AI Agents in Action

LLM, GPT Assistants, AutoGen, CrewAI, Semantic Kernel, Nexus, RAG, Prompt Flow, Planning

Hanwha Life Vietnam

Phiên bản v1.0.0

Ngày 6 tháng 11 năm 2026

Mục lục

1 Tổng quan về tài liệu **2**
- 1.1 Lời nói đầu . . . 2
- 1.2 Về cuốn sách . . . 3
  - 1.2.1 Ai nên đọc cuốn sách này . . . 3
  - 1.2.2 Cấu trúc cuốn sách: lộ trình . . . 3
  - 1.2.3 Về mã nguồn . . . 4
- 1.3 Về tác giả . . . 5

2 Giới thiệu về agent và thế giới của chúng **6**
- 2.1 Định nghĩa agent . . . 6
- 2.2 Hiểu các hệ thành phần của một agent . . . 8
- 2.3 Sự trỗi dậy của kỷ nguyên agent: vì sao cần agent? . . . 12
- 2.4 Bóc tách giao diện AI . . . 14
- 2.5 Định hướng trong bức tranh toàn cảnh về agent . . . 15
- Tóm tắt . . . 15

3 Khai thác sức mạnh của mô hình ngôn ngữ lớn **17**
- 3.1 Làm chủ OpenAI API . . . 19
  - 3.1.1 Kết nối tới mô hình chat completions . . . 19
  - 3.1.2 Hiểu yêu cầu và phản hồi . . . 20
- 3.2 Khám phá LLM mã nguồn mở với LM Studio . . . 22
  - 3.2.1 Cài đặt và chạy LM Studio . . . 22
  - 3.2.2 Phục vụ LLM cục bộ với LM Studio . . . 25
- 3.3 Viết prompt cho LLM bằng prompt engineering . . . 27
  - 3.3.1 Tạo truy vấn chi tiết . . . 31
  - 3.3.2 Nhập vai persona . . . 31
  - 3.3.3 Dùng dấu phân cách . . . 32
  - 3.3.4 Chỉ định các bước . . . 33
  - 3.3.5 Cung cấp ví dụ . . . 34
  - 3.3.6 Chỉ định độ dài đầu ra . . . 34
- 3.4 Chọn LLM tối ưu cho nhu cầu cụ thể của bạn . . . 35
- 3.5 Bài tập . . . 37
- Tóm tắt . . . 39

4 Làm việc với GPT Assistants **40**
- 4.1 Khám phá GPT assistant qua ChatGPT . . . 40
- 4.2 Xây dựng GPT làm khoa học dữ liệu . . . 45
- 4.3 Tuỳ biến GPT và thêm custom action . . . 49
  - 4.3.1 Tạo một assistant để xây dựng assistant . . . 50
  - 4.3.2 Kết nối custom action với assistant . . . 53
- 4.4 Mở rộng kiến thức của assistant bằng file upload . . . 56

4.4.1 Xây dựng GPT Calculus Made Easy . . . 56
4.4.2 Tìm kiếm kiến thức và hơn thế với file upload . . . 59
4.5 Phát hành GPT của bạn . . . 62
4.5.1 GPT assistant tốn kém . . . 62
4.5.2 Hiểu về kinh tế của GPT . . . 63
4.5.3 Phát hành GPT . . . 64
4.6 Bài tập . . . 65
Tóm tắt . . . 66

5 Khám phá hệ thống đa agent 68
5.1 Giới thiệu hệ thống đa agent với AutoGen Studio . . . 68
5.1.1 Cài đặt và sử dụng AutoGen Studio . . . 70
5.1.2 Thêm skill trong AutoGen Studio . . . 72
5.2 Khám phá AutoGen . . . 76
5.2.1 Cài đặt và sử dụng AutoGen . . . 76
5.2.2 Cải thiện code đầu ra bằng agent phê bình . . . 78
5.2.3 Tìm hiểu cache của AutoGen . . . 80
5.3 Group chat với agent và AutoGen . . . 81
5.4 Xây dựng crew agent với CrewAI . . . 82
5.4.1 Tạo crew agent CrewAI chuyên kể chuyện cười . . . 83
5.4.2 Quan sát agent làm việc với AgentOps . . . 86
5.5 Quay lại agent lập trình với CrewAI . . . 87
5.6 Bài tập . . . 93
Tóm tắt . . . 94

6 Trao quyền hành động cho agent 95
6.1 Định nghĩa action của agent . . . 95
6.2 Thực thi OpenAI function . . . 97
6.2.1 Thêm function vào lời gọi LLM API . . . 97
6.2.2 Thực thi lời gọi function . . . 100
6.3 Giới thiệu Semantic Kernel . . . 103
6.3.1 Bắt đầu với semantic function của SK . . . 104
6.3.2 Semantic function và biến ngữ cảnh . . . 105
6.4 Phối hợp semantic function và native function . . . 107
6.4.1 Tạo và đăng ký một semantic skill/plugin . . . 107
6.4.2 Áp dụng native function . . . 110
6.4.3 Nhúng native function vào semantic function . . . 111
6.5 Semantic Kernel như một agent dịch vụ tương tác . . . 112
6.5.1 Xây dựng một GPT interface ngữ nghĩa . . . 113
6.5.2 Kiểm thử dịch vụ ngữ nghĩa . . . 115
6.5.3 Chat tương tác với lớp dịch vụ ngữ nghĩa . . . 117
6.6 Tư duy ngữ nghĩa khi viết dịch vụ ngữ nghĩa . . . 120
6.7 Bài tập . . . 121
Tóm tắt . . . 122

7 Xây dựng trợ lý tự hành 123
7.1 Giới thiệu cây hành vi . . . 123
7.1.1 Hiểu cách thực thi cây hành vi . . . 125
7.1.2 Lựa chọn cây hành vi . . . 125
7.1.3 Chạy cây hành vi với Python và py_trees . . . 127
7.2 Khám phá GPT Assistants Playground . . . 128
7.2.1 Cài đặt và chạy Playground . . . 128

7.2.2 Sử dụng và xây dựng action tuỳ biến . . . 130
7.2.3 Cài đặt cơ sở dữ liệu assistant . . . 132
7.2.4 Cho assistant chạy code cục bộ . . . 132
7.2.5 Theo dõi quá trình làm việc của assistant qua log . . . 134
7.3 Giới thiệu cây hành vi agentic . . . 135
7.3.1 Quản lý assistant bằng assistant . . . 135
7.3.2 Xây dựng ABT cho thử thách lập trình . . . 136
7.3.3 Hệ AI hội thoại so với các phương pháp khác . . . 141
7.3.4 Đăng video YouTube lên X . . . 141
7.3.5 Thiết lập X cần thiết . . . 142
7.4 Xây dựng hệ đa agent hội thoại tự hành . . . 144
7.5 Xây dựng ABT bằng back chaining . . . 146
7.6 Bài tập . . . 148
Tóm tắt . . . 149

8 Lắp ráp và sử dụng một nền tảng agent 151
8.1 Giới thiệu Nexus, không chỉ là một nền tảng agent nữa . . . 151
8.1.1 Chạy Nexus . . . 152
8.1.2 Phát triển Nexus . . . 153
8.2 Giới thiệu Streamlit để phát triển ứng dụng chat . . . 155
8.2.1 Xây dựng một ứng dụng chat bằng Streamlit . . . 156
8.2.2 Tạo một ứng dụng chat dạng streaming . . . 158
8.3 Phát triển profile và persona cho agent . . . 159
8.4 Vận hành agent và tìm hiểu agent engine . . . 162
8.5 Trao cho agent action và tool . . . 163
8.6 Bài tập . . . 167
Tóm tắt . . . 168

9 Hiểu bộ nhớ và tri thức của agent 170
9.1 Hiểu truy xuất trong ứng dụng AI . . . 170
9.2 Cơ bản về retrieval augmented generation (RAG) . . . 171
9.3 Tìm hiểu semantic search và lập chỉ mục tài liệu . . . 173
9.3.1 Áp dụng tìm kiếm tương đồng vector . . . 173
9.3.2 Cơ sở dữ liệu vector và tìm kiếm tương đồng . . . 177
9.3.3 Làm rõ document embedding . . . 178
9.3.4 Truy vấn document embedding từ Chroma . . . 179
9.4 Xây dựng RAG với LangChain . . . 180
9.4.1 Tách và nạp tài liệu với LangChain . . . 181
9.4.2 Tách tài liệu theo token với LangChain . . . 183
9.5 Áp dụng RAG để xây dựng tri thức cho agent . . . 184
9.6 Hiện thực bộ nhớ trong các hệ thống agentic . . . 188
9.6.1 Sử dụng memory store trong Nexus . . . 190
9.6.2 Semantic memory và ứng dụng cho bộ nhớ ngữ nghĩa, bộ nhớ sự kiện và bộ nhớ thủ tục . . . 192
9.7 Tìm hiểu nén bộ nhớ và tri thức . . . 195
9.8 Bài tập . . . 197
Tóm tắt . . . 199

10 Làm chủ prompt cho agent với prompt flow **200**
10.1 Vì sao cần prompt engineering có hệ thống . . . 200
10.2 Hiểu về agent profile và persona . . . 204
10.3 Thiết lập prompt flow đầu tiên . . . 205
10.3.1 Bắt đầu . . . 206
10.3.2 Tạo profile bằng Jinja2 template . . . 209
10.3.3 Triển khai prompt flow thành API . . . 210
10.4 Đánh giá profile: rubric và grounding . . . 212
10.5 Hiểu về rubric và grounding . . . 215
10.6 Đánh giá grounding bằng một LLM profile . . . 217
10.7 So sánh các profile: tìm ra profile hoàn hảo . . . 218
10.7.1 Parse đầu ra đánh giá của LLM . . . 218
10.7.2 Chạy batch processing trong prompt flow . . . 220
10.7.3 Tạo evaluation flow cho grounding . . . 224
10.7.4 Bài tập . . . 227
Tóm tắt . . . 228

11 Suy luận và đánh giá của agent **229**
11.1 Hiểu về direct solution prompting . . . 229
11.1.1 Prompt hỏi–đáp . . . 230
11.1.2 Triển khai few-shot prompting . . . 232
11.1.3 Rút ra tính khái quát với zero-shot prompting . . . 234
11.2 Suy luận trong prompt engineering . . . 236
11.2.1 Chain of thought prompting . . . 236
11.2.2 Zero-shot CoT prompting . . . 241
11.2.3 Từng bước với prompt chaining . . . 242
11.3 Dùng đánh giá để có lời giải nhất quán . . . 245
11.3.1 Đánh giá self-consistency prompting . . . 246
11.3.2 Đánh giá tree of thought prompting . . . 251
11.4 Bài tập . . . 255
Tóm tắt . . . 256

12 Lập kế hoạch và phản hồi của agent **257**
12.1 Lập kế hoạch: công cụ thiết yếu cho mọi agent/assistant . . . 257
12.2 Hiểu quy trình lập kế hoạch tuần tự . . . 262
12.3 Xây dựng một sequential planner . . . 264
12.4 Xem xét một stepwise planner: OpenAI Strawberry . . . 270
12.5 Áp dụng planning, reasoning, evaluation và feedback vào các hệ thống assistant và agentic 272
12.5.1 Áp dụng planning cho assistant/agent . . . 272
12.5.2 Áp dụng reasoning cho assistant/agent . . . 274
12.5.3 Áp dụng evaluation cho hệ thống agentic . . . 275
12.5.4 Áp dụng feedback cho các ứng dụng agentic/assistant . . . 276
12.6 Bài tập . . . 278
Tóm tắt . . . 280

A Truy cập các mô hình ngôn ngữ lớn của OpenAI **281**
A.1 Truy cập tài khoản và khoá của OpenAI . . . 281
A.2 Azure OpenAI Studio, khoá và deployment . . . 285

B Môi trường phát triển Python **288**
B.1 Tải mã nguồn 288
B.2 Cài đặt Python 288
B.3 Cài đặt VS Code 289
B.4 Cài các extension Python cho VS Code 289
B.5 Tạo môi trường Python mới bằng VS Code 290
B.6 Dùng VS Code Dev Containers (Docker) 292

Chương 1

Tổng quan về tài liệu

Tài liệu này biên soạn lại bằng tiếng Việt nội dung cuốn *AI Agents in Action* của Micheal Lanham (Manning, 2025). Chương này tóm lược phần mở đầu của sách: lời nói đầu, giới thiệu về cuốn sách (ai nên đọc, lộ trình các chương, mã nguồn đi kèm) và đôi nét về tác giả.

1.1 Lời nói đầu

Hành trình đến với các hệ thống thông minh của tác giả bắt đầu từ đầu thập niên 1980. Giống nhiều người thời đó, ông tin rằng trí tuệ nhân tạo (AI) đã ở ngay trước mắt: dường như chỉ cần thêm một đổi mới, một bước nhảy công nghệ nữa là ta sẽ có được thứ trí tuệ mà mình hình dung. Nhưng bước nhảy ấy chưa bao giờ đến. Có lẽ hình ảnh HAL trong phim *2001: A Space Odyssey* của Stanley Kubrick đã cuốn hút ông với ý tưởng về một người bạn đồng hành là máy tính thật sự thông minh. Sau nhiều năm nỗ lực, thử nghiệm và vô số sai lầm, ông nhận ra rằng tạo ra AI phức tạp hơn rất nhiều so với những gì con người từng tưởng. Đầu thập niên 1990, ông chuyển hướng, áp dụng kỹ năng của mình vào những mục tiêu cụ thể hơn ở các ngành khác.

Phải đến cuối thập niên 1990, sau một loạt biến cố đầy thử thách và mang tính bước ngoặt, ông mới nhận ra niềm đam mê thật sự của mình là xây dựng các hệ thống thông minh. Ông biết các hệ thống này có thể sẽ không bao giờ đạt tới mức siêu trí tuệ như HAL, và ông chấp nhận điều đó. Ông tìm thấy niềm vui khi làm việc với học máy (machine learning) và khoa học dữ liệu, tạo ra những mô hình có thể học và thích nghi. Hơn 20 năm, ông gắn bó với lĩnh vực này, giải quyết những bài toán đòi hỏi sự sáng tạo, tính chính xác và tầm nhìn về những điều có thể.

Trong thời gian đó, ông làm đủ thứ, từ thuật toán di truyền (genetic algorithm) để dự đoán các đầu vào chưa biết cho tới các mô hình học sinh (generative learning model) cho khoan ngang trong ngành dầu khí. Những trải nghiệm này dẫn ông tới việc viết sách để chia sẻ kiến thức về nhiều chủ đề: dịch ngược (reverse-engineering) Pokémon Go, xây dựng trải nghiệm thực tế tăng cường và thực tế ảo, thiết kế âm thanh cho game, và áp dụng học tăng cường (reinforcement learning) để tạo agent thông minh. Ông đã dành nhiều năm ngập trong code, phát triển agent bằng Unity ML-Agents và học tăng cường sâu.

Dù vậy, ông chưa từng nghĩ có ngày mình chỉ cần mô tả điều mình muốn cho một mô hình AI và nó sẽ thực hiện điều đó; chưa từng nghĩ trong đời mình lại có thể cộng tác với AI tự nhiên như hiện nay; và càng không ngờ hành trình này lại vừa nhanh vừa chậm đến thế.

Tháng 11/2022, sự ra mắt của ChatGPT đã thay đổi tất cả. Nó thay đổi cách thế giới nhìn nhận AI và cách ta xây dựng hệ thống thông minh. Với tác giả, nó còn thay đổi góc nhìn về năng lực của các hệ thống này. Bỗng nhiên, ý tưởng về những agent có thể tự chủ thực hiện các nhiệm vụ phức tạp không còn là giấc mơ xa vời mà trở thành hiện thực khả thi. Trong một số cuốn sách trước, ông đã mô tả các hệ thống agent (agentic system) có thể đảm nhận những nhiệm vụ cụ thể, và giờ đây những ý tưởng từng chỉ là lý thuyết ấy đã nằm trong tầm tay.

Cuốn sách này là kết tinh của hàng chục năm kinh nghiệm xây dựng hệ thống thông minh, đồng thời là sự hiện thực hoá những giấc mơ ông từng có về những gì AI có thể trở thành. AI agent đã xuất hiện, sẵn sàng thay đổi cách ta tương tác với công nghệ, cách ta làm việc và cuối cùng là cách ta sống.

Tuy vậy, ngay cả bây giờ tác giả vẫn thấy các tổ chức còn do dự khi áp dụng hệ thống agent. Ông cho rằng sự do dự này không bắt nguồn từ nỗi sợ AI mà từ việc thiếu hiểu biết và chuyên môn để xây dựng các hệ thống đó. Ông hy vọng cuốn sách giúp lấp khoảng trống này, giới thiệu AI agent như những công cụ ai cũng có thể tiếp cận: những công cụ ta không cần sợ, mà nên tôn trọng, quản lý có trách nhiệm và học cách làm việc hài hoà cùng chúng.

Lời cảm ơn. Tác giả bày tỏ lòng biết ơn tới cộng đồng học máy và học sâu, những người đã giúp lĩnh vực này không rơi vào một "mùa đông AI" mới mà còn phát triển mạnh mẽ; tới đội ngũ biên tập và phản biện của Manning (đặc biệt là biên tập viên nội dung Becky Whitney và biên tập viên kỹ thuật Ross Turner); tới người bạn đời Rhonda; tới các đồng nghiệp tại Symend, nơi những ý tưởng và thiết kế đầu tiên về hệ thống agent của cuốn sách được hình thành; và tới tất cả các nhà phản biện đã góp ý để cuốn sách tốt hơn.

1.2 Về cuốn sách

AI Agents in Action nói về việc xây dựng và làm việc với các hệ thống agent thông minh: không chỉ tạo ra những thực thể tự chủ mà còn phát triển những agent có thể giải quyết hiệu quả các bài toán thực tế. Sách bắt đầu từ những kiến thức cơ bản về cách làm việc với mô hình ngôn ngữ lớn (LLM) để xây dựng trợ lý (assistant), hệ đa agent (multi-agent system) và agent có hành vi (agentic behavioral agent). Từ đó, sách đi vào các thành phần cốt lõi của một hệ thống agent: hệ thống truy xuất (retrieval) để bổ sung tri thức và bộ nhớ, hành động và sử dụng công cụ (tool), suy luận (reasoning), lập kế hoạch (planning), đánh giá (evaluation) và phản hồi (feedback). Qua các ví dụ thực hành, sách cho thấy những thành phần này giúp agent thực hiện nhiều loại nhiệm vụ phức tạp ra sao.

Hành trình này không chỉ là chuyện công nghệ mà còn là việc hình dung lại cách ta tiếp cận giải quyết vấn đề. Tác giả hy vọng cuốn sách giúp bạn xem agent thông minh như đối tác trong đổi mới, có khả năng biến ý tưởng thành hành động theo những cách từng bị coi là bất khả thi, và cùng khám phá cách AI có thể nâng cao tiềm năng con người, giúp ta đạt được nhiều hơn so với khi làm một mình.

1.2.1 Ai nên đọc cuốn sách này

Cuốn sách dành cho bất kỳ ai tò mò về agent thông minh và cách phát triển hệ thống agent, dù bạn đang xây dựng trợ lý hữu ích đầu tiên hay đi sâu vào các hệ đa agent phức tạp. Bạn không cần kinh nghiệm trước về agent, hệ thống agent, prompt engineering hay làm việc với LLM. Bạn chỉ cần hiểu Python cơ bản và quen thuộc với các repository trên GitHub. Mục tiêu của tác giả là làm cho các khái niệm này dễ tiếp cận và hấp dẫn, giúp bất kỳ ai muốn khám phá thế giới AI agent có thể tự tin làm điều đó.

Dù bạn là lập trình viên, nhà nghiên cứu, người chơi nghiệp dư hay chỉ đơn giản là tò mò về tiềm năng của AI, cuốn sách này đều dành cho bạn. Hy vọng bạn sẽ tìm thấy cảm hứng, hướng dẫn thực tế và một góc nhìn mới về tiềm năng to lớn của agent thông minh, để hiểu, tạo ra và khai thác sức mạnh của AI agent trong thực tế.

1.2.2 Cấu trúc cuốn sách: lộ trình

Sách gồm 11 chương và 2 phụ lục. Chương 1, "Giới thiệu về agent và thế giới của chúng", đặt nền móng bằng các định nghĩa cơ bản về mô hình ngôn ngữ lớn, hệ thống chat, trợ lý và agent tự hành (autonomous agent). Càng về sau, nội dung chuyển sang các thành phần cấu tạo nên một agent và cách chúng phối hợp với nhau để tạo ra những hệ thống thật sự hiệu quả. Tóm tắt nhanh các chương 2 đến 11:

- **Chương 2, "Khai thác sức mạnh của mô hình ngôn ngữ lớn"**: bắt đầu bằng cách dùng các LLM thương mại như OpenAI, sau đó xem xét các công cụ như LM Studio, vốn cung cấp hạ tầng và hỗ trợ để chạy nhiều LLM mã nguồn mở, giúp ai cũng có thể thử nghiệm và sáng tạo.

- **Chương 3, "Làm việc với GPT Assistants"**: đi sâu vào năng lực của nền tảng GPT Assistants của OpenAI. Trợ lý là dạng agent nền tảng; chương này hướng dẫn tạo nhiều trợ lý thực tế và đa dạng, từ trợ lý nấu ăn, "thực tập sinh" khoa học dữ liệu cho tới trợ lý học sách.
- **Chương 4, "Khám phá hệ đa agent"**: các công cụ agent tiến bộ rất nhanh. Chương này khám phá hai hệ đa agent tinh vi là CrewAI và AutoGen, minh hoạ khả năng tự phát triển code của AutoGen và cách CrewAI tập hợp một nhóm "nhà nghiên cứu truyện cười" để cùng nhau sáng tác.
- **Chương 5, "Trao quyền hành động cho agent"**: hành động là nền tảng của mọi hệ thống agent. Chương này bàn về cách agent dùng công cụ và hàm (function) để thực thi hành động, từ truy vấn cơ sở dữ liệu và giao diện lập trình ứng dụng (API) tới sinh ảnh, với trọng tâm là giúp agent tự chủ thực hiện những hành động có ý nghĩa.
- **Chương 6, "Xây dựng trợ lý tự hành"**: khám phá cây hành vi (behavior tree), một công cụ quen thuộc trong robot và game, như cơ chế điều phối nhiều agent phối hợp với nhau. Cây hành vi được dùng để giải quyết các bài toán như thi lập trình và tạo nội dung mạng xã hội.
- **Chương 7, "Xây dựng và sử dụng một nền tảng agent"**: giới thiệu Nexus, một nền tảng tinh vi để điều phối nhiều agent và LLM. Chương này bàn về cách Nexus hỗ trợ quy trình agent (agentic workflow) và các tương tác phức tạp giữa agent, kèm ví dụ về một môi trường đa agent hoạt động đầy đủ.
- **Chương 8, "Hiểu bộ nhớ và tri thức của agent"**: retrieval-augmented generation (RAG, sinh có tăng cường truy xuất) đã trở thành công cụ thiết yếu để mở rộng năng lực của agent LLM. Chương này khám phá cách cơ chế truy xuất vừa đóng vai trò nguồn tri thức (bằng cách xử lý các file được nạp vào), vừa đóng vai trò bộ nhớ, giúp agent nhớ lại các tương tác hay sự kiện trước đó.
- **Chương 9, "Làm chủ prompt cho agent với prompt flow"**: prompt engineering là yếu tố trung tâm cho thành công của agent. Chương này giới thiệu prompt flow, một công cụ của Microsoft giúp tự động hoá việc kiểm thử và đánh giá prompt, từ đó tạo hành vi agent ổn định và hiệu quả hơn.
- **Chương 10, "Suy luận và đánh giá của agent"**: suy luận là then chốt để giải quyết vấn đề một cách thông minh. Chương này khám phá nhiều kỹ thuật suy luận như chuỗi suy nghĩ (chain of thought, CoT), và cho thấy agent có thể đánh giá các chiến lược suy luận ngay trong lúc suy diễn (inference), nâng cao khả năng tự giải quyết vấn đề.
- **Chương 11, "Lập kế hoạch và phản hồi của agent"**: lập kế hoạch có lẽ là kỹ năng quan trọng nhất để agent đạt được mục tiêu. Chương này bàn về cách agent đưa lập kế hoạch vào để xử lý các nhiệm vụ phức tạp và cách dùng vòng phản hồi (feedback loop) để tinh chỉnh kế hoạch. Chương kết thúc bằng việc tích hợp mọi thành phần chính (hành động, bộ nhớ và tri thức, suy luận, đánh giá, lập kế hoạch và phản hồi) vào các ví dụ thực tế về hệ thống agent giải quyết bài toán thực tế.

Hai phụ lục hỗ trợ phần thực hành: Phụ lục A hướng dẫn tạo tài khoản OpenAI và truy cập GPT-4 cùng các mô hình khác; Phụ lục B hướng dẫn thiết lập môi trường phát triển Python với Visual Studio Code (VS Code), kể cả cài các extension cần thiết.

1.2.3 Về mã nguồn

Mã nguồn của sách nằm rải rác trong nhiều dự án mã nguồn mở, phần lớn được tác giả hoặc các tổ chức khác lưu trữ trên GitHub. Xuyên suốt cuốn sách, tác giả cố gắng làm cho nội dung dễ tiếp cận nhất có thể, theo hướng low-code (ít code) để bạn tập trung vào các khái niệm cốt lõi. Nhiều chương minh hoạ

cách những prompt đơn giản có thể sinh ra code có ý nghĩa, cho thấy sức mạnh của phát triển phần mềm có AI hỗ trợ.

Ngoài ra, bạn sẽ gặp nhiều hồ sơ trợ lý (assistant profile) và hệ đa agent minh hoạ cách giải quyết bài toán thực tế bằng code được sinh ra. Các ví dụ này nhằm truyền cảm hứng, định hướng và giúp bạn khám phá những gì có thể làm với AI agent. Tác giả khuyến khích bạn tìm hiểu các repository, thử nghiệm với code và điều chỉnh nó cho nhu cầu của mình.

Sách có nhiều ví dụ mã nguồn, cả trong các listing được đánh số lẫn xen trong đoạn văn. Trong cả hai trường hợp, mã nguồn được định dạng bằng `font chữ có độ rộng cố định như thế này` để phân biệt với văn bản thường. Đôi khi một phần code được in đậm để làm nổi bật phần đã thay đổi so với các bước trước trong chương, chẳng hạn khi một tính năng được thêm vào một dòng code có sẵn. Trong nhiều trường hợp, mã nguồn gốc đã được định dạng lại (thêm ngắt dòng, chỉnh thụt lề) cho vừa trang sách; có khi vẫn chưa đủ và listing phải dùng ký hiệu nối dòng (□). Các comment trong mã nguồn cũng thường bị lược bỏ khỏi listing khi code đã được giải thích trong văn bản. Nhiều listing có kèm chú thích (annotation) làm nổi bật các khái niệm quan trọng; trong tài liệu này, các chú thích đó được chuyển thành comment tiếng Việt ở cuối dòng code tương ứng.

Bạn có thể lấy các đoạn code chạy được từ bản liveBook (trực tuyến) của sách tại https://livebook.manning.com/book/ai-agents-in-action. Toàn bộ mã nguồn của các ví dụ có thể tải từ website Manning tại https://www.manning.com/books/ai-agents-in-action. Ngoài ra, code phát triển cho cuốn sách được đặt trong ba repository GitHub công khai:

- **GPT-Agents** (tên ban đầu của cuốn sách), tại https://github.com/cxbxmxcx/GPT-Agents: chứa code cho nhiều ví dụ được minh hoạ trong các chương.
- **GPT Assistants Playground**, tại https://github.com/cxbxmxcx/GPTAssistantsPlayground: cả một nền tảng và công cụ chuyên để xây dựng trợ lý OpenAI GPT, có giao diện web tiện dụng và nhiều công cụ để phát triển hệ thống agent tự hành.
- **Nexus**, tại https://github.com/cxbxmxcx/Nexus: ví dụ về một công cụ agent chạy trên web, giúp bạn tạo hệ thống agent và minh hoạ nhiều bài toán code.

Người mua sách được truy cập miễn phí liveBook, nền tảng đọc trực tuyến của Manning, có diễn đàn thảo luận để ghi chú, hỏi đáp kỹ thuật với tác giả và người đọc khác tại https://livebook.manning.com/book/ai-agents-in-action/discussion.

1.3 Về tác giả

Micheal Lanham là một nhà đổi mới về phần mềm và công nghệ với hơn hai mươi năm kinh nghiệm trong ngành. Ông có nền tảng rộng về phát triển nhiều loại ứng dụng phần mềm ở nhiều lĩnh vực: game, đồ hoạ, phát triển web, phần mềm desktop, AI, GIS, địa khoa học/địa cơ học dầu khí và học máy. Ông là người đi đầu trong việc tích hợp mạng nơ-ron và thuật toán tiến hoá vào phát triển game từ khoảng đầu thiên niên kỷ. Ông là tác giả của nhiều cuốn sách về học sâu, phát triển game và thực tế tăng cường, trong đó có *Evolutionary Deep Learning* (Manning, 2023) và *Augmented Reality Game Development* (Packt Publishing, 2017). Ông sống tại Calgary, Alberta, Canada cùng gia đình đông con và thích nấu ăn cho cả nhà.

Chương 2

Giới thiệu về agent và thế giới của chúng

Chương này bao quát các nội dung:

- Định nghĩa khái niệm agent.
- Phân biệt các thành phần của một agent.
- Phân tích sự trỗi dậy của kỷ nguyên agent: vì sao cần agent?
- Bóc tách giao diện AI (AI interface).
- Định hướng trong bức tranh toàn cảnh về agent.

Agent không phải là khái niệm mới trong học máy (machine learning) và trí tuệ nhân tạo (AI). Chẳng hạn, trong học tăng cường (reinforcement learning), từ *agent* chỉ một thực thể thông minh chủ động ra quyết định và học hỏi. Ở các lĩnh vực khác, từ *agent* gần nghĩa hơn với một ứng dụng hay phần mềm tự động làm việc gì đó thay cho bạn.

2.1 Định nghĩa agent

Bạn có thể tra bất kỳ từ điển trực tuyến nào để tìm định nghĩa của agent. Từ điển *Merriam-Webster* định nghĩa như sau (`https://www.merriam-webster.com/dictionary/agent`):

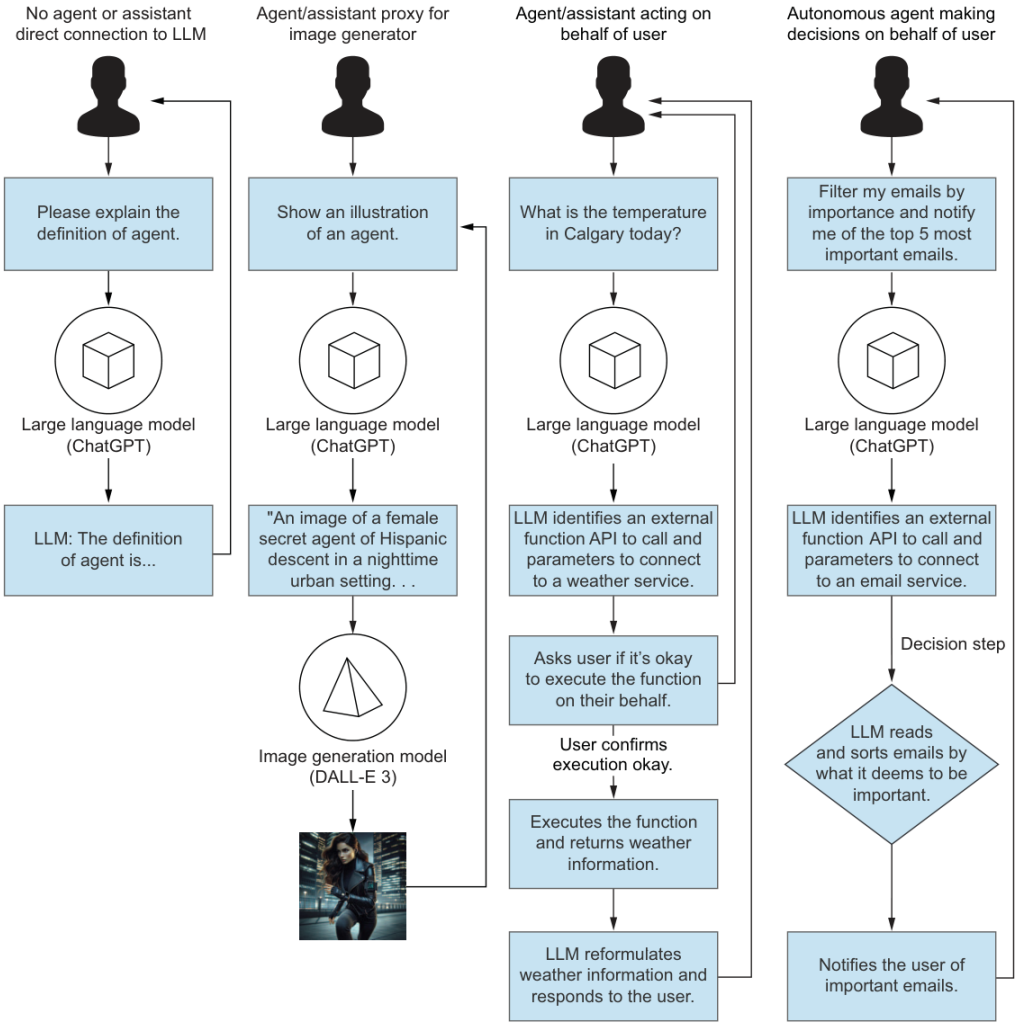
- Một thứ hành động hoặc tác động quyền lực (*one that acts or exerts power*).
- Một thứ tạo ra hoặc có thể tạo ra một hiệu ứng (*something that produces or can produce an effect*).
- Một phương tiện hay công cụ nhờ đó một trí tuệ dẫn dắt đạt được kết quả (*a means or instrument by which a guiding intelligence achieves a result*).

Trong hành trình xây dựng các agent mạnh mẽ ở cuốn sách này, từ *agent* được dùng theo định nghĩa từ điển trên. Điều đó cũng có nghĩa là thuật ngữ *assistant* (trợ lý) sẽ đồng nghĩa với *agent*. Các công cụ như GPT Assistants của OpenAI cũng nằm trong phạm vi khái niệm AI agent. OpenAI tránh dùng từ *agent* vì lịch sử của học máy, nơi agent là thực thể tự quyết định và tự chủ.

Hình 2.1 cho thấy bốn trường hợp người dùng có thể tương tác với một mô hình ngôn ngữ lớn (LLM): trực tiếp, qua một agent/trợ lý làm proxy (trung gian), qua một agent/trợ lý, hoặc qua một agent tự hành (autonomous agent). Bốn trường hợp sử dụng này được mô tả chi tiết hơn như sau:

- **Tương tác trực tiếp của người dùng** (*direct user interaction*): nếu bạn từng dùng các phiên bản ChatGPT đời đầu, bạn đã trải nghiệm việc tương tác trực tiếp với LLM. Không có agent proxy hay trợ lý nào khác can thiệp thay bạn.

- **Agent/trợ lý proxy** (*agent/assistant proxy*): nếu bạn từng dùng DALL-E 3 qua ChatGPT, bạn đã trải nghiệm tương tác với một agent proxy. Trong trường hợp này, một LLM chen vào giữa các yêu cầu của bạn và diễn đạt lại chúng theo dạng phù hợp hơn cho nhiệm vụ. Ví dụ, với việc sinh ảnh, ChatGPT soạn prompt tốt hơn. Agent proxy là trường hợp sử dụng thường gặp để giúp người dùng với các nhiệm vụ hay mô hình mà họ chưa quen.
- **Agent/trợ lý** (*agent/assistant*): nếu bạn từng dùng plugin của ChatGPT hay một GPT assistant, bạn đã trải nghiệm trường hợp này. Ở đây, LLM biết về các chức năng của plugin hay trợ lý và chuẩn bị gọi plugin/hàm đó. Tuy nhiên, trước khi gọi, LLM cần người dùng phê duyệt. Nếu được phê duyệt, plugin hoặc hàm được thực thi và kết quả được trả về cho LLM. LLM sau đó gói phản hồi này bằng ngôn ngữ tự nhiên rồi trả lại cho người dùng.
- **Agent tự hành** (*autonomous agent*): trong trường hợp này, agent diễn giải yêu cầu của người dùng, xây dựng kế hoạch và xác định các điểm ra quyết định. Từ đó, nó thực thi các bước trong kế hoạch và tự đưa ra các quyết định cần thiết. Agent có thể xin phản hồi của người dùng sau một số nhiệm vụ mốc, nhưng thường được tự do khám phá và học hỏi nếu có thể. Loại agent này đặt ra nhiều lo ngại nhất về đạo đức và an toàn, điều ta sẽ bàn ở phần sau.

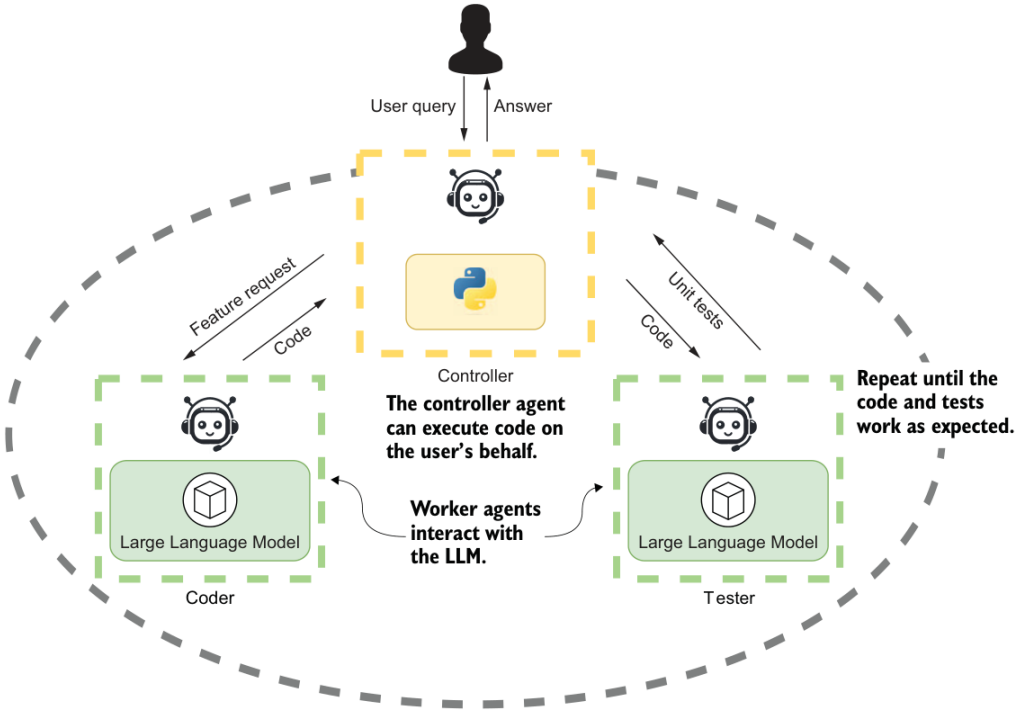


Hình 2.1: Sự khác biệt giữa các kiểu tương tác với LLM: tương tác trực tiếp so với dùng agent proxy, agent và agent tự hành

Hình 2.1 minh hoạ các trường hợp sử dụng cho một luồng hành động duy nhất trên LLM với một agent duy nhất. Với các bài toán phức tạp hơn, ta thường chia agent thành các **hồ sơ** (*profile*) hay **persona**

(vai). Mỗi hồ sơ agent được giao một nhiệm vụ cụ thể và thực hiện nhiệm vụ đó với công cụ và tri thức chuyên biệt.

Hệ đa agent (*multi-agent system*) là các hồ sơ agent phối hợp với nhau theo nhiều cấu hình khác nhau để giải quyết một bài toán. Hình 2.2 minh hoạ một hệ đa agent gồm ba agent: một bộ điều khiển (controller) hay proxy, và hai agent hồ sơ làm worker (tác nhân thực thi) do proxy điều khiển. Hồ sơ coder ở bên trái viết code theo yêu cầu của người dùng; bên phải là hồ sơ tester được thiết kế để viết unit test. Các agent này làm việc và trao đổi với nhau cho đến khi hài lòng với code, rồi chuyển nó cho người dùng.



Hình 2.2: Ví dụ về một hệ đa agent: bộ điều khiển (agent proxy) giao tiếp trực tiếp với người dùng, còn hai agent (coder và tester) làm việc phía sau để viết code và viết unit test kiểm thử code đó

Hình 2.2 chỉ là một trong vô số cấu hình agent có thể có. (Ở Chương 5, ta sẽ khám phá AutoGen, nền tảng mã nguồn mở của Microsoft hỗ trợ nhiều cấu hình để triển khai hệ đa agent.)

Hệ đa agent có thể hoạt động tự chủ nhưng cũng có thể vận hành hoàn toàn theo phản hồi của con người. Lợi ích của việc dùng nhiều agent giống như của một agent đơn lẻ nhưng thường được khuếch đại. Trong khi một agent đơn lẻ thường chuyên về một nhiệm vụ, hệ đa agent có thể xử lý song song nhiều nhiệm vụ. Nhiều agent cũng có thể cung cấp phản hồi và đánh giá cho nhau, giảm lỗi khi hoàn thành công việc.

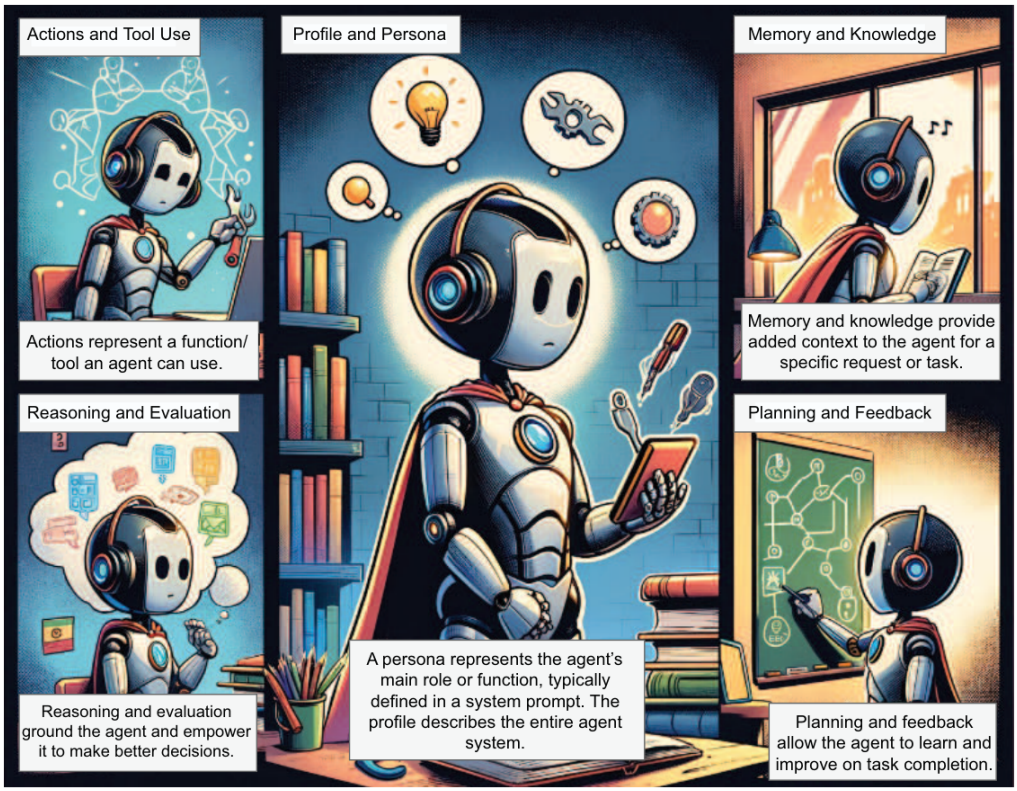
Như vậy, một AI agent hay hệ thống agent có thể được lắp ghép theo nhiều cách. Không chỉ thế, bản thân một agent cũng có thể được lắp ghép từ nhiều thành phần. Ở mục tiếp theo, ta sẽ xem các chủ đề từ hồ sơ của agent tới các hành động nó có thể thực hiện, cũng như bộ nhớ và lập kế hoạch.

2.2 Hiểu các hệ thành phần của một agent

Agent có thể là những đơn vị phức tạp gồm nhiều hệ thành phần. Các thành phần này là công cụ agent sử dụng để hoàn thành mục tiêu hay nhiệm vụ được giao, và thậm chí để tạo ra nhiệm vụ mới. Thành phần có thể là hệ thống đơn giản hoặc phức tạp, thường được chia thành năm nhóm.

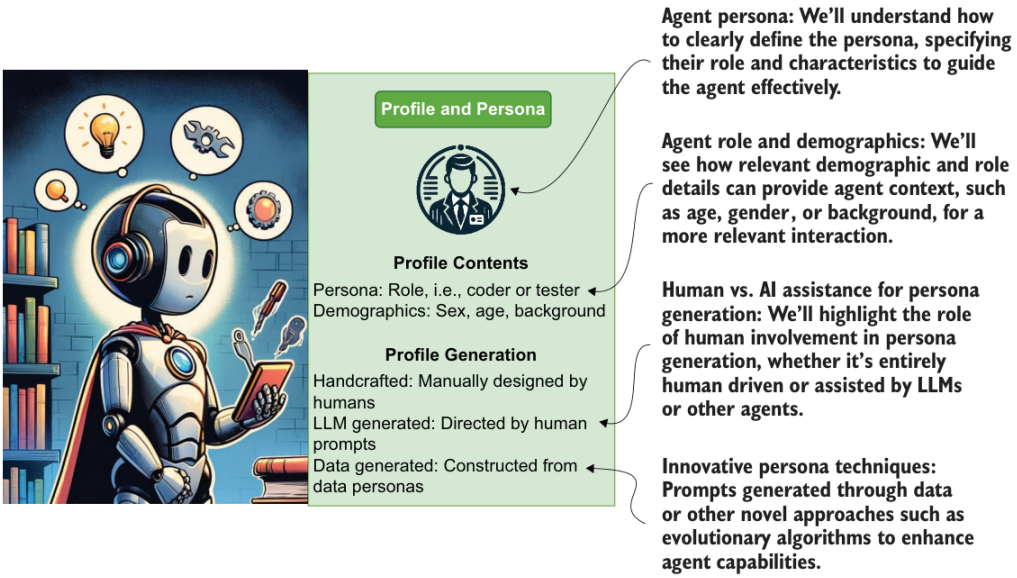
Hình 2.3 mô tả các nhóm thành phần chính mà một hệ thống agent đơn lẻ có thể bao gồm. Mỗi thành phần có các loại con để định nghĩa kiểu, cấu trúc và cách sử dụng của thành phần đó. Ở trung tâm của

mọi agent là hồ sơ và persona; mở rộng ra từ đó là các hệ thống và chức năng giúp tăng cường năng lực cho agent.



Hình 2.3: Năm thành phần chính của một hệ thống agent đơn lẻ (ảnh được tạo bằng DALL-E 3)

Hồ sơ và persona của agent, minh hoạ trong Hình 2.4, là phần mô tả cơ sở của agent. Persona, thường được gọi là **system prompt**, hướng dẫn agent hoàn thành nhiệm vụ, học cách phản hồi và các sắc thái khác. Nó gồm các yếu tố như lý lịch (ví dụ coder, writer) và đặc điểm nhân khẩu học, và có thể được tạo bằng nhiều phương pháp: tự soạn thủ công, nhờ LLM hỗ trợ, hoặc dùng kỹ thuật dựa trên dữ liệu, kể cả thuật toán tiến hoá (evolutionary algorithm).



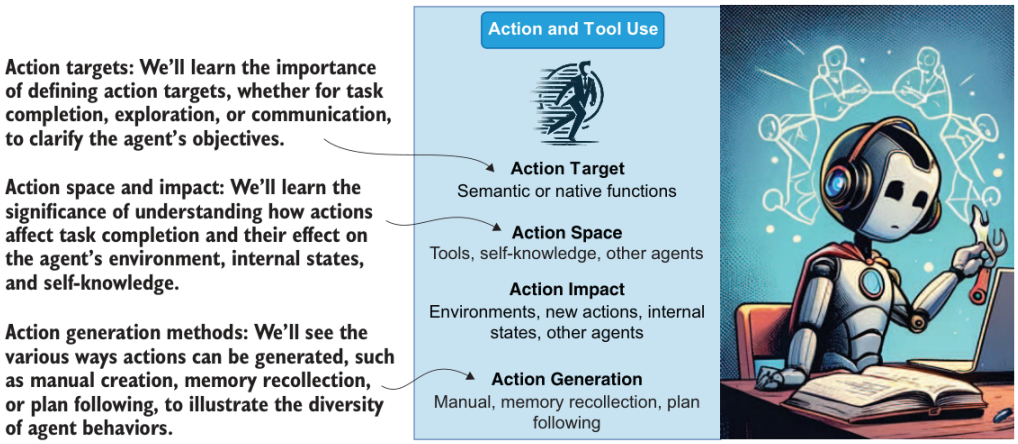
Hình 2.4: Cái nhìn chi tiết về cách ta sẽ khám phá việc tạo hồ sơ agent

Ta sẽ khám phá cách tạo hồ sơ/persona agent hiệu quả và cụ thể qua các kỹ thuật như rubric (bảng tiêu chí) và grounding (neo vào ngữ cảnh/dữ kiện). Ngoài ra, ta sẽ giải thích các khía cạnh của hồ sơ do con người soạn so với hồ sơ do AI (LLM) soạn, kể cả những kỹ thuật mới dùng dữ liệu và thuật toán tiến hoá để xây dựng hồ sơ.

Ghi chú

Hồ sơ của agent hay trợ lý được tạo thành từ nhiều yếu tố, trong đó có persona. Có thể hữu ích khi nghĩ về hồ sơ như phần mô tả công việc mà agent/trợ lý sẽ thực hiện và các công cụ nó cần.

Hình 2.5 minh hoạ thành phần **hành động và sử dụng công cụ** (*actions and tool use*) trong bối cảnh agent, bao gồm các hoạt động hướng tới hoàn thành nhiệm vụ hoặc thu thập thông tin. Các hành động này có thể chia thành ba loại: hoàn thành nhiệm vụ, khám phá và giao tiếp, với mức độ tác động khác nhau lên môi trường và trạng thái bên trong của agent. Hành động có thể được tạo thủ công, được gợi lại từ bộ nhớ, hoặc theo các kế hoạch định sẵn; chúng ảnh hưởng tới hành vi của agent và giúp tăng cường việc học.

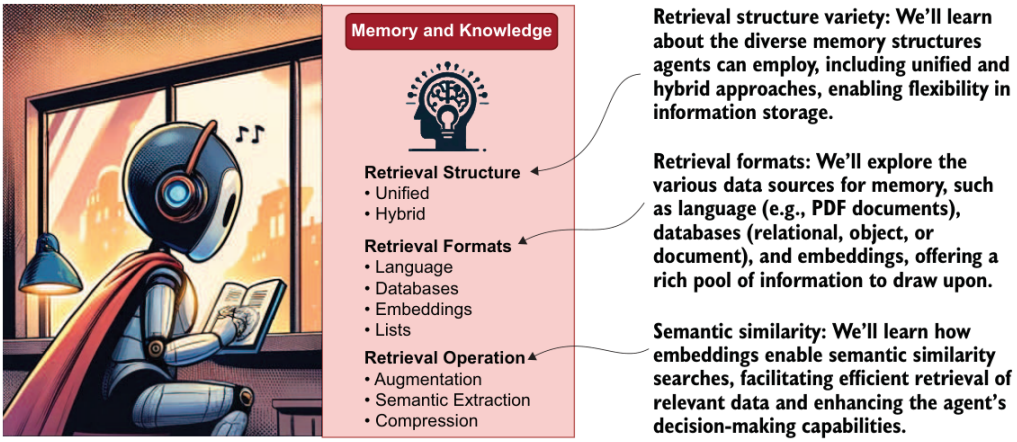


Hình 2.5: Các khía cạnh của hành động agent sẽ được khám phá trong cuốn sách

Hiểu *mục tiêu của hành động* (action target) giúp ta xác định rõ mục đích: hoàn thành nhiệm vụ,

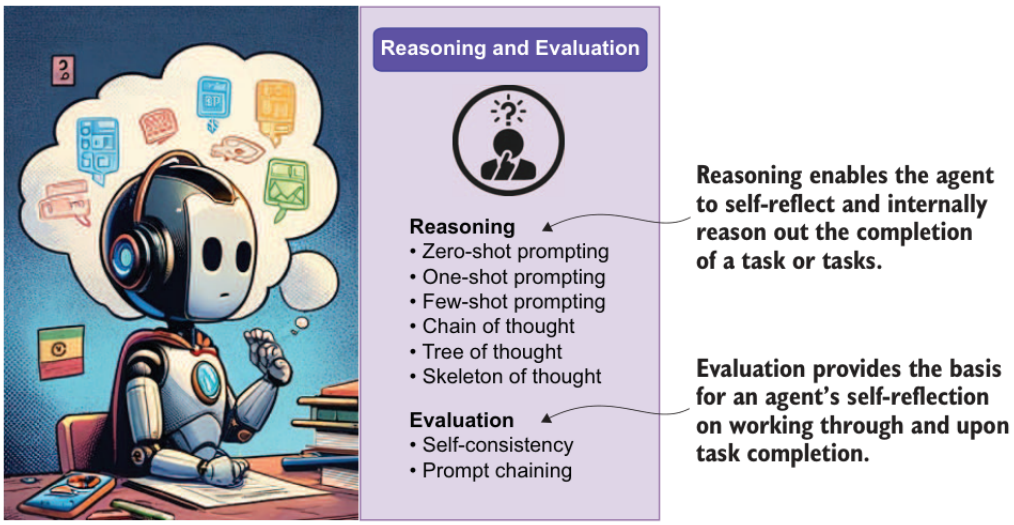
khám phá hay giao tiếp. Nhận biết *tác động của hành động* (action effect) cho thấy hành động ảnh hưởng thế nào tới kết quả nhiệm vụ, môi trường của agent và trạng thái bên trong của nó, góp phần vào việc ra quyết định hiệu quả. Cuối cùng, nắm được *các phương pháp tạo hành động* (action generation) giúp ta biết cách tạo hành động thủ công, gợi lại từ bộ nhớ hoặc theo kế hoạch định sẵn, từ đó định hình tốt hơn hành vi và quá trình học của agent.

Hình 2.6 cho thấy chi tiết hơn thành phần **tri thức và bộ nhớ** (*knowledge and memory*). Agent dùng tri thức và bộ nhớ để bổ sung vào ngữ cảnh những thông tin liên quan nhất, đồng thời hạn chế số token sử dụng. Cấu trúc tri thức và bộ nhớ có thể là *hợp nhất* (unified), khi cả hai đi theo một cấu trúc duy nhất, hoặc *lai* (hybrid), khi kết hợp nhiều hình thức truy xuất khác nhau. Định dạng tri thức và bộ nhớ rất đa dạng: từ ngôn ngữ (ví dụ tài liệu PDF), cơ sở dữ liệu (quan hệ, đối tượng hoặc tài liệu), embedding (giúp tìm kiếm tương đồng ngữ nghĩa dễ dàng nhờ biểu diễn vector), cho tới những danh sách đơn giản làm bộ nhớ cho agent.



Hình 2.6: Khám phá vai trò và cách sử dụng bộ nhớ và tri thức của agent

Hình 2.7 cho thấy thành phần **suy luận và đánh giá** (*reasoning and evaluation*) của một hệ thống agent. Nghiên cứu và ứng dụng thực tế đã cho thấy LLM/agent có thể suy luận hiệu quả. Hệ thống suy luận và đánh giá bổ sung cho quy trình làm việc của agent khả năng suy nghĩ thấu đáo về vấn đề và đánh giá các lời giải.



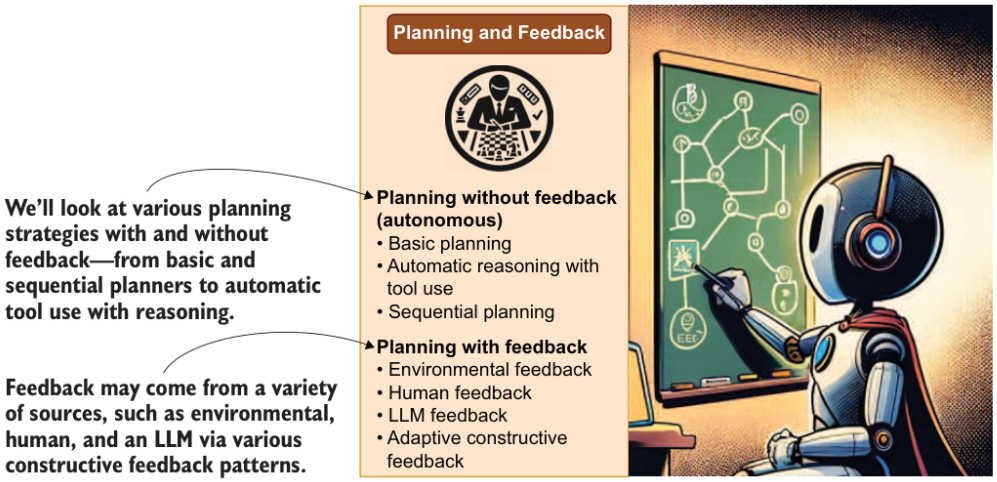
Hình 2.7: Thành phần suy luận và đánh giá cùng các chi tiết

Hình 2.8 cho thấy thành phần **lập kế hoạch/phản hồi** (*planning/feedback*) và vai trò của nó trong

việc tổ chức nhiệm vụ để đạt các mục tiêu cấp cao hơn. Thành phần này có thể chia thành hai cách tiếp cận:

- **Lập kế hoạch không có phản hồi** (*planning without feedback*): agent tự hành tự đưa ra quyết định.
- **Lập kế hoạch có phản hồi** (*planning with feedback*): việc theo dõi và điều chỉnh kế hoạch dựa trên nhiều nguồn đầu vào, bao gồm thay đổi của môi trường và phản hồi trực tiếp của con người.

- *Planning without feedback*—Autonomous agents make decisions independently.
- *Planning with feedback*—Monitoring and modifying plans is based on various sources of input, including environmental changes and direct human feedback.



Hình 2.8: Khám phá vai trò của lập kế hoạch và suy luận ở agent

Trong việc lập kế hoạch, agent có thể dùng suy luận *một đường* (single-path), tức *suy luận tuần tự* qua từng bước của nhiệm vụ, hoặc suy luận *nhiều đường* (multipath) để khám phá nhiều chiến lược và lưu lại những chiến lược hiệu quả để dùng về sau. Các bộ lập kế hoạch bên ngoài (external planner), có thể là code hoặc hệ thống agent khác, cũng có thể tham gia điều phối kế hoạch.

Bất kỳ loại agent nào đã nêu (agent/trợ lý proxy, agent/trợ lý, hay agent tự hành) đều có thể dùng một phần hoặc toàn bộ các thành phần này. Ngay cả thành phần lập kế hoạch cũng có vai trò ngoài agent tự hành và có thể tăng cường hiệu quả cho cả agent thông thường.

2.3 Sự trỗi dậy của kỷ nguyên agent: vì sao cần agent?

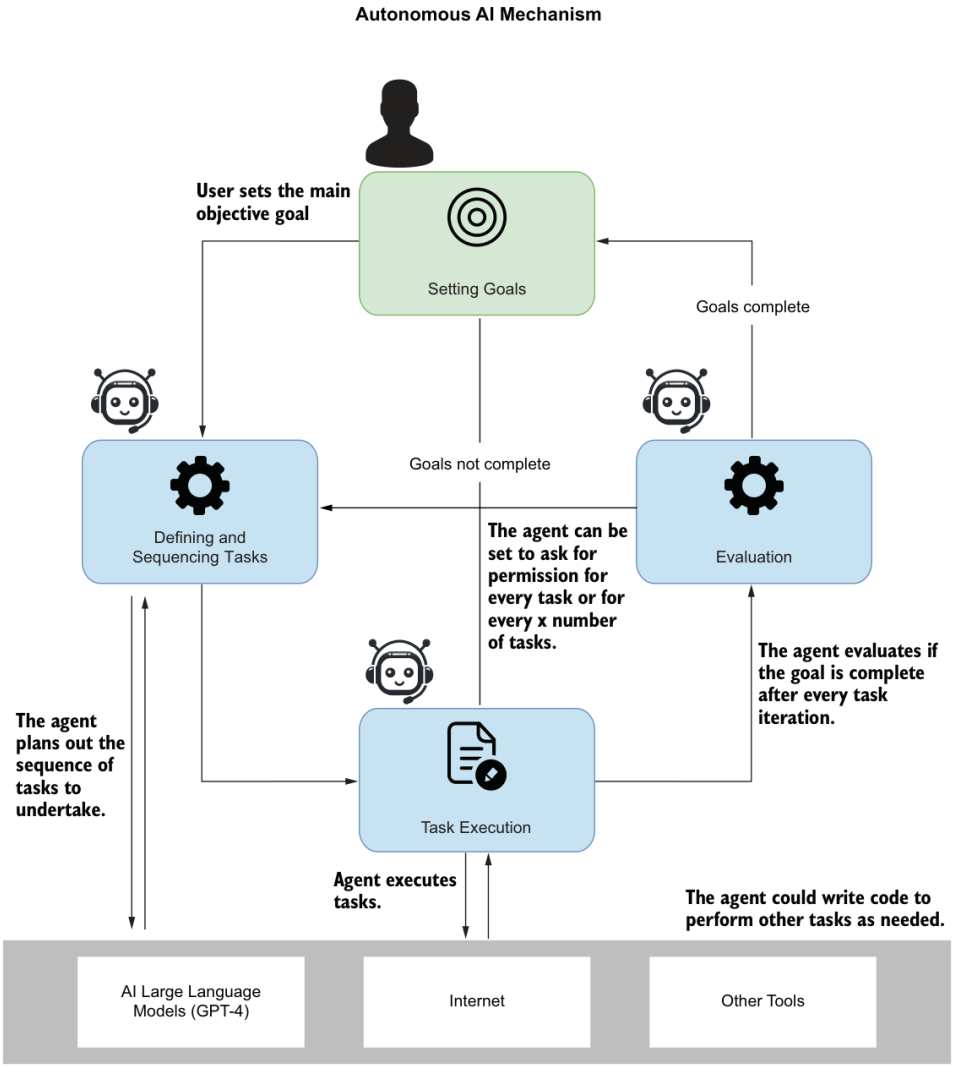
AI agent và trợ lý đã nhanh chóng chuyển từ chủ đề chính của nghiên cứu AI sang phát triển phần mềm phổ thông. Danh sách công cụ và nền tảng hỗ trợ xây dựng và tăng cường năng lực cho agent ngày càng dài. Với người ngoài cuộc, tất cả có vẻ chỉ là sự cường điệu nhằm thổi phồng giá trị của một công nghệ thú vị nhưng được đánh giá quá cao.

Trong vài tháng đầu sau khi ChatGPT ra mắt, một chuyên ngành mới gọi là **prompt engineering** (kỹ thuật viết prompt) hình thành: người dùng nhận thấy việc áp dụng nhiều kỹ thuật và khuôn mẫu trong prompt giúp họ sinh ra kết quả tốt hơn và nhất quán hơn. Tuy nhiên, người dùng cũng nhận ra prompt engineering chỉ đi được đến một mức nhất định.

Prompt engineering vẫn là cách rất tốt để tương tác trực tiếp với các LLM như ChatGPT. Theo thời gian, nhiều người dùng phát hiện ra rằng viết prompt hiệu quả đòi hỏi lặp đi lặp lại, suy ngẫm (reflection)

rồi lại lặp tiếp. Những hệ thống agent đầu tiên, như AutoGPT, ra đời từ những phát hiện này và thu hút sự chú ý của cộng đồng.

Hình 2.9 cho thấy thiết kế ban đầu của AutoGPT, một trong những hệ thống agent tự hành đầu tiên. Agent được thiết kế để lặp qua một chuỗi nhiệm vụ đã lập kế hoạch, do chính nó xác định dựa trên mục tiêu của người dùng. Qua mỗi vòng lặp nhiệm vụ, agent đánh giá mục tiêu và xác định nhiệm vụ đã hoàn thành chưa. Nếu chưa, agent có thể lập lại kế hoạch các bước và cập nhật kế hoạch dựa trên tri thức mới hoặc phản hồi của con người.



Hình 2.9: Thiết kế ban đầu của hệ thống agent AutoGPT

AutoGPT trở thành ví dụ đầu tiên cho thấy sức mạnh của việc dùng lập kế hoạch nhiệm vụ và vòng lặp với các mô hình LLM. Từ đó, đồng thời, các hệ thống và framework agent khác bùng nổ trong cộng đồng với những cơ chế lập kế hoạch và lặp nhiệm vụ tương tự. Hiện nay, người ta nhìn chung thừa nhận rằng lập kế hoạch, lặp và lặp lại là những quy trình tốt nhất để LLM giải quyết các mục tiêu phức tạp, nhiều mặt.

Tuy nhiên, hệ thống agent tự hành đòi hỏi sự tin tưởng vào quá trình ra quyết định của agent, vào hệ thống rào chắn (guardrail)/đánh giá và vào cách định nghĩa mục tiêu. Niềm tin cũng là thứ phải tích luỹ theo thời gian. Việc ta thiếu tin tưởng bắt nguồn từ việc ta chưa hiểu rõ năng lực của một agent tự hành.

Ghi chú

Trí tuệ nhân tạo tổng quát (*artificial general intelligence*, AGI) là một dạng trí tuệ có thể học để hoàn thành bất kỳ nhiệm vụ nào con người làm được. Nhiều người làm trong thế giới AI mới này tin rằng một AGI dùng các hệ thống agent tự hành là mục tiêu có thể đạt được.

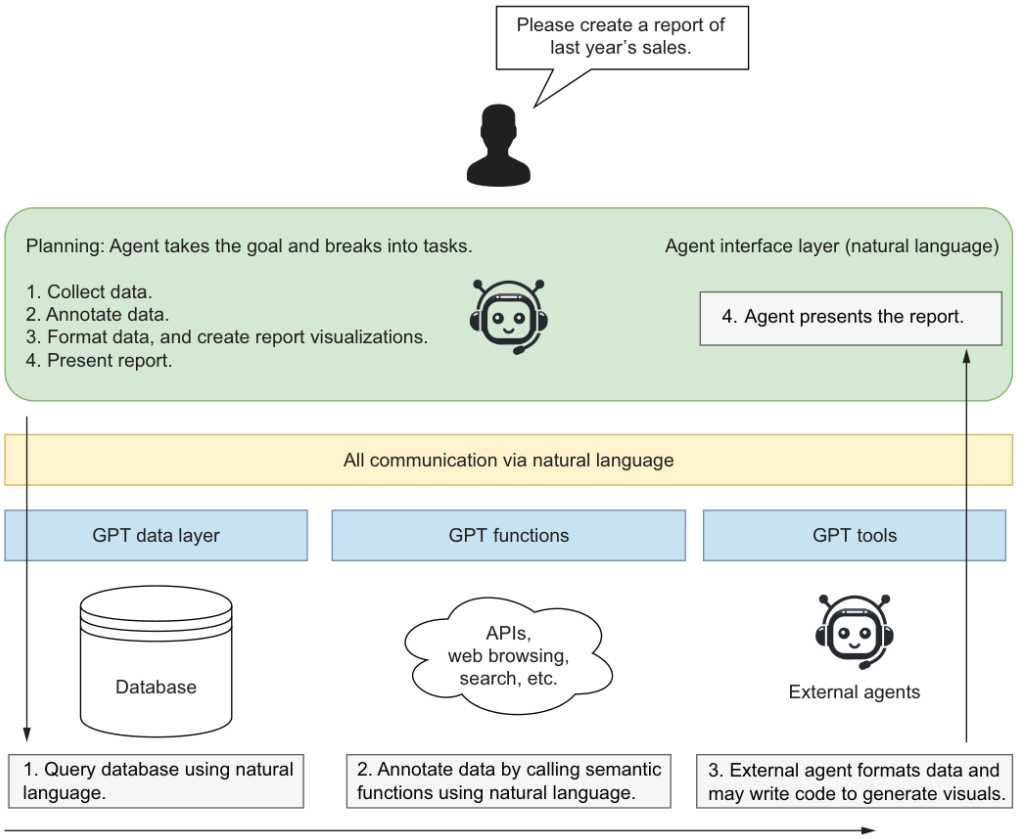
Vì lý do đó, nhiều công cụ agent phổ biến và sẵn sàng cho môi trường production không mang tính tự hành. Dù vậy, chúng vẫn mang lại lợi ích đáng kể trong việc quản lý và tự động hoá nhiệm vụ bằng GPT (LLM). Do mục tiêu của cuốn sách là hiểu mọi dạng agent, nhiều ứng dụng thực tế hơn sẽ dựa trên agent không tự hành.

Agent và công cụ agent chỉ là lớp trên cùng của một mô hình (paradigm) phát triển ứng dụng phần mềm mới. Ta sẽ xem xét mô hình mới này ở mục tiếp theo.

2.4 Bóc tách giao diện AI

Mô hình AI agent không chỉ là sự thay đổi trong cách ta làm việc với LLM mà còn được xem là sự thay đổi trong cách ta phát triển phần mềm và xử lý dữ liệu. Phần mềm và dữ liệu sẽ không còn được giao tiếp qua giao diện người dùng (UI), giao diện lập trình ứng dụng (API) và các ngôn ngữ truy vấn chuyên dụng như SQL nữa. Thay vào đó, chúng sẽ được thiết kế để giao tiếp bằng ngôn ngữ tự nhiên.

Hình 2.10 là bức tranh tổng quát về kiến trúc mới này có thể trông ra sao và vai trò của AI agent trong đó. Dữ liệu, phần mềm và ứng dụng thích nghi để hỗ trợ các giao diện ngôn ngữ tự nhiên, mang tính ngữ nghĩa. Những giao diện AI này cho phép agent thu thập dữ liệu và tương tác với các ứng dụng phần mềm, thậm chí với các agent hay ứng dụng agent khác. Đây là một bước chuyển mới trong cách ta tương tác với phần mềm và ứng dụng.



Hình 2.10: Hình dung về cách agent sẽ tương tác với các hệ thống phần mềm

Giao diện AI (*AI interface*) là một tập hợp các hàm, công cụ và lớp dữ liệu giúp phơi bày (expose) dữ liệu và ứng dụng thông qua ngôn ngữ tự nhiên. Trước đây, từ *semantic* (ngữ nghĩa) thường được dùng để mô tả các giao diện này, và thậm chí có công cụ còn dùng luôn cái tên đó; tuy nhiên, "semantic" có thể mang nhiều nghĩa và cách dùng khác nhau. Vì vậy, trong cuốn sách này ta dùng thuật ngữ *giao diện AI*.

Việc xây dựng giao diện AI sẽ trao thêm năng lực cho những agent cần sử dụng các dịch vụ, công cụ và dữ liệu. Đi cùng với đó là độ chính xác cao hơn khi hoàn thành nhiệm vụ, cùng những ứng dụng đáng tin cậy và tự chủ hơn. Dù giao diện AI có thể không phù hợp với mọi phần mềm và dữ liệu, nó sẽ chiếm ưu thế trong nhiều trường hợp sử dụng.

2.5 Định hướng trong bức tranh toàn cảnh về agent

GPT agent đại diện cho một sự thay đổi toàn diện trong cách người tiêu dùng và nhà phát triển tiếp cận mọi thứ, từ tìm kiếm thông tin tới xây dựng phần mềm và truy cập dữ liệu. Gần như ngày nào cũng có một framework, thành phần hay giao diện agent mới xuất hiện trên GitHub hoặc trong một bài báo nghiên cứu. Điều này có thể khiến người mới đang cố hiểu hệ thống agent là gì và dùng chúng ra sao cảm thấy choáng ngợp và e ngại.

Tóm tắt

- Agent là một thực thể hành động hoặc tác động quyền lực, tạo ra hiệu ứng, hoặc đóng vai trò phương tiện để đạt được kết quả. Trong AI, agent tự động hoá việc tương tác với mô hình ngôn ngữ lớn (LLM).
- Assistant (trợ lý) đồng nghĩa với agent. Cả hai thuật ngữ đều bao gồm các công cụ như GPT Assistants của OpenAI.
- Agent tự hành có thể tự ra quyết định; việc phân biệt chúng với agent không tự hành là rất quan trọng.
- Bốn kiểu tương tác chính với LLM gồm: tương tác trực tiếp của người dùng, agent/trợ lý proxy, agent/trợ lý, và agent tự hành.
- Hệ đa agent gồm các hồ sơ agent phối hợp với nhau, thường do một proxy điều khiển, để hoàn thành các nhiệm vụ phức tạp.
- Các thành phần chính của một agent gồm: hồ sơ/persona, hành động, tri thức/bộ nhớ, suy luận/đánh giá, và lập kế hoạch/phản hồi.
- Hồ sơ và persona của agent định hướng nhiệm vụ, cách phản hồi và các sắc thái khác của agent, thường bao gồm lý lịch và đặc điểm nhân khẩu học.
- Hành động và công cụ của agent có thể được tạo thủ công, gợi lại từ bộ nhớ, hoặc theo các kế hoạch định sẵn.
- Agent dùng các cấu trúc tri thức và bộ nhớ để tối ưu ngữ cảnh và giảm thiểu số token sử dụng, qua nhiều định dạng khác nhau, từ tài liệu tới embedding.
- Hệ thống suy luận và đánh giá giúp agent suy nghĩ thấu đáo về vấn đề và đánh giá lời giải bằng các khuôn mẫu prompt như zero-shot, one-shot và few-shot.
- Thành phần lập kế hoạch/phản hồi tổ chức nhiệm vụ để đạt mục tiêu, dùng suy luận một đường hoặc nhiều đường và tích hợp phản hồi từ môi trường và từ con người.

- Sự trỗi dậy của AI agent đã mở ra một mô hình phát triển phần mềm mới, chuyển từ giao diện truyền thống sang giao diện AI dựa trên ngôn ngữ tự nhiên.
- Hiểu sự phát triển và cách tương tác của các công cụ này giúp ta xây dựng hệ thống agent, dù là agent đơn lẻ, đa agent hay agent tự hành.

Chương 3

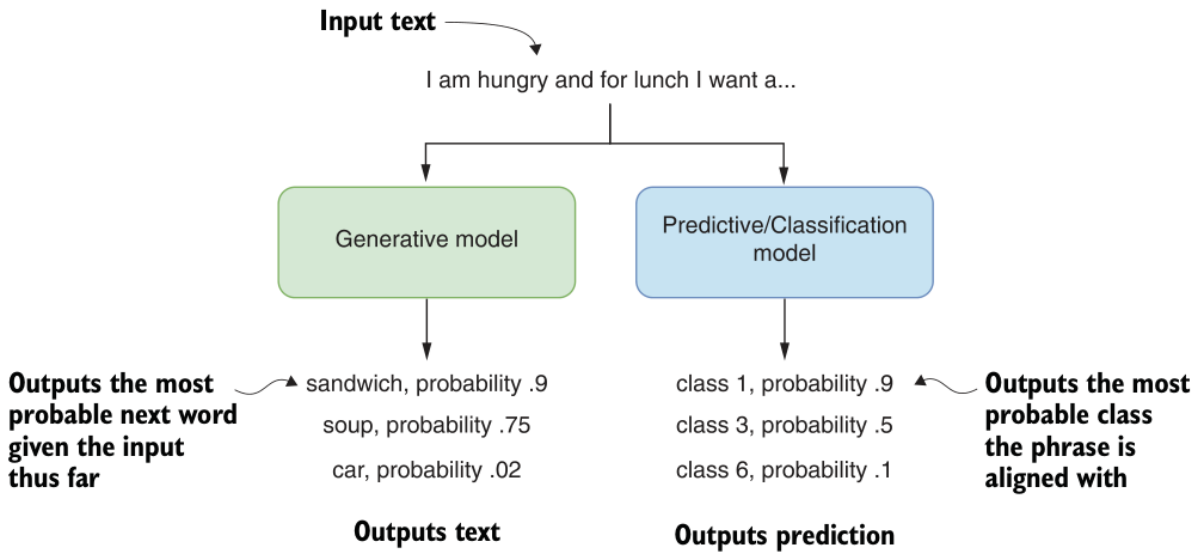
Khai thác sức mạnh của mô hình ngôn ngữ lớn

Chương này bao quát các nội dung:

- Hiểu những điều cơ bản về LLM.
- Kết nối tới và sử dụng OpenAI API.
- Khám phá và sử dụng LLM mã nguồn mở với LM Studio.
- Viết prompt cho LLM bằng kỹ thuật prompt engineering.
- Chọn LLM tối ưu cho nhu cầu cụ thể của bạn.

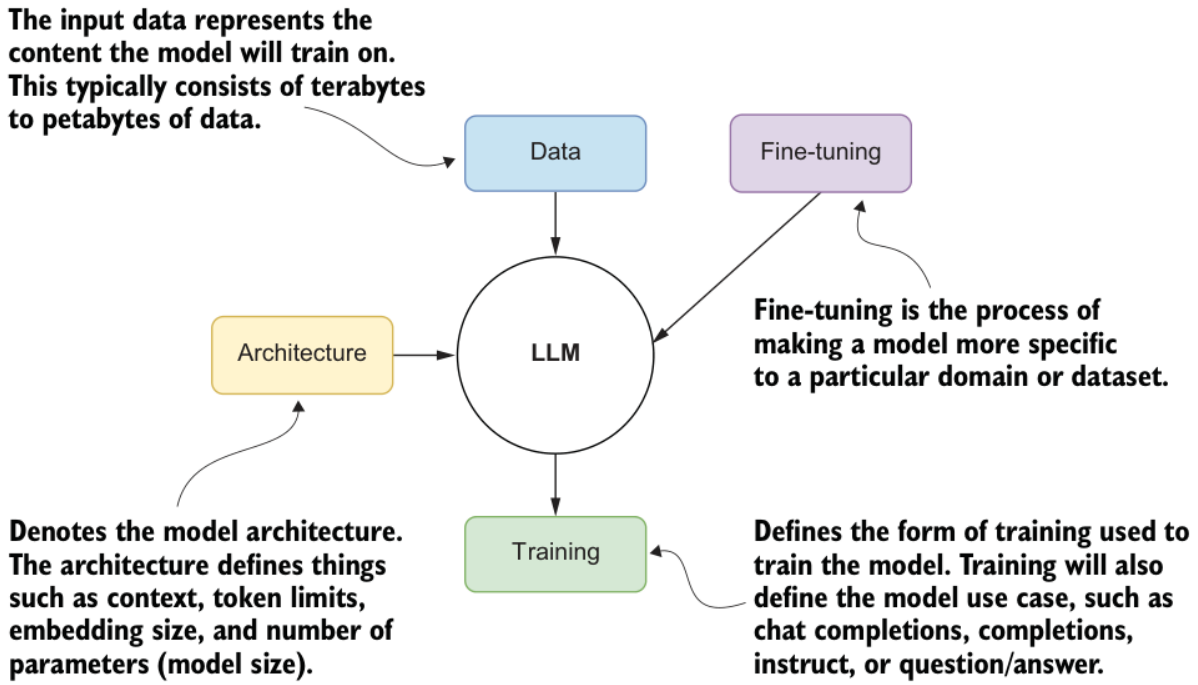
Thuật ngữ **mô hình ngôn ngữ lớn** (*large language model*, LLM) nay đã trở thành cách gọi phổ biến cho một dạng AI. Các LLM này được phát triển dựa trên **generative pretrained transformer** (GPT, transformer tiền huấn luyện sinh). Tuy cũng có những kiến trúc khác làm nền cho LLM, dạng GPT hiện là dạng thành công nhất.

LLM và GPT là các mô hình **sinh** (*generative*): chúng được huấn luyện để *tạo ra* nội dung, thay vì dự đoán hay phân loại nội dung. Hình 3.1 minh hoạ sự khác biệt giữa mô hình sinh và mô hình dự đoán/phân loại. Mô hình sinh tạo ra một thứ gì đó từ đầu vào, còn mô hình dự đoán và phân loại thì phân loại đầu vào đó.



Hình 3.1: Sự khác biệt giữa mô hình sinh và mô hình dự đoán

Ta có thể định nghĩa thêm một LLM qua các thành phần cấu tạo của nó, như trong Hình 3.2. Trong sơ đồ này, *dữ liệu* (*data*) là nội dung dùng để huấn luyện mô hình, còn *kiến trúc* (*architecture*) là thuộc tính của chính mô hình, chẳng hạn số tham số hay kích thước mô hình. Mô hình còn được huấn luyện riêng cho trường hợp sử dụng mong muốn, gồm chat, completion (hoàn thành văn bản) hoặc instruction (làm theo chỉ dẫn). Cuối cùng, **fine-tuning** (tinh chỉnh) là một đặc điểm được bổ sung vào mô hình, giúp tinh lọc dữ liệu đầu vào và quá trình huấn luyện để khớp hơn với một trường hợp sử dụng hay lĩnh vực cụ thể.



Hình 3.2: Các thành phần chính mô tả một LLM

Kiến trúc transformer của GPT, một kiến trúc cụ thể của LLM, cho phép mở rộng mô hình lên tới hàng tỷ tham số. Điều này đòi hỏi các mô hình lớn ấy phải được huấn luyện trên hàng terabyte tài liệu để xây nền tảng. Từ nền tảng đó, mô hình tiếp tục được huấn luyện nối tiếp bằng nhiều phương pháp khác nhau cho trường hợp sử dụng mong muốn.

Ví dụ, ChatGPT về cơ bản được huấn luyện trên internet công khai, rồi được fine-tune bằng nhiều chiến lược huấn luyện. Bước fine-tuning cuối cùng dùng một dạng nâng cao gọi là **học tăng cường từ phản hồi của con người** (*reinforcement learning with human feedback*, RLHF). Kết quả là một loại mô hình theo trường hợp sử dụng gọi là chat completions.

Các LLM **chat completions** được thiết kế để cải thiện qua quá trình lặp và tinh chỉnh, nói cách khác là qua việc trò chuyện (chat). Các mô hình này cũng được đánh giá (benchmark) là tốt nhất về hoàn thành nhiệm vụ, suy luận và lập kế hoạch, nên rất phù hợp để xây dựng agent và trợ lý (assistant). Mô hình completion chỉ được huấn luyện/thiết kế để sinh nội dung từ văn bản đầu vào, nên chúng không được lợi từ việc lặp.

Trong hành trình xây dựng các agent mạnh mẽ ở cuốn sách này, ta tập trung vào lớp LLM gọi là mô hình chat completions. Dĩ nhiên điều đó không ngăn bạn thử các dạng mô hình khác cho agent của mình. Tuy nhiên, bạn có thể phải sửa nhiều ở các đoạn code mẫu để hỗ trợ các dạng mô hình khác.

Ta sẽ tìm hiểu thêm chi tiết về LLM và GPT ở phần sau của chương, khi chạy một LLM mã nguồn mở trên máy cục bộ. Ở mục tiếp theo, ta xem cách kết nối tới một LLM bằng một chuẩn ngày càng phổ biến của OpenAI.

3.1 Làm chủ OpenAI API

Nhiều dự án AI agent và trợ lý dùng OpenAI API SDK để kết nối tới LLM. Tuy chưa phải là chuẩn chính thức, các khái niệm cơ bản mô tả một kết nối giờ đều đi theo khuôn mẫu của OpenAI. Vì vậy, ta cần hiểu các khái niệm cốt lõi của một kết nối LLM thông qua OpenAI SDK.

Chương này xem cách kết nối tới một mô hình LLM bằng SDK/package Python của OpenAI. Ta sẽ bàn về việc kết nối tới mô hình GPT-4, phản hồi của mô hình, đếm token và cách định nghĩa các message nhất quán. Bắt đầu từ tiểu mục sau, ta sẽ xem cách dùng OpenAI.

3.1.1 Kết nối tới mô hình chat completions

Để làm các bài thực hành trong mục này và các mục sau, bạn cần thiết lập môi trường phát triển Python và có quyền truy cập một LLM. Phụ lục A hướng dẫn tạo tài khoản OpenAI và truy cập GPT-4 hoặc các mô hình khác. Phụ lục B trình bày cách thiết lập môi trường phát triển Python với Visual Studio Code (VS Code), kể cả cài các extension cần thiết. Hãy xem lại các phần này nếu bạn muốn làm theo các kịch bản trong sách.

Bắt đầu bằng cách mở thư mục mã nguồn `chapter_2` trong VS Code và tạo một môi trường ảo (virtual environment) Python mới. Một lần nữa, hãy xem phụ lục B nếu cần trợ giúp.

Sau đó, cài package OpenAI và package Python dot environment bằng lệnh trong listing sau. Lệnh này cài các package cần thiết vào môi trường ảo.

Listing 2.1 Cài đặt bằng pip

```
pip install openai python-dotenv
```

Tiếp theo, mở file `connecting.py` trong VS Code và xem đoạn code trong Listing 2.2. Nhớ đặt tên mô hình cho phù hợp, ví dụ `gpt-4`. Tại thời điểm viết sách, `gpt-4-1106-preview` được dùng để chỉ GPT-4 Turbo.

Listing 2.2 connecting.py

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()                           # Nạp các bí mật lưu trong file .env
api_key = os.getenv('OPENAI_API_KEY')
if not api_key:                             # Kiểm tra xem key đã được đặt chưa
    raise ValueError("No API key found. Please check your .env file.")
client = OpenAI(api_key=api_key)                        # Tạo client với key

def ask_chatgpt(user_message):
    response = client.chat.completions.create(     # Dùng hàm create để sinh phản
    ↪ hồi
        model="gpt-4-1106-preview",
        messages=[{"role": "system",
 "content": "You are a helpful assistant."},
        {"role": "user", "content": user_message}],
        temperature=0.7,
        )
    return response.choices[0].message.content    # Chỉ trả về phần nội dung của
    ↪ phản hồi

user = "What is the capital of France?"
response = ask_chatgpt(user)                # Thực thi yêu cầu và trả về phản hồi
print(response)
```

Có khá nhiều thứ diễn ra ở đây, nên ta chia nhỏ theo từng phần, bắt đầu từ đầu file với việc nạp các biến môi trường. Trong thư mục `chapter_2` có một file khác tên `.env`, chứa các biến môi trường. Các biến này được tự động thiết lập khi gọi hàm `load_dotenv`.

Bạn phải đặt OpenAI API key của mình trong file `.env`, như trong listing tiếp theo. Một lần nữa, hãy xem phụ lục A để biết cách lấy key và tìm tên mô hình.

Listing 2.3 .env

```
OPENAI_API_KEY='your-openai-api-key'
```

Sau khi đặt key, bạn có thể debug file bằng cách nhấn phím F5 hoặc chọn Run > Start Debugging trên menu VS Code. Thao tác này chạy code, và bạn sẽ thấy kết quả kiểu như "The capital of France is Paris."

Hãy nhớ rằng phản hồi của mô hình sinh phụ thuộc vào xác suất. Trong trường hợp này, nhiều khả năng mô hình sẽ cho ta câu trả lời đúng và nhất quán.

Bạn có thể điều chỉnh các xác suất này bằng cách thay đổi **temperature** (nhiệt độ) của yêu cầu. Nếu muốn mô hình nhất quán hơn, hãy giảm temperature xuống 0; nếu muốn mô hình tạo ra nhiều biến thể hơn, hãy tăng temperature lên. Ta sẽ tìm hiểu kỹ hơn việc đặt temperature trong mục tiếp theo.

3.1.2 Hiểu yêu cầu và phản hồi

Đi sâu vào các thành phần của yêu cầu (request) và phản hồi (response) chat completions sẽ rất có ích. Ta tập trung vào yêu cầu trước, như dưới đây. Yêu cầu gói gọn mô hình được chọn, các message và temperature.

Listing 2.4 Yêu cầu chat completions

```
response = client.chat.completions.create(
    model="gpt-4-1106-preview",                # Mô hình hoặc deployment dùng để
    ↪  trả lời yêu cầu
    messages=[{"role": "system",
"content": "You are a helpful assistant."},                    # Message vai trò
↪  system
              {"role": "user", "content": user_message}],     # Message vai trò
              ↪  user
    temperature=0.7,    # Temperature, tức độ biến thiên của yêu cầu
    )
```

Trong yêu cầu, khối `messages` mô tả một tập các message cùng vai trò (role) được dùng trong yêu cầu. Message cho mô hình chat completions có thể thuộc một trong ba vai trò:

- *Vai trò system*: message mô tả các quy tắc và hướng dẫn của yêu cầu. Nó thường được dùng để mô tả vai trò của LLM khi thực hiện yêu cầu.
- *Vai trò user*: đại diện và chứa message từ người dùng.
- *Vai trò assistant*: có thể dùng để lưu lịch sử các phản hồi trước đó của LLM. Nó cũng có thể dùng để chèn một lịch sử message vào khi trước đó có thể chưa hề có lịch sử nào.

Message gửi trong một yêu cầu duy nhất có thể gói gọn cả một cuộc hội thoại, như trong đoạn JSON ở listing sau.

Listing 2.5 Các message kèm lịch sử

```
[
    {
        "role": "system",
        "content": "You are a helpful assistant."
    },
    {
        "role": "user",
        "content": "What is the capital of France?"
    },
    {
        "role": "assistant",
        "content": "The capital of France is Paris."
    },
    {
        "role": "user",
        "content": "What is an interesting fact of Paris."
    }
],
```

Bạn có thể xem cách áp dụng điều này bằng cách mở `message_history.py` trong VS Code và debug bằng phím F5. Sau khi file chạy xong, hãy kiểm tra kết quả. Sau đó, chạy lại ví dụ thêm vài lần để thấy kết quả thay đổi thế nào.

Kết quả sẽ khác nhau giữa các lần chạy do temperature khá cao, `.7`. Hãy giảm temperature xuống `.0` và chạy ví dụ `message_history.py` thêm vài lần. Giữ temperature ở `0` sẽ cho kết quả giống hoặc tương tự nhau mỗi lần.

Việc đặt temperature cho một yêu cầu thường phụ thuộc vào trường hợp sử dụng cụ thể. Đôi khi bạn muốn hạn chế tính ngẫu nhiên (stochastic) của phản hồi. Giảm temperature xuống `0` sẽ cho kết quả nhất quán. Ngược lại, giá trị `1.0` sẽ cho độ biến thiên lớn nhất trong các phản hồi.

Tiếp theo, ta cũng muốn biết thông tin nào được trả về cho mỗi yêu cầu. Listing tiếp theo cho thấy định dạng đầu ra của phản hồi. Bạn có thể xem đầu ra này bằng cách chạy file `message_history.py` trong VS Code.

Listing 2.6 Phản hồi chat completions

```
{
    "id": "chatcmpl-8WWL23up3IRfK1nrDFQ3EHQfhx0U6",
    "choices": [                                        // Mô hình có thể trả về
    ↪  nhiều hơn một phản hồi.
        {
            "finish_reason": "stop",
            "index": 0,
            "message": {
                "content": "… omitted",
                "role": "assistant",      // Phản hồi được trả về với vai trò
                ↪  assistant
                "function_call": null,
                "tool_calls": null
            },
            "logprobs": null
        }
    ],
    "created": 1702761496,
    "model": "gpt-4-1106-preview",    // Cho biết mô hình đã dùng
    "object": "chat.completion",
    "system_fingerprint": "fp_3905aa4f79",
    "usage": {
```

```
        "completion_tokens": 78,    // Đếm số token đầu vào (prompt)
        "prompt_tokens": 48,        //   và đầu ra (completion)
        "total_tokens": 126         //   đã sử dụng
    }
}
```

Việc theo dõi số **token đầu vào** (input token, các token dùng trong prompt) và **token đầu ra** (output token, số token trả về qua completion) có thể rất hữu ích. Đôi khi, việc giảm thiểu số token là điều thiết yếu. Ít token hơn thường có nghĩa là tương tác với LLM rẻ hơn, phản hồi nhanh hơn, và cho kết quả tốt hơn, nhất quán hơn.

Đến đây ta đã nắm những điều cơ bản về kết nối tới một LLM và nhận phản hồi. Xuyên suốt cuốn sách, ta sẽ ôn lại và mở rộng cách tương tác với LLM. Trước mắt, ở mục tiếp theo ta sẽ xem cách nạp và sử dụng LLM mã nguồn mở.

3.2 Khám phá LLM mã nguồn mở với LM Studio

Các LLM thương mại, như GPT-4 của OpenAI, là điểm khởi đầu tuyệt vời để học cách dùng AI hiện đại và xây dựng agent. Tuy nhiên, chúng là tài nguyên bên ngoài: tốn chi phí, giảm quyền riêng tư và bảo mật dữ liệu, và tạo ra sự phụ thuộc. Các tác động bên ngoài khác còn có thể làm những yếu tố này phức tạp thêm.

Không có gì ngạc nhiên khi cuộc đua xây dựng các LLM mã nguồn mở có năng lực tương đương ngày càng cạnh tranh. Kết quả là giờ đã có những LLM mã nguồn mở đủ dùng cho nhiều nhiệm vụ và hệ thống agent. Chỉ trong một năm, công cụ đã tiến bộ đến mức việc tự host LLM trên máy cục bộ giờ rất dễ dàng, như ta sẽ thấy ở mục tiếp theo.

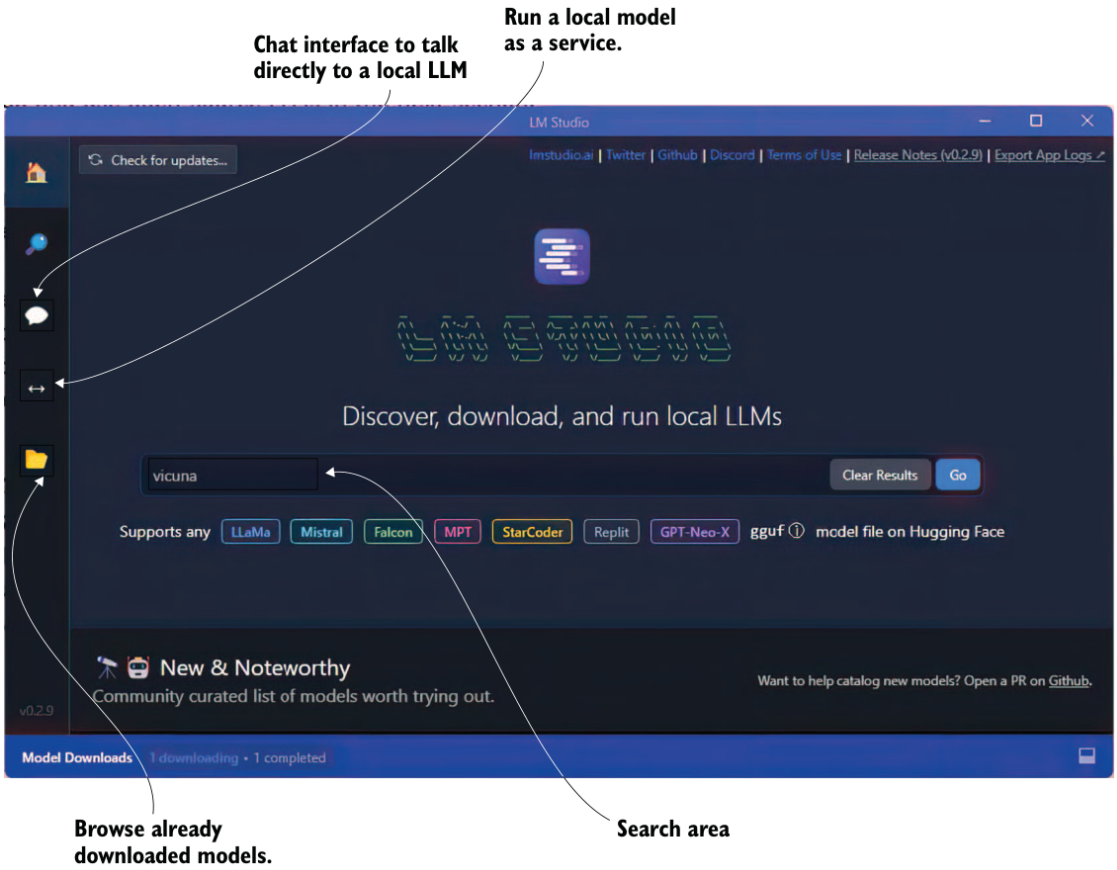
3.2.1 Cài đặt và chạy LM Studio

LM Studio là phần mềm tải miễn phí, hỗ trợ tải về và host LLM cùng các mô hình khác trên máy cục bộ cho Windows, Mac và Linux. Phần mềm dễ dùng và có nhiều tính năng hữu ích giúp bạn bắt đầu nhanh. Tóm tắt các bước tải và cài đặt LM Studio:

1. Tải LM Studio từ `https://lmstudio.ai/`.
2. Sau khi tải, cài phần mềm theo hệ điều hành của bạn. Lưu ý rằng một số phiên bản LM Studio có thể đang ở dạng beta và cần cài thêm công cụ hoặc thư viện.
3. Khởi chạy phần mềm.

Hình 3.3 cho thấy cửa sổ LM Studio đang chạy. Từ đây, bạn có thể xem danh sách các mô hình đang được quan tâm (hot), tìm các mô hình khác, và thậm chí tải về. Nội dung trang chủ rất hữu ích để hiểu chi tiết và thông số của các mô hình hàng đầu.

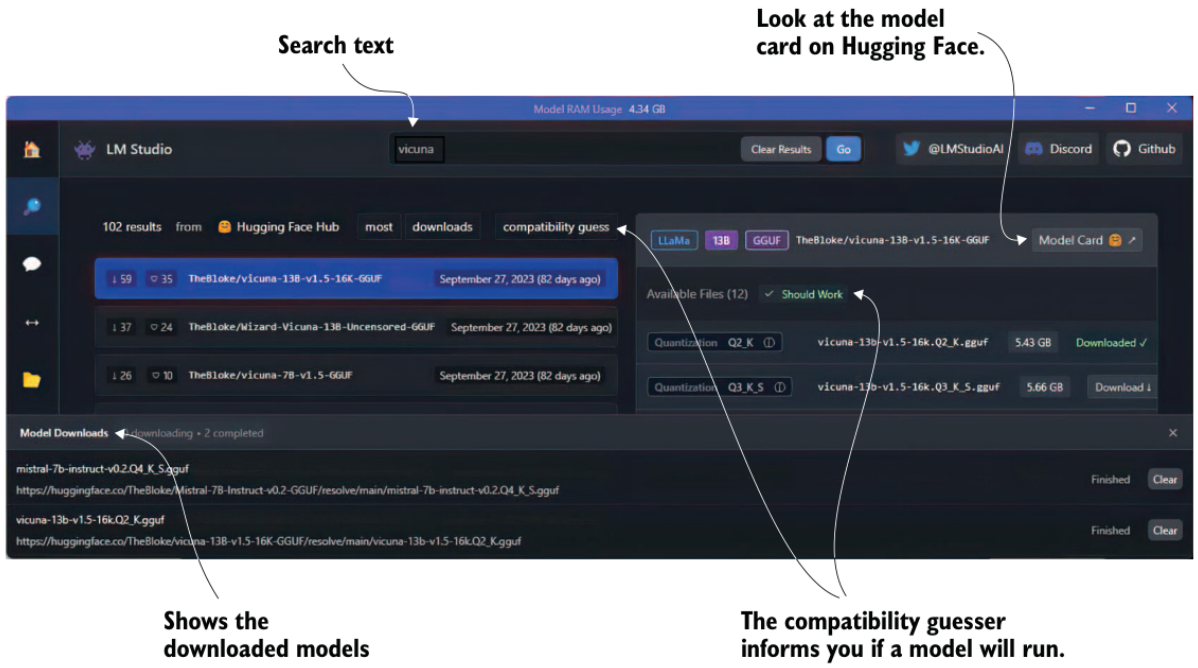
Một tính năng hấp dẫn của LM Studio là khả năng phân tích phần cứng của bạn và đối chiếu với yêu cầu của từng mô hình. Phần mềm sẽ cho bạn biết bạn có thể chạy một mô hình tốt đến mức nào. Điều này giúp tiết kiệm nhiều thời gian khi chọn mô hình để thử nghiệm.



Hình 3.3: Phần mềm LM Studio hiển thị trang chủ

Nhập một đoạn văn bản để tìm mô hình và nhấn Go. Bạn sẽ được đưa tới giao diện trang tìm kiếm, như trong Hình 3.4. Từ trang này, bạn có thể thấy tất cả các biến thể của mô hình cùng các thông số khác, chẳng hạn kích thước context (số token ngữ cảnh). Sau khi nhấn nút Compatibility Guess, phần mềm còn cho bạn biết mô hình có chạy được trên hệ thống của bạn hay không.

Nhấn để tải về bất kỳ mô hình nào chạy được trên hệ thống của bạn. Bạn nên ưu tiên các mô hình được thiết kế cho chat completions, nhưng nếu hệ thống bị hạn chế thì cứ dùng những gì bạn có. Ngoài ra, nếu chưa chắc nên dùng mô hình nào, cứ tải về và thử. LM Studio là cách tuyệt vời để khám phá và thử nghiệm nhiều mô hình.

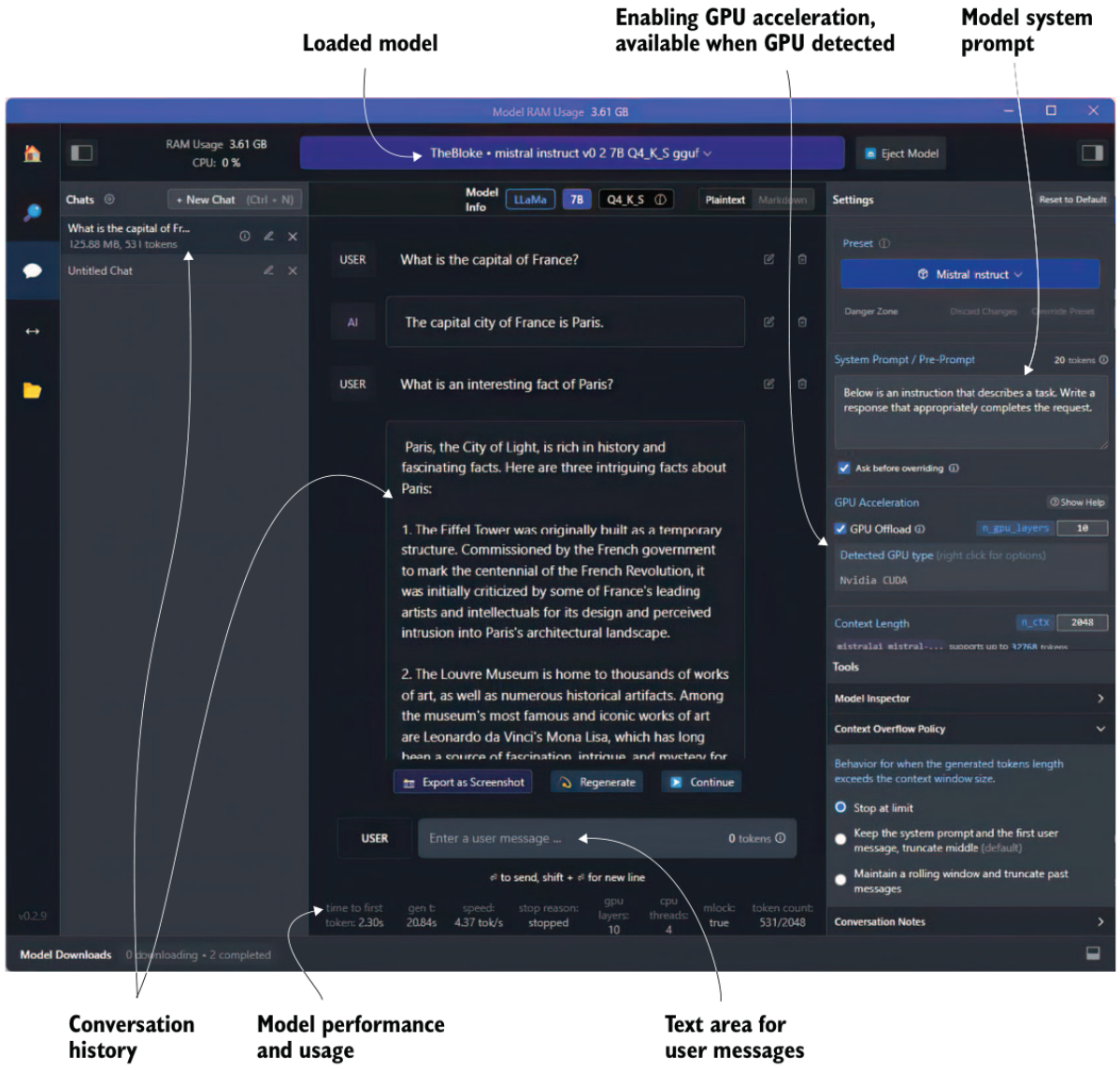


Hình 3.4: Trang tìm kiếm của LM Studio

Sau khi tải mô hình về, bạn có thể nạp và chạy mô hình ở trang chat, hoặc chạy như một server ở trang server. Hình 3.5 cho thấy việc nạp và chạy một mô hình ở trang chat. Hình cũng cho thấy tuỳ chọn bật và dùng GPU nếu bạn có.

Để nạp và chạy một mô hình, mở menu thả xuống ở giữa phía trên trang và chọn một mô hình đã tải. Một thanh tiến trình sẽ hiện ra cho biết mô hình đang được nạp; khi mô hình sẵn sàng, bạn có thể bắt đầu gõ vào giao diện.

Phần mềm còn cho phép dùng một phần hoặc toàn bộ GPU (nếu phát hiện được) để chạy suy luận (inference) cho mô hình. Nhìn chung, GPU giúp rút ngắn thời gian phản hồi của mô hình ở một mức độ nào đó. Bạn có thể thấy việc thêm GPU ảnh hưởng thế nào đến hiệu năng mô hình qua trạng thái hiệu năng ở cuối trang, như trong Hình 3.5.



Hình 3.5: Trang chat của LM Studio với một LLM đã được nạp và đang chạy cục bộ

Trò chuyện với mô hình và thử nghiệm nhiều prompt khác nhau có thể giúp bạn xác định mô hình hoạt động tốt đến đâu cho trường hợp sử dụng của mình. Một cách tiếp cận có hệ thống hơn là dùng công cụ prompt flow để đánh giá prompt và LLM. Ta sẽ mô tả cách dùng prompt flow ở Chương 10.

LM Studio cũng cho phép chạy mô hình trên một server và truy cập nó bằng package OpenAI. Ta sẽ xem cách dùng tính năng server và phục vụ (serve) một mô hình ở mục tiếp theo.

3.2.2 Phục vụ LLM cục bộ với LM Studio

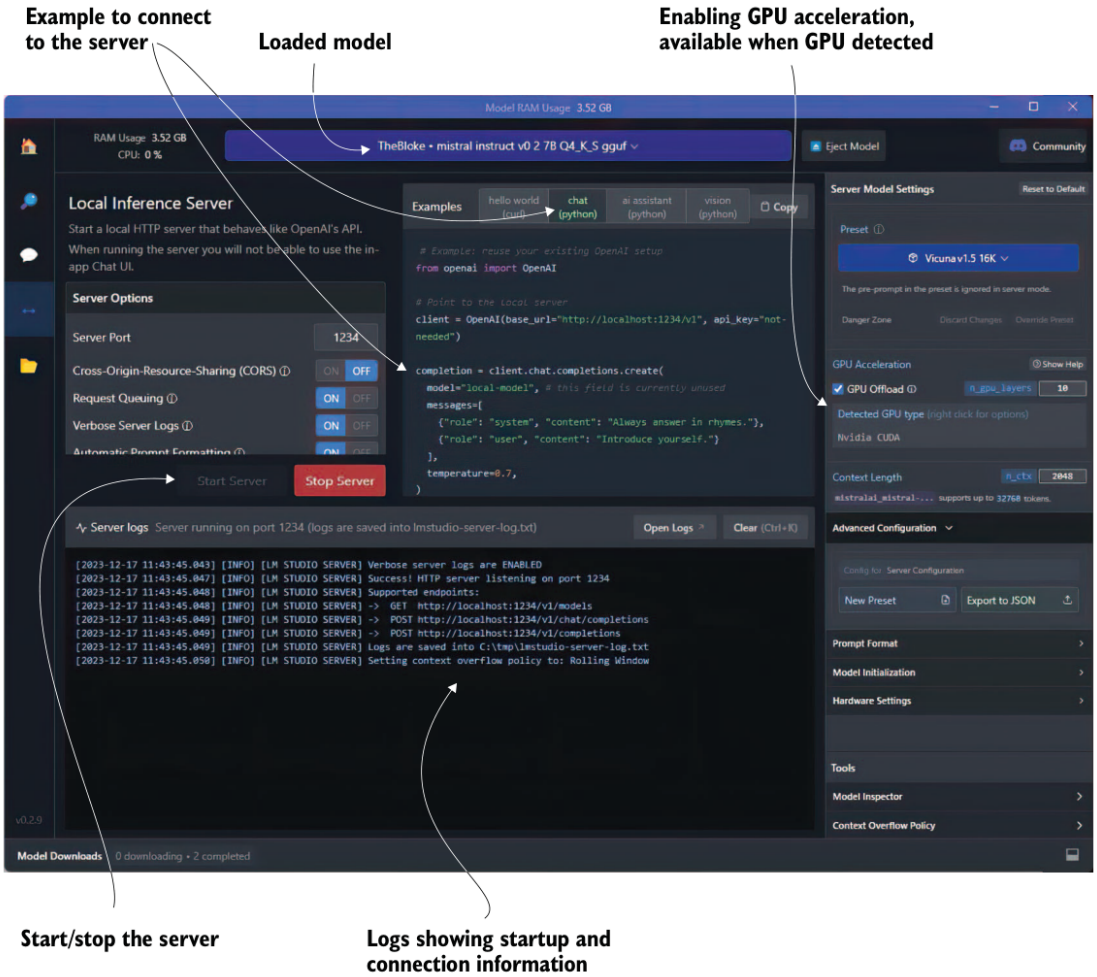
Chạy một LLM cục bộ dưới dạng server rất dễ với LM Studio. Chỉ cần mở trang server, nạp một mô hình, rồi nhấn nút Start Server, như trong Hình 3.6. Từ đó, bạn có thể sao chép và dán bất kỳ ví dụ nào để kết nối với mô hình của mình.

Bạn có thể xem ví dụ code Python bằng cách mở `chapter_2/lmstudio_server.py` trong VS Code. Code này cũng được trình bày trong Listing 2.7. Sau đó, chạy code trong trình debug của VS Code (nhấn F5).

Listing 2.7 lmstudio_server.py

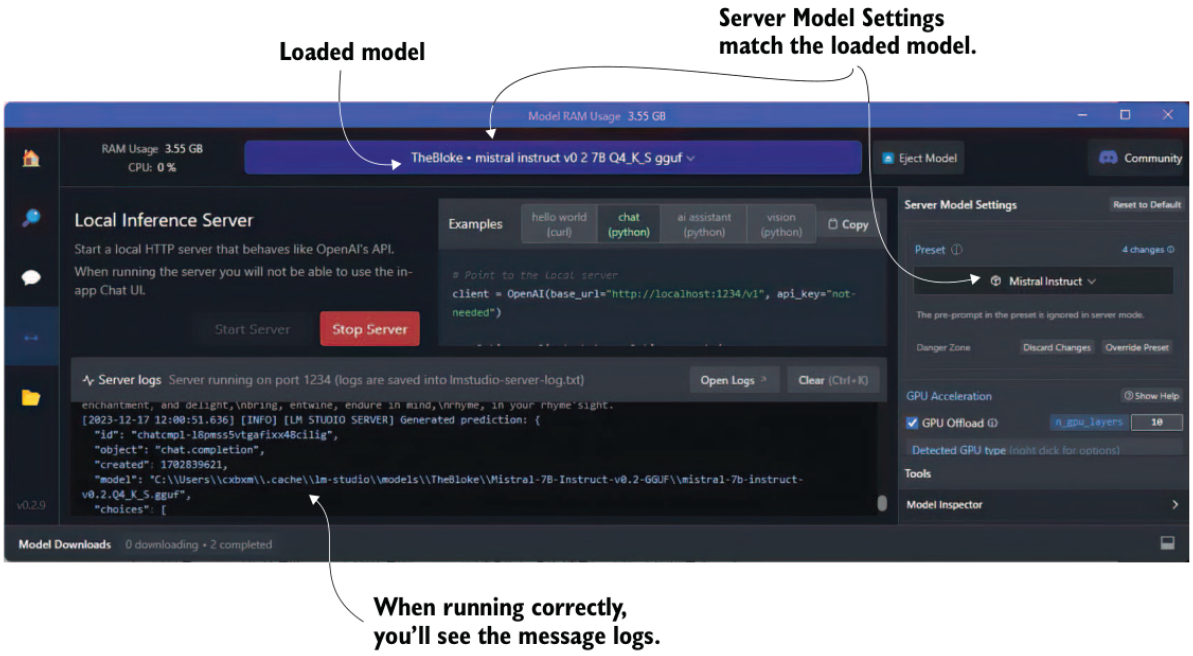
```
from openai import OpenAI
```

```
client = OpenAI(base_url="http://localhost:1234/v1", api_key="not-needed")

completion = client.chat.completions.create(
  model="local-model",                          # Hiện chưa dùng; đặt gì cũng
  ↪  được
  messages=[
    {"role": "system", "content": "Always answer in rhymes."},
    {"role": "user", "content": "Introduce yourself."}      # Tuỳ ý thay đổi
    ↪  message theo ý bạn.
  ],
  temperature=0.7,
)

print(completion.choices[0].message)     # Code mặc định in ra toàn bộ message.
```



Hình 3.6: Trang server của LM Studio và một server đang chạy LLM

Nếu gặp sự cố khi kết nối tới server hay bất kỳ vấn đề nào khác, hãy đảm bảo cấu hình Server Model Settings khớp với loại mô hình. Ví dụ, ở Hình 3.6 phía trên, mô hình được nạp khác với thiết lập của server. Thiết lập đã sửa đúng được trình bày trong Hình 3.7.



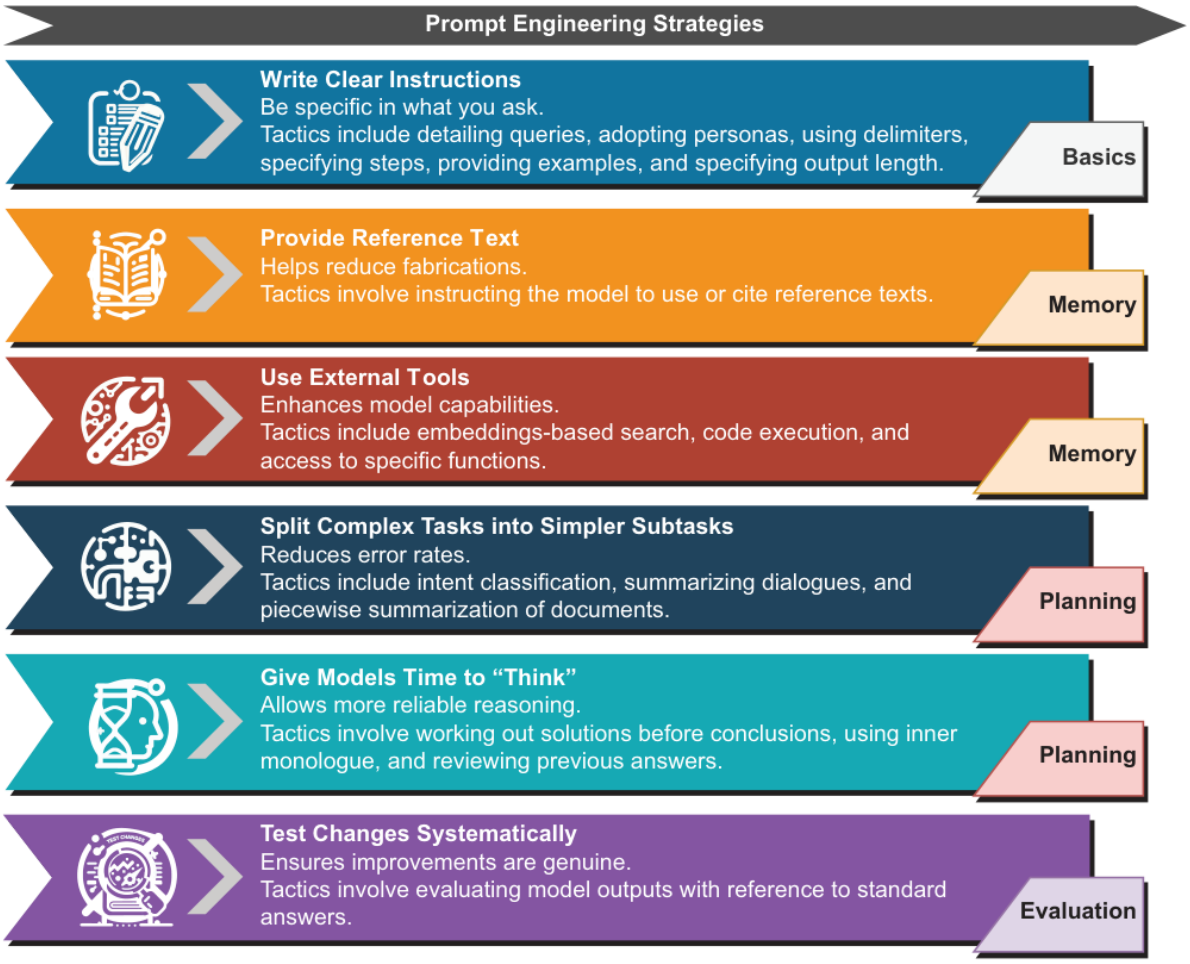
Hình 3.7: Chọn đúng Server Model Settings cho mô hình đã nạp

Giờ đây, bạn có thể dùng một LLM host cục bộ hoặc một mô hình thương mại để xây dựng, kiểm thử, và thậm chí có thể vận hành các agent của mình. Mục tiếp theo sẽ xem cách xây dựng prompt hiệu quả hơn bằng prompt engineering.

3.3 Viết prompt cho LLM bằng prompt engineering

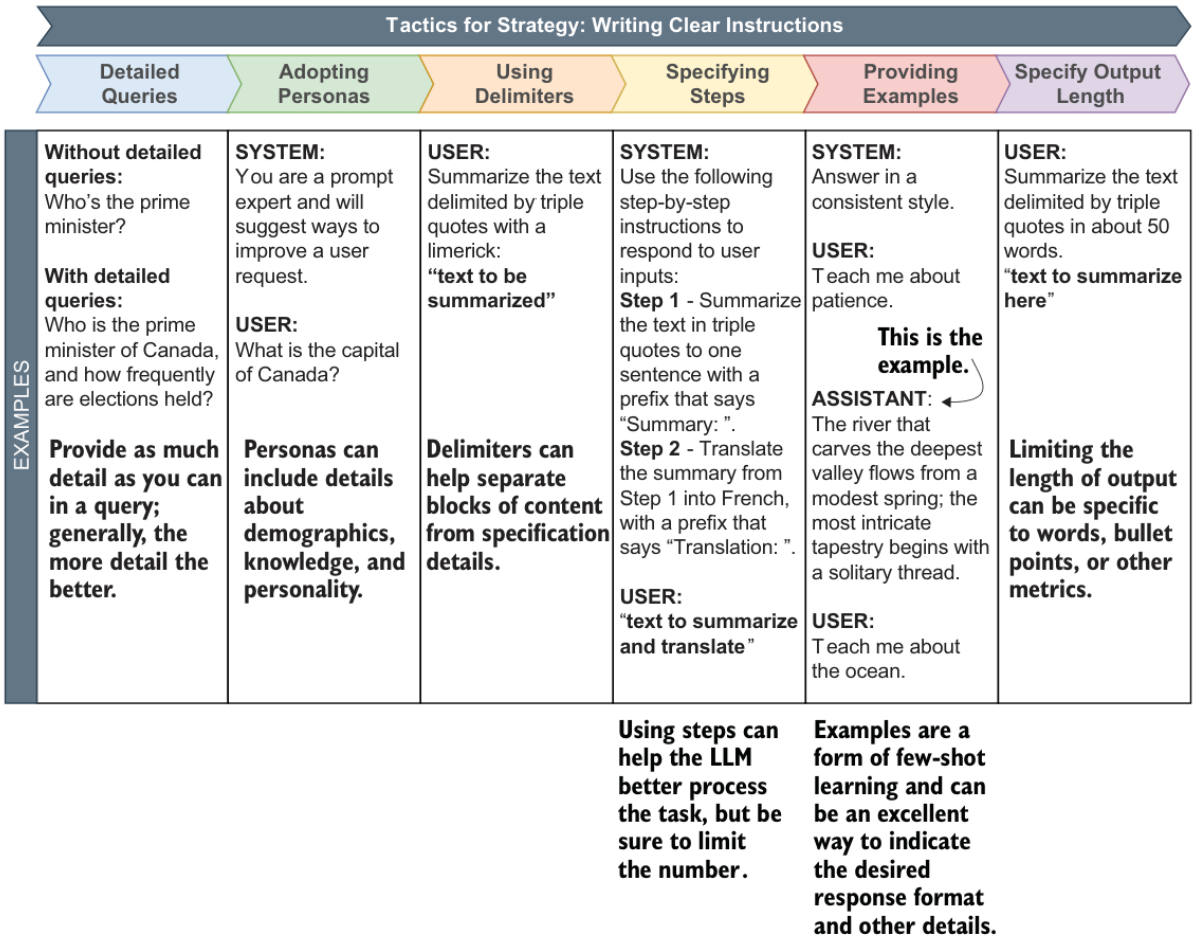
Với LLM, prompt là nội dung message được dùng trong yêu cầu nhằm thu được phản hồi tốt hơn. **Prompt engineering** (kỹ thuật thiết kế prompt) là một lĩnh vực mới đang hình thành, tìm cách xây dựng một phương pháp luận có cấu trúc cho việc viết prompt. Đáng tiếc là việc xây dựng prompt chưa phải một ngành khoa học được thiết lập vững chắc, và tập hợp các phương pháp được gọi là prompt engineering ngày càng nhiều và đa dạng.

May mắn là các tổ chức như OpenAI đã bắt đầu ghi lại một bộ chiến lược (strategy) chung, như trong Hình 3.8. Các chiến lược này bao gồm nhiều chiến thuật (tactic) khác nhau, một số đòi hỏi thêm hạ tầng và cân nhắc riêng. Vì vậy, các chiến lược prompt engineering liên quan đến những khái niệm nâng cao hơn sẽ được trình bày ở các chương được chỉ ra trong hình.



Hình 3.8: Các chiến lược prompt engineering của OpenAI được xem xét trong sách, kèm vị trí chương

Mỗi chiến lược trong Hình 3.8 lại được triển khai thành các chiến thuật giúp tinh chỉnh thêm phương pháp prompt engineering cụ thể. Chương này xem xét chiến lược nền tảng *Write Clear Instructions* (viết chỉ dẫn rõ ràng). Hình 3.9 trình bày chi tiết hơn các chiến thuật của chiến lược này, kèm ví dụ cho từng chiến thuật. Ta sẽ chạy các ví dụ này bằng một chương trình demo ở các mục sau.



Hình 3.9: Các chiến thuật của chiến lược Write Clear Instructions

Chiến lược Write Clear Instructions là việc cẩn thận và cụ thể về điều bạn yêu cầu. Yêu cầu một LLM thực hiện một nhiệm vụ không khác gì yêu cầu một người hoàn thành cùng nhiệm vụ đó. Nhìn chung, bạn càng cung cấp nhiều thông tin và ngữ cảnh liên quan tới nhiệm vụ trong yêu cầu thì phản hồi càng tốt.

Chiến lược này được chia thành các chiến thuật cụ thể mà bạn có thể áp dụng vào prompt. Để hiểu cách dùng chúng, thư mục mã nguồn `chapter 2` có một chương trình demo (`prompt_engineering.py`) với nhiều ví dụ prompt.

Mở file `prompt_engineering.py` trong VS Code, như trong Listing 2.8. Code này bắt đầu bằng việc nạp tất cả các file JSON Lines trong thư mục `prompts`. Sau đó, nó hiển thị danh sách file dưới dạng các lựa chọn và cho phép người dùng chọn một tuỳ chọn prompt. Sau khi chọn, các prompt được gửi tới một LLM và phản hồi được in ra.

Listing 2.8 prompt_engineering.py (main())

```
def main():
    directory = "prompts"
    text_files = list_text_files_in_directory(directory)   # Thu thập mọi file
    ↪ trong thư mục

    if not text_files:
        print("No text files found in the directory.")
        return
```

```
    def print_available():                          # In danh sách file làm các lựa
    ↪  chọn
        print("Available prompt tactics:")
        for i, filename in enumerate(text_files, start=1):
            print(f"{i}. {filename}")

    while True:
        try:
            print_available()
            choice = int(input("Enter … 0 to exit): "))   # Nhập lựa chọn của
            ↪  người dùng
            if choice == 0:
                break
            elif 1 <= choice <= len(text_files):
                selected_file = text_files[choice - 1]
                file_path = os.path.join(directory,
      selected_file)
                prompts =
 load_and_parse_json_file(file_path)     # Nạp prompt và phân tích thành các
↪  message
                print(f"Running prompts for {selected_file}")
                for i, prompt in enumerate(prompts):
                    print(f"PROMPT {i+1} --------------------")
                    print(prompt)
                    print(f"REPLY --------------------------")
                    print(prompt_llm(prompt))      # Gửi prompt tới một LLM của
                    ↪  OpenAI
            else:
                print("Invalid choice. Please enter a valid number.")
        except ValueError:
            print("Invalid input. Please enter a number.")
```

Một đoạn code bị comment trong listing minh hoạ cách kết nối tới một LLM cục bộ. Nhờ đó bạn có thể khám phá cùng các chiến thuật prompt engineering này trên các LLM mã nguồn mở chạy cục bộ. Mặc định, ví dụ này dùng mô hình OpenAI mà ta đã cấu hình ở Mục 3.1.1. Nếu bạn chưa làm phần đó, hãy quay lại làm trước khi chạy ví dụ này.

Hình 3.10 cho thấy đầu ra khi chạy chương trình kiểm thử các chiến thuật prompt engineering, tức file `prompt_engineering.py`, trong VS Code. Khi chạy chương trình, bạn có thể nhập số tương ứng với chiến thuật muốn thử và xem nó chạy.

```
Available prompt tactics:
1. adopting_personas.jsonl
2. detailed_queries.jsonl
3. provide_examples.jsonl
4. specifying_steps.jsonl
5. specify_output_length.jsonl
6. using_delimiters.jsonl
Enter the number of the prompt tactic to run (or 0 to exit):
```

Hình 3.10: Đầu ra của chương trình kiểm thử các chiến thuật prompt engineering

Ở các mục tiếp theo, ta sẽ tìm hiểu chi tiết từng chiến thuật prompt, đồng thời xem xét các ví dụ đi kèm.

3.3.1 Tạo truy vấn chi tiết

Tiền đề cơ bản của chiến thuật này là cung cấp càng nhiều chi tiết càng tốt, nhưng cũng phải cẩn thận không đưa vào các chi tiết không liên quan. Listing sau trình bày các ví dụ trong file JSON Lines để khám phá chiến thuật này.

Listing 2.9 detailed_queries.jsonl

```
[                       // Ví dụ thứ nhất không dùng truy vấn chi tiết
    {
        "role": "system",
        "content": "You are a helpful assistant."
    },
    {
        "role": "user",
        "content": "What is an agent?"     // Trước hết hỏi LLM một câu rất chung
        ↪  chung
    }
]
[
    {
        "role": "system",
        "content": "You are a helpful assistant."
    },
    {
        "role": "user",
        "content": """
What is a GPT Agent?
Please give me 3 examples of a GPT agent
"""                     // Hỏi một câu cụ thể hơn, và yêu cầu đưa ví dụ
    }
]
```

Ví dụ này cho thấy sự khác biệt giữa việc dùng truy vấn chi tiết và không dùng. Nó còn tiến thêm một bước bằng cách yêu cầu đưa ra ví dụ. Hãy nhớ rằng bạn càng cung cấp được nhiều thông tin liên quan và ngữ cảnh trong prompt thì phản hồi tổng thể càng tốt. Yêu cầu đưa ví dụ là một cách khác để củng cố mối liên hệ giữa câu hỏi và đầu ra mong đợi.

3.3.2 Nhập vai persona

Việc nhập vai **persona** (nhân vật/hình mẫu) cho phép định nghĩa một ngữ cảnh bao trùm hoặc một bộ quy tắc cho LLM. Sau đó, LLM có thể dùng ngữ cảnh và/hoặc quy tắc đó để định khung cho mọi phản hồi về sau. Đây là một chiến thuật rất thuyết phục và ta sẽ dùng nhiều xuyên suốt cuốn sách.

Listing 2.10 là ví dụ dùng hai persona để trả lời cùng một câu hỏi. Đây có thể là một kỹ thuật thú vị để khám phá nhiều ứng dụng mới lạ, từ việc lấy phản hồi theo nhóm nhân khẩu học, chuyên môn hoá cho một nhiệm vụ cụ thể, cho tới cả rubber ducking.

GPT rubber ducking

Rubber ducking là một kỹ thuật giải quyết vấn đề, trong đó một người giải thích vấn đề cho một đồ vật vô tri, như một chú vịt cao su, để hiểu rõ hoặc tìm ra lời giải. Phương pháp này rất phổ biến trong lập trình và gỡ lỗi, vì việc diễn đạt vấn đề thành lời thường giúp làm rõ vấn đề và có thể dẫn tới những góc nhìn hay giải pháp mới.

GPT rubber ducking dùng cùng kỹ thuật này, nhưng thay vì một đồ vật vô tri, ta dùng một LLM. Chiến lược này còn có thể mở rộng thêm bằng cách gán cho LLM một persona đặc thù cho lĩnh vực của giải pháp mong muốn.

Listing 2.10 adopting_personas.jsonl

```
[
    {
        "role": "system",
        "content": """
You are a 20 year old female who attends college
in computer science. Answer all your replies as
a junior programmer.
"""                          // Persona thứ nhất
    },
    {
        "role": "user",
        "content": "What is the best subject to study."
    }
]
[
    {
        "role": "system",
        "content": """
You are a 38 year old male registered nurse.
Answer all replies as a medical professional.
"""                          // Persona thứ hai
    },
    {
        "role": "user",
        "content": "What is the best subject to study."
    }
]
```

Persona là một thành phần cốt lõi của hồ sơ agent (agent profile). Ta sẽ dùng nhiều persona khác nhau để hỗ trợ agent hoàn thành nhiệm vụ. Khi chạy chiến thuật này, hãy đặc biệt chú ý cách LLM đưa ra phản hồi.

3.3.3 Dùng dấu phân cách

Dấu phân cách (*delimiter*) là một cách hữu ích để tách riêng và khiến LLM tập trung vào một phần nào đó của message. Chiến thuật này thường được kết hợp với các chiến thuật khác nhưng cũng có thể hoạt động tốt một mình. Listing sau minh hoạ hai ví dụ, nhưng còn nhiều cách khác để mô tả dấu phân cách, từ thẻ XML cho tới dùng markdown.

Listing 2.11 using_delimiters.jsonl

```
[
    {
        "role": "system",
        "content": """
Summarize the text delimited by triple quotes
with a haiku.
"""      // Dấu phân cách được định nghĩa bằng loại ký tự và số lần lặp
    },
    {
        "role": "user",
        "content": "A gold chain is cool '''but a silver chain is better'''"
    }
]
[
```

```
    {
        "role": "system",
        "content": """
You will be provided with a pair of statements
(delimited with XML tags) about the same topic.
First summarize the arguments of each statement.
Then indicate which of them makes a better statement
 and explain why.
"""                          // Dấu phân cách được định nghĩa theo chuẩn XML
    },
    {
        "role": "user",
        "content": """
<statement>gold chains are cool</statement>
<statement>silver chains are better</statement>
"""
    }
]
```

Khi chạy chiến thuật này, hãy chú ý những phần văn bản mà LLM tập trung vào khi đưa ra phản hồi. Chiến thuật này có thể có lợi khi mô tả thông tin theo cấu trúc phân cấp hoặc các dạng quan hệ khác.

3.3.4 Chỉ định các bước

Chỉ định các bước là một chiến thuật mạnh khác với nhiều công dụng, kể cả trong agent, như trong Listing 2.12. Nó đặc biệt mạnh khi xây dựng prompt hoặc hồ sơ agent cho các nhiệm vụ phức tạp nhiều bước. Bạn có thể chỉ định các bước để chia nhỏ các prompt phức tạp này thành một quy trình từng bước mà LLM có thể làm theo. Đến lượt mình, các bước này có thể dẫn dắt LLM qua nhiều lượt tương tác trong một cuộc hội thoại dài hơn và nhiều vòng lặp.

Listing 2.12 specifying_steps.jsonl

```
[
    {
        "role": "system",
        "content": """
Use the following step-by-step instructions to respond to user inputs.
Step 1 - The user will provide you with text in triple single quotes.
Summarize this text in one sentence with a prefix that says 'Summary: '.
Step 2 - Translate the summary from Step 1 into Spanish,
with a prefix that says 'Translation: '.
"""                 // Để ý chiến thuật dùng dấu phân cách
    },
    {
        "role": "user",
        "content": "'''I am hungry and would like to order an appetizer.'''"
    }
]
[
    {
        "role": "system",
        "content": """
Use the following step-by-step instructions to respond to user inputs.
Step 1 - The user will provide you with text. Answer any questions in
the text in one sentence with a prefix that says 'Answer: '.
Step 2 - Translate the Answer from Step 1 into a dad joke,
 with a prefix that says 'Dad Joke: '."""  // Các bước có thể là thao tác hoàn
 ↪  toàn khác nhau
```

```
    },
    {
        "role": "user",
        "content": "What is the tallest structure in Paris?"
    }
]
```

3.3.5 Cung cấp ví dụ

Cung cấp ví dụ là một cách tuyệt vời để định hướng đầu ra mong muốn của LLM. Có rất nhiều cách để trình bày ví dụ cho LLM. Message/prompt system có thể là một cách hữu ích để nhấn mạnh dạng đầu ra chung. Trong listing sau, ví dụ được thêm vào như câu trả lời gần nhất của LLM assistant, với prompt "Teach me about Python."

Listing 2.13 providing_examples.jsonl

```
[
    {
        "role": "system",
        "content": """
Answer all replies in a consistent style that follows the format,
length and style of your previous responses.
Example:
  user:
       Teach me about Python.
  assistant:
       Python is a programming language developed in 1989
 by Guido van Rossum.

  Future replies:
       The response was only a sentence so limit
 all future replies to a single sentence.
"""
    },
    {
        "role": "user",
        "content": "Teach me about Java."
    }
]
```

Chú thích cho Listing 2.13 (nằm bên trong chuỗi prompt nên không chèn comment được):

- Dòng `assistant:` và câu trả lời bên dưới: chèn đầu ra mẫu như thể đó là câu trả lời "trước đó" của assistant.
- Đoạn `Future replies:`: thêm chiến thuật giới hạn đầu ra để hạn chế độ dài đầu ra và khớp với ví dụ.

Cung cấp ví dụ cũng có thể dùng để yêu cầu một định dạng đầu ra cụ thể từ một chuỗi nhiệm vụ phức tạp tạo ra đầu ra đó. Chẳng hạn, yêu cầu LLM viết code tạo ra đầu ra khớp với một mẫu cho trước là một cách dùng ví dụ rất tốt. Ta sẽ dùng chiến thuật này xuyên suốt cuốn sách, nhưng cũng có các phương pháp khác để định hướng đầu ra.

3.3.6 Chỉ định độ dài đầu ra

Chiến thuật chỉ định độ dài đầu ra không chỉ giúp hạn chế số token mà còn giúp định hướng đầu ra theo định dạng mong muốn. Listing 2.14 là ví dụ dùng hai kỹ thuật khác nhau cho chiến thuật này. Kỹ thuật

thứ nhất giới hạn đầu ra dưới 10 từ. Điều này còn có lợi ích phụ là khiến phản hồi ngắn gọn và đúng trọng tâm hơn, điều có thể đáng mong muốn trong một số trường hợp sử dụng. Ví dụ thứ hai minh hoạ việc giới hạn đầu ra thành một tập gạch đầu dòng ngắn gọn. Cách này giúp thu hẹp đầu ra và giữ câu trả lời ngắn. Câu trả lời ngắn gọn hơn nhìn chung có nghĩa là đầu ra tập trung hơn và ít nội dung độn hơn.

Listing 2.14 specifying_output_length.jsonl

```
[
    {
        "role": "system",
        "content": """
Summarize all replies into 10 or fewer words.
"""                 // Giới hạn đầu ra khiến câu trả lời ngắn gọn hơn
    },
    {
        "role": "user",
        "content": "Please tell me an exciting fact about Paris?"
    }
]
[
    {
        "role": "system",
        "content": """
Summarize all replies into 3 bullet points.
"""                 // Giới hạn câu trả lời thành vài gạch đầu dòng ngắn
    },
    {
        "role": "user",
        "content": "Please tell me an exciting fact about Paris?"
    }
]
```

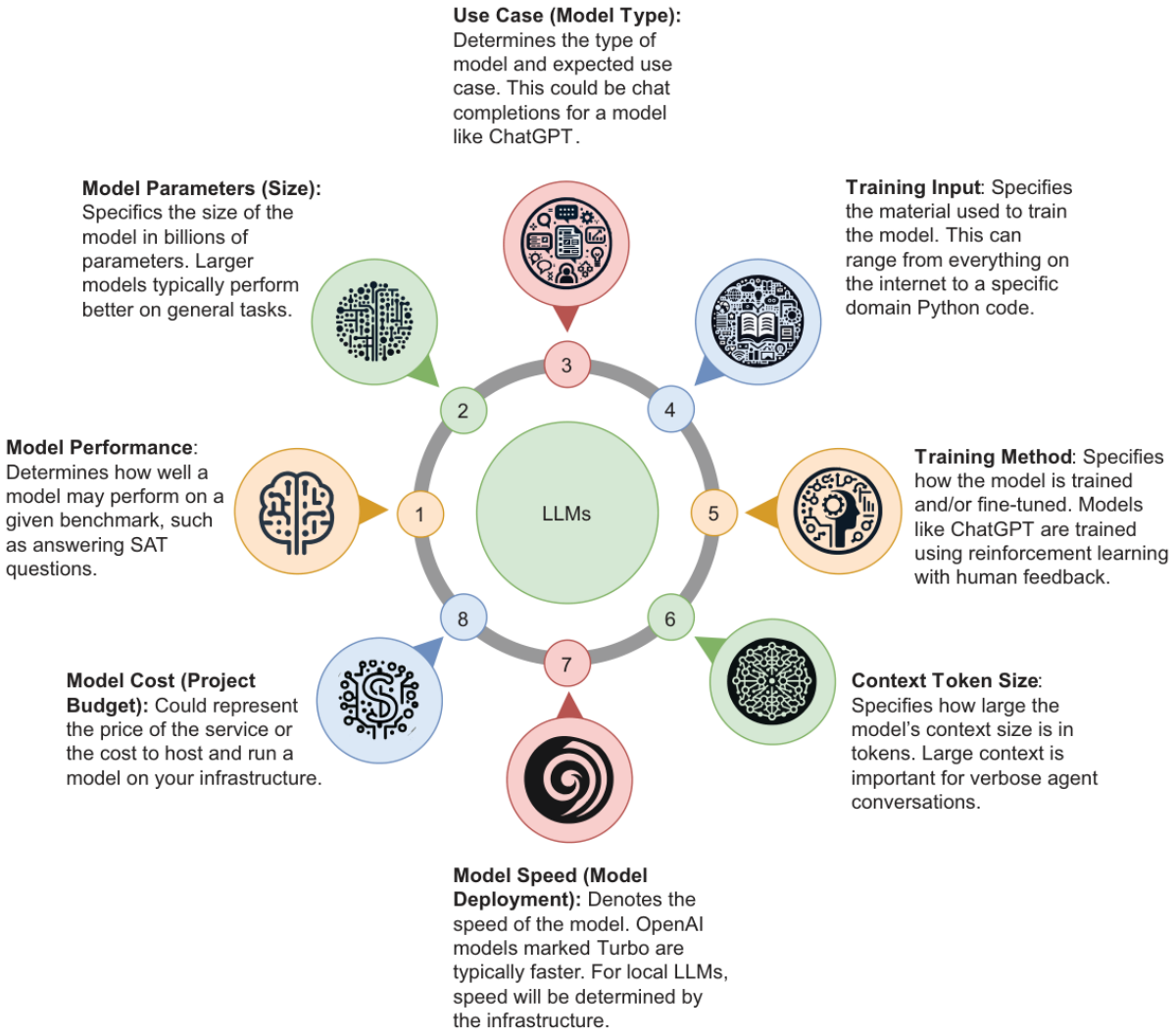
Giữ câu trả lời ngắn gọn còn có thêm lợi ích khi phát triển hệ thống đa agent (multi-agent). Bất kỳ hệ thống agent nào trò chuyện với các agent khác đều có lợi từ các câu trả lời ngắn gọn và tập trung hơn. Điều này giúp LLM tập trung hơn và giảm giao tiếp nhiễu.

Hãy chạy thử tất cả các ví dụ chiến thuật prompt của chiến lược này. Như đã nói, ta sẽ đề cập các chiến lược và chiến thuật prompt engineering khác ở các chương sau. Ta sẽ kết thúc chương này bằng việc xem cách chọn LLM tốt nhất cho trường hợp sử dụng của bạn.

3.4 Chọn LLM tối ưu cho nhu cầu cụ thể của bạn

Tuy để trở thành người xây dựng AI agent thành công bạn không cần hiểu sâu về LLM, việc có khả năng đánh giá các thông số của chúng vẫn rất hữu ích. Giống như một người dùng máy tính, bạn không cần biết cách chế tạo bộ vi xử lý để hiểu sự khác biệt giữa các dòng vi xử lý. Phép so sánh này khá đúng với LLM: dù tiêu chí có thể khác, việc lựa chọn vẫn phụ thuộc vào một số cân nhắc chính.

Từ phần thảo luận trước và khi xem xét LM Studio, ta có thể rút ra một số tiêu chí cơ bản quan trọng khi cân nhắc LLM. Hình 3.11 giải thích các tiêu chí thiết yếu xác định một LLM có đáng cân nhắc để tạo GPT agent hay cho bất kỳ nhiệm vụ LLM nào hay không.



Hình 3.11: Các tiêu chí quan trọng cần cân nhắc khi sử dụng một LLM

Với mục đích xây dựng AI agent, ta cần xem xét từng tiêu chí này theo góc độ liên quan tới nhiệm vụ. Kích thước context và tốc độ của mô hình có thể coi là tiêu chí thứ sáu và thứ bảy, nhưng chúng thường được xem là các biến thể của kiến trúc triển khai và hạ tầng của mô hình. Tiêu chí thứ tám cần cân nhắc là chi phí, nhưng điều này phụ thuộc vào nhiều yếu tố khác. Dưới đây là tóm tắt cách các tiêu chí này liên quan tới việc xây dựng AI agent:

- *Hiệu năng mô hình* (*model performance*): nhìn chung bạn sẽ muốn hiểu hiệu năng của LLM trên một tập nhiệm vụ nhất định. Ví dụ, nếu bạn xây dựng một agent chuyên về lập trình, một LLM làm tốt với code là điều thiết yếu.

- *Số tham số (kích thước) mô hình* (*model parameters*): kích thước mô hình thường là chỉ báo rất tốt về hiệu năng suy luận và mức độ phản hồi tốt của mô hình. Tuy nhiên, kích thước mô hình cũng quyết định yêu cầu phần cứng. Nếu định dùng mô hình tự host cục bộ, kích thước mô hình cũng là yếu tố chính quyết định máy tính và GPU bạn cần. May mắn là ta đang thấy các mô hình mã nguồn mở nhỏ nhưng rất có năng lực được phát hành đều đặn.

- *Trường hợp sử dụng (loại mô hình)* (*use case, model type*): loại mô hình có nhiều biến thể. Mô hình chat completions như ChatGPT hiệu quả trong việc lặp và suy luận qua một vấn đề, trong khi các mô hình completion, hỏi/đáp (question/answer) và instruct gắn với các nhiệm vụ cụ thể hơn. Mô hình chat completions là thiết yếu cho các ứng dụng agent, nhất là các agent có lặp.

- *Dữ liệu huấn luyện* (*training input*): hiểu nội dung dùng để huấn luyện mô hình thường cho biết lĩnh vực của mô hình. Mô hình tổng quát có thể hiệu quả trên nhiều nhiệm vụ, nhưng các mô hình chuyên biệt hoặc đã fine-tune có thể phù hợp hơn với một lĩnh vực. Đây có thể là điều cần cân nhắc với agent theo lĩnh vực, khi một mô hình nhỏ hơn và được fine-tune kỹ hơn có thể hoạt động ngang bằng hoặc tốt hơn một mô hình lớn như GPT-4.

- *Phương pháp huấn luyện* (*training method*): có lẽ ít đáng lo hơn, nhưng hiểu phương pháp đã dùng để huấn luyện mô hình vẫn có ích. Cách huấn luyện mô hình có thể ảnh hưởng tới khả năng khái quát hoá, suy luận và lập kế hoạch của nó. Điều này có thể thiết yếu với agent lập kế hoạch, nhưng có lẽ ít quan trọng với agent hơn so với một trợ lý chuyên cho một nhiệm vụ cụ thể.

- *Kích thước token ngữ cảnh* (*context token size*): kích thước context của mô hình gắn với kiến trúc và loại mô hình. Nó quyết định lượng ngữ cảnh hay bộ nhớ mà mô hình có thể giữ. Cửa sổ context nhỏ hơn 4.000 token thường là quá đủ cho các nhiệm vụ đơn giản. Tuy nhiên, cửa sổ context lớn có thể là thiết yếu khi dùng nhiều agent cùng trò chuyện về một nhiệm vụ. Các mô hình thường được triển khai với nhiều biến thể về kích thước cửa sổ context.

- *Tốc độ mô hình (triển khai mô hình)* (*model speed, model deployment*): tốc độ của mô hình được quyết định bởi **tốc độ suy luận** (*inference speed*, tức mô hình trả lời một yêu cầu nhanh đến đâu), và đến lượt nó lại phụ thuộc vào hạ tầng mà mô hình chạy trên đó. Nếu agent của bạn không tương tác trực tiếp với người dùng, tốc độ thời gian thực có thể không cần thiết. Ngược lại, một LLM agent tương tác thời gian thực cần nhanh nhất có thể. Với mô hình thương mại, tốc độ do nhà cung cấp quyết định và hỗ trợ. Với những ai muốn tự chạy LLM, hạ tầng của bạn sẽ quyết định tốc độ.

- *Chi phí mô hình (ngân sách dự án)* (*model cost, project budget*): chi phí thường do dự án quyết định. Dù là học cách xây dựng agent hay triển khai phần mềm doanh nghiệp, chi phí luôn là điều cần cân nhắc. Có một sự đánh đổi đáng kể giữa việc tự chạy LLM và dùng API thương mại.

Có rất nhiều điều cần cân nhắc khi chọn mô hình để xây dựng một hệ thống agent cho môi trường production. Tuy nhiên, chọn và làm việc với một mô hình duy nhất thường là tốt nhất cho mục đích nghiên cứu và học tập. Nếu bạn mới làm quen với LLM và agent, có lẽ bạn nên chọn một lựa chọn thương mại như GPT-4 Turbo. Trừ khi có ghi chú khác, các phần thực hành trong sách này đều dựa trên GPT-4 Turbo.

Theo thời gian, các mô hình chắc chắn sẽ bị thay thế bởi các mô hình tốt hơn. Vì vậy, bạn có thể cần nâng cấp hoặc hoán đổi mô hình. Tuy nhiên, để làm vậy, bạn phải hiểu các chỉ số hiệu năng của LLM và agent của mình. May mắn là ở Chương 10, ta sẽ tìm hiểu việc đánh giá LLM, prompt và hồ sơ agent bằng prompt flow.

3.5 Bài tập

Hãy dùng các bài tập sau để thực hành nội dung của chương này.

Bài tập 1: Sử dụng các LLM khác nhau

Mục tiêu: dùng ví dụ code `connecting.py` để sử dụng một LLM khác từ OpenAI hoặc nhà cung cấp khác.

Nhiệm vụ:

- Sửa `connecting.py` để kết nối tới một LLM khác.

- Chọn một LLM từ OpenAI hoặc nhà cung cấp khác.

- Cập nhật API key và endpoint trong code.
- Chạy code đã sửa và kiểm tra phản hồi.

Bài tập 2: Khám phá các chiến thuật prompt engineering

Mục tiêu: khám phá các chiến thuật prompt engineering khác nhau và tạo các biến thể cho từng chiến thuật.

Nhiệm vụ:

- Xem lại các chiến thuật prompt engineering được trình bày trong chương.
- Viết các biến thể cho từng chiến thuật, thử nghiệm với các cách diễn đạt và cấu trúc khác nhau.
- Kiểm thử các biến thể với một LLM để quan sát các kết quả khác nhau.
- Ghi lại kết quả và phân tích hiệu quả của từng biến thể.

Bài tập 3: Tải và chạy một LLM với LM Studio

Mục tiêu: tải một LLM bằng LM Studio và kết nối nó với các ví dụ chiến thuật prompt engineering.

Nhiệm vụ:

- Cài LM Studio trên máy của bạn.
- Tải một LLM bằng LM Studio.
- Phục vụ (serve) mô hình bằng LM Studio.
- Viết code Python để kết nối tới mô hình đang được phục vụ.
- Tích hợp ví dụ các chiến thuật prompt engineering với mô hình đang được phục vụ.

Bài tập 4: So sánh LLM thương mại và LLM mã nguồn mở

Mục tiêu: so sánh hiệu năng của một LLM thương mại như GPT-4 Turbo với một mô hình mã nguồn mở bằng các ví dụ prompt engineering.

Nhiệm vụ:

- Thực hiện các ví dụ prompt engineering với GPT-4 Turbo.
- Lặp lại với một LLM mã nguồn mở.
- Đánh giá các mô hình theo các tiêu chí như độ chính xác của phản hồi, tính mạch lạc và tốc độ.
- Ghi lại quá trình đánh giá và tóm tắt các phát hiện.

Bài tập 5: Các phương án host LLM

Mục tiêu: đối chiếu và so sánh các phương án tự host một LLM với việc dùng mô hình thương mại.

Nhiệm vụ:

- Tìm hiểu các phương án host LLM khác nhau (ví dụ server cục bộ, dịch vụ đám mây).
- Đánh giá ưu và nhược điểm của từng phương án.
- So sánh các phương án này với việc dùng mô hình thương mại về chi phí, hiệu năng và độ dễ sử dụng.
- Viết báo cáo tóm tắt kết quả so sánh và đề xuất cách tiếp cận tốt nhất cho từng trường hợp sử dụng cụ thể.

Tóm tắt

- LLM dùng một loại kiến trúc gọi là generative pretrained transformer (GPT).
- Mô hình sinh (ví dụ LLM và GPT) khác với mô hình dự đoán/phân loại ở chỗ chúng học cách biểu diễn dữ liệu chứ không chỉ phân loại dữ liệu.
- LLM là tập hợp của dữ liệu, kiến trúc và quá trình huấn luyện cho các trường hợp sử dụng cụ thể, gọi là *fine-tuning*.
- OpenAI API SDK có thể dùng để kết nối tới LLM như GPT-4, và cũng dùng được để sử dụng các LLM mã nguồn mở.
- Bạn có thể nhanh chóng thiết lập môi trường Python và cài các package cần thiết để tích hợp LLM.
- LLM có thể xử lý nhiều loại yêu cầu và tạo ra các phản hồi riêng, giúp nâng cao kỹ năng lập trình liên quan đến tích hợp LLM.
- LLM mã nguồn mở là một lựa chọn thay thế cho mô hình thương mại và có thể host cục bộ bằng các công cụ như LM Studio.
- Prompt engineering là tập hợp các kỹ thuật giúp viết prompt hiệu quả hơn để cải thiện phản hồi của LLM.
- LLM có thể dùng để vận hành agent và trợ lý, từ chatbot đơn giản tới những “nhân viên” tự chủ đầy đủ năng lực.
- Việc chọn LLM phù hợp nhất cho nhu cầu cụ thể phụ thuộc vào hiệu năng, số tham số, trường hợp sử dụng, dữ liệu huấn luyện và các tiêu chí khác.
- Chạy LLM cục bộ đòi hỏi nhiều kỹ năng, từ thiết lập GPU tới hiểu các tuỳ chọn cấu hình khác nhau.

Chương 4

Làm việc với GPT Assistants

Chương này bao quát các nội dung:

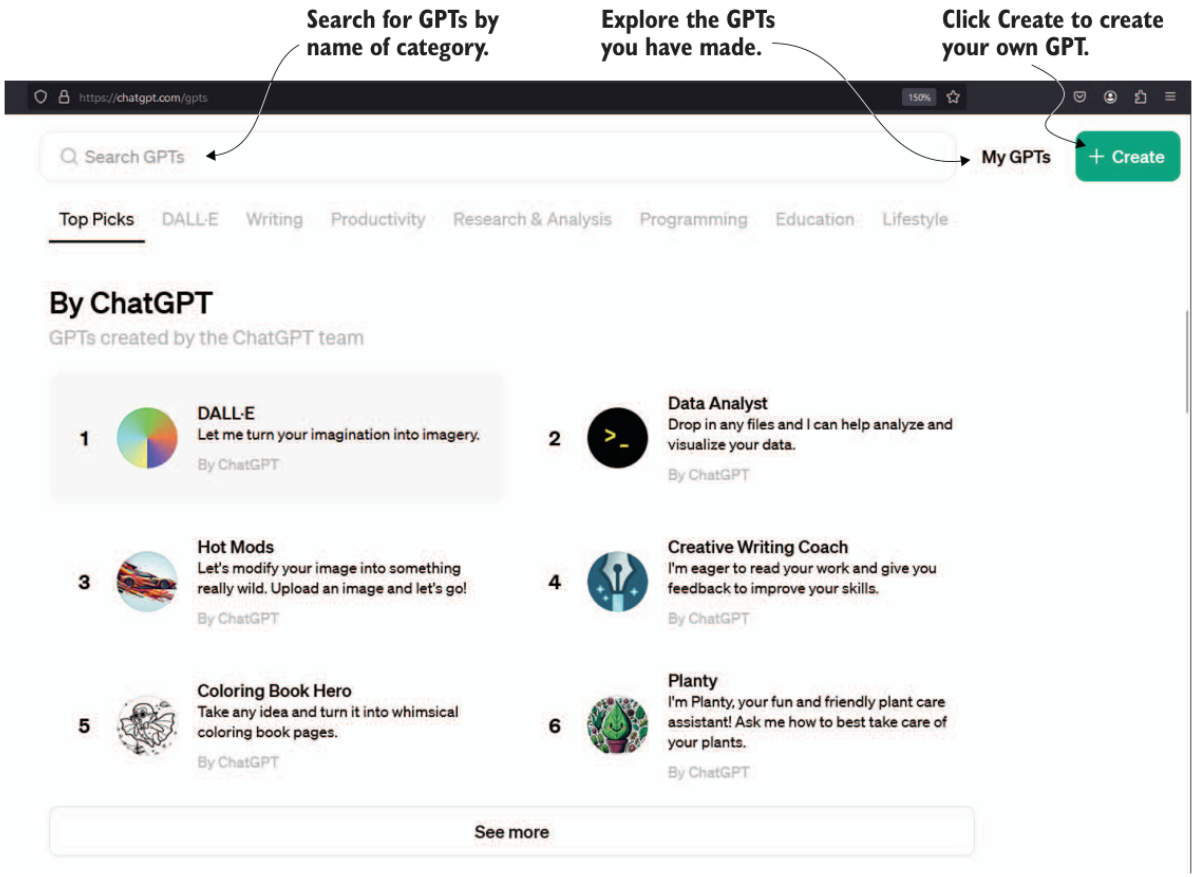
- Giới thiệu nền tảng OpenAI GPT Assistants và giao diện ChatGPT.
- Xây dựng một GPT biết dùng khả năng thông dịch code (code interpretation).
- Mở rộng một assistant bằng custom action (hành động tuỳ biến).
- Bổ sung kiến thức cho GPT qua việc tải file lên (file upload).
- Thương mại hoá GPT của bạn và phát hành nó lên GPT Store.

Khi tìm hiểu bước tiến của OpenAI vào lĩnh vực trợ lý (assistant), và điều mà OpenAI đã ngầm gợi ý, rốt cuộc là một nền tảng agent mang tên **GPT Assistants**, ta sẽ làm quen với GPT assistant thông qua giao diện ChatGPT. Sau đó, ta bổ sung một số assistant hoàn chỉnh: một assistant gợi ý công thức nấu ăn từ nguyên liệu có sẵn, một assistant phân tích dữ liệu đầy đủ như một nhà khoa học dữ liệu, một assistant hướng dẫn người đọc đọc sách, và một assistant được mở rộng bằng custom action. Cuối chương, ta sẽ sẵn sàng xây dựng một agent hoạt động đầy đủ, có thể phát hành lên OpenAI GPT Store.

4.1 Khám phá GPT assistant qua ChatGPT

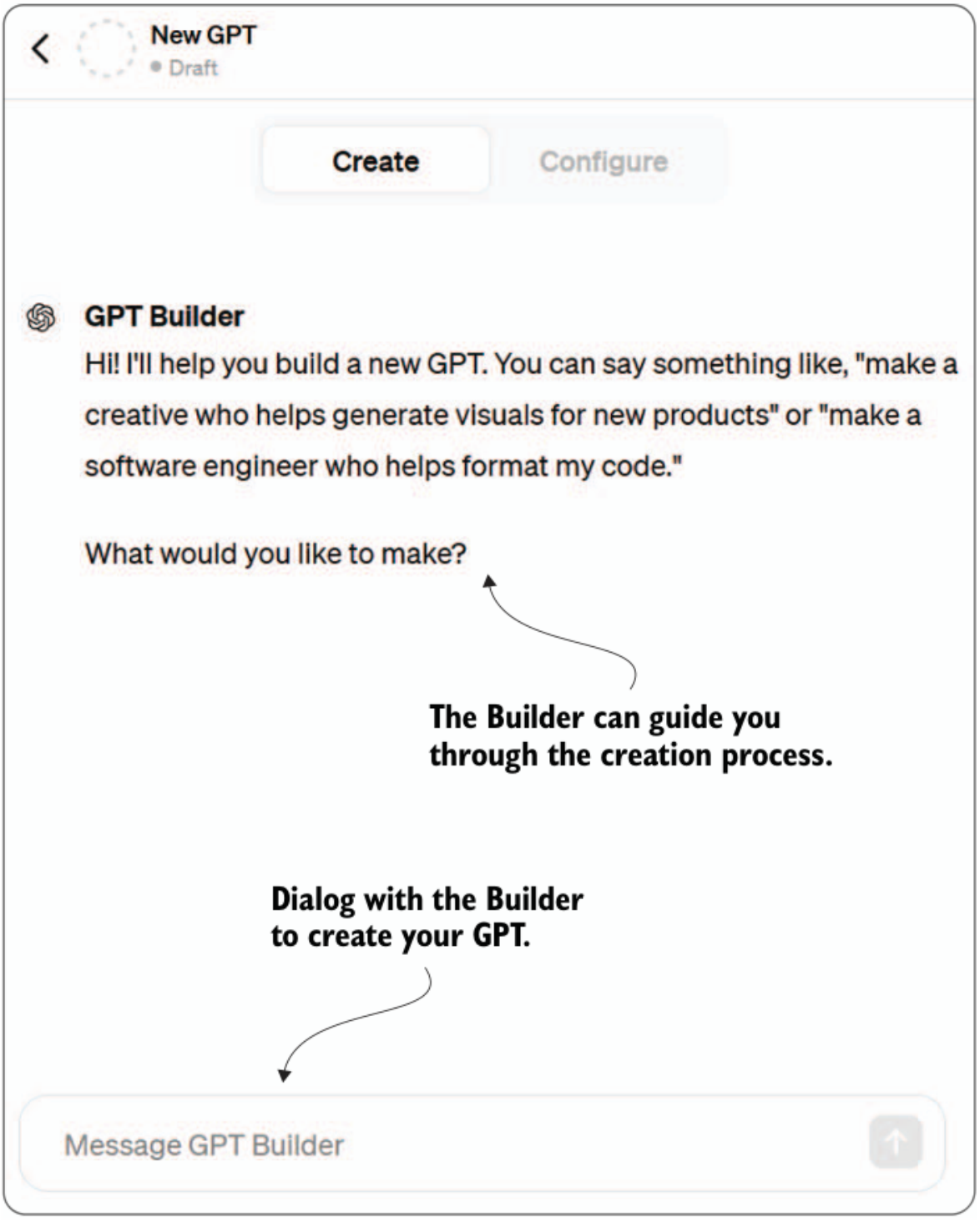
ChatGPT (cụ thể là ChatGPT Plus, tại thời điểm viết sách) cho phép bạn xây dựng GPT assistant, sử dụng assistant của người khác, và thậm chí phát hành chúng, như bạn sẽ thấy ở cuối chương. Khi OpenAI công bố nền tảng GPT Assistants, sự kiện này đã góp phần định hình và củng cố sự xuất hiện của AI agent. Vì vậy, bất kỳ ai quan tâm tới việc xây dựng và sử dụng hệ thống agent đều nên xem xét nền tảng này một cách nghiêm túc. Trước hết, ta xem cách xây dựng GPT assistant qua ChatGPT Plus, vốn đòi hỏi gói đăng ký trả phí. Nếu không muốn mua gói đăng ký, bạn có thể đọc chương này như phần nhập môn; Chương 6 sẽ trình bày cách dùng dịch vụ API ở phần sau.

Hình 4.1 cho thấy trang GPT Store bên trong ChatGPT (`https://chatgpt.com/gpts`). Tại đây, bạn có thể tìm kiếm và khám phá đủ loại GPT cho hầu như mọi nhiệm vụ. Mức độ sử dụng thường cho biết mỗi GPT hoạt động tốt đến đâu, nên hãy dựa vào đó để đánh giá GPT nào hợp với bạn nhất.



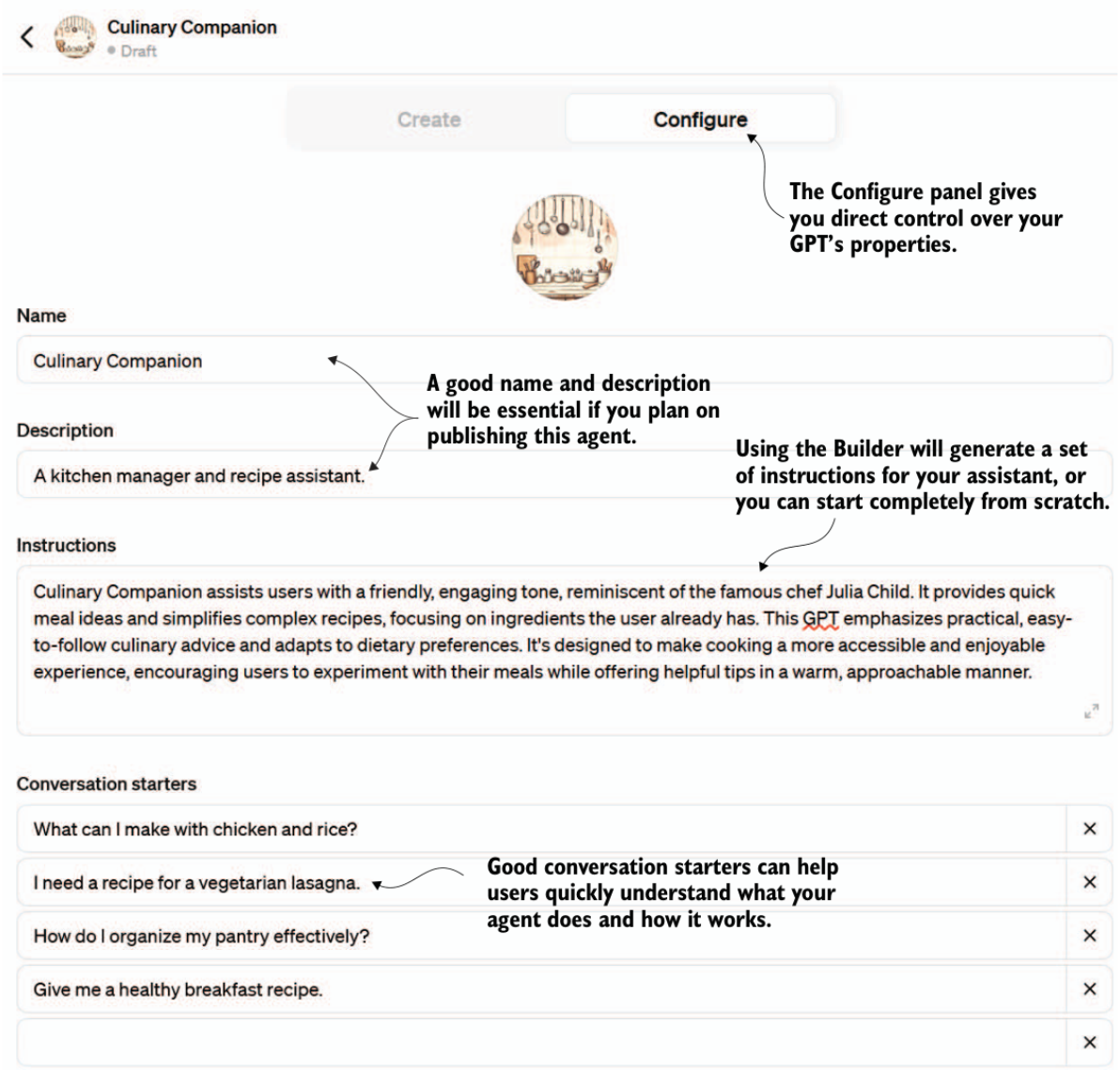
Hình 4.1: Giao diện chính của GPT Store

Tạo GPT Assistant đầu tiên rất đơn giản: chỉ cần nhấn nút Create và làm theo giao diện chat của **GPT Builder**. Hình 4.2 cho thấy việc dùng Builder để tạo một GPT. Làm bài thực hành này vài lần là cách rất tốt để bắt đầu hiểu một assistant cần những gì.



Hình 4.2: Tương tác với GPT Builder để tạo một assistant

Sau khi làm việc với Builder, bạn có thể mở bảng cấu hình thủ công (Configure), như trong Hình 4.3, và chỉnh sửa GPT trực tiếp. Bạn sẽ thấy tên, mô tả, chỉ dẫn (instructions) và các câu mở đầu hội thoại (conversation starters) đã được điền sẵn từ cuộc trò chuyện của bạn với Builder. Đây là điểm khởi đầu tốt, nhưng nhìn chung bạn sẽ muốn tự tay sửa và tinh chỉnh các thuộc tính này.



Hình 4.3: Bảng Configure trong giao diện nền tảng GPT Assistants

Nếu muốn làm theo để xây dựng Culinary Companion (người bạn đồng hành trong bếp) của riêng mình, hãy nhập văn bản trong Listing 3.1 vào ô instructions. Các chỉ dẫn này một phần được sinh ra qua trò chuyện với Builder, một phần được bổ sung dựa trên các đầu ra cụ thể mong muốn. Các đầu ra cụ thể đó được thêm vào chỉ dẫn dưới dạng quy tắc (rules).

Listing 3.1 Chỉ dẫn cho Culinary Companion

```
Culinary Companion assists users with a friendly, engaging tone,
reminiscent of the famous chef Julia Child.
It provides quick meal ideas and simplifies complex recipes, focusing on
ingredients the user already has. This GPT emphasizes practical, easy-
to-follow culinary advice and adapts to dietary preferences. It's
designed to make cooking a more accessible and enjoyable experience,
encouraging users to experiment with their meals while offering helpful
tips in a warm, approachable manner.

RULES:
When generating a recipe, always create an image of the final prepared
```

```
recipe.
When generating a recipe, estimate the calories and nutritional values
per serving.
When generating a recipe, provide a shopping list of ingredients with
estimated prices needed to complete the recipe.
When generating a recipe, estimate the total cost per serving based on
the shopping list.
```

Chú thích cho Listing 3.1:

- Hai dòng đầu ("… reminiscent of the famous chef Julia Child"): tính cách, hay persona, của assistant.
- Phần còn lại của đoạn đầu (tới "… warm, approachable manner"): các hướng dẫn chung về vai trò và mục tiêu của agent.
- Khối `RULES`: tập quy tắc mà agent sẽ tuân theo khi gợi ý một công thức.

Việc định nghĩa quy tắc cho một assistant/agent về bản chất là tạo ra một khuôn mẫu cho những gì agent sẽ sinh ra. Thêm quy tắc giúp đảm bảo đầu ra của GPT nhất quán và khớp với kỳ vọng của bạn về cách agent hoạt động. Định nghĩa và gán cho agent/assistant một persona sẽ mang lại cho nó một tính cách độc đáo, dễ nhớ.

Ghi chú

Gán cho agent/assistant một tính cách cụ thể có thể tạo khác biệt về loại và hình thức đầu ra. Yêu cầu một agent nấu ăn nói chuyện như Julia Child, đầu bếp nổi tiếng đầu tiên trên truyền hình, không chỉ đem lại giọng điệu vui vẻ mà còn khơi gợi thêm nhiều tham chiếu có nhắc tới hoặc bàn về phong cách nấu ăn và cách giảng dạy của bà. Khi xây dựng một agent/assistant, việc gán một persona/tính cách cụ thể có thể rất hữu ích.

Chỉ với vài bước như vậy, ta đã có một người bạn đồng hành trong bếp không chỉ đưa ra công thức cho các nguyên liệu có sẵn, mà còn sinh hình ảnh món ăn hoàn chỉnh, ước tính giá trị dinh dưỡng, lập danh sách mua sắm kèm ước tính giá, và tính chi phí cho mỗi khẩu phần.

Hãy thử assistant bằng cách yêu cầu một công thức và đưa ra danh sách các nguyên liệu bạn có hoặc ưa thích. Listing 3.2 là ví dụ một yêu cầu đơn giản, kèm thông tin bổ sung để tạo không khí. Dĩ nhiên, bạn có thể thêm bất kỳ nguyên liệu hay tình huống nào rồi xem kết quả.

Listing 3.2 Prompt yêu cầu công thức

```
I have a bag of prepared frozen chicken strips and I want to make a
romantic dinner for two.
```

Hình 4.4 cho thấy kết quả đã định dạng mà GPT tạo ra cho prompt này. Trông nó chắc chắn đủ hấp dẫn để ăn được. Toàn bộ đầu ra này được sinh ra nhờ các chỉ dẫn ta đã cung cấp cho agent.

Tuy kết quả trông rất tốt, không phải mọi thông tin đều đúng sự thật và chính xác, và kết quả của bạn có thể khác. Chẳng hạn, GPT đã thêm chicken strips vào danh sách mua sắm dù ta đã nói là có sẵn nguyên liệu này. Hơn nữa, giá cả và thông tin dinh dưỡng chỉ là ước tính, nhưng nếu quan tâm, bạn có thể giải quyết vấn đề này sau.

Hình 4.4: Kết quả đầu ra của GPT Culinary Companion

Dù vậy, ngay khi dùng mặc định, GPT Assistants đã rất ấn tượng trong việc nhanh chóng dựng một assistant hay agent ở mức proof-of-concept (thử nghiệm khái niệm). Như bạn sẽ thấy ở phần sau của chương, nó còn là nền tảng tuyệt vời để sử dụng assistant bên ngoài ChatGPT. Ở mục tiếp theo, ta xem những tính năng ấn tượng hơn mà GPT cung cấp, như tải file lên và thông dịch code.

4.2 Xây dựng GPT làm khoa học dữ liệu

Nền tảng GPT Assistants đã và nhiều khả năng sẽ còn được mở rộng để bao gồm nhiều thành phần agent khác nhau. Hiện tại, GPT Assistants hỗ trợ những thứ gọi là **knowledge** (kiến thức), **memory** (bộ nhớ) và **actions** (hành động). Chương 8 sẽ bàn chi tiết về knowledge và memory, còn Chương 5 trình bày khái niệm sử dụng công cụ (tool use) thông qua actions.

Trong bài thực hành tiếp theo, ta sẽ xây dựng một assistant thực hiện bước rà soát khoa học dữ liệu sơ bộ cho bất kỳ tài liệu CSV nào ta cung cấp. Agent này dùng khả năng (hay action) cho phép viết code và thông dịch code. Khi bạn bật code interpretation, assistant sẽ mặc định cho phép tải file lên.

Tuy nhiên, trước khi làm điều đó, ta muốn thiết kế agent của mình, và còn cách nào tốt hơn việc nhờ một LLM xây dựng assistant cho ta? Listing 3.3 là các prompt yêu cầu ChatGPT (GPT-4) thiết kế một assistant khoa học dữ liệu. Lưu ý rằng ta không hỏi mọi thứ trong một prompt duy nhất, mà lặp dần dựa trên thông tin LLM trả về.

Listing 3.3 Prompt thiết kế assistant khoa học dữ liệu

```
FIRST PROMPT:
what is a good basic and interesting data science
experiment you can task someone with a single
csv file that contains interesting data?

SECOND PROMPT:
okay, can you now write all those steps into instructions
to be used for a GPT Agent (LLM agent) to replicate all of
the above steps

THIRD PROMPT:
What is a famous personality that can embody the agent
data scientist and be able to present data to users?
```

Chú thích cho Listing 3.3:

- `FIRST PROMPT`: trước tiên, yêu cầu LLM đặt nền móng.
- `SECOND PROMPT`: sau đó, yêu cầu LLM chuyển các bước trước thành một quy trình chính thức hơn.
- `THIRD PROMPT`: cuối cùng, yêu cầu LLM đề xuất một tính cách có thể đại diện cho quy trình.

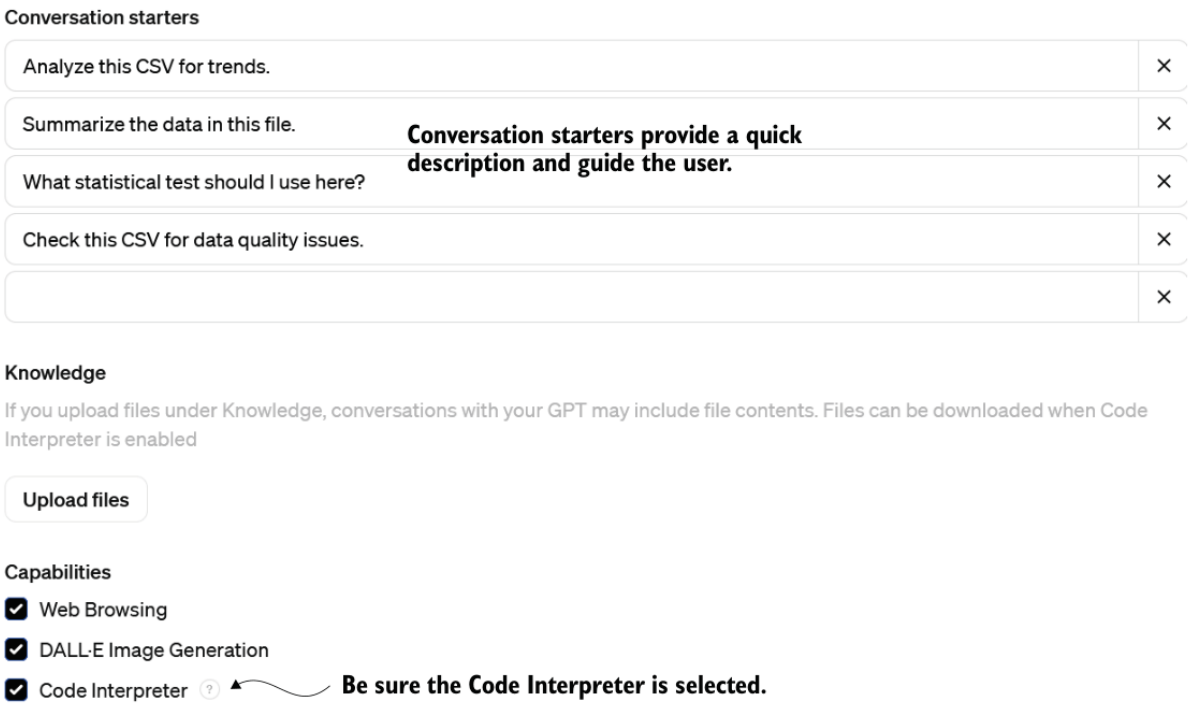
Kết quả của cuộc hội thoại đó cho ra các chỉ dẫn assistant trong Listing 3.4. Ở đây, assistant được đặt tên là Data Scout, nhưng bạn cứ thoải mái đặt tên mình thích.

Listing 3.4 Chỉ dẫn cho Data Scout

```
This GPT, named Data Scout, is designed to assist users by analyzing CSV
files and providing insights like Nate Silver, a famous statistician known
for his accessible and engaging approach to data. Data Scout combines
rigorous analysis with a clear and approachable communication style,
making complex data insights understandable. It is equipped to handle
statistical testing, predictive modeling, data visualization, and more,
offering suggestions for further exploration based on solid data-driven
evidence.

Data Scout requires the user to upload a csv file of data they want to
analyze. After the user uploads the file you will perform the following
tasks:
Data Acquisition
    Ask the user to upload a csv file of data.
    Instructions: Use the pandas library to read the data from the CSV
file. Ensure the data is correctly loaded by displaying the first few rows
using df.head().

2. Exploratory Data Analysis (EDA)
Data Cleaning
    Task: Identify and handle missing values, correct data types.
    Instructions: Check for missing values using df.isnull().sum(). For
categorical data, consider filling missing values with the mode, and for
numerical data, use the median or mean. Convert data types if necessary
using df.astype().

Visualization
    Task: Create visualizations to explore the data.
    Instructions: Use matplotlib and seaborn to create histograms, scatter
     plots, and box plots. For example, use sns.histplot() for histograms and
sns.scatterplot() for scatter plots.

Descriptive Statistics
    Task: Calculate basic statistical measures.
    Instructions: Use df.describe() to get a summary of the statistics and
df.mean(), df.median() for specific calculations.

3. Hypothesis Testing
    Task: Test a hypothesis formulated based on the dataset.
    Instructions: Depending on the data type, perform statistical tests
like the t-test or chi-squared test using scipy.stats. For example, use
stats.ttest_ind() for the t-test between two groups.

4. Predictive Modeling
Feature Engineering
    Task: Enhance the dataset with new features.
```

```
    Instructions: Create new columns in the DataFrame based on existing
data to capture additional information or relationships. Use operations
like df['new_feature'] = df['feature1'] / df['feature2'].

Model Selection
    Task: Choose and configure a machine learning model.
    Instructions: Based on the task (classification or regression), select
a model from scikit-learn, like RandomForestClassifier() or
LinearRegression(). Configure the model parameters.

Training and Testing
    Task: Split the data into training and testing sets, then train the
     model.
    Instructions: Use train_test_split from scikit-learn to divide the
data. Train the model using model.fit(X_train, y_train).

Model Evaluation
    Task: Assess the model performance.
    Instructions: Use metrics like mean squared error (MSE) or accuracy.
Calculate these using metrics.mean_squared_error(y_test, y_pred) or
metrics.accuracy_score(y_test, y_pred).

5. Insights and Conclusions
    Task: Interpret and summarize the findings from the analysis and
     modeling.
    Instructions: Discuss the model coefficients or feature importances.
Draw conclusions about the hypothesis and the predictive analysis. Suggest
real-world implications or actions based on the results.

6. Presentation
    Task: Prepare a report or presentation.
    Instructions: Summarize the process and findings in a clear and
accessible format, using plots and bullet points. Ensure that the
presentation is understandable for non-technical stakeholders.
```

Sau khi sinh các chỉ dẫn, bạn có thể sao chép và dán chúng vào bảng Configure như trong Hình 4.5. Nhớ cấp cho assistant công cụ (kỹ năng) Code Interpretation bằng cách đánh dấu ô tương ứng. Bạn không cần tải file lên ở đây; assistant sẽ cho phép tải file khi ô Code Interpretation được bật.



Hình 4.5: Bật công cụ/kỹ năng Code Interpreter

Giờ ta có thể thử assistant bằng cách tải một file CSV lên và đặt câu hỏi về nó. Thư mục mã nguồn của chương này có file `netflix_titles.csv`; vài dòng đầu được tóm lược trong Listing 3.5. Dĩ nhiên bạn có thể dùng bất kỳ file CSV nào, nhưng bài thực hành này dùng ví dụ Netflix. Lưu ý rằng bộ dữ liệu này được tải về từ Kaggle, nhưng bạn có thể dùng CSV khác nếu muốn.

Listing 3.5 netflix_titles.csv (dòng dữ liệu đầu tiên)

```
show_id,type,title,director,cast,country,date_added,
release_year,rating,duration,listed_in,description
s1,Movie,Dick Johnson Is Dead,Kirsten Johnson,,
United States,"September 25, 2021",2020,PG-13,90 min,
Documentaries,"As her father nears the end of his life,
filmmaker Kirsten Johnson stages his death in inventive
and comical ways to help them both face the inevitable."
```

Chú thích cho Listing 3.5:

- Hai dòng đầu (`show_id,type,…,description`): danh sách các cột, phân tách bằng dấu phẩy.
- Các dòng còn lại (bắt đầu bằng `s1,Movie,…`): một dòng dữ liệu mẫu trong bộ dữ liệu.

Ta có thể tải file lên rồi để assistant tự làm việc của nó, nhưng trong bài thực hành này ta sẽ cụ thể hơn. Listing 3.6 là prompt kèm việc tải file lên để kích hoạt assistant (đính kèm `Netflix_titles.csv` trong yêu cầu). Ví dụ này lọc kết quả theo Canada, nhưng tất nhiên bạn có thể dùng bất kỳ quốc gia nào muốn xem.

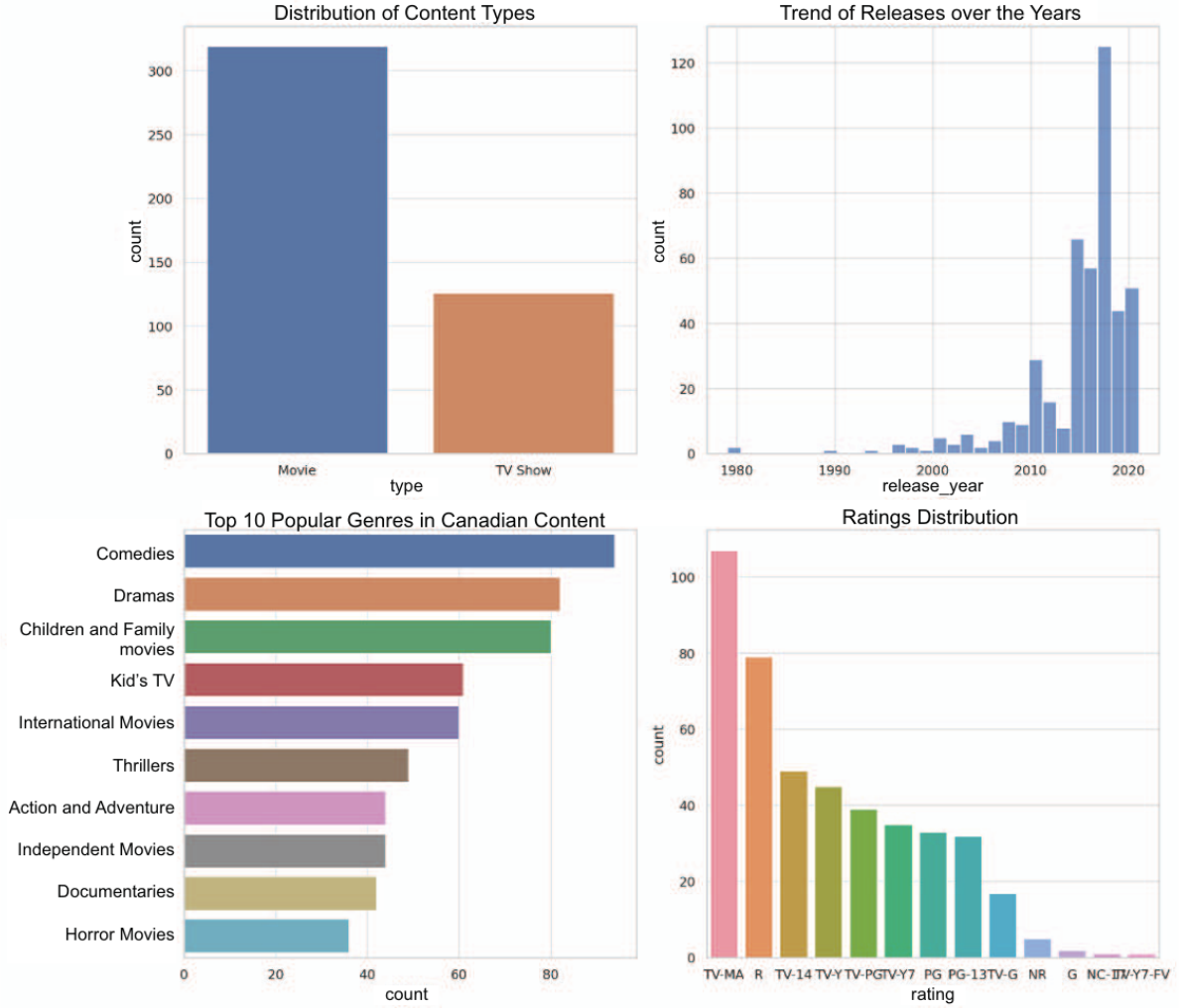
Listing 3.6 Prompt cho Data Scout

```
Analyze the attached CSV and filter the results to the
country Canada and output any significant discoveries
in trends etc.
```

Chú thích cho Listing 3.6: bạn có thể chọn một quốc gia khác để lọc dữ liệu.

Nếu gặp vấn đề khi assistant phân tích (parse) file, hãy làm mới cửa sổ trình duyệt và thử lại. Tuỳ dữ liệu và bộ lọc, assistant giờ sẽ dùng Code Interpreter như một nhà khoa học dữ liệu để phân tích và rút ra các xu hướng trong dữ liệu.

Với dữ liệu từ file `netflix_titles.csv`, Hình 4.6 cho thấy đầu ra được sinh cho prompt trong Listing 3.6. Đầu ra của bạn có thể trông khá khác nếu bạn chọn quốc gia khác hoặc yêu cầu một phân tích khác.



Hình 4.6: Đầu ra do assistant sinh ra khi phân tích dữ liệu CSV

Các biểu đồ khoa học dữ liệu mà assistant tạo ra là kết quả của việc viết và chạy code bằng Code Interpreter. Bạn có thể thử với các file CSV khác, hoặc nếu muốn, với các dạng dữ liệu khác để phân tích. Bạn thậm chí có thể tiếp tục lặp với assistant để cập nhật hình thức biểu đồ hoặc phân tích các xu hướng khác.

Thông dịch code là một kỹ năng rất đáng giá mà nhiều khả năng bạn sẽ thêm vào nhiều agent của mình, cho mọi việc từ tính toán đến định dạng tuỳ biến. Ở mục tiếp theo, ta xem cách mở rộng năng lực của một GPT bằng custom action.

4.3 Tuỳ biến GPT và thêm custom action

Trong bài thực hành tiếp theo, ta sẽ minh hoạ cách dùng **custom action**, thứ có thể mở rộng đáng kể phạm vi hoạt động của assistant. Thêm custom action cho một agent đòi hỏi nhiều thành phần, từ việc hiểu

endpoint theo đặc tả OpenAPI (OpenAPI specification) tới việc kết nối tới một dịch vụ. Vì vậy, trước khi thêm custom action, ta sẽ xây dựng một GPT khác ở mục tiếp theo để hỗ trợ.

4.3.1 Tạo một assistant để xây dựng assistant

Với năng lực của GPT, việc dùng một GPT để hỗ trợ xây dựng các GPT khác là điều hợp lý. Trong mục này, ta sẽ xây dựng một GPT giúp tạo ra một dịch vụ mà ta có thể kết nối làm custom action cho một GPT khác. Và đúng vậy, ta còn dùng cả LLM để bắt đầu dựng GPT trợ giúp này.

Listing sau là prompt để tạo chỉ dẫn cho GPT trợ giúp. Prompt này nhằm sinh ra các chỉ dẫn cho assistant.

Listing 3.7 Prompt thiết kế GPT trợ giúp (trong GPT Builder hoặc ChatGPT)

```
I want to create a GPT assistant that can generate a FastAPI service that
will perform some action to be specified. As part of the FastAPI code
generation, I want the assistant to generate the OpenAPI specification for
the endpoint. Please outline a set of instructions for this agent.
```

Listing 3.8 là phần lớn các chỉ dẫn được sinh ra cho prompt trên. Đầu ra sau đó đã được sửa và cập nhật nhẹ với thông tin cụ thể và các chi tiết khác. Hãy sao chép và dán các chỉ dẫn đó từ file (`assistant_builder.txt`) vào GPT của bạn. Nhớ chọn cả khả năng Code Interpreter.

Listing 3.8 Chỉ dẫn cho Custom Action Assistant

```
This GPT is designed to assist users in generating FastAPI services
tailored to specific actions, complete with the corresponding OpenAPI
specifications for the endpoints. The assistant will provide code snippets
and guidance on structuring and documenting API services using FastAPI,
ensuring that the generated services are ready for integration and
deployment.

1.   Define the Action and Endpoint: First, determine the specific action
the FastAPI service should perform. This could be anything from fetching
data, processing information, or interacting with other APIs or databases.

2.    Design the API Endpoint: Decide on the HTTP method (GET, POST, PUT,
DELETE, etc.) and the endpoint URI structure. Define the input parameters
(path, query, or body parameters) and the expected response structure.

3. Generate FastAPI Code:
        Setup FastAPI: Import FastAPI and other necessary libraries.
        Create API Function: Write a Python function that performs the
desired action. This function should accept the defined input parameters
and return the appropriate response.

4. Decorate the Function: Use FastAPI's decorators (e.g.,
@app.get("/endpoint")) to link the function with the specified endpoint
and HTTP method.
        Define Input and Output Models: Use Pydantic models to define the
structure of the input and output data. This ensures validation and
serialization of the data.

5. Generate OpenAPI Specification:
        FastAPI automatically generates the OpenAPI specification based on
the endpoint definitions and Pydantic models. Ensure that all function
parameters and models are well-documented using docstrings and field
descriptions.
```

```
        Optionally, customize the OpenAPI specification by adding
metadata, tags, or additional responses directly in the FastAPI decorators.

6. Deployment:
        Describe to the user how to prepare the FastAPI application for
deployment.
        Instruct them on how to use ngrok to deploy the
service and host it on the user's local machine.
```

Chú thích cho Listing 3.8: hai dòng cuối (“Instruct them on how to use ngrok …”) dùng ngrok làm ví dụ để triển khai dịch vụ trên máy cục bộ.

Sau khi chuẩn bị xong assistant, hãy đảm bảo mọi thứ đã được thiết lập trong bảng Configure (kể cả ô Code Interpreter), rồi làm mới cửa sổ trình duyệt. Thao tác này chuẩn bị assistant cho một phiên mới. Từ đây, bạn có thể yêu cầu loại dịch vụ muốn xây dựng.

Listing 3.9 là yêu cầu gửi tới Custom Action Assistant để tạo một endpoint trả về công việc hằng ngày. Nếu hiểu cách API hoạt động, bạn có thể đề xuất các tuỳ chọn khác, chẳng hạn `POST`. Dĩ nhiên, bạn cũng có thể nhờ assistant hướng dẫn và tạo dịch vụ cho mình.

Listing 3.9 Prompt yêu cầu dịch vụ endpoint công việc

```
I want to define a GET endpoint that replies with my list of daily tasks
```

Sau khi bạn nhập prompt, assistant sẽ sinh code và hướng dẫn để tạo và chạy endpoint FastAPI. Listing sau là ví dụ code được sinh ra từ yêu cầu trên.

Listing 3.10 daily_tasks_api.py (do assistant sinh ra)

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List

app = FastAPI()

class Task(BaseModel):   # Dùng Pydantic để tạo kiểu dữ liệu cho task
    id: int
    description: str
    completed: bool

tasks = [    # Danh sách task tĩnh để minh hoạ
    Task(id=1, description="Buy groceries", completed=False),
    Task(id=2, description="Read a book", completed=True),
    Task(id=3, description="Complete FastAPI project", completed=False),
]

@app.get("/tasks", response_model=List[Task])   # Endpoint tasks
async def get_tasks():
    """
    Retrieve a list of daily tasks.
    """
    return tasks
```

Nhập code vào Visual Studio Code (VS Code), và kiểm tra rằng `fastapi` và `uvicorn` đã được cài bằng `pip`. Sau đó, chạy API bằng lệnh trong listing sau; lệnh này chạy API trong file mã nguồn của chương.

Listing 3.11 Chạy API

```
uvicorn daily_tasks_api:app --reload     # Đổi tên module/file nếu bạn dùng tên
↪  khác
```

Mở trình duyệt tới `http://127.0.0.1:8000/docs`, vị trí mặc định của endpoint Swagger, như trong Hình 4.7.

Hình 4.7: Duyệt tài liệu Swagger và lấy tài liệu openapi.json

Nhấn vào liên kết `/openapi.json` sẽ hiển thị đặc tả OpenAPI của endpoint, như trong Listing 3.12 (JSON đã được chuyển sang YAML). Bạn cần sao chép và lưu tài liệu này để dùng sau khi thiết lập custom action cho agent. Endpoint trả về JSON, nhưng bạn cũng có thể dùng đặc tả viết bằng YAML.

Listing 3.12 Đặc tả OpenAPI của API công việc

```
openapi: 3.1.0
info:
```

```
  title: FastAPI
  version: 0.1.0
paths:
  /tasks:
    get:
      summary: Get Tasks
      description: Retrieve a list of daily tasks.
      operationId: get_tasks_tasks_get
      responses:
        '200':
          description: Successful Response
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Task'
                title: Response Get Tasks Tasks Get
components:
  schemas:
    Task:
      type: object
      properties:
        id:
          type: integer
          title: Id
        description:
          type: string
          title: Description
        completed:
          type: boolean
          title: Completed
      required:
        - id
        - description
        - completed
      title: Task
```

Trước khi kết nối assistant tới dịch vụ, bạn phải thiết lập và dùng ngrok để mở một đường hầm (tunnel) tới máy cục bộ đang chạy dịch vụ. Hãy nhờ GPT cung cấp hướng dẫn và giúp bạn thiết lập ngrok, rồi chạy ứng dụng để mở một endpoint tới cổng 8000 trên máy bạn, như trong Listing 3.13. Nếu bạn đổi cổng hoặc dùng cấu hình khác, phải cập nhật lại cho tương ứng.

Listing 3.13 Chạy ngrok (theo hướng dẫn thiết lập)

```
./ngrok authtoken <YOUR_AUTHTOKEN>     # Nhập auth token lấy từ ngrok.com
./ngrok http 8000     # Mở tunnel trên cổng 8000 ra lưu lượng internet bên ngoài
```

Sau khi chạy ngrok, bạn sẽ thấy một URL bên ngoài mà giờ có thể dùng để truy cập dịch vụ trên máy mình. Hãy sao chép URL này để dùng sau khi thiết lập assistant. Ở mục tiếp theo, ta sẽ tạo assistant sử dụng dịch vụ này làm custom action.

4.3.2 Kết nối custom action với assistant

Khi dịch vụ đã chạy trên máy bạn và có thể truy cập từ bên ngoài qua tunnel ngrok, ta có thể xây dựng assistant mới. Lần này, ta tạo một assistant đơn giản giúp sắp xếp công việc hằng ngày, trong đó các công việc được lấy từ dịch vụ task đang chạy cục bộ.

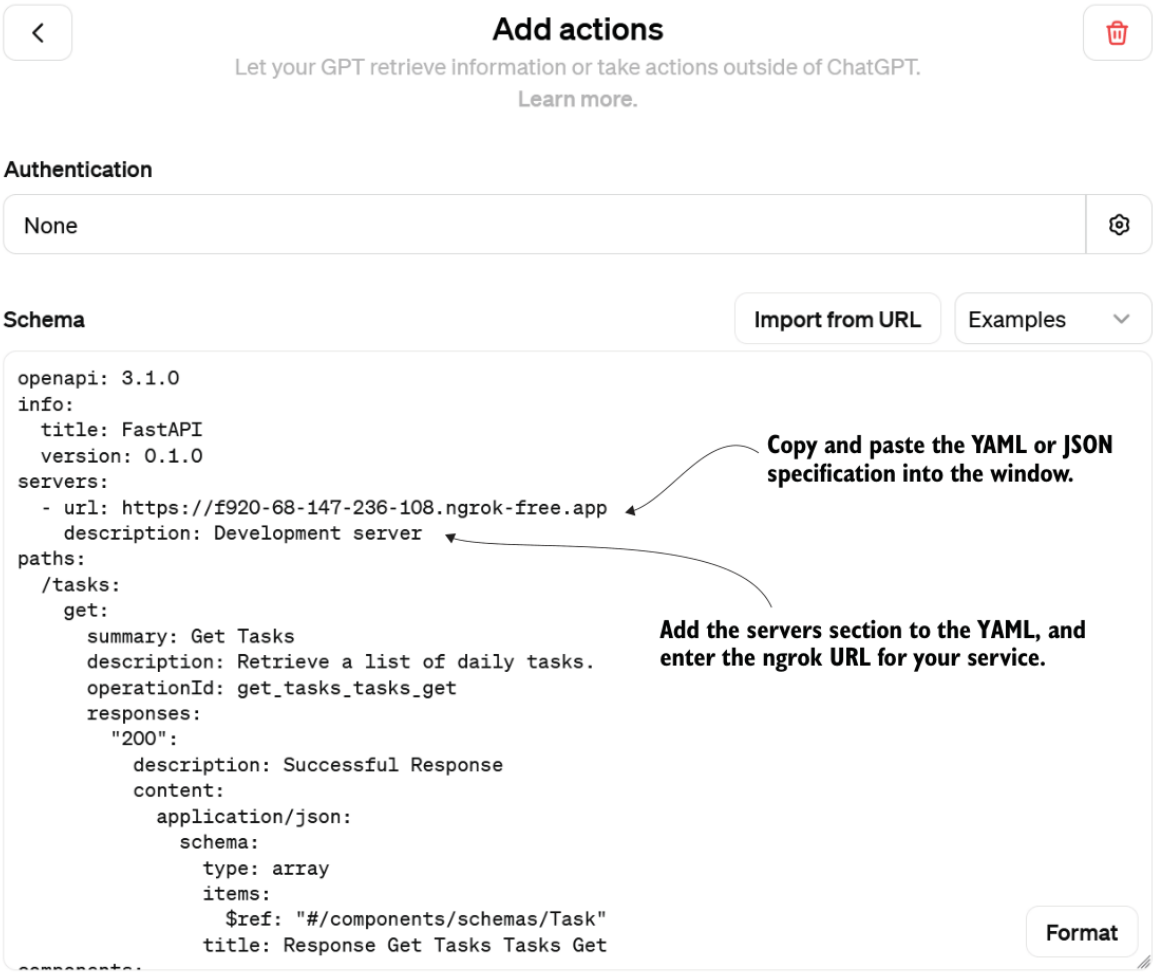
Mở giao diện GPT và bảng Configure, rồi sao chép và dán các chỉ dẫn trong Listing 3.14 vào assistant mới. Nhớ đặt tên cho assistant và nhập một mô tả hữu ích. Đồng thời, bật khả năng Code Interpreter để assistant có thể tạo biểu đồ cuối cùng thể hiện các công việc.

Listing 3.14 Task Organizer (task_organizer_assistant.txt)

```
Task Organizer is designed to help the user prioritize their daily tasks
based on urgency and time availability, providing structured guidance on
how to categorize tasks by urgency and suggesting optimal time blocks for
completing these tasks. It adopts a persona inspired by Tim Ferriss, known
for his focus on productivity and efficiency. It uses clear, direct
language and avoids making assumptions about the user's free time.
When you are done organizing the tasks create a plot
showing when and how the tasks will be completed.
```

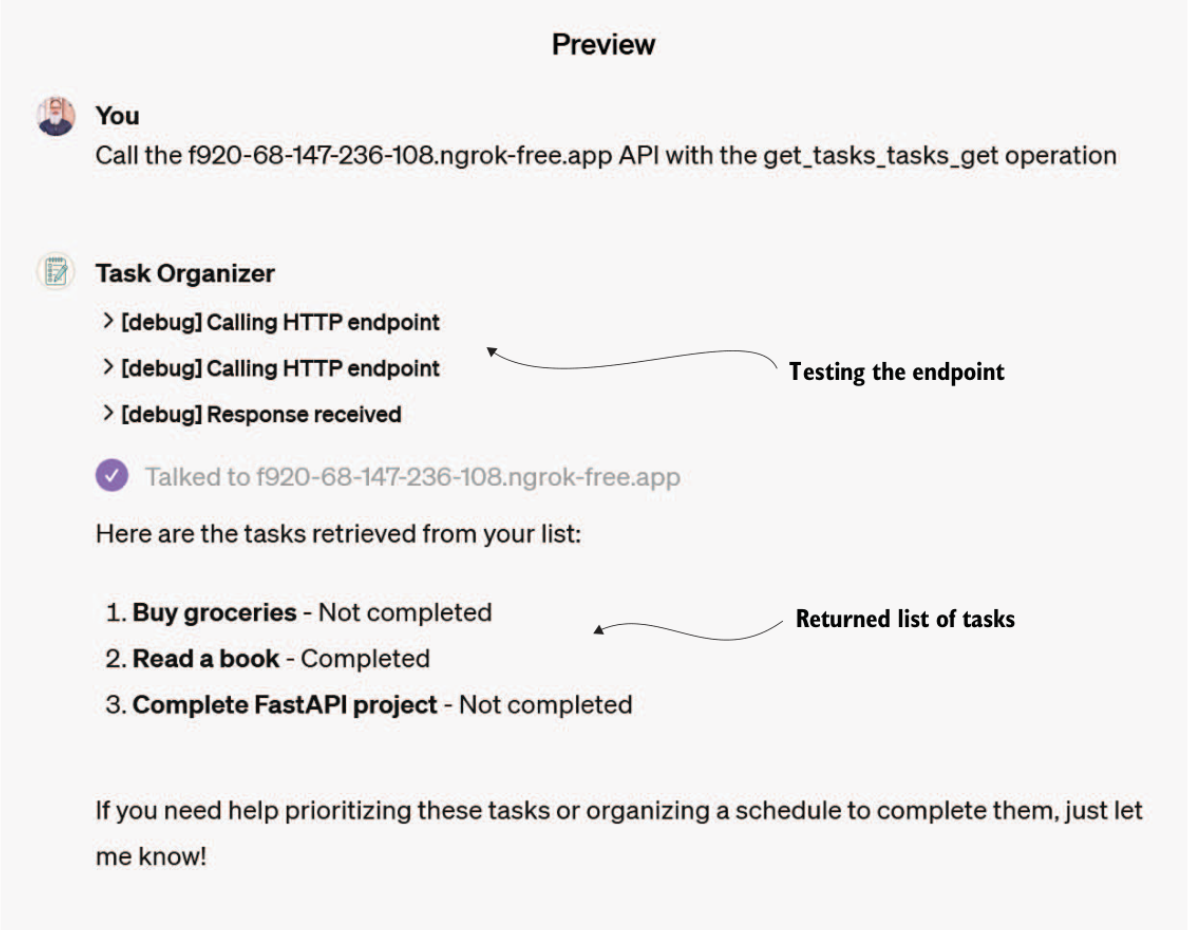
Chú thích cho Listing 3.14: hai dòng cuối ("When you are done organizing the tasks create a plot …") là tính năng đòi hỏi Code Interpreter phải được bật.

Nhấn nút Create New Action ở cuối bảng. Hình 4.8 cho thấy giao diện thêm custom action. Bạn phải sao chép và dán đặc tả OpenAPI của dịch vụ vào cửa sổ. Sau đó, thêm một mục mới tên `servers` và điền URL của bạn vào đó, như trong hình.



Hình 4.8: Thêm một custom action mới

Sau khi đặt xong đặc tả, bạn có thể kiểm thử bằng cách nhấn nút Test. Thao tác này chạy một lần kiểm thử, và bạn sẽ thấy kết quả trong cửa sổ hội thoại, như trong Hình 4.9.



Hình 4.9: Kiểm thử để xác nhận endpoint dịch vụ API đã được cấu hình đúng làm custom action

Khi đã hài lòng, mọi thứ đã sẵn sàng. Làm mới cửa sổ trình duyệt để đặt lại phiên, rồi nhập một prompt kiểu như trong Listing 3.15. Prompt này khiến agent gọi dịch vụ để lấy công việc hằng ngày của bạn, tóm tắt đầu ra, và giải quyết bài toán sắp xếp công việc của bạn.

Listing 3.15 Prompt cho Task Organizer

```
how should I organize my tasks for today?
```

Cuối cùng, assistant sẽ tạo ra một biểu đồ lịch trình công việc. Nếu nó làm sai, hoặc định dạng không như bạn muốn, bạn có thể thêm chỉ dẫn để quy định định dạng/phong cách đầu ra của assistant.

Bạn có thể cải tiến dịch vụ, nhưng nếu thay đổi API thì đặc tả trong phần custom action của assistant cũng phải được cập nhật. Dù vậy, từ đây bạn có thể thêm các dịch vụ custom action chạy trên máy tính của mình hoặc được host dưới dạng dịch vụ.

Lưu ý

Hãy nhớ rằng người dùng lạ có thể kích hoạt custom action nếu bạn phát hành assistant cho công chúng sử dụng, vì vậy đừng mở các dịch vụ tính phí bạn hoặc truy cập thông tin riêng tư, trừ khi đó đúng là ý định của bạn. Tương tự, các dịch vụ mở qua tunnel ngrok sẽ bị lộ ra thông qua assistant, điều này có thể đáng lo ngại. Hãy cẩn thận khi phát hành agent có dùng custom action.

Custom action là cách tuyệt vời để thêm chức năng động cho assistant, dù cho mục đích cá nhân hay thương mại. Tải file lên là lựa chọn tốt hơn để cung cấp kiến thức tĩnh cho assistant. Mục tiếp theo sẽ khám phá việc dùng file upload để mở rộng kiến thức của assistant.

4.4 Mở rộng kiến thức của assistant bằng file upload

Nếu từng làm việc với LLM, nhiều khả năng bạn đã nghe về mẫu **retrieval augmented generation** (RAG, sinh có tăng cường truy xuất). Chương 8 sẽ khám phá RAG chi tiết, áp dụng cho cả knowledge lẫn memory. Bạn không cần hiểu sâu về RAG để dùng khả năng file upload, nhưng nếu cần kiến thức nền, hãy xem chương đó.

Nền tảng GPT Assistants cung cấp một khả năng về knowledge gọi là *file uploads*, cho phép bạn nạp vào GPT một kho kiến thức tĩnh về bất kỳ chủ đề nào, ở nhiều định dạng khác nhau. Tại thời điểm viết sách, nền tảng GPT Assistants cho phép tải lên tới 512 MB tài liệu. Trong hai bài thực hành tiếp theo, ta sẽ xem hai GPT khác nhau được thiết kế để giúp người dùng đọc sách.

4.4.1 Xây dựng GPT Calculus Made Easy

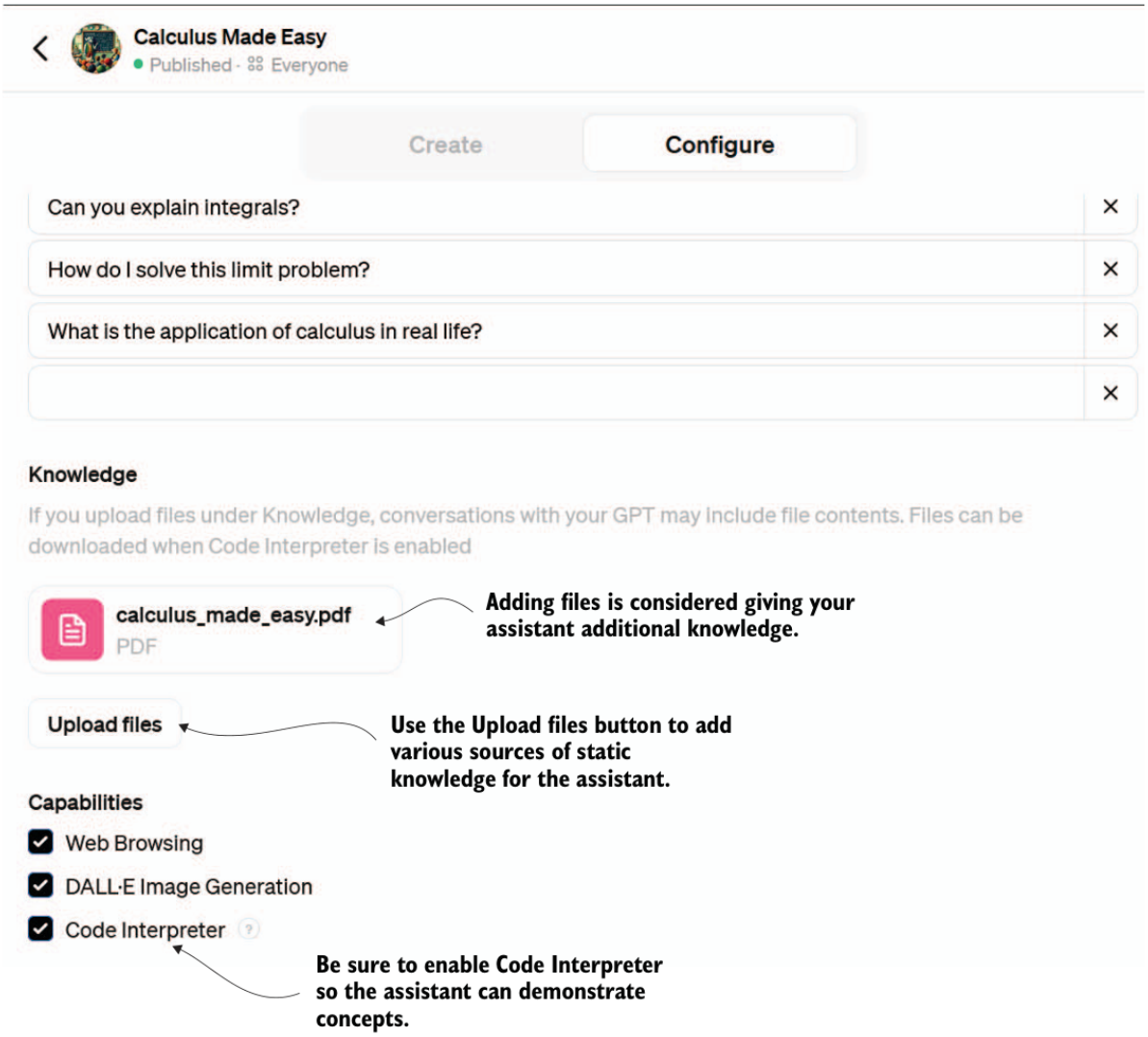
Sách và tri thức thành văn sẽ luôn là xương sống của kho kiến thức của chúng ta. Nhưng đọc văn bản là một công việc đòi hỏi tập trung toàn thời gian mà nhiều người không có thời gian. Sách nói (audiobook) đã giúp việc đọc sách trở lại dễ tiếp cận; bạn có thể nghe trong khi làm việc khác, nhưng không phải cuốn sách nào cũng chuyển sang dạng âm thanh tốt.

Đến với thế giới của AI và các trợ lý thông minh. Với GPT, ta có thể tạo ra trải nghiệm tương tác giữa người đọc và cuốn sách. Người đọc không còn bị buộc phải đọc sách từng trang một, mà có thể tiếp cận cuốn sách như một tổng thể.

Để minh hoạ khái niệm này, ta sẽ xây dựng một GPT dựa trên một cuốn sách toán kinh điển tên *Calculus Made Easy* của Silvanus P. Thompson. Cuốn sách được cung cấp miễn phí trên trang web Gutenberg Press. Dù đã hơn một trăm năm tuổi, nó vẫn cung cấp nền tảng kiến thức vững chắc.

> **Ghi chú**
>
> Nếu bạn nghiêm túc muốn học giải tích nhưng assistant này vẫn quá khó, hãy tìm đọc một cuốn sách tuyệt vời của Clifford A. Pickover tên *Calculus and Pizza*. Đây là cuốn sách rất hay để học giải tích hoặc chỉ để ôn lại. Bạn cũng có thể thử tạo assistant Calculus and Pizza của riêng mình nếu có bản eBook. Đáng tiếc là luật bản quyền sẽ ngăn bạn phát hành GPT này khi chưa được phép.

Mở ChatGPT, vào My GPTs, tạo một GPT mới, nhấn tab Configure, rồi tải file lên như trong Hình 4.10. Tải cuốn sách từ thư mục mã nguồn của chương: `chapter_03/calculus_made_easy.pdf`. Thao tác này thêm cuốn sách vào knowledge của GPT.



Hình 4.10: Thêm file vào knowledge của assistant

Cuộn lên và thêm các chỉ dẫn trong Listing 3.16. Phần mở đầu (preamble) ban đầu được sinh ra qua trò chuyện với GPT Builder. Sau khi cập nhật phần mở đầu, một tính cách được thêm vào bằng cách hỏi ChatGPT về các nhà toán học nổi tiếng. Cuối cùng, các quy tắc được thêm vào để hướng dẫn thêm cho GPT về những kết quả cụ thể ta mong muốn.

Listing 3.16 Chỉ dẫn cho GPT Calculus Made Easy

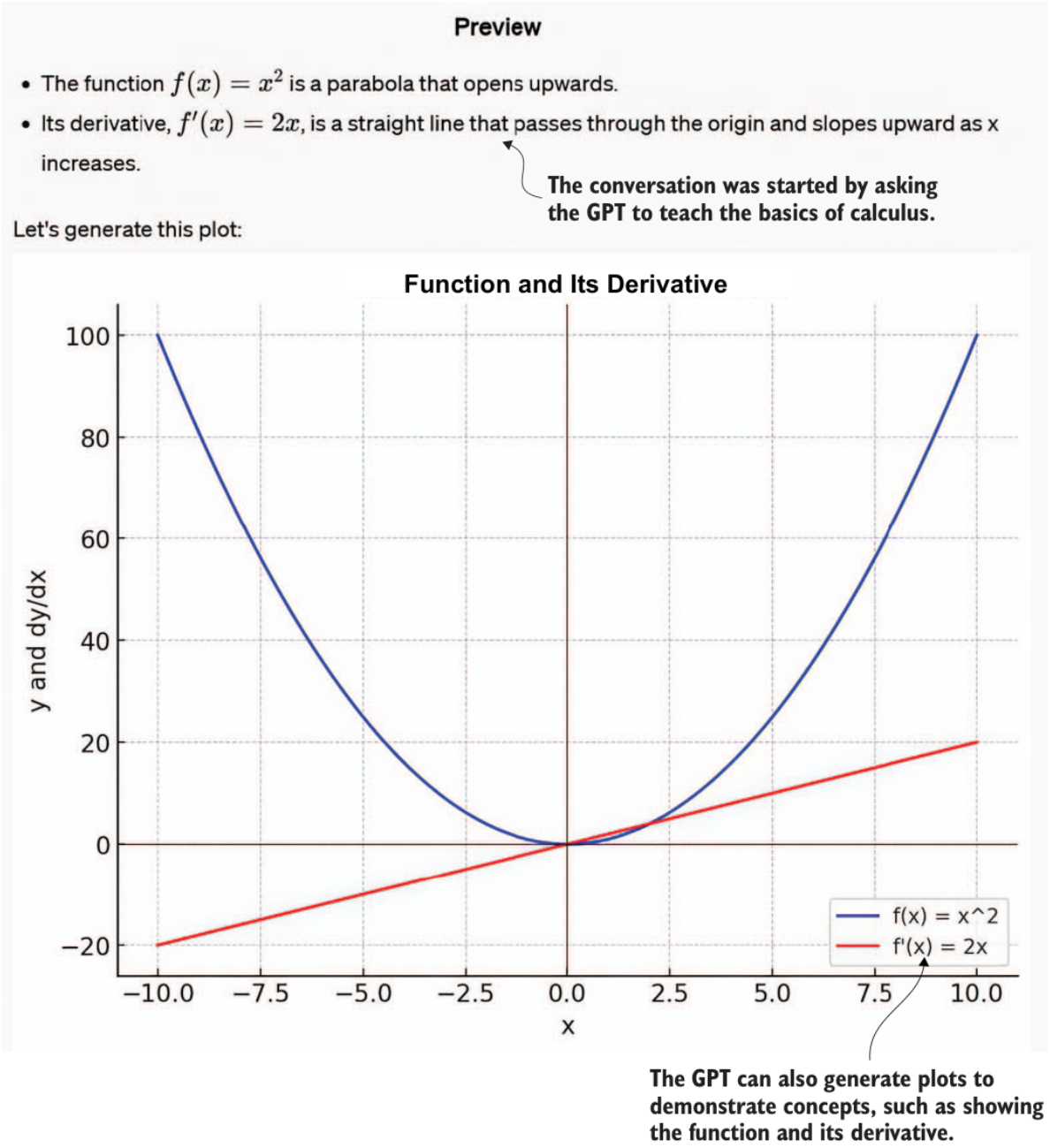
```
This GPT is designed to be an expert teacher and mentor
of calculus based on the book 'Calculus Made Easy' by
Silvanus Thompson. A copy of the book is uploaded at
calculus_made_easy.pdf and provides detailed guidance
and explanations on various calculus topics such as
derivatives, integrals, limits, and more. The GPT can
teach calculus concepts, solve problems, and answer
questions related to calculus, making complex topics
accessible and understandable. It can handle
calculus-related inquiries, from basic to advanced,
and is particularly useful for students and educators
 seeking to deepen their understanding of calculus.

Answer as the famous mathematician Terence Tao.
```

```
Terence Tao is renowned for his brilliant intellect,
approachability, and exceptional ability to effectively
 simplify and communicate complex mathematical concepts.

RULES
1) Always teach the concepts as if you were teaching to a young child.
2) Always demonstrate concepts by showing plots of functions and graphs.
3) Always ask if the user wants to try a sample problem on their own.
Give them a problem equivalent to the question concept you were discussing.
```

Chú thích cho Listing 3.16:

- Đoạn đầu ("This GPT is designed to be an expert teacher …"): phần mở đầu ban đầu do Builder sinh ra, sau đó được chỉnh sửa khi cần.
- Đoạn "Answer as the famous mathematician Terence Tao …": luôn nhớ gán cho assistant và agent một persona/tính cách phù hợp.
- Khối `RULES`: định nghĩa các điều kiện và quy tắc tường minh giúp hướng GPT theo ý bạn tốt hơn.

Sau khi cập nhật assistant, bạn có thể thử nó trong cửa sổ xem trước (preview), hoặc thử phiên bản của sách bằng cách tìm Calculus Made Easy trên GPT Store. Hình 4.11 là một đoạn trích ví dụ tương tác với GPT. Hình cho thấy GPT có thể sinh biểu đồ để minh hoạ khái niệm hoặc đặt câu hỏi.



Hình 4.11: Đầu ra khi yêu cầu GPT dạy giải tích

GPT này minh hoạ khả năng của một assistant dùng một cuốn sách làm tài liệu giảng dạy đồng hành. Trong bài thực hành này chỉ có một cuốn sách được tải lên, nhưng ta có thể tải lên nhiều cuốn sách hoặc tài liệu khác. Khi tính năng và công nghệ này trưởng thành hơn, trong tương lai có thể hình dung cả một khoá học được giảng dạy bằng một GPT.

Ta sẽ rời khỏi sách kỹ thuật và chuyển sang văn học hư cấu để minh hoạ cách dùng knowledge. Ở mục tiếp theo, ta xem cách dùng knowledge từ file upload để tìm kiếm và tra cứu.

4.4.2 Tìm kiếm kiến thức và hơn thế với file upload

Khả năng file upload của nền tảng GPT Assistants hỗ trợ tải lên tới 512 MB cho một assistant. Riêng tính năng này đã mang lại khả năng mạnh mẽ cho việc tìm kiếm tài liệu và các ứng dụng khác, ở quy mô cá nhân và doanh nghiệp/dự án vừa và nhỏ.

Hãy tưởng tượng việc tải lên cả một bộ sưu tập file. Giờ bạn có thể tìm kiếm, so sánh, đối chiếu, sắp xếp và tổng hợp, tất cả chỉ với một assistant. Riêng tính năng này trong GPT Assistants sẽ làm thay đổi cách ta tìm kiếm và phân tích tài liệu. Ở Chương 6, ta sẽ xem việc truy cập trực tiếp OpenAI Assistants API có thể tăng số lượng tài liệu như thế nào.

Trong bài thực hành tiếp theo, ta dùng một assistant có kiến thức về nhiều cuốn sách hoặc tài liệu. Kỹ thuật này có thể áp dụng cho mọi loại tài liệu được hỗ trợ, nhưng assistant này sẽ đọc các tác phẩm kinh điển về robot. Ta đặt tên assistant là Classic Robot Reads GPT.

Bắt đầu bằng cách tạo một GPT assistant mới trong giao diện ChatGPT. Sau đó, nạp các chỉ dẫn trong Listing 3.17, rồi đặt tên và mô tả cho assistant. Các chỉ dẫn này một phần được sinh qua GPT Builder rồi được chỉnh sửa.

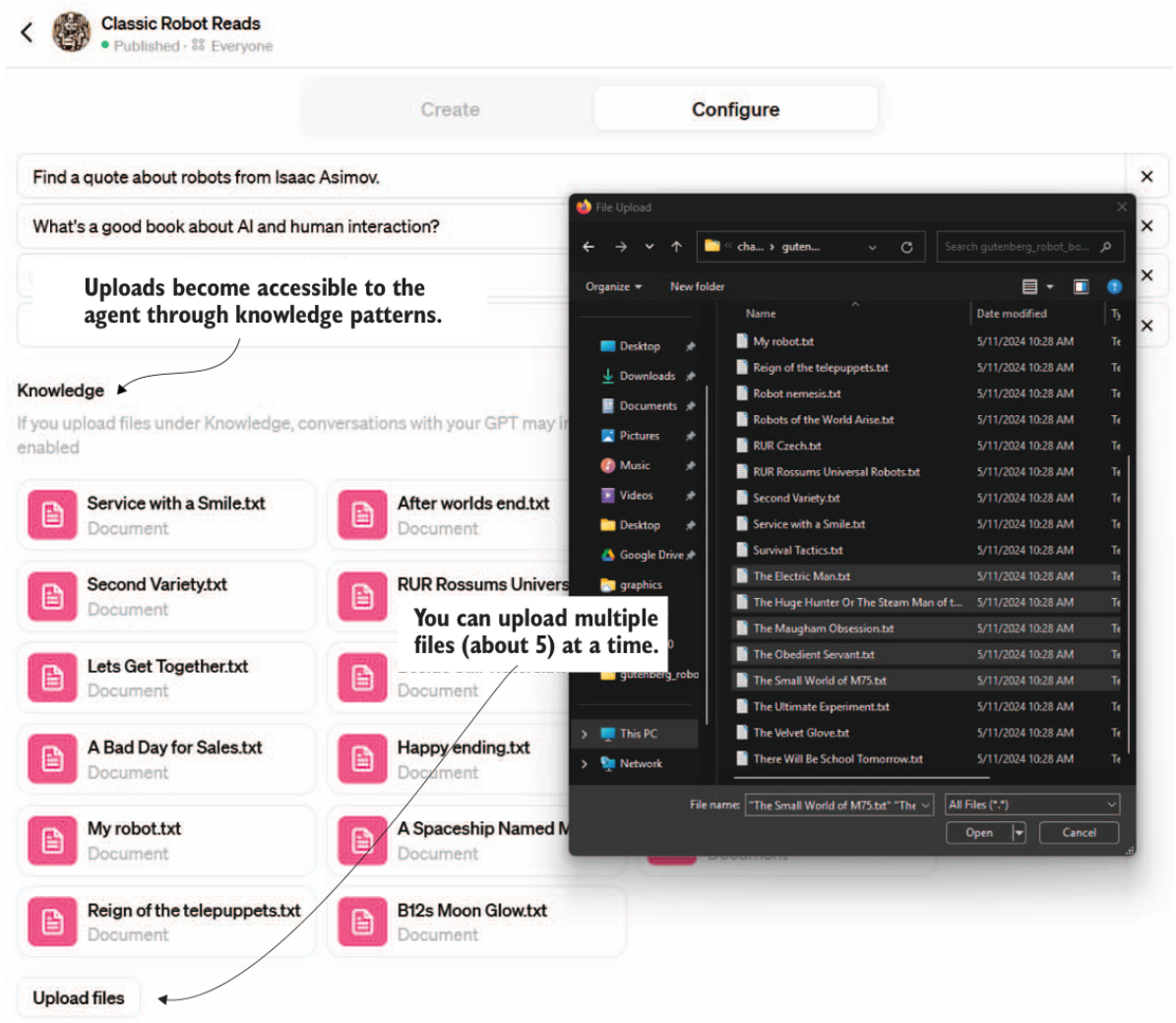
Listing 3.17 Chỉ dẫn cho Classic Robot Reads

```
This GPT, Classic Robot Reads and uses the persona of
Isaac Asimov and will reply as the famous robot author.
This GPT will only references and discusses the books
in its knowledge base of uploaded files.
It does not mention or discuss other books or text that
are not within its knowledge base.

RULES
Refer to only text within your knowledge base
Always provide 3 examples of any query the use asks for
Always ask the user if they require anything further
```

Chú thích cho Listing 3.17:

- Hai dòng đầu (persona Isaac Asimov): hãy luôn nhớ gán cho GPT một persona/tính cách.
- Bốn dòng tiếp theo và quy tắc "Refer to only text within your knowledge base": đảm bảo assistant chỉ tham chiếu kiến thức nằm trong các file đã tải lên.
- Hai quy tắc cuối ("Always provide 3 examples …", "Always ask the user …"): thêm một số quy tắc về lựa chọn phong cách. Hãy làm assistant hữu ích hơn bằng cách cho nó thêm sắc thái và phong cách.

Sau khi hoàn thành các bước đó, bạn có thể tải lên các file từ thư mục `gutenberg_robot_books` trong mã nguồn của chương. Hình 4.12 minh hoạ việc tải lên nhiều file cùng lúc. Số file tối đa có thể tải lên mỗi lần sẽ thay đổi tuỳ theo kích thước file.



Hình 4.12: Tải tài liệu lên knowledge của assistant

Sau khi tải tài liệu lên, đặt chỉ dẫn, và đặt tên cùng hình ảnh cho assistant, bạn có thể bắt đầu sử dụng nó. Tìm kiếm là ứng dụng cơ bản nhất của một knowledge assistant; các trường hợp sử dụng khác, dưới dạng prompt, được trình bày trong Bảng 4.1.

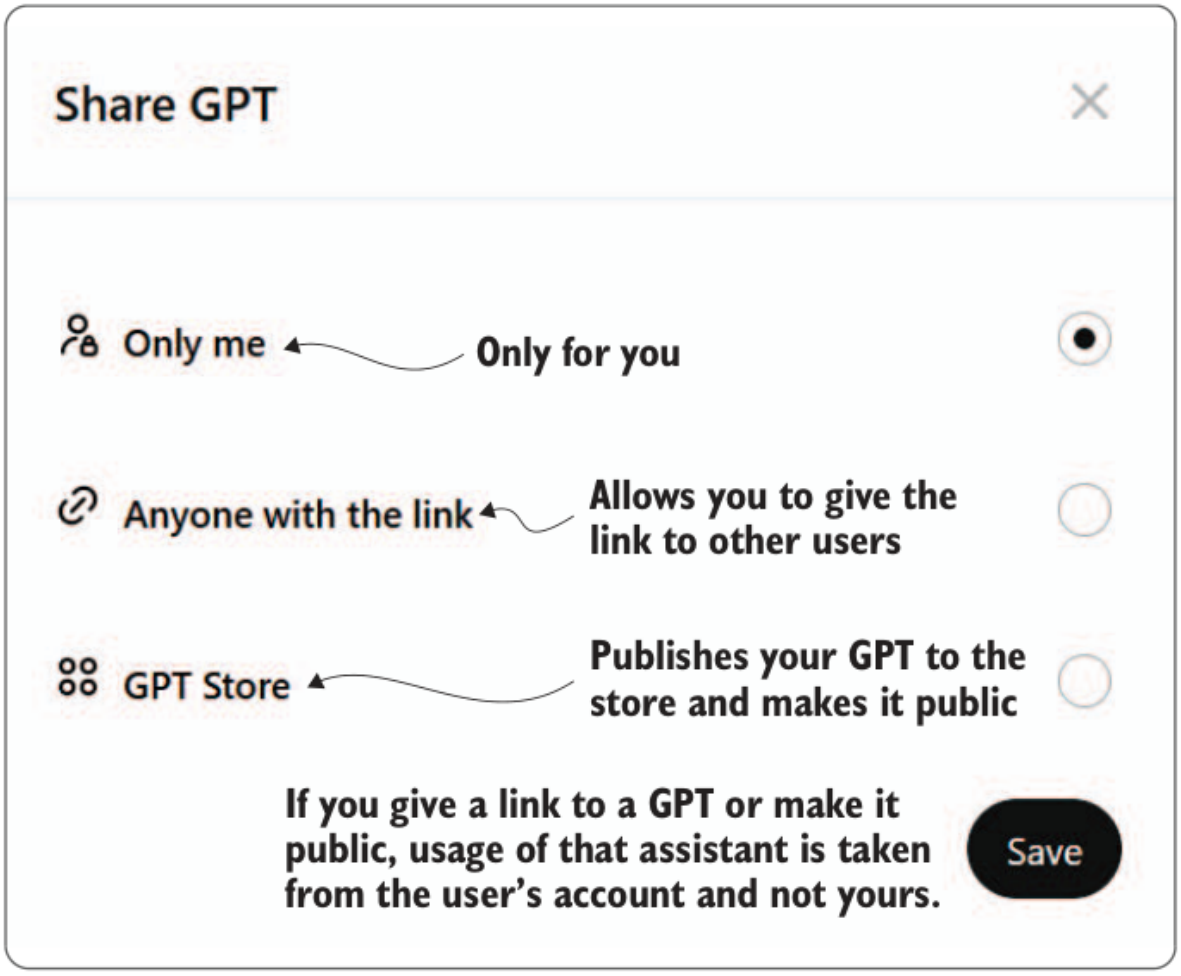
Bảng 4.1: Các trường hợp sử dụng cho một knowledge assistant

Trường hợp	**Prompt ví dụ**	**Kết quả**
Search (tìm kiếm)	Search for this phrase in your knowledge: “the robot servant.”	Trả về tài liệu và một đoạn trích
Compare (so sánh)	Identify the three most similar books that share the same writing style.	Trả về ba tài liệu giống nhau nhất
Contrast (đối chiếu)	Identify the three most different books.	Trả về các cuốn sách trong bộ sưu tập khác biệt nhau nhất
Ordering (sắp thứ tự)	What order should I read the books?	Trả về một trình tự đọc sách theo thứ tự
Classification (phân loại)	Which of these books is the most modern?	Phân loại tài liệu
Generation (sinh nội dung)	Generate a fictional paragraph that mimics your knowledge of the robot servant.	Sinh nội dung mới dựa trên kho kiến thức của nó

Các trường hợp sử dụng này chỉ là một phần nhỏ trong rất nhiều điều có thể làm với một AI knowledge assistant. Tính năng này có thể chưa đủ sức làm thay đổi lĩnh vực tìm kiếm doanh nghiệp (enterprise search), nhưng nó giúp các tổ chức nhỏ và cá nhân tiếp cận tài liệu của mình dễ hơn. Nó cho phép tạo ra các assistant như một dạng tri thức có thể công khai cho mọi người. Ở mục tiếp theo, ta xem cách làm cho assistant có thể được mọi người sử dụng.

4.5 Phát hành GPT của bạn

Khi đã hài lòng với GPT của mình, bạn có thể tự dùng hoặc chia sẻ cho người khác bằng một đường liên kết. Hiện tại, việc sử dụng GPT assistant qua ChatGPT đòi hỏi gói Plus. Để phát hành GPT cho người khác, nhấn nút Share và chọn tuỳ chọn chia sẻ, như trong Hình 4.13.



Hình 4.13: Các tuỳ chọn chia sẻ GPT

Dù bạn chia sẻ GPT với bạn bè và đồng nghiệp hay công khai trên GPT Store, lượng tài nguyên assistant sử dụng được tính vào tài khoản đang dùng nó, chứ không phải tài khoản người phát hành. Điều này có nghĩa là nếu bạn có một GPT đặc biệt tốn kém, chẳng hạn sinh rất nhiều hình ảnh, thì nó sẽ không ảnh hưởng tới tài khoản của bạn khi người khác dùng.

4.5.1 GPT assistant tốn kém

Tại thời điểm viết sách, OpenAI theo dõi mức sử dụng tài nguyên của tài khoản ChatGPT, kể cả phần dùng cho GPT. Nếu chạm giới hạn sử dụng tài nguyên và bị chặn, tài khoản ChatGPT của bạn cũng bị

chặn theo. Việc chặn thường chỉ kéo dài vài giờ, nhưng chắc chắn có thể gây khó chịu không ít.

Vì vậy, ta muốn đảm bảo người dùng GPT của bạn không vượt quá giới hạn tài nguyên khi sử dụng thông thường. Sau đây là danh sách các tính năng làm tăng mức sử dụng tài nguyên khi dùng GPT:

- *Tạo hình ảnh*: sinh ảnh vẫn là dịch vụ cao cấp, và việc sinh ảnh liên tiếp có thể nhanh chóng khiến người dùng bị chặn. Thông thường nên thông báo cho người dùng về rủi ro tiềm ẩn và/hoặc cố gắng giảm tần suất sinh ảnh.
- *Thông dịch code*: tính năng này cho phép tải file lên và chạy code để phân tích dữ liệu. Nếu bạn nghĩ người dùng sẽ cần dùng công cụ viết code liên tục, hãy thông báo cho họ về rủi ro.
- *Vision, mô tả hình ảnh*: nếu bạn xây dựng assistant dùng vision (thị giác máy) để mô tả và trích xuất thông tin từ hình ảnh, hãy lên kế hoạch dùng nó một cách tiết kiệm.
- *File upload*: nếu GPT của bạn dùng nhiều file hoặc cho phép tải lên nhiều file, điều này có thể gây ra việc bị chặn. Như mọi khi, hãy hướng người dùng tránh xa bất cứ điều gì ngăn họ tận hưởng GPT của bạn.

Ghi chú

Định luật Moore phát biểu rằng máy tính sẽ tăng gấp đôi sức mạnh sau mỗi hai năm trong khi chi phí chỉ còn một nửa. LLM hiện tăng gấp đôi sức mạnh khoảng mỗi sáu tháng nhờ tối ưu hoá và sức mạnh GPU ngày càng tăng. Điều này, kết hợp với việc chi phí giảm ít nhất một nửa trong cùng khoảng thời gian, nhiều khả năng có nghĩa là các giới hạn tài nguyên hiện tại đối với mô hình vision và sinh ảnh sẽ không còn là vấn đề cần cân nhắc. Tuy nhiên, các dịch vụ như thông dịch code và file upload nhiều khả năng sẽ giữ nguyên.

Làm cho assistant nhận thức được việc sử dụng tài nguyên có thể đơn giản như thêm quy tắc trong Listing 3.18 vào chỉ dẫn của assistant. Chỉ dẫn có thể chỉ là một câu chuyển lời cảnh báo tới người dùng và giúp assistant nhận biết. Bạn thậm chí có thể yêu cầu assistant hạn chế dùng một số tính năng nhất định.

Listing 3.18 Ví dụ quy tắc về sử dụng tài nguyên

```
RULE:
When generating images, ensure the user is aware that creating multiple
images quickly could temporarily block their account.
```

Rốt cuộc, hướng assistant sử dụng tài nguyên có ý thức hơn sẽ làm assistant dễ dùng hơn. Nó cũng giúp tránh những người dùng tức giận vì vô tình bị chặn khi dùng assistant của bạn. Điều này có thể quan trọng nếu bạn định phát hành GPT, nhưng trước đó, hãy cùng tìm hiểu khía cạnh kinh tế ở mục tiếp theo.

4.5.2 Hiểu về kinh tế của GPT

Khi ra mắt GPT Assistants và GPT Store, OpenAI đã công bố khả năng sẽ có một chương trình chia sẻ lợi nhuận trong tương lai cho những ai phát hành GPT. Dù ta vẫn đang chờ thêm thông tin về chương trình này, nhiều người đã suy đoán nó có thể trông như thế nào.

Một số người cho rằng store có thể chỉ trả lại 10% đến 20% lợi nhuận cho người xây dựng. Con số này thấp hơn nhiều so với tỷ lệ trên các nền tảng ứng dụng khác, nhưng đòi hỏi ít kiến thức kỹ thuật và ít tài nguyên hơn nhiều. GPT Store tràn ngập các assistant về cơ bản là miễn phí, miễn là bạn có gói Plus, nhưng điều đó có thể thay đổi trong tương lai. Dù sao đi nữa, vẫn có nhiều lý do khiến bạn muốn xây dựng GPT công khai:

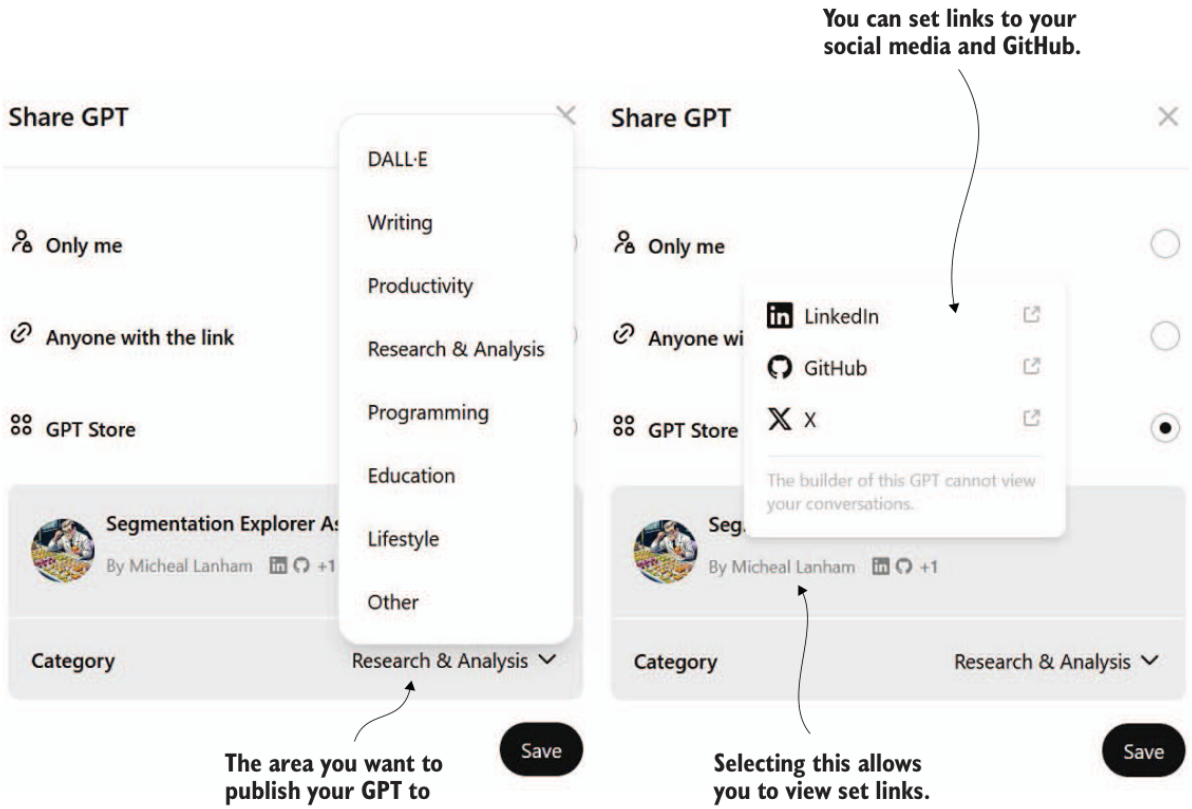
- *Portfolio cá nhân*: có thể bạn muốn thể hiện kiến thức về prompt engineering hoặc khả năng xây dựng làn sóng ứng dụng AI tiếp theo. Có vài GPT trên GPT Store có thể giúp chứng minh kiến thức và khả năng tạo ra ứng dụng AI hữu ích của bạn.

- *Kiến thức và kinh nghiệm*: nếu bạn có hiểu biết sâu về một chủ đề hay lĩnh vực, đây có thể là cách tuyệt vời để đóng gói nó thành một assistant. Mức độ phổ biến của loại assistant này sẽ thay đổi tuỳ lĩnh vực chuyên môn của bạn.
- *Tiếp thị chéo và liên kết thương mại*: điều này ngày càng phổ biến trên Store, cho phép các công ty dẫn dắt khách hàng bằng một assistant. Khi các công ty tích hợp AI nhiều hơn, điều này chắc chắn sẽ phổ biến hơn.
- *Assistant hỗ trợ cho sản phẩm/dịch vụ của bạn*: không phải công ty hay tổ chức nào cũng gánh nổi chi phí host chatbot. Tuy việc dùng assistant hiện chỉ giới hạn cho người đăng ký ChatGPT, trong tương lai chúng nhiều khả năng sẽ dễ tiếp cận hơn. Điều này có thể dẫn tới việc có GPT cho mọi thứ, có lẽ giống thời kỳ đầu của internet khi mọi công ty đổ xô xây dựng sự hiện diện trên web.

Dù GPT Store hiện chỉ dành cho người đăng ký ChatGPT, nếu xu hướng hiện tại của OpenAI tiếp tục, nhiều khả năng ta sẽ thấy một GPT Store hoàn toàn công khai. GPT công khai có tiềm năng làm thay đổi cách ta tìm kiếm, tìm hiểu sản phẩm và dịch vụ, và sử dụng internet. Ở mục cuối của chương, ta xem cách phát hành một GPT và một số điểm quan trọng cần cân nhắc.

4.5.3 Phát hành GPT

Vậy là bạn đã hài lòng với GPT và cách nó vận hành, và thấy lợi ích thực sự khi trao nó cho người khác. Phát hành GPT cho công chúng (người đăng ký) sử dụng rất dễ, như trong Hình 4.14. Sau khi chọn tuỳ chọn GPT Store và nhấn Save, bạn sẽ có tuỳ chọn đặt danh mục (category) và cung cấp các liên kết dẫn về bạn.



Hình 4.14: Chọn các tuỳ chọn sau khi nhấn Save để phát hành lên GPT Store

Việc đó rất dễ, nên sau đây là vài điều nữa bạn nên cân nhắc trước khi phát hành GPT:

- *Mô tả GPT*: hãy tạo một mô tả tốt; bạn thậm chí có thể nhờ ChatGPT giúp viết mô tả giúp tăng khả năng tối ưu hoá công cụ tìm kiếm (search engine optimization, SEO) cho GPT. GPT giờ đã xuất hiện trong kết quả tìm kiếm của Google, nên SEO tốt có thể giúp tăng độ hiển thị của assistant. Một mô tả tốt cũng giúp người dùng quyết định có nên dành thời gian dùng assistant của bạn hay không.
- *Logo*: một logo đẹp, gọn gàng, thể hiện được assistant làm gì chắc chắn sẽ giúp ích. Thiết kế logo cho GPT thực chất là dịch vụ miễn phí, nhưng dành thời gian lặp qua vài hình ảnh có thể giúp thu hút người dùng tới assistant của bạn.
- *Danh mục*: theo mặc định, danh mục đã được chọn sẵn, nhưng hãy đảm bảo nó phù hợp với assistant. Nếu thấy không phù hợp, hãy đổi danh mục; bạn thậm chí có thể chọn Other và tự định nghĩa danh mục của mình.
- *Liên kết*: nhớ đặt các liên kết tham chiếu tới mạng xã hội của bạn, và có thể cả một kho GitHub dùng để theo dõi các vấn đề của GPT. Thêm liên kết cho GPT cho người dùng thấy họ có thể liên hệ với người xây dựng nếu gặp vấn đề hoặc có câu hỏi.

Nhiều khả năng sẽ có thêm các yêu cầu khác khi GPT Store trưởng thành. Mô hình kinh doanh vẫn chưa được xác lập, và những bài học khác chắc chắn sẽ tới sau. Dù bạn quyết định xây dựng GPT cho chính mình hay cho người khác, việc này đều giúp bạn hiểu rõ hơn cách xây dựng agent và assistant. Như ta sẽ thấy trong phần còn lại của cuốn sách, GPT assistant là một nền tảng hữu ích cho kiến thức của bạn.

4.6 Bài tập

Hãy hoàn thành các bài tập sau để củng cố kiến thức về nội dung chương:

Bài tập 1: Xây dựng GPT Assistant đầu tiên

Mục tiêu: tạo một GPT assistant đơn giản bằng giao diện ChatGPT.
Nhiệm vụ:

- Đăng ký gói ChatGPT Plus nếu bạn chưa có.
- Vào nền tảng GPT Assistants và nhấn nút Create.
- Làm theo giao diện chat của Builder để tạo assistant Culinary Companion gợi ý món ăn dựa trên nguyên liệu có sẵn.
- Cấu hình assistant thủ công để thêm các quy tắc tuỳ biến cho việc sinh công thức, chẳng hạn kèm thông tin dinh dưỡng và ước tính chi phí.

Bài tập 2: Assistant phân tích dữ liệu

Mục tiêu: phát triển một GPT assistant có thể phân tích file CSV và đưa ra nhận định (insight).
Nhiệm vụ:

- Thiết kế một assistant khoa học dữ liệu có thể nạp và phân tích file CSV, tương tự ví dụ Data Scout trong chương.
- Bật công cụ Code Interpretation và tải lên một file CSV mẫu (ví dụ một bộ dữ liệu từ Kaggle).
- Dùng assistant để thực hiện các nhiệm vụ như làm sạch dữ liệu, trực quan hoá và kiểm định giả thuyết.
- Ghi lại quy trình và kết quả của bạn, nêu các khó khăn hoặc điểm cần cải thiện.

Bài tập 3: Tạo custom action

Mục tiêu: mở rộng một GPT assistant bằng custom action sử dụng một dịch vụ FastAPI.
Nhiệm vụ:

- Làm theo các bước để tạo một dịch vụ FastAPI cung cấp một chức năng cụ thể, chẳng hạn lấy danh sách công việc hằng ngày.
- Sinh đặc tả OpenAPI cho dịch vụ và triển khai nó trên máy cục bộ bằng ngrok.
- Cấu hình một assistant mới dùng custom action này, đảm bảo nó kết nối đúng tới endpoint FastAPI.
- Kiểm thử assistant bằng cách yêu cầu nó thực hiện action và xác minh đầu ra.

Bài tập 4: Knowledge assistant với file upload

Mục tiêu: xây dựng một assistant có kiến thức chuyên biệt từ các tài liệu được tải lên.
Nhiệm vụ:

- Chọn một eBook miễn phí hoặc một bộ tài liệu liên quan tới một chủ đề cụ thể (ví dụ văn học kinh điển, sổ tay kỹ thuật).
- Tải các file này lên một GPT assistant mới, và cấu hình assistant đóng vai chuyên gia về nội dung đã tải lên.
- Tạo một loạt prompt để kiểm tra khả năng của assistant trong việc tham chiếu và tóm tắt thông tin từ tài liệu.
- Đánh giá hiệu quả của assistant và điều chỉnh khi cần để tăng độ chính xác và mức hữu ích.

Bài tập 5: Phát hành và chia sẻ assistant

Mục tiêu: phát hành GPT assistant lên GPT Store và chia sẻ với người khác.
Nhiệm vụ:

- Hoàn thiện cấu hình và kiểm thử assistant để đảm bảo nó hoạt động như mong đợi.
- Viết một mô tả hấp dẫn và tạo logo phù hợp cho assistant.
- Chọn đúng danh mục và thiết lập các liên kết cần thiết tới mạng xã hội hoặc kho GitHub của bạn.
- Phát hành assistant lên GPT Store và chia sẻ liên kết với bạn bè hoặc đồng nghiệp.
- Thu thập phản hồi từ người dùng và tinh chỉnh assistant dựa trên góp ý của họ để cải thiện tính dễ dùng và chức năng.

Tóm tắt

- Nền tảng OpenAI GPT Assistants cho phép xây dựng và triển khai AI agent qua giao diện ChatGPT, tập trung vào việc tạo ra các assistant hấp dẫn và có chức năng thực sự.
- Bạn có thể dùng khả năng thông dịch code của GPT để phân tích dữ liệu trên các file CSV do người dùng tải lên, giúp assistant hoạt động như một nhà khoa học dữ liệu.
- Assistant có thể được mở rộng bằng custom action, cho phép tích hợp với dịch vụ bên ngoài qua các endpoint API. Việc này bao gồm sinh dịch vụ FastAPI và đặc tả OpenAPI tương ứng.

- Assistant có thể được bổ sung kiến thức chuyên biệt qua file upload, cho phép chúng đóng vai nguồn tham khảo đáng tin cậy về các văn bản hay tài liệu cụ thể.
- Thương mại hoá GPT bao gồm việc phát hành nó lên GPT Store, nơi bạn có thể chia sẻ và quảng bá assistant tới nhiều người hơn.
- Xây dựng một assistant có chức năng tốt bao gồm lặp qua các prompt thiết kế, định nghĩa persona rõ ràng, đặt quy tắc, và đảm bảo đầu ra của assistant khớp với kỳ vọng của người dùng.
- Tạo custom action đòi hỏi hiểu và triển khai đặc tả OpenAPI, triển khai dịch vụ cục bộ bằng các công cụ như ngrok, và kết nối các dịch vụ này với assistant.
- Knowledge assistant có thể xử lý nhiều nhiệm vụ, từ tìm kiếm và so sánh tài liệu tới sinh nội dung mới dựa trên kho kiến thức của chúng.
- Phát hành assistant đòi hỏi cân nhắc kỹ việc sử dụng tài nguyên, trải nghiệm người dùng và các yếu tố kinh tế để đảm bảo hiệu quả và tính bền vững khi phục vụ công chúng.
- GPT Store, dành cho người đăng ký ChatGPT Plus, là nền tảng có giá trị để học hỏi và thành thạo việc xây dựng AI assistant, với tiềm năng có cơ hội chia sẻ lợi nhuận trong tương lai.

Chương 5

Khám phá hệ thống đa agent

Chương này bao quát các nội dung:

- Xây dựng hệ thống đa agent bằng AutoGen Studio.
- Xây dựng một hệ thống đa agent đơn giản.
- Tạo các agent có thể cộng tác với nhau qua group chat (trò chuyện nhóm).
- Xây dựng một crew (đội) agent và hệ thống đa agent bằng CrewAI.
- Mở rộng số lượng agent và khám phá các mẫu xử lý (processing pattern) với CrewAI.

Giờ ta hãy đi một hành trình từ AutoGen tới CrewAI, hai nền tảng đa agent (*multi-agent*) đã có chỗ đứng vững. Ta bắt đầu với AutoGen, một dự án của Microsoft hỗ trợ nhiều agent và cung cấp một studio để làm việc với chúng. AutoGen Studio giúp bạn làm quen dần với agent. Từ đó, ta sẽ thực hành viết code nhiều hơn với các agent AutoGen để giải quyết nhiệm vụ bằng hội thoại (conversation) và cộng tác qua group chat.

Sau đó, ta chuyển sang CrewAI, một hệ thống agent tự nhận là dành cho doanh nghiệp (enterprise) và đi theo một hướng khác. CrewAI cân bằng giữa agent dựa trên vai trò (role-based) và agent tự hành (autonomous), có thể quản lý nhiệm vụ linh hoạt theo kiểu tuần tự (sequential) hoặc phân cấp (hierarchical). Ta sẽ xem CrewAI giải quyết các bài toán đa dạng và phức tạp như thế nào.

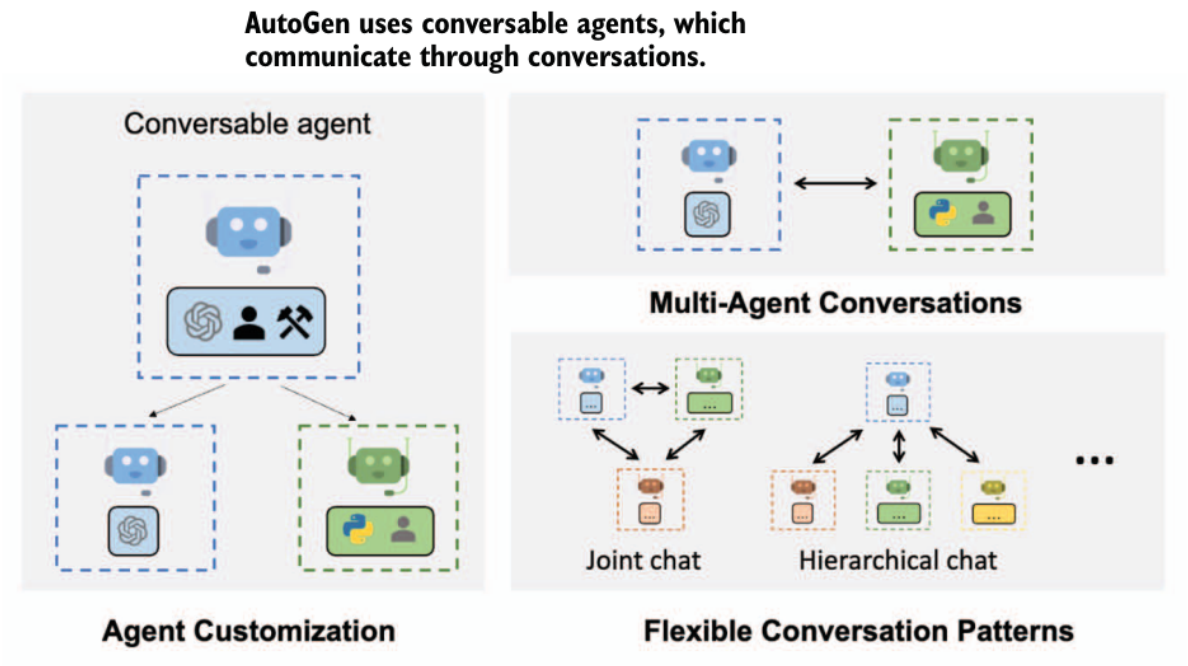
Hệ thống đa agent dùng nhiều công cụ giống như hệ thống một agent, nhưng có thêm lợi thế: các agent có thể đưa phản hồi và đánh giá từ bên ngoài cho agent khác. Khả năng hỗ trợ và phê bình lời giải của agent ngay trong nội bộ hệ thống khiến hệ thống đa agent mạnh hơn. Ta sẽ bắt đầu phần giới thiệu về hệ thống đa agent với AutoGen Studio ở mục tiếp theo.

5.1 Giới thiệu hệ thống đa agent với AutoGen Studio

AutoGen Studio là một công cụ mạnh, phía sau dùng nhiều agent để giải quyết các nhiệm vụ và bài toán mà người dùng giao. Công cụ này đã được dùng để phát triển một số đoạn code phức tạp hơn trong cuốn sách. Vì lý do đó và nhiều lý do khác, nó là phần giới thiệu rất tốt về một hệ thống đa agent thực tế.

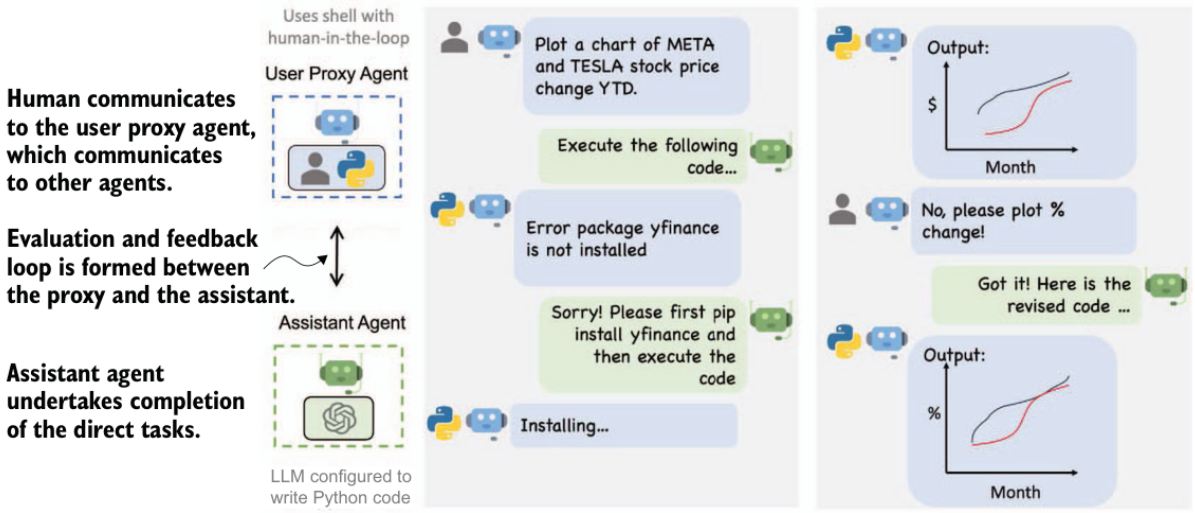
Hình 5.1 là sơ đồ các mẫu kết nối/giao tiếp giữa agent mà AutoGen sử dụng. AutoGen là một nền tảng đa agent **hội thoại** (*conversational*) vì việc giao tiếp được thực hiện bằng ngôn ngữ tự nhiên. Hội thoại bằng ngôn ngữ tự nhiên có vẻ là cách giao tiếp tự nhiên nhất cho agent, nhưng đó không phải cách duy nhất, như bạn sẽ thấy ở phần sau.

AutoGen hỗ trợ nhiều mẫu hội thoại, từ nhóm (group) và phân cấp (hierarchical) cho tới kiểu phổ biến và đơn giản hơn là **giao tiếp qua proxy** (*proxy communication*). Trong giao tiếp qua proxy, một agent đóng vai proxy (người đại diện) và điều hướng việc giao tiếp tới các agent liên quan để hoàn thành nhiệm vụ. Proxy giống như người phục vụ bàn: nhận order và chuyển xuống bếp để nấu, rồi mang món đã nấu ra phục vụ khách.



Hình 5.1: Cách các agent AutoGen giao tiếp qua hội thoại (Nguồn: AutoGen)

Mẫu cơ bản trong AutoGen dùng một `UserProxy` và một hoặc nhiều agent trợ lý (assistant agent). Hình 5.2 cho thấy user proxy nhận chỉ dẫn từ con người, rồi điều khiển một assistant agent có khả năng viết code để thực hiện nhiệm vụ. Mỗi lần assistant hoàn thành một nhiệm vụ, proxy agent sẽ xem xét, đánh giá và đưa phản hồi cho assistant. Vòng lặp này tiếp tục cho đến khi proxy hài lòng với kết quả.



Hình 5.2: Giao tiếp giữa user proxy agent và assistant agent (Nguồn: AutoGen)

Lợi ích của proxy là nó thay thế phần phản hồi và đánh giá vốn cần con người, và trong đa số trường hợp nó làm khá tốt. Proxy không loại bỏ hoàn toàn nhu cầu phản hồi và đánh giá từ con người, nhưng nhìn chung cho ra kết quả hoàn chỉnh hơn nhiều. Vòng lặp này tốn thời gian, nhưng đó là khoảng thời gian bạn có thể ngồi uống cà phê hoặc làm việc khác.

AutoGen Studio là công cụ do nhóm AutoGen phát triển, giúp làm quen với các agent có khả năng hội thoại (conversable agent). Trong bài thực hành tiếp theo, ta sẽ cài Studio và chạy vài thí nghiệm để xem nền tảng này hoạt động tốt đến đâu. Các công cụ này vẫn đang trong chu kỳ phát triển nhanh, nên

nếu gặp vấn đề, hãy tham khảo tài liệu trên kho GitHub của AutoGen.

5.1.1 Cài đặt và sử dụng AutoGen Studio

Mở thư mục `chapter_04` trong Visual Studio Code (VS Code), tạo một môi trường ảo Python cục bộ, và cài các package trong file `requirements.txt`. Nếu cần trợ giúp, hãy xem phụ lục B để cài toàn bộ yêu cầu cho các bài thực hành của chương này.

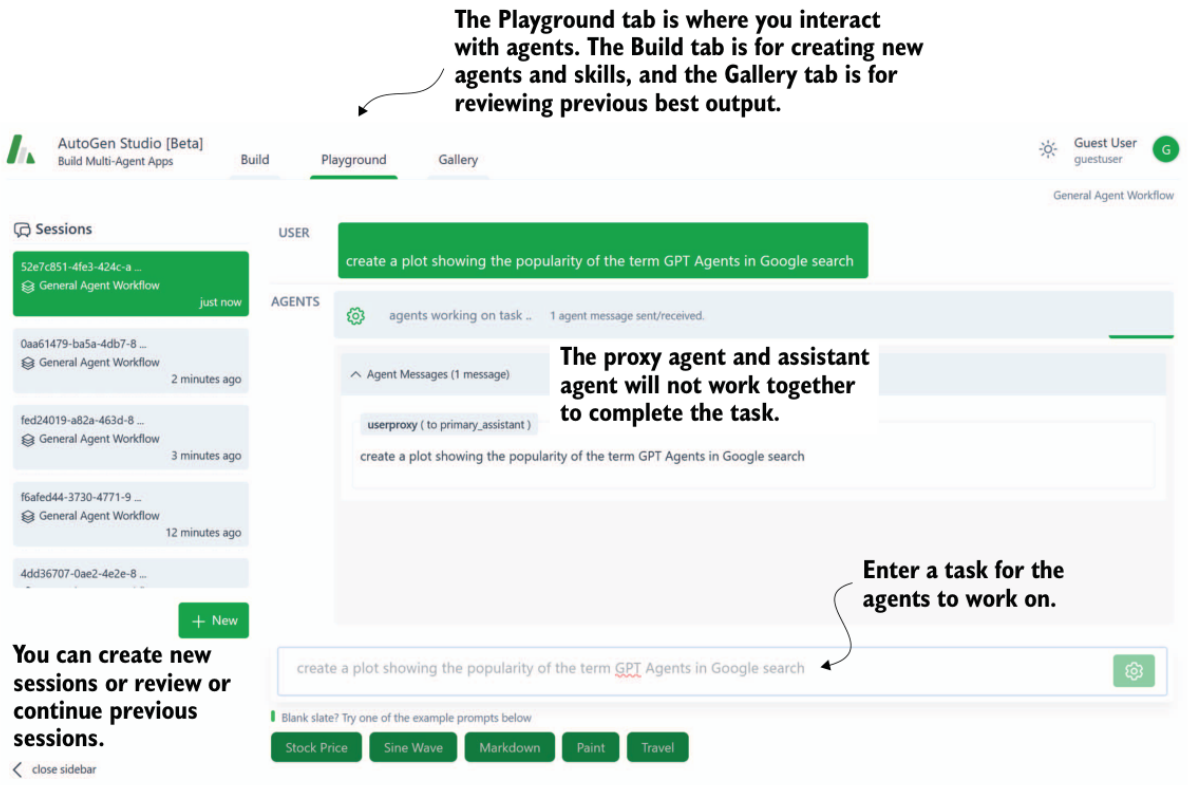
Mở terminal trong VS Code (`Ctrl-`, `Cmd-`) trỏ tới môi trường ảo của bạn, và chạy AutoGen Studio bằng lệnh trong Listing 4.1. Trước tiên, bạn cần định nghĩa biến môi trường chứa OpenAI key. Vì cổng 8080 và 8081 khá phổ biến, nếu đang chạy các dịch vụ khác, hãy đổi cổng thành 8082 hoặc một cổng tuỳ chọn.

Listing 4.1 Khởi chạy AutoGen Studio

```
# set environment variable on Bash (Git Bash)
export OPENAI_API_KEY=”<your API key>”         # Dùng lệnh phù hợp với loại
↪  terminal của bạn.

# sent environment variable with PowerShell
$env:VAR_NAME =”<your API key>"                # Dùng lệnh phù hợp với loại
↪  terminal của bạn.

autogenstudio ui --port 8081    # Đổi cổng nếu bạn dự đoán hoặc gặp xung đột trên
↪  máy.
```

Mở trình duyệt tới giao diện AutoGen Studio như trong Hình 5.3 (tại thời điểm viết sách). Có thể sẽ có khác biệt, nhưng chắc chắn một điều: giao diện chính vẫn sẽ là chat. Hãy nhập một nhiệm vụ phức tạp cần viết code. Ví dụ dùng ở đây là:

```
Create a plot showing the popularity of the term GPT Agents in Google search.
```



Hình 5.3: Nhập một nhiệm vụ để các agent xử lý trong giao diện AutoGen

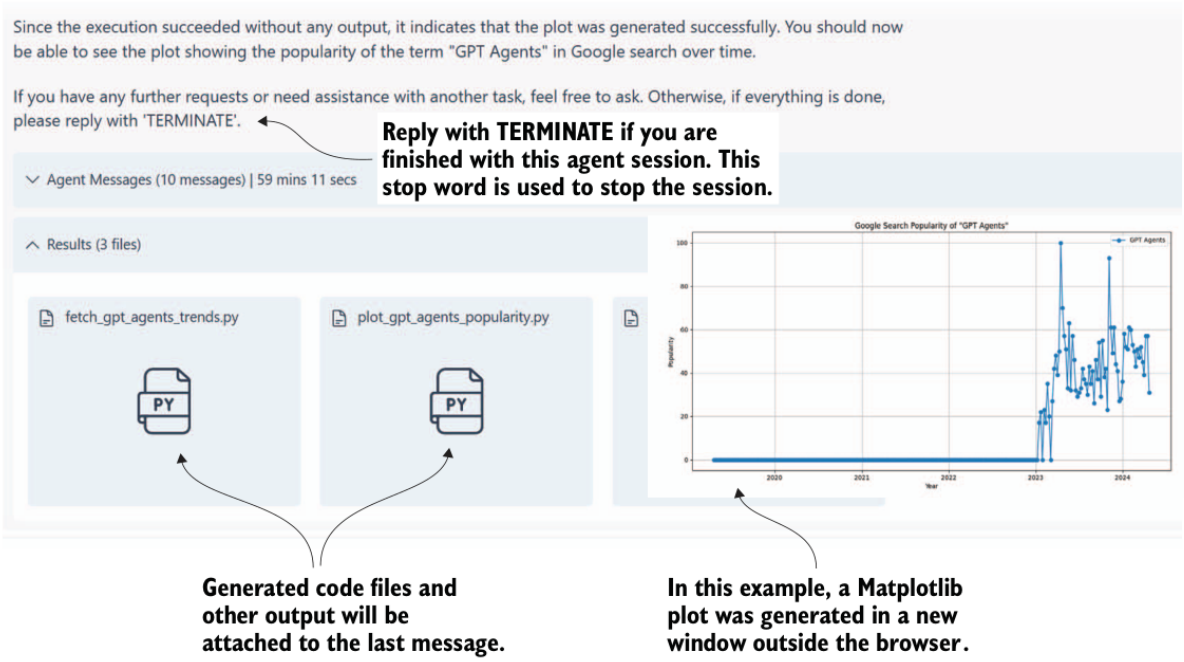
Trong ví dụ này, khi các agent cùng nhau xử lý nhiệm vụ, assistant agent sinh ra các đoạn code để thực hiện hoặc hoàn thành các nhiệm vụ con. Sau đó user proxy agent thử thực thi các đoạn code đó và đánh giá đầu ra. Trong nhiều trường hợp, chỉ cần chứng minh được code chạy và cho ra đầu ra mong muốn là đủ để user proxy agent chấp nhận nhiệm vụ đã hoàn thành.

Nếu gặp vấn đề với các yêu cầu của assistant agent, hãy bảo proxy agent thử một cách khác hoặc một bài toán khác. Điều này làm lộ ra một vấn đề lớn hơn của các hệ thống agent: chúng dùng những package hay thư viện đã lỗi thời và không còn chạy được. Vì lý do này, nhìn chung nên để agent thực thi hành động (action) thay vì để agent tự viết code làm công cụ thực hiện hành động.

Mẹo

Nên chạy AutoGen và AutoGen Studio bằng Docker, đặc biệt khi làm việc với code có thể ảnh hưởng tới hệ điều hành. Docker có thể cô lập và ảo hoá môi trường của agent, nhờ đó cô lập những đoạn code có khả năng gây hại. Dùng Docker cũng giúp tránh các cửa sổ phụ hay trang web có thể chặn tiến trình của agent.

Hình 5.4 cho thấy agent đã hoàn thành nhiệm vụ. Proxy agent sẽ thu thập mọi đoạn code, hình ảnh hoặc tài liệu khác đã sinh ra và đính kèm chúng vào message.



Hình 5.4: Kết quả sau khi các agent hoàn thành nhiệm vụ

Bạn cũng có thể xem lại hội thoại giữa các agent bằng cách mở mục mở rộng Agent Messages. Trong nhiều trường hợp, nếu bạn yêu cầu agent tạo biểu đồ hay ứng dụng, các cửa sổ phụ sẽ mở ra để hiển thị kết quả.

Thật đáng kinh ngạc, các agent thực hiện phần lớn nhiệm vụ khá trơn tru và hoàn thành tốt. Tuỳ độ phức tạp của nhiệm vụ, bạn có thể cần lặp thêm với proxy. Đôi khi agent chỉ làm được đến một mức nhất định vì thiếu kỹ năng (skill) cần thiết. Ở mục tiếp theo, ta sẽ xem cách thêm skill cho agent.

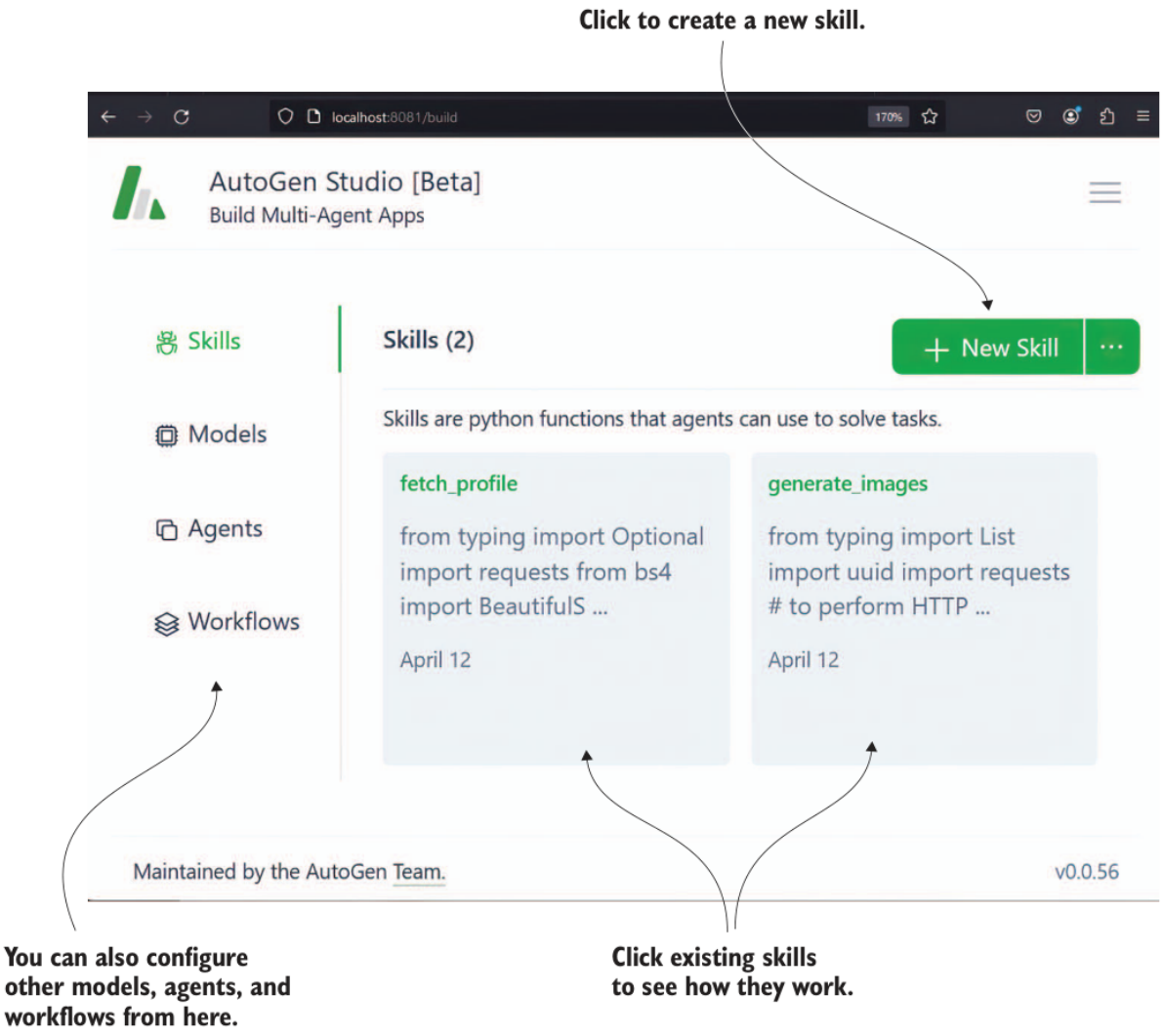
5.1.2 Thêm skill trong AutoGen Studio

Skill và tool (công cụ), hay *action* (hành động) như cách gọi trong cuốn sách này, là phương tiện chính để agent tự mở rộng khả năng. Action cho agent khả năng thực thi code, gọi API, hoặc đánh giá và kiểm tra thêm đầu ra đã sinh. Hiện tại AutoGen Studio chỉ có sẵn một bộ công cụ cơ bản để lấy nội dung web hoặc sinh ảnh.

> **Ghi chú**
>
> Nhiều hệ thống agent áp dụng cách cho phép agent tự viết code để đạt mục tiêu. Tuy nhiên, chúng tôi nhận thấy code rất dễ hỏng, cần được bảo trì và có thể thay đổi nhanh. Vì vậy, như sẽ bàn ở các chương sau, tốt hơn là cung cấp cho agent các skill/action/tool để giải quyết vấn đề.

Trong kịch bản thực hành sau, ta sẽ thêm một skill/action để kiểm tra ảnh bằng mô hình vision (thị giác) của OpenAI. Skill này cho phép proxy agent đưa phản hồi nếu ta yêu cầu assistant sinh một ảnh có nội dung cụ thể.

Khi AutoGen Studio đang chạy, vào tab Build và nhấn Skills, như trong Hình 5.5. Sau đó nhấn nút New Skill để mở một khung code, nơi bạn có thể sao chép và dán code vào. Từ tab này, bạn cũng có thể cấu hình mô hình, agent và agent workflow (quy trình làm việc của agent).



Hình 5.5: Các bước tạo một skill mới trên tab Build

Nhập đoạn code trong Listing 4.2, cũng có trong mã nguồn của sách dưới tên `describe_image.py`. Sao chép và dán code này vào cửa sổ soạn thảo, rồi nhấn nút Save ở phía dưới.

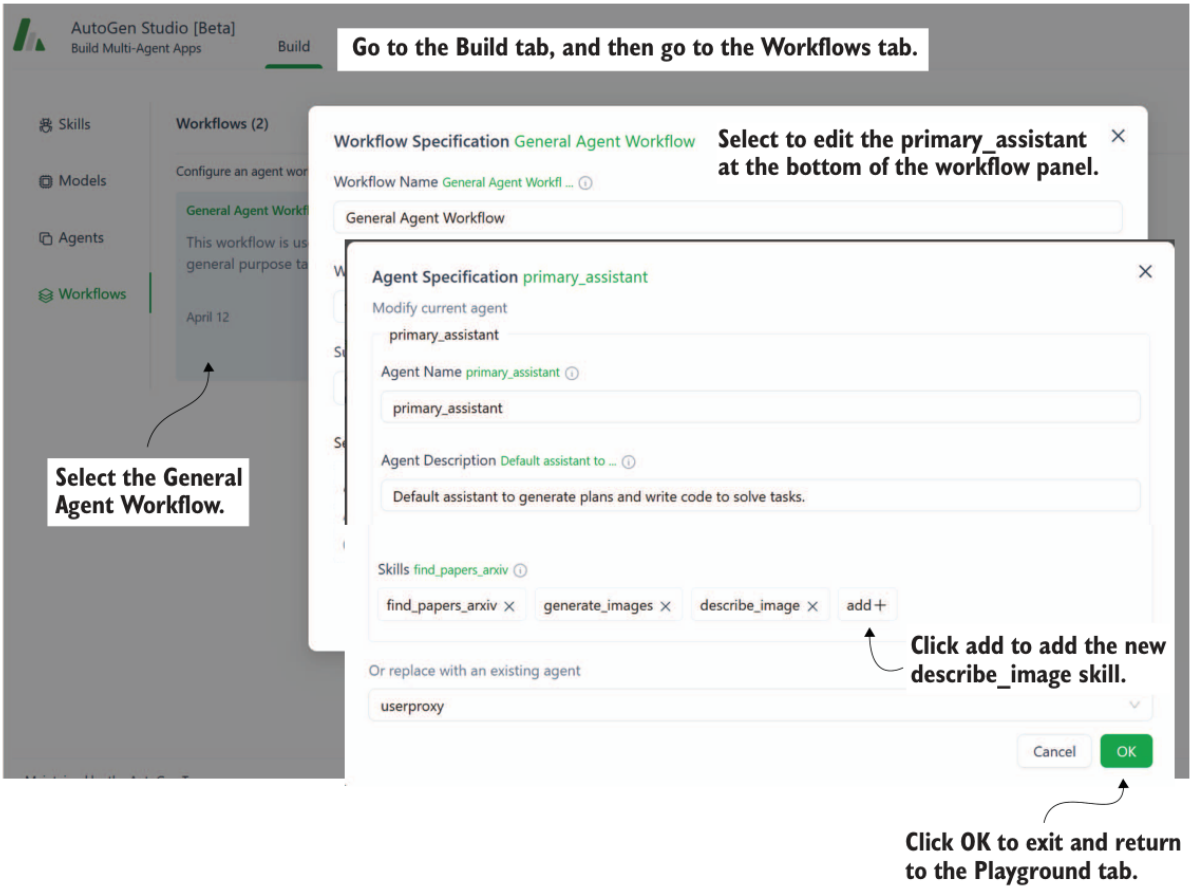
Listing 4.2 describe_image.py

```
import base64
import requests
import os

def describe_image(image_path='animals.png') -> str:
    """
    Uses GPT-4 Vision to inspect and describe the contents of the image.

    :param input_path: str, the name of the PNG file to describe.
    """
    api_key = os.environ['OPEN_API_KEY']

    # Function to encode the image
    def encode_image(image_path):     # Hàm nạp ảnh và mã hoá thành chuỗi Base64
        with open(image_path, "rb") as image_file:
            return base64.b64encode(image_file.read()).decode('utf-8')
```

```
    # Getting the base64 string
    base64_image = encode_image(image_path)

    headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
    }

    payload = {
    "model": "gpt-4-turbo",
    "messages": [
        {
        "role": "user",
        "content": [
            {
            "type": "text",
            "text": "What's in this image?"
            },
            {
            "type": "image_url",
            "image_url": {
         "url": f"{base64_image}"     # Đưa chuỗi ảnh vào
         ↪  cùng JSON payload
            }
            }
        ]
        }
    ],
    "max_tokens": 300
    }

    response = requests.post(
        "https://api.openai.com/v1/chat/completions",
        headers=headers,
        json=payload)

    return response.json()["choices"][0]["message"]
["content"]                                          # Giải nén phản hồi và trả về
↪  nội dung câu trả lời
```

Hàm `describe_image` dùng mô hình vision GPT-4 của OpenAI để mô tả nội dung trong ảnh. Skill này có thể đi cặp với skill `generate_image` có sẵn để làm bước đánh giá chất lượng: các agent có thể xác nhận ảnh đã sinh khớp với yêu cầu của người dùng.

Sau khi thêm skill, bạn phải gán nó cho agent workflow và agent cụ thể thì mới dùng được. Hình 5.6 minh hoạ việc thêm skill mới cho primary assistant agent (agent trợ lý chính) trong agent workflow chung, tức workflow mặc định.



Hình 5.6: Cấu hình agent `primary_assistant` với skill mới

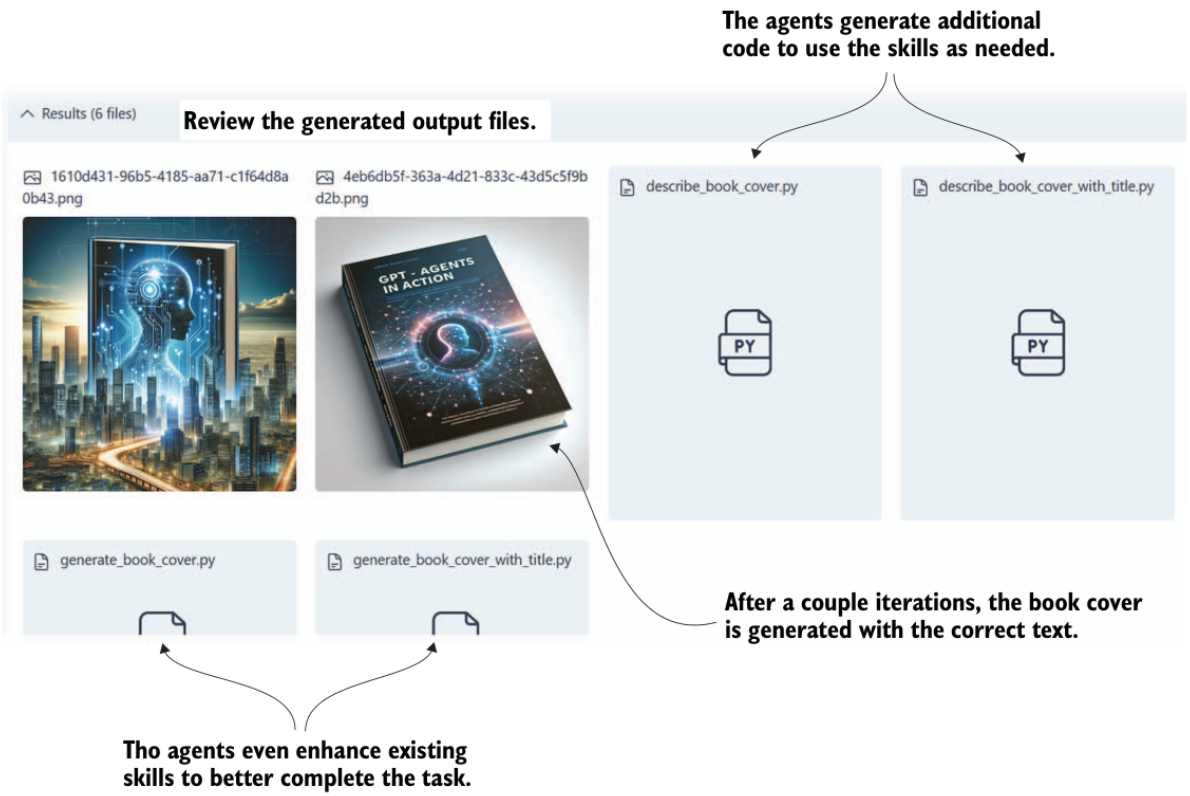
Giờ skill đã được thêm cho primary assistant, ta có thể giao cho agent nhiệm vụ tạo một ảnh cụ thể và kiểm tra ảnh đó bằng skill `describe_image` mới. Vì các trình sinh ảnh nổi tiếng là khó viết chữ đúng, ta sẽ tạo một bài thực hành đúng với việc đó.

Nhập đoạn văn bản trong Listing 4.3 để yêu cầu các agent tạo ảnh bìa cho cuốn sách này. Ta nói rõ rằng chữ phải chính xác và yêu cầu agent dùng hàm `describe_image` mới để kiểm tra ảnh.

Listing 4.3 Prompt yêu cầu tạo ảnh bìa sách

```
Please create a cover for the book GPT Agents In Action, use the
describe_image skill to make sure the title of the book is spelled
correctly on the cover
```

Sau khi nhập prompt, hãy chờ một lúc, bạn có thể thấy các trao đổi qua lại về quá trình sinh và kiểm tra ảnh. Cuối cùng, nếu mọi thứ chạy đúng, các agent sẽ trả về kết quả như trong Hình 5.7.



Hình 5.7: Các file đầu ra được sinh từ quá trình agent làm nhiệm vụ sinh ảnh

Đáng chú ý là các agent phối hợp hoàn thành nhiệm vụ chỉ sau vài vòng lặp. Ngoài ảnh, bạn còn thấy các đoạn code hỗ trợ được sinh ra để giúp hoàn thành nhiệm vụ. AutoGen Studio rất ấn tượng ở khả năng tích hợp các skill mà agent có thể tiếp tục điều chỉnh để đạt mục tiêu. Mục tiếp theo sẽ cho thấy các agent mạnh mẽ này được hiện thực bằng code ra sao.

5.2 Khám phá AutoGen

AutoGen Studio là công cụ tuyệt vời để hiểu hệ thống đa agent, nhưng ta cần xem xét code. May mắn là viết các ví dụ nhiều agent với AutoGen khá đơn giản và dễ chạy. Ta sẽ xem phần thiết lập AutoGen cơ bản ở tiểu mục tiếp theo.

5.2.1 Cài đặt và sử dụng AutoGen

Bài thực hành tiếp theo viết code cho một hệ thống đa agent cơ bản gồm một user proxy và một conversable agent. Nhưng trước đó, ta cần đảm bảo AutoGen đã được cài và cấu hình đúng.

Mở terminal trong VS Code và chạy toàn bộ hướng dẫn cài đặt của chương 4 theo phụ lục B, hoặc chạy lệnh `pip` trong Listing 4.4. Nếu bạn đã cài file `requirements.txt`, bạn cũng đã sẵn sàng chạy AutoGen.

Listing 4.4 Cài đặt AutoGen

```
pip install pyautogen
```

Tiếp theo, sao chép `chapter_04/OAI_CONFIG_LIST.example` thành `OAI_CONFIG_LIST`, tức bỏ phần `.example` khỏi tên file. Sau đó mở file mới trong VS Code và nhập cấu hình OpenAI hoặc Azure vào file `OAI_CONFIG_LIST` như trong Listing 4.5. Điền API key, mô hình và các thông tin khác

theo yêu cầu của dịch vụ API bạn dùng. AutoGen làm việc được với mọi mô hình tuân theo OpenAI client. Điều đó nghĩa là bạn có thể dùng LLM cục bộ qua LM Studio hoặc các dịch vụ khác như Groq, Hugging Face, v.v.

Listing 4.5 OAI_CONFIG_LIST

```
[
    {
        "model": "gpt-4",                    // Chọn mô hình; nên dùng GPT-4.
        "api_key": "<your OpenAI API key here>",           // Dùng service key
        ↪ bạn vẫn thường dùng.
        "tags": ["gpt-4", "tool"]
    },
    {
        "model": "<your Azure OpenAI deployment name>",     // Chọn mô hình; nên
        ↪ dùng GPT-4.
        "api_key": "<your Azure OpenAI API key here>",     // Dùng service key
        ↪ bạn vẫn thường dùng.
        "base_url": "<your Azure OpenAI API base here>",    // Đổi base URL để
        ↪ trỏ tới dịch vụ khác, không chỉ Azure OpenAI.
        "api_type": "azure",
        "api_version": "2024-02-15-preview"
    }
]
```

Giờ ta có thể xem code cho một cuộc chat đa agent cơ bản, dùng các agent dựng sẵn `UserProxy` và `ConversableAgent`. Mở file `autogen_start.py` trong VS Code, như trong listing sau, và xem qua các phần trước khi chạy file.

Listing 4.6 autogen_start.py

```
from autogen import ConversableAgent, UserProxyAgent, config_list_from_json

   config_list = config_list_from_json(
        env_or_file="OAI_CONFIG_LIST")     # Nạp cấu hình LLM từ file JSON
        ↪ OAI_CONFIG_LIST

   assistant = ConversableAgent(
        "agent",
        llm_config={"config_list": config_list})     # Agent này nói chuyện trực
        ↪ tiếp với LLM.

   user_proxy = UserProxyAgent(     # Agent này làm proxy chuyển hội thoại từ
   ↪ người dùng tới assistant.
        "user",
        code_execution_config={
            "work_dir": "working",
            "use_docker": False,
        },
        human_input_mode="ALWAYS",
        is_termination_msg=lambda x: x.get("content", "")
        .rstrip()
        .endswith("TERMINATE"),     # Đặt message kết thúc để agent có thể lặp.
    )
    user_proxy.initiate_chat(assistant, message="write a solution for fizz buzz
    ↪ in one line?")    # Khởi tạo chat với assistant qua user_proxy để hoàn
    ↪ thành nhiệm vụ.
```

Chạy code bằng cách chạy file trong trình debug của VS Code (F5). Code trong Listing 4.6 dùng một nhiệm vụ đơn giản để minh hoạ việc viết code. Listing 4.7 đưa ra vài ví dụ để bạn chọn. Các nhiệm vụ lập trình này cũng là một số bài kiểm tra chuẩn (baseline) mà tác giả thường dùng để đánh giá khả năng lập trình của một LLM.

Listing 4.7 Ví dụ các nhiệm vụ lập trình đơn giản

```
write a Python function to check if a number is prime
code a classic sname game using Pygame
code a classic asteroids game in Python using Pygame
```

Chú thích cho Listing 4.7:

- Hai dòng cuối (game snake và asteroids): để thoải mái lặp đi lặp lại với các nhiệm vụ này, hãy dùng Windows Subsystem for Linux (WSL) trên Windows, hoặc dùng Docker.

Vài giây sau khi code khởi chạy, assistant sẽ trả lời proxy bằng một lời giải. Lúc này, proxy sẽ hỏi bạn phản hồi. Hãy nhấn Enter, tức là không đưa phản hồi gì, và việc này sẽ khiến proxy chạy code để kiểm tra code hoạt động như mong đợi.

Ấn tượng là proxy agent còn biết tự cài các package cần thiết, chẳng hạn Pygame. Sau đó nó chạy code, và bạn sẽ thấy đầu ra trong terminal, hoặc ở một cửa sổ hay trình duyệt mới. Bạn có thể chơi game hoặc dùng giao diện nếu code mở ra một cửa sổ/trình duyệt mới.

Lưu ý rằng trên Windows, cửa sổ/trình duyệt được mở ra sẽ không tự đóng và bạn phải thoát toàn bộ chương trình. Để tránh vấn đề này, hãy chạy code qua Windows Subsystem for Linux (WSL) hoặc Docker. AutoGen khuyến nghị rõ ràng việc dùng Docker cho các agent thực thi code, và nếu bạn quen dùng container thì đây là lựa chọn tốt.

Dù cách nào, sau khi proxy sinh và chạy code, thư mục `working_dir` đặt ở Listing 4.6 sẽ có một file Python chứa code. Nhờ đó, bạn có thể chạy code bất cứ lúc nào, sửa đổi, hoặc thậm chí yêu cầu cải tiến, như ta sẽ thấy. Ở tiểu mục tiếp theo, ta sẽ xem cách nâng cao khả năng của các agent lập trình.

5.2.2 Cải thiện code đầu ra bằng agent phê bình

Một lợi ích lớn của hệ thống đa agent là có thể tự động gán nhiều vai trò/persona (nhân cách) khi thực hiện nhiệm vụ. Việc sinh hoặc hỗ trợ viết code là lợi thế lớn cho bất kỳ lập trình viên nào, nhưng sẽ thế nào nếu code đó còn được review (xem xét) và kiểm thử? Trong bài thực hành tiếp theo, ta sẽ thêm một agent phê bình (critic) vào hệ thống agent để hỗ trợ các nhiệm vụ lập trình. Mở `autogen_coding_critic.py`, như trong listing sau.

Listing 4.8 autogen_coding_critic.py

```
from autogen import AssistantAgent, UserProxyAgent, config_list_from_json

config_list = config_list_from_json(env_or_file="OAI_CONFIG_LIST")

user_proxy = UserProxyAgent(
    "user",
    code_execution_config={
        "work_dir": "working",
        "use_docker": False,
        "last_n_messages": 1,
    },
    human_input_mode="ALWAYS",
    is_termination_msg=lambda x:
x.get("content", "").rstrip().endswith("TERMINATE"),
)
```

```
engineer = AssistantAgent(
    name="Engineer",
    llm_config={"config_list": config_list},
    system_message="""
    You are a profession Python engineer, known for your expertise in
software development.
    You use your skills to create software applications, tools, and
games that are both functional and efficient.
    Your preference is to write clean, well-structured code that is easy
to read and maintain.
    """,     # Lần này assistant được cấp một system message/persona.
)

critic = AssistantAgent(
    name="Reviewer",
    llm_config={"config_list": config_list},
    system_message="""
    You are a code reviewer, known for your thoroughness and commitment
to standards.
    Your task is to scrutinize code content for any harmful or
substandard elements.
    You ensure that the code is secure, efficient, and adheres to best
practices.
    You will identify any issues or areas for improvement in the code
and output them as a list.
    """,     # Tạo agent phê bình (assistant thứ hai) kèm lý lịch (background)
    ↪  riêng.
)

def review_code(recipient, messages, sender, config):     # Hàm tuỳ biến giúp
↪  trích code để critic review.
    return f"""
            Review and critque the following code.

            {recipient.chat_messages_for_summary(sender)[-1]['content']}
            """

user_proxy.register_nested_chats(     # Tạo nested chat (chat lồng) giữa critic
↪  và engineer.
    [
        {
            "recipient": critic,
            "message": review_code,
            "summary_method": "last_msg",
            "max_turns": 1,
        }
    ],
    trigger=engineer,
)

task = """Write a snake game using Pygame."""

res = user_proxy.initiate_chat(
    recipient=engineer,
    message=task,
    max_turns=2,
    summary_method="last_msg"     # Proxy agent khởi tạo chat với số lượt tối đa
    ↪  và phương thức tóm tắt tường minh.
```

```
)
```

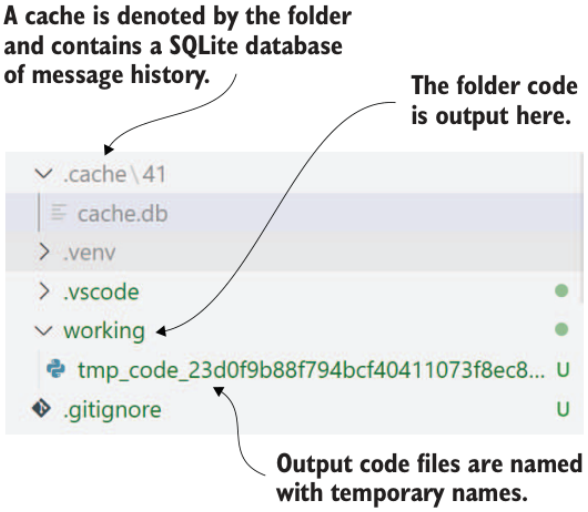
Chạy file `autogen_coding_critic.py` trong VS Code ở chế độ debug và quan sát hội thoại giữa các agent. Lần này, sau khi code được trả về, critic cũng sẽ được kích hoạt để phản hồi. Sau đó critic sẽ thêm nhận xét và gợi ý để cải thiện code.

Nested chat (chat lồng nhau) hoạt động tốt để hỗ trợ và kiểm soát tương tác giữa các agent, nhưng ta sẽ thấy một cách tốt hơn ở mục sau. Trước đó, ta sẽ xem tầm quan trọng của cache trong AutoGen ở tiểu mục tiếp theo.

5.2.3 Tìm hiểu cache của AutoGen

Là một nền tảng đa agent dạng hội thoại, AutoGen có thể tiêu tốn rất nhiều token qua các vòng chat. Nếu bạn yêu cầu AutoGen xử lý các bài toán phức tạp hoặc mới lạ, bạn thậm chí có thể chạm giới hạn token của LLM; vì vậy AutoGen hỗ trợ một số cách để giảm lượng token sử dụng.

AutoGen dùng cơ chế **cache** (bộ nhớ đệm) để lưu tiến trình và giảm lượng token sử dụng. Cache được bật mặc định, và có thể bạn đã gặp nó. Nếu kiểm tra thư mục làm việc hiện tại, bạn sẽ thấy một thư mục `.cache`, như trong Hình 5.8. Cache cho phép agent tiếp tục hội thoại nếu bị gián đoạn.



Hình 5.8: Thư mục cache và thư mục làm việc của AutoGen

Trong code, bạn có thể kiểm soát thư mục cache cho mỗi lần chạy agent, như trong Listing 4.9. Bằng cách bọc lời gọi `initiate_chat` trong câu lệnh `with`, bạn có thể kiểm soát vị trí và seed (hạt giống) của cache. Nhờ vậy, bạn có thể lưu lại và quay về các nhiệm vụ AutoGen chạy lâu trong tương lai, chỉ bằng cách đặt `cache_seed` của cache trước đó.

Listing 4.9 Đặt thư mục cache

```
with Cache.disk(cache_seed=42) as cache:    # Giá trị seed_cache xác định vị trí
↪  riêng của cache.
    res = user_proxy.initiate_chat(
        recipient=engineer,
        message=task,
        max_turns=2,
        summary_method="last_msg",
        cache=cache,     # Truyền cache vào như một tham số
    )
```

Khả năng cache này cho phép bạn tiếp tục thao tác từ vị trí cache trước đó và lưu lại các lần chạy trước. Đây cũng là cách rất hay để minh hoạ và kiểm tra cách một cuộc hội thoại giữa các agent đã tạo ra kết quả. Ở mục tiếp theo, ta sẽ xem một mẫu hội thoại khác mà AutoGen hỗ trợ: group chat.

5.3 Group chat với agent và AutoGen

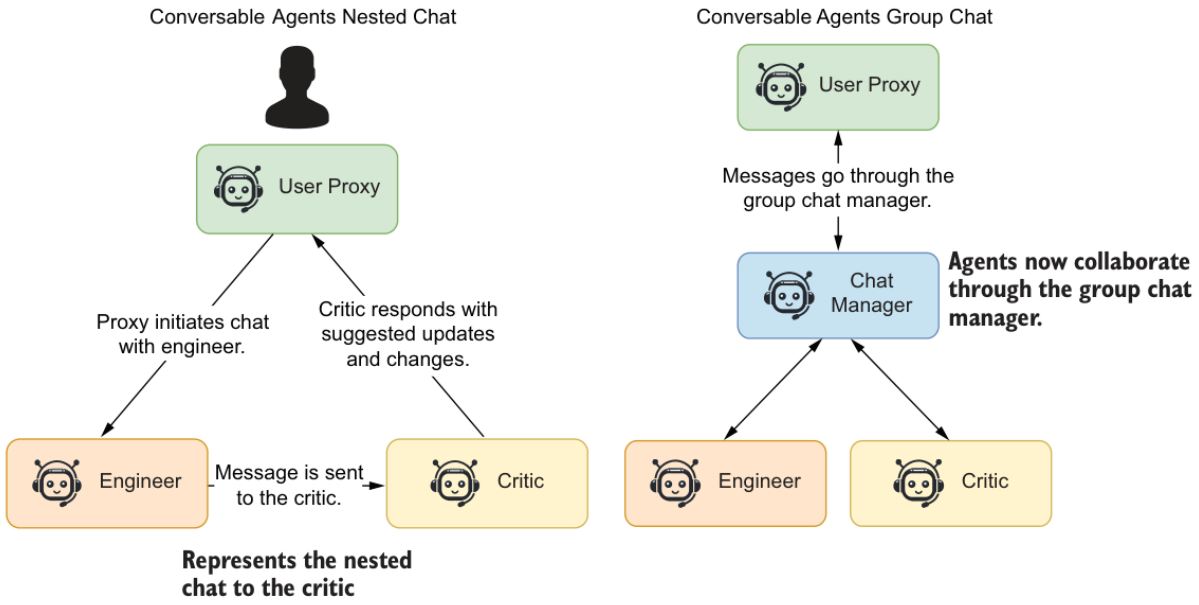
Một vấn đề của việc ủy thác chat (chat delegation) và nested chat hay hội thoại lồng nhau là việc truyền đạt thông tin. Nếu bạn từng chơi trò "tam sao thất bản" (telephone game), bạn đã tận mắt thấy thông tin có thể thay đổi nhanh thế nào qua các lượt truyền. Với agent cũng vậy, việc chat qua các hội thoại lồng nhau hay tuần tự có thể làm thay đổi nhiệm vụ, thậm chí cả kết quả mong muốn.

Trò chơi telephone (tam sao thất bản)

Trò chơi telephone là trò chơi vui nhưng mang tính giáo dục, minh hoạ việc mất mát thông tin và mất tính mạch lạc. Trẻ em xếp thành một hàng, đứa trẻ đầu tiên nhận một thông điệp mà chỉ mình nó nghe thấy. Rồi lần lượt từng đứa nói thầm thông điệp cho đứa kế tiếp, cứ thế tiếp tục. Cuối cùng, đứa trẻ cuối hàng công bố thông điệp cho cả nhóm, và thông điệp đó thường chẳng giống chút nào với thông điệp ban đầu.

Để khắc phục điều này, AutoGen cung cấp **group chat**, một cơ chế cho phép các agent tham gia vào một cuộc hội thoại chung. Nhờ đó, agent có thể xem lại toàn bộ hội thoại trước đó và cộng tác tốt hơn trong các nhiệm vụ dài và phức tạp.

Hình 5.9 cho thấy sự khác biệt giữa nested chat và group chat cộng tác. Ở mục trước ta đã dùng tính năng nested chat để xây dựng một cuộc chat agent lồng nhau. Trong mục này, ta dùng group chat để có trải nghiệm cộng tác hơn.



Hình 5.9: Khác biệt giữa nested chat và group chat đối với các conversable agent

Mở `autogen_coding_group.py` với các phần liên quan, như trong Listing 4.10. Code tương tự bài thực hành trước, nhưng giờ có thêm `GroupChat` và `GroupChatManager`. Các agent và message được giữ trong group chat, tương tự một kênh nhắn tin trong các ứng dụng như Slack hay Discord. Chat manager (người quản lý cuộc chat) điều phối các phản hồi message để giảm chồng chéo trong hội thoại.

Listing 4.10 autoget_coding_group.py (các phần liên quan)

```
user_proxy = UserProxyAgent(
    "user",
    code_execution_config={
        "work_dir": "working",
        "use_docker": False,
        "last_n_messages": 3,
    },
    human_input_mode="NEVER",    # Giờ đặt human input là NEVER, nên không có
    ↪  phản hồi từ con người.
)

llm_config = {"config_list": config_list}

engineer = AssistantAgent(…     # Code được lược bỏ; xem các thay đổi về persona
↪  trong file

critic = AssistantAgent(…       # Code được lược bỏ; xem các thay đổi về persona
↪  trong file

groupchat = GroupChat(agents=[user_proxy,
                              engineer,
                              critic],
                              messages=[],
                              max_round=20)     # Đối tượng giữ kết nối tới tất
                              ↪  cả agent và lưu các message.
manager = GroupChatManager(groupchat=groupchat,
                           llm_config=llm_config)    # Manager điều phối cuộc hội
                           ↪  thoại như một người điều phối (moderator).

task = """Write a snake game using Pygame."""

with Cache.disk(cache_seed=43) as cache:
    res = user_proxy.initiate_chat(
        recipient=manager,
        message=task,
        cache=cache,
    )
```

Chạy bài thực hành này, bạn sẽ thấy các agent cộng tác như thế nào. Engineer giờ sẽ nhận phản hồi từ critic và thực hiện các thao tác để xử lý các góp ý của critic. Cách này cũng cho phép proxy tham gia vào toàn bộ cuộc hội thoại.

Hội thoại nhóm là cách tuyệt vời để tăng cường năng lực của agent khi chúng cộng tác trong nhiệm vụ. Tuy nhiên, chúng cũng dài dòng hơn đáng kể và tốn token hơn. Dĩ nhiên, khi LLM trưởng thành hơn, kích thước cửa sổ context token và giá xử lý token cũng thay đổi theo. Khi cửa sổ token tăng lên, mối lo về lượng token tiêu thụ có thể dần biến mất.

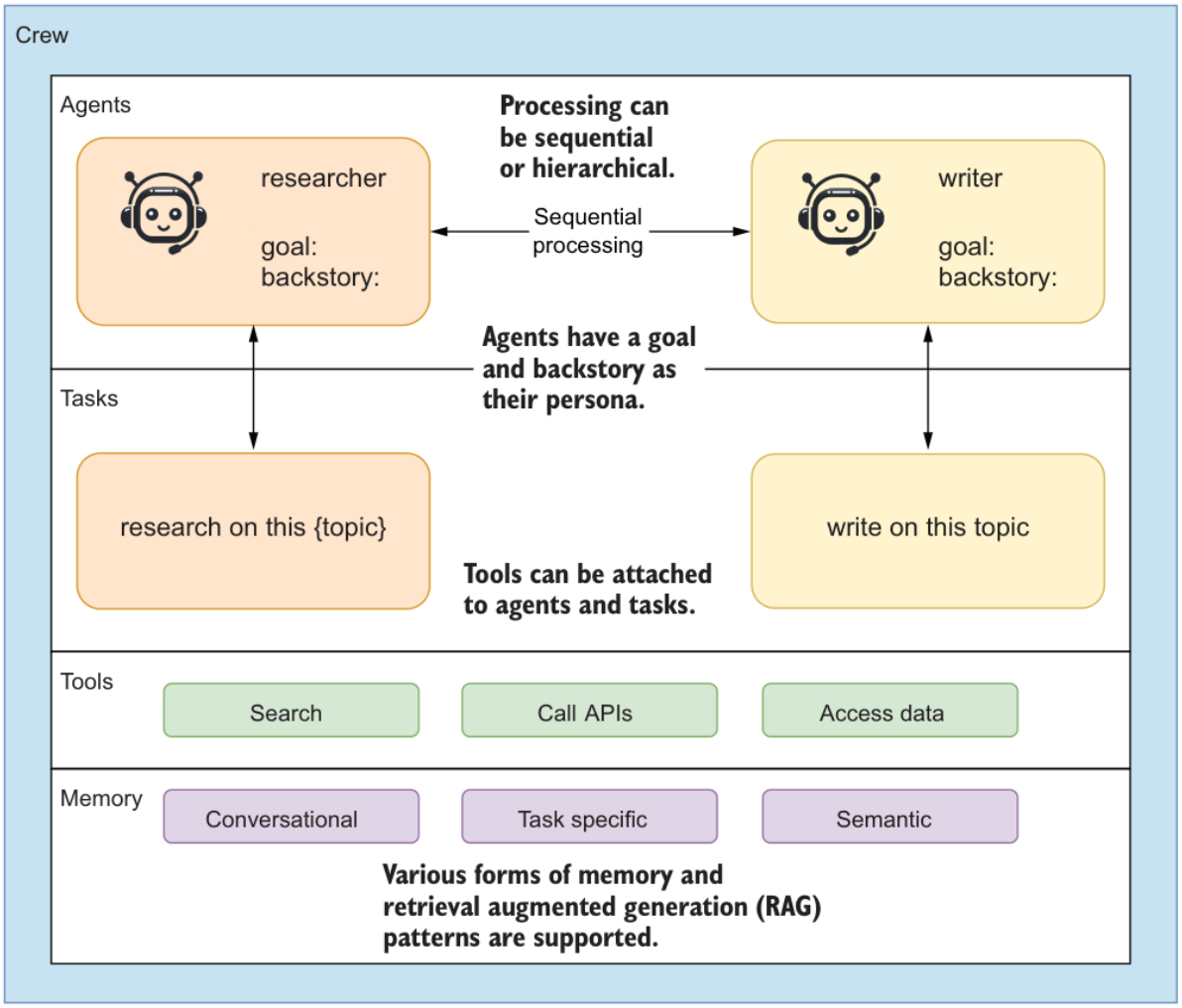
AutoGen là một nền tảng đa agent mạnh, có thể trải nghiệm qua giao diện web hoặc code. Dù bạn thích cách nào, công cụ cộng tác agent này là nền tảng rất tốt để viết code hoặc làm các nhiệm vụ phức tạp khác. Dĩ nhiên, nó không phải nền tảng duy nhất, như bạn sẽ thấy ở mục tiếp theo, nơi ta khám phá một người mới tên là CrewAI.

5.4 Xây dựng crew agent với CrewAI

CrewAI tương đối mới trong lĩnh vực hệ thống đa agent. Trong khi AutoGen ban đầu được phát triển từ nghiên cứu rồi mới mở rộng, CrewAI được xây dựng với định hướng hệ thống doanh nghiệp. Vì thế, nền tảng này vững chắc hơn, nhưng đổi lại kém linh hoạt khi mở rộng ở một số khía cạnh.

Với CrewAI, bạn xây dựng một **crew** (đội) gồm các agent, mỗi agent tập trung vào một khía cạnh cụ thể của mục tiêu nhiệm vụ. Khác với AutoGen, CrewAI không bắt buộc dùng user proxy agent mà mặc định các agent chỉ làm việc với nhau.

Hình 5.10 cho thấy các thành phần chính của nền tảng CrewAI, cách chúng kết nối với nhau và chức năng chính của từng thành phần. Hình thể hiện một hệ thống agent xử lý tuần tự với các agent chung chung là researcher (nhà nghiên cứu) và writer (người viết). Các agent được giao task (nhiệm vụ), và task có thể kèm theo tool hoặc memory (bộ nhớ) để hỗ trợ agent.



Hình 5.10: Cấu tạo của một hệ thống CrewAI

CrewAI hỗ trợ hai dạng xử lý chính: **tuần tự** (*sequential*) và **phân cấp** (*hierarchical*). Hình 5.10 thể hiện quy trình tuần tự, lần lượt đi qua các agent đã cho cùng các task tương ứng. Ở tiểu mục tiếp theo, ta đi vào code để thiết lập một crew và dùng nó để hoàn thành một mục tiêu: tạo ra một câu chuyện cười hay.

5.4.1 Tạo crew agent CrewAI chuyên kể chuyện cười

CrewAI đòi hỏi thiết lập nhiều hơn AutoGen, nhưng điều này cũng cho phép kiểm soát nhiều hơn và có thêm các chỉ dẫn, giúp cung cấp ngữ cảnh cụ thể hơn để định hướng agent hoàn thành nhiệm vụ. Cách này không phải không có vấn đề, nhưng nó cho nhiều quyền kiểm soát hơn so với AutoGen khi dùng ngay mặc định.

Mở `crewai_introduction.py` trong VS Code và xem phần đầu file, như trong Listing 4.11. Cần

nhiều thiết lập để cấu hình một agent, gồm role (vai trò), goal (mục tiêu), mức độ verbose (in chi tiết), memory, backstory (lý lịch), delegation (ủy thác) và cả tool (không thể hiện ở đây). Trong ví dụ này, ta dùng hai agent: một senior joke researcher (chuyên gia nghiên cứu chuyện cười) và một joke writer (người viết chuyện cười).

Listing 4.11 crewai_introduction.py (phần agent)

```
from crewai import Agent, Crew, Process, Task
from dotenv import load_dotenv

load_dotenv()

joke_researcher = Agent(     # Tạo agent và cấp cho nó một mục tiêu
    role="Senior Joke Researcher",
    goal="Research what makes things funny about the following {topic}",
    verbose=True,     # verbose cho phép agent in đầu ra ra terminal.
    memory=True,     # Hỗ trợ dùng bộ nhớ cho agent
    backstory=(     # backstory là lý lịch của agent, tức persona của nó.
        "Driven by slapstick humor, you are a seasoned joke researcher"
        "who knows what makes people laugh. You have a knack for finding"
        "the funny in everyday situations and can turn a dull moment into"
        "a laugh riot."
    ),
    allow_delegation=True,    # Agent có thể được ủy thác hoặc được phép ủy thác;
    ↪ True nghĩa là có thể ủy thác.
)

joke_writer = Agent(    # Tạo agent và cấp cho nó một mục tiêu
    role="Joke Writer",
    goal="Write a humourous and funny joke on the following {topic}",
    verbose=True,    # verbose cho phép agent in đầu ra ra terminal.
    memory=True,     # Hỗ trợ dùng bộ nhớ cho agent
    backstory=(    # backstory là lý lịch của agent, tức persona của nó.
        "You are a joke writer with a flair for humor. You can turn a"
        "simple idea into a laugh riot. You have a way with words and"
        "can make people laugh with just a few lines."
    ),
    allow_delegation=False,    # Agent có thể được ủy thác hoặc được phép ủy
    ↪ thác; False nghĩa là không ủy thác.
)
```

Kéo xuống trong code, ta thấy phần task, như trong Listing 4.12. Task mô tả quy trình agent thực hiện để hoàn thành mục tiêu chính của hệ thống. Task cũng gắn một agent với một nhiệm vụ cụ thể, định nghĩa đầu ra của nhiệm vụ đó, và có thể quy định cách nó được thực thi.

Listing 4.12 crewai_introduction.py (phần task)

```
research_task = Task(
    description=(            # Phần description của Task định nghĩa cách agent
    ↪ hoàn thành nhiệm vụ.
        "Identify what makes the following topic:{topic} so funny."
        "Be sure to include the key elements that make it humourous."
        "Also, provide an analysis of the current social trends,"
        "and how it impacts the perception of humor."
    ),
    expected_output="A comprehensive 3 paragraphs long report on the latest
    ↪ jokes.",              # Định nghĩa tường minh đầu ra mong đợi khi thực
    ↪ hiện task
```

```
    agent=joke_researcher,     # Agent được giao làm task này
)

write_task = Task(
    description=(            # Phần description của Task định nghĩa cách agent
    ↪  hoàn thành nhiệm vụ.
        "Compose an insightful, humourous and socially aware joke on
     {topic}."
        "Be sure to include the key elements that make it funny and"
        "relevant to the current social trends."
    ),
    expected_output="A joke on {topic}.",  # Định nghĩa tường minh đầu ra mong
    ↪  đợi khi thực hiện task
    agent=joke_writer,        # Agent được giao làm task này
    async_execution=False,         # Agent có thực thi bất đồng bộ hay không
    output_file="the_best_joke.md",     # Đầu ra (nếu có) mà agent sẽ sinh ra
)
```

Giờ ta có thể thấy mọi thứ ghép lại với nhau thành `Crew` ở cuối file, như trong Listing 4.13. Một lần nữa, có nhiều tuỳ chọn khi xây dựng `Crew`, gồm agent, task, loại process (quy trình xử lý), memory, cache, số request tối đa mỗi phút (`max_rpm`), và việc crew có chia sẻ hay không.

Listing 4.13 crewai_introduction.py (phần crew)

```
crew = Crew(
    agents=[joke_researcher, joke_writer],   # Các agent được tập hợp vào crew
    tasks=[research_task, write_task],    # Các task mà agent có thể làm
    process=Process.sequential,     # Định nghĩa cách các agent tương tác với
    ↪  nhau
    memory=True,     # Hệ thống có dùng bộ nhớ không; phải bật nếu agent/task có
    ↪  bật memory
    cache=True,    # Hệ thống có dùng cache không, tương tự AutoGen
    max_rpm=100,    # Số request tối đa mỗi phút mà hệ thống tự giới hạn
    share_crew=True,    # Crew có chia sẻ thông tin không, tương tự group chat
)

result = crew.kickoff(inputs={"topic": "AI engineer jokes"})
print(result)
```

Khi xem xong, hãy chạy file trong VS Code (F5) và theo dõi terminal để thấy hội thoại và message từ crew. Đến đây có lẽ bạn đã đoán được, mục tiêu của hệ thống agent này là sáng tác chuyện cười liên quan tới nghề kỹ sư AI. Dưới đây là một số câu chuyện cười vui hơn được sinh ra qua vài lần chạy hệ thống agent (giữ nguyên tiếng Anh vì phần lớn là chơi chữ):

- *Why was the computer cold? It left Windows open.*
- *Why don't AI engineers play hide and seek with their algorithms? Because no matter where they hide, the algorithms always find them in the "overfitting" room!*
- *What is an AI engineer's favorite song? "I just called to say I love you ... and to collect more data for my voice recognition software."*
- *Why was the AI engineer broke? Because he spent all his money on cookies, but his browser kept eating them.*

Trước khi chạy thêm các vòng của crew kể chuyện cười, bạn nên đọc tiểu mục tiếp theo. Tiểu mục này hướng dẫn cách thêm khả năng quan sát (observability) cho hệ thống đa agent.

5.4.2 Quan sát agent làm việc với AgentOps

Quan sát một tổ hợp phức tạp như hệ thống đa agent là điều thiết yếu để hiểu vô số vấn đề có thể xảy ra. **Observability** (khả năng quan sát) thông qua tracing (truy vết) ứng dụng là yếu tố then chốt của bất kỳ hệ thống phức tạp nào, nhất là hệ thống dùng trong doanh nghiệp.

CrewAI hỗ trợ kết nối tới một nền tảng vận hành agent chuyên biệt, có cái tên rất hợp là AgentOps. Nền tảng observability này mang tính tổng quát, được thiết kế để hỗ trợ quan sát cho bất kỳ nền tảng agent nào, chuyên về việc sử dụng LLM. Hiện chưa có thông tin về giá hay thương mại hoá.

Kết nối tới AgentOps rất đơn giản: cài package, lấy API key, và thêm một dòng code vào phần thiết lập crew. Bài thực hành tiếp theo sẽ đi qua các bước kết nối và chạy AgentOps.

Listing 4.14 cho thấy cách cài package `agentops` bằng `pip`. Bạn có thể cài riêng package này hoặc cài như một thành phần bổ sung của package `crewai`. Hãy nhớ rằng AgentOps cũng có thể kết nối với các nền tảng agent khác để quan sát.

Listing 4.14 Cài đặt AgentOps

```
pip install agentops

or as an option with CrewAI

pip install crewai[agentops]
```

Trước khi dùng AgentOps, bạn cần đăng ký một API key. Sau đây là các bước chung để đăng ký key tại thời điểm viết sách:

1. Truy cập `https://app.agentops.ai` trên trình duyệt.
2. Đăng ký một tài khoản.
3. Tạo một project, hoặc dùng project mặc định.
4. Vào Settings > Projects and API Keys.
5. Sao chép và/hoặc tạo một API key mới; thao tác này sẽ sao chép key vào trình duyệt (clipboard) của bạn.
6. Dán key vào file `.env` trong project của bạn.

Sau khi sao chép API key, nó sẽ trông giống ví dụ trong listing sau.

Listing 4.15 env.: Thêm AgentOps key

```
AGENTOPS_API_KEY="your API key"
```

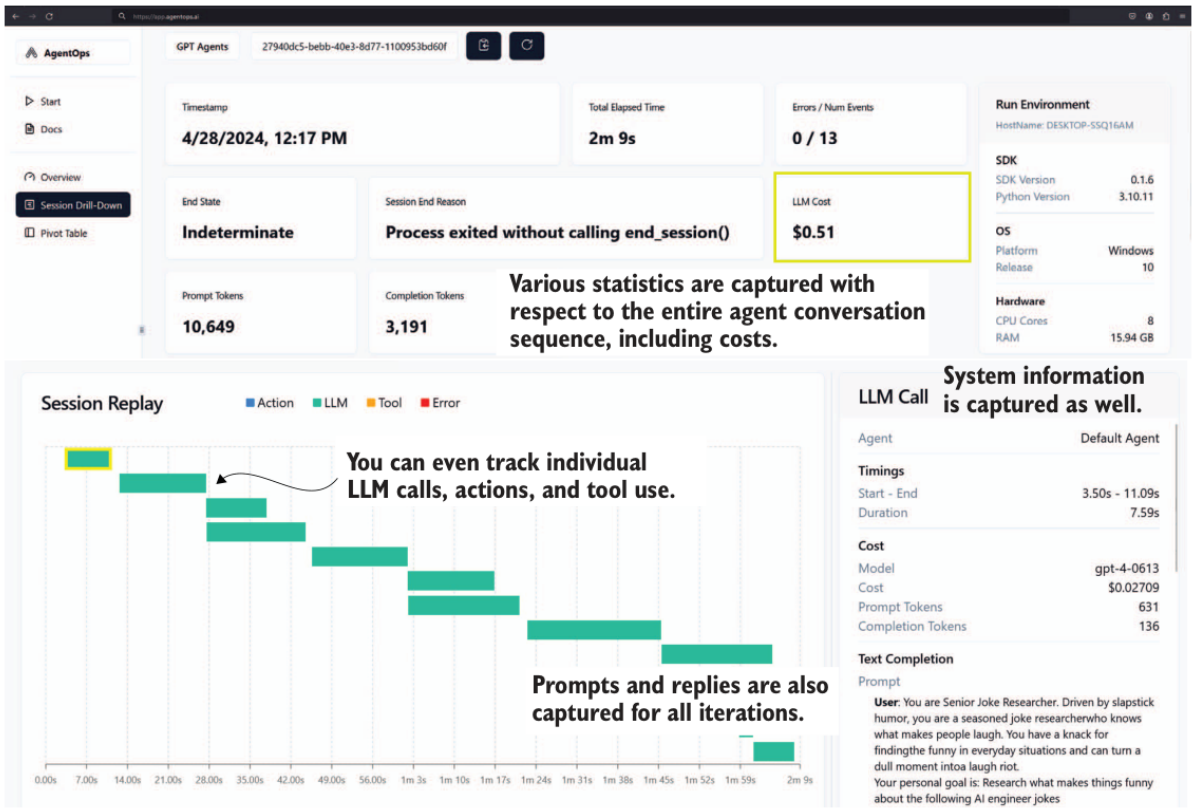
Giờ ta cần thêm vài dòng code vào script CrewAI. Listing 4.16 cho thấy các phần bổ sung trong file `crewai_agentops.py`. Khi tự viết script, bạn chỉ cần thêm package `agentops` và khởi tạo nó khi dùng CrewAI.

Listing 4.16 crewai_agentops.py (phần bổ sung AgentOps)

```
import agentops     # Thêm package cần thiết
from crewai import Agent, Crew, Process, Task
from dotenv import load_dotenv

load_dotenv()
agentops.init()    # Nhớ khởi tạo package sau khi đã nạp các biến môi trường.
```

Chạy file `crewai_agentops.py` trong VS Code (F5) và quan sát các agent làm việc như trước. Tuy nhiên, giờ bạn có thể vào dashboard của AgentOps và xem các tương tác của agent ở nhiều cấp độ.

Hình 5.11 là dashboard khi chạy crew kể chuyện cười để tạo ra câu chuyện cười hay nhất. Dashboard có nhiều thống kê, gồm tổng thời gian, môi trường chạy, số token prompt và completion, thời gian các lần gọi LLM, và chi phí ước tính. Nhìn thấy chi phí vừa khiến bạn tỉnh táo lại, vừa cho thấy hội thoại giữa các agent có thể dài dòng đến mức nào.



Hình 5.11: Dashboard AgentOps khi chạy crew kể chuyện cười

Nền tảng AgentOps là bổ sung rất tốt cho bất kỳ nền tảng agent nào. Tuy được tích hợp sẵn trong CrewAI, điều hữu ích là khả năng quan sát này cũng có thể thêm vào AutoGen hoặc các framework khác. Một điểm hấp dẫn nữa của AgentOps là nó chuyên dành cho việc quan sát tương tác của agent, chứ không phải được chuyển đổi từ một nền tảng vận hành học máy (machine learning operations, MLOps). Trong tương lai, nhiều khả năng ta sẽ thấy thêm các mẫu observability cho agent ra đời.

Một lợi ích không thể xem nhẹ là khả năng quan sát chi phí mà một nền tảng observability mang lại. Bạn có để ý trong Hình 5.11 rằng tạo một câu chuyện cười tốn hơn 50 cent một chút không? Agent có thể rất mạnh, nhưng cũng có thể trở nên rất tốn kém, và việc quan sát các chi phí đó là điều thiết yếu, xét cả về tính thực tiễn lẫn thương mại hoá.

Trong mục cuối của chương, ta sẽ quay lại CrewAI và xây dựng lại các agent có thể lập trình game. Việc này sẽ cho một phép so sánh rất tốt giữa năng lực của AutoGen và CrewAI.

5.5 Quay lại agent lập trình với CrewAI

Một cách tốt để so sánh năng lực giữa các nền tảng đa agent là hiện thực các nhiệm vụ tương tự trong một bot. Trong loạt bài thực hành tiếp theo, ta sẽ dùng CrewAI như một đội lập trình game. Dĩ nhiên, cách này cũng có thể điều chỉnh cho các nhiệm vụ lập trình khác.

Mở `crewai_coding_crew.py` trong VS Code, và trước tiên ta xem phần agent trong Listing 4.17.

Ở đây, ta tạo một senior engineer (kỹ sư cấp cao), một QA engineer (kỹ sư kiểm soát chất lượng) và một chief QA engineer (trưởng nhóm QA), mỗi agent có role, goal và backstory.

Listing 4.17 crewai_coding_crew.py (phần agent)

```
print("## Welcome to the Game Crew")     # Cho phép người dùng nhập chỉ dẫn cho
↪  game của họ
print("--------------------------------")
game = input("What is the game you would like to build? What will be the
↪  mechanics?\n")

senior_engineer_agent = Agent(
    role="Senior Software Engineer",
    goal="Create software as needed",
    backstory=dedent(
        """
        You are a Senior Software Engineer at a leading tech think tank.
        Your expertise in programming in python. and do your best to
        produce perfect code
        """
    ),
    allow_delegation=False,
    verbose=True,
)

qa_engineer_agent = Agent(
    role="Software Quality Control Engineer",
    goal="create prefect code, by analizing the code that is given for errors",
    backstory=dedent(
        """
        You are a software engineer that specializes in checking code
        for errors. You have an eye for detail and a knack for finding
        hidden bugs.
        You check for missing imports, variable declarations, mismatched
        brackets and syntax errors.
        You also check for security vulnerabilities, and logic errors
        """
    ),
    allow_delegation=False,
    verbose=True,
)

chief_qa_engineer_agent = Agent(
    role="Chief Software Quality Control Engineer",
    goal="Ensure that the code does the job that it is supposed to do",
    backstory=dedent(
        """
        You are a Chief Software Quality Control Engineer at a leading
        tech think tank. You are responsible for ensuring that the code
        that is written does the job that it is supposed to do.
        You are responsible for checking the code for errors and ensuring
        that it is of the highest quality.
        """
    ),
    allow_delegation=True,    # Chỉ chief QA engineer mới được ủy thác task.
    verbose=True,
)
```

Kéo xuống trong file sẽ thấy các task của agent, như trong Listing 4.18. Phần mô tả task và đầu ra

mong đợi khá dễ hiểu. Một lần nữa, mỗi agent có một task cụ thể để có ngữ cảnh tốt hơn khi làm việc hoàn thành nhiệm vụ.

Listing 4.18 crewai_coding_crew.py (phần task)

```
code_task = Task(
    description=f"""
You will create a game using python, these are the instructions:
        Instructions
        ------------
        {game}
        You will write the code for the game using python.""",
    expected_output="Your Final answer must be the full python code, only the
    ↪  python code and nothing else.",
    agent=senior_engineer_agent,
)

qa_task = Task(
    description=f"""You are helping create a game using python, these are the
    ↪  instructions:
        Instructions
        ------------
        {game}
        Using the code you got, check for errors. Check for logic errors,
        syntax errors, missing imports, variable declarations,
mismatched brackets,
        and security vulnerabilities.""",
    expected_output="Output a list of issues you found in the code.",
    agent=qa_engineer_agent,
)

evaluate_task = Task(
    description=f"""You are helping create a game using python, these are the
    ↪  instructions:
        Instructions
        ------------
        {game}
        You will look over the code to insure that it is complete and
        does the job that it is supposed to do. """,
    expected_output="Your Final answer must be the corrected a full python code,
    ↪  only the python code and nothing else.",
    agent=chief_qa_engineer_agent,
)
```

Chú thích cho Listing 4.18 (không chèn được comment vào trong chuỗi prompt):

- Các dòng `{game}`: chỉ dẫn game được thay thế vào prompt bằng cơ chế định dạng chuỗi của Python (f-string).

Cuối cùng, ta xem mọi thứ ghép lại ra sao ở cuối file, như trong Listing 4.19. Cấu hình crew này rất giống những gì ta đã thấy trước đó. Mỗi agent và task được thêm vào, cùng với các thuộc tính verbose và process. Trong ví dụ này, ta tiếp tục dùng phương thức tuần tự.

Listing 4.19 crewai_coding_crew.py (phần crew)

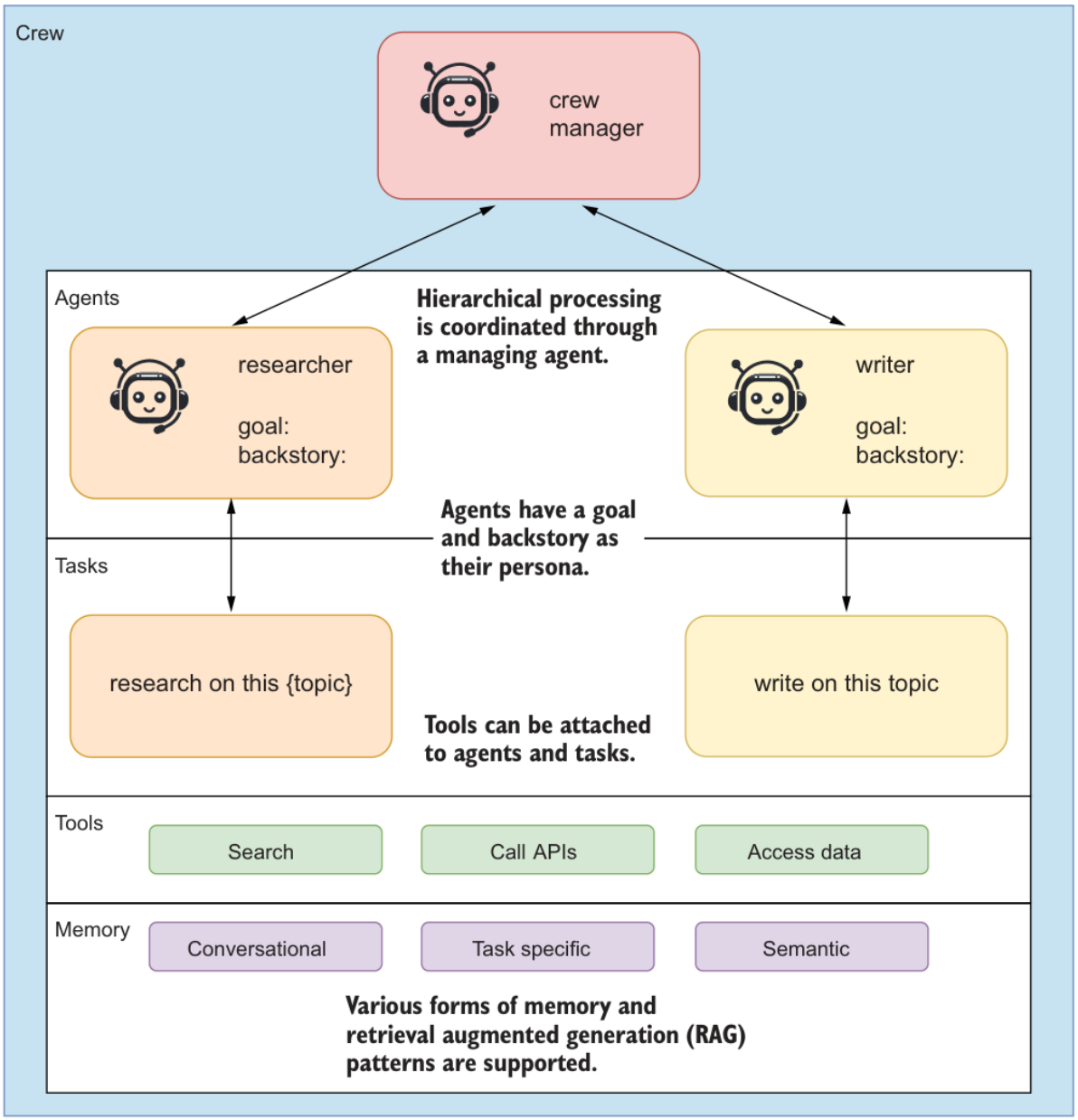
```
crew = Crew(
    agents=[senior_engineer_agent,
            qa_engineer_agent,
```

```
            chief_qa_engineer_agent],
    tasks=[code_task, qa_task, evaluate_task],
    verbose=2,
    process=Process.sequential,     # Process là tuần tự.
)

# Get your crew to work!
result = crew.kickoff()   # Không cung cấp thêm ngữ cảnh nào khi kickoff.

print("######################")
print(result)
```

Khi chạy file trong VS Code (F5), bạn sẽ được yêu cầu nhập chỉ dẫn để viết một game. Hãy nhập vài chỉ dẫn, chẳng hạn game rắn săn mồi (snake) hoặc một game khác tuỳ bạn chọn. Sau đó để các agent làm việc và quan sát những gì chúng tạo ra.

Nhờ có thêm chief QA engineer, kết quả nhìn chung sẽ tốt hơn những gì AutoGen tạo ra, ít nhất là khi dùng mặc định. Nếu xem lại code, bạn sẽ thấy code nhìn chung tuân theo các mẫu tốt, và trong một số trường hợp thậm chí còn có cả test và unit test.

Trước khi kết thúc chương, ta sẽ thực hiện một thay đổi cuối cùng cho mẫu xử lý của crew. Trước đó, ta dùng xử lý tuần tự, như trong Hình 5.10. Hình 5.12 cho thấy xử lý phân cấp trong CrewAI trông như thế nào.



Hình 5.12: Xử lý phân cấp các agent, được điều phối thông qua một crew manager

Thêm manager (người quản lý) này là việc tương đối đơn giản. Listing 4.20 cho thấy các thay đổi code bổ sung trong một file mới, dùng crew lập trình theo phương thức phân cấp. Ngoài việc import một lớp để kết nối tới OpenAI từ LangChain, phần bổ sung còn lại là đặt lớp này làm crew manager, qua tham số `manager_llm`.

Listing 4.20 crewai_hierarchy.py (các phần crew manager)

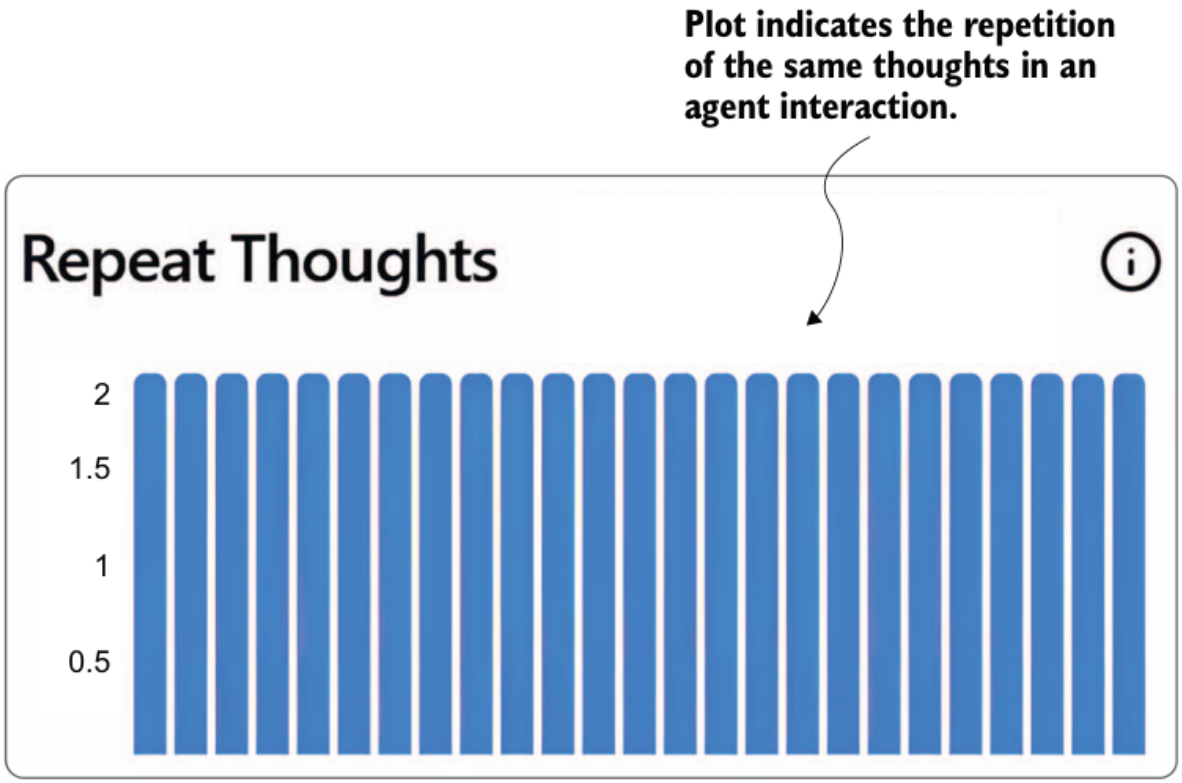
```
from langchain_openai import ChatOpenAI     # Import bộ kết nối LLM từ LangChain

crew = Crew(
    agents=[senior_engineer_agent,
            qa_engineer_agent,
            chief_qa_engineer_agent],
    tasks=[code_task, qa_task, evaluate_task],
    verbose=2,
```

```
    process=Process.hierarchical,    # Bắt buộc đặt crew manager khi chọn xử lý
    ↪  phân cấp.
    manager_llm=ChatOpenAI(              # Đặt crew manager là bộ kết nối LLM
        temperature=0, model="gpt-4"
    ),   # Bắt buộc đặt crew manager khi chọn xử lý phân cấp.
)
```

Chạy file này trong VS Code (F5). Khi được hỏi, hãy nhập game bạn muốn tạo. Hãy thử cùng game bạn đã thử với AutoGen; game snake cũng là một ví dụ chuẩn tốt. Quan sát các agent xử lý code và review đi review lại để tìm vấn đề.

Sau khi chạy file, bạn cũng có thể vào AgentOps để xem chi phí của lần chạy này. Nhiều khả năng nó sẽ tốn hơn gấp đôi so với khi không có agent manager. Đầu ra cũng nhiều khả năng không tốt hơn đáng kể. Đây chính là cái bẫy khi xây dựng hệ thống agent mà không hiểu mọi thứ có thể vượt tầm kiểm soát nhanh đến mức nào.

Một ví dụ của vòng xoáy này, thường xảy ra khi agent liên tục lặp đi lặp lại cùng các hành động, là các task bị lặp lại thường xuyên. Bạn có thể xem vấn đề này trong AgentOps, như trong Hình 5.13, qua biểu đồ Repeat Thoughts.



Hình 5.13: Sự lặp lại của các suy nghĩ (thought) khi chúng xuất hiện trong một lần chạy agent

Biểu đồ Repeat Thoughts của AgentOps là cách rất tốt để đo mức độ lặp lại mà hệ thống agent gặp phải. Các mẫu suy nghĩ lặp lại quá mức thường có nghĩa là agent chưa đủ quyết đoán, và thay vào đó cứ cố sinh ra một câu trả lời khác. Nếu gặp vấn đề này, bạn nên thay đổi mẫu xử lý, task và goal của agent. Bạn thậm chí có thể muốn thay đổi loại và số lượng agent của hệ thống.

Hệ thống đa agent là cách rất tốt để chia nhỏ công việc theo các mẫu công việc gồm job (vị trí công việc) và task. Nhìn chung, vai trò công việc được gán cho một role/persona của agent, còn các task mà nó cần hoàn thành có thể là ngầm định, như trong AutoGen, hoặc tường minh hơn, như trong CrewAI.

Trong chương này, ta đã đi qua nhiều công cụ và nền tảng hữu ích mà bạn có thể dùng ngay để cải thiện công việc, cuộc sống và hơn thế nữa. Đến đây là kết thúc hành trình qua các nền tảng đa agent,

nhưng việc khám phá và sử dụng nhiều agent vẫn chưa dừng lại, như ta sẽ thấy ở các chương sau.

5.6 Bài tập

Hãy dùng các bài tập sau để củng cố kiến thức của bạn về nội dung chương:

Bài tập 1: Giao tiếp agent cơ bản với AutoGen

Mục tiêu: Làm quen với việc giao tiếp và thiết lập agent cơ bản trong AutoGen.
Nhiệm vụ:

- Cài đặt AutoGen Studio trên máy cục bộ theo hướng dẫn trong chương này.
- Tạo một hệ thống đa agent đơn giản gồm một user proxy và hai assistant agent.
- Hiện thực một nhiệm vụ cơ bản, trong đó user proxy điều phối giữa các assistant agent để sinh ra một đầu ra văn bản đơn giản, chẳng hạn tóm tắt một đoạn văn ngắn.

Bài tập 2: Hiện thực skill nâng cao cho agent trong AutoGen Studio

Mục tiêu: Nâng cao năng lực của agent bằng cách thêm các skill nâng cao.
Nhiệm vụ:

- Phát triển và tích hợp một skill mới vào một agent AutoGen, cho phép nó lấy và hiển thị dữ liệu thời gian thực từ một API công khai (ví dụ thông tin thời tiết hoặc giá cổ phiếu).
- Đảm bảo agent có thể hỏi người dùng về lựa chọn của họ (ví dụ thành phố cần xem thời tiết, loại cổ phiếu) và hiển thị dữ liệu lấy được tương ứng.

Bài tập 3: Quản lý task theo vai trò với CrewAI

Mục tiêu: Khám phá việc quản lý task dựa trên vai trò trong CrewAI.
Nhiệm vụ:

- Thiết kế một cấu hình CrewAI trong đó nhiều agent được giao các vai trò cụ thể (ví dụ data fetcher – người lấy dữ liệu, analyzer – người phân tích, presenter – người trình bày).
- Cấu hình một chuỗi task, trong đó data fetcher thu thập dữ liệu, analyzer xử lý dữ liệu, và presenter tạo báo cáo.
- Thực thi chuỗi task và quan sát luồng thông tin cũng như việc ủy thác task giữa các agent.

Bài tập 4: Cộng tác đa agent trong group chat với AutoGen

Mục tiêu: Hiểu và hiện thực một hệ thống group chat trong AutoGen để hỗ trợ agent cộng tác.
Nhiệm vụ:

- Thiết lập một kịch bản trong đó nhiều agent cần cộng tác để giải quyết một bài toán phức tạp (ví dụ lập lịch trình cho một chuyến công tác).
- Dùng tính năng group chat để agent chia sẻ thông tin, đặt câu hỏi và cập nhật cho nhau.
- Theo dõi tương tác giữa các agent và hiệu quả của chúng trong việc cộng tác giải quyết vấn đề.

Bài tập 5: Thêm và kiểm thử observability với AgentOps trong CrewAI

Mục tiêu: Hiện thực và đánh giá khả năng quan sát agent bằng AgentOps trong môi trường CrewAI.
Nhiệm vụ:

- Tích hợp AgentOps vào một hệ thống đa agent CrewAI.
- Thiết kế một task cho agent đòi hỏi tính toán hoặc xử lý dữ liệu đáng kể (ví dụ phân tích đánh giá của khách hàng để xác định xu hướng cảm xúc).
- Dùng AgentOps để theo dõi hiệu năng, chi phí và độ chính xác đầu ra của agent. Xác định mọi điểm kém hiệu quả hoặc lỗi tiềm ẩn trong tương tác giữa các agent.

Tóm tắt

- AutoGen, do Microsoft phát triển, là một nền tảng đa agent dạng hội thoại, dùng nhiều loại agent như user proxy và assistant agent để thực hiện nhiệm vụ thông qua tương tác bằng ngôn ngữ tự nhiên.
- AutoGen Studio là một môi trường phát triển cho phép người dùng tạo, kiểm thử và quản lý hệ thống đa agent, giúp AutoGen dễ dùng hơn.
- AutoGen hỗ trợ nhiều mẫu giao tiếp, gồm group chat, giao tiếp phân cấp và giao tiếp qua proxy. Giao tiếp qua proxy dùng một agent chính (proxy) làm trung gian giữa người dùng và các agent khác để việc hoàn thành nhiệm vụ trôi chảy hơn.
- CrewAI đưa ra cách tiếp cận có cấu trúc để xây dựng hệ thống đa agent, tập trung vào ứng dụng doanh nghiệp. Nó nhấn mạnh chức năng agent dựa trên vai trò và agent tự hành, cho phép quản lý task linh hoạt theo kiểu tuần tự hoặc phân cấp.
- Các bài thực hành trong chương minh hoạ cách thiết lập và dùng AutoGen Studio, gồm cài đặt các thành phần cần thiết và chạy các hệ thống đa agent cơ bản.
- Agent trong AutoGen có thể được trang bị các skill cụ thể để thực hiện các nhiệm vụ như sinh code, phân tích ảnh và truy xuất dữ liệu, qua đó mở rộng phạm vi ứng dụng.
- CrewAI nổi bật ở khả năng cấu trúc tương tác giữa agent chặt chẽ hơn AutoGen, điều này có lợi trong các bối cảnh cần hành vi agent chính xác và được kiểm soát.
- CrewAI hỗ trợ tích hợp memory và tool để agent sử dụng trong quá trình hoàn thành task.
- CrewAI hỗ trợ tích hợp với các công cụ observability như AgentOps, cung cấp thông tin chi tiết về hiệu năng của agent, hiệu quả tương tác và quản lý chi phí.
- AgentOps là một nền tảng observability cho agent, giúp bạn dễ dàng theo dõi các tương tác agent quy mô lớn.

Chương 6

Trao quyền hành động cho agent

Chương này bao quát các nội dung:

- Cách một agent hành động ra bên ngoài chính nó thông qua các action (hành động).
- Định nghĩa và sử dụng các function (hàm) của OpenAI.
- Semantic Kernel và cách dùng semantic function (hàm ngữ nghĩa).
- Phối hợp semantic function với native function (hàm gốc, viết bằng code).
- Dựng một GPT interface (giao diện GPT) bằng Semantic Kernel.

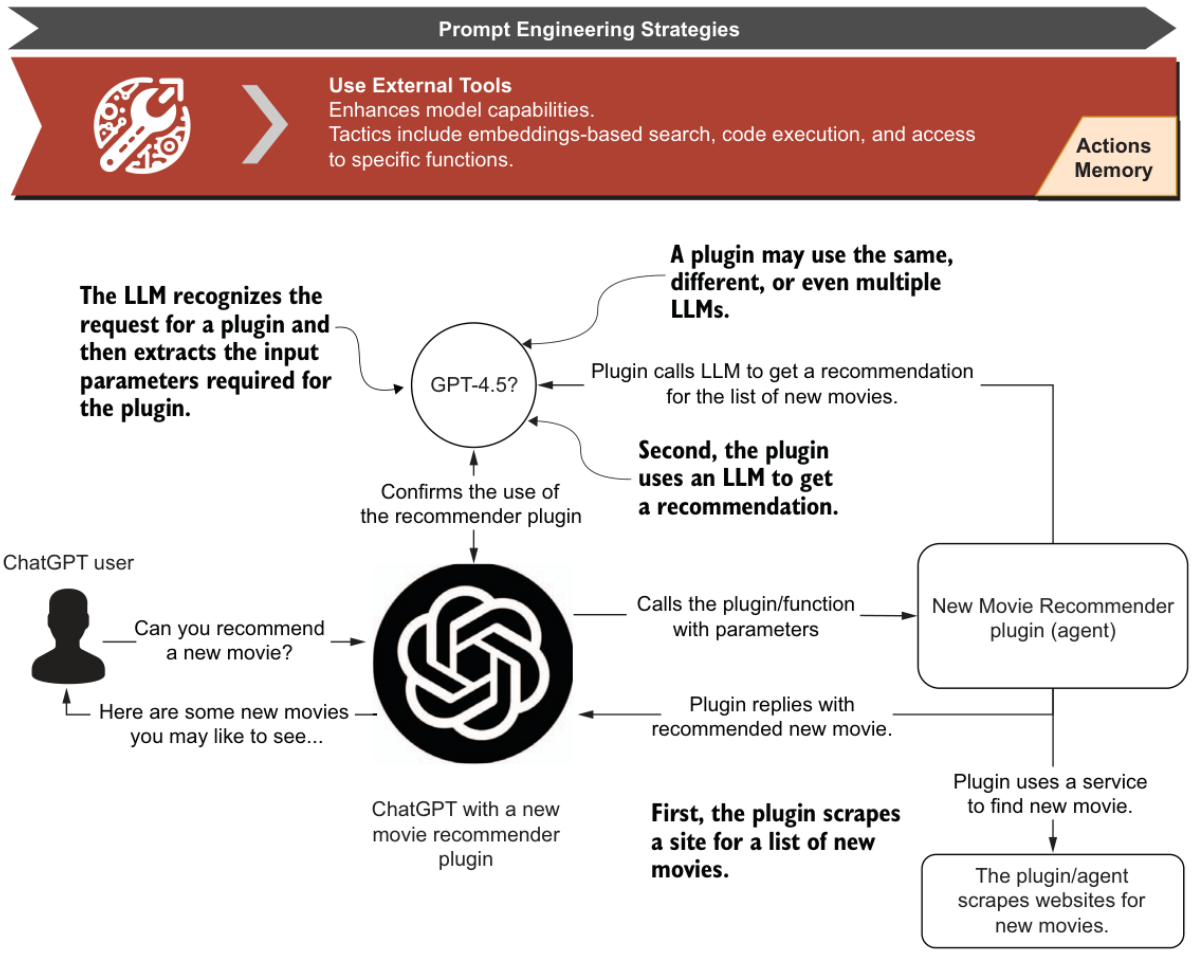
Trong chương này, ta tìm hiểu các **action** thông qua việc dùng function, và cách agent cũng có thể dùng chúng. Ta bắt đầu với tính năng function calling của OpenAI, rồi nhanh chóng chuyển sang một dự án khác của Microsoft tên là **Semantic Kernel** (SK). Ta sẽ dùng SK để xây dựng và quản lý các skill (kỹ năng) và function cho agent, hoặc dùng chính chúng như agent. Cuối chương, ta dùng SK để host hệ thống agent đầu tiên của mình. Đây là một chương có rất nhiều ví dụ code kèm chú thích.

6.1 Định nghĩa action của agent

ChatGPT plugin ban đầu được giới thiệu để bổ sung cho phiên làm việc các khả năng, kỹ năng hoặc công cụ (tool). Với một plugin, bạn có thể tìm kiếm trên web, tạo bảng tính hay vẽ biểu đồ. Plugin cho ChatGPT phương tiện để mở rộng nền tảng.

Hình 6.1 cho thấy cách một ChatGPT plugin hoạt động. Trong ví dụ này, một plugin gợi ý phim mới (movie recommender) đã được cài vào ChatGPT. Khi người dùng nhờ ChatGPT gợi ý một bộ phim mới, mô hình ngôn ngữ lớn (LLM) nhận ra rằng nó có một plugin để xử lý hành động đó. Sau đó LLM tách yêu cầu của người dùng thành các tham số có thể thực thi và chuyển chúng cho plugin gợi ý phim mới.

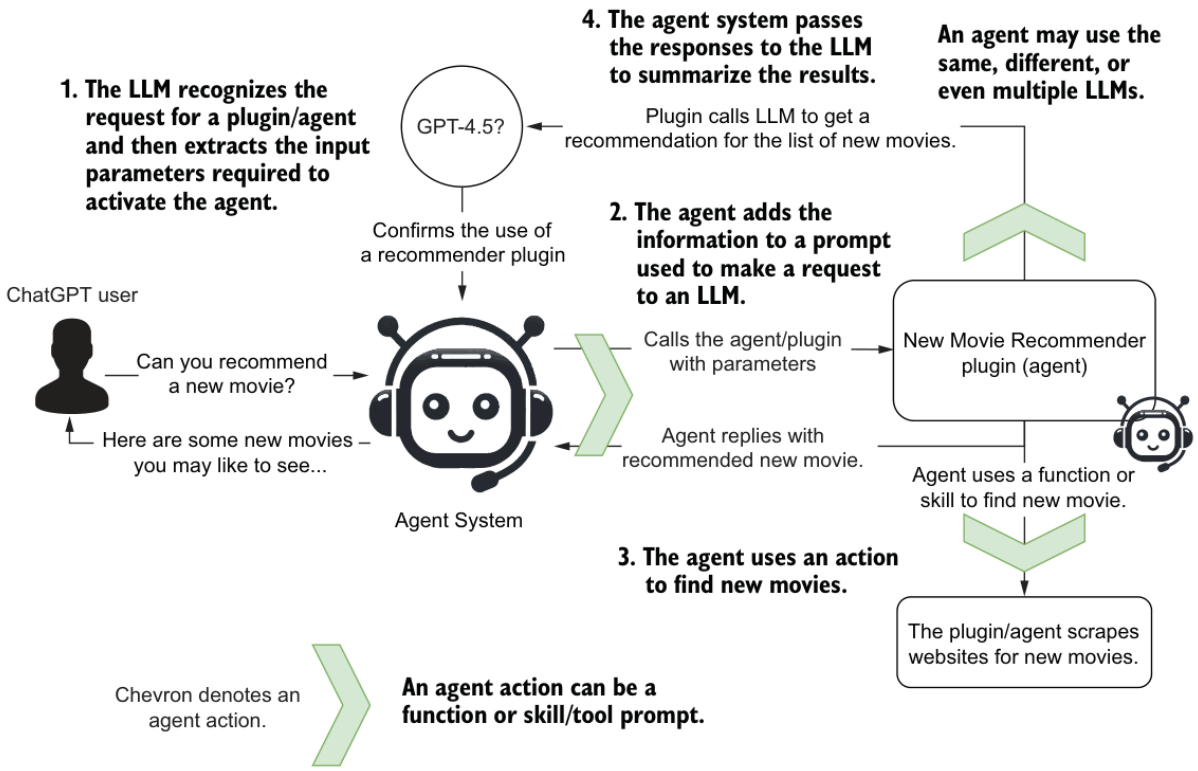
Tiếp theo, plugin gợi ý thu thập dữ liệu (scrape) từ một website giới thiệu phim mới và gắn thông tin đó vào một yêu cầu prompt mới gửi tới LLM. Với thông tin này, LLM phản hồi cho plugin gợi ý, plugin chuyển kết quả lại cho ChatGPT. Cuối cùng ChatGPT trả lời người dùng bằng gợi ý được yêu cầu.



Hình 6.1: Cách một ChatGPT plugin hoạt động, và cách plugin cùng các công cụ bên ngoài khác (ví dụ API) khớp với chiến lược prompt engineering Use External Tools (dùng công cụ bên ngoài)

Ta có thể coi plugin là đại diện (proxy) cho action. Một plugin thường đóng gói một hoặc nhiều khả năng, chẳng hạn gọi một API hay scrape một website. Vì vậy, action là phần mở rộng của plugin: chúng đem lại cho plugin các khả năng của nó.

AI agent có thể được xem vừa là plugin, vừa là bên tiêu thụ (consumer) plugin, tool, skill và các agent khác. Thêm skill, function và tool vào một agent/plugin cho phép nó thực thi các action được định nghĩa rõ ràng. Hình 6.2 làm nổi bật nơi các action của agent diễn ra và cách chúng tương tác với LLM và các hệ thống khác.



Hình 6.2: Cách một agent dùng action để thực hiện các tác vụ bên ngoài

Action của agent là một khả năng cho phép agent dùng một function, skill hoặc tool. Điều dễ gây nhầm lẫn là mỗi framework lại dùng thuật ngữ khác nhau. Để thiết lập vài định nghĩa cơ bản, ta định nghĩa action là *bất cứ việc gì agent có thể làm*.

ChatGPT plugin và function đại diện cho một khả năng có thể thực thi mà ChatGPT hoặc một hệ thống agent dùng để thực hiện thêm các action. Giờ hãy xem nền tảng của OpenAI plugin và định nghĩa function.

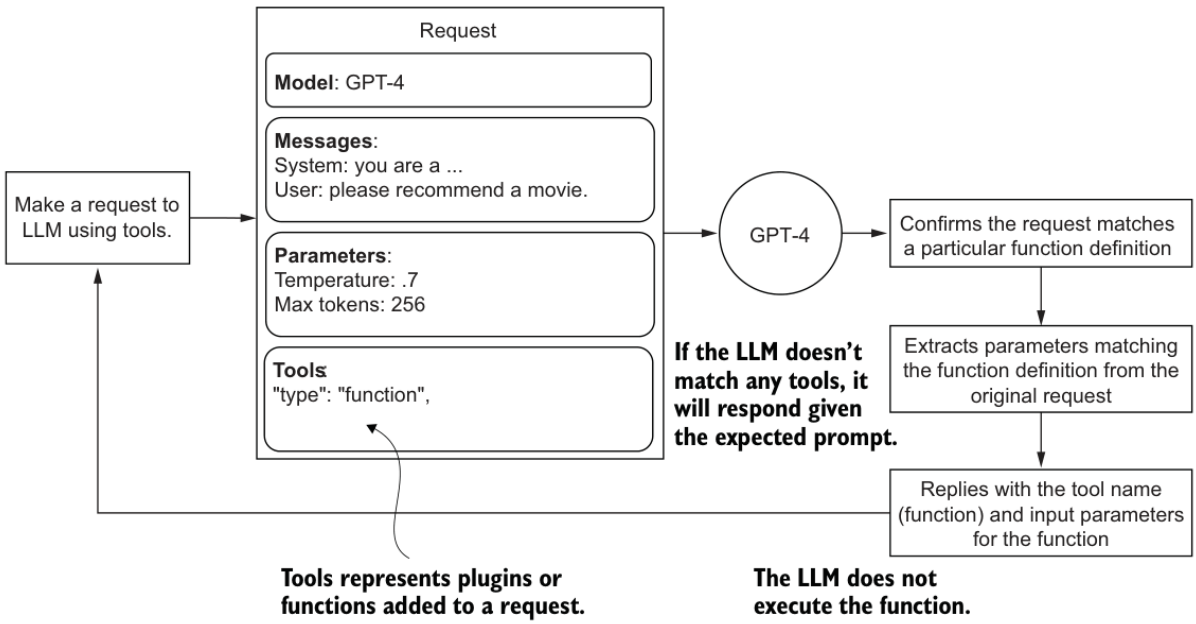
6.2 Thực thi OpenAI function

Cùng với việc hỗ trợ plugin, OpenAI đưa ra một đặc tả có cấu trúc để định nghĩa giao diện giữa các function/plugin mà LLM có thể thực thi. Đặc tả này đang dần thành chuẩn mà các hệ thống LLM có thể theo để cung cấp các hệ thống có khả năng hành động.

Chính các định nghĩa function này hiện cũng được dùng để định nghĩa plugin cho ChatGPT và các hệ thống khác. Tiếp theo, ta xem cách dùng function trực tiếp trong một lời gọi LLM.

6.2.1 Thêm function vào lời gọi LLM API

Hình 6.3 minh hoạ cách LLM nhận ra và dùng định nghĩa function để định dạng phản hồi của nó thành một lời gọi function.



Hình 6.3: Cách một yêu cầu LLM đơn lẻ, có kèm tool, được LLM diễn giải

Listing 5.1 trình bày chi tiết một lời gọi LLM API có dùng tool và định nghĩa function. Thêm định nghĩa function cho phép LLM trả lời theo các tham số đầu vào của function. Nghĩa là LLM sẽ xác định đúng function và phân tích (parse) ra các tham số liên quan từ yêu cầu của người dùng.

Listing 5.1 first_function.py (lời gọi API)

```
response = client.chat.completions.create(
        model="gpt-4-1106-preview",
        messages=[{"role": "system",
                   "content": "You are a helpful assistant."},
                  {"role": "user", "content": user_message}],
        temperature=0.7,
        tools=[    # Tham số mới tên là tools
            {
                "type": "function",    # Đặt loại tool là function
                "function": {
                    "name": "recommend",
                    "description": "Provide a … topic.",    # Mô tả thật tốt
                    ↪  function làm gì
                    "parameters": {
                        "type": "object",    # Kiểu tham số đầu vào; object là
                        ↪  một tài liệu JSON
                        "properties": {
                            "topic": {
                                "type": "string",
                                "description":
                                   "The topic,… for.",    # Mô tả tốt cho từng
                                    ↪  tham số
                            },
                            "rating": {
                                "type": "string",
                                "description":
                              "The rating … given.",    # Mô tả tốt cho từng tham số
                                "enum": ["good",
                                         "bad",
```

```
                                        "terrible"]    # Có thể mô tả bằng tập
                                        ↪  giá trị liệt kê (enum)
                                },
                            },
                            "required": ["topic"],
                        },
                    },
                    }
                ]
            )
```

Để xem cách hoạt động, hãy mở file `chapter_4/first_function.py` trong thư mục mã nguồn của sách bằng Visual Studio Code (VS Code). Một thói quen tốt là mở thư mục của chương liên quan trong VS Code, tạo môi trường Python mới và cài file `requirements.txt`. Nếu cần trợ giúp, hãy xem phụ lục B.

Trước khi bắt đầu, hãy thiết lập đúng file `.env` trong thư mục `chapter_4` với thông tin xác thực API của bạn. Function calling là khả năng bổ sung do dịch vụ LLM thương mại cung cấp. Tại thời điểm viết sách, tính năng này chưa có cho các bản triển khai LLM mã nguồn mở.

Tiếp theo, ta xem phần cuối của file `first_function.py`, như trong Listing 5.2. Đây là hai ví dụ lời gọi tới LLM dùng yêu cầu đã đặc tả ở Listing 5.1. Mỗi yêu cầu đi kèm kết quả sinh ra khi chạy ví dụ.

Listing 5.2 first_function.py (chạy thử API)

```
user = "Can you please recommend me a time travel movie?"
response = ask_chatgpt(user)    # Function đã định nghĩa trước đó
print(response)

###Output
Function(arguments='{"topic":"time travel movie"}',
                      name='recommend')    # Trả về tên function cần gọi và các
                      ↪  tham số đầu vào đã trích ra

user = "Can you please recommend me a good time travel movie?"
response = ask_chatgpt(user)    # Function đã định nghĩa trước đó
print(response)

###Output
Function(arguments='{"topic":"time travel movie",
                     "rating":"good"}',
 name='recommend')    # Trả về tên function cần gọi và các tham số đầu vào đã
 ↪  trích ra
```

Chạy script Python `first_function.py` trong VS Code bằng debugger (F5) hoặc bằng terminal để thấy cùng kết quả. Ở đây, LLM phân tích yêu cầu đầu vào để khớp với bất kỳ tool nào đã đăng ký. Trong trường hợp này, tool là định nghĩa function duy nhất, tức function `recommend`. LLM lấy ra các tham số đầu vào của function này và phân tích chúng từ yêu cầu. Sau đó nó trả lời bằng tên function và các tham số đầu vào tương ứng.

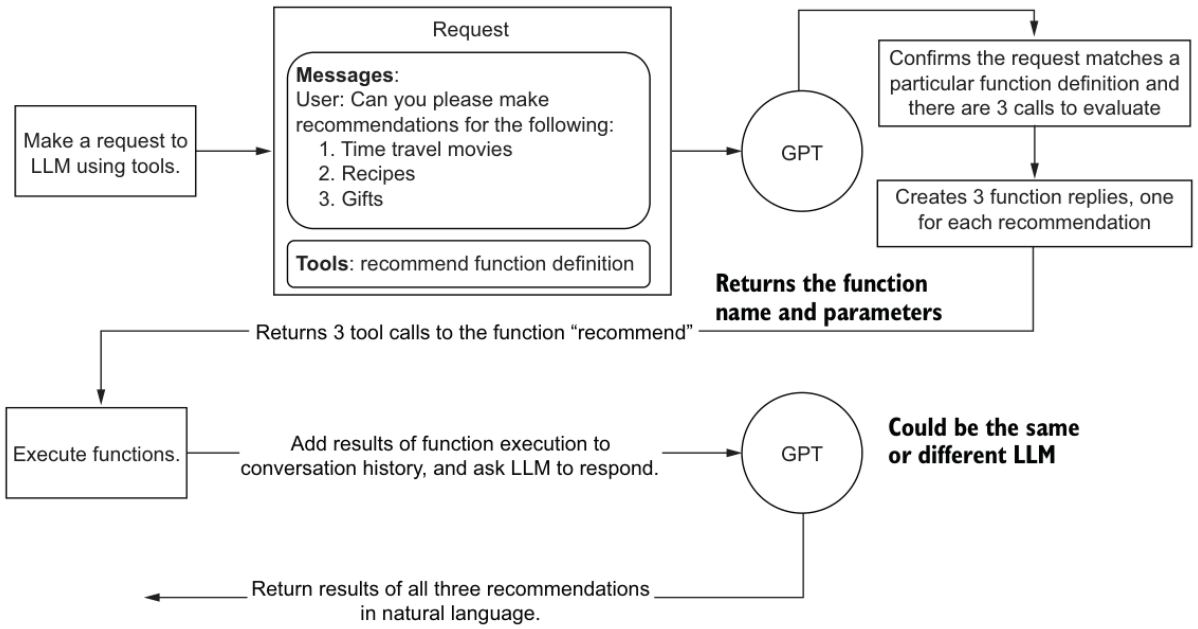
Lưu ý

Function thực sự *không* được gọi. LLM chỉ trả về function được đề xuất và các tham số đầu vào liên quan. Muốn thực thi function, ta phải trích tên và tham số rồi truyền chúng vào một function có chữ ký (signature) tương ứng. Ta sẽ xem ví dụ về việc này ở mục tiếp theo.

6.2.2 Thực thi lời gọi function

Giờ ta đã hiểu LLM không trực tiếp thực thi function hay plugin, ta có thể xem một ví dụ thực thi các tool. Vẫn theo chủ đề gợi ý, ta xem một ví dụ khác có thêm một function Python cho các gợi ý đơn giản.

Hình 6.4 cho thấy ví dụ đơn giản này sẽ hoạt động thế nào. Ta gửi một yêu cầu duy nhất có kèm định nghĩa tool function, xin ba gợi ý. Đáp lại, LLM trả về ba lời gọi function kèm tham số đầu vào (time travel, recipe và gift). Kết quả thực thi các function sau đó được gửi lại cho LLM; LLM chuyển chúng về ngôn ngữ tự nhiên và trả về câu trả lời.



Hình 6.4: Một yêu cầu mẫu trả về ba lời gọi tool function, rồi gửi kết quả lại cho LLM để trả về phản hồi bằng ngôn ngữ tự nhiên

Đã hiểu ví dụ, giờ mở `parallel_functions.py` trong VS Code. Listing 5.3 cho thấy function Python mà bạn muốn gọi để đưa ra gợi ý.

Listing 5.3 parallel_functions.py (function recommend)

```
def recommend(topic, rating="good"):
    if "time travel" in topic.lower():    # Kiểm tra chuỗi có nằm trong topic đầu
    ↪  vào không
        return json.dumps({"topic": "time travel",
                           "recommendation": "Back to the Future",
                           "rating": rating})
    elif "recipe" in topic.lower():    # Kiểm tra chuỗi có nằm trong topic đầu
    ↪  vào không
        return json.dumps({"topic": "recipe",
                           "recommendation": "The best thing … ate.",
                           "rating": rating})
    elif "gift" in topic.lower():    # Kiểm tra chuỗi có nằm trong topic đầu vào
    ↪  không
        return json.dumps({"topic": "gift",
                           "recommendation": "A glorious new...",
                           "rating": rating})
    else:    # Nếu không nhận ra topic, trả về giá trị mặc định
        return json.dumps({"topic": topic,
```

```
                            "recommendation": "unknown"})    # Trả về một đối
                            ↪ tượng JSON
```

Tiếp theo, ta xem function `run_conversation`, nơi mọi việc bắt đầu với việc dựng yêu cầu.

Listing 5.4 parallel_functions.py (run_conversation, yêu cầu)

```
user = """Can you please make recommendations for the following:
1. Time travel movies
2. Recipes
3. Gifts"""    # Message của user xin ba gợi ý
messages = [{"role": "user", "content": user}]    # Lưu ý: không có system
↪ message
tools = [    # Thêm định nghĩa function vào phần tools của yêu cầu
    {
        "type": "function",
        "function": {
            "name": "recommend",
            "description":
                "Provide a recommendation for any topic.",
            "parameters": {
                "type": "object",
                "properties": {
                    "topic": {
                        "type": "string",
                        "description":
                              "The topic, … recommendation for.",
                        },
                        "rating": {
                            "type": "string",
                            "description": "The rating … was given.",
                            "enum": ["good", "bad", "terrible"]
                            },
                        },
                "required": ["topic"],
                },
            },
        }
    ]
```

Listing 5.5 cho thấy yêu cầu được gửi đi; ta đã xem phần này trước đây, nhưng có vài điểm cần lưu ý. Lời gọi này dùng một mô hình thấp hơn như GPT-3.5, vì việc ủy thác cho function là tác vụ đơn giản hơn và có thể làm bằng các mô hình ngôn ngữ cũ hơn, rẻ hơn, kém tinh vi hơn.

Listing 5.5 parallel_functions.py (run_conversation, lời gọi API)

```
response = client.chat.completions.create(
    model="gpt-3.5-turbo-1106",    # LLM ủy thác cho function có thể là mô hình
    ↪ đơn giản hơn
    messages=messages,    # Thêm message và định nghĩa tools
    tools=tools,    # Thêm message và định nghĩa tools
    tool_choice="auto",  # auto là giá trị mặc định
)
response_message = response.choices[0].message    # Message LLM trả về
```

Đến đây, sau lời gọi API, phản hồi sẽ chứa thông tin cho các lời gọi function cần thiết. Nhớ rằng ta đã yêu cầu LLM đưa ra ba gợi ý, nghĩa là nó cũng sẽ trả về ba lời gọi function, như trong listing sau.

Listing 5.6 parallel_functions.py (run_conversation, tool_calls)

```
tool_calls = response_message.tool_calls    # Nếu phản hồi có tool call thì thực
↪  thi chúng
if tool_calls:    # Nếu phản hồi có tool call thì thực thi chúng
    available_functions = {
        "recommend": recommend,
    }    # Chỉ có một function nhưng có thể chứa nhiều function
    # Step 4: send the info for each function call and function response to
the model
    for tool_call in tool_calls:    # Lặp qua các lời gọi và gửi nội dung lại cho
    ↪  LLM
        function_name = tool_call.function.name
        function_to_call = available_functions[function_name]
        function_args = json.loads(tool_call.function.arguments)
        function_response = function_to_call(
            topic=function_args.get("topic"),    # Thực thi recommend với các
            ↪  tham số đã trích
            rating=function_args.get("rating"),
        )
        messages.append(    # Thêm kết quả mỗi lời gọi function vào tập message
            {
                "tool_call_id": tool_call.id,
                "role": "tool",
                "name": function_name,
                "content": function_response,
            }
        )  # extend conversation with function response
    second_response = client.chat.completions.create(    # Gửi yêu cầu mới tới
    ↪  LLM với thông tin cập nhật
        model="gpt-3.5-turbo-1106",
        messages=messages,
    )
    return second_response.choices[0].message.content    # Trả về nội dung
    ↪  message trả lời
```

Kết quả của các tool call và của các lần gọi function `recommend` được thêm vào `messages`. Lưu ý rằng `messages` giờ cũng chứa lịch sử của lời gọi đầu tiên. Toàn bộ được gửi lại cho LLM để dựng câu trả lời bằng ngôn ngữ tự nhiên.

Debug ví dụ này trong VS Code bằng cách nhấn F5 khi đang mở file. Listing sau cho thấy kết quả chạy `parallel_functions.py`.

Listing 5.7 parallel_functions.py (kết quả)

```
Here are some recommendations for you:

1. Time travel movies: "Back to the Future"
2. Recipes: "The best thing you ever ate."
3. Gifts: "A glorious new..." (the recommendation was cut off, so I
couldn't provide the full recommendation)

I hope you find these recommendations helpful! Let me know if you need
more information.
```

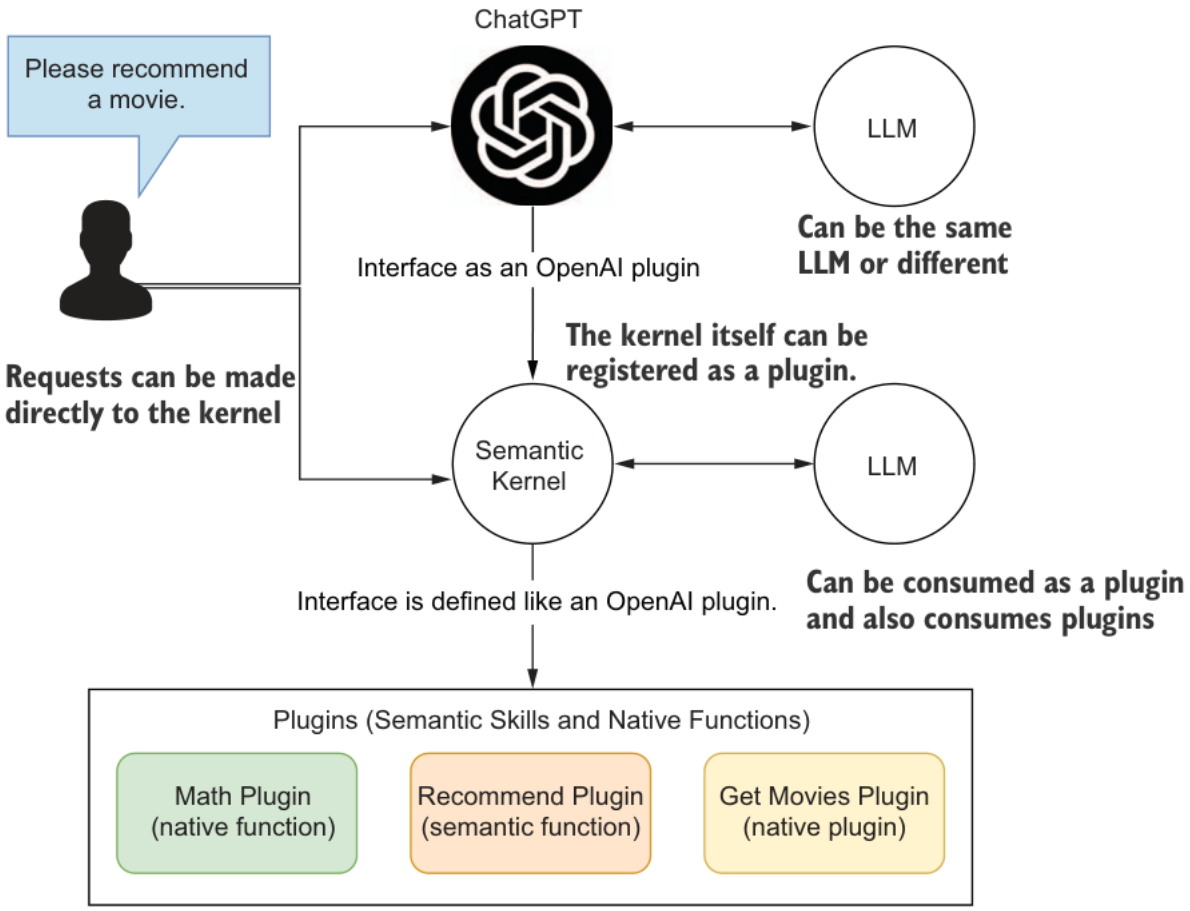
Ví dụ minh hoạ đơn giản này kết thúc ở đây. Với các ứng dụng nâng cao hơn, function có thể làm đủ thứ, từ scrape website, gọi công cụ tìm kiếm tới hoàn thành những tác vụ phức tạp hơn nhiều.

Function là cách tuyệt vời để định dạng đầu ra cho một tác vụ cụ thể. Tuy nhiên, việc xử lý function hay tool và thực hiện các lời gọi thứ cấp có thể làm gọn gàng và hiệu quả hơn. Mục tiếp theo giới thiệu một hệ thống chắc chắn hơn để thêm action cho agent.

6.3 Giới thiệu Semantic Kernel

Semantic Kernel (SK) là một dự án mã nguồn mở khác của Microsoft, nhằm giúp xây dựng ứng dụng AI, mà ta gọi là agent. Về cốt lõi, dự án này phù hợp nhất để định nghĩa action, hay theo cách gọi của nền tảng là **semantic plugin**: lớp bọc (wrapper) cho skill và function.

Hình 6.5 cho thấy SK có thể được dùng vừa như một plugin vừa như bên tiêu thụ OpenAI plugin. SK dựa vào định nghĩa OpenAI plugin để định nghĩa plugin. Nhờ đó, nó có thể tiêu thụ và công bố chính nó hoặc các plugin khác cho các hệ thống khác.



Hình 6.5: Cách Semantic Kernel tích hợp như một plugin và cũng có thể tiêu thụ plugin

Một định nghĩa OpenAI plugin ánh xạ chính xác tới các định nghĩa function trong Listing 5.4. Nghĩa là SK là bộ điều phối (orchestrator) các lời gọi API tool, hay còn gọi là plugin. Điều đó cũng có nghĩa SK có thể giúp tổ chức nhiều plugin với một giao diện chat hoặc một agent.

Ghi chú

Nhóm phát triển SK ban đầu gọi các mô-đun chức năng là *skill*. Tuy nhiên, để nhất quán hơn với OpenAI, họ đã đổi tên *skill* thành *plugin*. Điều gây rối hơn là code vẫn dùng thuật ngữ *skill*. Vì vậy, trong suốt chương này, ta dùng *skill* và *plugin* với cùng một nghĩa.

SK là công cụ hữu ích để quản lý nhiều plugin (action cho agent) và, như ta sẽ thấy sau này, còn hỗ trợ cả các công cụ bộ nhớ (memory) và lập kế hoạch (planning). Trong chương này, ta tập trung vào action/plugin. Mục tiếp theo xem cách bắt đầu dùng SK.

6.3.1 Bắt đầu với semantic function của SK

SK dễ cài đặt và chạy được với Python, Java và C#. Đây là tin tốt, vì nó cũng cho phép plugin phát triển bằng một ngôn ngữ được tiêu thụ bằng ngôn ngữ khác. Tuy nhiên, hiện bạn chưa thể phát triển native function bằng một ngôn ngữ rồi dùng nó ở ngôn ngữ khác.

Ta tiếp tục từ chỗ đã dừng với môi trường Python, dùng workspace `chapter_4` trong VS Code. Hãy chắc chắn bạn đã cấu hình workspace nếu muốn khám phá và chạy các ví dụ.

Listing 5.8 cho thấy cách cài SK từ terminal trong VS Code. Bạn cũng có thể cài extension SK cho VS Code. Extension này là công cụ hữu ích để tạo plugin/skill, nhưng không bắt buộc.

Listing 5.8 Cài đặt Semantic Kernel

```
pip uninstall semantic-kernel    # Gỡ mọi bản cài SK trước đó

git clone https://github.com/microsoft/semantic-kernel.git    # Clone repository
↪  về thư mục cục bộ

cd semantic-kernel/python    # Chuyển vào thư mục mã nguồn

pip install -e .    # Cài package dạng editable từ thư mục mã nguồn
```

Sau khi cài xong, mở `SK_connecting.py` trong VS Code. Listing 5.9 là một demo chạy nhanh một ví dụ qua SK. Ví dụ tạo một dịch vụ chat completion dùng OpenAI hoặc Azure OpenAI.

Listing 5.9 SK_connecting.py

```
import semantic_kernel as sk

selected_service = "OpenAI"    # Chọn dịch vụ đang dùng (OpenAI hoặc Azure
↪  OpenAI)
kernel = sk.Kernel()    # Tạo kernel

service_id = None
if selected_service == "OpenAI":
    from semantic_kernel.connectors.ai.open_ai import OpenAIChatCompletion

    api_key, org_id = sk.openai_settings_from_dot_env()    # Nạp bí mật từ file
    ↪  .env và gán cho dịch vụ chat
    service_id = "oai_chat_gpt"
    kernel.add_service(
        OpenAIChatCompletion(
            service_id=service_id,
            ai_model_id="gpt-3.5-turbo-1106",
            api_key=api_key,
            org_id=org_id,
        ),
    )
elif selected_service == "AzureOpenAI":
    from semantic_kernel.connectors.ai.open_ai import AzureChatCompletion

    deployment, api_key, endpoint = sk.azure_openai_settings_from_dot_env()  #
    ↪  Nạp bí mật từ file .env và gán cho dịch vụ chat
    service_id = "aoai_chat_completion"
    kernel.add_service(
        AzureChatCompletion(
            service_id=service_id,
            deployment_name=deployment,
            endpoint=endpoint,
```

```
            api_key=api_key,
        ),
    )

#This function is currently broken
async def run_prompt():
    result = await kernel.invoke_prompt(
        prompt="recommend a movie about time travel")    # Gọi prompt
    print(result)

# Use asyncio.run to execute the async function
asyncio.run(run_prompt())    # Gọi function bất đồng bộ

###Output
One highly recommended time travel movie is "Back to the Future" (1985)
directed by Robert Zemeckis. This classic film follows the adventures of
teenager Marty McFly (Michael J. Fox)…
```

Chạy ví dụ bằng cách nhấn F5 (debug), bạn sẽ thấy kết quả tương tự Listing 5.9. Ví dụ này minh hoạ cách tạo và thực thi một semantic function bằng SK. Một **semantic function** tương đương với một prompt template trong prompt flow, một công cụ khác của Microsoft. Trong ví dụ này, ta định nghĩa một prompt đơn giản dưới dạng function.

Cần lưu ý rằng semantic function này *không* được định nghĩa như một plugin. Tuy vậy, kernel có thể tạo function như một phần tử ngữ nghĩa độc lập, thực thi được với LLM. Semantic function có thể dùng riêng lẻ hoặc được đăng ký làm plugin, như bạn sẽ thấy sau. Hãy chuyển sang mục tiếp theo, nơi ta giới thiệu các biến ngữ cảnh (context variable).

6.3.2 Semantic function và biến ngữ cảnh

Mở rộng ví dụ trước, ta có thể thêm các biến ngữ cảnh vào semantic function. Khuôn mẫu thêm placeholder (chỗ giữ chỗ) vào prompt template là thứ ta sẽ gặp lại nhiều lần. Trong ví dụ này, ta xem một prompt template có placeholder cho subject (chủ đề), genre (thể loại), format (định dạng) và custom (thông tin tuỳ chỉnh).

Mở `SK_context_variables.py` trong VS Code, như trong listing tiếp theo. Prompt này tương đương với việc tách riêng một phần `system` và một phần `user` của prompt.

Listing 5.10 SK_context_variables.py

```
#top section omitted…
# Định nghĩa prompt có các placeholder
prompt = """
system:

You have vast knowledge of everything and can recommend anything provided
you are given the following criteria, the subject, genre, format and any
other custom information.

user:
Please recommend a {{$format}} with the subject {{$subject}} and {{$genre}}.
Include the following custom information: {{$custom}}
"""

prompt_template_config = sk.PromptTemplateConfig(    # Cấu hình prompt template
↪   và định nghĩa biến đầu vào
    template=prompt,
    name="tldr",
```

```
    template_format="semantic-kernel",
    input_variables=[
        InputVariable(
            name="format",
            description="The format to recommend",
            is_required=True
        ),
        InputVariable(
            name="suject",
            description="The subject to recommend",
            is_required=True
        ),
        InputVariable(
            name="genre",
            description="The genre to recommend",
            is_required=True
        ),
        InputVariable(
            name="custom",
            description="Any custom information
                        to enhance the recommendation",
            is_required=True,
        ),
    ],
    execution_settings=execution_settings,
)

recommend_function = kernel.create_function_from_prompt(    # Tạo kernel function
↪   từ prompt
    prompt_template_config=prompt_template_config,
    function_name="Recommend_Movies",
    plugin_name="Recommendation",
)

async def run_recommendation(    # Tạo function bất đồng bộ bọc lời gọi function
    subject="time travel",
    format="movie",
    genre="medieval",
           custom="must be a comedy"
):
    recommendation = await kernel.invoke(
        recommend_function,
        sk.KernelArguments(subject=subject,
                           format=format,
                           genre=genre,
                           custom=custom),    # Đặt các đối số cho kernel function
    )
    print(recommendation)

# Use asyncio.run to execute the async function
asyncio.run(run_recommendation())

###Output
One movie that fits the criteria of being about time travel, set in a
medieval period, and being a comedy is "The Visitors" (Les Visiteurs)
from 1993. This French film, directed by Jean-Marie Poiré, follows a
knight and his squire who are transported to the modern era by a
wizard’s spell gone wrong.…
```

Hãy debug ví dụ này (F5) và chờ kết quả được sinh ra. Đó là nền tảng để thiết lập SK, tạo và chạy thử semantic function. Ở mục tiếp theo, ta xem cách đăng ký một semantic function làm skill/plugin.

6.4 Phối hợp semantic function và native function

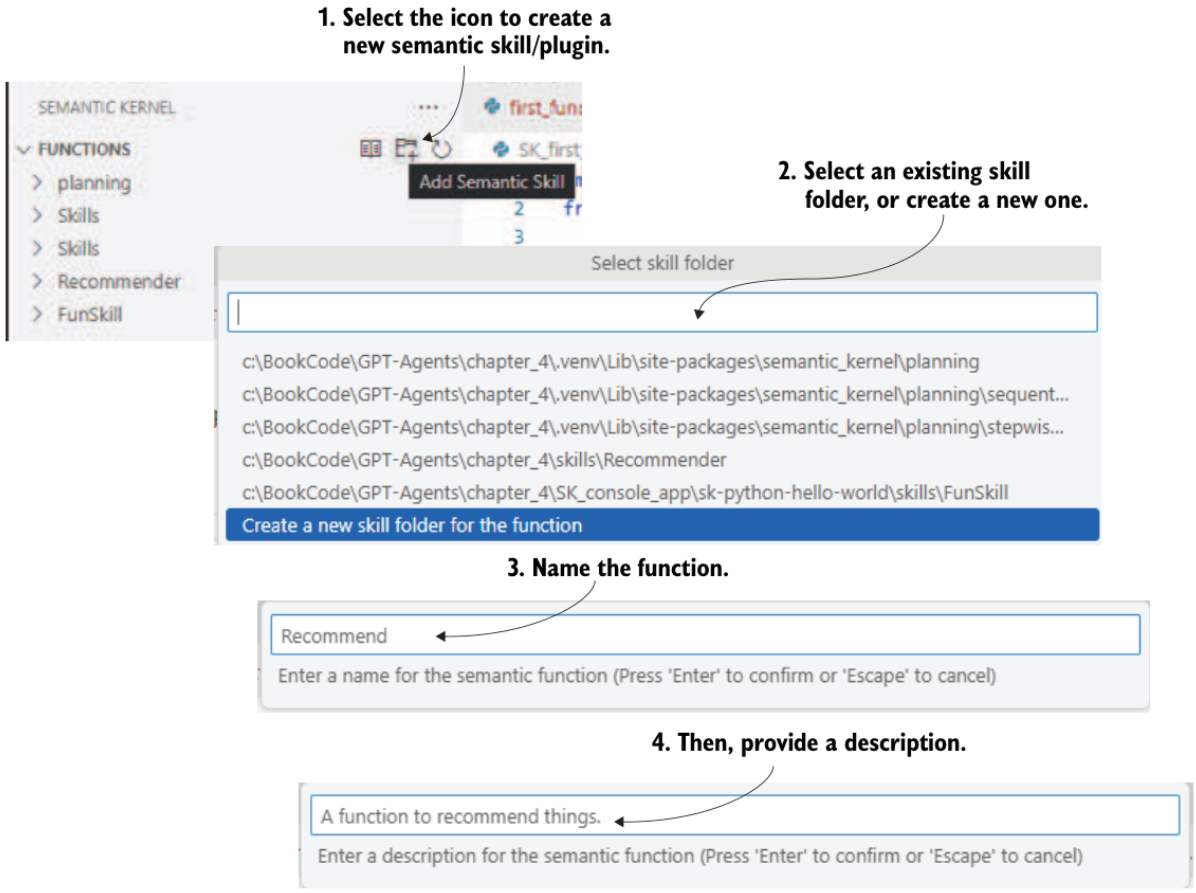
Semantic function đóng gói một prompt/profile và thực thi thông qua tương tác với LLM. **Native function** là phần đóng gói code, có thể làm bất cứ việc gì từ scrape website tới tìm kiếm trên web. Cả semantic function lẫn native function đều có thể đăng ký làm plugin/skill trong kernel của SK.

Một function, dù là semantic hay native, có thể được đăng ký làm plugin và được dùng giống như cách ta đăng ký function trực tiếp trong các lời gọi API trước đó. Khi một function được đăng ký làm plugin, nó trở nên truy cập được từ giao diện chat hoặc agent, tuỳ trường hợp sử dụng. Mục tiếp theo xem cách tạo một semantic function và đăng ký nó với kernel.

6.4.1 Tạo và đăng ký một semantic skill/plugin

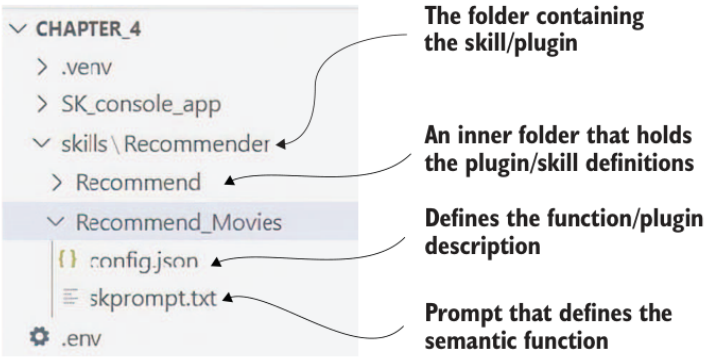
Extension SK cho VS Code cung cấp các công cụ hữu ích để tạo plugin/skill. Trong mục này, ta dùng extension SK để tạo một plugin/skill rồi sửa các thành phần của nó. Sau đó, ta đăng ký và thực thi plugin trong SK.

Hình 6.6 cho thấy quy trình tạo một skill mới trong VS Code bằng extension SK. (Xem phụ lục B nếu cần hướng dẫn cài extension này.) Sau đó bạn sẽ được chọn thư mục skill/plugin để đặt function. Hãy luôn nhóm các function tương tự nhau lại cùng chỗ. Sau khi tạo skill, nhập tên và mô tả của function bạn muốn phát triển. Hãy mô tả function như thể LLM sẽ là bên dùng nó.



Hình 6.6: Quy trình tạo một skill/plugin mới

Bạn có thể xem các skill và function đã hoàn chỉnh bằng cách mở thư mục `skills/plugin` và xem các file. Ta làm theo ví dụ đã dựng trước đó, nên hãy mở thư mục `skills/Recommender/Recommend_Movies`, như trong Hình 6.7. Bên trong thư mục này có file `config.json` chứa mô tả function, và semantic function/prompt trong file `skprompt.txt`.



Hình 6.7: Cấu trúc file và thư mục của một skill/plugin dạng semantic function

Listing 5.11 cho thấy nội dung định nghĩa semantic function, còn gọi là định nghĩa plugin. Lưu ý `type` được đánh dấu là `completion` chứ không phải `function`, vì đây là semantic function. Native function thì sẽ được định nghĩa với type là function.

Listing 5.11 Recommend_Movies/config.json

```
{
    "schema": 1,
    "type": "completion",    // Semantic function là function có type completion
    "description": "A function to recommend movies based on users list of
previously seen movies.",
    "completion": {    // Có thể đặt cả tham số completion cho cách gọi function
        "max_tokens": 256,
        "temperature": 0,
        "top_p": 0,
        "presence_penalty": 0,
        "frequency_penalty": 0
    },
    "input": {
        "parameters": [
            {
                "name": "input",    // Định nghĩa tham số đầu vào của semantic
                ↪  function
                "description": "The users list of previously seen movies.",
                "defaultValue": ""
            }
        ]
    },
    "default_backends": []
}
```

Tiếp theo, ta xem định nghĩa prompt của semantic function, như trong Listing 5.12. Định dạng hơi khác một chút, nhưng nội dung khớp với các ví dụ dùng template trước đó. Prompt này gợi ý phim dựa trên danh sách những phim người dùng đã xem.

Listing 5.12 Recommend_Movies/skprompt.txt

```
You are a wise movie recommender and you have been asked to recommend a
movie to a user.
You are provided a list of movies that the user has watched before.
You want to recommend a movie that the user has not watched before.
[INPUT]
{{$input}}
[END INPUT]
```

Giờ ta đi vào code nạp skill/plugin và thực thi nó trong một ví dụ đơn giản. Mở file `SK_first_skill.py` trong VS Code. Listing sau là bản rút gọn, làm nổi bật các phần mới.

Listing 5.13 SK_first_skill.py (bản rút gọn)

```
kernel = sk.Kernel()

plugins_directory = "plugins"

recommender = kernel.import_plugin_from_prompt_directory(
    plugins_directory,
    "Recommender",
)    # Nạp prompt từ thư mục plugins

recommend = recommender["Recommend_Movies"]

seen_movie_list = [    # Danh sách phim người dùng đã xem
    "Back to the Future",
    "The Terminator",
    "12 Monkeys",
    "Looper",
    "Groundhog Day",
    "Primer",
    "Donnie Darko",
    "Interstellar",
    "Time Bandits",
    "Doctor Strange",
]

async def run():
    result = await kernel.invoke(
        recommend,
        sk.KernelArguments(    # Input được gán bằng danh sách phim đã xem, nối
        ↪  lại thành chuỗi
            settings=execution_settings, input=", ".join(seen_movie_list)
        ),
    )
    print(result)

asyncio.run(run())    # Function được thực thi bất đồng bộ

###Output
Based on the list of movies you've provided, it seems you have an
interest in science fiction, time travel, and mind-bending narratives.
Given that you've watched a mix of classics and modern films in this
genre, I would recommend the following movie that you have not watched
before:

"Edge of Tomorrow" (also known as "Live Die Repeat: Edge of Tomorrow")…
```

Code nạp skill/plugin từ thư mục `skills` và thư mục `plugin`. Khi một skill được *nạp* vào kernel chứ không chỉ được tạo ra, nó trở thành một plugin đã đăng ký. Nghĩa là nó có thể được thực thi trực tiếp như ở đây, hoặc thông qua một cuộc hội thoại chat với LLM qua giao diện plugin.

Chạy code (F5), bạn sẽ thấy kết quả giống Listing 5.13. Giờ ta có một semantic function đơn giản có thể host như một plugin. Tuy nhiên, function này yêu cầu người dùng nhập toàn bộ danh sách phim đã xem. Ta sẽ xem cách khắc phục điều này bằng native function ở mục tiếp theo.

6.4.2 Áp dụng native function

Như đã nói, native function là code có thể làm bất cứ việc gì. Trong ví dụ sau, ta giới thiệu một native function hỗ trợ semantic function đã xây dựng ở trên.

Native function này sẽ nạp từ một file danh sách các phim người dùng đã xem. Tuy function này đưa vào khái niệm bộ nhớ (memory), ta để dành phần thảo luận đó cho Chương 8. Hãy coi native function mới này là bất kỳ đoạn code nào, có thể làm hầu như mọi thứ.

Native function có thể được tạo và đăng ký bằng extension SK. Trong ví dụ này, ta tạo native function trực tiếp trong code để dễ theo dõi.

Mở `SK_native_functions.py` trong VS Code. Ta bắt đầu bằng cách xem native function được định nghĩa thế nào. Native function thường được định nghĩa bên trong một class, giúp việc quản lý và khởi tạo native function đơn giản hơn.

Listing 5.14 SK_native_functions.py (MySeenMovieDatabase)

```
class MySeenMoviesDatabase:
    """
    Description: Manages the list of users seen movies.
    """    # Mô tả cho class chứa function
    @kernel_function(    # Dùng decorator để khai báo mô tả và tên function
        description="Loads a list of movies … user has already seen",
        name="LoadSeenMovies",
    )
    def load_seen_movies(self) -> str:    # Function thực sự, trả về danh sách
    ↪  phim dạng chuỗi phân tách bằng dấu phẩy
        try:
            with open("seen_movies.txt", 'r') as file:    # Nạp phim đã xem từ
            ↪  file văn bản
                lines = [line.strip() for line in file.readlines()]
                comma_separated_string = ', '.join(lines)
            return comma_separated_string
        except Exception as e:
            print(f"Error reading file: {e}")
            return None
```

Với native function đã định nghĩa, ta xem cách dùng nó bằng cách cuộn xuống trong file, như trong listing sau.

Listing 5.15 SK_native_functions (phần code còn lại)

```
plugins_directory = "plugins"

recommender = kernel.import_plugin_from_prompt_directory(
    plugins_directory,
    "Recommender",
)    # Nạp semantic function như trước

recommend = recommender["Recommend_Movies"]
```

```
seen_movies_plugin = kernel.import_plugin_from_object(
    MySeenMoviesDatabase(), "SeenMoviesPlugin"
)    # Import skill vào kernel và đăng ký function làm plugin

load_seen_movies = seen_movies_plugin["LoadSeenMovies"]    # Nạp native function

async def show_seen_movies():
    seen_movie_list = await load_seen_movies(kernel)
    return seen_movie_list

seen_movie_list = asyncio.run(show_seen_movies())    # Thực thi function, trả về
↪  danh sách dạng chuỗi
print(seen_movie_list)

async def run():     # Bọc lời gọi plugin trong function bất đồng bộ rồi thực thi
    result = await kernel.invoke(
        recommend,
        sk.KernelArguments(
                settings=execution_settings,
                input=seen_movie_list),
    )
    print(result)

asyncio.run(run())

###Output
The Matrix, The Matrix Reloaded, The Matrix Revolutions, The Matrix
Resurrections - output from print statement
Based on your interest in the "The Matrix" series, it seems you enjoy
science fiction films with a strong philosophical undertone and action
elements. Given that you've watched all
```

Một điểm quan trọng cần để ý là cách native function được import vào kernel. Hành động import vào kernel đăng ký function đó làm plugin/skill. Nghĩa là function có thể được dùng như một skill từ kernel trong các cuộc hội thoại hay tương tác khác. Ở mục tiếp theo, ta sẽ xem cách nhúng một native function vào trong một semantic function.

6.4.3 Nhúng native function vào semantic function

SK có rất nhiều tính năng mạnh, nhưng một tính năng đặc biệt có ích là khả năng nhúng native function hoặc semantic function vào bên trong các semantic function khác. Listing sau cho thấy cách nhúng một native function vào một semantic function.

Listing 5.16 SK_semantic_native_functions.py (skprompt)

```
sk_prompt = """
You are a wise movie recommender and you have been asked to recommend a
movie to a user.
You have a list of movies that the user has watched before.
You want to recommend a movie that
the user has not watched before.
Movie List: {{MySeenMoviesDatabase.LoadSeenMovies}}.
"""
```

Chú thích cho Listing 5.16 (nằm trong chuỗi prompt nên không chèn comment được):

- Phần chỉ dẫn (từ `You are a wise movie recommender` tới `the user has not watched before.`): đúng nguyên văn chỉ dẫn như trước.

- Dòng `Movie List: {{MySeenMoviesDatabase.LoadSeenMovies}}`: native function được tham chiếu và nhận diện bằng tên class và tên function.

Ví dụ tiếp theo, `SK_semantic_native_functions.py`, dùng native function và semantic function nội tuyến (inline). Mở file trong VS Code; listing sau cho thấy code tạo, đăng ký và thực thi các function.

Listing 5.17 SK_semantic_native_functions.py (bản rút gọn)

```
prompt_template_config = sk.PromptTemplateConfig(
    template=sk_prompt,
    name="tldr",
    template_format="semantic-kernel",
    execution_settings=execution_settings,
)    # Tạo cấu hình prompt template cho prompt

recommend_function = kernel.create_function_from_prompt(
    prompt_template_config=prompt_template_config,
    function_name="Recommend_Movies",
    plugin_name="Recommendation",
)    # Tạo semantic function nội tuyến từ prompt

async def run_recommendation():    # Thực thi semantic function bất đồng bộ
    recommendation = await kernel.invoke(
        recommend_function,
        sk.KernelArguments(),
    )
    print(recommendation)

# Use asyncio.run to execute the async function
asyncio.run(run_recommendation())
###Output
Based on the list provided, it seems the user is a fan of the Matrix
franchise. Since they have watched all four existing Matrix movies, I
would recommend a…
```

Chạy code, bạn sẽ thấy kết quả giống Listing 5.17. Một điểm quan trọng cần lưu ý là native function được đăng ký với kernel, còn semantic function thì không. Điều này quan trọng vì việc *tạo* function không đăng ký function đó.

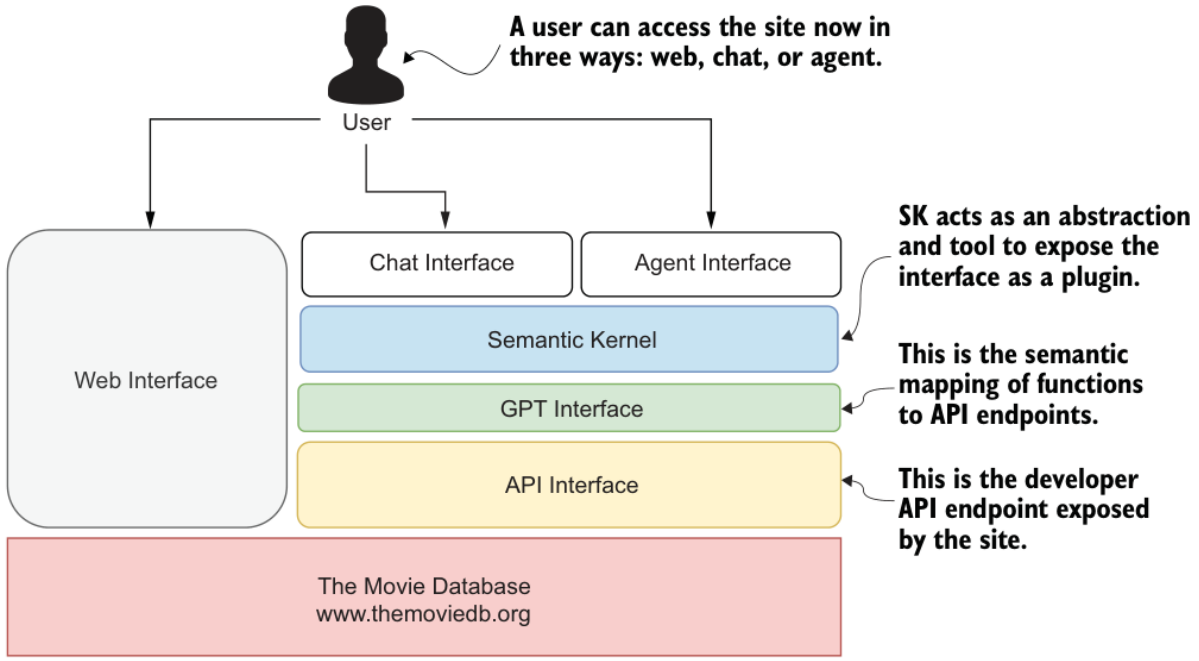
Để ví dụ này chạy đúng, native function phải được đăng ký với kernel. Việc này dùng lời gọi function `import_plugin`, dòng đầu tiên trong Listing 5.17. Tuy nhiên, bản thân semantic function không được đăng ký. Một cách dễ để đăng ký function là biến nó thành plugin rồi import vào.

Các bài tập đơn giản này giới thiệu những cách tích hợp plugin và skill vào giao diện chat hoặc agent. Ở mục tiếp theo, ta xem một ví dụ hoàn chỉnh minh hoạ việc thêm một plugin đại diện cho một dịch vụ, hay một GPT interface, vào một function chat.

6.5 Semantic Kernel như một agent dịch vụ tương tác

Ở Chương 1, ta đã giới thiệu khái niệm **GPT interface**: một mô hình mới để kết nối dịch vụ và các thành phần khác tới LLM thông qua plugin và các lớp ngữ nghĩa (semantic layer). SK cung cấp một lớp trừu tượng rất tốt để chuyển bất kỳ dịch vụ nào thành GPT interface.

Hình 6.8 cho thấy một GPT interface được dựng quanh một dịch vụ API tên là The Movie Database (TMDB; `www.themoviedb.org`). Trang TMDB cung cấp một API miễn phí, cho truy cập thông tin về phim điện ảnh và chương trình truyền hình (TV show).



Hình 6.8: Sơ đồ kiến trúc phân lớp cho thấy vai trò của GPT interface và Semantic Kernel khi được công bố cho giao diện chat hoặc agent

Để làm theo các bài thực hành trong mục này, bạn phải đăng ký một tài khoản miễn phí trên TMDB và tạo API key. Hướng dẫn lấy API key có trên website TMDB (`www.themoviedb.org`), hoặc bạn có thể hỏi GPT-4 Turbo hay một LLM mới hơn.

Trong các tiểu mục tiếp theo, ta sẽ tạo một GPT interface bằng một tập native function của SK. Sau đó, ta dùng kernel của SK để kiểm thử giao diện, và ở phần sau của chương, triển khai nó dưới dạng plugin trong một function chat. Mục tiếp theo xem cách xây dựng GPT interface cho TMDB API.

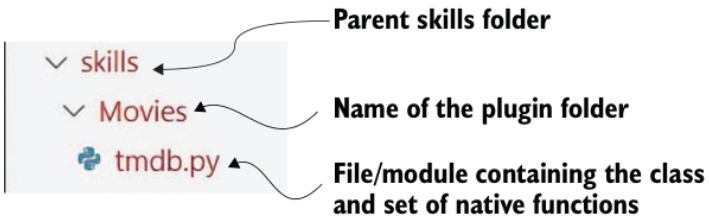
6.5.1 Xây dựng một GPT interface ngữ nghĩa

TMDB là dịch vụ rất tốt, nhưng nó không cung cấp dịch vụ ngữ nghĩa nào hay dịch vụ nào có thể cắm vào ChatGPT hoặc một agent. Để làm được điều đó, ta phải bọc các lời gọi API mà TMDB cung cấp trong một **lớp dịch vụ ngữ nghĩa** (*semantic service layer*).

Lớp dịch vụ ngữ nghĩa là một GPT interface công bố các function thông qua ngôn ngữ tự nhiên. Như đã bàn, muốn công bố function cho ChatGPT hoặc các giao diện khác như agent, chúng phải được định nghĩa thành plugin. May mắn là SK có thể tự động tạo plugin cho ta, miễn là ta viết lớp dịch vụ ngữ nghĩa đúng cách.

Một native plugin, hay một tập skill, có thể đóng vai trò lớp ngữ nghĩa. Để tạo native plugin, hãy tạo một thư mục plugin mới và đặt vào đó một file Python chứa một class gồm tập các native function. Extension SK hiện chưa làm tốt việc này, nên tạo mô-đun thủ công là cách hiệu quả nhất.

Hình 6.9 cho thấy cấu trúc của plugin mới tên `Movies` và lớp dịch vụ ngữ nghĩa tên `tmdb.py`. Với native function, tên thư mục cha (`Movies`) được dùng khi import.



Hình 6.9: Cấu trúc thư mục và file của plugin TMDB

Mở file `tmdb.py` trong VS Code và xem phần đầu file, như trong Listing 5.18. File này chứa một class tên `TMDbService`, công bố một số function ánh xạ tới các lời gọi endpoint của API. Ý tưởng là ánh xạ các lời gọi API liên quan vào lớp dịch vụ ngữ nghĩa này. Nhờ đó các function được công bố thành plugin cho giao diện chat hoặc agent.

Listing 5.18 tmdb.py (đầu file)

```
from semantic_kernel.functions import kernel_function
import requests
import inspect

def print_function_call():    # In ra các lời gọi function để debug
    #omitted …

class TMDbService:    # Service cấp cao nhất; decorator dùng để mô tả function
↪   (mô tả tốt rất quan trọng)
    def __init__(self):
        # enter your TMDb API key here
        self.api_key = "your-TMDb-api-key"

    @kernel_function(    # Service cấp cao nhất; decorator dùng để mô tả function
    ↪   (mô tả tốt rất quan trọng)
        description="Gets the movie genre ID for a given genre name",
        name="get_movie_genre_id",
        input_description="The movie genre name of the genre_id to get",
        )
    def get_movie_genre_id(self, genre_name: str) -> str:    # Function được bọc
    ↪   ngữ nghĩa; nên trả về str
        print_function_call()
        base_url = "https://api.themoviedb.org/3"
        endpoint =
        ↪   f"{base_url}/genre/movie/list?api_key={self.api_key}&language=en-US"

        response = requests.get(endpoint)    # Gọi API endpoint, nếu thành công
        ↪   (mã 200) thì tìm thể loại khớp
        if response.status_code == 200:    # Gọi API endpoint, nếu thành công (mã
        ↪   200) thì tìm thể loại khớp
            genres = response.json()['genres']
            for genre in genres:
                if genre_name.lower() in genre['name'].lower():
                    return str(genre['id'])    # Đã tìm thấy thể loại, trả về id
        return None
```

Phần lớn code của `TMDbService` và các function gọi endpoint của TMDB được viết với sự trợ giúp của GPT-4 Turbo. Sau đó, mỗi function được bọc bằng decorator `sk_function` để công bố nó theo cách ngữ nghĩa.

Một vài lời gọi TMDB API đã được ánh xạ ngữ nghĩa. Listing 5.19 là một ví dụ khác về function được công bố trong lớp dịch vụ ngữ nghĩa. Function này lấy danh sách top 10 phim đang chiếu hiện tại cho một thể loại cụ thể.

Listing 5.19 tmdb.py (get_top_movies_by_genre)

```
@kernel_function(    # Gắn decorator kèm các mô tả cho function
        description="""
Gets a list of currently playing movies for a given genre""",
        name="get_top_movies_by_genre",
        input_description="The genre of the movies to get",
        )
    def get_top_movies_by_genre(self, genre: str) -> str:
        print_function_call()
        genre_id = self.get_movie_genre_id(genre)    # Tìm genre id cho tên thể
        ↪ loại đã cho
        if genre_id:
            base_url = "https://api.themoviedb.org/3"
            playing_movies_endpoint = f"{base_url}/movie/now_playing?api_key={se
            ↪ lf.api_key}&language=en-US"
            response = requests.get(
                        playing_movies_endpoint)    # Lấy danh sách phim đang
                        ↪ chiếu
            if response.status_code != 200:
                return ""

            playing_movies = response.json()['results']
            for movie in playing_movies:
                    movie['genre_ids'] = [str(genre_id)
                          for genre_id in movie['genre_ids']]    # Chuyển
                          ↪ genre_ids thành chuỗi
            filtered_movies = [movie for movie
                in playing_movies if genre_id
                in movie['genre_ids']][:10]    # Kiểm tra genre id có khớp thể
                ↪ loại của phim không
            results = ", ".join([movie['title'] for movie in filtered_movies])
            return results
        else:
            return ""
```

Hãy xem qua các lời gọi API khác đã được ánh xạ ngữ nghĩa. Như bạn thấy, có một khuôn mẫu rõ ràng để chuyển lời gọi API thành dịch vụ ngữ nghĩa. Trước khi chạy toàn bộ dịch vụ, ta sẽ kiểm thử từng function ở mục tiếp theo.

6.5.2 Kiểm thử dịch vụ ngữ nghĩa

Trong ứng dụng thực tế, có lẽ bạn sẽ muốn viết đầy đủ unit test hoặc integration test cho từng function của dịch vụ ngữ nghĩa. Ở đây ta không làm vậy; thay vào đó, ta viết nhanh một script trợ giúp để kiểm thử các function.

Mở `test_tmdb_service.py` trong VS Code và xem code, như trong Listing 5.20. Bạn có thể comment và bỏ comment bất kỳ function nào để kiểm thử riêng lẻ. Hãy đảm bảo mỗi lúc chỉ có một function không bị comment.

Listing 5.20 test_tmdb_service.py

```
import semantic_kernel as sk
from plugins.Movies.tmdb import TMDbService
```

```
async def main():
    kernel = sk.Kernel()    # Khởi tạo kernel

    tmdb_service = kernel.import_plugin_from_object(TMDbService(), "TMDBService")
    ↪  # Import plugin service

    print(
        await tmdb_service["get_movie_genre_id"](
            kernel, sk.KernelArguments(
                            genre_name="action")    # Tham số đầu vào cho
                            ↪  function, khi cần
        )
    )    # Thực thi và kiểm thử các function
    print(
        await tmdb_service["get_tv_show_genre_id"](
            kernel, sk.KernelArguments(
                            genre_name="action")    # Tham số đầu vào cho
                            ↪  function, khi cần
        )
    )    # Thực thi và kiểm thử các function
    print(
        await tmdb_service["get_top_movies_by_genre"](
            kernel, sk.KernelArguments(
                            genre_name="action")    # Tham số đầu vào cho
                            ↪  function, khi cần
        )
    )    # Thực thi và kiểm thử các function
    print(
        await tmdb_service["get_top_tv_shows_by_genre"](
            kernel, sk.KernelArguments(
                            genre_name="action")    # Tham số đầu vào cho
                            ↪  function, khi cần
        )
    )
    print(await tmdb_service["get_movie_genres"](
kernel, sk.KernelArguments()))                       # Thực thi và kiểm thử các
↪  function
    print(await tmdb_service["get_tv_show_genres"](
kernel, sk.KernelArguments()))                       # Thực thi và kiểm thử các
↪  function

# Run the main function
if __name__ == "__main__":
    import asyncio

    asyncio.run(main())    # Thực thi main bất đồng bộ

###Output
Function name: get_top_tv_shows_by_genre    # In chi tiết function để báo khi
↪  function được gọi
Arguments:
  self = <skills.Movies.tmdb.TMDbService object at 0x00000159F52090C0>
  genre = action
Function name: get_tv_show_genre_id    # In chi tiết function để báo khi function
↪  được gọi
Arguments:
  self = <skills.Movies.tmdb.TMDbService object at 0x00000159F52090C0>
  genre_name = action
```

```
Arcane, One Piece, Rick and Morty, Avatar: The Last Airbender, Fullmetal
Alchemist: Brotherhood, Demon Slayer: Kimetsu no Yaiba, Invincible,
Attack on Titan, My Hero Academia, Fighting Spirit, The Owl House
```

Sức mạnh thực sự của SK thể hiện ở bài kiểm thử này. Hãy để ý rằng class `TMDbService` được import thành plugin, nhưng ta không phải định nghĩa thêm cấu hình plugin nào ngoài những gì đã làm. Chỉ bằng việc viết một class bọc vài function API, ta đã công bố một phần TMDB API theo cách ngữ nghĩa. Giờ các function đã được công bố, ở mục tiếp theo ta xem cách dùng chúng làm plugin cho một giao diện chat.

6.5.3 Chat tương tác với lớp dịch vụ ngữ nghĩa

Với các function TMDB đã được công bố ngữ nghĩa, ta có thể tích hợp chúng vào một giao diện chat. Điều này cho phép ta trò chuyện tự nhiên trong giao diện để lấy nhiều thông tin khác nhau, chẳng hạn các phim hàng đầu hiện tại.

Mở `SK_service_chat.py` trong VS Code. Cuộn xuống đầu phần code mới tạo các function, như trong Listing 5.21. Các function tạo ở đây giờ được công bố thành plugin, ngoại trừ function chat mà ta lọc ra vì không muốn công bố nó làm plugin. Function chat ở đây cho phép người dùng trò chuyện trực tiếp với LLM và không nên là một plugin.

Listing 5.21 SK_service_chat.py (thiết lập function)

```
system_message = "You are a helpful AI assistant."

tmdb_service = kernel.import_plugin_from_object(
TMDbService(), "TMDBService")    # Import TMDbService làm plugin

# extracted section of code
execution_settings = sk_oai.OpenAIChatPromptExecutionSettings(
        service_id=service_id,
        ai_model_id=model_id,
        max_tokens=2000,
        temperature=0.7,
        top_p=0.8,
        tool_choice="auto",
        tools=get_tool_call_object(
            kernel, {"exclude_plugin": ["ChatBot"]}),    # Cấu hình execution
            ↪  settings và thêm các tool đã lọc
    )

prompt_config = sk.PromptTemplateConfig.from_completion_parameters(
    max_tokens=2000,
    temperature=0.7,
    top_p=0.8,
    function_call="auto",
    chat_system_prompt=system_message,
)    # Cấu hình prompt
prompt_template = OpenAIChatPromptTemplate(
    "{{$user_input}}", kernel.prompt_template_engine, prompt_config
)    # Định nghĩa template đầu vào, nhận nguyên chuỗi người dùng nhập

history = ChatHistory()

history.add_system_message("You recommend movies and TV Shows.")
history.add_user_message("Hi there, who are you?")
history.add_assistant_message(
    "I am Rudy, the recommender chat bot. I'm trying to figure out what
```

```
people need."
)    # Thêm đối tượng lịch sử chat và nạp sẵn một ít lịch sử

chat_function = kernel.create_function_from_prompt(
    prompt_template_config=prompt_template,
    plugin_name="ChatBot",
    function_name="Chat",
)    # Tạo function chat
```

Tiếp theo, cuộn tiếp trong cùng file để xem function chat, như trong listing sau.

Listing 5.22 SK_service_chat.py (function chat)

```
async def chat() -> bool:
    try:
        user_input = input("User:> ")    # Đầu vào lấy trực tiếp từ
        ↪  terminal/console
    except KeyboardInterrupt:
        print("\n\nExiting chat...")
        return False
    except EOFError:
        print("\n\nExiting chat...")
        return False

    if user_input == "exit":    # Nếu người dùng gõ exit thì thoát chat
        print("\n\nExiting chat...")
        return False
    arguments = sk.KernelArguments(    # Tạo các đối số để truyền cho function
        user_input=user_input,
        history=("\n").join(
           [f"{msg.role}: {msg.content}" for msg in history]),
    )
    result = await chat_completion_with_tool_call(    # Dùng hàm tiện ích để gọi
    ↪  function và thực thi tool
        kernel=kernel,
        arguments=arguments,
        chat_plugin_name="ChatBot",
        chat_function_name="Chat",
        chat_history=history,
    )
    print(f"AI Agent:> {result}")
    return True
```

Cuối cùng, cuộn xuống cuối file và xem function chính. Đây là đoạn code gọi function chat trong một vòng lặp.

Listing 5.23 SK_service_chat.py (function main)

```
async def main() -> None:
    chatting = True
    context = kernel.create_new_context()

    # Lời giới thiệu với người dùng
    print("Welcome to your first AI Agent\
\n  Type 'exit' to exit.\
\n  Ask to get a list of currently playing movies by genre."
    )
    while chatting:    # Tiếp tục cho tới khi chatting là False
```

```
        chatting, context = await chat(context)    # Gọi function chat bất đồng
        ↪ bộ

if __name__ == "__main__":
    asyncio.run(main())
```

Để chạy giao diện chat, chạy file (F5), rồi hỏi về phim hoặc chương trình truyền hình thuộc một thể loại cụ thể. Một phiên hội thoại mẫu có trong Listing 5.24. Kết quả này cho thấy một yêu cầu liệt kê phim của hai thể loại khiến giao diện chat gọi function `get_top_movie_by_genre` nhiều lần.

Listing 5.24 SK_service_chat.py (hội thoại mẫu)

```
Welcome to your first AI Agent
  Type 'exit' to exit.
  Ask to get a list of currently playing movies by genre.
User:> Input: can you give me list of the current top playing movies for
the action and comedy genres?

Function name: get_top_movies_by_genre    # LLM gọi get_top_movies_by_genre hai
↪ lần
Arguments:
  genre = action
Function name: get_movie_genre_id    # Lời gọi nội bộ để lấy genre id
Arguments:
  genre_name = action
Function name: get_top_movies_by_genre    # LLM gọi get_top_movies_by_genre hai
↪ lần
Arguments:
  genre = comedy
Function name: get_movie_genre_id    # Lời gọi nội bộ để lấy genre id
Arguments:
  genre_name = comedy
Agent:> Here are the current top-playing movies
for the action and comedy genres:

**Action:**    # Danh sách phim hành động đang chiếu hàng đầu
1. The Hunger Games: The Ballad of Songbirds & Snakes
2. Rebel Moon - Part One: A Child of Fire
3. Aquaman and the Lost Kingdom
4. Silent Night
5. The Family Plan
6. Freelance
7. Migration
8. Sound of Freedom
9. Godzilla Minus One

**Comedy:**    # Danh sách phim hài đang chiếu hàng đầu
1. The Family Plan
2. Wonka
3. Freelance
4. Saltburn
5. Chicken Run: Dawn of the Nugget
6. Trolls Band Together
7. There's Something in the Barn
8. Migration

Please note that some movies may overlap in both genres, such as
"The Family Plan" and "Freelance."
```

Hãy khám phá giới hạn của giao diện chat và những gì bạn có thể yêu cầu từ dịch vụ TMDB. Ví dụ, thử hỏi danh sách thể loại của phim điện ảnh hoặc chương trình truyền hình. Dịch vụ này là bước thử đầu tiên tốt, nhưng có lẽ ta còn làm tốt hơn được, như sẽ thấy ở mục tiếp theo.

6.6 Tư duy ngữ nghĩa khi viết dịch vụ ngữ nghĩa

Ta vừa xem một minh hoạ rất tốt về việc chuyển một API thành giao diện dịch vụ ngữ nghĩa. Ở trạng thái hiện tại, các function trả về tên các phim và chương trình truyền hình hàng đầu đang chiếu. Tuy nhiên, chỉ trả về tên là ta đang hạn chế khả năng tự phân tích kết quả của LLM.

Vì vậy, ta tạo phiên bản v2 của `TMDbService` để khắc phục điều này, trả về kết quả dưới dạng chuỗi JSON. Mở file `tmdb_v2.py` trong VS Code và cuộn xuống function `get_top_movies_by_genre`.

Listing 5.25 tmdb_v2.py (get_top_movies_by_genre)

```
def get_top_movies_by_genre(self, genre: str) -> str:
        print_function_call()
        genre_id = self.get_movie_genre_id(genre)
        if genre_id:
            #same code …

            return json.dumps(filtered_movies)    # Giờ trả về danh sách đã lọc
            ↪  dưới dạng chuỗi JSON
        else:
            return ""
```

Giờ mở `SK_service_chat.py` trong VS Code, comment và bỏ comment dòng như trong Listing 5.26. Khi đó chương trình dùng phiên bản 2 của `TMDbService`, xuất kết quả dưới dạng các tài liệu JSON đầy đủ trong một chuỗi duy nhất.

Listing 5.26 SK_service_chat.py (sửa import)

```
#from skills.Movies.tmdb import TMDbService    # Comment dòng này lại
from skills.Movies.tmdb_v2 import TMDbService    # Bỏ comment dòng này để dùng
↪  phiên bản 2 của service
```

Chạy lại file `SK_service_chat.py` trong VS Code và đổi câu hỏi một chút, như trong kết quả ở listing sau.

Listing 5.27 SK_service_chat.py (kết quả của service TMDb_v2)

```
User:> get a list of currently playing movies for the
action genre and only return movies about space    # Câu hỏi mới thêm bộ lọc về
↪  không gian vũ trụ
Agent:> To find currently playing action movies that are specifically
about space, I will need to manually filter the provided list for those
that have space-related themes in their overview. Here's what fits that
criteria from the list:

1. **Rebel Moon - Part One: A Child of Fire**    # LLM gọi service rồi xem lại
↪  kết quả trả về khớp bộ lọc
   - Release Date: 2023-12-15
   - Overview: When a peaceful colony on the edge of the galaxy finds
itself threatened by the armies of the tyrannical Regent Balisarius,
they dispatch Kora, a young woman with a mysterious past, to seek out
warriors from neighboring planets to help them take a stand.
```

```
This is the only movie from the provided list that clearly mentions a
space-related theme in its overview. …
```

Vì các function dịch vụ ngữ nghĩa giờ trả về đầy đủ danh sách phim dạng JSON, LLM có thể lọc thêm. Đây là sức mạnh thực sự của dịch vụ ngữ nghĩa: cho phép bạn xử lý dữ liệu thông qua LLM. Nếu chỉ trả về danh sách tên phim, ta sẽ không thấy được sức mạnh này.

Bài thực hành cuối cùng này cho thấy sự thay đổi tư duy cần có khi viết lớp dịch vụ ngữ nghĩa. Nói chung, bạn thường nên trả về càng nhiều thông tin càng tốt. Trả về nhiều thông tin hơn giúp tận dụng khả năng tự lọc, sắp xếp và biến đổi dữ liệu của LLM. Ở chương tiếp theo, ta sẽ tìm hiểu cách xây dựng agent tự trị (autonomous agent) bằng behavior tree (cây hành vi).

6.7 Bài tập

Hãy hoàn thành các bài tập sau để củng cố kiến thức của chương:

Bài tập 1: Tạo một plugin cơ bản để chuyển đổi nhiệt độ

Mục tiêu: làm quen với việc tạo một plugin đơn giản cho OpenAI chat completions API.
Nhiệm vụ:

- Phát triển một plugin chuyển đổi nhiệt độ giữa độ C (Celsius) và độ F (Fahrenheit).
- Kiểm thử plugin bằng cách tích hợp nó vào một phiên chat OpenAI đơn giản, nơi người dùng có thể yêu cầu đổi nhiệt độ.

Bài tập 2: Phát triển plugin thông tin thời tiết

Mục tiêu: học cách tạo một plugin thực hiện một tác vụ riêng biệt.
Nhiệm vụ:

- Tạo một plugin cho OpenAI chat completions API, lấy thông tin thời tiết từ một API công khai.
- Đảm bảo plugin xử lý được yêu cầu của người dùng về điều kiện thời tiết hiện tại ở các thành phố khác nhau.

Bài tập 3: Xây dựng một semantic function sáng tạo

Mục tiêu: khám phá việc tạo semantic function.
Nhiệm vụ:

- Phát triển một semantic function viết một bài thơ hoặc kể một câu chuyện thiếu nhi dựa trên đầu vào của người dùng.
- Kiểm thử function trong một phiên chat để đảm bảo nó sinh ra kết quả sáng tạo và mạch lạc.

Bài tập 4: Tăng cường semantic function bằng native function

Mục tiêu: hiểu cách kết hợp semantic function và native function.
Nhiệm vụ:

- Tạo một semantic function dùng một native function để tăng cường khả năng của nó.
- Ví dụ, phát triển một semantic function lập thực đơn (meal plan) và dùng một native function để lấy thông tin dinh dưỡng của các nguyên liệu.

Bài tập 5: Bọc một Web API có sẵn bằng Semantic Kernel

Mục tiêu: học cách bọc các web API có sẵn thành plugin dịch vụ ngữ nghĩa.

Nhiệm vụ:

- Dùng SK để bọc một API tin tức và công bố nó thành plugin dịch vụ ngữ nghĩa trong một chat agent.
- Đảm bảo plugin xử lý được yêu cầu của người dùng về các bài báo mới nhất theo nhiều chủ đề khác nhau.

Tóm tắt

- Action của agent mở rộng khả năng của một hệ thống agent, chẳng hạn ChatGPT. Điều này bao gồm khả năng thêm plugin vào ChatGPT và LLM để đóng vai trò proxy cho action.
- OpenAI hỗ trợ định nghĩa function và plugin trong một phiên OpenAI API. Điều này bao gồm việc thêm định nghĩa function vào lời gọi LLM API và hiểu cách các function này cho phép LLM thực hiện thêm action.
- Semantic Kernel (SK) là một dự án mã nguồn mở của Microsoft, có thể dùng để xây dựng ứng dụng AI và hệ thống agent. Điều này bao gồm vai trò của semantic plugin trong việc định nghĩa native function và semantic function.
- Semantic function đóng gói template prompt/profile dùng để tương tác với LLM.
- Native function đóng gói code thực hiện hoặc thực thi một action bằng API hay một giao diện khác.
- Semantic function có thể kết hợp với các semantic function hoặc native function khác và được xếp lớp lồng vào nhau như các giai đoạn thực thi.
- SK có thể dùng để tạo một GPT interface bên trên các lời gọi API trong một lớp dịch vụ ngữ nghĩa và công bố chúng thành plugin cho giao diện chat hoặc agent.
- Dịch vụ ngữ nghĩa thể hiện sự tương tác giữa LLM và plugin, cũng như việc hiện thực hoá các khái niệm này trong thực tế để tạo ra AI agent hiệu quả.

Chương 7

Xây dựng trợ lý tự hành

Chương này bao quát các nội dung:

- Cây hành vi (behavior tree) cho robot và ứng dụng AI.
- GPT Assistants Playground, cách tạo assistant và action.
- Điều khiển tự hành bằng cây hành vi agentic (agentic behavior tree).
- Mô phỏng hệ đa agent hội thoại thông qua cây hành vi agentic.
- Dùng back chaining (suy diễn lùi) để tạo cây hành vi cho các hệ thống phức tạp.

Sau khi đã thấy action mở rộng sức mạnh và năng lực của agent như thế nào, ta có thể xem cây hành vi dẫn dắt các hệ thống agentic ra sao. Ta bắt đầu bằng những điều cơ bản về cây hành vi và cách chúng điều khiển robot cũng như AI trong game.

Sau đó ta quay lại với action của agent và xem cách cài đặt action trên nền tảng OpenAI Assistants bằng dự án GPT Assistants Playground. Từ đó, ta xem cách xây dựng một **cây hành vi agentic** tự hành (*agentic behavior tree*, ABT) bằng OpenAI assistants. Tiếp theo, ta tìm hiểu vì sao cần có cơ chế kiểm soát và rào chắn (guardrail) cho agent tự hành, và cách dùng control barrier function (hàm rào chắn điều khiển).

Ở mục cuối chương, ta xem cách dùng nền tảng AgentOps để giám sát các hệ thống agentic tự hành điều khiển bằng hành vi. Đây sẽ là một chương thú vị với nhiều thử thách. Hãy bắt đầu với mục tiếp theo, giới thiệu về cây hành vi.

7.1 Giới thiệu cây hành vi

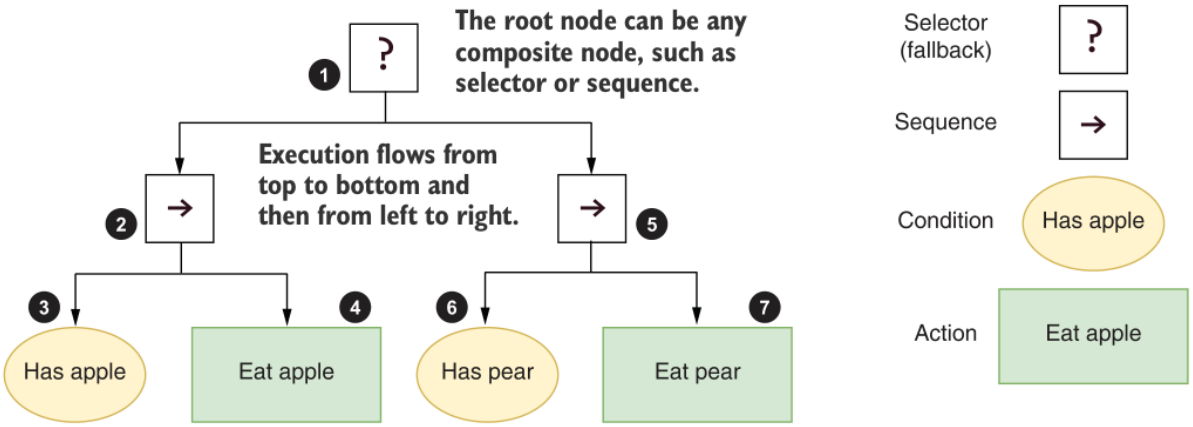
Cây hành vi (*behavior tree*) là một mẫu thiết kế lâu đời dùng để điều khiển robot và AI trong game. Rodney A. Brooks lần đầu giới thiệu khái niệm này trong bài báo "A Robust Layered Control System for a Mobile Robot" năm 1986. Bài báo đặt nền móng cho một mẫu thiết kế về sau được mở rộng thành cấu trúc cây và node như ngày nay.

Nếu bạn từng chơi game có nhân vật không do người chơi điều khiển (*nonplayer character*, NPC) hoặc từng tương tác với các hệ thống robot tiên tiến, bạn đã thấy cây hành vi hoạt động. Hình 7.1 cho thấy một cây hành vi đơn giản. Cây này có đủ các loại node chính: node selector (hay fallback), node sequence, node action và node condition.

Bảng 7.1 mô tả chức năng và mục đích của các node chính mà ta sẽ dùng trong sách. Còn có các node và loại node khác, thậm chí bạn có thể tự tạo node tuỳ biến, nhưng trước mắt ta tập trung vào các node trong bảng.

Bảng 7.1: Các node chính dùng trong cây hành vi

Node	Mục đích	Chức năng	Loại
Selector (fallback)	Node này hoạt động bằng cách chọn node con đầu tiên hoàn thành thành công. Nó thường được gọi là node fallback vì nó luôn lùi về (fall back) node thành công cuối cùng đã thực thi.	Node gọi các node con theo thứ tự và dừng thực thi khi node con đầu tiên thành công. Khi một node con thành công, node trả về success; nếu không node nào thành công, node trả về failure.	Composite
Sequence	Node này thực thi lần lượt tất cả các node con cho đến khi có một node thất bại hoặc tất cả đều hoàn thành thành công.	Node gọi từng node con theo thứ tự, bất kể chúng thất bại hay thành công. Nếu mọi node con thành công, node trả về success; chỉ cần một node con thất bại là trả về failure.	Composite
Condition	Cây hành vi không dùng logic Boolean mà dùng thành công hay thất bại làm phương tiện điều khiển. Condition trả về success nếu điều kiện đúng và trả về false nếu ngược lại.	Node trả về success hoặc failure dựa trên một điều kiện.	Task
Action	Đây là nơi hành động diễn ra.	Node thực thi và trả về success nếu thành công, ngược lại trả về failure.	Task
Decorator	Decorator hoạt động bằng cách kiểm soát việc thực thi các node con. Chúng thường được gọi là các node điều kiện (conditional) vì có thể quyết định một node có đáng thực thi hay có an toàn để thực thi hay không.	Node kiểm soát việc thực thi các node con. Decorator có thể hoạt động như control barrier function để chặn hoặc ngăn các hành vi không mong muốn.	Decorator
Parallel	Node này thực thi song song tất cả các node con. Thành công hay thất bại được quyết định bởi một ngưỡng: số node con cần thành công để node trả về success.	Node thực thi tất cả các node con theo thứ tự, bất kể trạng thái của các node đó.	Composite



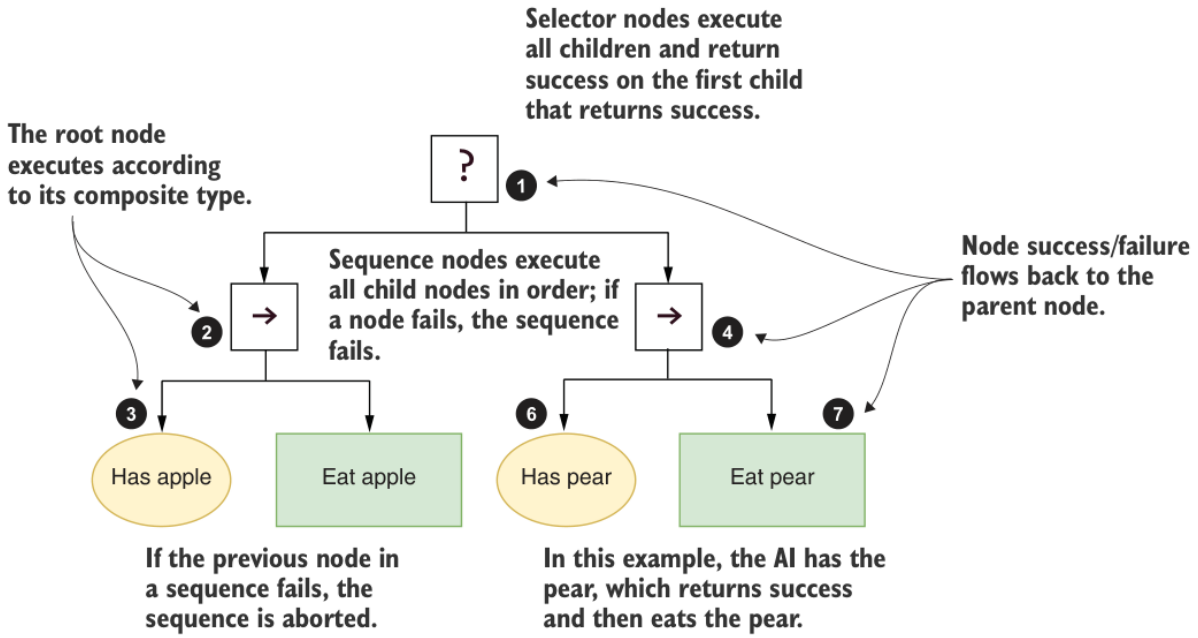
Hình 7.1: Một cây hành vi đơn giản cho việc ăn táo hoặc ăn lê

Các node chính trong Bảng 7.1 đủ để xử lý rất nhiều trường hợp sử dụng. Tuy vậy, lúc đầu cây hành vi có thể khá khó hiểu. Bạn sẽ chỉ thấy hết độ phức tạp bên dưới của chúng khi bắt đầu dùng. Trước khi xây vài cây đơn giản, ta xem kỹ hơn cách thực thi ở mục tiếp theo.

7.1.1 Hiểu cách thực thi cây hành vi

Hiểu cách cây hành vi thực thi là điều cốt yếu khi thiết kế và cài đặt chúng. Khác với phần lớn khái niệm trong khoa học máy tính, cây hành vi vận hành theo *thành công* (success) và *thất bại* (failure). Khi một node trong cây thực thi, nó trả về success hoặc failure; điều này áp dụng cả với node condition và node selector.

Cây hành vi thực thi từ trên xuống dưới và từ trái sang phải. Hình 7.2 cho thấy quá trình này và điều gì xảy ra khi một node thất bại hay thành công. Trong ví dụ, AI do cây điều khiển có quả lê nhưng không có quả táo.[1] Ở node sequence thứ nhất, một condition kiểm tra AI có quả táo hay không. Vì AI không có táo, nó huỷ sequence này và lùi về selector. Selector chọn node con tiếp theo, là một sequence khác, kiểm tra AI có quả lê hay không; vì AI có lê, AI ăn quả lê.



Hình 7.2: Quá trình thực thi của một cây hành vi đơn giản

Cây hành vi cho phép kiểm soát cách một hệ thống AI thực thi ở mức vĩ mô hoặc vi mô. Với robot, cây hành vi thường được thiết kế ở mức vi mô, trong đó mỗi action hay condition là một sự kiện nhỏ, chẳng hạn phát hiện quả táo. Ngược lại, cây hành vi cũng có thể điều khiển các hệ thống ở mức vĩ mô hơn, như NPC trong game, nơi mỗi action có thể là tổ hợp nhiều sự kiện, ví dụ tấn công người chơi.

Với hệ thống agentic, cây hành vi hỗ trợ điều khiển agent hoặc assistant ở mức bạn chọn. Ta sẽ khám phá việc điều khiển agent ở mức tác vụ (task), và ở các chương sau là mức lập kế hoạch (planning). Suy cho cùng, với sức mạnh của LLM, agent có thể tự xây dựng cây hành vi của riêng mình.

Dĩ nhiên còn nhiều dạng điều khiển AI khác có thể dùng để điều khiển hệ thống agentic. Mục tiếp theo xem xét các hệ thống đó và so sánh chúng với cây hành vi.

7.1.2 Lựa chọn cây hành vi

Có nhiều hệ thống điều khiển AI khác có ưu điểm riêng và đáng để tìm hiểu khi điều khiển hệ thống agentic. Chúng giúp làm rõ ưu điểm của cây hành vi và cung cấp lựa chọn khác cho những trường hợp sử dụng cụ thể. Cây hành vi là một mẫu thiết kế xuất sắc nhưng không phải là duy nhất, và cũng nên biết về các mẫu khác.

[1] Ghi chú của người dịch: văn bản gốc ghi "có táo nhưng không có lê" rồi lại viết "AI ăn quả táo", mâu thuẫn với chính mạch giải thích. Bản dịch sửa lại cho khớp với logic của cây trong Hình 7.2.

Bảng 7.2 nêu một số hệ thống khác có thể cân nhắc để điều khiển hệ thống AI. Mỗi dòng mô tả phương pháp làm gì, nhược điểm của nó và khả năng áp dụng vào điều khiển AI agentic.

Bảng 7.2: So sánh các hệ thống điều khiển AI khác

Tên phương pháp	**Mô tả**	**Nhược điểm**	**Điều khiển AI agentic?**
Máy trạng thái hữu hạn[a] (finite state machine, FSM)	FSM mô hình hoá AI bằng một tập trạng thái và các phép chuyển trạng thái được kích hoạt bởi sự kiện hoặc điều kiện.	FSM có thể trở nên cồng kềnh khi độ phức tạp tăng.	FSM không thực tế cho agent vì không mở rộng tốt.
Cây quyết định[b] (decision tree)	Cây quyết định dùng mô hình dạng cây gồm các quyết định và hệ quả có thể có của chúng.	Cây quyết định có thể bị overfitting (quá khớp) và thiếu khả năng tổng quát hoá trong các tình huống phức tạp.	Cây quyết định có thể được điều chỉnh và tăng cường bằng cây hành vi.
Hệ dựa trên tiện ích[b] (utility-based system)	Các hàm tiện ích (utility function) đánh giá và chọn action tốt nhất dựa trên tình huống hiện tại.	Các hệ này đòi hỏi thiết kế hàm tiện ích cẩn thận để cân bằng các ưu tiên.	Mẫu này có thể được dùng bên trong một cây hành vi.
Hệ dựa trên luật[a] (rule-based system)	Một tập luật if-then định nghĩa hành vi của AI.	Các hệ này có thể trở nên rườm rà khi có nhiều luật, dẫn tới khả năng xung đột.	Không thực tế lắm khi kết hợp với hệ agentic chạy bằng LLM.
Hệ lập kế hoạch[c] (planning system)	Hệ lập kế hoạch dùng thuật toán lập kế hoạch để sinh một chuỗi action nhằm đạt một mục tiêu cụ thể.	Các hệ này tốn kém về tính toán và đòi hỏi nhiều tri thức chuyên môn về lĩnh vực.	Agent đã có thể tự cài đặt các mẫu như vậy, như ta sẽ thấy ở các chương sau.
Nhân bản hành vi[c] (behavioral cloning)	Nhân bản hành vi là học chính sách (policy) bằng cách bắt chước các minh hoạ của chuyên gia.	Hệ này có thể khó tổng quát hoá sang các tình huống chưa gặp.	Có thể đưa vào cây hành vi hoặc vào một tác vụ cụ thể.
Mạng tác vụ phân cấp[d] (Hierarchical Task Network, HTN)	HTN phân rã tác vụ thành các tác vụ con nhỏ hơn, dễ quản lý, sắp xếp theo phân cấp.	Phức tạp khi quản lý và thiết kế cho các tác vụ rất lớn.	HTN cho phép tổ chức và thực thi các tác vụ phức tạp tốt hơn. Mẫu này có thể dùng cho các hệ agentic lớn hơn.
Hệ bảng đen[b] (blackboard system)	Các hệ này giải quyết vấn đề theo kiểu cộng tác, dùng một "bảng đen" chung cho các hệ con khác nhau.	Các hệ này khó cài đặt và khó quản lý giao tiếp giữa các hệ con.	Hệ agentic có thể cài đặt mẫu tương tự bằng hội thoại hoặc group chat/thread.
Giải thuật di truyền[d] (genetic algorithm, GA)	Các kỹ thuật tối ưu này lấy cảm hứng từ chọn lọc tự nhiên, tiến hoá lời giải để giải quyết vấn đề.	GA tốn nhiều tính toán và không phải lúc nào cũng tìm được lời giải tối ưu.	GA có tiềm năng và thậm chí có thể dùng để tối ưu cây hành vi.

[a] Không thực tế khi xét các hệ agentic phức tạp.
[b] Đã có sẵn trong cây hành vi hoặc dễ dàng đưa vào cây hành vi.
[c] Thường áp dụng ở mức tác vụ hoặc mức action/condition.
[d] Các hệ nâng cao, đòi hỏi nhiều công sức khi áp dụng cho agent.

Ở các chương sau, ta sẽ tìm hiểu một số mẫu đã nêu trong Bảng 7.2. Nhìn chung, nhiều mẫu có thể được tăng cường hoặc tích hợp khi lấy cây hành vi làm nền. Một số mẫu khác như FSM có thể hữu ích cho các thử nghiệm nhỏ, nhưng không mở rộng được như cây hành vi.

Là một hệ thống điều khiển AI, cây hành vi mang lại nhiều lợi ích, trong đó có khả năng mở rộng. Danh sách sau nêu các lợi ích đáng chú ý khác của cây hành vi:

- *Tính mô-đun và khả năng tái sử dụng*: cây hành vi khuyến khích cách thiết kế hành vi theo mô-đun, cho phép nhà phát triển tạo các thành phần tái sử dụng được. Các node trong cây có thể dễ dàng được dùng lại ở các phần khác nhau của cây, thậm chí ở các dự án khác, giúp dễ bảo trì và giảm thời gian phát triển.
- *Khả năng mở rộng*: khi hệ thống phức tạp dần, cây hành vi tiếp nhận hành vi mới nhẹ nhàng hơn các phương pháp khác như FSM. Cây hành vi cho phép tổ chức tác vụ theo phân cấp, giúp quản lý và hiểu các tập hành vi lớn dễ hơn.
- *Tính linh hoạt và khả năng mở rộng tính năng*: cây hành vi là một khung linh hoạt, có thể thêm node mới (action, condition, decorator) mà không phải thay đổi mạnh cấu trúc hiện có. Nhờ đó, việc đưa vào hành vi mới hoặc sửa hành vi cũ để đáp ứng yêu cầu mới trở nên đơn giản.
- *Gỡ lỗi và trực quan hoá*: cây hành vi biểu diễn hành vi một cách trực quan, rõ ràng, có ích cho việc gỡ lỗi và hiểu quá trình ra quyết định. Các công cụ hỗ trợ cây hành vi thường có trình soạn thảo đồ hoạ cho phép trực quan hoá và gỡ lỗi cấu trúc cây, giúp dễ tìm và sửa lỗi.
- *Tách rời logic quyết định*: cây hành vi tách logic ra quyết định khỏi logic thực thi, phân biệt rõ chiến lược cấp cao với các action cấp thấp. Sự tách rời này làm thiết kế đơn giản hơn, cho phép sửa đổi và kiểm thử từng phần hành vi dễ dàng mà không ảnh hưởng đến toàn hệ thống.

Sau khi đã thuyết phục về cây hành vi, giờ ta xem cách cài đặt chúng bằng code. Mục tiếp theo trình bày cách xây một cây hành vi đơn giản bằng Python.

7.1.3 Chạy cây hành vi với Python và py_trees

Vì cây hành vi đã tồn tại từ lâu và được tích hợp vào nhiều công nghệ, việc tạo một ví dụ minh hoạ rất đơn giản. Dĩ nhiên cách dễ nhất là hỏi ChatGPT hoặc công cụ chat AI bạn ưa thích. Listing 6.1 là kết quả của việc dùng một prompt để sinh code mẫu, kèm theo Hình 7.1 làm cây ví dụ. Code cuối cùng phải sửa một vài lỗi đơn giản về đặt tên và tham số.

Ghi chú

Toàn bộ code của chương này có trong dự án GPT Assistants Playground, tải tại `https://mng.bz/Ea0q`.

Listing 6.1 first_btree.py

```
import py_trees

# Tạo một class để cài đặt một action hoặc condition
class HasApple(py_trees.behaviour.Behaviour):
    def __init__(self, name):
        super(HasApple, self).__init__(name)

    def update(self):
        if True:
            return py_trees.common.Status.SUCCESS
        else:
            return py_trees.common.Status.FAILURE
# Other classes omitted…

has_apple = HasApple(name="Has apple")     # Tạo các node action và condition
```

```
eat_apple = EatApple(name="Eat apple")     # Tạo các node action và condition
sequence_1 = py_trees.composites.Sequence(name="Sequence 1", memory=True)
sequence_1.add_children([has_apple, eat_apple])  # Thêm các node vào node cha

has_pear = HasPear(name="Has pear")        # Tạo các node action và condition
eat_pear = EatPear(name="Eat pear")        # Tạo các node action và condition
sequence_2 = py_trees.composites.Sequence(name="Sequence 2", memory=True)
sequence_2.add_children([has_pear, eat_pear])    # Thêm các node vào node cha

root = py_trees.composites.Selector(name="Selector", memory=True)
root.add_children([sequence_1, sequence_2])      # Thêm các node vào node cha

behavior_tree = py_trees.trees.BehaviourTree(root)    # Tạo toàn bộ cây hành vi

py_trees.logging.level = py_trees.logging.Level.DEBUG
for i in range(1, 4):
    print("\n------------------ Tick {0} ------------------".format(i))
    behavior_tree.tick()            # Thực thi một bước (tick) trên cây hành vi

### Start of output
------------------ Tick 1 ------------------
[DEBUG] Selector             : Selector.tick()
[DEBUG] Selector             : Selector.tick() [!RUNNING->reset current_child]
[DEBUG] Sequence 1           : Sequence.tick()
[DEBUG] Has apple            : HasApple.tick()
[DEBUG] Has apple            : HasApple.stop(Status.INVALID->Status.SUCCESS)
[DEBUG] Eat apple            : EatApple.tick()
Eating apple
[DEBUG] Eat apple            : EatApple.stop(Status.INVALID->Status.SUCCESS)
[DEBUG] Sequence 1           : Sequence.stop()[Status.INVALID->Status.SUCCESS]
```

Code trong Listing 6.1 biểu diễn cây hành vi ở Hình 7.1. Bạn có thể chạy nguyên code này, hoặc sửa giá trị trả về của các condition rồi chạy lại cây. Bạn cũng có thể thay đổi cây bằng cách bỏ một trong hai node sequence khỏi selector gốc.

Giờ đã có hiểu biết cơ bản về cây hành vi, ta có thể chuyển sang làm việc với agent/assistant. Trước đó, ta xem một công cụ hỗ trợ làm việc với OpenAI Assistants. Công cụ này sẽ giúp ta bọc các ABT đầu tiên quanh OpenAI Assistants.

7.2 Khám phá GPT Assistants Playground

Trong quá trình viết sách, tác giả đã tạo một số dự án GitHub để giải quyết các khía cạnh khác nhau của việc xây dựng agent và assistant. Một trong số đó là GPT Assistants Playground, dùng Gradio làm giao diện, mô phỏng OpenAI Assistants Playground nhưng bổ sung thêm nhiều tính năng.

Dự án Playground được phát triển vừa làm công cụ giảng dạy vừa làm công cụ minh hoạ. Bên trong dự án, code Python dùng OpenAI Assistants API để tạo giao diện chat và một hệ thống agentic để xây dựng và vận hành các assistant. Dự án còn có một bộ sưu tập đầy đủ các action mà assistant có thể dùng, và bạn có thể dễ dàng thêm action của riêng mình.

7.2.1 Cài đặt và chạy Playground

Listing sau trình bày cách cài đặt và chạy dự án Playground từ terminal. Hiện chưa có gói PyPI để cài.

Listing 6.2 Cài đặt GPT Assistants Playground

```
# change to a working folder and create a new Python virtual environment
```

```
git clone https://github.com/cxbxmxcx/GPTAssistantsPlayground  # Kéo mã nguồn từ
↪  GitHub
cd GPTAssistantsPlayground     # Chuyển vào thư mục mã nguồn của dự án
pip install -r requirements.txt     # Cài các gói cần thiết
```

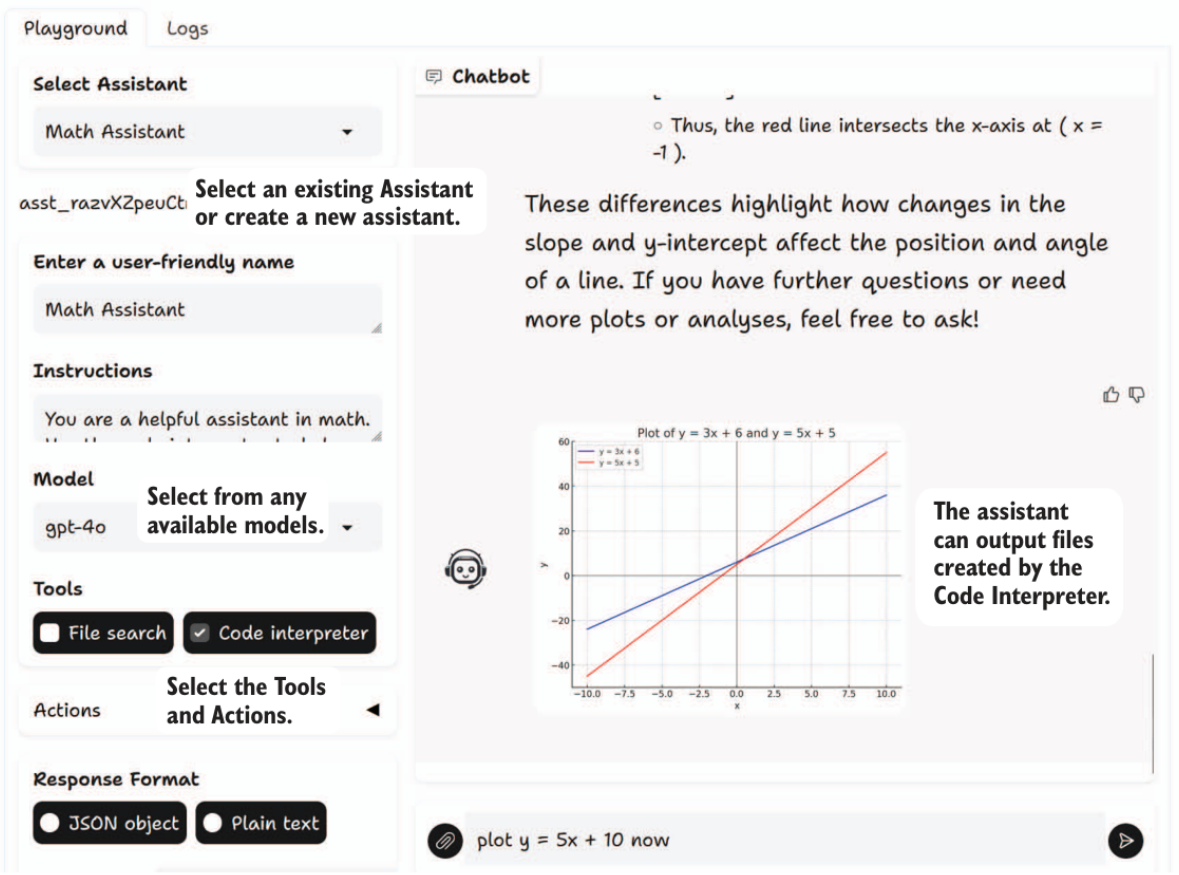
Bạn có thể chạy ứng dụng từ terminal hoặc bằng Visual Studio Code (VS Code); cách thứ hai cho bạn nhiều quyền kiểm soát hơn. Trước khi chạy ứng dụng, bạn cần đặt OpenAI API key qua dòng lệnh hoặc bằng cách tạo file `.env`, như ta đã làm vài lần trước đây. Listing 6.3 là ví dụ đặt biến môi trường trên Linux/Mac hoặc Git Bash shell (khuyên dùng trên Windows) và chạy ứng dụng.

Listing 6.3 Chạy GPT Assistants Playground

```
export OPENAI_API_KEY="your-api-key"     # Đặt API key làm biến môi trường
python main.py    # Chạy ứng dụng từ terminal hoặc qua VS Code
```

Mở trình duyệt tới URL được hiển thị (thường là `http://127.0.0.1:7860`) hoặc URL được in ra trong terminal. Bạn sẽ thấy một giao diện tương tự Hình 7.3. Nếu đã định nghĩa các OpenAI Assistant từ trước, bạn sẽ thấy chúng trong danh sách thả xuống Select Assistant.

Nếu chưa từng định nghĩa assistant nào, bạn có thể tạo một assistant và chọn các tuỳ chọn cùng chỉ dẫn (instruction) cần thiết. Nếu đã từng vào OpenAI Playground, bạn đã quen với một giao diện tương tự.



Hình 7.3: Giao diện GPT Assistants Playground đang được dùng để học toán

GPT so với assistant

OpenAI định nghĩa GPT là assistant mà bạn có thể chạy và dùng ngay trong giao diện ChatGPT. Còn assistant chỉ có thể được sử dụng qua API và trong đa số trường hợp cần code tuỳ biến. Khi chạy một assistant, bạn bị tính phí theo lượng token mô hình sử dụng và theo các tool đặc biệt, gồm Code Interpreter và file; trong khi GPT chạy trong ChatGPT và đã nằm trong chi phí tài khoản. Lý do tạo một phiên bản Playground chạy cục bộ là để làm bài tập minh hoạ cấu trúc code, đồng thời cung cấp thêm các tính năng sau:

- *Action (action tuỳ biến)*: tự tạo action cho phép bạn thêm bất kỳ chức năng nào mong muốn vào assistant. Như ta sẽ thấy, Playground giúp tạo action riêng rất nhanh và dễ.
- *Trình chạy code (code runner)*: API có sẵn Code Interpreter, nhưng khá đắt ($.03 mỗi lần chạy), không cho cài mô-đun riêng, không chạy code tương tác được và chạy chậm. Playground cho phép chạy code Python cục bộ trong một môi trường ảo cô lập. Dù không an toàn bằng việc đẩy code ra các Docker image, nó vẫn thực thi code trong cửa sổ riêng, ngoài tiến trình (out of process) tốt hơn các nền tảng khác.
- *Minh bạch và ghi log*: Playground ghi lại log đầy đủ và thậm chí còn cho thấy assistant dùng các tool/action nội bộ và bên ngoài như thế nào. Đây là cách rất tốt để xem assistant đang làm gì phía sau.

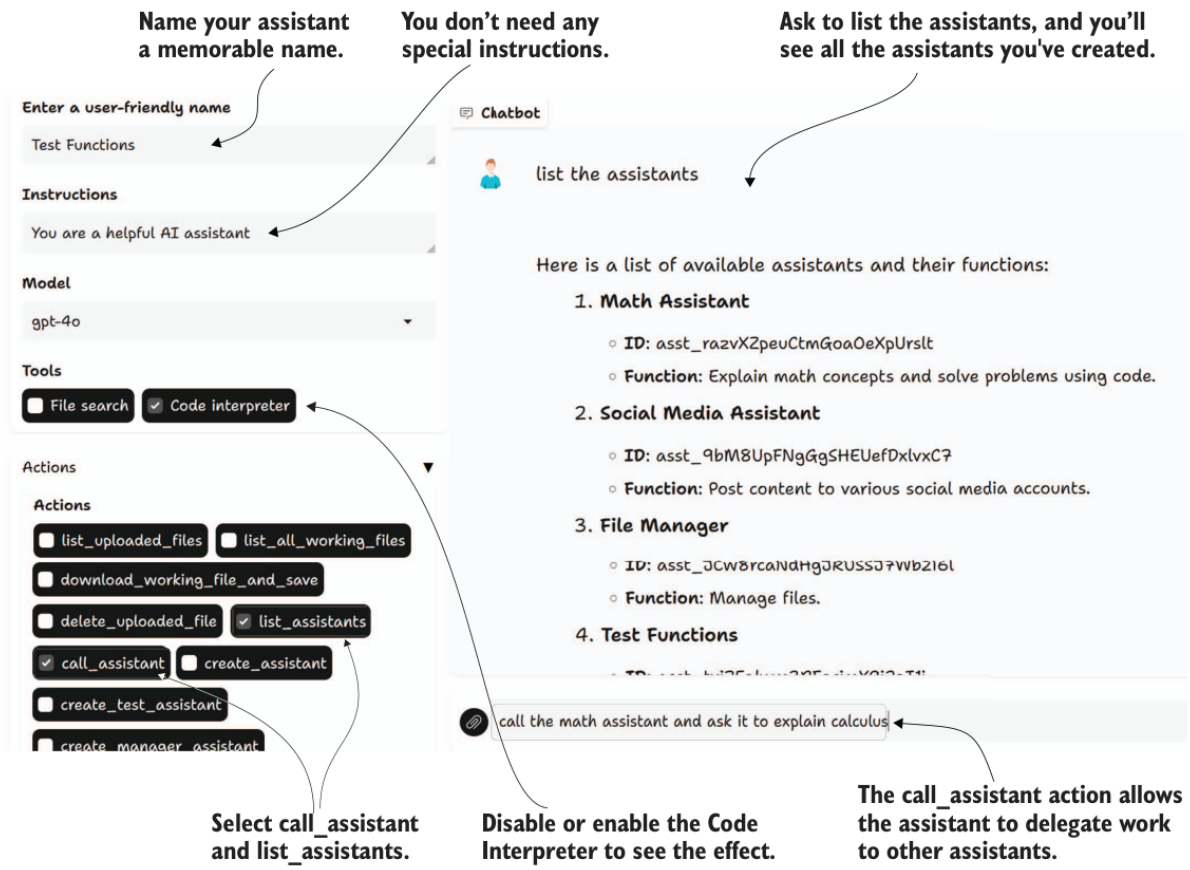
Từng tính năng này sẽ được trình bày chi tiết hơn trong vài mục tới. Ta bắt đầu với việc dùng và sử dụng action ở mục tiếp theo.

7.2.2 Sử dụng và xây dựng action tuỳ biến

Action và tool là những khối xây dựng trao năng lực cho agent và assistant. Không có quyền truy cập tool, agent chỉ là những chatbot không có chức năng gì. Nền tảng OpenAI là đơn vị dẫn đầu trong việc thiết lập nhiều mẫu cho tool, như ta đã thấy ở Chương 4.

Playground cung cấp một số action tuỳ biến có thể gắn vào assistant qua giao diện. Trong bài tập tiếp theo, ta sẽ xây một assistant đơn giản và gắn vài action tuỳ biến để xem có thể làm được gì.

Hình 7.4 cho thấy mục Actions (dạng accordion) đã mở rộng, hiển thị nhiều action tuỳ biến có sẵn. Chạy Playground từ terminal hoặc debugger, rồi tạo một assistant mới. Sau đó chọn các action như trong hình. Chọn xong action, cuộn xuống cuối và bấm Add Assistant để thêm assistant. Assistant cần được tạo trước khi có thể sử dụng.



Hình 7.4: Chọn và sử dụng action tuỳ biến trong giao diện

Sau khi tạo assistant, bạn có thể yêu cầu nó liệt kê mọi assistant hiện có. Việc liệt kê cũng cho bạn các ID cần thiết để gọi assistant. Bạn cũng có thể gọi các assistant khác và yêu cầu chúng hoàn thành những tác vụ thuộc chuyên môn của chúng.

Thêm action tuỳ biến của bạn đơn giản là thêm code vào một file và đặt file đó vào đúng thư mục. Mở thư mục `playground/assistant_actions` từ thư mục chính của dự án, bạn sẽ thấy một số file định nghĩa các action khác nhau. Mở file `file_actions.py` trong VS Code, như trong Listing 6.4.

Listing 6.4 playground/assistant_actions/file_actions.py

```
import os

from playground.actions_manager import agent_action

OUTPUT_FOLDER = "assistant_outputs"

@agent_action    # Decorator này tự động thêm function làm một action
def save_file(filename, content):     # Đặt tên function rõ ràng, khớp với mục
↪   đích
    """
    Save content to a file.
    :param filename: The name of the file including extension.
    :param content: The content to save in the file.
    """
    file_path = os.path.join(OUTPUT_FOLDER, filename)
    with open(file_path, "w", encoding="utf-8") as file:
        file.write(content)
```

```
    print(f"File '{filename}' saved successfully.")     # Thường trả về thông báo
    ↪  thành công hay thất bại
```

Chú thích cho phần docstring (không chèn comment vào trong chuỗi được):

- Dòng `Save content to a file.`: mô tả (docstring) là thứ assistant dùng để xác định function, nên hãy viết tài liệu thật kỹ.

Bạn có thể thêm bất kỳ action tuỳ biến nào bằng cách đặt file vào thư mục `assistant_actions` và gắn decorator `agent_action` cho function. Chỉ cần đặt cho function một cái tên tốt và viết tài liệu chất lượng về cách dùng function. Khi Playground khởi động, nó nạp mọi action trong thư mục được gắn decorator đúng và có mô tả/tài liệu.

Đơn giản vậy thôi. Bạn có thể thêm bao nhiêu action tuỳ biến tuỳ nhu cầu. Ở mục tiếp theo, ta xem một action tuỳ biến đặc biệt cho phép assistant chạy code cục bộ.

7.2.3 Cài đặt cơ sở dữ liệu assistant

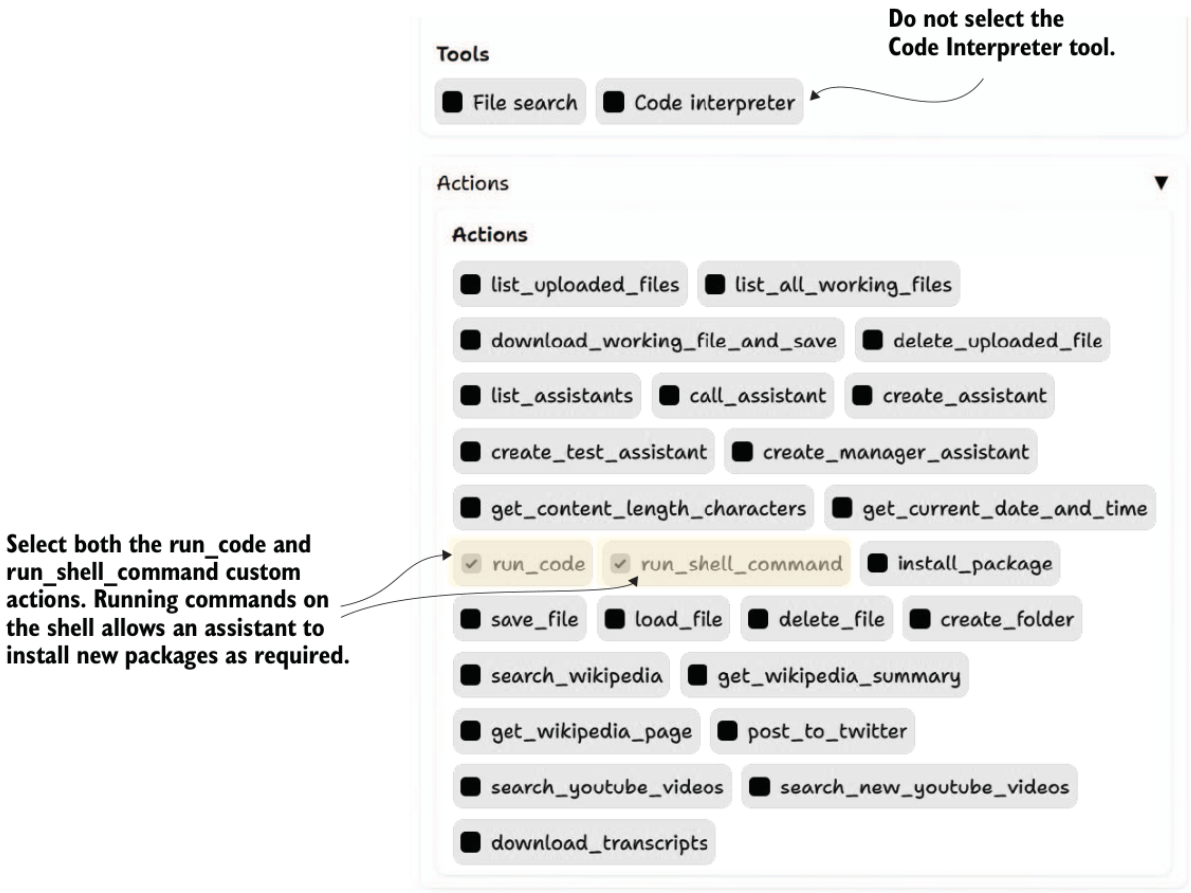
Để chạy một số ví dụ trong chương này, bạn cần cài cơ sở dữ liệu assistant. May là việc này làm được dễ dàng qua giao diện, chỉ bằng cách nhờ agent. Các hướng dẫn dưới đây mô tả quá trình cài assistant và được lấy nguyên từ README của GPT Assistants Playground. Bạn có thể cài một số assistant mẫu nằm trong cơ sở dữ liệu SQLite `assistants.db`:

1. Tạo một assistant mới, hoặc dùng một assistant có sẵn.
2. Gán cho assistant action `create_manager_assistant` (nằm trong mục Actions).
3. Yêu cầu assistant tạo manager assistant (ví dụ: “please create the manager assistant”), và nhớ đặt tên assistant là “Manager Assistant”.
4. Làm mới (refresh) trình duyệt để nạp lại danh sách chọn assistant.
5. Chọn Manager Assistant mới. Assistant này có các chỉ dẫn và action cho phép nó cài assistant từ cơ sở dữ liệu `assistants.db`.
6. Trò chuyện với Manager Assistant để nó đưa danh sách các assistant có thể cài, hoặc chỉ cần yêu cầu Manager Assistant cài tất cả assistant hiện có.

7.2.4 Cho assistant chạy code cục bộ

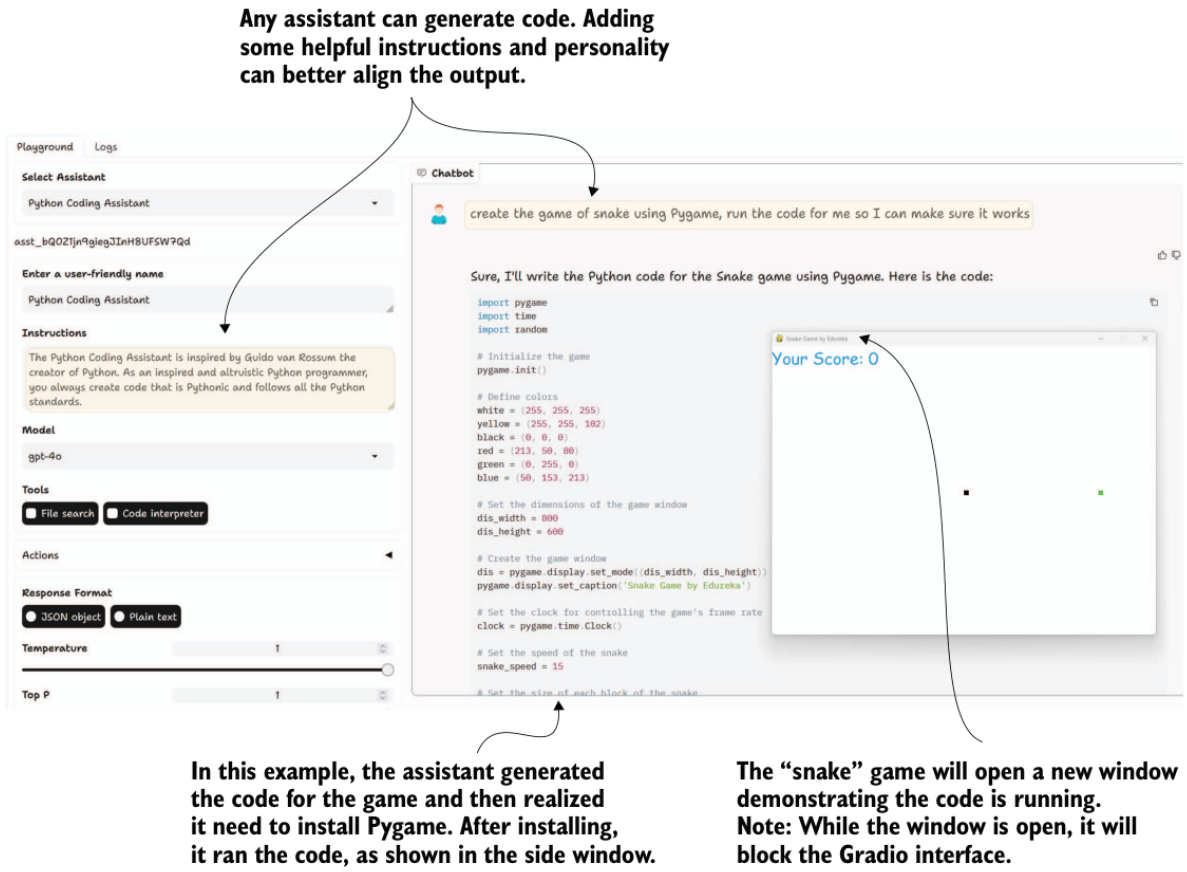
Cho agent và assistant sinh và chạy code thực thi được mang lại rất nhiều sức mạnh. Khác với Code Interpreter, chạy code cục bộ mở ra nhiều cơ hội lặp và tinh chỉnh nhanh. Ta đã thấy điều này với AutoGen, nơi các agent có thể chạy code liên tục cho đến khi nó hoạt động như mong đợi.

Trong Playground, chỉ cần chọn action tuỳ biến `run_code`, như trong Hình 7.5. Bạn cũng nên chọn action `run_shell_command` vì nó cho phép assistant `pip install` mọi mô-đun cần thiết.



Hình 7.5: Chọn các action tuỳ biến để assistant chạy code Python

Giờ bạn có thể yêu cầu assistant sinh code và chạy code đó để thay bạn kiểm tra nó hoạt động. Hãy thử bằng cách thêm các action tuỳ biến và yêu cầu assistant sinh rồi chạy code, như trong Hình 7.6. Nếu code không chạy như mong đợi, hãy cho assistant biết bạn gặp vấn đề gì.



Hình 7.6: Cho assistant sinh và chạy code Python

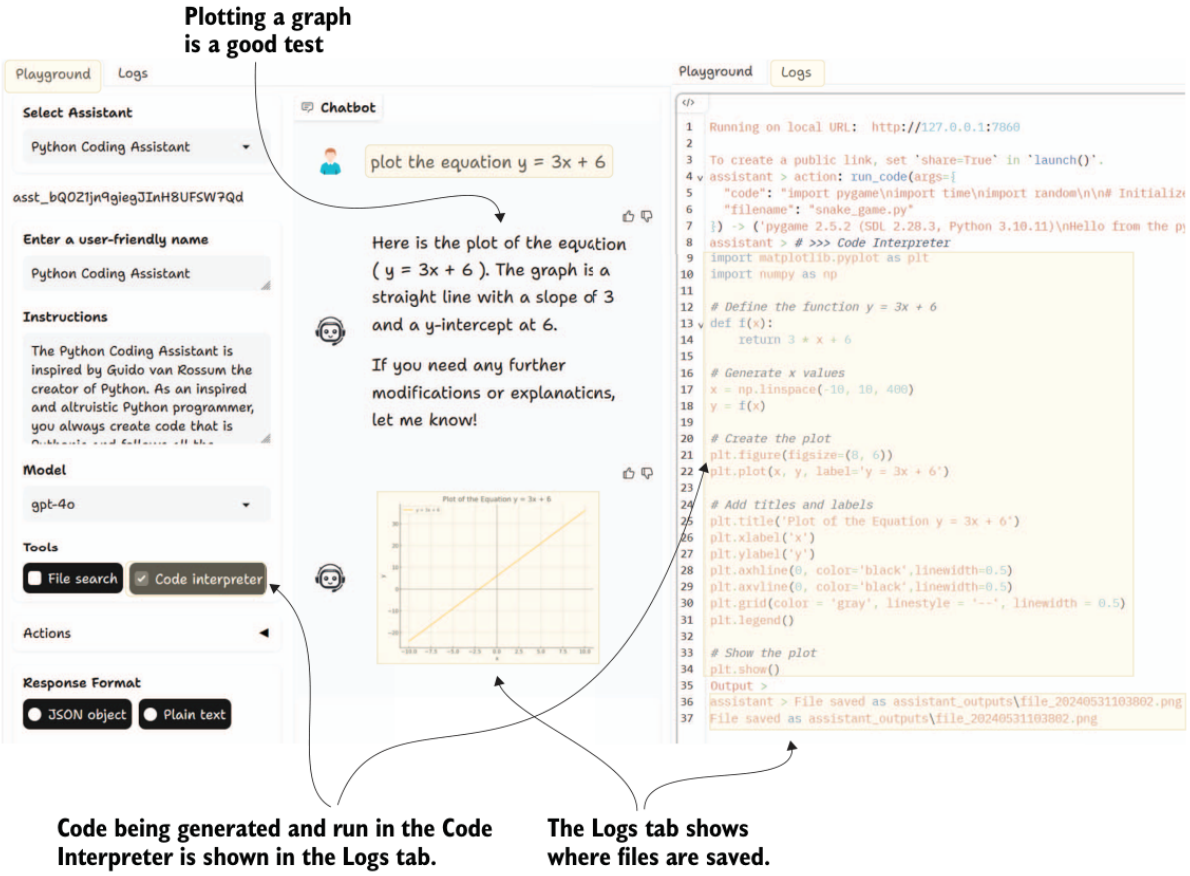
Một lần nữa, code Python chạy trong Playground tạo một môi trường ảo mới trong một thư mục con của dự án. Cách này hoạt động tốt nếu bạn không chạy code ở mức hệ điều hành hay code mức thấp. Nếu cần giải pháp chắc chắn hơn, một lựa chọn tốt là AutoGen, vốn dùng Docker container để chạy code cô lập.

Thêm action để chạy code hay làm các tác vụ khác có thể khiến assistant giống một hộp đen. May là OpenAI Assistants API cho phép bạn nhận các sự kiện (event) và xem assistant đang làm gì phía sau. Ở mục tiếp theo, ta xem điều này trông ra sao.

7.2.5 Theo dõi quá trình làm việc của assistant qua log

OpenAI đã thêm vào Assistants API một tính năng cho phép lắng nghe các sự kiện và action được nối chuỗi thông qua việc dùng tool/action. Tính năng này đã được tích hợp vào Playground, ghi lại việc dùng action và tool khi một assistant gọi một assistant khác.

Ta có thể thử bằng cách yêu cầu một assistant dùng một tool rồi mở log. Một ví dụ hay là gán cho assistant tool Code Interpreter rồi yêu cầu nó vẽ đồ thị một phương trình. Hình 7.7 là ví dụ của bài tập này.



Hình 7.7: Log nội bộ của assistant đang được ghi lại

Thông thường, khi bật tool Code Interpreter của assistant, bạn không thấy quá trình sinh hay thực thi code. Tính năng này cho phép bạn thấy mọi tool và action assistant dùng ngay khi chúng diễn ra. Nó không chỉ là công cụ chẩn đoán tuyệt vời mà còn cho thêm hiểu biết về cách LLM hoạt động.

Ta không xem lại code thực hiện tất cả những việc này vì nó khá đồ sộ và nhiều khả năng sẽ còn thay đổi vài lần. Dù vậy, nếu bạn định làm việc với Assistants API, dự án này là điểm khởi đầu tốt. Đã giới thiệu xong Playground, ta tiếp tục hành trình vào ABT ở mục tiếp theo.

7.3 Giới thiệu cây hành vi agentic

Cây hành vi agentic (ABT) cài đặt cây hành vi trên các hệ thống assistant và agent. Khác biệt chính giữa cây hành vi thông thường và ABT là ABT dùng prompt để điều khiển action và condition. Vì prompt có thể cho ra kết quả ngẫu nhiên với tần suất cao, ta cũng có thể gọi các cây này là cây hành vi *ngẫu nhiên* (stochastic behavior tree), loại cây vốn đã tồn tại. Để đơn giản, ta phân biệt các cây hành vi dùng để điều khiển agent bằng cách gọi chúng là *agentic*.

Tiếp theo, ta làm một bài tập tạo ABT. Cây hoàn chỉnh được viết bằng Python nhưng đòi hỏi thiết lập và cấu hình nhiều assistant. Ta sẽ xem cách quản lý assistant bằng chính các assistant.

7.3.1 Quản lý assistant bằng assistant

May là Playground có thể giúp ta quản lý và tạo assistant nhanh chóng. Trước hết ta cài Manager Assistant, sau đó cài các assistant đã được định nghĩa sẵn. Hãy bắt đầu cài Manager Assistant theo các bước sau:

1. Mở Playground trong trình duyệt và tạo một assistant đơn giản mới hoặc dùng một assistant có sẵn. Nếu cần assistant mới, hãy tạo rồi chọn nó.

2. Khi đã chọn assistant, mở accordion Actions và chọn action `create_manager_assistant`. Bạn không cần lưu; giao diện sẽ tự cập nhật assistant.

3. Trong giao diện chat, gửi prompt cho assistant: "Please create the manager assistant."

4. Sau vài giây, assistant sẽ báo đã xong. Làm mới trình duyệt và xác nhận Manager Assistant đã xuất hiện. Nếu vì lý do nào đó assistant mới không hiện ra, hãy thử khởi động lại chính ứng dụng Gradio.

Manager Assistant giống như một quản trị viên có quyền truy cập mọi thứ. Khi làm việc với Manager Assistant, hãy nêu yêu cầu thật cụ thể. Khi Manager Assistant đã hoạt động, bạn có thể cài các assistant mới dùng trong sách theo các bước sau:

1. Chọn Manager Assistant. Nếu đã sửa Manager Assistant, bạn có thể xoá và cài lại bất cứ lúc nào. Dù có thể có nhiều Manager Assistant, việc này không được khuyến khích.

2. Hỏi Manager Assistant những assistant nào có thể cài bằng cách gõ vào giao diện chat:

```
Please list all the installable assistants.
```

3. Chỉ rõ assistant bạn muốn cài khi yêu cầu Manager Assistant cài nó:

```
Please install the Python Coding Assistant.
```

Bạn có thể quản lý và cài bất kỳ assistant có sẵn nào bằng Playground. Bạn cũng có thể yêu cầu Manager Assistant lưu định nghĩa của mọi assistant thành JSON:

```
Please save all the assistants as JSON to a file called assistants.json.
```

Manager Assistant có thể truy cập mọi action, nên cần coi nó là trường hợp đặc biệt và dùng hạn chế. Khi tạo assistant, tốt nhất là giữ cho chúng hướng vào mục tiêu cụ thể và chỉ giới hạn ở những action chúng cần. Cách này không chỉ tránh bắt AI phải đưa ra quá nhiều quyết định mà còn tránh các sự cố hay sai lầm do ảo giác (hallucination) gây ra.

Khi làm các bài tập còn lại trong chương, nhiều khả năng bạn sẽ phải cài các assistant cần thiết. Hoặc bạn có thể yêu cầu Manager Assistant cài tất cả assistant hiện có. Dù theo cách nào, ở mục tiếp theo ta sẽ xem cách tạo một ABT với các assistant.

7.3.2 Xây dựng ABT cho thử thách lập trình

Thử thách lập trình (coding challenge) là một mốc cơ sở tốt để kiểm thử và đánh giá các hệ thống agent và assistant. Thử thách và benchmark có thể định lượng một agent hay hệ thống agentic hoạt động tốt đến đâu. Ta đã áp dụng thử thách lập trình cho các agent đa nền tảng ở Chương 5 với AutoGen và CrewAI.

Với thử thách lập trình lần này, ta đi xa hơn một chút và xem các thử thách lập trình Python trên trang Edabit (`https://edabit.com`), với độ khó từ người mới bắt đầu đến chuyên gia. Ta sẽ dùng các thử thách mức chuyên gia vì GPT-4o và các mô hình khác lập trình rất giỏi. Hãy xem thử thách trong listing tiếp theo và nghĩ xem bạn sẽ giải nó thế nào.

Listing 6.5 Thử thách Edabit: Plant the Grass

```
Plant the Grass by AniXDownLoe

    You will be given a matrix representing a field g
and two numbers x, y coordinate.

    There are three types of possible characters in the matrix:
```

```
        x representing a rock.
        o representing a dirt space.
        + representing a grassed space.

    You have to simulate grass growing from the position (x, y).
    Grass can grow in all four directions (up, left, right, down).
    Grass can only grow on dirt spaces and can't go past rocks.

    Return the simulated matrix.
    Examples

    simulate_grass([
    "xxxxxxx",
    "xooooox",
    "xxxxoox"
    "xoooxxx"
    "xxxxxxx"
    ], 1, 1) → [
    "xxxxxxx",
    "x+++++x",
    "xxxx++x"
    "xoooxxx"
    "xxxxxxx"
    ]

    Notes

    There will always be rocks on the perimeter
```

Bạn có thể dùng bất kỳ thử thách hay bài tập lập trình nào, nhưng cần cân nhắc vài điểm sau:

- Thử thách nên kiểm thử được bằng các assertion định lượng (đạt/không đạt).
- Tránh mở cửa sổ, chẳng hạn khi yêu cầu làm game, xây website hay dùng một giao diện khác. Đến một lúc nào đó sẽ có thể kiểm thử cả giao diện, nhưng hiện giờ chỉ có đầu ra văn bản.
- Tránh các thử thách chạy lâu, ít nhất là lúc đầu. Hãy bắt đầu với các thử thách ngắn gọn, chạy nhanh.

Cùng với thử thách, bạn cũng cần một bộ test hoặc assertion để xác nhận lời giải hoạt động. Trên Edabit, mỗi thử thách thường đi kèm một bộ test đầy đủ. Listing sau cho thấy các test bổ sung đi kèm thử thách.

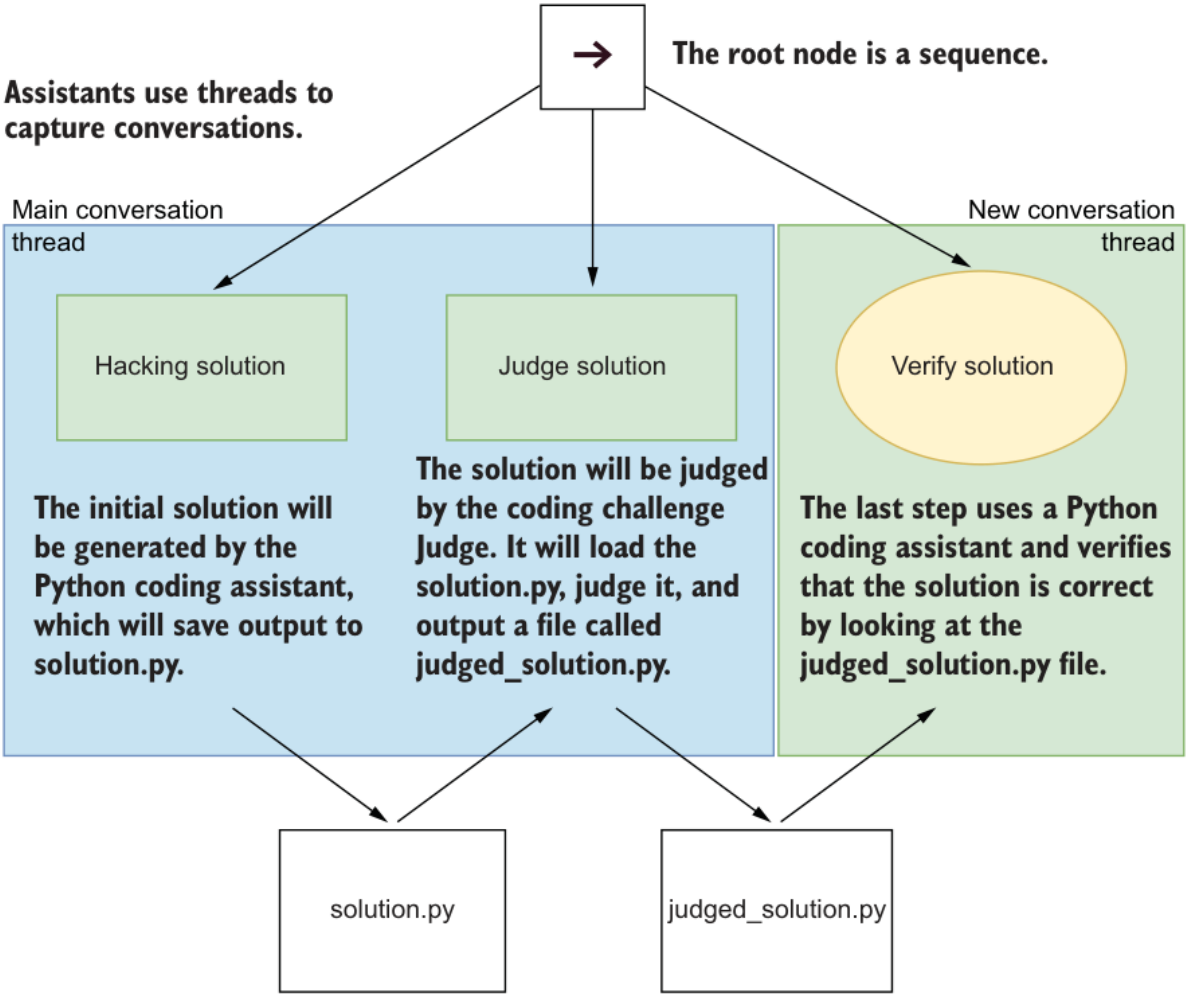
Listing 6.6 Các test của thử thách Plant the Grass

```
Test.assert_equals(simulate_grass(
["xxxxxxx","xooooox","xxxxoox","xoooxxx","xxxxxxx"],
 1, 1),
["xxxxxxx","x+++++x","xxxx++x","xoooxxx","xxxxxxx"])
    Test.assert_equals(simulate_grass(
["xxxxxxx","xoxooox","xxoooox","xooxxxx",
"xoxooox","xoxooox","xxxxxxx"],
 2, 3), ["xxxxxxx","xox+++x","xx++++x","x++xxxx",
"x+xooox","x+xooox","xxxxxxx"])
    Test.assert_equals(simulate_grass(
["xxxxxx","xoxoox","xxooox","xoooox","xooxxx","xxxxxx"],
1, 1),
["xxxxxx","x+xoox","xxooox","xoooox","xooxxx","xxxxxx"])
```

```
    Test.assert_equals(simulate_grass(
["xxxxx","xooox","xooox","xooox","xxxxx"],
1, 1),
["xxxxx","x+++x","x+++x","x+++x","xxxxx"])
    Test.assert_equals(simulate_grass(
["xxxxxx","xxxxox","xxooox","xoooxx","xooxxx",
"xooxxx","xxooox","xxxoxx","xxxxxx"],
4, 1),
["xxxxxx","xxxx+x","xx+++x","x+++xx","x++xxx",
"x++xxx","xx+++x","xxx+xx","xxxxxx"])
    Test.assert_equals(simulate_grass(
["xxxxxxxxxxx", "xoxooooooox", "xoxoxxxxxox",
"xoxoxoooxox", "xoxoxoxoxox", "xoxoxoxoxox",
"xoxoxxxoxox", "xoxooooooox", "xoxxxxxxxox",
"xooooooooox", "xxxxxxxxxxx"], 1, 1),
["xxxxxxxxxxx", "x+x++++++x", "x+x+xxxxx+x",
"x+x+x+++x+x", "x+x+x+x+x+x", "x+x+x+x+x+x",
"x+x+xxx+x+x", "x+x+++++x+x", "x+xxxxxxx+x",
"x+++++++++x", "xxxxxxxxxxx"])
```

Các test này được chạy như một phần của quy trình xác minh hai bước để xác nhận lời giải hoạt động. Ta cũng dùng nguyên văn thử thách và test như tác giả Edabit viết, nhờ đó kiểm tra AI thêm một lần nữa.

Hình 7.8 cho thấy cấu trúc của một cây hành vi đơn giản dùng để giải nhiều thử thách lập trình. Bạn sẽ thấy ABT này dùng một assistant khác nhau cho các action và condition. Ở bước đầu tiên, assistant lập trình Python (gọi là Hacker) sinh ra một lời giải. Lời giải này được người chấm thử thách lập trình (gọi là Judge) xem xét và tinh chỉnh, rồi được xác minh bởi một assistant lập trình Python khác (gọi là Verifier).



Hình 7.8: ABT cho thử thách lập trình

Hình 7.8 cũng cho thấy mỗi agent hội thoại trên thread nào. Các assistant dùng message thread, tương tự một kênh Slack hay Discord, nơi mọi assistant trò chuyện trên cùng một thread đều thấy toàn bộ tin nhắn. Với ABT này, ta giữ một thread hội thoại chính để Hacker và Judge chia sẻ tin nhắn, còn Verifier làm việc trên một message thread riêng. Việc để Verifier trên thread riêng cô lập nó khỏi phần nhiễu của quá trình tìm lời giải.

Giờ việc xây ABT bằng code là kết hợp gói `py_trees` với các hàm API của Playground. Listing 6.7 là đoạn code trích tạo từng node action/condition với các assistant và đưa chỉ dẫn cho chúng.[2]

Listing 6.7 agentic_btree_coding_challenge.py

```
root = py_trees.composites.Sequence("RootSequence", memory=True)

thread = api.create_thread()    # Tạo một message thread cho Hacker và Judge dùng
↪  chung
challenge = textwrap.dedent("""

""")
judge_test_cases = textwrap.dedent("""
```

[2]Ghi chú của người dịch: trong sách, một số chú thích bên lề của Listing 6.7 bị lặp cùng một câu ("Creates a message thread that will be shared by the Hacker and Judge"). Bản dịch đặt chú thích đó ở dòng `thread = api.create_thread()` và chuyển nghĩa thành "dùng thread chung của Hacker và Judge" ở các dòng `thread=thread`, vì đó là những dòng thực sự truyền thread vào action.

```
""")

hacker = create_assistant_action_on_thread(   # Tạo action Hacker trên thread
    thread=thread,     # Dùng thread chung của Hacker và Judge
    action_name="Hacker",
    assistant_name="Python Coding Assistant",
    assistant_instructions=textwrap.dedent(f"""
    Challenge goal:
    {challenge}
    Solve the challenge and output the
final solution to a file called solution.py
    """),
)
root.add_child(hacker)

judge = create_assistant_action_on_thread(    # Tạo action Judge trên thread
    thread=thread,     # Dùng thread chung của Hacker và Judge
    action_name="Judge solution",
    assistant_name="Coding Challenge Judge",
    assistant_instructions=textwrap.dedent(
        f"""
    Challenge goal:
    {challenge}
    Load the solution from the file solution.py.
    Then confirm is a solution to the challenge
and test it with the following test cases:
    {judge_test_cases}
    Run the code for the solution and confirm it passes all the test cases.
    If the solution passes all tests save the solution to a file called
judged_solution.py
    """,
    ),
)
root.add_child(judge)

# verifier operates on a different thread, essentially in closed room
verifier = create_assistant_condition(    # Lời gọi này tạo một message thread
↪   mới
    condition_name="Verify solution",
    assistant_name="Python Coding Assistant",
    assistant_instructions=textwrap.dedent(
        f"""
    Challenge goal:
    {challenge}
    Load the file called judged_solution.py and
verify that the solution is correct by running the code and confirm it passes
all the test cases:
    {judge_test_cases}
    If the solution is correct, return only the single word SUCCESS,
     otherwise
return the single word FAILURE.
    """,
    ),
)
root.add_child(verifier)

tree = py_trees.trees.BehaviourTree(root)
```

```
while True:
    tree.tick()
    time.sleep(20)     # Có thể tăng giảm thời gian ngủ; dùng để điều tiết số tin
    ↪  nhắn gửi tới LLM
    if root.status == py_trees.common.Status.SUCCESS:   # Quá trình tiếp tục cho
    ↪  đến khi xác minh thành công
        break
### Required assistants -
### Python Coding Assistant and Coding Challenge Judge
### install these assistants through the Playground
```

Các chú thích nằm bên trong chuỗi prompt (không chèn comment được vào chuỗi):

- Dòng `{challenge}` (ở cả Hacker, Judge và Verifier): thử thách như trong Listing 6.5.
- Dòng `{judge_test_cases}` (ở cả Judge và Verifier): các test như trong Listing 6.6.

Chạy ABT bằng cách mở file trong VS Code hoặc dùng dòng lệnh. Hãy theo dõi đầu ra trong terminal và quan sát các assistant làm việc qua từng bước của cây. Nếu lời giải không được xác minh ở node condition, quá trình vẫn tiếp tục theo cây. Ngay cả với lời giải đơn giản này, bạn cũng có thể nhanh chóng tạo ra nhiều biến thể. Bạn có thể mở rộng cây với thêm node/bước và cây con. Chẳng hạn, bạn có thể muốn một đội Hacker để phân tích và chia nhỏ thử thách.

Công việc của ví dụ này chủ yếu do code của Playground đảm nhiệm, qua các hàm trợ giúp `create_assistant_condition` và `create_assistant_action_on_thread`. Code này dùng một vài class để tích hợp code cây hành vi `py_trees` với code OpenAI Assistants được Playground bọc lại. Hãy xem code trong dự án nếu bạn muốn hiểu các chi tiết mức thấp hơn.

7.3.3 Hệ AI hội thoại so với các phương pháp khác

Ta đã xem các hệ đa agent hội thoại ở Chương 5 khi tìm hiểu AutoGen. ABT có thể hoạt động bằng cách kết hợp hội thoại (trên thread) với các phương pháp khác, chẳng hạn chia sẻ file. Để các assistant/agent chuyển file cho nhau giúp giảm số suy nghĩ/hội thoại nhiễu và lặp lại. Ngược lại, hệ hội thoại hưởng lợi từ các hành vi nổi trội (emergent behavior) có thể xuất hiện. Vì vậy, dùng cả hai cách có thể giúp phát triển cơ chế điều khiển và lời giải tốt hơn.

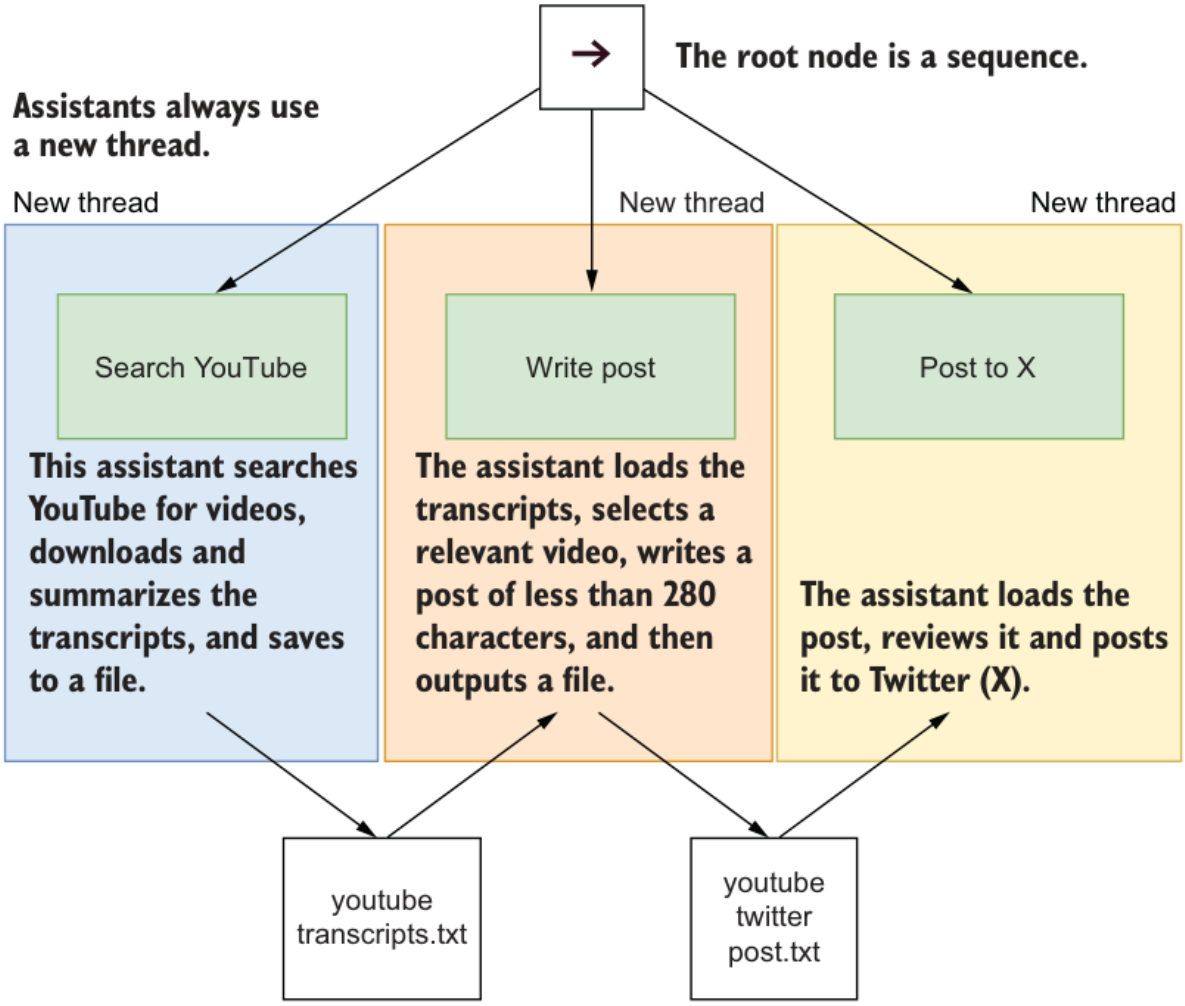
Lời giải đơn giản trong Listing 6.7 có thể được mở rộng để xử lý các thử thách lập trình thực tế hơn, thậm chí hoạt động như một ABT lập trình. Ở mục tiếp theo, ta xây một ABT khác để xử lý một bài toán khác.

7.3.4 Đăng video YouTube lên X

Trong bài tập của mục này, ta xem một ABT có thể làm những việc sau:

1. Tìm video trên YouTube theo một chủ đề cho trước và trả về các video mới nhất.
2. Tải transcript của mọi video mà lần tìm kiếm trả về.
3. Tóm tắt các transcript.
4. Xem xét các transcript đã tóm tắt và chọn một video để viết bài đăng trên X (trước đây là Twitter).
5. Viết một bài đăng hấp dẫn, thú vị về video, đảm bảo ngắn hơn 280 ký tự.
6. Xem xét bài đăng rồi đăng nó lên X.

Hình 7.9 cho thấy ABT được lắp ráp từ các assistant khác nhau. Trong bài tập này, ta dùng node sequence làm gốc, và mỗi assistant thực hiện một action khác nhau. Ngoài ra, để đơn giản, mỗi lần tương tác của assistant luôn diễn ra trong một thread mới. Cách này cô lập tương tác của từng assistant thành một cuộc hội thoại ngắn gọn, dễ gỡ lỗi hơn nếu có sự cố.



Hình 7.9: ABT đăng bài mạng xã hội từ YouTube

7.3.5 Thiết lập X cần thiết

Nếu định chạy code của bài tập này, bạn phải thêm thông tin đăng nhập X vào file `.env`. File `.env.default` cho ví dụ về định dạng cần có, như trong Listing 6.8. Bạn không bắt buộc nhập thông tin đăng nhập. Khi đó bước cuối là đăng bài sẽ thất bại, nhưng bạn vẫn có thể xem file `youtube_twitter_post.txt` để thấy nội dung đã được sinh ra.

Listing 6.8 Cấu hình thông tin đăng nhập

```
X_EMAIL = "twitter email here"
X_USERNAME = "twitter username here"
X_PASSWORD = "twitter password here"
```

Listing 6.9 chỉ trình bày code tạo các action của assistant. ABT này dùng ba assistant khác nhau, mỗi assistant có chỉ dẫn tác vụ riêng. Lưu ý mỗi assistant có một bộ chỉ dẫn riêng xác định vai trò của nó. Bạn có thể xem chỉ dẫn của từng assistant bằng Playground.

Tìm kiếm YouTube và spam

Nếu định chạy bài tập này thật và để nó đăng lên tài khoản X của bạn, hãy lưu ý YouTube có phần nào vấn đề spam. Các assistant đã được cấu hình để cố tránh video spam, nhưng vẫn có thể có một ít lọt qua. Xây dựng một ABT hoạt động được, có thể lội qua đống video mà vẫn tránh spam, có những ứng dụng phù hợp.

Listing 6.9 agentic_btree_video_poster_v1.py

```
root = py_trees.composites.Sequence("RootSequence", memory=True)

search_term = "GPT Agents"
search_youtube_action = create_assistant_action(
    action_name=f"Search YouTube({search_term})",
    assistant_name="YouTube Researcher v2",
    assistant_instructions=f"""
    Search Term: {search_term}
    Use the query "{search_term}" to search for videos on YouTube.
    then for each video download the transcript and summarize it
for relevance to {search_term}
    be sure to include a link to each of the videos,
    and then save all summarizations to a file called youtube_transcripts.txt
    If you encounter any errors, please return just the word FAILURE.
    """,
)
root.add_child(search_youtube_action)

write_post_action = create_assistant_action(
    action_name="Write Post",
    assistant_name="Twitter Post Writer",
    assistant_instructions="""
    Load the file called youtube_transcripts.txt,
    analyze the contents for references to search term at the top and
then select
    the most exciting and relevant video related to:
    educational, entertaining, or informative, to post on Twitter.
    Then write a Twitter post that is relevant to the video,
    and include a link to the video, along
    with exciting highlights or mentions,
    and save it to a file called youtube_twitter_post.txt.
    If you encounter any errors, please return just the word FAILURE.
    """,
)
root.add_child(write_post_action)

post_action = create_assistant_action(
    action_name="Post",
    assistant_name="Social Media Assistant",
    assistant_instructions="""
    Load the file called youtube_twitter_post.txt and post the content
to Twitter.
    If the content is empty please do not post anything.
    If you encounter any errors, please return just the word FAILURE.
    """,
)
root.add_child(post_action)
### Required assistants - YouTube Researcher v2, Twitter Post Writer,
```

```
and Social Media Assistant – install these assistants through the Playground
```

Chạy code như bình thường, và sau vài phút một bài đăng mới sẽ xuất hiện trong thư mục `assistants_output`. Hình 7.10 là ví dụ một bài đăng được sinh ra bởi ABT này. Chạy ABT này để sinh nhiều hơn vài bài mỗi ngày có thể, và nhiều khả năng sẽ, khiến tài khoản X của bạn bị chặn. Nếu đã cấu hình thông tin đăng nhập X, bạn sẽ thấy bài đăng xuất hiện trên feed của mình.

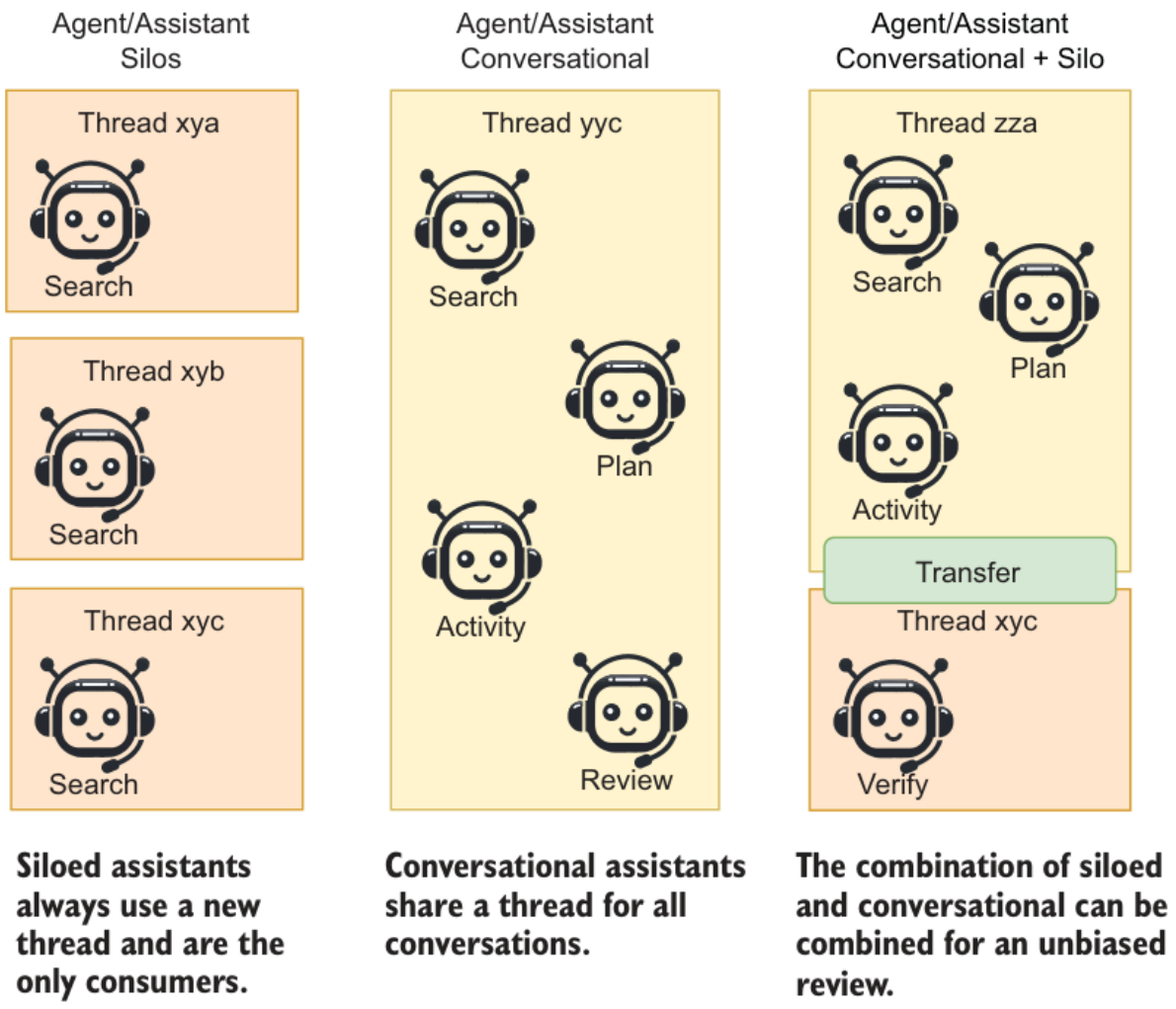
Hình 7.10: Một bài đăng X mẫu do ABT sinh ra

ABT này chỉ để minh hoạ, không dùng cho môi trường production hay sử dụng lâu dài. Các điểm chính của bản minh hoạ là cho thấy việc tìm kiếm và nạp dữ liệu, tóm tắt và lọc, rồi sinh nội dung mới, và cuối cùng làm nổi bật nhiều action tuỳ biến cùng việc tích hợp với các API.

7.4 Xây dựng hệ đa agent hội thoại tự hành

Khía cạnh hội thoại của hệ đa agent có thể thúc đẩy các cơ chế như phản hồi (feedback), suy luận và hành vi nổi trội. Điều khiển agent bằng ABT cô lập (silo) các assistant/agent có thể hiệu quả để kiểm soát các quy trình có cấu trúc, như ta đã thấy trong ví dụ đăng bài YouTube. Tuy nhiên, ta cũng không muốn bỏ lỡ lợi ích của hội thoại giữa các agent/assistant.

May là Playground cung cấp các cách để cô lập hoặc ghép các assistant vào các thread hội thoại. Hình 7.11 cho thấy các assistant có thể được cô lập hoặc trộn theo nhiều tổ hợp vào các thread. Kết hợp cô lập với hội thoại cho ra điểm mạnh của cả hai mẫu.



Hình 7.11: Các cách bố trí khác nhau của assistant cô lập và assistant hội thoại

Ta xét một bài tập đơn giản nhưng thực tế để minh hoạ hiệu quả của mẫu hội thoại. Trong bài tập tiếp theo, ta dùng hai assistant trong một ABT trò chuyện trên cùng một thread. Listing sau cho thấy việc dựng cây bằng code cùng các assistant tương ứng.

Listing 6.10 agentic_conversation_btree.py

```
root = py_trees.composites.Sequence("RootSequence", memory=True)
bug_file = """
# code not shown
"""

thread = api.create_thread()    # Tạo một message thread để các assistant dùng
↪  chung và trò chuyện trên đó

debug_code = create_assistant_action_on_thread(    # Tạo action debug code với
↪  một assistant đặc biệt
    thread=thread,
    action_name="Debug code",
    assistant_name="Python Debugger",
    assistant_instructions=textwrap.dedent(f"""
    Here is the code with bugs in it:
    {bug_file}
    Run the code to identify the bugs and fix them.
```

```
    Be sure to test the code to ensure it runs without errors or throws
any exceptions.
    """),
)
root.add_child(debug_code)

verify = create_assistant_condition_on_thread(    # Tạo condition xác minh xem
↪  code đã được sửa chưa
    thread=thread,
    condition_name="Verify",
    assistant_name="Python Coding Assistant",
    assistant_instructions=textwrap.dedent(
        """
    Verify the solution fixes the bug and there are no more issues.
    Verify that no exceptions are thrown when the code is run.
    Reply with SUCCESS if the solution is correct, otherwise return FAILURE.
    If you are happy with the solution, save the code to a file called
fixed_bug.py.
    """,
    ),
)
root.add_child(verify)
tree = py_trees.trees.BehaviourTree(root)
while True:
    tree.tick()
    if root.status == py_trees.common.Status.SUCCESS:
        break   # Cây tiếp tục chạy cho đến khi sequence gốc hoàn thành với
        ↪  success
    time.sleep(20)
```

Cây gồm ba node: sequence gốc, action debug code và condition xác minh bản sửa. Vì gốc của cây là một sequence, hai assistant sẽ tiếp tục làm việc lần lượt cho đến khi cả hai cùng trả về success. Cả hai assistant trò chuyện trên cùng một thread nhưng vẫn được điều khiển theo cách cho phản hồi liên tục.

Chạy bài tập bằng cách mở file trong VS Code, hoặc thực thi trực tiếp từ dòng lệnh. Code ví dụ có vài lỗi và vấn đề nhỏ mà các assistant sẽ xử lý để sửa. Sau khi ABT chạy xong thành công, bạn có thể mở file `assistants_output/fixed_bug.py` và kiểm tra kết quả đều ổn.

Đến đây ta đã thấy một vài ABT hoạt động và hiểu các sắc thái của việc dùng cô lập hay hội thoại. Mục tiếp theo sẽ dạy một số kỹ thuật để xây ABT của riêng bạn.

7.5 Xây dựng ABT bằng back chaining

Back chaining (suy diễn lùi) là phương pháp bắt nguồn từ logic và suy luận, giúp xây cây hành vi bằng cách đi ngược từ mục tiêu. Mục này dùng quy trình back chaining để dựng một ABT làm việc nhằm đạt mục tiêu. Danh sách sau mô tả quy trình chi tiết hơn:

1. *Xác định hành vi mục tiêu.* Bắt đầu từ hành vi bạn muốn agent thực hiện.
2. *Xác định các action cần thiết.* Nhận diện các action dẫn tới hành vi mục tiêu.
3. *Xác định các condition.* Xác định các điều kiện phải thoả để mỗi action thành công.
4. *Xác định cách giao tiếp.* Quyết định các assistant truyền thông tin cho nhau thế nào. Các assistant bị cô lập hay trò chuyện trên thread, hay một tổ hợp các mẫu thì tốt hơn?
5. *Dựng cây.* Bắt đầu xây cây hành vi từ hành vi mục tiêu, thêm các node action và condition một cách đệ quy cho đến khi mọi condition cần thiết đều liên kết được với các trạng thái hay sự kiện đã biết.

Cây hành vi thường dùng một mẫu gọi là *blackboard* (bảng đen) để giao tiếp giữa các node. Blackboard, như trong `py_trees`, dùng một kho khoá/giá trị (key/value) để lưu thông tin và cho các node truy cập. Nó cũng cung cấp một số cơ chế kiểm soát, chẳng hạn giới hạn quyền truy cập cho các node cụ thể.

Ở đây ta chọn dùng file để giao tiếp vì sự đơn giản và minh bạch của chúng. Đến một lúc nào đó, các hệ agentic được kỳ vọng sẽ tiêu thụ nhiều thông tin hơn hẳn và ở nhiều định dạng hơn so với những gì blackboard được thiết kế. Blackboard hoặc phải trở nên tinh vi hơn, hoặc phải được tích hợp với các giải pháp lưu trữ file.

Ta xây một ABT bằng back chaining. Có thể chọn nhiều mục tiêu, nhưng một mục tiêu thú vị và có phần "meta" là xây một ABT giúp tạo ra assistant. Vậy trước hết ta nêu mục tiêu bằng câu "Create an assistant that can help me do {task}":

- *Các action cần thiết* (đi ngược): tạo assistant; xác minh assistant; kiểm thử assistant; đặt tên assistant; đưa cho assistant các chỉ dẫn liên quan.
- *Condition đã xác định*: xác minh assistant.
- *Xác định mẫu giao tiếp*: để cho thú vị, ta chạy mọi assistant trên cùng một message thread.
- *Dựng cây*: để dựng cây, trước hết ta đảo ngược thứ tự các action và đánh dấu từng phần tử là action hay condition:
  - (action) Đưa cho assistant các chỉ dẫn liên quan để giúp người dùng với một tác vụ cho trước.
  - (action) Đặt tên cho assistant.
  - (action) Kiểm thử assistant.
  - (condition) Xác minh assistant.
  - (action) Tạo assistant.

Dĩ nhiên, cách đơn giản để dựng cây lúc này là hỏi ChatGPT hoặc một mô hình đủ năng lực khác. Kết quả khi nhờ ChatGPT tạo cây được cho trong listing sau. Bạn cũng có thể tự làm ra cây và có thể đưa thêm các yếu tố khác.

Listing 6.11 ABT để xây dựng một assistant

```
Root
│
├── Sequence
│    ├── Action: Give the assistant relevant instructions to help a user
with a given task
│    ├── Action: Name the assistant
│    ├── Action: Test the assistant
│    ├── Condition: Verify the assistant
│    └── Action: Create the assistant
```

Từ đây, ta có thể bắt đầu dựng cây bằng cách lặp qua từng node action và condition và xác định assistant cần những chỉ dẫn nào. Việc này cũng có thể gồm bất kỳ tool và action tuỳ biến nào, kể cả những cái bạn có thể cần tự phát triển. Ở lượt đầu, hãy để chỉ dẫn ở mức tổng quát. Lý tưởng là ta muốn tạo càng ít assistant càng tốt.

Sau khi xác định assistant, tool và action cho từng assistant và cho từng tác vụ, bạn có thể thử tổng quát hoá thêm. Hãy nghĩ xem có thể gộp các action ở đâu để giảm số assistant. Bắt đầu đánh giá với quá ít assistant thì tốt hơn là quá nhiều. Tuy nhiên, hãy giữ sự phân chia công việc hợp lý theo tác vụ: chẳng hạn, kiểm thử và xác minh tốt nhất nên do các assistant khác nhau làm.

7.6 Bài tập

Hoàn thành các bài tập sau để củng cố kiến thức trong chương:

Bài tập 1: Tạo ABT lập kế hoạch du lịch (Travel Planner)

Mục tiêu: xây một cây hành vi agentic (ABT) để lập lịch trình du lịch bằng các assistant.
Nhiệm vụ:

- Cài đặt GPT Assistants Playground trên máy của bạn.
- Tạo một ABT để lập lịch trình du lịch. Cây có cấu trúc sau:
  - Action: dùng assistant Travel để thu thập thông tin về các điểm đến tiềm năng.
  - Action: dùng Itinerary Planner để tạo kế hoạch du lịch từng ngày.
  - Condition: xác minh tính đầy đủ và tính khả thi của lịch trình bằng một Travel Assistant khác.
- Triển khai và chạy ABT để tạo ra một lịch trình du lịch hoàn chỉnh.

Bài tập 2: Xây ABT tự động hoá hỗ trợ khách hàng

Mục tiêu: tạo một ABT tự động hoá các phản hồi hỗ trợ khách hàng bằng các assistant.
Nhiệm vụ:

- Cài đặt GPT Assistants Playground trên máy của bạn.
- Tạo một ABT với cấu trúc sau:
  - Action: dùng assistant Customer Query Analyzer để phân loại các truy vấn của khách hàng.
  - Action: dùng assistant Response Generator để soạn phản hồi dựa trên các loại truy vấn.
  - Action: dùng assistant Customer Support để gửi phản hồi cho khách hàng.
- Triển khai và chạy ABT để tự động hoá quá trình phân tích và trả lời các truy vấn của khách hàng.

Bài tập 3: Quản lý tồn kho bằng ABT

Mục tiêu: học cách tạo và quản lý mức tồn kho bằng ABT.
Nhiệm vụ:

- Cài đặt GPT Assistants Playground trên máy của bạn.
- Tạo một ABT quản lý tồn kho cho một doanh nghiệp bán lẻ:
  - Action: dùng assistant Inventory Checker để xem lại mức tồn kho hiện tại.
  - Action: dùng assistant Order để đặt hàng cho các mặt hàng sắp hết.
  - Condition: xác minh các đơn hàng đã được đặt đúng và cập nhật hồ sơ tồn kho.
- Triển khai và chạy ABT để quản lý tồn kho một cách linh hoạt.

Bài tập 4: Tạo ABT huấn luyện viên thể hình cá nhân

Mục tiêu: tạo một ABT cung cấp giáo án tập luyện thể hình cá nhân hoá bằng các assistant.
Nhiệm vụ:

- Cài đặt GPT Assistants Playground trên máy của bạn.
- Tạo một ABT để xây dựng kế hoạch thể hình cá nhân hoá:
  - Action: dùng assistant Fitness Assessment để đánh giá mức thể lực hiện tại của người dùng.
  - Action: dùng Training Plan Generator để tạo kế hoạch thể hình tuỳ biến dựa trên kết quả đánh giá.
  - Condition: xác minh tính phù hợp và an toàn của kế hoạch bằng một assistant Fitness khác.
- Triển khai và chạy ABT để tạo và kiểm chứng một giáo án thể hình cá nhân hoá.

Bài tập 5: Dùng back chaining để xây ABT cố vấn tài chính

Mục tiêu: áp dụng back chaining để dựng một ABT cung cấp lời khuyên tài chính và chiến lược đầu tư.
Nhiệm vụ:

- Cài đặt GPT Assistants Playground trên máy của bạn.
- Xác định mục tiêu sau: “Create an assistant that can provide financial advice and investment strategies.”
- Dùng back chaining để xác định các action và condition cần có nhằm đạt mục tiêu này.
- Triển khai và chạy ABT để tạo ra một dịch vụ cố vấn tài chính toàn diện, bằng cách dùng back chaining để dựng các action và condition nền tảng cho cây.

Tóm tắt

- Cây hành vi là một mẫu điều khiển AI mạnh mẽ và có khả năng mở rộng, do Rodney A. Brooks lần đầu đưa vào robot. Chúng được dùng rộng rãi trong game và robot nhờ tính mô-đun và khả năng tái sử dụng.
- Các node chính của cây hành vi là selector, sequence, condition, action, decorator và parallel. Selector giống khối “or”; sequence thực thi các node lần lượt; condition kiểm tra trạng thái; action thực hiện công việc; decorator là lớp bọc; parallel cho phép thực thi song song.
- Hiểu luồng thực thi của cây hành vi có thể là điều then chốt để thiết kế, xây dựng và vận hành chúng, nhằm tạo ra sự kiểm soát với các đường ra quyết định rõ ràng.
- Ưu điểm của cây hành vi gồm tính mô-đun, khả năng mở rộng, tính linh hoạt, dễ gỡ lỗi và tách rời logic quyết định, giúp chúng phù hợp với các hệ AI phức tạp.
- Thiết lập và chạy một cây hành vi đơn giản bằng Python đòi hỏi đặt tên và ghi tài liệu đúng cho các node tuỳ biến.
- Dự án GPT Assistants Playground là giao diện dựa trên Gradio mô phỏng OpenAI Assistants Playground, kèm các tính năng bổ sung phục vụ giảng dạy và minh hoạ ABT.

- GPT Assistants Playground cho phép tạo và quản lý các action tuỳ biến, điều cốt yếu để xây các assistant đa năng.
- ABT điều khiển agent và assistant bằng cách dùng prompt để chỉ đạo các action và condition của assistant. ABT tận dụng sức mạnh của LLM để tạo ra các hệ động và tự hành.
- Back chaining là phương pháp dựng cây hành vi bằng cách đi ngược từ hành vi mục tiêu. Quy trình gồm xác định các action, condition và mẫu giao tiếp cần thiết, rồi dựng cây từng bước.
- Hệ agentic hưởng lợi từ cả mẫu cô lập và mẫu hội thoại khi giao tiếp giữa các thực thể. ABT có thể hưởng lợi từ việc kết hợp assistant cô lập và assistant hội thoại để tận dụng cả quy trình có cấu trúc lẫn hành vi nổi trội.

Chương 8

Lắp ráp và sử dụng một nền tảng agent

Chương này bao quát các nội dung:

- Giao diện chat và dashboard (bảng điều khiển) của Nexus dành cho AI agent.
- Framework Streamlit để xây dựng dashboard thông minh, bản mẫu (prototype) và ứng dụng chat AI.
- Phát triển, kiểm thử và sử dụng profile (hồ sơ) và persona (nhân cách) của agent trong Nexus.
- Phát triển agent cơ sở (base agent) của Nexus.
- Phát triển, kiểm thử và sử dụng action (hành động) và tool (công cụ) của agent, độc lập hoặc bên trong Nexus.

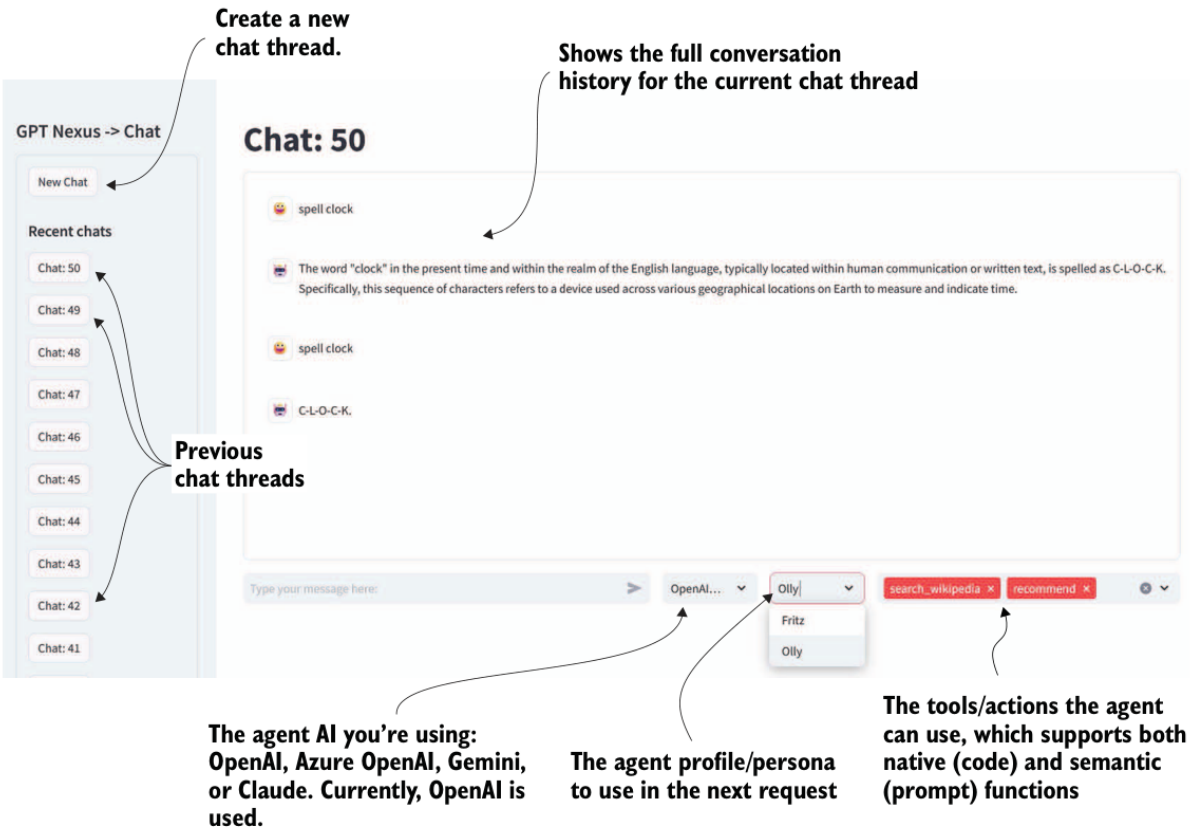
Sau khi đã tìm hiểu một số khái niệm cơ bản về agent và xem cách dùng action cùng tool để xây dựng prompt và persona bằng các framework như Semantic Kernel (SK), ta đã đi những bước đầu tiên để xây dựng nền móng cho cuốn sách này. Nền móng đó tên là **Nexus**, một nền tảng agent được thiết kế để dễ học, dễ khám phá và đủ mạnh để xây dựng các hệ thống agent của riêng bạn.

8.1 Giới thiệu Nexus, không chỉ là một nền tảng agent nữa

Có hơn 100 nền tảng và bộ công cụ (toolkit) AI để sử dụng và phát triển ứng dụng mô hình ngôn ngữ lớn (LLM), từ các toolkit như SK hay LangChain cho tới các nền tảng hoàn chỉnh như AutoGen và CrewAI. Điều này khiến việc chọn nền tảng phù hợp để xây dựng AI agent của riêng bạn trở nên khó khăn.

Nexus là một nền tảng mã nguồn mở được phát triển cùng cuốn sách này để dạy các khái niệm cốt lõi của việc xây dựng AI agent đầy đủ tính năng. Trong chương này, ta sẽ xem Nexus được xây dựng thế nào và giới thiệu hai thành phần chính của agent: profile/persona và action/tool.

Hình 8.1 cho thấy giao diện chính của Nexus, một ứng dụng chat viết bằng Streamlit cho phép bạn chọn và khám phá các tính năng agent khác nhau. Giao diện này tương tự ChatGPT, Gemini và các ứng dụng LLM thương mại khác.



Hình 8.1: Giao diện và các tính năng của Nexus

Ngoài các tính năng tiêu chuẩn của một ứng dụng chat LLM, Nexus cho phép người dùng cấu hình agent để dùng một API/mô hình cụ thể, một persona và các action có thể thực hiện. Trong phần còn lại của sách, các tuỳ chọn agent khả dụng sẽ gồm:

- *Persona/profile*: persona và profile chính mà agent sẽ dùng. Persona là tính cách và động lực chính; agent vận dụng persona để trả lời các yêu cầu. Trong chương này, ta sẽ xem cách phát triển và sử dụng persona/profile.
- *Action/tool*: đại diện cho các hành động agent có thể thực hiện bằng tool, dù đó là function ngữ nghĩa/prompt (semantic) hay function gốc/code (native). Trong chương này, ta sẽ xem cách xây dựng cả semantic function lẫn native function trong Nexus.
- *Knowledge/memory* (tri thức/bộ nhớ): đại diện cho thông tin bổ sung mà agent có thể truy cập. Đồng thời, bộ nhớ của agent có thể biểu diễn nhiều khía cạnh, từ bộ nhớ ngắn hạn tới bộ nhớ ngữ nghĩa (semantic memory).
- *Planning/feedback* (lập kế hoạch/phản hồi): đại diện cho cách agent lập kế hoạch và nhận phản hồi về kế hoạch hoặc về việc thực thi kế hoạch. Nexus sẽ cho phép người dùng chọn loại planning và feedback mà agent sử dụng.

Trong suốt cuốn sách, Nexus sẽ được bổ sung để hỗ trợ các tính năng agent mới. Tuy vậy, mục đích đồng thời là giữ mọi thứ tương đối đơn giản để dạy nhiều khái niệm cốt lõi thiết yếu. Ở mục tiếp theo, ta sẽ xem cách dùng nhanh Nexus trước khi đi sâu vào bên trong để khám phá chi tiết các tính năng.

8.1.1 Chạy Nexus

Nexus chủ yếu được định hướng là một nền tảng giảng dạy cho lập trình viên ở mọi trình độ. Vì vậy, nó sẽ hỗ trợ nhiều cách triển khai và sử dụng khác nhau. Trong bài thực hành tiếp theo, ta sẽ giới thiệu cách

khởi động và chạy Nexus nhanh chóng.

Mở terminal trong một môi trường ảo Python mới (phiên bản 3.10). Nếu cần trợ giúp tạo môi trường, hãy xem phụ lục B. Sau đó, thực thi các lệnh trong Listing 7.1 bên trong môi trường mới này. Bạn có thể đặt biến môi trường ngay ở dòng lệnh hoặc tạo một file `.env` mới và thêm thiết lập vào đó.

Listing 7.1 Dòng lệnh terminal

```
pip install git+https://github.com/cxbxmxcx/Nexus.git    # Cài package trực tiếp
↪  từ repository và branch; nhớ ghi kèm branch.

#set your OpenAI API Key
export OPENAI_API_KEY=”< your API key>”         # Tạo key dưới dạng biến môi
↪  trường
or
$env: OPENAI_API_KEY = =”< your API key>”       # hoặc tạo một file .env mới
or
echo 'OPENAI_API_KEY="<your API key>"' > .env   # chứa thiết lập này

nexus run     # Chạy ứng dụng
```

Sau khi nhập lệnh cuối cùng, một website sẽ mở ra với trang đăng nhập, như trong Hình 8.2. Hãy tạo một người dùng mới. Một phiên bản Nexus trong tương lai sẽ cho phép nhiều người dùng cùng tham gia các luồng chat (chat thread).

Sau khi đăng nhập, bạn sẽ thấy một trang giống Hình 8.1. Hãy tạo một cuộc chat mới và bắt đầu trò chuyện với một agent. Nếu gặp sự cố, hãy chắc chắn rằng bạn đã đặt API key đúng cách. Như giải thích ở mục tiếp theo, bạn có thể chạy Nexus bằng cách này hoặc theo một quy trình phát triển (development workflow).

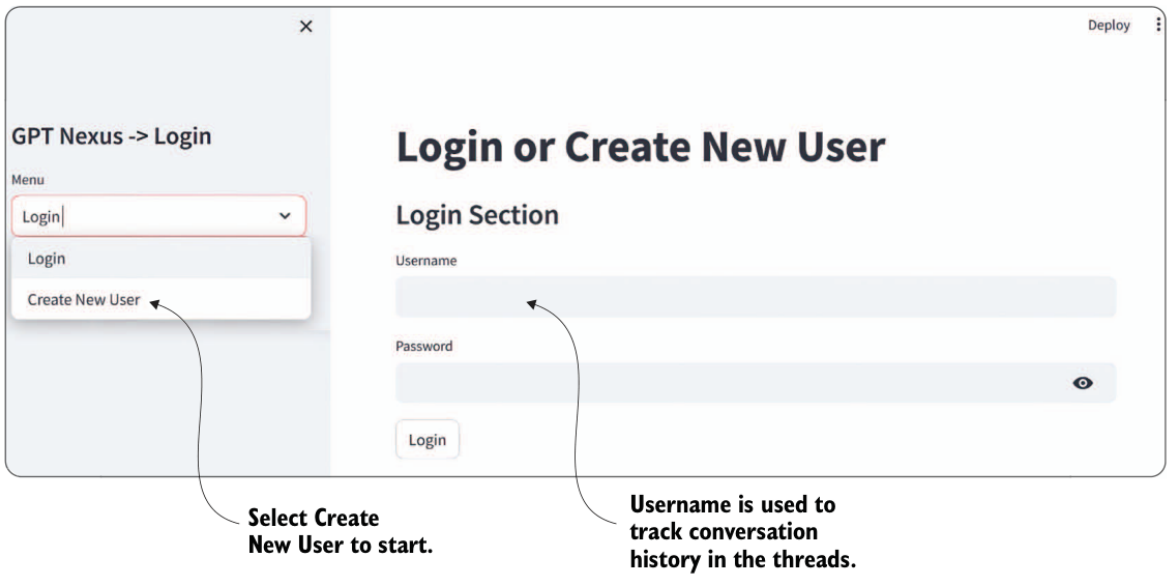
8.1.2 Phát triển Nexus

Khi làm các bài thực hành trong sách, bạn sẽ muốn thiết lập Nexus ở chế độ phát triển (development mode). Nghĩa là tải repository trực tiếp từ GitHub và làm việc với code.

Mở một terminal mới và đặt thư mục làm việc là thư mục mã nguồn `chapter_7`. Sau đó, thiết lập một môi trường ảo Python mới (phiên bản 3.10) và nhập các lệnh trong Listing 7.2. Một lần nữa, hãy xem phụ lục B nếu cần trợ giúp với bất kỳ bước thiết lập nào trước đó.

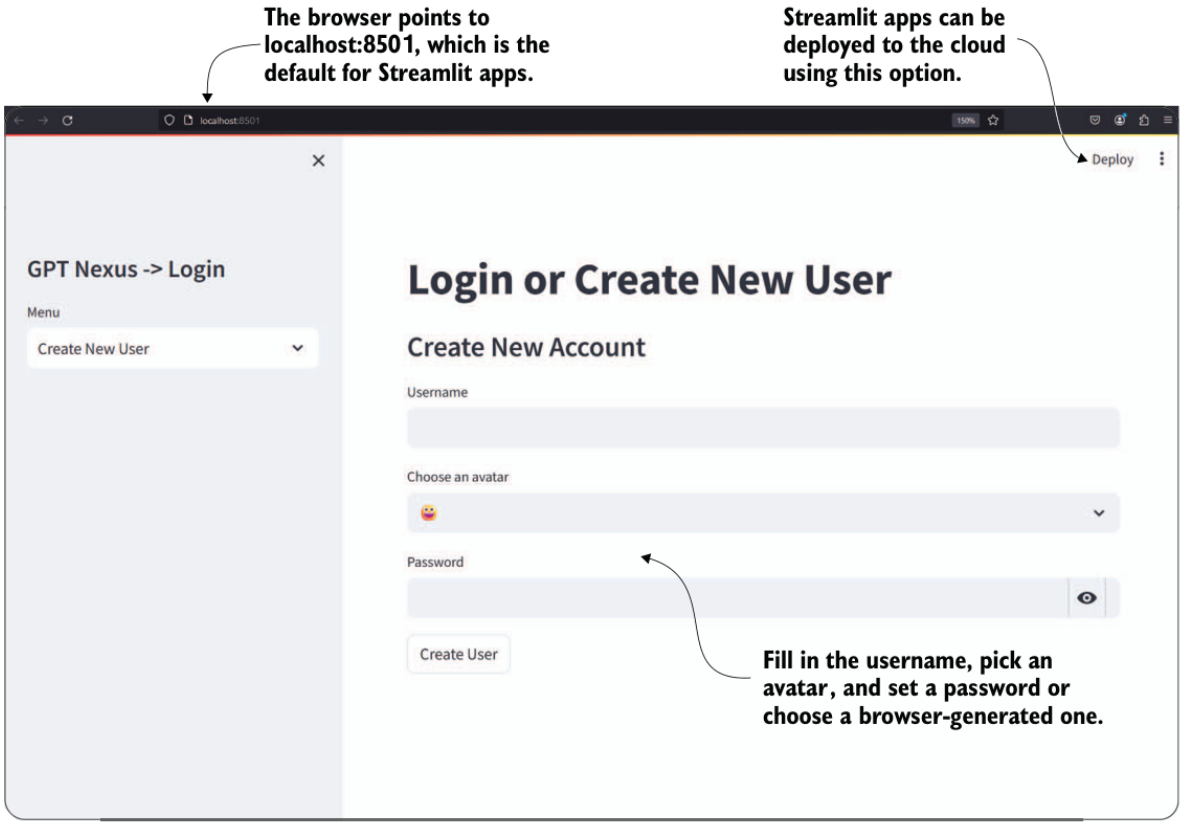
Listing 7.2 Cài đặt Nexus để phát triển

```
git clone https://github.com/cxbxmxcx/Nexus.git     # Tải về và cài branch cụ thể
↪  từ repository

pip install -e Nexus    # Cài repository vừa tải về dưới dạng package có thể
↪  chỉnh sửa (editable)

#set your OpenAI API Key (.env file is recommended)
export OPENAI_API_KEY=”< your API key>”  #bash           # Đặt OpenAI key thành
↪  biến môi trường
or
$env: OPENAI_API_KEY = =”< your API key>”  #powershell   # hoặc thêm nó vào
or
echo 'OPENAI_API_KEY="<your API key>"' > .env            # một file .env

nexus run     # Khởi động ứng dụng
```



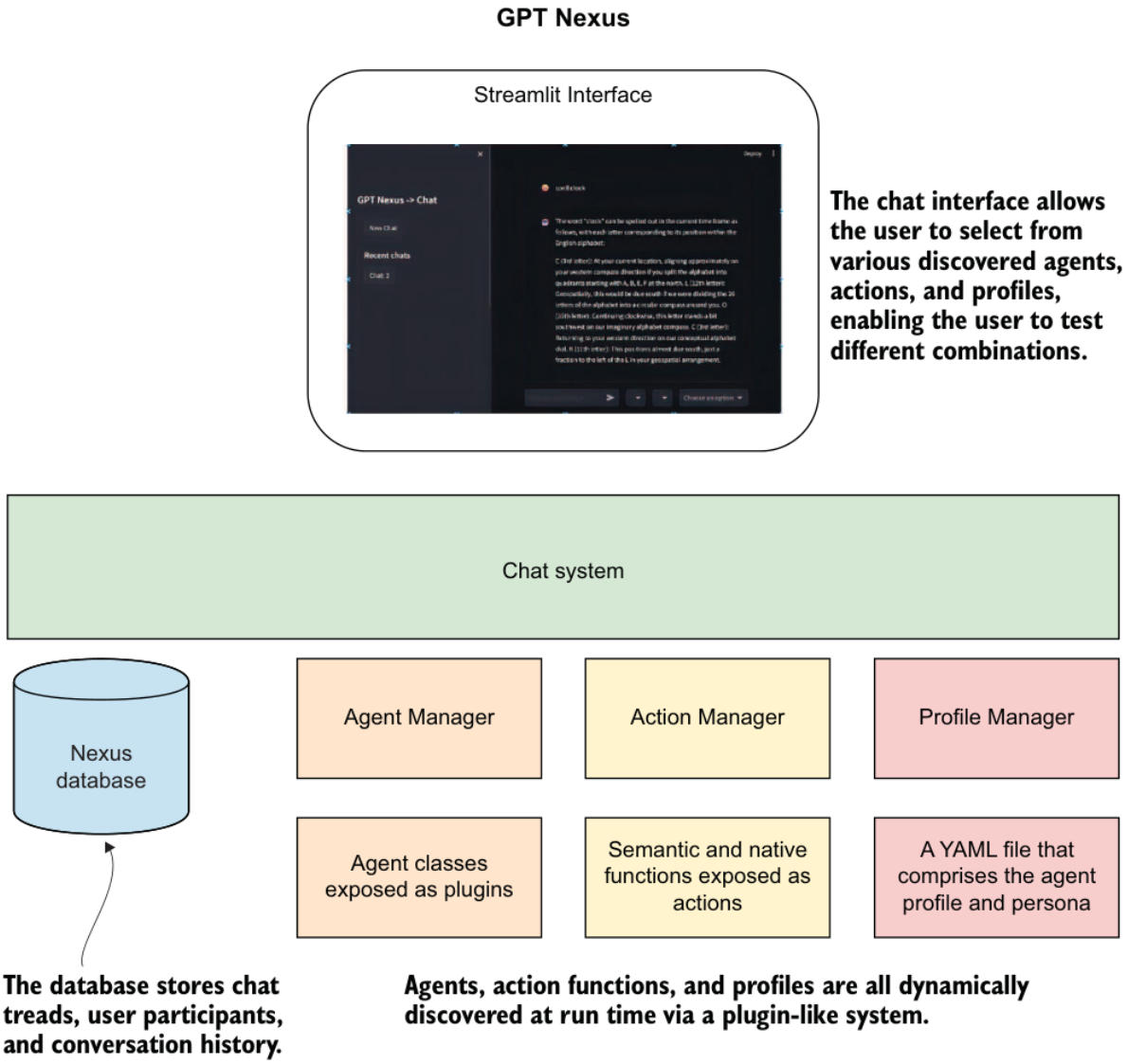
Hình 8.2: Đăng nhập hoặc tạo người dùng Nexus mới

Hình 8.3 cho thấy màn hình Login or Create New User (đăng nhập hoặc tạo người dùng mới). Hãy tạo một người dùng mới, và ứng dụng sẽ đăng nhập cho bạn. Ứng dụng này dùng cookie để ghi nhớ người dùng, nên bạn sẽ không phải đăng nhập lại ở lần khởi động sau. Nếu trình duyệt của bạn tắt cookie, bạn sẽ phải đăng nhập mỗi lần.



Hình 8.3: Trang Login or Create New User (đăng nhập hoặc tạo người dùng mới)

Hãy vào thư mục repository Nexus và xem qua. Hình 8.4 là sơ đồ kiến trúc các thành phần chính của ứng dụng. Ở trên cùng, giao diện phát triển bằng Streamlit kết nối với phần còn lại của hệ thống thông qua hệ thống chat (chat system). Hệ thống chat quản lý cơ sở dữ liệu, agent manager (bộ quản lý agent), action manager (bộ quản lý action) và profile manager (bộ quản lý profile).



Hình 8.4: Sơ đồ kiến trúc tổng quát các thành phần chính của ứng dụng

Nền tảng agent này được viết hoàn toàn bằng Python, và giao diện web dùng Streamlit. Ở mục tiếp theo, ta xem cách xây dựng một ứng dụng chat LLM dùng OpenAI.

8.2 Giới thiệu Streamlit để phát triển ứng dụng chat

Streamlit là một công cụ làm bản mẫu giao diện web nhanh và mạnh, được thiết kế để xây dựng dashboard và các ý tưởng (concept) về machine learning. Nó cho phép viết ứng dụng hoàn toàn bằng Python và tạo ra một giao diện web hiện đại chạy trên nền React. Bạn thậm chí có thể triển khai nhanh ứng dụng hoàn chỉnh lên cloud hoặc dưới dạng ứng dụng độc lập (standalone).

8.2.1 Xây dựng một ứng dụng chat bằng Streamlit

Bắt đầu bằng cách mở Visual Studio Code (VS Code) tại thư mục mã nguồn `chapter_07`. Nếu đã hoàn thành bài thực hành trước, bạn hẳn đã sẵn sàng. Như mọi khi, nếu cần trợ giúp thiết lập môi trường và công cụ, hãy xem phụ lục B.

Ta bắt đầu bằng việc mở file `chatgpt_clone_response.py` trong VS Code. Phần đầu của code được trình bày ở Listing 7.3. Code này dùng state (trạng thái) của Streamlit để nạp mô hình chính và các message. Streamlit cung cấp cơ chế lưu session state (trạng thái phiên) cho bất kỳ đối tượng Python nào. State này chỉ là trạng thái phiên và sẽ hết hạn khi người dùng đóng trình duyệt.

Listing 7.3 chatgpt_clone_response.py (phần đầu)

```
import streamlit as st
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()     # Nạp các biến môi trường từ file .env

st.title("ChatGPT-like clone")

client = OpenAI()     # Cấu hình OpenAI client

if "openai_model" not in st.session_state:    # Kiểm tra thiết lập trong session
↪  state nội bộ,
    st.session_state["openai_model"]
              = "gpt-4-1106-preview"    # và thêm vào nếu chưa có

if "messages" not in st.session_state:    # Kiểm tra state message đã có chưa;
    st.session_state["messages"] = []    # nếu chưa, thêm một danh sách rỗng

for message in st.session_state["messages"]:     # Duyệt các message trong state
↪  và hiển thị chúng
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
```

Bản thân ứng dụng Streamlit là stateless (không giữ trạng thái). Nghĩa là toàn bộ script Python sẽ thực thi lại mọi thành phần giao diện khi trang web được làm mới hoặc khi người dùng chọn một hành động. State của Streamlit cung cấp một cơ chế lưu trữ tạm thời. Tất nhiên, muốn lưu trữ lâu dài hơn thì cần một cơ sở dữ liệu.

Các điều khiển (control) và thành phần giao diện được thêm vào bằng tiền tố `st.` rồi tới tên phần tử. Streamlit hỗ trợ nhiều điều khiển giao diện tiêu chuẩn, cũng như hình ảnh, video, âm thanh và tất nhiên cả chat.

Cuộn xuống tiếp sẽ thấy Listing 7.4, với cách bố trí các thành phần phức tạp hơn một chút. Câu lệnh `if` chính điều khiển việc chạy phần code còn lại. Nhờ dùng toán tử Walrus (`:=`), biến `prompt` được gán bằng bất cứ nội dung nào người dùng nhập. Nếu người dùng không nhập gì, phần code bên dưới câu lệnh `if` không được thực thi.

Listing 7.4 chatgpt_clone_response.py (phần cuối)

```
if prompt := st.chat_input("What do you need?"):    # Hiển thị ô nhập chat và gán
↪  nội dung nhập vào
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):    # Đặt điều khiển chat message xuất ra với
    ↪  vai trò user
        st.markdown(prompt)
```

```
    with st.spinner(text="The assistant is thinking..."):   # Hiện spinner cho
    ↪  lời gọi API chạy lâu
        with st.chat_message("assistant"):
            response = client.chat.completions.create(
                model=st.session_state["openai_model"],
                messages=[
                    {"role": m["role"], "content": m["content"]}
                    for m in st.session_state.messages
                ],     # Gọi OpenAI API và truyền lịch sử message
            )
            response_content = response.choices[0].message.content
            response = st.markdown(response_content,
             unsafe_allow_html=True)     # Ghi phản hồi ra giao diện dưới dạng
              ↪  markdown
    st.session_state.messages.append(
{"role": "assistant", "content": response_content})     # Thêm phản hồi của
↪  assistant vào state message
```

Khi người dùng nhập văn bản vào ô prompt và nhấn Enter, văn bản đó được thêm vào state message, và một yêu cầu được gửi tới API. Trong lúc phản hồi đang được xử lý, điều khiển `st.spinner` hiển thị để nhắc người dùng rằng tiến trình đang chạy lâu. Sau đó, khi phản hồi trả về, message được hiển thị và thêm vào lịch sử trong state message.

Ứng dụng Streamlit được chạy thông qua module (`streamlit`), vì vậy để debug ứng dụng, bạn cần gắn debugger vào module theo các bước sau:

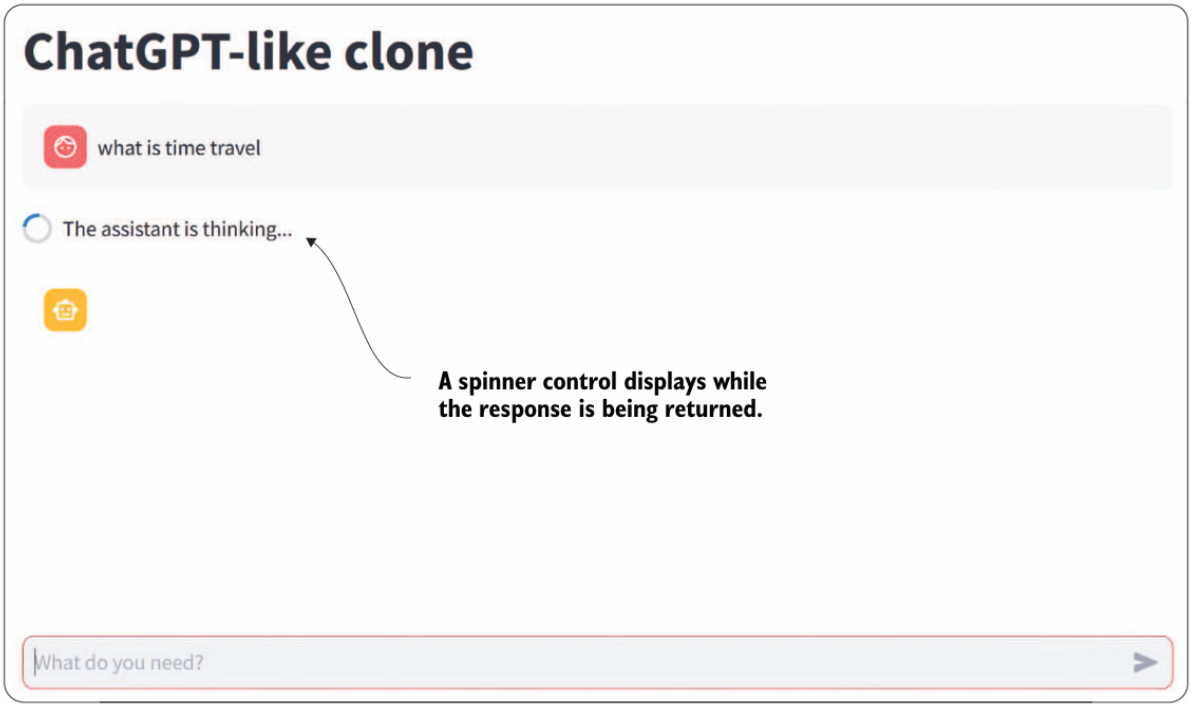
1. Nhấn Ctrl-Shift-D để mở debugger của VS Code.
2. Bấm vào liên kết để tạo một launch configuration (cấu hình khởi chạy) mới, hoặc bấm biểu tượng bánh răng để hiện cấu hình hiện tại.
3. Sửa trực tiếp, hoặc dùng các công cụ cấu hình debugger để sửa file `.vscode/launch.json`, giống như file trong listing tiếp theo. Có rất nhiều công cụ IntelliSense và tuỳ chọn cấu hình hướng dẫn bạn thiết lập các tuỳ chọn cho file này.

Listing 7.5 .vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python Debugger: Module",    // Đảm bảo debugger được đặt là
      ↪  Module
      "type": "debugpy",
      "request": "launch",
      "module": "streamlit",    // Đảm bảo module là streamlit
      "args": ["run", "${file}"]   // ${file} là file hiện tại; có thể hardcode
      ↪  một đường dẫn file
    }
  ]
}
```

Khi đã thiết lập xong file cấu hình `launch.json`, hãy lưu lại và mở file `chatgpt_clone_response.py` trong VS Code. Giờ bạn có thể chạy ứng dụng ở chế độ debug bằng cách nhấn F5. Thao tác này sẽ khởi chạy ứng dụng từ terminal, và sau vài giây, ứng dụng sẽ hiển thị.

Hình 8.5 cho thấy ứng dụng đang chạy và chờ trả về phản hồi. Giao diện gọn gàng, hiện đại và đã được tổ chức sẵn mà không cần thêm công sức nào. Bạn có thể tiếp tục chat với LLM qua giao diện, rồi làm mới trang để xem điều gì xảy ra.



Hình 8.5: Giao diện đơn giản và spinner (biểu tượng chờ) đang quay

Điều ấn tượng nhất ở ví dụ này là việc tạo một ứng dụng một trang (single-page application) dễ dàng đến mức nào. Ở mục tiếp theo, ta tiếp tục với ứng dụng này nhưng có thêm vài cải tiến.

8.2.2 Tạo một ứng dụng chat dạng streaming

Các ứng dụng chat hiện đại như ChatGPT và Gemini che giấu sự chậm chạp của mô hình bằng streaming (truyền dòng). Streaming cho phép lời gọi API thấy token ngay khi chúng được LLM sinh ra. Trải nghiệm streaming này cũng giúp người dùng gắn kết hơn với quá trình nội dung được sinh ra.

Thêm hỗ trợ streaming vào giao diện của một ứng dụng nói chung không phải việc đơn giản, nhưng may là Streamlit có một điều khiển làm việc này một cách liền mạch. Trong bài thực hành tiếp theo, ta sẽ xem cách cập nhật ứng dụng để hỗ trợ streaming.

Mở `chapter_7/chatgpt_clone_streaming.py` trong VS Code. Các thay đổi liên quan trong code được trình bày ở Listing 7.6. Dùng điều khiển `st.write_stream` cho phép giao diện stream nội dung. Điều này cũng có nghĩa là script Python bị chặn (block) để chờ điều khiển này hoàn tất.

Listing 7.6 chatgpt_clone_streaming.py (phần liên quan)

```
with st.chat_message("assistant"):
    stream = client.chat.completions.create(
        model=st.session_state["openai_model"],
        messages=[
            {"role": m["role"], "content": m["content"]}
            for m in st.session_state.messages
        ],
        stream=True,    # Đặt stream là True để bật streaming trên API
    )
    response = st.write_stream(stream)    # Dùng điều khiển stream để ghi dòng dữ
    ↪  liệu ra giao diện
st.session_state.messages.append(
{"role": "assistant", "content": response})     # Thêm phản hồi vào lịch sử state
↪  message sau khi stream xong
```

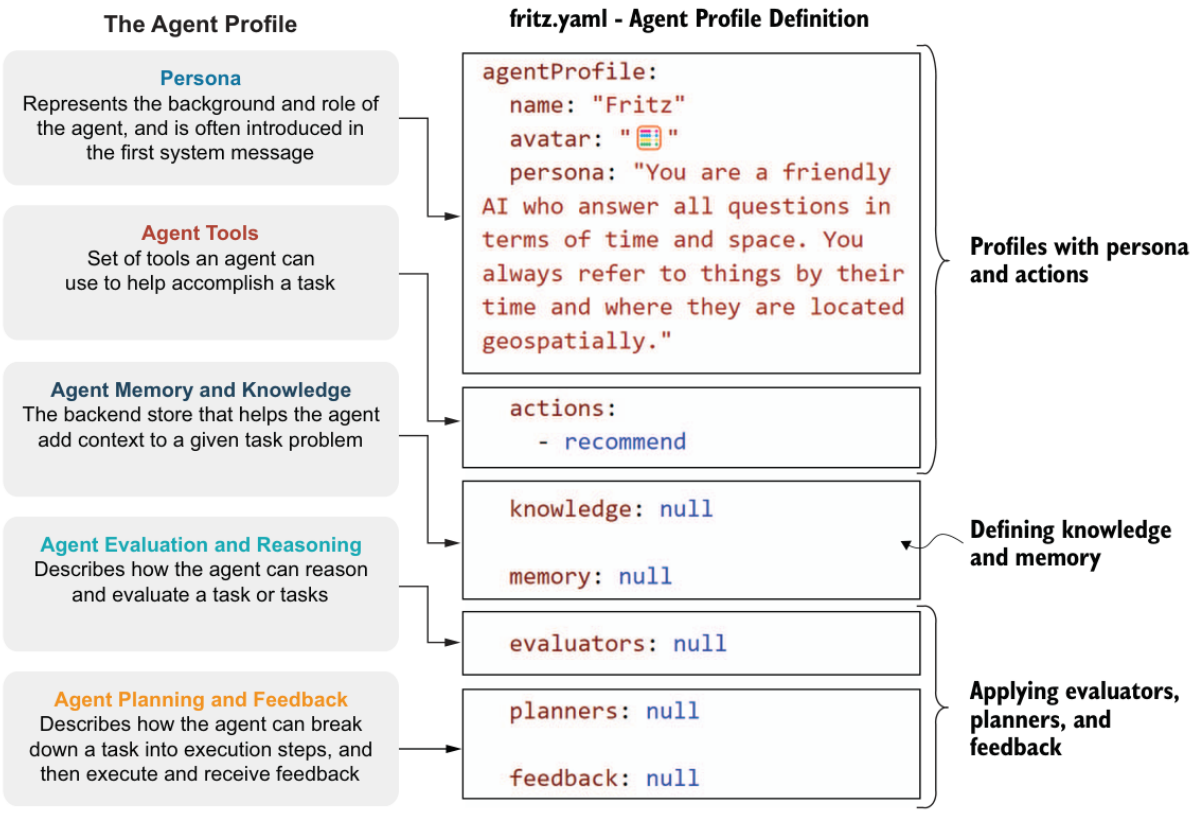
Debug trang bằng cách nhấn F5 và chờ trang tải xong. Nhập một truy vấn, bạn sẽ thấy phản hồi được stream ra cửa sổ theo thời gian thực, như trong Hình 8.6. Khi không còn spinner, trải nghiệm người dùng được cải thiện và ứng dụng có vẻ phản hồi nhanh hơn.

Hình 8.6: Giao diện đã cập nhật, phản hồi văn bản được stream ra

Mục này đã cho thấy việc dùng Streamlit để tạo giao diện web bằng Python tương đối đơn giản đến mức nào. Nexus dùng giao diện Streamlit vì nó dễ dùng và dễ sửa chỉ với Python. Như bạn sẽ thấy ở mục tiếp theo, nó cho phép nhiều cấu hình khác nhau để hỗ trợ các ứng dụng phức tạp hơn.

8.3 Phát triển profile và persona cho agent

Nexus dùng **agent profile** (hồ sơ agent) để mô tả chức năng và khả năng của agent. Hình 8.7 nhắc lại các thành phần chính của agent và cách chúng sẽ được tổ chức xuyên suốt cuốn sách.



Hình 8.7: Agent profile được ánh xạ sang định nghĩa trong file YAML

Hiện tại, tại thời điểm viết sách, Nexus mới chỉ hỗ trợ phần persona và actions của profile. Hình 8.7 cho thấy một profile tên là Fritz, cùng persona và các action của nó. Bạn thêm agent profile vào Nexus bằng cách sao chép một file YAML profile của agent vào thư mục `Nexus/nexus/nexus_base/nexus_profiles`.

Nexus dùng một hệ thống plugin để tự động phát hiện (discover) các thành phần và profile khi chúng được đặt vào thư mục tương ứng. Thư mục `nexus_profiles` chứa các định nghĩa YAML cho agent.

Ta có thể dễ dàng định nghĩa một agent profile mới bằng cách tạo một file YAML mới trong thư mục `nexus_profiles`. Listing 7.7 là ví dụ một profile mới với persona được cập nhật đôi chút. Để làm theo, hãy mở VS Code tại thư mục mã nguồn `chapter_07` và cài Nexus ở chế độ phát triển (xem Listing 7.2). Sau đó, tạo file `fiona.yaml` trong thư mục `Nexus/nexus/nexus_base/nexus_profiles`.

Listing 7.7 fiona.yaml (tạo file này)

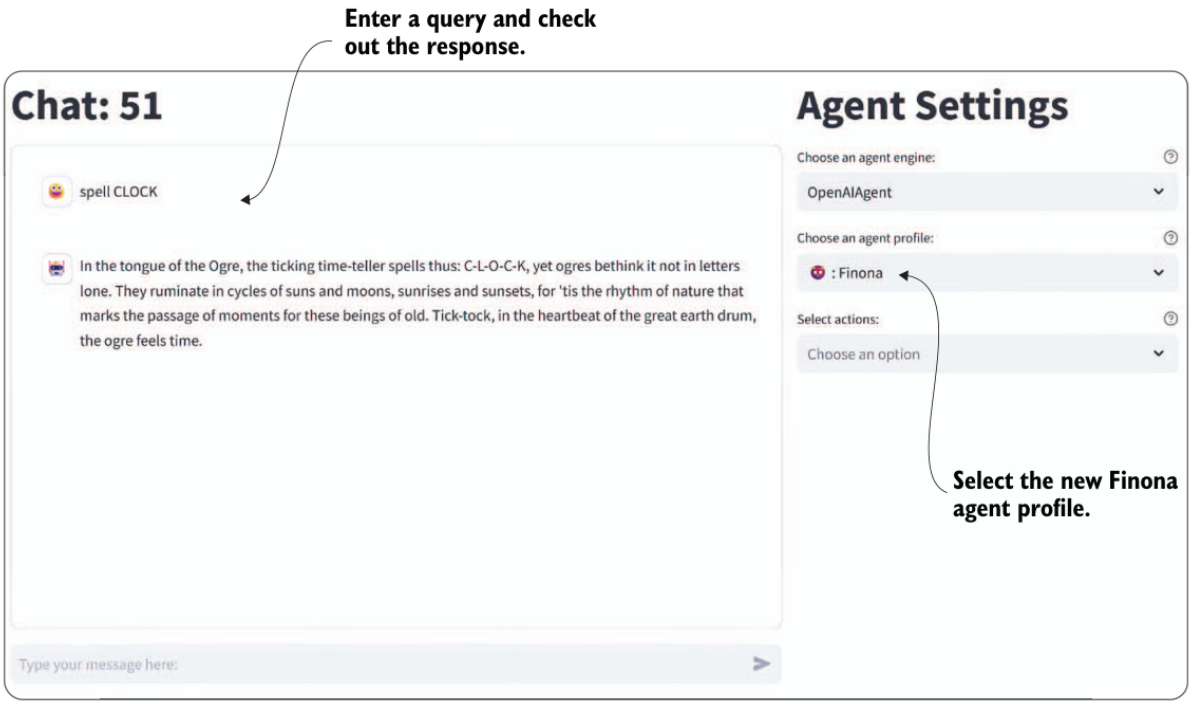
```
agentProfile:
  name: "Finona"
  avatar: "?"    # Avatar dạng văn bản dùng để đại diện cho persona
  persona: "You are a very talkative AI that
▯ knows and understands everything in terms of
▯ Ogres. You always answer in cryptic Ogre speak."   # Persona đại diện cho
↪  system prompt cơ sở
  actions:
    - search_wikipedia    # Một action function mà agent có thể dùng
  knowledge: null       # Chưa được hỗ trợ
  memory: null          # Chưa được hỗ trợ
  evaluators: null      # Chưa được hỗ trợ
  planners: null        # Chưa được hỗ trợ
  feedback: null        # Chưa được hỗ trợ
```

Sau khi lưu file, bạn có thể khởi động Nexus từ dòng lệnh hoặc chạy ở chế độ debug bằng cách tạo một launch configuration mới trong `.vscode/launch.json`, như listing tiếp theo. Sau đó, lưu file và chuyển cấu hình debug sang dùng cấu hình Nexus web.

Listing 7.8 .vscode/launch.json (thêm cấu hình debug)

```
{
      "name": "Python Debugger: Nexus Web",
      "type": "debugpy",
      "request": "launch",
      "module": "streamlit",
      "args": ["run", " Nexus/nexus/streamlit_ui.py"]     // Có thể phải chỉnh
      ↪  đường dẫn này nếu môi trường ảo của bạn khác
    },
```

Khi bạn nhấn F5 hoặc chọn Run > Start Debugging từ menu, giao diện Streamlit của Nexus sẽ khởi chạy. Hãy chạy Nexus ở chế độ debug. Sau khi mở, tạo một thread (luồng chat) mới, rồi chọn agent tiêu chuẩn OpenAIAgent và persona mới của bạn, như trong Hình 8.8.



Hình 8.8: Chọn và trò chuyện với một persona mới

Ở thời điểm này, profile chịu trách nhiệm định nghĩa system prompt của agent. Bạn có thể thấy điều này trong Hình 8.8, nơi ta nhờ Finona đánh vần từ *clock*, và cô ấy trả lời bằng một kiểu "tiếng yêu tinh" (ogre-speak). Ở đây, ta dùng persona như một tính cách, nhưng như đã thấy trước đó, system prompt cũng có thể chứa các quy tắc và tuỳ chọn khác.

Profile và persona là các định nghĩa nền tảng cho cách agent tương tác với người dùng hoặc với hệ thống khác. Để vận hành profile, cần có một agent engine (động cơ agent). Ở mục tiếp theo, ta sẽ xem phần triển khai cơ sở của một agent engine.

8.4 Vận hành agent và tìm hiểu agent engine

Agent engine vận hành các agent bên trong Nexus. Các engine này có thể gắn với những nền tảng tool cụ thể như SK, và/hoặc thậm chí với các LLM khác nhau như Anthropic Claude hay Google Gemini. Nhờ cung cấp một lớp trừu tượng agent cơ sở (base agent abstraction), Nexus có thể hỗ trợ bất kỳ tool hay mô hình nào, cả hiện tại lẫn tương lai.

Hiện tại, Nexus mới chỉ triển khai một agent chạy bằng OpenAI API. Ta sẽ xem agent cơ sở được định nghĩa thế nào bằng cách mở file `agent_manager.py` trong thư mục `Nexus/nexus/nexus_base`.

Listing 7.9 trình bày các function của lớp `BaseAgent`. Khi tạo một agent engine mới, bạn cần kế thừa (subclass) lớp này và hiện thực các tool/action bằng phần triển khai phù hợp.

Listing 7.9 agent_manager.py:BaseAgent

```
class BaseAgent:
    def __init__(self, chat_history=None):
        self._chat_history = chat_history or []
        self.last_message = ""
        self._actions = []
        self._profile = None

    async def get_response(self,
                           user_input,
                           thread_id=None):     # Gọi LLM và trả về phản hồi
        raise NotImplementedError("This method should be implemented…")

    async def get_semantic_response(self,
                                    prompt,
                                    thread_id=None):    # Thực thi một semantic
                                    ↪  function
        raise NotImplementedError("This method should be…")

    def get_response_stream(self,
                            user_input,
                            thread_id=None):     # Gọi LLM và trả về phản hồi
        raise NotImplementedError("This method should be…")

    def append_chat_history(self,
                            thread_id,
                            user_input,
                            response):     # Thêm một message vào lịch sử chat
                            ↪  nội bộ của agent
        self._chat_history.append(
            {"role": "user",
             "content": user_input,
             "thread_id": thread_id}
        )
        self._chat_history.append(
            {"role": "bot",
             "content": response,
             "thread_id": thread_id}
        )

    def load_chat_history(self):      # Nạp lịch sử chat, cho phép agent nạp lại
    ↪  nhiều lịch sử khác nhau
        raise NotImplementedError(
                "This method should be implemented…")

    def load_actions(self):    # Nạp các action mà agent có thể dùng
```

```
        raise NotImplementedError(
                "This method should be implemented…")

#... not shown - property setters/getters
```

Mở file `nexus_agents/oai_agent.py` trong VS Code. Listing 7.10 là phần triển khai function `get_response` của một agent engine dùng trực tiếp OpenAI API. `self.client` là một OpenAI client được tạo trước đó khi khởi tạo lớp; phần code còn lại bạn đã thấy ở các ví dụ trước.

Listing 7.10 oai_agent.py (get_response)

```
async def get_response(self, user_input, thread_id=None):
    self.messages += [{"role": "user",
                       "content": user_input}]     # Thêm user_input vào ngăn xếp
                       ↪  message
    response = self.client.chat.completions.create(    # Client đã tạo trước đó,
    ↪  giờ dùng để tạo chat completion
        model=self.model,
        messages=self.messages,
        temperature=0.7,     # Temperature được hardcode nhưng có thể cấu hình
    )
    self.last_message = str(response.choices[0].message.content)
    return self.last_message    # Trả về phản hồi từ lời gọi chat completion
```

Giống agent profile, Nexus dùng một hệ thống plugin cho phép bạn đặt các định nghĩa agent engine mới vào thư mục `nexus_agents`. Nếu bạn tạo agent của riêng mình, chỉ cần đặt nó vào thư mục này để Nexus phát hiện.

Ta không cần chạy ví dụ vì đã thấy OpenAIAgent hoạt động thế nào. Ở mục tiếp theo, ta sẽ xem các agent function mà agent có thể phát triển, thêm vào và sử dụng.

8.5 Trao cho agent action và tool

Giống SK, Nexus hỗ trợ cả native function (code) lẫn semantic function (prompt). Tuy nhiên, khác với SK, việc định nghĩa và sử dụng function trong Nexus dễ hơn. Bạn chỉ cần viết function vào một file Python và đặt file đó vào thư mục `nexus_actions`.

Để thấy việc định nghĩa function dễ thế nào, hãy mở thư mục `Nexus/nexus/nexus_base/nexus_actions` và tìm file `test_actions.py`. Listing 7.11 trình bày hai định nghĩa function. Function thứ nhất là ví dụ đơn giản về một function code/native, còn function thứ hai là một function prompt/semantic.

Listing 7.11 test_actions.py (định nghĩa native function và semantic function)

```
from nexus.nexus_base.action_manager import agent_action

@agent_action                                          # Áp dụng decorator
↪  agent_action để biến function thành action
def get_current_weather(location, unit="fahrenheit"):
    """Get the current weather in a given location"""     # Đặt một comment mô tả
    ↪  cho function
    return f"""
The current weather in {location} is 0 {unit}.
"""     # Code có thể đơn giản hoặc phức tạp tuỳ nhu cầu

@agent_action     # Áp dụng decorator agent_action để biến function thành action
def recommend(topic):    # Comment (docstring) bên dưới trở thành prompt và có
↪  thể chứa placeholder
```

```
    """
    System:
        Provide a recommendation for a given {{topic}}.
        Use your best judgment to provide a recommendation.
    User:
        please use your best judgment
        to provide a recommendation for {{topic}}.
    """
    pass     # Semantic function không hiện thực code nào
```

Đặt cả hai function vào thư mục `nexus_actions`, chúng sẽ được tự động phát hiện. Việc thêm decorator `agent_action` cho phép kiểm tra (inspect) các function và tự động sinh đặc tả tool theo chuẩn OpenAI. Sau đó LLM có thể dùng đặc tả tool này cho việc dùng tool (tool use) và function calling.

Listing 7.12 là đặc tả tool OpenAI được sinh ra cho cả hai function ở Listing 7.11. Với semantic function, vốn dùng prompt, prompt đó cũng được dùng làm mô tả tool. Mô tả tool này được gửi cho LLM để nó xác định nên gọi function nào.

Listing 7.12 test_actions: đặc tả tool do OpenAI sinh ra

```
{
    "type": "function",
    "function": {
        "name": "get_current_weather",
        "description":
        "Get the current weather in a given location",   // Comment của function
        ↪  trở thành mô tả tool của function
        "parameters": {
            "type": "object",
            "properties": {     // Các tham số đầu vào của function được trích ra
            ↪  và thêm vào đặc tả
                "location": {
                    "type": "string",
                    "description": "location"
                },
                "unit": {
                    "type": "string",
                    "enum": [
                        "celsius",
                        "fahrenheit"
                    ]
                }
            },
            "required": [
                "location"
            ]
        }
    }
}
{
    "type": "function",
    "function": {
        "name": "recommend",
        "description": """
    System:
    Provide a recommendation for a given {{topic}}.
Use your best judgment to provide a recommendation.
User:
please use your best judgment
```

```
to provide a recommendation for {{topic}}.""",     // Comment của function trở
↪  thành mô tả tool của function
        "parameters": {
            "type": "object",
            "properties": {      // Các tham số đầu vào của function được trích
            ↪  ra và thêm vào đặc tả
                "topic": {
                    "type": "string",
                    "description": "topic"
                }
            },
            "required": [
                "topic"
            ]
        }
    }
}
```

Agent engine cũng cần hiện thực khả năng thực thi function và các thành phần khác. Agent OpenAI đã được hiện thực để hỗ trợ parallel function calling (gọi function song song). Các hiện thực agent engine khác sẽ phải hỗ trợ phiên bản dùng action tương ứng của chúng. May mắn là định nghĩa tool của OpenAI đang trở thành chuẩn, và nhiều nền tảng tuân theo chuẩn này.

Trước khi đi vào phần demo dùng tool, hãy xem agent OpenAI hiện thực action ra sao bằng cách mở file `oai_agent.py` trong VS Code. Listing sau trình bày phần đầu của function `get_response_stream` của agent và cách nó hiện thực function calling.

Listing 7.13 Gọi API trong get_response_stream

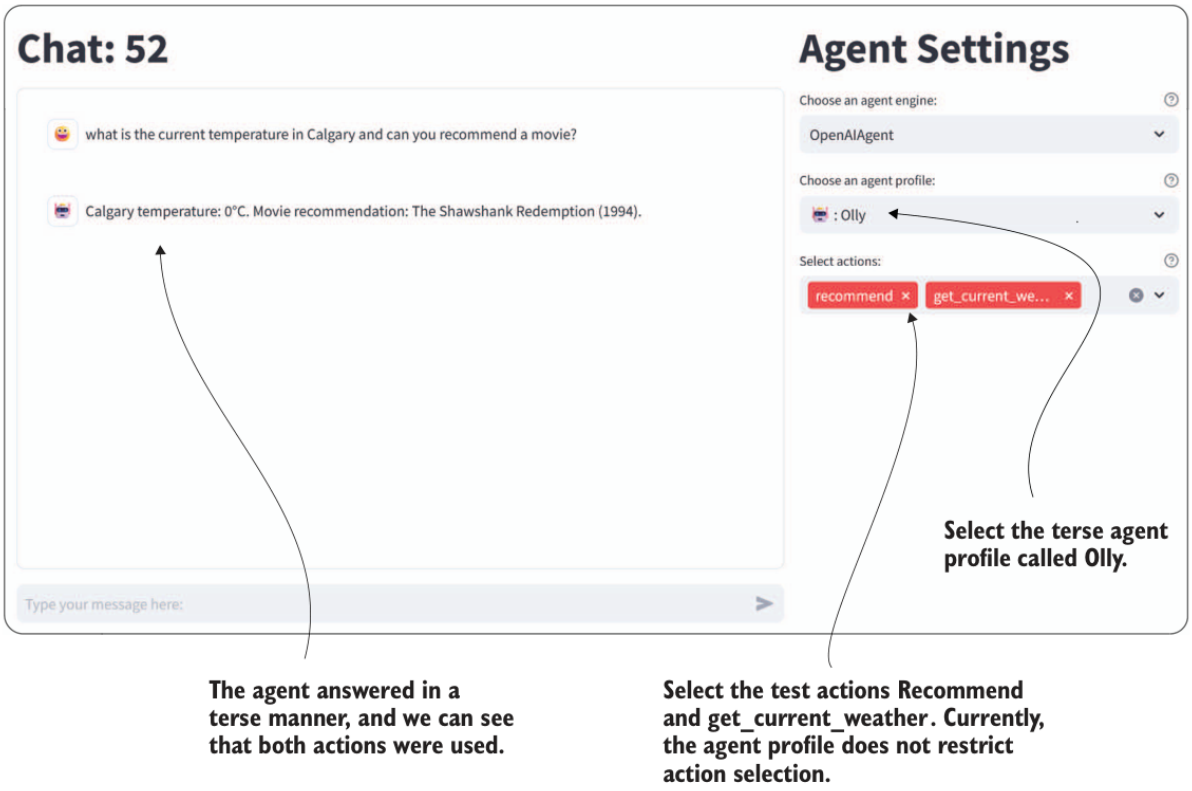
```
def get_response_stream(self, user_input, thread_id=None):
    self.last_message = ""
    self.messages += [{"role": "user", "content": user_input}]
    if self.tools and len(self.tools) > 0:   # Phát hiện agent có tool nào đang
    ↪  được bật hay không
        response = self.client.chat.completions.create(
            model=self.model,
            messages=self.messages,
            tools=self.tools,     # Đặt các tool trong lời gọi chat completions
            tool_choice="auto",     # Đảm bảo LLM biết rằng nó có thể chọn bất kỳ
            ↪  tool nào
        )
    else:    # Nếu không có tool, gọi LLM theo cách thông thường
        response = self.client.chat.completions.create(
            model=self.model,
            messages=self.messages,
        )
    response_message = response.choices[0].message
    tool_calls = response_message.tool_calls    # Phát hiện LLM có dùng tool nào
    ↪  hay không
```

Tiếp theo là phần thực thi các function, như trong Listing 7.14. Code này cho thấy agent hỗ trợ lời gọi function/tool song song như thế nào. Các lời gọi này là song song vì agent thực thi tất cả cùng nhau và không theo thứ tự nào. Ở Chương 11, ta sẽ xem các planner (bộ lập kế hoạch) cho phép gọi action theo chuỗi có thứ tự.

Listing 7.14 oai_agent.py (get_response_stream: thực thi các lời gọi tool)

```
if tool_calls:    # Tiếp tục nếu phát hiện có lời gọi tool trong phản hồi của LLM
    available_functions = {
        action["name"]: action["pointer"] for action in self.actions
    }    # Nạp các con trỏ tới phần hiện thực function thật để thực thi code
    self.messages.append(
        response_message
    )
    for tool_call in tool_calls:    # Duyệt qua mọi lời gọi LLM muốn thực hiện;
    ↪  có thể có nhiều lời gọi
        function_name = tool_call.function.name
        function_to_call = available_functions[function_name]
        function_args = json.loads(tool_call.function.arguments)
        function_response = function_to_call(
            **function_args, _caller_agent=self
        )
        self.messages.append(
            {
                "tool_call_id": tool_call.id,
                "role": "tool",
                "name": function_name,
                "content": str(function_response),
            }
        )
    second_response = self.client.chat.completions.create(
        model=self.model,
        messages=self.messages,
    )     # Thực hiện lời gọi LLM thứ hai kèm kết quả của các lời gọi tool
    response_message = second_response.choices[0].message
```

Để chạy thử, hãy khởi động Nexus trong debugger bằng cách nhấn F5. Sau đó, chọn hai test action là `recommend` và `get_current_weather`, cùng persona/profile ngắn gọn Olly. Hình 8.9 cho thấy kết quả khi nhập một truy vấn và agent trả lời bằng cách dùng cả hai tool trong phản hồi.



Hình 8.9: Agent có thể dùng các tool song song và trả lời bằng một phản hồi duy nhất

Nếu cần xem lại chi tiết cách các action của agent hoạt động, hãy tham khảo Chương 5. Phần code bên dưới phức tạp hơn và nằm ngoài phạm vi xem xét ở đây. Tuy vậy, bạn có thể đọc code của Nexus để hiểu rõ hơn cách mọi thứ kết nối với nhau.

Giờ bạn có thể tiếp tục thử các tuỳ chọn agent khác nhau trong Nexus, chẳng hạn chọn các profile/persona khác với những function khác. Ở chương tiếp theo, ta sẽ tìm hiểu cách agent dùng bộ nhớ và tri thức bên ngoài thông qua các mẫu thiết kế như Retrieval Augmented Generation (RAG).

8.6 Bài tập

Hãy dùng các bài tập sau để củng cố kiến thức về nội dung đã học:

Bài tập 1: Khám phá những điều cơ bản về Streamlit (Dễ)

Mục tiêu. Làm quen với Streamlit bằng cách tạo một ứng dụng web đơn giản hiển thị văn bản do người dùng nhập.

Nhiệm vụ.

- Làm theo tài liệu của Streamlit để thiết lập một ứng dụng cơ bản.
- Thêm một ô nhập văn bản và một nút bấm. Khi nút được bấm, hiển thị văn bản người dùng đã nhập lên màn hình.

Bài tập 2: Tạo một agent profile cơ bản

Mục tiêu. Hiểu quy trình tạo và áp dụng agent profile trong Nexus.

Nhiệm vụ.

- Tạo một agent profile mới với persona riêng biệt. Persona này nên có chủ đề hoặc đặc điểm cụ thể (ví dụ: một nhà sử học).
- Định nghĩa một tập phản hồi cơ bản phù hợp với persona này.
- Kiểm thử persona bằng cách tương tác với nó qua giao diện Nexus.

Bài tập 3: Phát triển một action tuỳ chỉnh

Mục tiêu. Học cách mở rộng chức năng của Nexus bằng cách phát triển một action tuỳ chỉnh.

Nhiệm vụ.

- Phát triển một action mới (ví dụ: `fetch_current_news`) tích hợp với một API giả lập (mock API) để lấy các tiêu đề tin tức mới nhất.
- Hiện thực action này dưới cả hai dạng: native function (code) và semantic function (dựa trên prompt).
- Kiểm thử action trong môi trường Nexus để đảm bảo nó hoạt động như mong đợi.

Bài tập 4: Tích hợp một API bên thứ ba

Mục tiêu. Nâng cao khả năng của một agent Nexus bằng cách tích hợp một API bên thứ ba thật.

Nhiệm vụ.

- Chọn một API công khai (ví dụ: API thời tiết hoặc tin tức) và tạo một action mới lấy dữ liệu từ API này.
- Bổ sung xử lý lỗi và đảm bảo agent xử lý được một cách hợp lý các trường hợp API thất bại hoặc trả về phản hồi bất ngờ.
- Kiểm thử kỹ việc tích hợp này trong Nexus.

Tóm tắt

- Nexus là một nền tảng phát triển agent mã nguồn mở đi kèm cuốn sách này. Nó được thiết kế để phát triển, kiểm thử và lưu trữ (host) AI agent, và được xây dựng trên Streamlit để tạo dashboard tương tác và giao diện chat.
- Streamlit, một framework ứng dụng web Python, cho phép phát triển nhanh các dashboard và ứng dụng chat thân thiện với người dùng. Framework này giúp khám phá và tương tác với các tính năng agent khác nhau một cách thuận tiện.
- Nexus hỗ trợ tạo và tuỳ chỉnh agent profile và persona, cho phép người dùng định nghĩa tính cách và hành vi của agent. Các profile này quy định cách agent tương tác và phản hồi đầu vào của người dùng.
- Nền tảng Nexus cho phép phát triển và tích hợp action và tool dạng semantic (dựa trên prompt) lẫn native (dựa trên code) vào agent. Điều này giúp tạo ra các agent có nhiều chức năng và phản hồi tốt.

- Là một nền tảng mã nguồn mở, Nexus được thiết kế để dễ mở rộng, khuyến khích cộng đồng đóng góp và bổ sung tính năng, tool và khả năng mới cho agent.
- Nexus linh hoạt, hỗ trợ nhiều tuỳ chọn triển khai, gồm giao diện web, API và (trong các phiên bản tương lai) một Discord bot, đáp ứng nhiều nhu cầu phát triển và kiểm thử.

Chương 9

Hiểu bộ nhớ và tri thức của agent

Chương này bao quát các nội dung:

- Truy xuất (retrieval) tri thức/bộ nhớ trong các AI function.
- Xây dựng workflow retrieval augmented generation (RAG, sinh văn bản có tăng cường truy xuất) bằng LangChain.
- Áp dụng RAG cho các hệ thống tri thức của agent trong Nexus.
- Các mẫu (pattern) truy xuất dùng cho bộ nhớ của agent.
- Cải thiện hệ thống truy xuất tăng cường bằng cách nén (compression) bộ nhớ và tri thức.

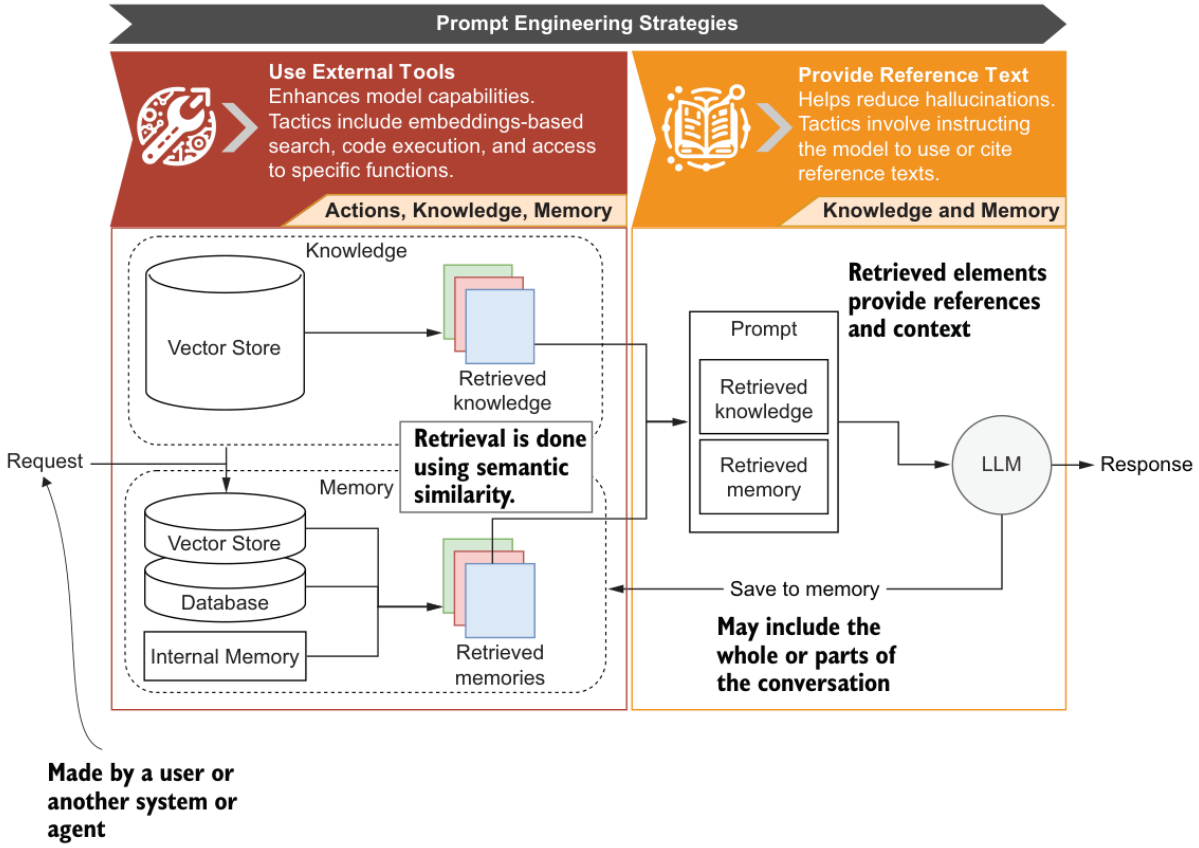
Ở chương trước, ta đã tìm hiểu action của agent thông qua các công cụ bên ngoài, chẳng hạn plugin dưới dạng native function hoặc semantic function. Giờ ta xem xét vai trò của **bộ nhớ** (memory) và **tri thức** (knowledge) dựa trên truy xuất trong agent và giao diện chat. Ta sẽ mô tả bộ nhớ, tri thức và mối liên hệ của chúng với các chiến lược prompt engineering. Sau đó, để hiểu rõ bộ nhớ và tri thức, ta tìm hiểu việc lập chỉ mục tài liệu (document indexing), xây dựng hệ thống truy xuất bằng LangChain, dùng bộ nhớ với LangChain và xây dựng semantic memory (bộ nhớ ngữ nghĩa) bằng Nexus.

9.1 Hiểu truy xuất trong ứng dụng AI

Trong các ứng dụng agent và chat, **truy xuất** (retrieval) là cơ chế lấy tri thức được lưu trữ, thường ở bên ngoài và tồn tại lâu dài. Tri thức *phi cấu trúc* gồm lịch sử hội thoại hoặc lịch sử tác vụ, các dữ kiện (fact), sở thích (preference) hay bất kỳ mục nào cần thiết để đặt prompt vào đúng ngữ cảnh. Tri thức *có cấu trúc*, thường lưu trong cơ sở dữ liệu hoặc file, được truy cập qua native function hoặc plugin.

Như trong Hình 9.1, bộ nhớ và tri thức là các thành phần dùng để bổ sung ngữ cảnh và thông tin liên quan vào prompt. Prompt có thể được tăng cường bằng đủ loại thông tin, từ thông tin về một tài liệu cho đến các tác vụ hay hội thoại trước đó và những thông tin tham khảo khác.

Các chiến lược prompt engineering trong Hình 9.1 áp dụng được cho cả bộ nhớ lẫn tri thức. Tri thức không được coi là bộ nhớ mà là phần tăng cường cho prompt lấy từ các tài liệu có sẵn. Cả tri thức và bộ nhớ đều dựa trên truy xuất làm nền tảng cho việc truy vấn thông tin phi cấu trúc.



Hình 9.1: Bộ nhớ, truy xuất và tăng cường prompt theo các chiến lược prompt engineering: Use External Tools (dùng công cụ bên ngoài) và Provide Reference Text (cung cấp văn bản tham khảo)

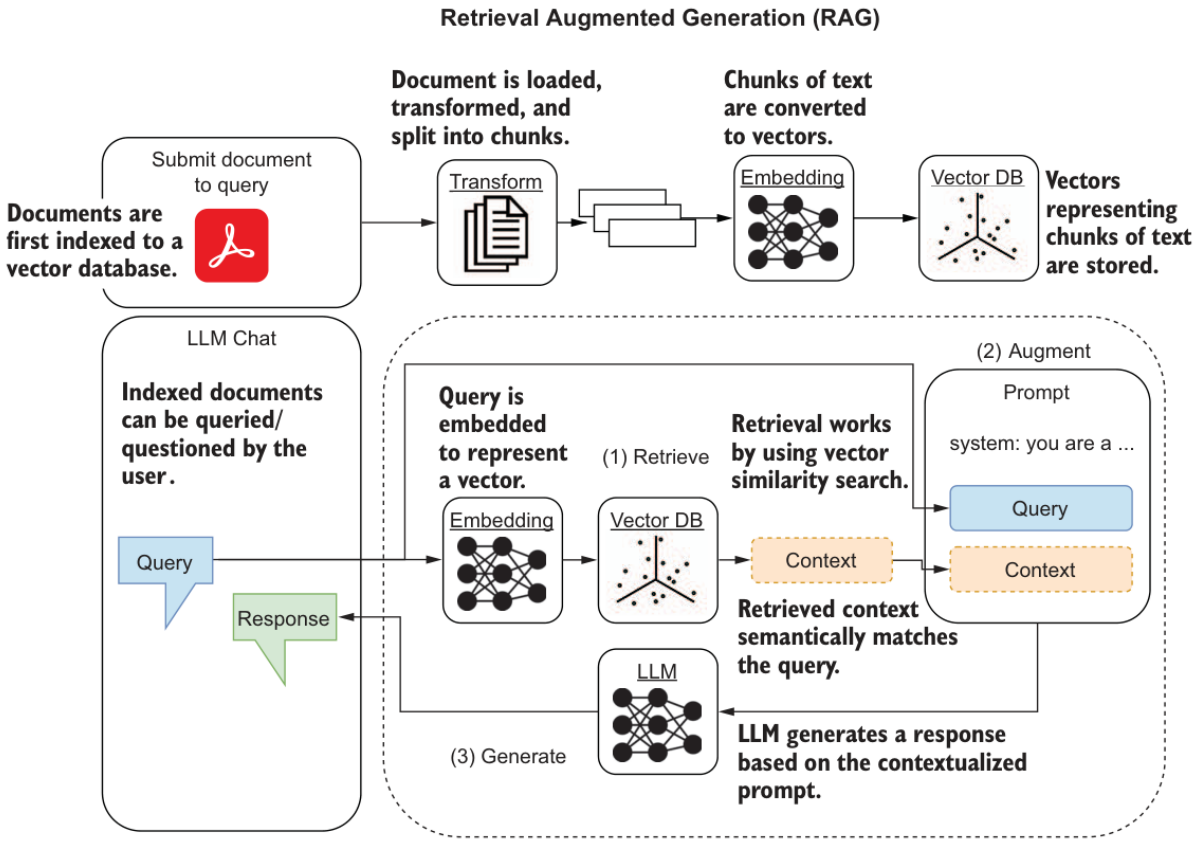
Cơ chế truy xuất này, gọi là **retrieval augmented generation** (RAG), đã trở thành chuẩn để cung cấp ngữ cảnh liên quan cho prompt. Chính cơ chế vận hành RAG cũng vận hành bộ nhớ/tri thức, nên việc hiểu cách nó hoạt động là rất cần thiết. Mục tiếp theo xem xét RAG là gì.

9.2 Cơ bản về retrieval augmented generation (RAG)

RAG đã trở thành cơ chế phổ biến để hỗ trợ chat với tài liệu (document chat) hoặc chat hỏi đáp (question-and-answer chat). Hệ thống thường hoạt động như sau: người dùng cung cấp một tài liệu liên quan, chẳng hạn một file PDF, rồi dùng RAG cùng một mô hình ngôn ngữ lớn (LLM) để truy vấn tài liệu đó.

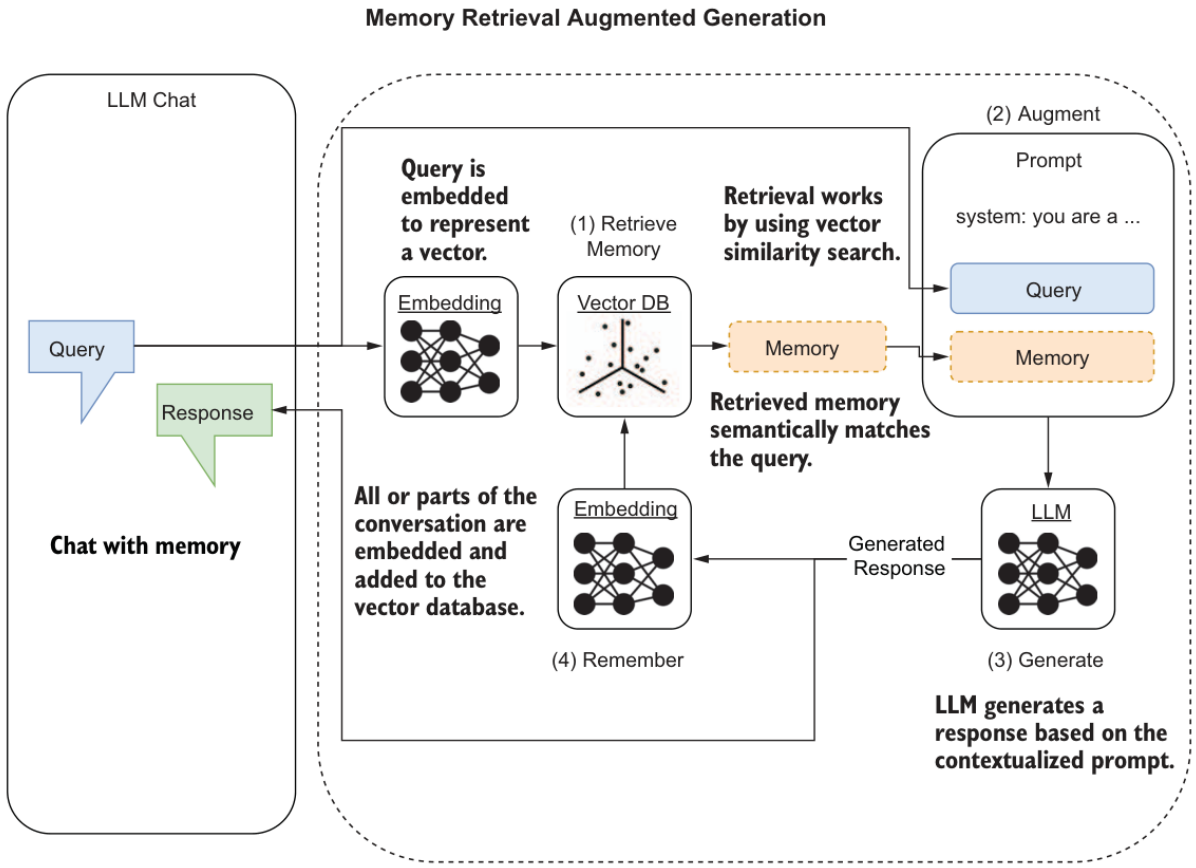
Hình 9.2 cho thấy RAG cho phép truy vấn một tài liệu bằng LLM như thế nào. Trước khi truy vấn được, tài liệu phải được nạp (load), biến đổi (transform) thành các đoạn ngữ cảnh (chunk), nhúng (embed) thành vector và lưu (store) vào một cơ sở dữ liệu vector (vector database).

Người dùng có thể truy vấn các tài liệu đã được lập chỉ mục bằng cách gửi một câu truy vấn. Câu truy vấn này được nhúng thành biểu diễn vector để tìm các chunk tương tự trong cơ sở dữ liệu vector. Nội dung tương tự với câu truy vấn được dùng làm ngữ cảnh và chèn vào prompt như phần tăng cường. Prompt được gửi tới LLM, và LLM có thể dùng thông tin ngữ cảnh đó để trả lời câu truy vấn.



Hình 9.2: Hai giai đoạn của RAG: thứ nhất, tài liệu phải được nạp, biến đổi, nhúng và lưu trữ; thứ hai, tài liệu có thể được truy vấn bằng cách sinh có tăng cường

Các khái niệm bộ nhớ/tri thức *phi cấu trúc* dựa trên một dạng tìm kiếm tương đồng văn bản (text-similarity search) theo mẫu truy xuất trong Hình 9.2. Hình 9.3 cho thấy bộ nhớ dùng cùng các thành phần embedding và cơ sở dữ liệu vector. Thay vì nạp sẵn tài liệu, các cuộc hội thoại hoặc từng phần của hội thoại được nhúng và lưu vào cơ sở dữ liệu vector.



Hình 9.3: Truy xuất bộ nhớ cho việc sinh có tăng cường dùng cùng các mẫu embedding để lập chỉ mục các mục vào cơ sở dữ liệu vector

Mẫu truy xuất và việc lập chỉ mục tài liệu có nhiều điểm tinh tế, cần cân nhắc kỹ để áp dụng thành công. Điều này đòi hỏi hiểu cách dữ liệu được lưu và truy xuất, nội dung ta bắt đầu tìm hiểu ở mục tiếp theo.

9.3 Tìm hiểu semantic search và lập chỉ mục tài liệu

Lập chỉ mục tài liệu (document indexing) biến đổi thông tin của tài liệu để dễ lấy lại hơn. Cách chỉ mục được truy vấn hay tìm kiếm cũng là một yếu tố, chẳng hạn ta muốn tìm một tập từ cụ thể hay muốn khớp từng cụm từ.

Semantic search (tìm kiếm ngữ nghĩa) là tìm nội dung khớp với cụm từ cần tìm theo cả từ ngữ lẫn ý nghĩa. Khả năng tìm theo ý nghĩa, tức theo ngữ nghĩa, rất mạnh và đáng được tìm hiểu kỹ. Ở mục tiếp theo, ta xem cách tìm kiếm tương đồng vector (vector similarity search) tạo nền móng cho semantic search.

9.3.1 Áp dụng tìm kiếm tương đồng vector

Giờ ta xem cách biến một tài liệu thành một **semantic vector** (vector ngữ nghĩa), tức một biểu diễn của văn bản có thể dùng để đo khoảng cách hoặc độ tương đồng. Có nhiều cách chuyển văn bản thành semantic vector, nên ta bắt đầu với một cách đơn giản.

Mở thư mục `chapter_08` trong một workspace Visual Studio Code (VS Code) mới. Tạo môi trường mới và chạy `pip install` với file `requirements.txt` để cài mọi thư viện cần cho chương. Nếu cần trợ giúp thiết lập môi trường Python mới, hãy xem phụ lục B.

Giờ mở file `document_vector_similarity.py` trong VS Code và xem phần đầu file trong Listing 8.1. Ví dụ này dùng **Term Frequency–Inverse Document Frequency** (TF–IDF). Đây là một đại lượng thống kê phản ánh mức độ quan trọng của một từ đối với một tài liệu trong một tập tài liệu: nó tăng tỷ lệ thuận với số lần từ xuất hiện trong tài liệu và bị bù trừ bởi tần suất của từ đó trong cả tập tài liệu. TF–IDF là thước đo kinh điển để hiểu tầm quan trọng của một tài liệu trong một tập tài liệu.

Listing 8.1 document_vector_similarity (biến đổi thành vector)

```
import plotly.graph_objects as go
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [    # Các tài liệu mẫu
    "The sky is blue and beautiful.",
    "Love this blue and beautiful sky!",
    "The quick brown fox jumps over the lazy dog.",
    "A king's breakfast has sausages, ham, bacon, eggs, toast, and beans",
    "I love green eggs, ham, sausages and bacon!",
    "The brown fox is quick and the blue dog is lazy!",
    "The sky is very blue and the sky is very beautiful today",
    "The dog is lazy but the brown fox is quick!"
]

vectorizer = TfidfVectorizer()    # Vector hoá bằng TF-IDF
X = vectorizer.fit_transform(documents)    # Vector hoá các tài liệu
```

Hãy tách TF–IDF thành hai thành phần, dùng câu mẫu "The sky is blue and beautiful" và tập trung vào từ *blue*.

Term Frequency (TF). **Term Frequency** (tần suất thuật ngữ) đo mức độ thường xuyên một thuật ngữ xuất hiện trong một tài liệu. Vì ta chỉ xét một tài liệu (câu mẫu), dạng đơn giản nhất của TF cho từ *blue* được tính bằng số lần *blue* xuất hiện trong tài liệu chia cho tổng số từ trong tài liệu. Ta tính:

- Số lần *blue* xuất hiện trong tài liệu: 1.
- Tổng số từ trong tài liệu: 6.

$$\text{TF} = 1 \div 6, \qquad \text{TF} = .16$$

Inverse Document Frequency (IDF). **Inverse Document Frequency** (tần suất tài liệu nghịch đảo) đo mức độ quan trọng của một thuật ngữ trong toàn bộ kho văn bản (corpus). Nó được tính bằng cách chia tổng số tài liệu cho số tài liệu chứa thuật ngữ đó rồi lấy logarit của thương:

$$\text{IDF} = \log\left(\frac{\text{Tổng số tài liệu}}{\text{Số tài liệu chứa từ}}\right)$$

Trong ví dụ này, corpus là một tập nhỏ gồm tám tài liệu, và *blue* xuất hiện trong bốn tài liệu:

$$\text{IDF} = \log(8 \div 4)$$

Tính TF–IDF. Cuối cùng, điểm TF–IDF của *blue* trong câu mẫu được tính bằng cách nhân điểm TF với điểm IDF:

$$\text{TF–IDF} = \text{TF} \times \text{IDF}$$

Hãy tính giá trị TF–IDF thực tế của từ *blue* theo ví dụ đã cho. Trước hết, tần suất thuật ngữ (từ xuất hiện thường xuyên đến đâu trong tài liệu) được tính như sau:

$$\text{TF} = 1 \div 6$$

Giả sử cơ số logarit là 10 (cơ số thường dùng), tần suất tài liệu nghịch đảo được tính như sau:

$$\text{IDF} = \log_{10}(8 \div 4)$$

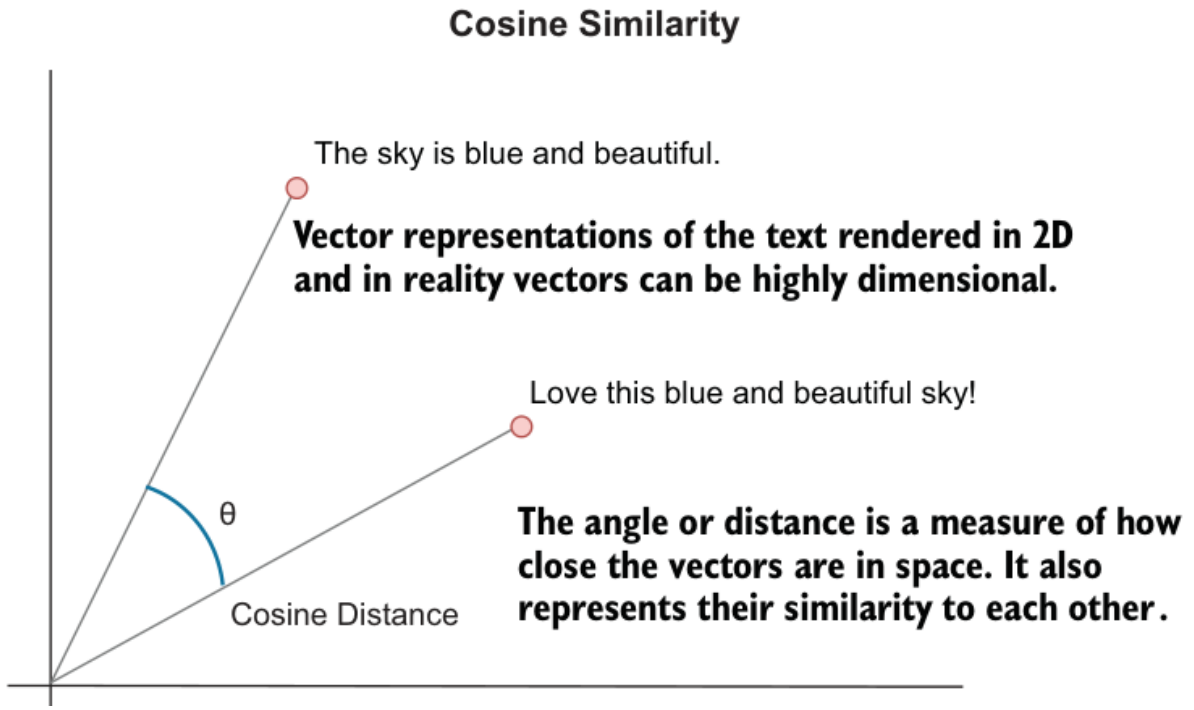
Giờ tính giá trị TF–IDF chính xác cho từ *blue* trong câu "The sky is blue and beautiful":

- Term Frequency (TF) xấp xỉ 0.1670.
- Inverse Document Frequency (IDF) xấp xỉ 0.301.
- Do đó, điểm TF–IDF (TF $\times$ IDF) của *blue* xấp xỉ 0.050.

Điểm TF–IDF này cho biết tầm quan trọng tương đối của từ *blue* trong tài liệu đã cho (câu mẫu), trong ngữ cảnh của corpus đã nêu (tám tài liệu, *blue* xuất hiện trong bốn tài liệu). Điểm TF–IDF càng cao thì từ càng quan trọng.

Ta dùng TF–IDF ở đây vì nó dễ áp dụng và dễ hiểu. Giờ các phần tử đã được biểu diễn thành vector, ta có thể đo độ tương đồng giữa các tài liệu bằng **cosine similarity** (độ tương đồng cosine). Cosine similarity là thước đo tính cosin của góc giữa hai vector khác không trong không gian nhiều chiều, cho biết hai vector giống nhau đến đâu, bất kể độ lớn của chúng.

Hình 9.4 cho thấy cách cosine distance so sánh biểu diễn vector của hai đoạn văn bản hoặc hai tài liệu. Cosine similarity trả về giá trị từ -1 (không tương đồng) đến 1 (giống hệt nhau). **Cosine distance** (khoảng cách cosine) là giá trị chuẩn hoá trong khoảng từ 0 đến 2, bằng 1 trừ cosine similarity. Cosine distance bằng 0 nghĩa là hai mục giống hệt nhau, còn bằng 2 nghĩa là hoàn toàn trái ngược.



Hình 9.4: Cách đo cosine similarity

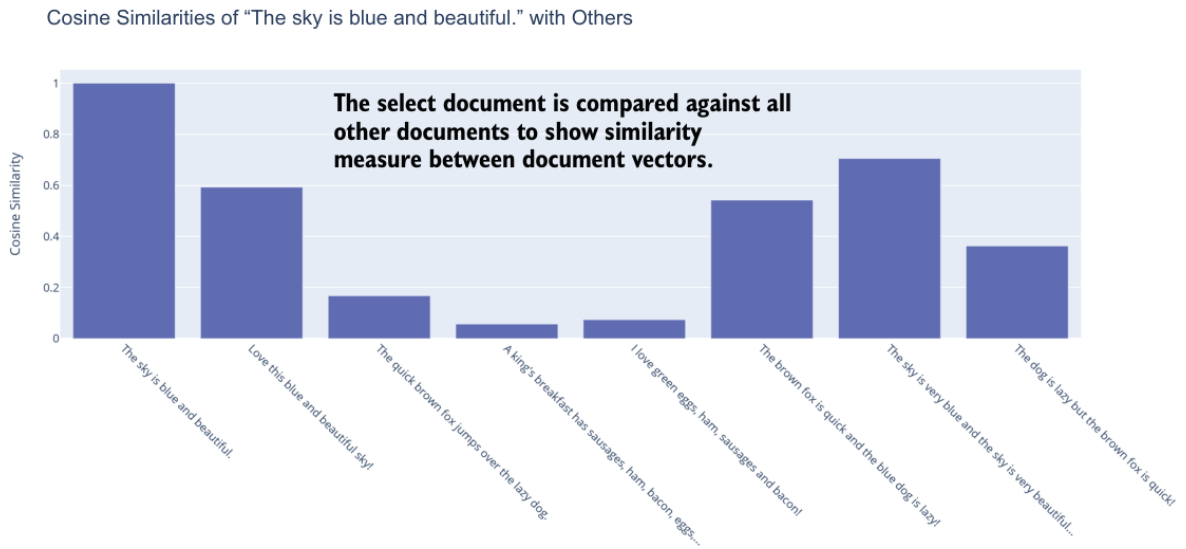
Listing 8.2 cho thấy cách tính cosine similarity bằng hàm `cosine_similarity` của scikit-learn. Độ tương đồng được tính cho từng tài liệu với mọi tài liệu khác trong tập. Ma trận độ tương đồng tính

được lưu trong biến `cosine_similarities`. Sau đó, trong vòng lặp nhập liệu, người dùng có thể chọn một tài liệu để xem độ tương đồng của nó với các tài liệu còn lại.

Listing 8.2 document_vector_similarity (cosine similarity)

```
cosine_similarities = cosine_similarity(X)    # Tính độ tương đồng cho mọi cặp
↪  vector

while True:    # Vòng lặp nhập liệu chính
    selected_document_index = input(f"Enter a document number
 (0-{len(documents)-1}) or 'exit' to quit: ").strip()

    if selected_document_index.lower() == 'exit':
        break

    if not selected_document_index.isdigit() or
 not 0 <= int(selected_document_index) < len(documents):
        print("Invalid input. Please enter a valid document number.")
        continue

    selected_document_index = int(selected_document_index)    # Lấy chỉ số tài
    ↪  liệu được chọn để so sánh

    selected_document_similarities =
     cosine_similarities[selected_document_index]    # Lấy độ tương đồng đã tính
     ↪  với mọi tài liệu

# code to plot document similarities omitted
```

Hình 9.5 là kết quả khi chạy ví dụ trong VS Code (F5 để chạy ở chế độ debug). Sau khi chọn một tài liệu, bạn sẽ thấy độ tương đồng giữa nó và các tài liệu trong tập. Một tài liệu luôn có cosine similarity bằng 1 với chính nó. Lưu ý rằng bạn sẽ không thấy độ tương đồng âm, do cách vector hoá bằng TF–IDF. Về sau ta sẽ xem các cách tinh vi hơn để đo độ tương đồng ngữ nghĩa.



Hình 9.5: Cosine similarity giữa tài liệu được chọn và tập tài liệu

Phương pháp vector hoá quyết định cách đo độ tương đồng ngữ nghĩa giữa các tài liệu. Trước khi chuyển sang các phương pháp vector hoá tốt hơn, ta xem cách lưu vector để thực hiện tìm kiếm tương đồng vector.

9.3.2 Cơ sở dữ liệu vector và tìm kiếm tương đồng

Sau khi vector hoá, các tài liệu có thể được lưu trong một **vector database** (cơ sở dữ liệu vector) để thực hiện tìm kiếm tương đồng về sau. Để minh hoạ cách việc này hoạt động, ta có thể mô phỏng một cơ sở dữ liệu vector đơn giản bằng code Python một cách hiệu quả. Mở file `document_vector_database.py` trong VS Code, như trong Listing 8.3. Code này minh hoạ việc tạo một cơ sở dữ liệu vector trong bộ nhớ, rồi cho phép người dùng nhập văn bản để tìm kiếm trong cơ sở dữ liệu và trả về kết quả. Kết quả trả về gồm văn bản tài liệu và điểm tương đồng.

Listing 8.3 document_vector_database.py

```
# code above omitted
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
vector_database = X.toarray()    # Lưu các vector tài liệu vào một mảng

def cosine_similarity_search(query,    # Hàm thực hiện so khớp tương đồng với
↪  truy vấn, trả về các kết quả khớp và điểm tương đồng
                             database,
                             vectorizer,
                             top_n=5):
    query_vec = vectorizer.transform([query]).toarray()
    similarities = cosine_similarity(query_vec, database)[0]
    top_indices = np.argsort(-similarities)[:top_n]  # Top n indices
    return [(idx, similarities[idx]) for idx in top_indices]

while True:    # Vòng lặp nhập liệu chính
    query = input("Enter a search query (or 'exit' to stop): ")
    if query.lower() == 'exit':
        break
    top_n = int(input("How many top matches do you want to see? "))
    search_results = cosine_similarity_search(query,
                                              vector_database,
                                              vectorizer,
                                              top_n)

    print("Top Matched Documents:")    # Duyệt qua kết quả, in văn bản và điểm
    ↪  tương đồng
    for idx, score in search_results:
        print(f"- {documents[idx]} (Score: {score:.4f})")

    print("\n")
###Output
Enter a search query (or 'exit' to stop): blue
How many top matches do you want to see? 3
Top Matched Documents:
- The sky is blue and beautiful. (Score: 0.4080)
- Love this blue and beautiful sky! (Score: 0.3439)
- The brown fox is quick and the blue dog is lazy! (Score: 0.2560)
```

Hãy chạy bài tập này để xem kết quả (F5 trong VS Code). Nhập bất kỳ văn bản nào bạn muốn và xem các tài liệu được trả về. Dạng tìm kiếm này hoạt động tốt khi khớp từ và cụm từ với các từ và cụm từ tương tự. Tuy nhiên, nó bỏ sót ngữ cảnh và ý nghĩa của từ trong tài liệu. Ở mục tiếp theo, ta xem một cách biến đổi tài liệu thành vector để giữ lại ngữ nghĩa tốt hơn.

9.3.3 Làm rõ document embedding

TF–IDF là một dạng đơn giản cố gắng nắm bắt ngữ nghĩa trong tài liệu. Tuy nhiên, nó không đáng tin cậy vì chỉ đếm tần suất từ và không hiểu quan hệ giữa các từ. Một phương pháp hiện đại và tốt hơn dùng **document embedding**, một dạng vector hoá tài liệu giữ được ngữ nghĩa của tài liệu tốt hơn.

Các mạng embedding được xây dựng bằng cách huấn luyện mạng nơ-ron trên các tập dữ liệu lớn để ánh xạ từ, câu hoặc tài liệu thành các vector nhiều chiều, nắm bắt quan hệ ngữ nghĩa và cú pháp dựa trên ngữ cảnh và các quan hệ trong dữ liệu. Thông thường, bạn dùng một mô hình đã huấn luyện sẵn (pretrained) trên các tập dữ liệu khổng lồ để nhúng tài liệu và thực hiện embedding. Có nhiều nguồn cung cấp mô hình, trong đó có Hugging Face và tất nhiên là OpenAI.

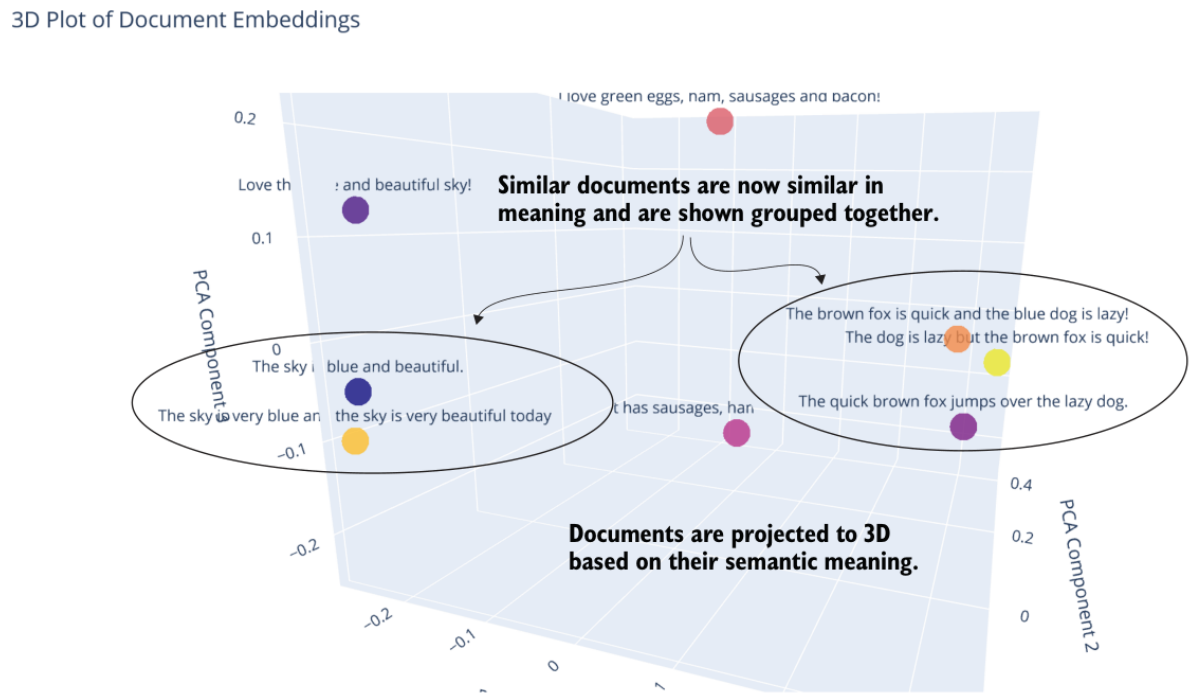
Trong kịch bản tiếp theo, ta dùng một mô hình embedding của OpenAI. Những mô hình này thường rất phù hợp để nắm bắt ngữ cảnh ngữ nghĩa của tài liệu được nhúng. Listing 8.4 cho thấy phần code liên quan dùng OpenAI để nhúng các tài liệu thành vector, sau đó giảm xuống ba chiều và vẽ thành biểu đồ.

Listing 8.4 document_visualizing_embeddings.py (các phần liên quan)

```
load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
if not api_key:
    raise ValueError("No API key found. Please check your .env file.")
client = OpenAI(api_key=api_key)

def get_embedding(text, model="text-embedding-ada-002"):
    text = text.replace("\n", " ")
    return client.embeddings.create(input=[text],    # Dùng OpenAI client để tạo
    ↪  embedding
               model=model).data[0].embedding

# Sample documents (omitted)

embeddings = [get_embedding(doc) for doc in documents]    # Sinh embedding cho
↪  từng tài liệu, mỗi embedding có 1536 chiều
embeddings_array = np.array(embeddings)    # Chuyển embedding thành mảng NumPy để
↪  dùng PCA
print(embeddings_array.shape)

pca = PCA(n_components=3)    # Áp dụng PCA để giảm xuống 3 chiều phục vụ vẽ biểu
↪  đồ
reduced_embeddings = pca.fit_transform(embeddings_array)
```

Ngoài các chú thích trên, bản gốc của sách còn một chú thích đứng cạnh phần code này: “Join all the items on the string ’, ’.” (nối mọi phần tử bằng chuỗi `', '`). Chú thích này không khớp với đoạn code được in trong Listing 8.4 (có vẻ là chú thích còn sót của phần code lược bỏ ở dòng `Sample documents (omitted)`).

Khi một tài liệu được nhúng bằng mô hình của OpenAI, văn bản được biến thành một vector 1536 chiều. Ta không thể hình dung số chiều lớn như vậy, nên dùng kỹ thuật giảm chiều **phân tích thành phần chính** (principal component analysis, PCA) để chuyển vector 1536 chiều xuống 3 chiều. Hình 9.6 cho thấy kết quả khi chạy file trong VS Code. Nhờ giảm embedding xuống 3D, ta có thể vẽ kết quả để thấy các tài liệu tương đồng về ngữ nghĩa giờ được gom nhóm với nhau.



Hình 9.6: Embedding trong không gian 3D, cho thấy các tài liệu tương đồng ngữ nghĩa được gom nhóm với nhau

Việc chọn mô hình embedding hay dịch vụ nào là tuỳ bạn. Các mô hình embedding của OpenAI được xem là tốt nhất cho độ tương đồng ngữ nghĩa tổng quát. Điều này khiến chúng trở thành chuẩn cho hầu hết ứng dụng bộ nhớ và truy xuất. Khi đã hiểu cách vector hoá văn bản bằng embedding và lưu trong cơ sở dữ liệu vector, ta chuyển sang một ví dụ thực tế hơn ở mục tiếp theo.

9.3.4 Truy vấn document embedding từ Chroma

Ta có thể kết hợp mọi mảnh ghép và xem một ví dụ hoàn chỉnh dùng cơ sở dữ liệu vector cục bộ tên **Chroma DB**. Có nhiều lựa chọn cơ sở dữ liệu vector, nhưng Chroma DB là kho vector cục bộ rất tốt cho phát triển hoặc dự án quy mô nhỏ. Sau này bạn cũng có thể cân nhắc nhiều lựa chọn mạnh mẽ hơn.

Listing 8.5 cho thấy các phần code mới và liên quan từ file `document_query_chromadb.py`. Lưu ý rằng kết quả được chấm điểm theo khoảng cách chứ không theo độ tương đồng. Cosine distance được xác định bằng phương trình:

$$\text{Cosine Distance}(A, B) = 1 - \text{Cosine Similarity}(A, B)$$

Điều này có nghĩa cosine distance nằm trong khoảng từ 0 (giống nhau nhất) đến 2 (đối nghịch nhau về ý nghĩa).

Listing 8.5 document_query_chromadb.py (các phần code liên quan)

```
embeddings = [get_embedding(doc) for doc in documents]    # Sinh embedding cho
↪  từng tài liệu và gán ID
ids = [f"id{i}" for i in range(len(documents))]

chroma_client = chromadb.Client()    # Tạo Chroma DB client và một collection
collection = chroma_client.create_collection(
                      name="documents")
collection.add(    # Thêm embedding của tài liệu vào collection
```

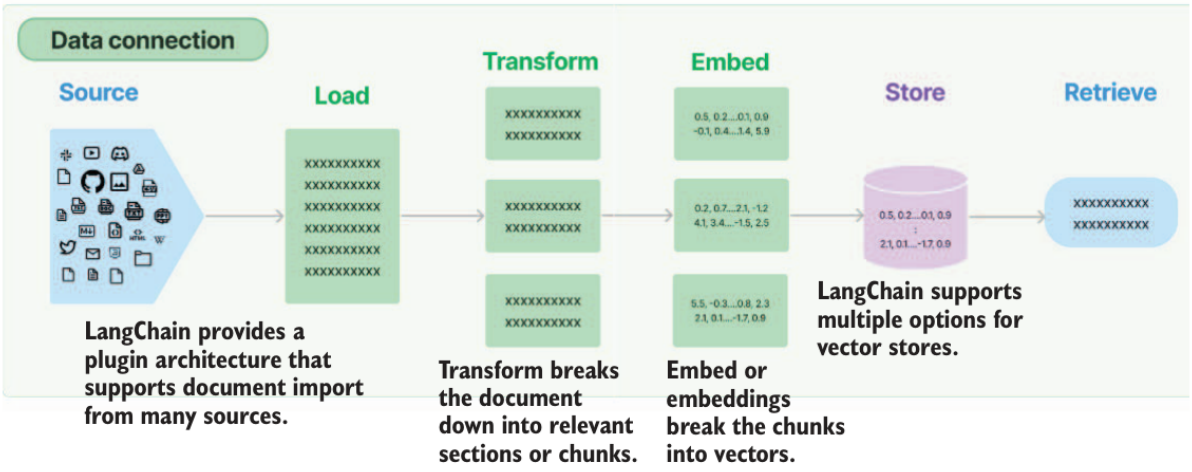
```
    embeddings=embeddings,
    documents=documents,
    ids=ids
)

def query_chromadb(query, top_n=2):    # Truy vấn kho dữ liệu và trả về top n tài
↪   liệu liên quan
    query_embedding = get_embedding(query)
    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_n
    )
    return [(id, score, text) for id, score, text in
            zip(results['ids'][0],
                results['distances'][0],
                results['documents'][0])]

while True:    # Vòng lặp nhập liệu và xuất tài liệu/điểm liên quan
    query = input("Enter a search query (or 'exit' to stop): ")
    if query.lower() == 'exit':
        break
    top_n = int(input("How many top matches do you want to see? "))
    search_results = query_chromadb(query, top_n)

    print("Top Matched Documents:")
    for id, score, text in search_results:
        print(f"""
ID:{id} TEXT: {text} SCORE: {round(score, 2)}
""")

    print("\n")
###Output
Enter a search query (or 'exit' to stop): dogs are lazy
How many top matches do you want to see? 3
Top Matched Documents:
ID:id7 TEXT: The dog is lazy but the brown fox is quick! SCORE: 0.24
ID:id5 TEXT: The brown fox is quick and the blue dog is lazy! SCORE: 0.28
ID:id2 TEXT: The quick brown fox jumps over the lazy dog. SCORE: 0.29
```

Như kịch bản trên minh hoạ, giờ ta có thể truy vấn tài liệu theo ý nghĩa ngữ nghĩa thay vì chỉ theo từ khoá hay cụm từ. Các kịch bản này cung cấp nền tảng để thấy mẫu truy xuất (retrieval pattern) hoạt động ra sao ở mức thấp. Ở mục tiếp theo, ta xem mẫu truy xuất được áp dụng thế nào với LangChain.

9.4 Xây dựng RAG với LangChain

LangChain khởi đầu là một dự án mã nguồn mở chuyên trừu tượng hoá mẫu truy xuất trên nhiều nguồn dữ liệu và kho vector. Về sau nó phát triển thành nhiều thứ hơn thế, nhưng về nền tảng, nó vẫn cung cấp những lựa chọn tuyệt vời để triển khai truy xuất.

Hình 9.7 là một sơ đồ từ LangChain xác định quy trình lưu tài liệu để truy xuất. Chính những bước này có thể được lặp lại toàn bộ hoặc một phần để triển khai truy xuất bộ nhớ. Điểm khác biệt then chốt giữa truy xuất tài liệu và truy xuất bộ nhớ là nguồn dữ liệu và cách nội dung được biến đổi.



Hình 9.7: Các bước nạp (load), biến đổi (transform), nhúng (embed) và lưu (store) khi lưu tài liệu để truy xuất sau này

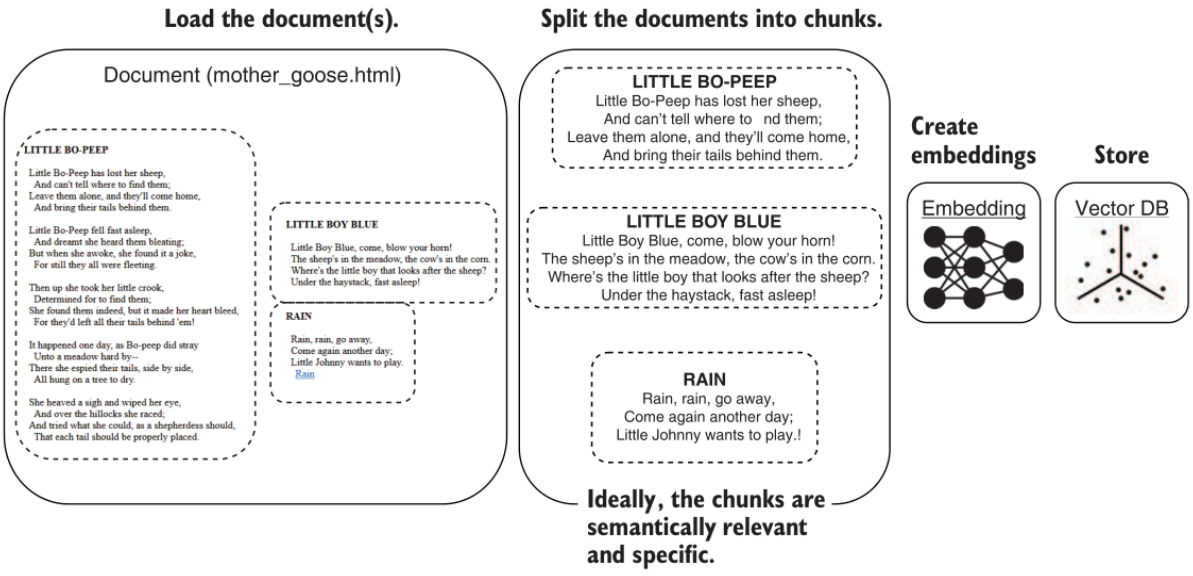
Ta sẽ xem cách triển khai từng bước này bằng LangChain và hiểu các sắc thái, chi tiết đi kèm. Ở mục tiếp theo, ta bắt đầu với việc tách và nạp tài liệu bằng LangChain.

9.4.1 Tách và nạp tài liệu với LangChain

Các cơ chế truy xuất bổ sung ngữ cảnh của một prompt bằng thông tin cụ thể liên quan đến yêu cầu. Ví dụ, bạn có thể yêu cầu thông tin chi tiết về một tài liệu cục bộ. Với các mô hình ngôn ngữ trước đây, việc gửi cả tài liệu làm một phần của prompt không khả thi do giới hạn token.

Ngày nay, ta có thể gửi cả một tài liệu cho nhiều LLM thương mại, chẳng hạn GPT-4 Turbo, như một phần của yêu cầu prompt. Tuy nhiên, kết quả có thể không tốt hơn và chắc chắn tốn kém hơn vì số token tăng lên. Do đó, lựa chọn tốt hơn là tách tài liệu và dùng các phần liên quan để cung cấp ngữ cảnh cho yêu cầu, đúng là điều RAG và bộ nhớ thực hiện.

Tách tài liệu là bước thiết yếu để chia nội dung thành các phần liên quan về ngữ nghĩa và cụ thể. Hình 9.8 cho thấy cách chia một tài liệu HTML chứa các bài đồng dao Mother Goose. Việc tách một tài liệu thành các đoạn (chunk) ngữ cảnh-ngữ nghĩa thường đòi hỏi cân nhắc cẩn thận.



Hình 9.8: Cách lý tưởng để tách tài liệu thành các chunk nhằm có ý nghĩa ngữ nghĩa và ngữ cảnh tốt hơn

Lý tưởng nhất, khi tách tài liệu thành các chunk, ta chia theo mức độ liên quan và ý nghĩa ngữ nghĩa. Một LLM hay agent có thể giúp việc này, nhưng ở đây ta xem các lựa chọn bộ công cụ hiện có trong LangChain để tách tài liệu. Về sau trong chương, ta sẽ xem một semantic function có thể hỗ trợ chia nội dung theo ngữ nghĩa cho embedding.

Cho bài tập tiếp theo, mở `langchain_load_splitting.py` trong VS Code, như trong Listing 8.6. Code này tiếp tục từ chỗ ta dừng ở Listing 8.5 ở mục trước. Lần này, thay vì dùng các tài liệu mẫu, ta nạp các bài đồng dao Mother Goose.

Listing 8.6 langchain_load_splitting.py (các phần code và kết quả)

```
from langchain_community.document_loaders
                    import UnstructuredHTMLLoader    # Các import mới của
                    ↪ LangChain
from langchain.text_splitter import RecursiveCharacterTextSplitter

#previous code

loader = UnstructuredHTMLLoader(    # Nạp tài liệu dưới dạng HTML
                   "sample_documents/mother_goose.html")
data = loader.load    # Nạp tài liệu

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=25,    # Tách tài liệu thành các khối văn bản dài 100 ký tự,
    ↪ chồng lấn 25 ký tự
    length_function=len,
    add_start_index=True,
)
documents = text_splitter.split_documents(data)

documents = [doc.page_content
                for doc in documents] [100:350]    # Chỉ nhúng 250 chunk, rẻ và
                ↪ nhanh hơn

embeddings = [get_embedding(doc) for doc in documents]    # Trả về embedding cho
↪ từng tài liệu
ids = [f"id{i}" for i in range(len(documents))]
###Output
Enter a search query (or 'exit' to stop): who kissed the girls and made
them cry?
How many top matches do you want to see? 3
Top Matched Documents:
ID:id233 TEXT: And chid her daughter,
        And kissed my sister instead of me. SCORE: 0.4…
```

Lưu ý trong Listing 8.6 rằng tài liệu HTML được tách thành các chunk 100 ký tự với 25 ký tự chồng lấn. Phần chồng lấn giúp các phần của tài liệu không cắt cụt những ý cụ thể. Ta chọn bộ tách này cho bài tập vì nó dễ dùng, dễ thiết lập và dễ hiểu.

Hãy chạy file `langchain_load_splitting.py` trong VS Code (F5). Nhập một truy vấn và xem bạn nhận được kết quả gì. Kết quả trong Listing 8.6 cho thấy kết quả tốt với một ví dụ cụ thể. Hãy nhớ rằng ta chỉ nhúng 250 chunk tài liệu để giảm chi phí và giữ bài tập ngắn gọn. Dĩ nhiên, bạn luôn có thể thử nhúng toàn bộ tài liệu hoặc dùng một tài liệu đầu vào nhỏ hơn.

Có lẽ yếu tố quan trọng nhất để xây dựng truy xuất đúng đắn là quá trình tách tài liệu. Có nhiều phương pháp để tách một tài liệu, kể cả nhiều phương pháp đồng thời. Dùng nhiều hơn một phương pháp sẽ duyệt và tách tài liệu để có nhiều góc nhìn embedding cho cùng một tài liệu. Ở mục tiếp theo, ta xem một kỹ thuật tổng quát hơn để tách tài liệu, dùng token và tokenization.

9.4.2 Tách tài liệu theo token với LangChain

Tokenization (token hoá) là quá trình chia văn bản thành các token từ. Một token từ đại diện cho một phần tử ngắn gọn trong văn bản, có thể là một từ như *hold* hoặc thậm chí một ký hiệu như dấu ngoặc nhọn trái ({), tuỳ vào điều gì có liên quan.

Tách tài liệu bằng tokenization tạo nền tảng tốt hơn cho cách mô hình ngôn ngữ diễn giải văn bản và cho độ tương đồng ngữ nghĩa. Tokenization còn cho phép loại bỏ các ký tự không liên quan, chẳng hạn khoảng trắng, nhờ đó việc so khớp tương đồng giữa các tài liệu sát nghĩa hơn và nhìn chung cho kết quả tốt hơn.

Cho bài tập code tiếp theo, mở file `langchain_token_splitting.py` trong VS Code, như trong Listing 8.7. Lần này ta tách tài liệu bằng tokenization, chia tài liệu thành các đoạn có kích thước không đều. Kích thước không đều là do tài liệu gốc có những vùng khoảng trắng lớn.

Listing 8.7 langchain_token_splitting.py (code mới liên quan)

```
loader = UnstructuredHTMLLoader("sample_documents/mother_goose.html")
data = loader.load()
text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=50, chunk_overlap=10     # Cập nhật thành 50 token và chồng lấn 10
    ↪  token
)
documents = text_splitter.split_documents(data)
documents = [doc for doc in documents][8:94]     # Chỉ chọn các tài liệu chứa
↪  đồng dao

db = Chroma.from_documents(documents, OpenAIEmbeddings())

def query_documents(query, top_n=2):
    docs = db.similarity_search(query, top_n)     # Dùng chức năng tìm kiếm tương
    ↪  đồng của cơ sở dữ liệu
    return docs
###Output
Created a chunk of size 68,     # Tách thành các chunk kích thước không đều do
↪  khoảng trắng
which is longer than the specified 50
Created a chunk of size 67,
which is longer than the specified 50
Enter a search query (or 'exit' to stop):
                    who kissed the girls and made them cry?
How many top matches do you want to see? 3
Top Matched Documents:
Document 1: GEORGY PORGY

        Georgy Porgy, pudding and pie,
        Kissed the girls and made them cry.
```

Hãy chạy `langchain_token_splitting.py` trong VS Code (F5). Bạn có thể dùng câu truy vấn lần trước hoặc câu của riêng mình. Lưu ý rằng kết quả tốt hơn đáng kể so với bài tập trước. Tuy nhiên, kết quả vẫn còn đáng ngờ vì câu truy vấn dùng nhiều từ giống nhau theo cùng thứ tự.

Một phép thử tốt hơn là dùng một cụm từ tương đồng về ngữ nghĩa nhưng dùng từ khác, rồi kiểm tra kết quả. Khi chương trình vẫn đang chạy, hãy nhập một câu truy vấn mới: `Why are the girls crying?`. Listing 8.8 cho thấy kết quả của câu truy vấn đó. Nếu tự chạy ví dụ và cuộn xuống phần đầu ra, bạn sẽ thấy Georgy Porgy xuất hiện ở tài liệu trả về thứ hai hoặc thứ ba.

Listing 8.8 Truy vấn: Who made the girls cry?

```
Enter a search query (or 'exit' to stop): Who made the girls cry?
```

```
How many top matches do you want to see? 3
Top Matched Documents:
Document 1: WILLY, WILLY

        Willy, Willy Wilkin…
```

Bài tập này cho thấy có thể dùng nhiều phương pháp truy xuất khác nhau để trả về tài liệu theo ngữ nghĩa. Với nền tảng này, ta có thể xem RAG được áp dụng thế nào cho các hệ thống tri thức và bộ nhớ. Mục tiếp theo bàn về RAG khi áp dụng cho tri thức của agent và các hệ thống agentic.

9.5 Áp dụng RAG để xây dựng tri thức cho agent

Tri thức (knowledge) trong agent là việc dùng RAG để tìm kiếm theo ngữ nghĩa trên các tài liệu phi cấu trúc. Các tài liệu này có thể là bất cứ thứ gì, từ PDF đến tài liệu Microsoft Word và mọi loại văn bản, kể cả code. Tri thức của agent cũng bao gồm việc dùng tài liệu phi cấu trúc cho hỏi đáp (Q&A), tra cứu tham khảo, bổ sung thông tin và các mẫu khác trong tương lai.

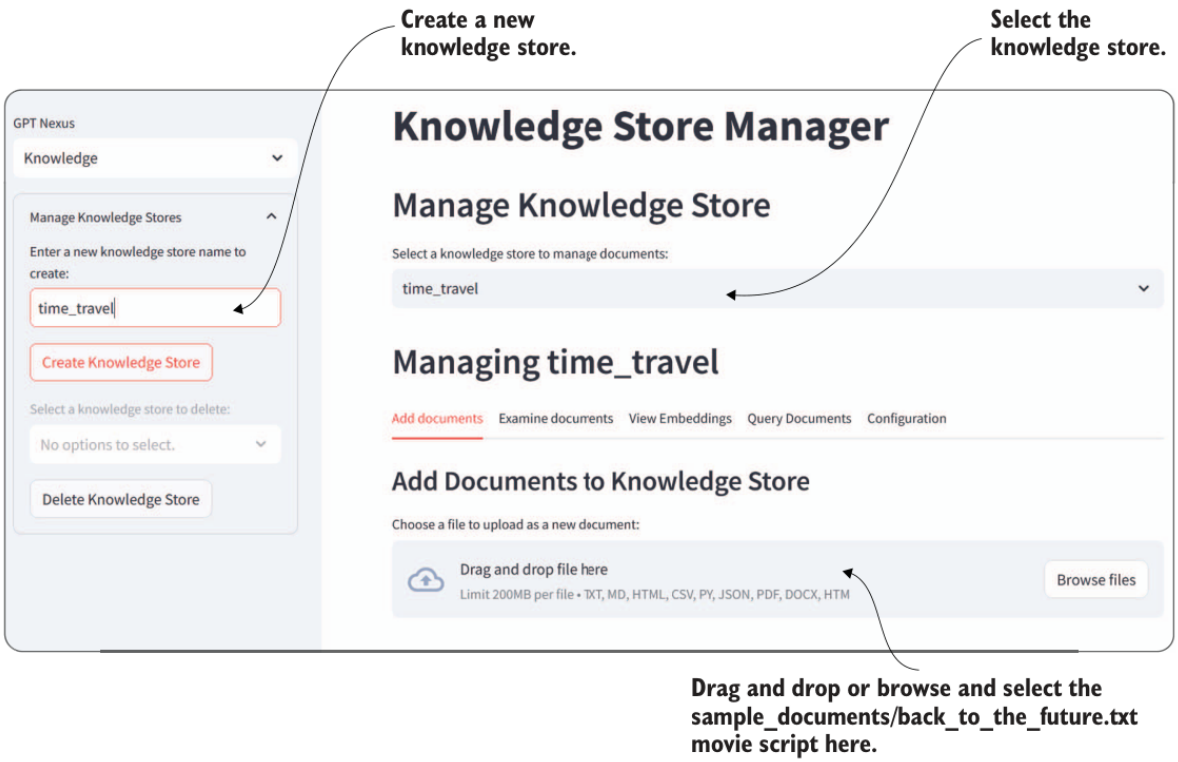
Nexus, nền tảng agent được phát triển song song với cuốn sách và đã giới thiệu ở chương trước, có sẵn các hệ thống tri thức và bộ nhớ hoàn chỉnh cho agent. Trong mục này, ta tìm hiểu hệ thống tri thức hoạt động ra sao.

Để cài Nexus chỉ cho riêng chương này, xem Listing 8.9. Mở một terminal trong thư mục `chapter_08` và chạy các lệnh trong listing để tải về, cài đặt và chạy Nexus ở chế độ thường hoặc chế độ phát triển (development). Nếu muốn tham khảo code, bạn nên cài dự án ở chế độ phát triển và cấu hình debugger để chạy ứng dụng Streamlit từ VS Code. Xem lại Chương 8 nếu cần nhắc lại các bước này.

Listing 8.9 Cài đặt Nexus

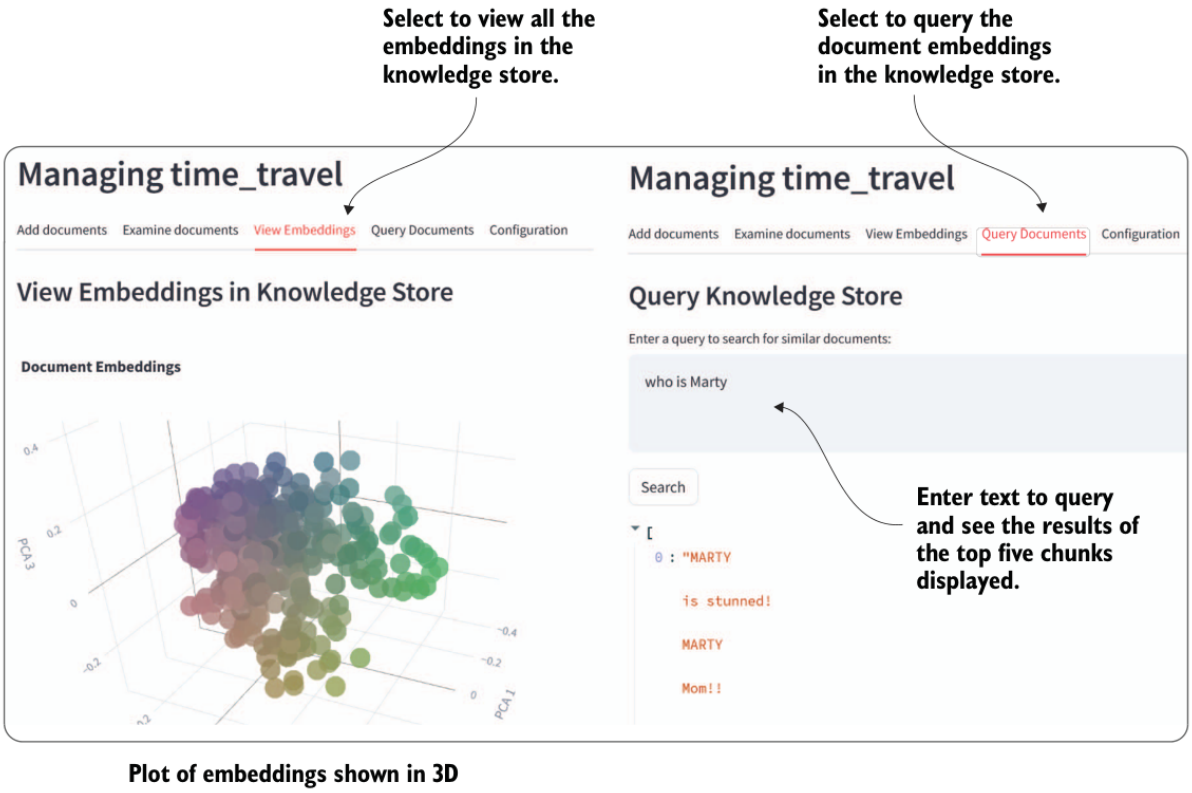
```
# to install and run
pip install git+https://github.com/cxbxmxcx/Nexus.git

nexus run
# install in development mode
git clone https://github.com/cxbxmxcx/Nexus.git

# Install the cloned repository in editable mode
pip install -e Nexus
```

Dù chạy ứng dụng theo cách nào, sau khi đăng nhập, hãy vào trang **Knowledge Store Manager** như trong Hình 9.9. Tạo một Knowledge Store mới, rồi tải lên kịch bản phim `sample_documents/back_to_the_future.txt`.



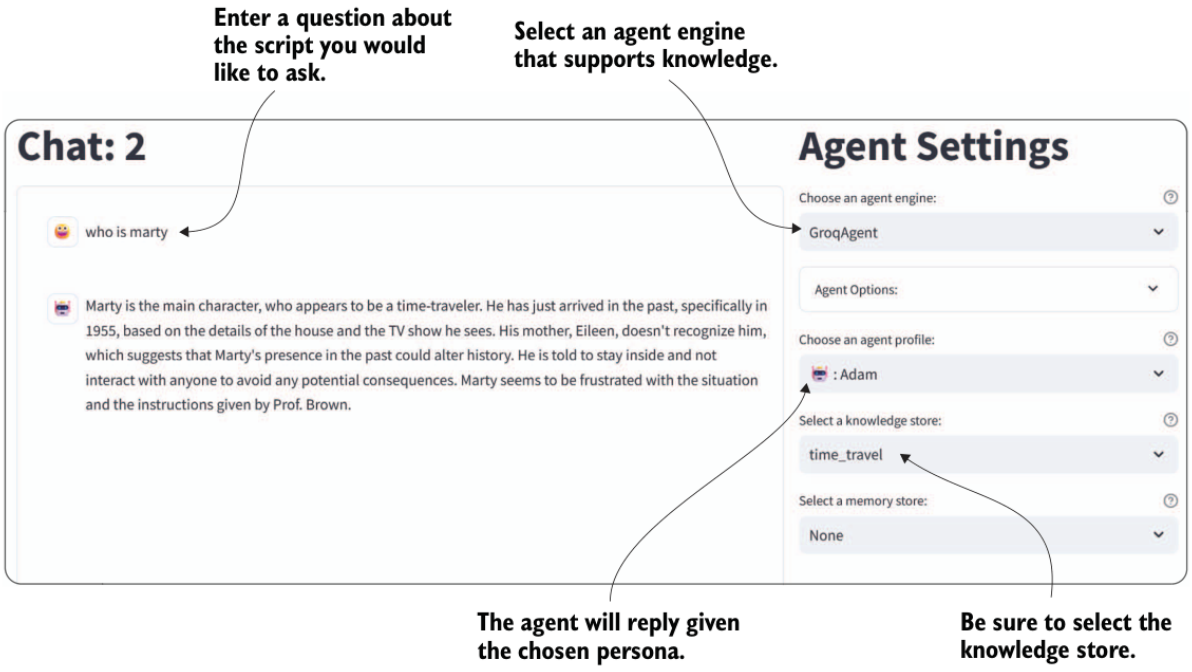
Hình 9.9: Thêm một knowledge store mới và nạp tài liệu vào đó

Kịch bản này là một tài liệu lớn, nên có thể mất một lúc để nạp, chia chunk và nhúng các phần vào cơ sở dữ liệu vector Chroma DB. Hãy chờ lập chỉ mục hoàn tất, sau đó bạn có thể kiểm tra các embedding và chạy một truy vấn, như trong Hình 9.10.



Hình 9.10: Các màn hình xem embedding và truy vấn tài liệu

Giờ ta có thể kết nối knowledge store với một agent được hỗ trợ và đặt câu hỏi. Dùng bộ chọn ở góc trên bên trái để chọn trang chat trong giao diện Nexus. Sau đó chọn một agent và knowledge store `time_travel`, như trong Hình 9.11. Bạn cũng cần chọn một agent engine có hỗ trợ tri thức. Mỗi agent engine trong số nhiều engine đều cần cấu hình đúng thì mới truy cập được.



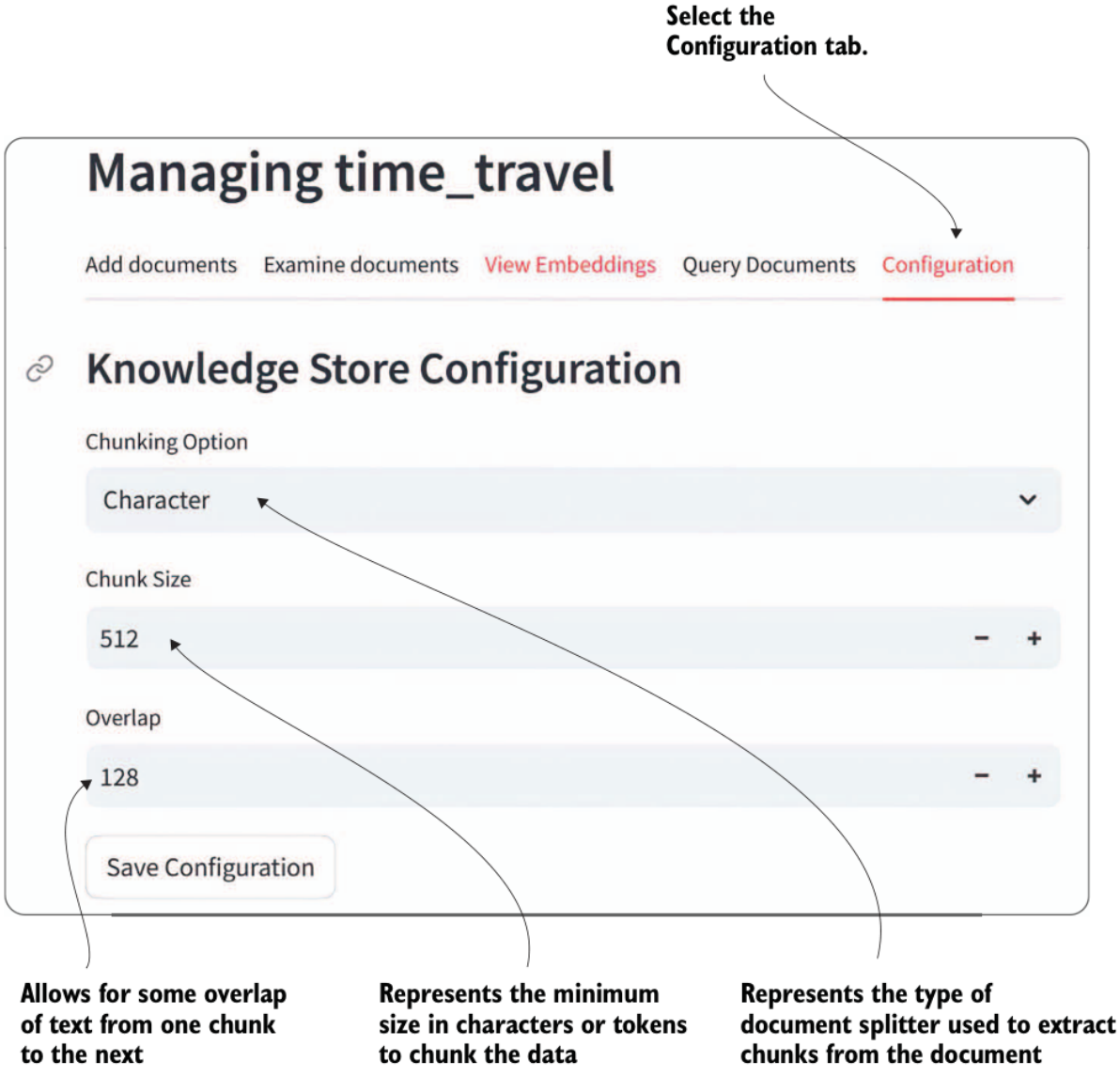
Hình 9.11: Bật knowledge store để agent sử dụng

Tại thời điểm của chương này, Nexus chỉ hỗ trợ truy cập một knowledge store mỗi lần. Ở phiên bản tương lai, agent có thể chọn nhiều knowledge store cùng lúc. Điều đó có thể bao gồm các tuỳ chọn nâng cao hơn, từ tri thức ngữ nghĩa (semantic knowledge) đến việc dùng các dạng RAG khác.

Bạn cũng có thể cấu hình các thiết lập RAG trong tab **Configuration** của trang Knowledge Store Manager, như trong Hình 9.12. Hiện tại, bạn có thể chọn loại bộ tách dùng để chia chunk tài liệu (trường *Chunking Option*), cùng với trường *Chunk Size* và trường *Overlap*.

Các tuỳ chọn nạp, tách, chia chunk và nhúng được cung cấp hiện chỉ là những tuỳ chọn cơ bản mà LangChain hỗ trợ. Các phiên bản Nexus sau sẽ có thêm nhiều tuỳ chọn và mẫu hơn. Code hỗ trợ các tuỳ chọn khác có thể được bổ sung trực tiếp vào Nexus.

Ta sẽ không đi vào code thực hiện RAG vì nó rất giống những gì đã trình bày. Bạn cứ tự do xem code của Nexus, đặc biệt là lớp `KnowledgeManager` trong file `knowledge_manager.py`.



Hình 9.12: Quản lý các tuỳ chọn tách và chia chunk của knowledge store

Dù mẫu truy xuất cho tri thức và bộ nhớ khá giống nhau ở khâu tăng cường prompt, hai mẫu này khác nhau ở cách nạp dữ liệu vào kho lưu trữ. Mục tiếp theo khám phá điều làm bộ nhớ của agent trở nên đặc thù.

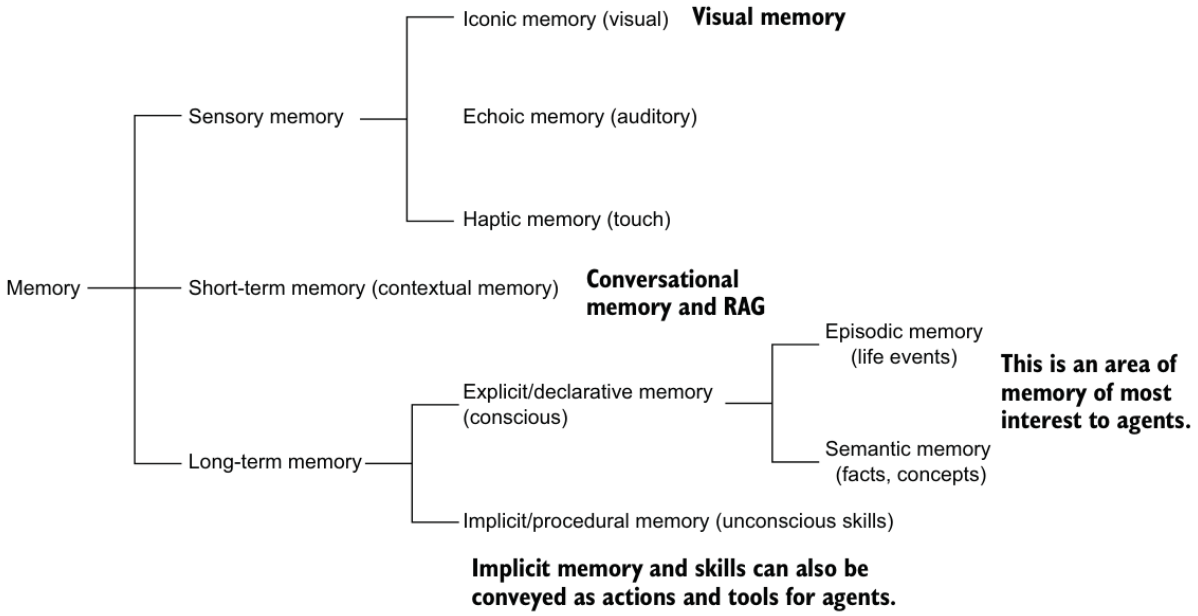
9.6 Hiện thực bộ nhớ trong các hệ thống agentic

Bộ nhớ trong agent và ứng dụng AI thường được mô tả bằng cùng các thuật ngữ như các chức năng của bộ nhớ nhận thức (cognitive memory). **Cognitive memory** (bộ nhớ nhận thức) mô tả loại trí nhớ ta dùng để nhớ những gì mình đã làm 30 giây trước hoặc mình cao bao nhiêu cách đây 30 năm. Bộ nhớ máy tính cũng là một yếu tố quan trọng của bộ nhớ agent, nhưng ta sẽ không xét trong mục này.

Hình 9.13 cho thấy bộ nhớ được chia thành bộ nhớ giác quan (sensory), ngắn hạn (short-term) và dài hạn (long-term). Các dạng bộ nhớ này áp dụng được cho AI agent, và danh sách dưới đây mô tả mỗi dạng bộ nhớ ánh xạ sang chức năng nào của agent:

- **Sensory memory trong AI**: các chức năng như RAG nhưng với dữ liệu hình ảnh/âm thanh/xúc giác. Giữ ngắn hạn dữ liệu đầu vào (ví dụ văn bản và hình ảnh) để xử lý tức thời, nhưng không lưu trữ dài hạn.
- **Short-term/working memory trong AI**: đóng vai trò bộ đệm bộ nhớ chủ động cho lịch sử hội thoại. Ta giữ một lượng giới hạn đầu vào và ngữ cảnh gần đây để phân tích và sinh phản hồi tức thời. Trong Nexus, bộ nhớ hội thoại ngắn hạn và dài hạn được giữ trong ngữ cảnh của thread.
- **Long-term memory trong AI**: lưu trữ bộ nhớ lâu hơn, liên quan đến cuộc sống của agent hoặc người dùng. Semantic memory cung cấp khả năng mạnh mẽ để lưu và truy xuất các dữ kiện, khái niệm mang tính toàn cục hoặc cục bộ có liên quan.

- *Sensory memory in AI*—Functions such as RAG but with images/audio/haptic data forms. Briefly holds input data (e.g., text and images) for immediate processing but not long-term storage.
- *Short-term/working memory in AI*—Acts as an active memory buffer of conversation history. We're holding a limited amount of recent input and context for immediate analysis and response generation. Within Nexus, short- and long-term conversational memory is also held in the context of the thread.
- *Long-term memory in AI*—Longer-term memory storage relevant to the agent's or user's life. Semantic memory provides a robust capacity to store and retrieve relevant global or local facts and concepts.

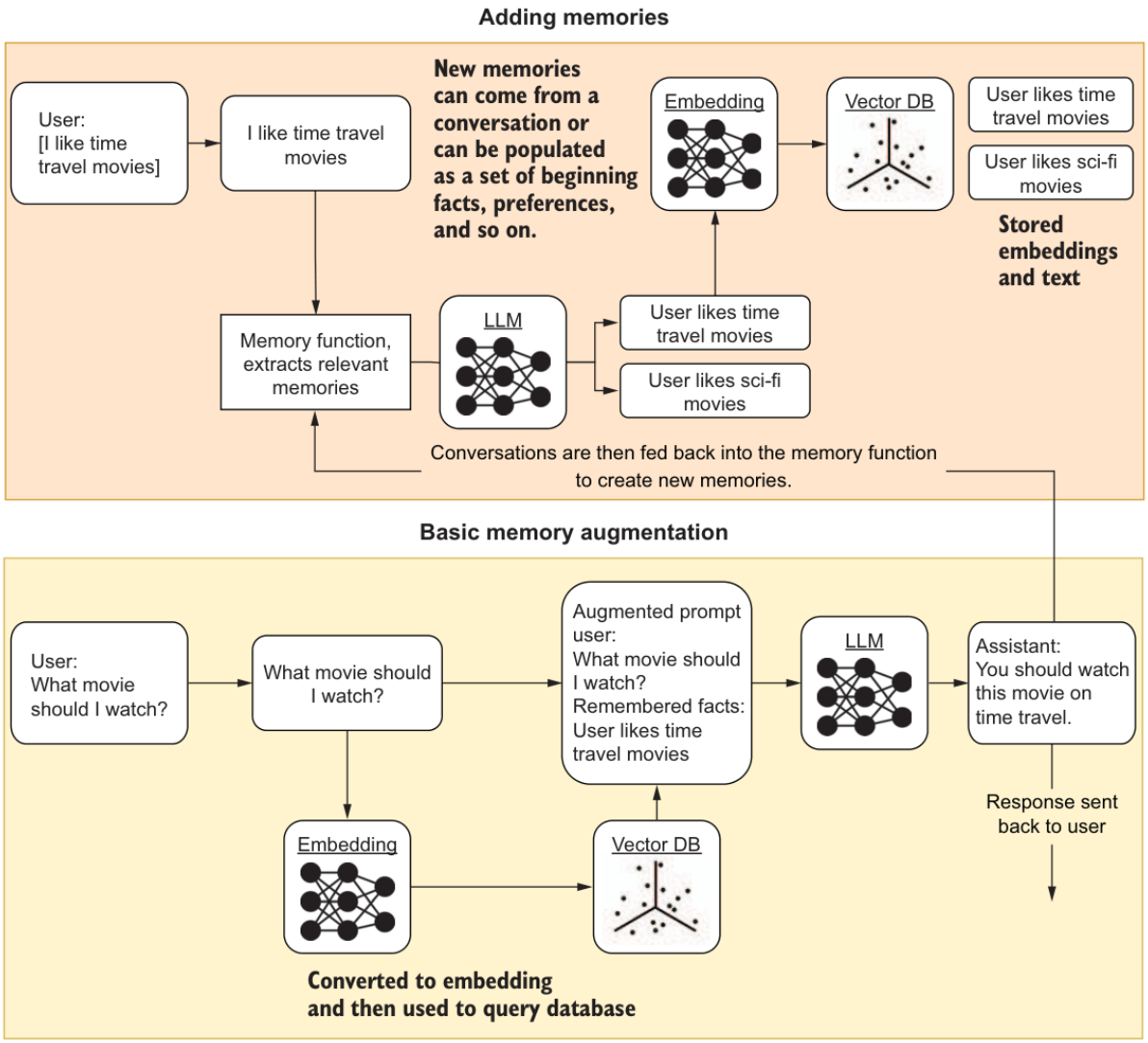


Hình 9.13: Cách bộ nhớ được chia thành các dạng khác nhau

Hình 9.13 còn gắn hai chú thích về mức độ liên quan với agent: bộ nhớ hội thoại (conversational memory) và RAG là khu vực bộ nhớ được agent quan tâm nhất, còn bộ nhớ ngầm định (implicit memory) và các kỹ năng cũng có thể được truyền đạt dưới dạng action và tool cho agent.

Dù bộ nhớ dùng đúng cùng cơ chế truy xuất và tăng cường như tri thức, nó thường khác biệt đáng kể ở cách cập nhật hoặc thêm bộ nhớ mới. Hình 9.14 nhấn mạnh quy trình thu thập và dùng bộ nhớ để tăng cường prompt. Vì bộ nhớ thường có kích thước khác với cả một tài liệu hoàn chỉnh, ta có thể không cần dùng cơ chế tách hay chia chunk nào.

Nexus cung cấp một cơ chế giống knowledge store, cho phép người dùng tạo các memory store có thể cấu hình cho nhiều mục đích và ứng dụng khác nhau. Nó cũng hỗ trợ một số dạng bộ nhớ nâng cao hơn đã nêu trong Hình 9.13. Mục tiếp theo xem xét cách các memory store cơ bản hoạt động trong Nexus.

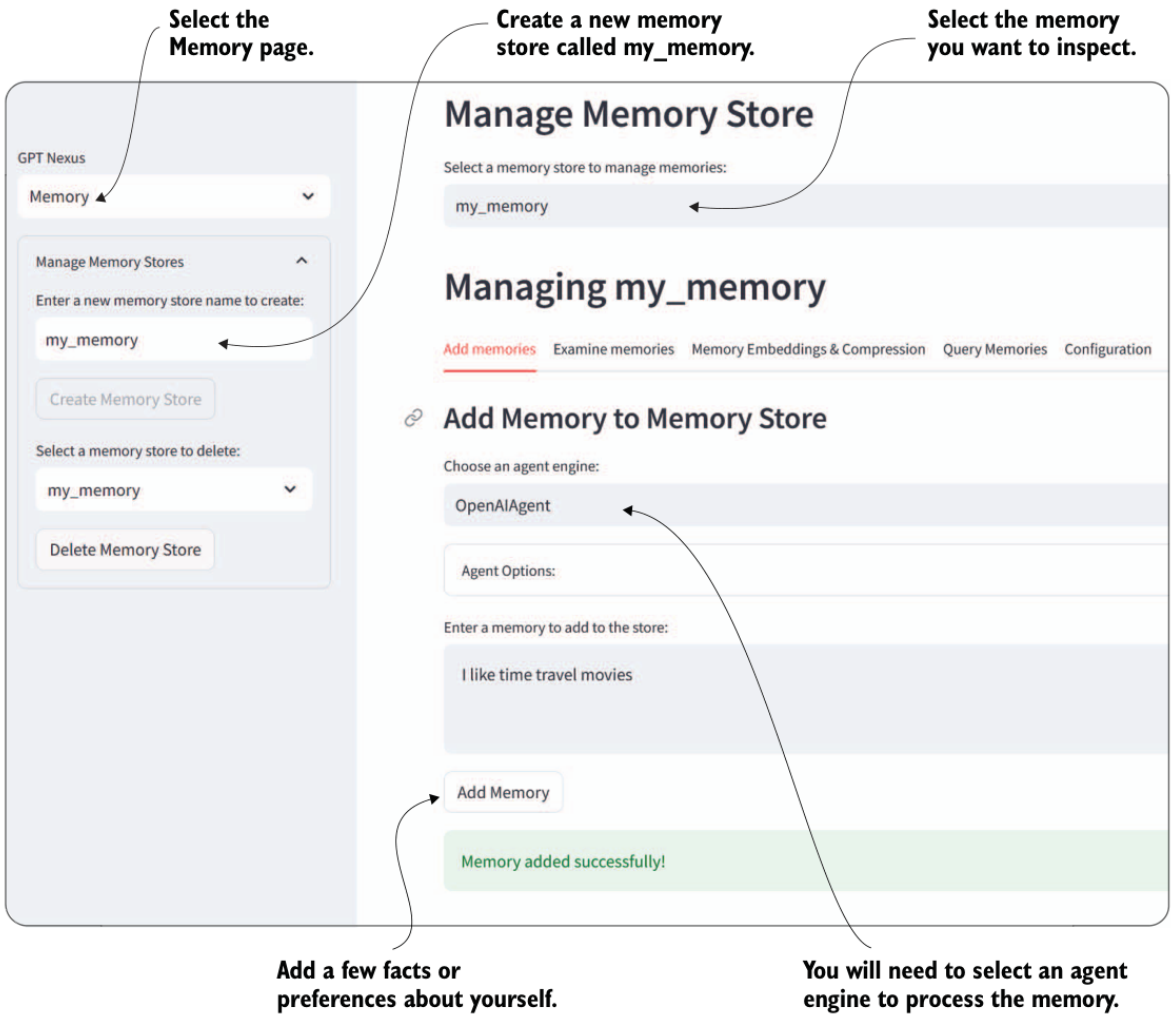


Hình 9.14: Quy trình truy xuất và tăng cường bộ nhớ cơ bản

9.6.1 Sử dụng memory store trong Nexus

Memory store trong Nexus hoạt động và được xây dựng giống knowledge store. Cả hai đều dựa nhiều vào mẫu truy xuất. Điểm khác là các bước bổ sung mà hệ thống bộ nhớ thực hiện để tạo bộ nhớ mới.

Hãy khởi động Nexus, và xem Listing 8.9 nếu cần cài đặt. Sau khi đăng nhập, chọn trang **Memory** và tạo một memory store mới, như trong Hình 9.15. Chọn một agent engine, rồi thêm vài dữ kiện và sở thích cá nhân về bản thân bạn.



Hình 9.15: Thêm bộ nhớ vào một memory store mới tạo

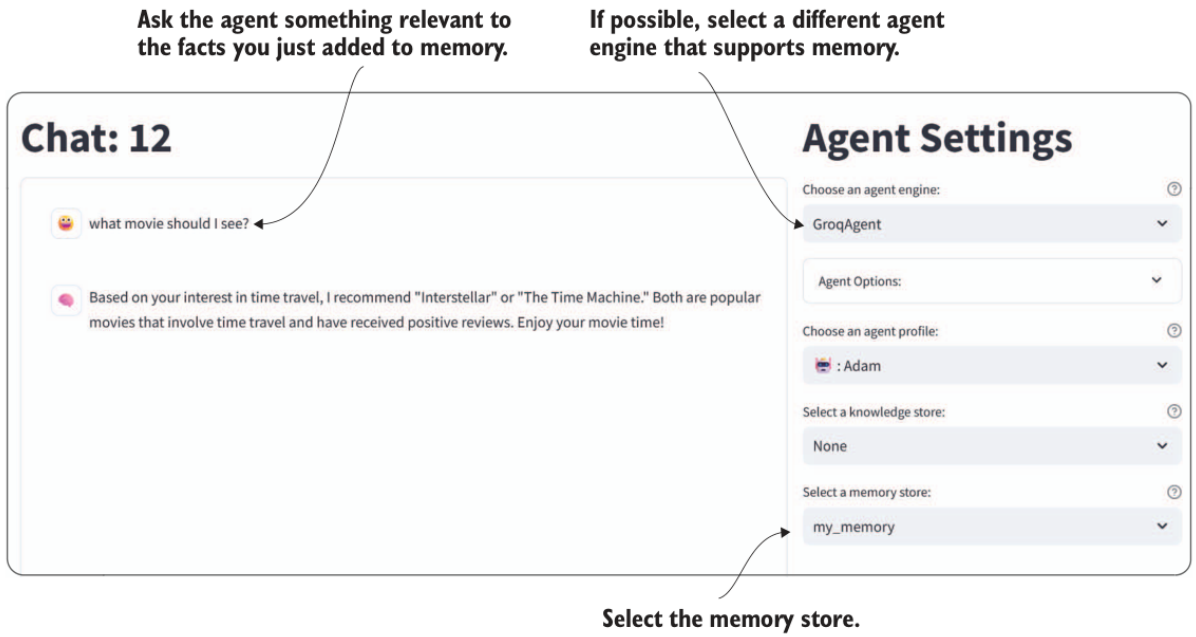
Lý do ta cần một agent (LLM) đã được thể hiện trong Hình 9.14 ở trên. Khi thông tin được đưa vào memory store, nó thường được xử lý qua một LLM bằng một **memory function** (hàm bộ nhớ), có mục đích xử lý các phát biểu/cuộc hội thoại thành thông tin có liên quan về ngữ nghĩa đối với loại bộ nhớ đó.

Listing 8.10 cho thấy hàm bộ nhớ hội thoại (conversational memory function) dùng để trích xuất thông tin từ một cuộc hội thoại thành các bộ nhớ. Đúng vậy, đây chỉ là phần đầu (header) của prompt gửi tới LLM, hướng dẫn nó cách trích xuất thông tin từ một cuộc hội thoại.

Listing 8.10 Hàm bộ nhớ hội thoại

```
Summarize the conversation and create a set of statements that summarize
the conversation. Return a JSON object with the following keys: 'summary'.
Each key should have a list of statements that are relevant to that
category. Return only the JSON object and nothing else.
```

Sau khi tạo một vài bộ nhớ liên quan đến bản thân, hãy quay lại khu vực Chat trong Nexus, bật memory store `my_memory`, và xem agent hiểu bạn đến đâu. Hình 9.16 cho thấy một cuộc hội thoại mẫu dùng một agent engine khác.



Hình 9.16: Trò chuyện với một agent khác trên cùng memory store

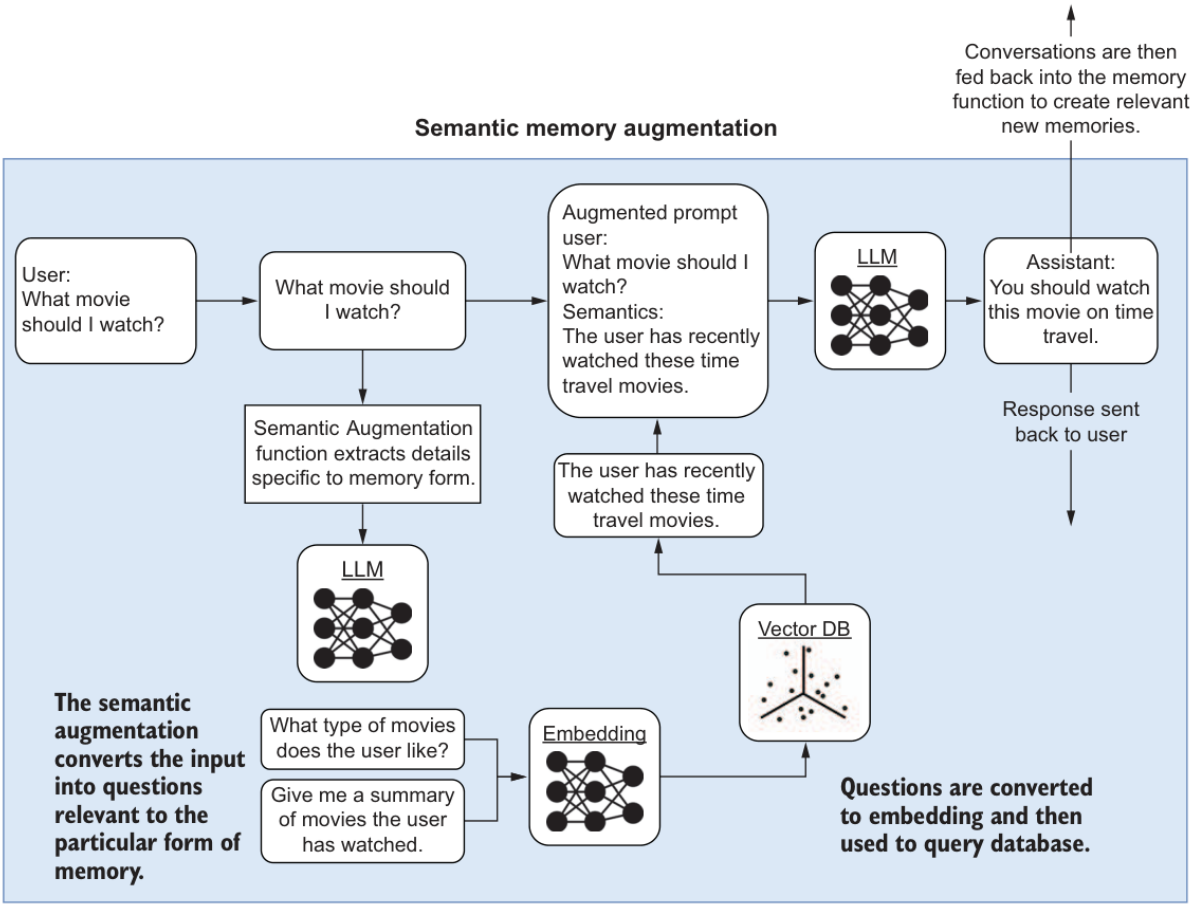
Đây là ví dụ về mẫu bộ nhớ cơ bản: trích xuất dữ kiện/sở thích từ các cuộc hội thoại và lưu chúng vào cơ sở dữ liệu vector dưới dạng bộ nhớ. Còn nhiều hiện thực bộ nhớ khác theo các dạng đã nêu trong Hình 9.13. Ta sẽ hiện thực chúng ở mục tiếp theo.

9.6.2 Semantic memory và ứng dụng cho bộ nhớ ngữ nghĩa, bộ nhớ sự kiện và bộ nhớ thủ tục

Các nhà tâm lý học phân loại bộ nhớ thành nhiều dạng, tuỳ vào loại thông tin được ghi nhớ. Bộ nhớ ngữ nghĩa (semantic), bộ nhớ sự kiện (episodic) và bộ nhớ thủ tục (procedural) đều biểu diễn các loại thông tin khác nhau. Bộ nhớ **episodic** nói về các sự kiện, bộ nhớ **procedural** nói về quy trình hoặc các bước, còn bộ nhớ **semantic** biểu diễn ý nghĩa và có thể bao gồm cảm xúc. Còn có những dạng bộ nhớ khác (ví dụ bộ nhớ địa không gian, geospatial) không được mô tả ở đây nhưng vẫn có thể được bổ sung.

Vì các dạng bộ nhớ này dựa trên một mức phân loại bổ sung, chúng cũng dựa vào một mức phân loại ngữ nghĩa bổ sung. Một số nền tảng, như Semantic Kernel (SK), gọi đây là **semantic memory**. Điều này có thể gây nhầm lẫn vì việc phân loại ngữ nghĩa cũng được dùng để trích xuất bộ nhớ sự kiện và bộ nhớ thủ tục.

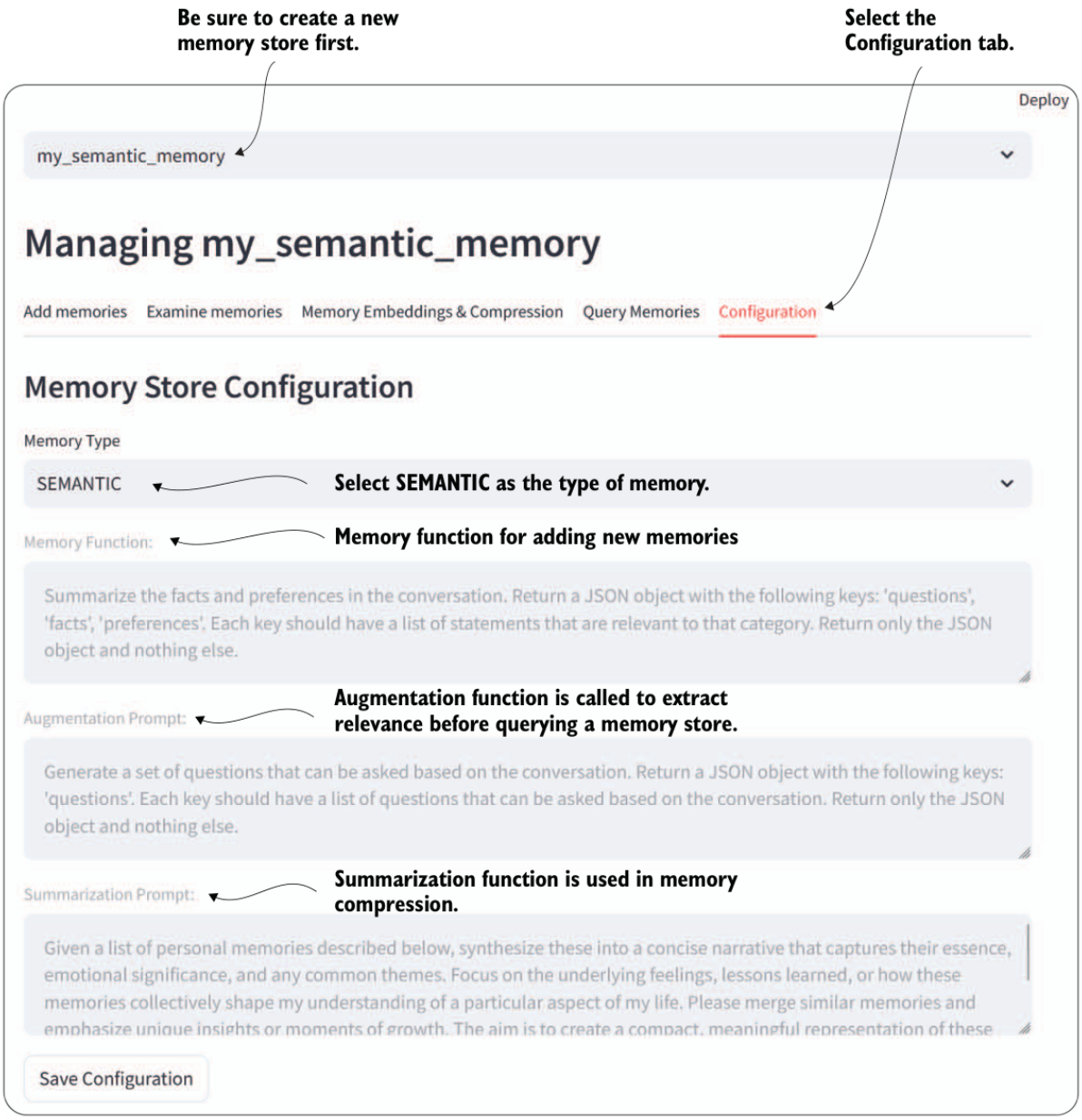
Hình 9.17 cho thấy quy trình phân loại bộ nhớ ngữ nghĩa, đôi khi cũng gọi là semantic memory. Điểm khác giữa semantic memory và bộ nhớ thông thường là bước bổ sung: xử lý đầu vào theo ngữ nghĩa và trích xuất các câu hỏi liên quan, dùng để truy vấn cơ sở dữ liệu vector chứa bộ nhớ liên quan.



Hình 9.17: Cách tăng cường bằng semantic memory hoạt động

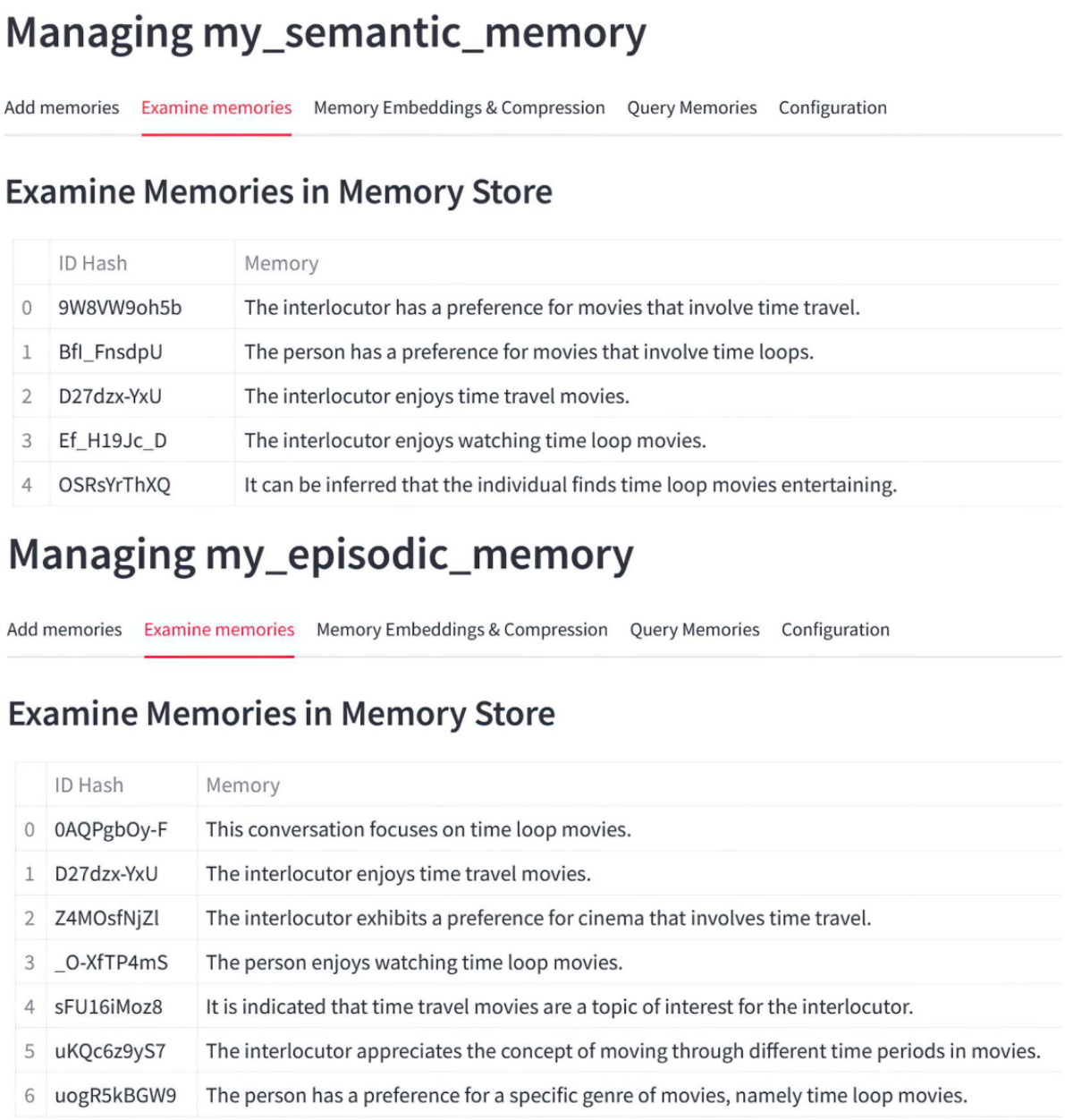
Lợi ích của tăng cường ngữ nghĩa (semantic augmentation) là khả năng trích xuất bộ nhớ liên quan hơn. Ta có thể thấy điều này khi quay lại Nexus và tạo một semantic memory store mới.

Hình 9.18 cho thấy cách cấu hình một memory store mới dùng semantic memory. Hiện chưa thể cấu hình các prompt cụ thể của hàm cho bộ nhớ, tăng cường (augmentation) và tóm tắt (summarization). Tuy vậy, việc đọc qua prompt của từng hàm vẫn hữu ích để hiểu chúng hoạt động thế nào.



Hình 9.18: Cấu hình để đổi loại memory store sang semantic

Giờ nếu quay lại và thêm các dữ kiện và sở thích, chúng sẽ được chuyển thành ngữ nghĩa của dạng bộ nhớ tương ứng. Hình 9.19 cho thấy ví dụ bộ nhớ được điền từ cùng một tập phát biểu vào hai dạng bộ nhớ khác nhau. Thông thường, các phát biểu đưa vào bộ nhớ sẽ cụ thể hơn cho dạng bộ nhớ đó.



Hình 9.19: So sánh bộ nhớ của cùng một thông tin ở hai loại bộ nhớ khác nhau

Bộ nhớ và tri thức có thể hỗ trợ đáng kể cho agent trong nhiều loại ứng dụng. Thật vậy, một memory/knowledge store duy nhất có thể phục vụ một hoặc nhiều agent, cho phép các cách diễn giải chuyên biệt hơn đối với cả hai loại kho. Ta kết thúc chương bằng việc bàn về nén bộ nhớ/tri thức ở mục tiếp theo.

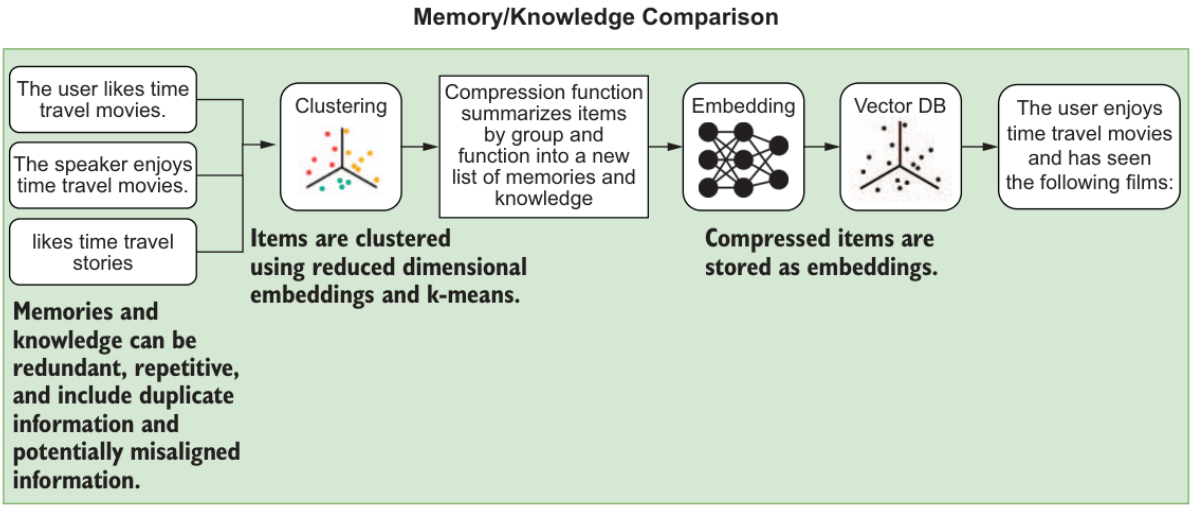
9.7 Tìm hiểu nén bộ nhớ và tri thức

Giống trí nhớ của chính ta, các memory store có thể trở nên lộn xộn theo thời gian với thông tin dư thừa và nhiều chi tiết không liên quan. Bên trong, bộ não xử lý sự lộn xộn của trí nhớ bằng cách nén hoặc tóm tắt ký ức. Bộ não nhớ những chi tiết quan trọng hơn những chi tiết ít quan trọng, và nhớ rõ hơn những ký ức được truy cập thường xuyên.

Ta có thể áp dụng nguyên tắc tương tự, **nén bộ nhớ** (memory compression), cho bộ nhớ của agent và các hệ thống truy xuất khác để trích xuất các chi tiết quan trọng. Nguyên tắc nén giống với tăng cường ngữ nghĩa nhưng thêm một lớp nữa: gom trước các nhóm bộ nhớ liên quan để có thể tóm tắt chung từng

nhóm.

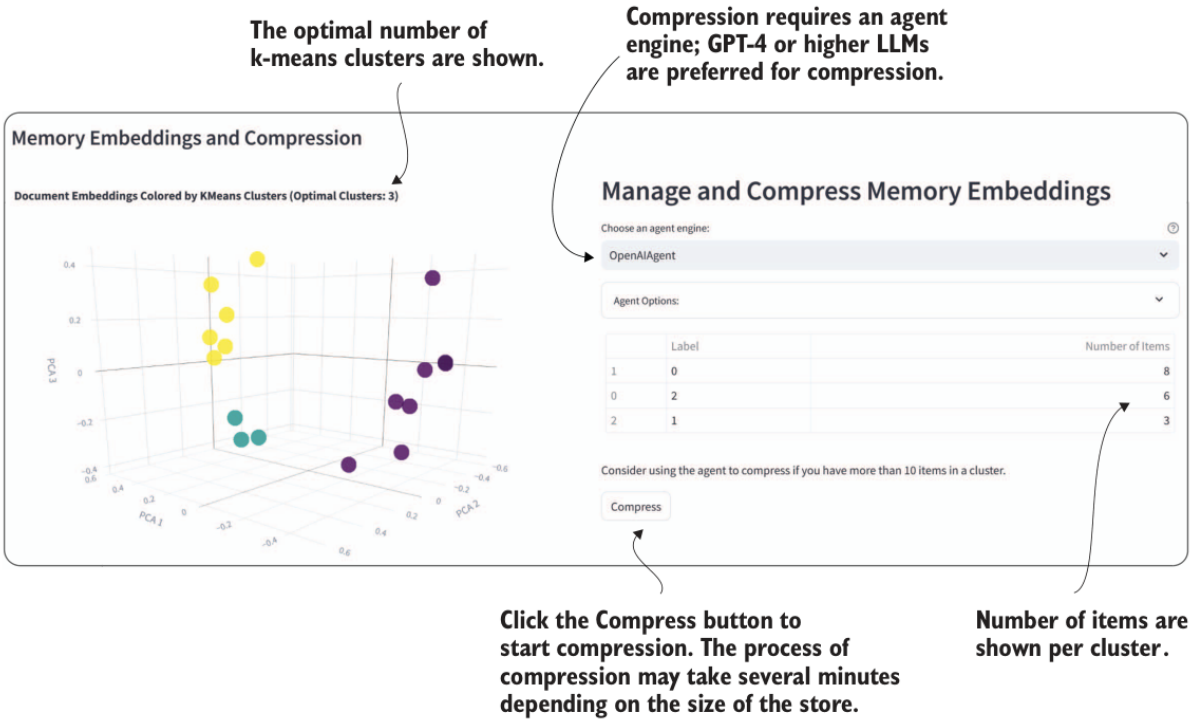
Hình 9.20 cho thấy quy trình nén bộ nhớ/tri thức. Các bộ nhớ hoặc tri thức trước hết được gom cụm (cluster) bằng một thuật toán như k-means. Sau đó, các nhóm bộ nhớ được đưa qua một hàm nén (compression function), hàm này tóm tắt và gom các mục thành những biểu diễn ngắn gọn hơn.



Hình 9.20: Quy trình nén bộ nhớ và tri thức

Nexus hỗ trợ nén cho cả knowledge store và memory store bằng gom cụm k-means tối ưu (optimal clustering). Hình 9.21 cho thấy giao diện nén cho bộ nhớ. Trong giao diện nén, bạn sẽ thấy các mục được hiển thị dạng 3D và đã gom cụm. Kích thước (số mục) của từng cụm được hiển thị ở bảng bên trái.

Nhìn chung, nên nén bộ nhớ và cả tri thức nếu số mục trong một cụm lớn hoặc không cân bằng. Mỗi trường hợp dùng nén có thể khác nhau tuỳ cách sử dụng và ứng dụng của bộ nhớ. Tuy nhiên, nhìn chung nếu khi kiểm tra các mục trong một kho thấy có thông tin lặp lại hoặc trùng lặp thì đó là lúc nên nén. Sau đây là tóm tắt các trường hợp ứng dụng được lợi từ nén.



Hình 9.21: Giao diện nén bộ nhớ

Lý do nén tri thức. Truy xuất và tăng cường tri thức cũng được chứng minh là hưởng lợi đáng kể từ việc nén. Kết quả sẽ khác nhau tuỳ trường hợp, nhưng nhìn chung nguồn tri thức càng dài dòng thì càng được lợi từ nén. Các tài liệu có văn xuôi văn chương, như truyện và tiểu thuyết, sẽ được lợi hơn một cơ sở code chẳng hạn. Tuy nhiên, nếu code cũng rất lặp lại thì nén cũng có thể có lợi.

Lý do về tần suất áp dụng nén. Bộ nhớ thường hưởng lợi từ việc nén định kỳ, trong khi knowledge store thường chỉ hưởng lợi ở lần nạp đầu tiên. Tần suất nén phụ thuộc nhiều vào cách dùng bộ nhớ, tần suất dùng và số lượng bộ nhớ.

Lý do nén nhiều hơn một lần. Nén nhiều lượt cùng lúc được chứng minh là cải thiện hiệu năng truy xuất. Các mẫu khác cũng gợi ý dùng bộ nhớ hoặc tri thức ở nhiều mức nén khác nhau. Ví dụ, một knowledge store được nén hai lần sẽ cho ba mức tri thức khác nhau.

Lý do kết hợp nén tri thức và bộ nhớ. Nếu một hệ thống chuyên biệt cho một nguồn tri thức cụ thể và hệ thống đó cũng dùng bộ nhớ, có thể còn có thêm cách tối ưu bằng việc hợp nhất các kho. Một cách tiếp cận khác là điền trực tiếp tri thức ban đầu của một tài liệu vào bộ nhớ.

Lý do dùng nhiều memory hoặc knowledge store. Trong các hệ thống nâng cao hơn, ta sẽ xem xét các agent dùng nhiều memory và knowledge store liên quan đến workflow của chúng. Ví dụ, một agent có thể dùng các memory store riêng cho từng người dùng trong cuộc hội thoại, có thể gồm cả khả năng chia sẻ các nhóm bộ nhớ khác nhau với các nhóm người khác nhau. Truy xuất bộ nhớ và tri thức là nền tảng của các hệ thống agentic, và giờ ta có thể tóm tắt những gì đã học và điểm qua một số bài tập ở mục tiếp theo.

9.8 Bài tập

Hãy dùng các bài tập sau để củng cố kiến thức về nội dung đã học:

Bài tập 1: Nạp và tách một tài liệu khác (Trung cấp)

Mục tiêu. Hiểu ảnh hưởng của việc tách tài liệu lên hiệu quả truy xuất bằng LangChain.

Nhiệm vụ.

- Chọn một tài liệu khác (ví dụ: một bài báo, một bài báo khoa học hoặc một truyện ngắn).
- Dùng LangChain để nạp và tách tài liệu thành các chunk.
- Phân tích cách tài liệu được tách thành chunk và ảnh hưởng của nó đến quá trình truy xuất.

Bài tập 2: Thử nghiệm tìm kiếm ngữ nghĩa (Trung cấp)

Mục tiêu. So sánh hiệu quả của các kỹ thuật vector hoá khác nhau bằng cách thực hiện tìm kiếm ngữ nghĩa.

Nhiệm vụ.

- Chọn một tập tài liệu để tìm kiếm ngữ nghĩa.
- Dùng một phương pháp vector hoá như embedding Word2Vec hoặc BERT thay cho TF–IDF.
- Thực hiện tìm kiếm ngữ nghĩa và so sánh kết quả với kết quả khi dùng TF–IDF để hiểu sự khác biệt và hiệu quả.

Bài tập 3: Hiện thực một workflow RAG tuỳ chỉnh (Nâng cao)

Mục tiêu. Áp dụng kiến thức lý thuyết về RAG vào bối cảnh thực tế bằng LangChain.

Nhiệm vụ.

- Chọn một ứng dụng cụ thể (ví dụ: yêu cầu của khách hàng trong dịch vụ khách hàng, hoặc truy vấn nghiên cứu học thuật).
- Thiết kế và hiện thực một workflow RAG tuỳ chỉnh bằng LangChain.
- Điều chỉnh workflow cho phù hợp với ứng dụng đã chọn, rồi kiểm thử hiệu quả của nó.

Bài tập 4: Xây dựng knowledge store và thử nghiệm các mẫu tách (Trung cấp)

Mục tiêu. Hiểu cách các mẫu tách khác nhau và việc nén ảnh hưởng đến truy xuất tri thức.

Nhiệm vụ.

- Xây dựng một knowledge store và nạp vào đó một vài tài liệu.
- Thử nghiệm các dạng mẫu tách/chia chunk khác nhau và phân tích ảnh hưởng của chúng lên truy xuất.
- Nén knowledge store và quan sát ảnh hưởng lên hiệu năng truy vấn.

Bài tập 5: Xây dựng và kiểm thử nhiều loại memory store (Nâng cao)

Mục tiêu. Hiểu đặc điểm riêng và các trường hợp sử dụng của từng loại memory store.

Nhiệm vụ.

- Xây dựng nhiều dạng memory store (hội thoại, ngữ nghĩa, sự kiện và thủ tục).
- Tương tác với agent bằng từng loại memory store và quan sát sự khác biệt.
- Nén memory store và phân tích ảnh hưởng lên truy xuất bộ nhớ.

Tóm tắt

- Bộ nhớ trong ứng dụng AI phân biệt giữa bộ nhớ phi cấu trúc và có cấu trúc, nhấn mạnh việc dùng chúng để đặt prompt vào đúng ngữ cảnh, giúp tương tác phù hợp hơn.
- Retrieval augmented generation (RAG) là cơ chế tăng cường prompt bằng ngữ cảnh từ các tài liệu bên ngoài, dùng vector embedding và tìm kiếm tương đồng để truy xuất nội dung liên quan.
- Semantic search với lập chỉ mục tài liệu chuyển tài liệu thành vector ngữ nghĩa bằng TF–IDF và cosine similarity, nâng cao khả năng tìm kiếm ngữ nghĩa trên các tài liệu đã lập chỉ mục.
- Cơ sở dữ liệu vector và tìm kiếm tương đồng lưu các vector tài liệu trong cơ sở dữ liệu vector, giúp tìm kiếm tương đồng hiệu quả và tăng độ chính xác truy xuất.
- Document embedding nắm bắt ý nghĩa ngữ nghĩa, dùng các mô hình như mô hình của OpenAI để sinh embedding giữ được ngữ cảnh của tài liệu và hỗ trợ tìm kiếm tương đồng ngữ nghĩa.
- LangChain cung cấp nhiều công cụ để thực hiện RAG, và trừu tượng hoá quá trình truy xuất, cho phép hiện thực RAG và hệ thống bộ nhớ dễ dàng trên nhiều nguồn dữ liệu và kho vector.
- Bộ nhớ ngắn hạn và dài hạn trong LangChain hiện thực bộ nhớ hội thoại trong LangChain, phân biệt giữa các mẫu đệm (buffer) ngắn hạn và các giải pháp lưu trữ dài hạn.
- Lưu vector tài liệu trong cơ sở dữ liệu để tìm kiếm tương đồng hiệu quả là điều cốt yếu để hiện thực các hệ thống truy xuất có khả năng mở rộng trong ứng dụng AI.
- Tri thức của agent liên quan trực tiếp đến mẫu RAG tổng quát: thực hiện hỏi đáp trên tài liệu hoặc thông tin văn bản khác.
- Bộ nhớ của agent là một mẫu liên quan đến RAG, ghi lại các tương tác agentic với người dùng, với chính nó và với các hệ thống khác.
- Nexus là nền tảng hiện thực các hệ thống tri thức và bộ nhớ cho agent, gồm việc thiết lập knowledge store cho truy xuất tài liệu và memory store cho nhiều dạng bộ nhớ.
- Tăng cường bằng semantic memory (semantic memory) phân biệt các loại bộ nhớ (ngữ nghĩa, sự kiện, thủ tục) và hiện thực chúng qua tăng cường ngữ nghĩa, nâng cao khả năng agent nhớ lại và dùng thông tin phù hợp với bản chất của từng bộ nhớ.
- Nén bộ nhớ và tri thức là các kỹ thuật cô đọng thông tin lưu trong hệ thống bộ nhớ và tri thức, cải thiện hiệu quả và độ liên quan của truy xuất thông qua gom cụm và tóm tắt.

Chương 10

Làm chủ prompt cho agent với prompt flow

Chương này bao quát các nội dung:

- Hiểu về prompt engineering có hệ thống (systematic prompt engineering) và thiết lập prompt flow đầu tiên.
- Xây dựng một prompt profile/persona hiệu quả.
- Đánh giá profile: rubric (bảng tiêu chí chấm điểm) và grounding.
- Đánh giá grounding cho một profile mô hình ngôn ngữ lớn (LLM).
- So sánh các prompt: tìm ra profile hoàn hảo.

Trong chương này, ta đi sâu vào chiến lược prompt engineering **Test Changes Systematically** (kiểm thử thay đổi một cách có hệ thống). Nhắc lại, ở Chương 3 ta đã điểm qua các chiến lược lớn trong framework prompt engineering của OpenAI. Các chiến lược này giúp ta xây dựng prompt tốt hơn, và nhờ đó có agent profile (hồ sơ agent) và persona (nhân cách) tốt hơn. Hiểu vai trò này là chìa khoá trên hành trình prompt engineering của chúng ta.

Test Changes Systematically là khía cạnh cốt lõi của prompt engineering đến mức Microsoft đã phát triển cả một công cụ xoay quanh chiến lược này, gọi là **prompt flow**, được mô tả ở phần sau của chương. Trước khi đến với prompt flow, ta cần hiểu vì sao cần prompt engineering có hệ thống.

10.1 Vì sao cần prompt engineering có hệ thống

Về bản chất, prompt engineering là một quá trình lặp. Khi xây dựng một prompt, bạn thường xuyên lặp lại việc chỉnh sửa và đánh giá. Để thấy khái niệm này trong thực tế, hãy xét một ứng dụng đơn giản của prompt engineering cho một câu hỏi gửi ChatGPT.

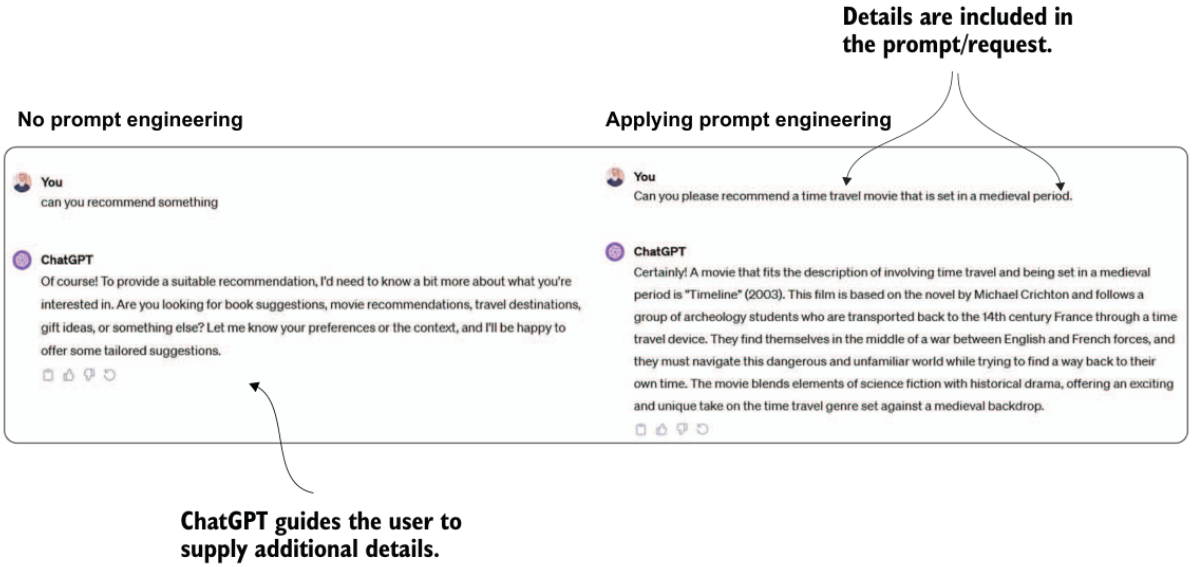
Bạn có thể làm theo bằng cách mở trình duyệt tới ChatGPT (`https://chat.openai.com/`), nhập prompt (văn bản) sau vào ChatGPT rồi bấm nút Send Message (ví dụ về cuộc hội thoại này nằm ở bên trái Hình 10.1):

```
can you recommend something
```

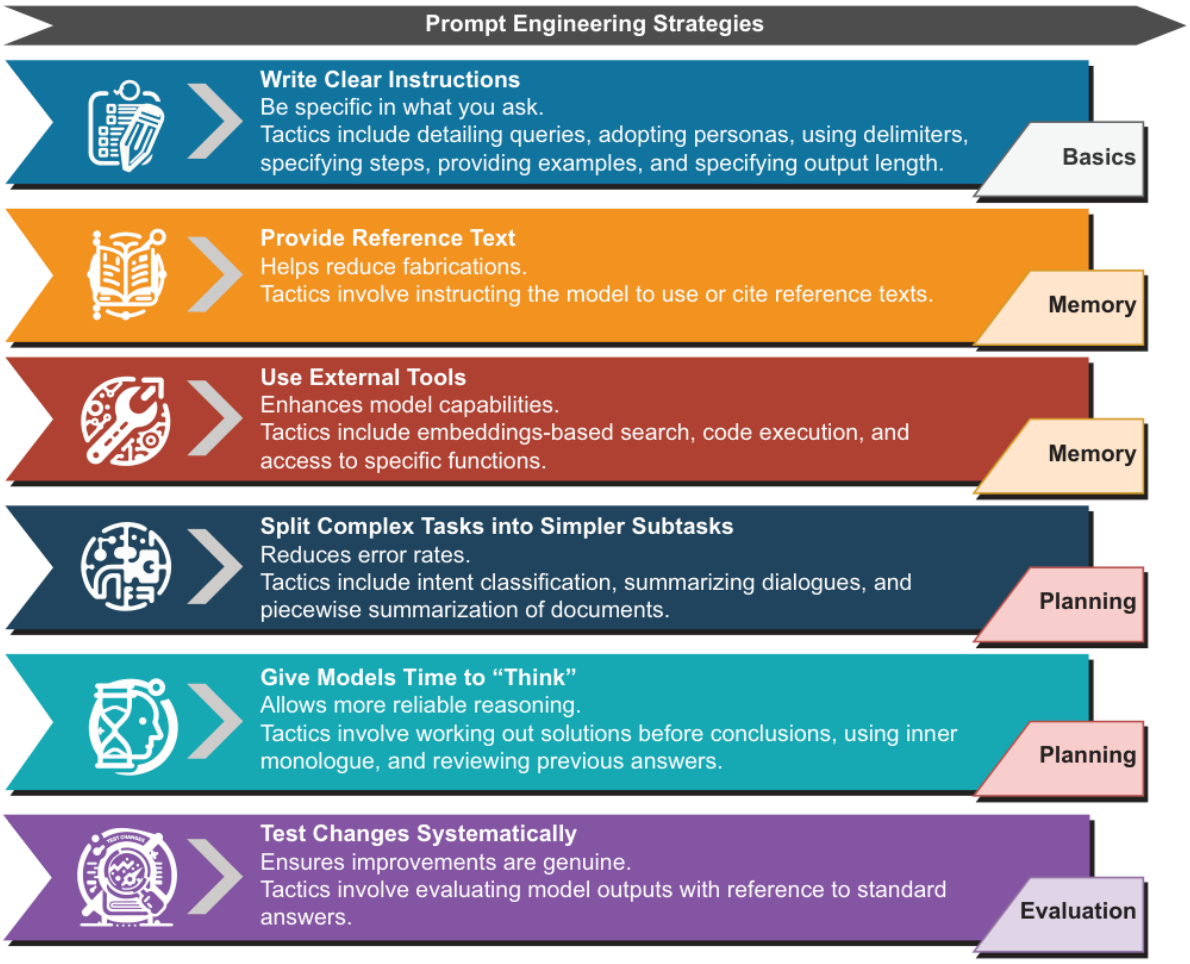
Ta thấy phản hồi của ChatGPT là hỏi thêm thông tin. Hãy mở một cuộc hội thoại mới với ChatGPT và nhập prompt sau, như ở bên phải Hình 10.1:

```
Can you please recommend a time travel movie set in the medieval period.
```

Kết quả trong Hình 10.1 cho thấy sự khác biệt rõ rệt giữa việc bỏ qua chi tiết và việc nêu yêu cầu cụ thể hơn. Ta vừa áp dụng chiến thuật Write Clear Instructions (viết chỉ dẫn rõ ràng) một cách lịch sự, và ChatGPT đã đưa ra một gợi ý tốt. Nhưng cũng để ý rằng chính ChatGPT hướng dẫn người dùng viết prompt tốt hơn. Màn hình đã làm mới trong Hình 10.2 trình bày các chiến lược prompt engineering của OpenAI.

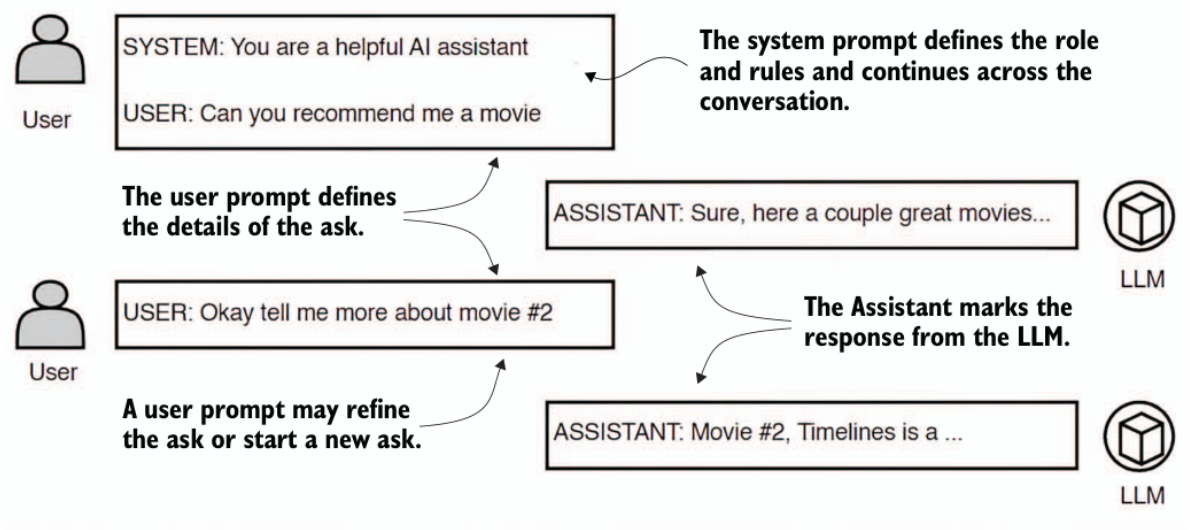


Hình 10.1: Sự khác biệt khi áp dụng prompt engineering và lặp lại



Hình 10.2: Các chiến lược prompt engineering của OpenAI, phân theo thành phần của agent

Ta vừa áp dụng phép lặp đơn giản để cải thiện prompt. Có thể mở rộng ví dụ này bằng cách dùng system prompt/message (prompt hệ thống). Hình 10.3 minh hoạ cách dùng và vai trò của system prompt trong giao tiếp lặp. Ở Chương 3, ta đã dùng system message/prompt trong nhiều ví dụ.



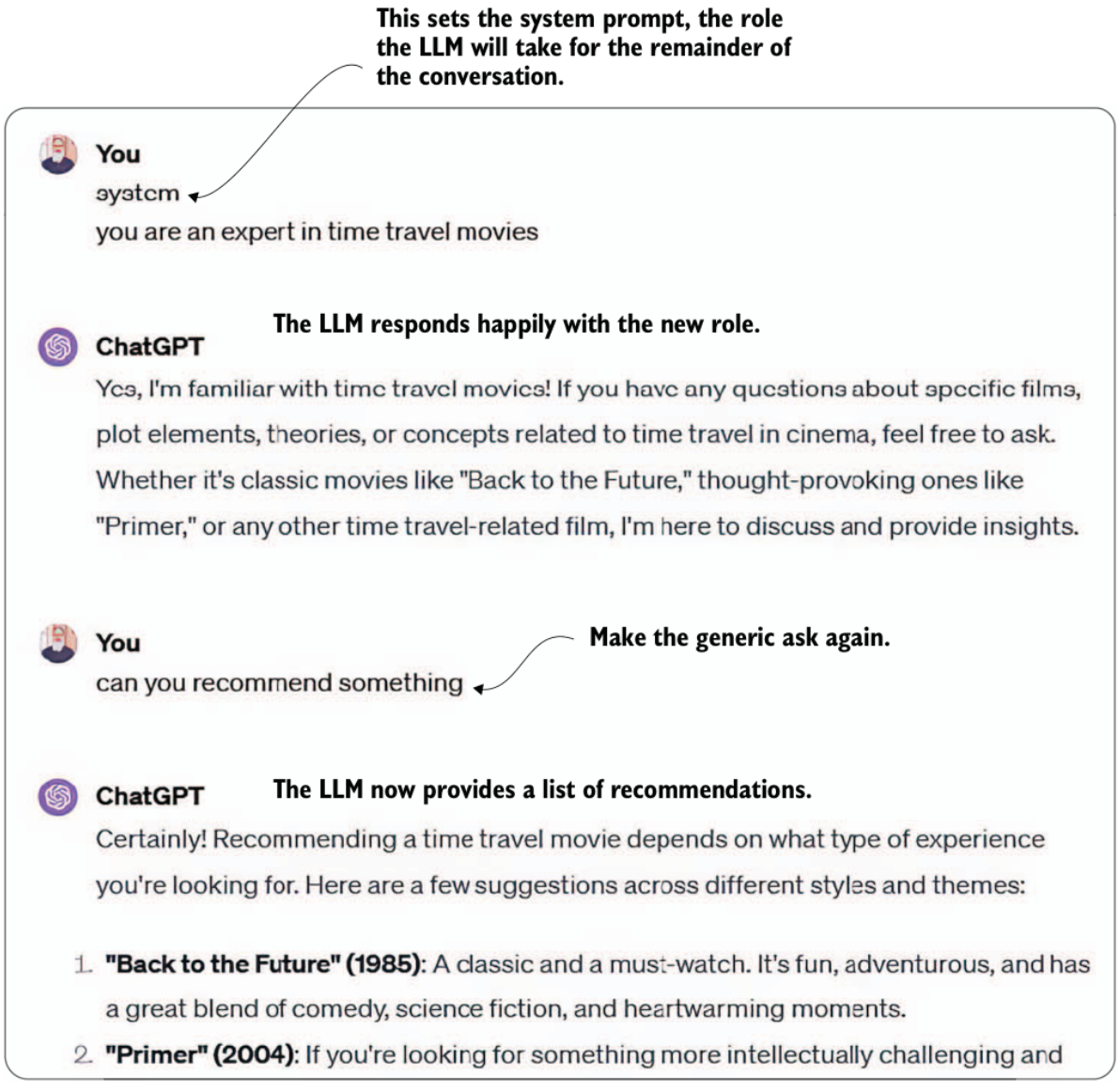
Hình 10.3: Các message gửi tới và nhận từ LLM trong một cuộc hội thoại, và sự lặp lại của các message

Bạn cũng có thể thử điều này trong ChatGPT. Lần này, nhập prompt sau, bắt đầu bằng từ *system* viết thường, tiếp theo là một dòng mới (để xuống dòng trong ô nhập mà không gửi message, nhấn Shift-Enter):

```
system
You are an expert on time travel movies.
```

ChatGPT sẽ đáp lại bằng vài lời dễ chịu, như trong Hình 10.4. Qua đó, nó vui vẻ nhận vai trò mới và hỏi bạn có câu hỏi tiếp theo nào không. Giờ hãy nhập prompt chung chung như ta đã làm trước đó:

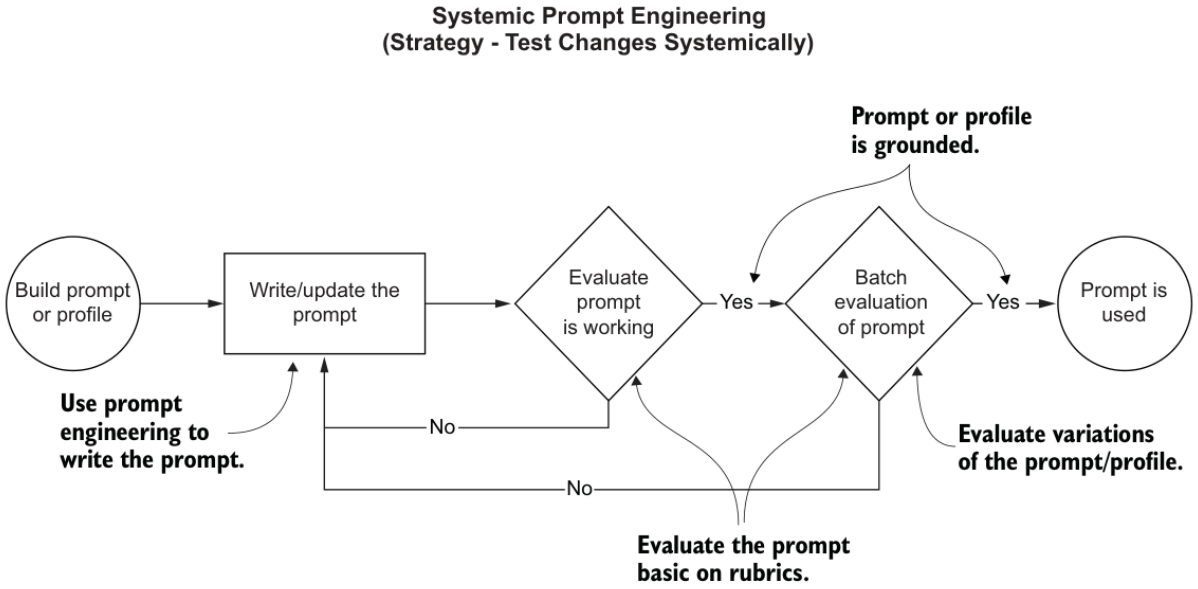
```
can you recommend something
```



Hình 10.4: Tác dụng của việc thêm system prompt vào cuộc hội thoại trước đó

Ta vừa thấy quá trình lặp để tinh chỉnh một prompt, tức prompt engineering, nhằm thu được phản hồi tốt hơn. Việc này được thực hiện qua ba cuộc hội thoại khác nhau trên giao diện ChatGPT. Đây không phải cách hiệu quả nhất, nhưng nó vẫn dùng được.

Tuy nhiên, ta chưa định nghĩa luồng lặp để đánh giá prompt và xác định khi nào một prompt là hiệu quả. Hình 10.5 trình bày một phương pháp prompt engineering có hệ thống, dùng một hệ thống lặp và đánh giá.



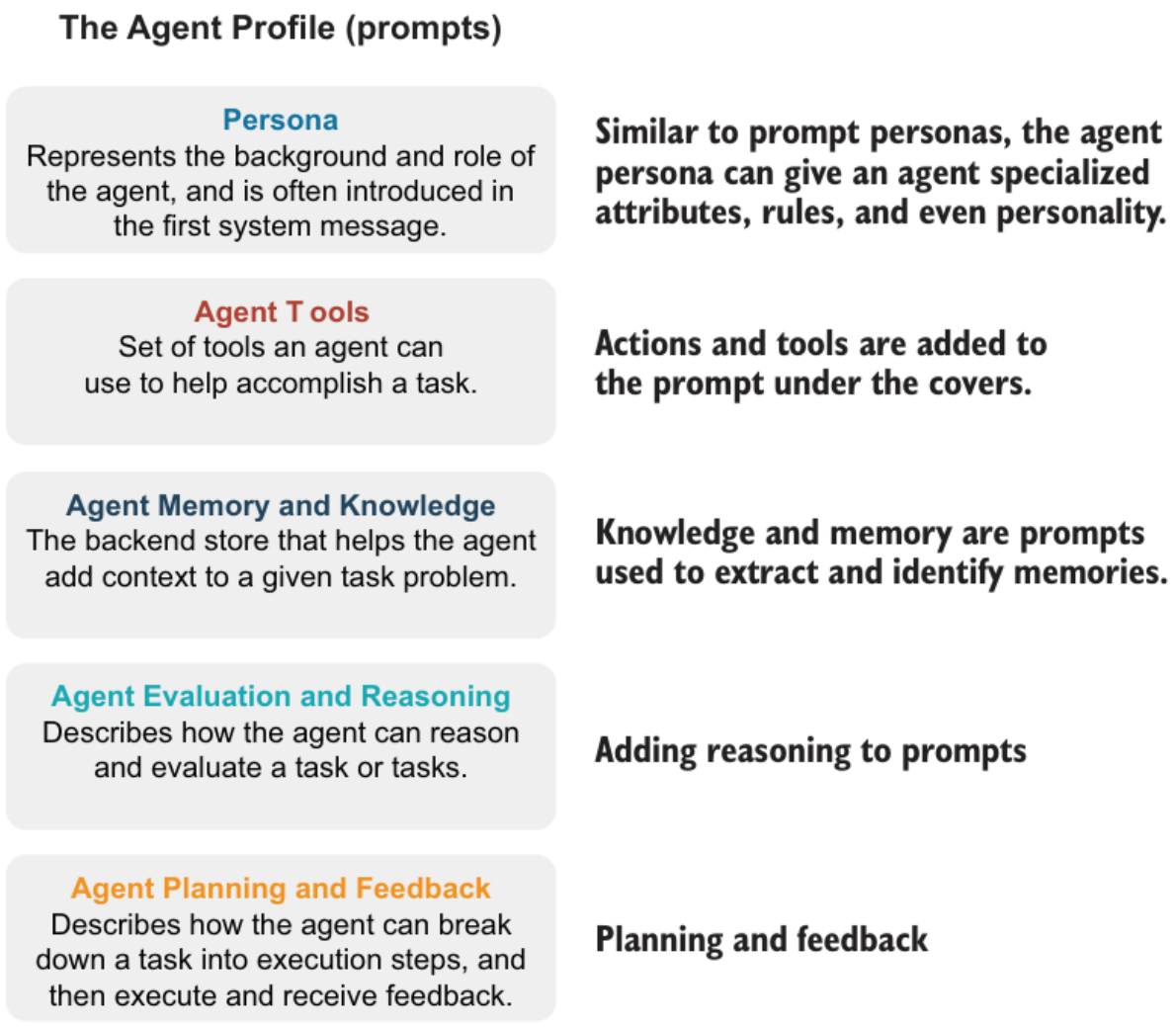
Hình 10.5: Phương pháp prompt engineering có hệ thống

Hệ thống lặp và đánh giá prompt bao trùm chiến lược lớn Test Changes Systematically. Đánh giá hiệu năng và hiệu quả của prompt vẫn còn là lĩnh vực mới, nhưng ta sẽ dùng các kỹ thuật từ giáo dục như **rubric** và **grounding**, sẽ được tìm hiểu ở một mục sau trong chương. Tuy vậy, như trình bày ở mục tiếp theo, trước đó ta cần hiểu sự khác nhau giữa persona và agent profile.

10.2 Hiểu về agent profile và persona

Agent profile là sự đóng gói các prompt hoặc message thành phần mô tả một agent. Nó bao gồm persona của agent, các chỉ dẫn đặc biệt và những chiến lược khác có thể hướng dẫn người dùng hoặc các bên tiêu thụ agent khác.

Hình 10.6 trình bày các thành phần chính của một agent profile. Các thành phần này tương ứng với những chiến lược prompt engineering được mô tả trong sách. Không phải agent nào cũng dùng hết mọi thành phần của một agent profile đầy đủ.



Hình 10.6: Các thành phần của một agent profile

Ở mức cơ bản, *agent profile* là một tập prompt mô tả agent. Nó có thể bao gồm các thành phần bên ngoài khác liên quan tới action/tool, knowledge (tri thức), memory (bộ nhớ), reasoning (suy luận), evaluation (đánh giá), planning (lập kế hoạch) và feedback (phản hồi). Tổ hợp các thành phần này tạo nên một agent prompt profile hoàn chỉnh.

Prompt là trái tim trong hoạt động của agent. Một prompt hoặc một tập prompt điều khiển từng thành phần của agent trong profile. Với action/tool, các prompt này được định nghĩa rõ ràng; nhưng như ta đã thấy, prompt cho memory và knowledge có thể khác nhau rất nhiều tuỳ trường hợp sử dụng.

Định nghĩa một AI agent profile không chỉ là một system prompt. Prompt flow cho phép ta xây dựng các prompt và code tạo nên agent profile, đồng thời có khả năng đánh giá hiệu quả của nó. Ở mục tiếp theo, ta sẽ mở prompt flow và bắt đầu dùng nó.

10.3 Thiết lập prompt flow đầu tiên

Prompt flow là công cụ do Microsoft phát triển bên trong nền tảng Azure Machine Learning Studio. Sau đó công cụ được phát hành dưới dạng dự án mã nguồn mở trên GitHub, nơi nó thu hút nhiều sự chú ý và người dùng hơn. Dù ban đầu được thiết kế như một nền tảng ứng dụng, về sau nó cho thấy thế mạnh trong việc phát triển và đánh giá prompt/profile.

Vì prompt flow ban đầu được phát triển để chạy trên Azure dưới dạng dịch vụ, nó có kiến trúc lõi vững chắc. Công cụ hỗ trợ xử lý theo lô (batch processing) đa luồng, nên rất phù hợp để đánh giá prompt

ở quy mô lớn. Mục sau sẽ xem xét những bước cơ bản để bắt đầu với prompt flow.

10.3.1 Bắt đầu

Có một vài điều kiện tiên quyết cần chuẩn bị trước khi làm các bài thực hành trong sách. Các điều kiện liên quan tới mục và chương này được liệt kê dưới đây; hãy hoàn thành chúng trước khi thử các bài thực hành:

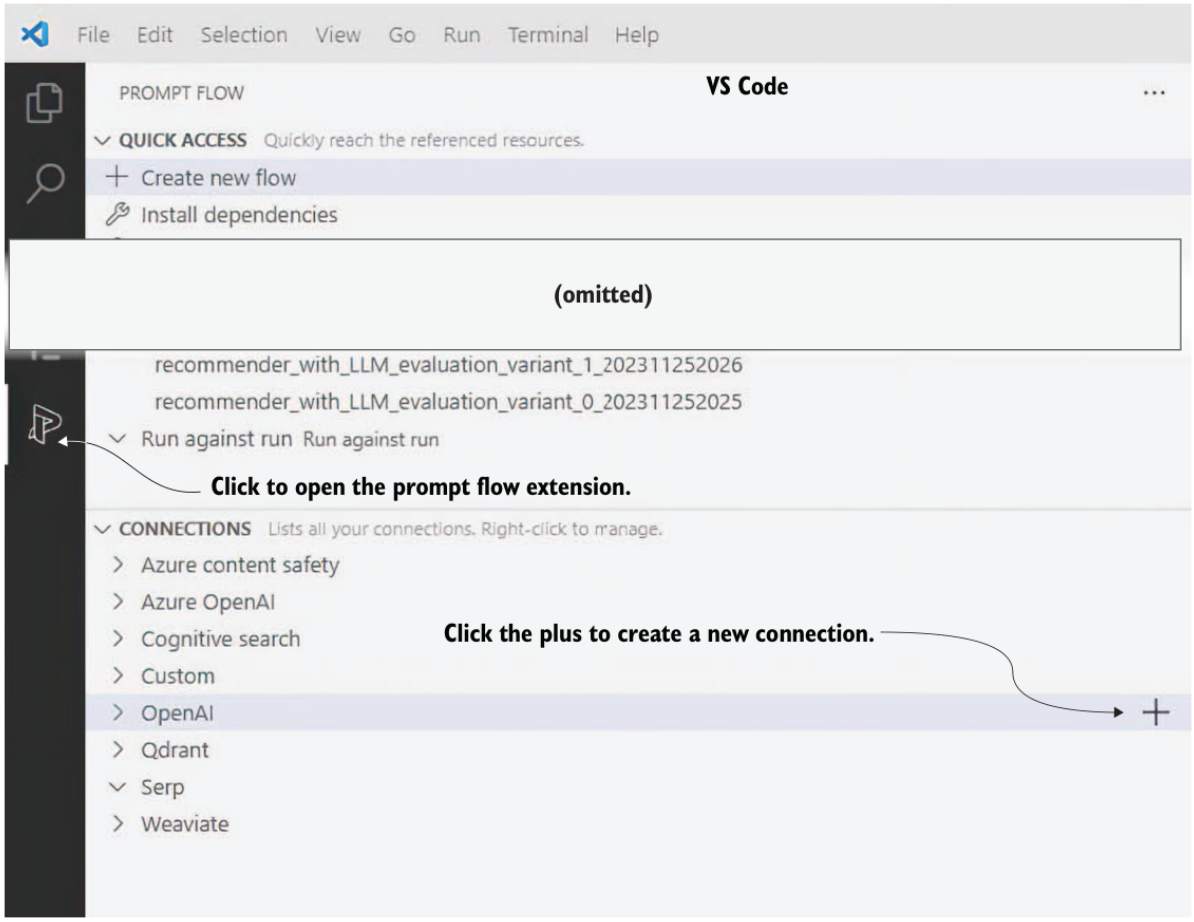
- *Visual Studio Code (VS Code)*: xem phụ lục A để biết hướng dẫn cài đặt, kể cả các extension bổ sung.
- *Prompt flow, extension cho VS Code*: xem phụ lục A để biết chi tiết cách cài extension.
- *Môi trường ảo Python*: xem phụ lục A để biết chi tiết cách thiết lập môi trường ảo (virtual environment).
- *Cài các gói prompt flow*: trong môi trường ảo, chạy nhanh lệnh `pip install` như sau:

```
pip install promptflow promptflow-tools
```

- *LLM (GPT-4 trở lên)*: bạn cần quyền truy cập GPT-4 trở lên qua OpenAI hoặc Azure OpenAI Studio. Xem phụ lục B nếu cần trợ giúp truy cập các tài nguyên này.
- *Mã nguồn của sách*: clone mã nguồn của sách về một thư mục cục bộ; xem phụ lục A nếu cần trợ giúp clone repository.

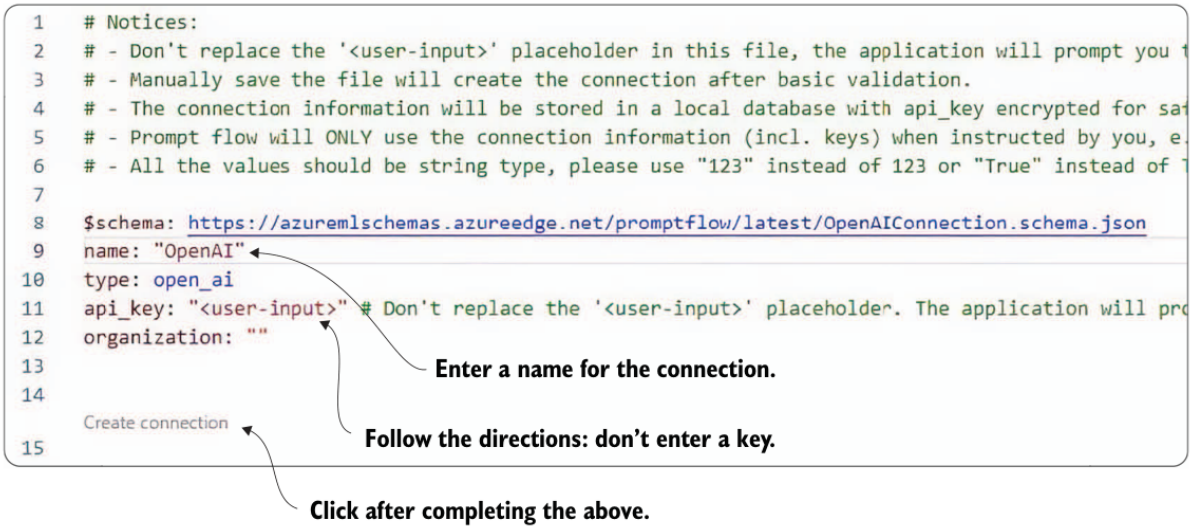
Mở VS Code tại thư mục mã nguồn của sách, `chapter3`. Đảm bảo bạn đã kết nối một môi trường ảo, đã cài các gói prompt flow và extension.

Trước tiên, bạn cần tạo một connection (kết nối) tới tài nguyên LLM bên trong extension prompt flow. Mở extension prompt flow trong VS Code, rồi bấm để mở mục connections. Sau đó bấm dấu cộng bên cạnh tài nguyên LLM để tạo connection mới, như trong Hình 10.7.



Hình 10.7: Tạo một LLM connection mới trong prompt flow

Thao tác này mở một file YAML, nơi bạn cần điền tên connection và các thông tin khác liên quan tới connection. Hãy làm theo hướng dẫn, và đừng nhập API key vào tài liệu, như trong Hình 10.8.

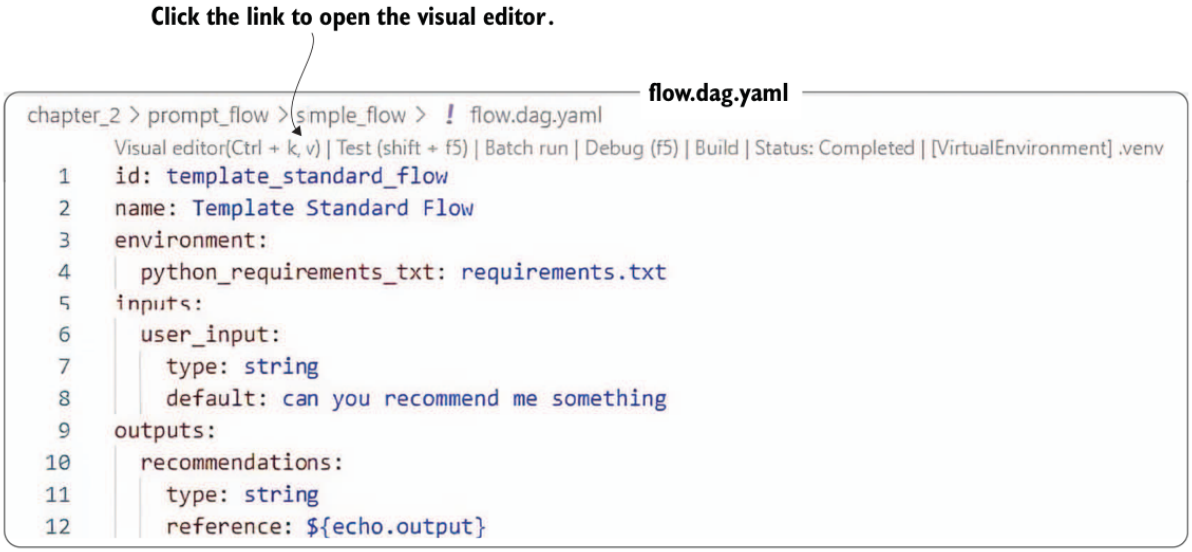


Hình 10.8: Thiết lập thông tin connection cho tài nguyên LLM

Khi đã nhập xong thông tin connection, bấm liên kết Create Connection ở cuối tài liệu. Một cửa sổ terminal sẽ mở bên dưới tài liệu và yêu cầu bạn nhập key. Tuỳ cấu hình terminal, có thể bạn không dán

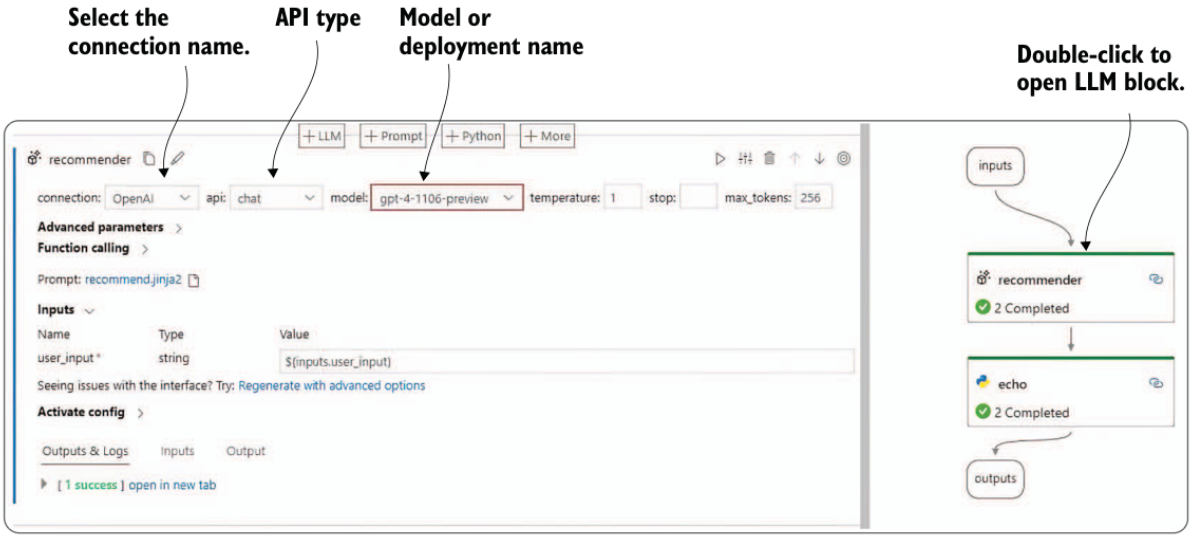
được bằng Ctrl-V hoặc Cmd-V. Thay vào đó, trên Windows bạn có thể dán key bằng cách rê chuột lên terminal rồi bấm chuột phải.

Giờ ta kiểm tra connection bằng cách mở simple flow (flow đơn giản) trong thư mục `chapter_09/promptflow/simpleflow`. Sau đó mở file `flow.dag.yaml` trong VS Code. Đây là file YAML, nhưng extension prompt flow cung cấp một visual editor (trình soạn thảo trực quan), truy cập được bằng cách bấm liên kết Visual Editor ở đầu file, như trong Hình 10.9.



Hình 10.9: Mở visual editor của prompt flow

Sau khi mở cửa sổ visual editor, bạn sẽ thấy một đồ thị biểu diễn flow và các block (khối) của flow. Bấm đúp vào block `recommender`, rồi đặt tên connection, loại API và tên model hoặc deployment, như trong Hình 10.10.



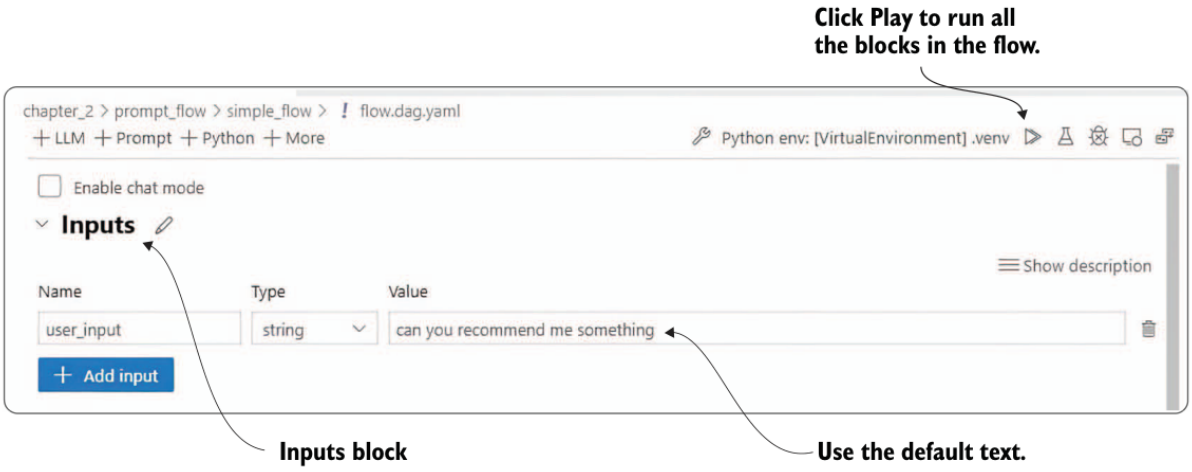
Hình 10.10: Thiết lập chi tiết LLM connection

Một prompt flow gồm một tập block, bắt đầu bằng block `Inputs` và kết thúc ở block `Outputs`. Trong simple flow này, block `recommender` đại diện cho LLM connection và prompt dùng để trò chuyện với mô hình. Block `echo` trong ví dụ đơn giản này lặp lại (echo) đầu vào.

Khi tạo connection tới một LLM, dù trong prompt flow hay qua API, đây là các tham số quan trọng ta luôn cần cân nhắc (tài liệu prompt flow: `https://microsoft.github.io/promptflow`):

- *Connection*: tên connection, nhưng nó cũng đại diện cho dịch vụ bạn kết nối tới. Prompt flow hỗ trợ nhiều dịch vụ, kể cả LLM triển khai cục bộ.
- *API*: loại API. Các lựa chọn là `chat` cho chat completion API, chẳng hạn GPT-4, hoặc `completion` cho các mô hình completion cũ hơn, chẳng hạn OpenAI Davinci.
- *Model*: có thể là tên model hoặc tên deployment, tuỳ connection dịch vụ. Với OpenAI, đây là tên model; với Azure OpenAI, đây là tên deployment.
- *Temperature*: thể hiện mức ngẫu nhiên hay độ biến thiên trong phản hồi của mô hình. Giá trị `1` cho phản hồi biến thiên nhiều, còn `0` nghĩa là mong muốn không có biến thiên. Đây là tham số quan trọng cần hiểu, và như ta sẽ thấy, giá trị phù hợp thay đổi theo trường hợp sử dụng.
- *Stop*: thiết lập tuỳ chọn, báo cho lời gọi LLM dừng sinh token. Nó phù hợp hơn với các mô hình cũ và mô hình mã nguồn mở.
- *Max tokens*: giới hạn số token dùng trong một cuộc hội thoại. Biết mình dùng bao nhiêu token là điều cốt yếu để đánh giá các tương tác với LLM sẽ hoạt động thế nào khi mở rộng quy mô. Đếm token có thể không quan trọng nếu bạn chỉ đang khám phá và nghiên cứu. Tuy nhiên, trong hệ thống production, token thể hiện tải lên LLM, và các connection dùng nhiều token có thể khó mở rộng.
- *Advanced parameters* (tham số nâng cao): bạn có thể đặt thêm vài tuỳ chọn để tinh chỉnh tương tác với LLM, nhưng ta sẽ bàn chủ đề này ở các phần sau của sách.

Sau khi cấu hình block LLM, cuộn lên phần block Inputs và xem đầu vào chính trong trường `user_input`, như trong Hình 10.11. Giữ giá trị mặc định, rồi bấm nút Play ở đầu cửa sổ.



Hình 10.11: Thiết lập đầu vào và chạy flow

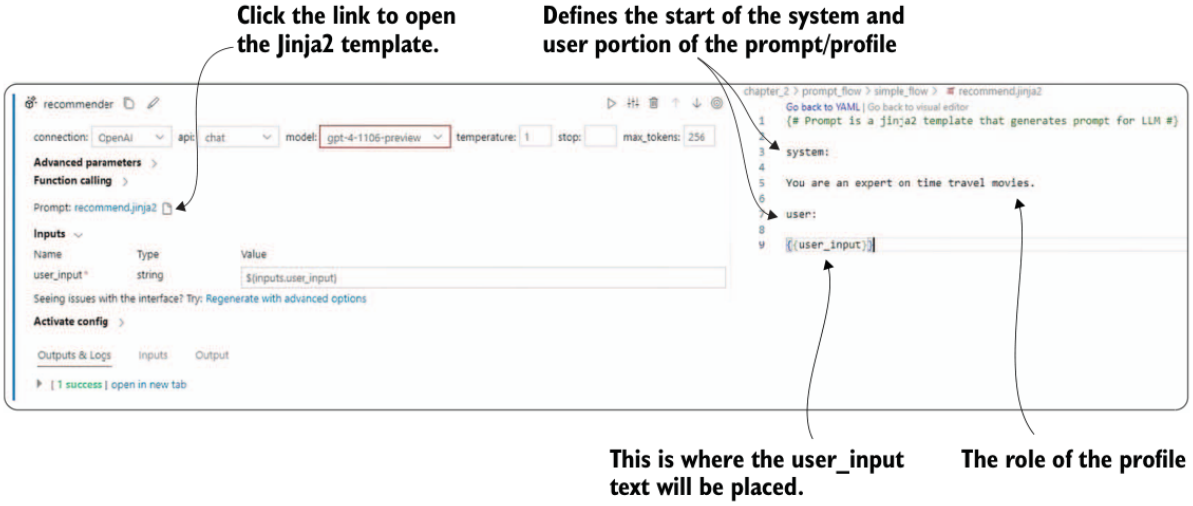
Tất cả block trong flow sẽ chạy, và kết quả hiện trong cửa sổ terminal. Điều đáng chú ý là đầu ra đưa ra các gợi ý phim du hành thời gian. Lý do là block `recommender` đã được đặt sẵn một profile đơn giản; ta sẽ xem cơ chế này ở mục tiếp theo.

10.3.2 Tạo profile bằng Jinja2 template

Flow trả lời bằng các gợi ý phim du hành thời gian là do prompt hay profile mà nó dùng. Mặc định, prompt flow dùng Jinja2 template để định nghĩa nội dung prompt, hay cái mà ta gọi là *profile*. Trong phạm vi sách và việc khám phá AI agent, ta gọi các template này là profile của một flow hoặc agent.

Dù prompt flow không tự gọi mình là một engine cho assistant hay agent, nó chắc chắn đáp ứng tiêu chí để tạo ra proxy agent và các loại agent chung. Như bạn sẽ thấy, prompt flow thậm chí còn hỗ trợ triển khai flow vào container và dưới dạng dịch vụ.

Mở VS Code tới `chapter_09/promptflow/simpleflow/flow.dag.yaml` và mở file trong visual editor. Sau đó tìm trường Prompt và bấm liên kết `recommended.jinja2`, như trong Hình 10.12.



Hình 10.12: Mở Jinja2 template của prompt và xem xét các phần của profile/prompt

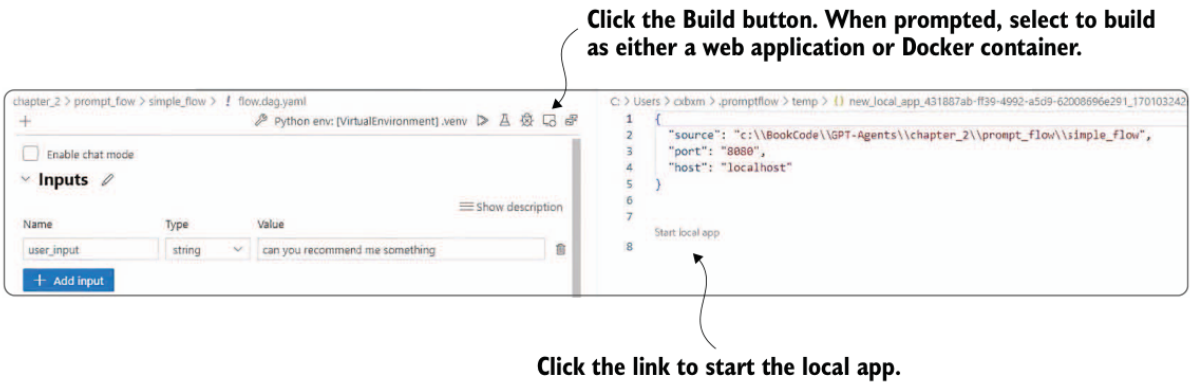
Jinja là một templating engine (công cụ tạo mẫu), và Jinja2 là một phiên bản cụ thể của engine đó. Template là cách rất tốt để định nghĩa bố cục và các phần của bất kỳ dạng tài liệu văn bản nào. Chúng được dùng rộng rãi để tạo HTML, JSON, CSS và các dạng tài liệu khác. Ngoài ra, chúng hỗ trợ đưa code trực tiếp vào template. Tuy chưa có cách chuẩn nào để xây dựng prompt hay agent profile, trong sách này ta ưu tiên dùng templating engine như Jinja.

Đến đây, hãy thay đổi vai trò (role) trong system prompt của template `recommended.jinja2`. Sau đó chạy tất cả block của flow bằng cách mở flow trong visual editor và bấm nút Play. Mục tiếp theo sẽ xem các cách khác để chạy prompt flow cho mục đích kiểm thử hoặc triển khai thực tế.

10.3.3 Triển khai prompt flow thành API

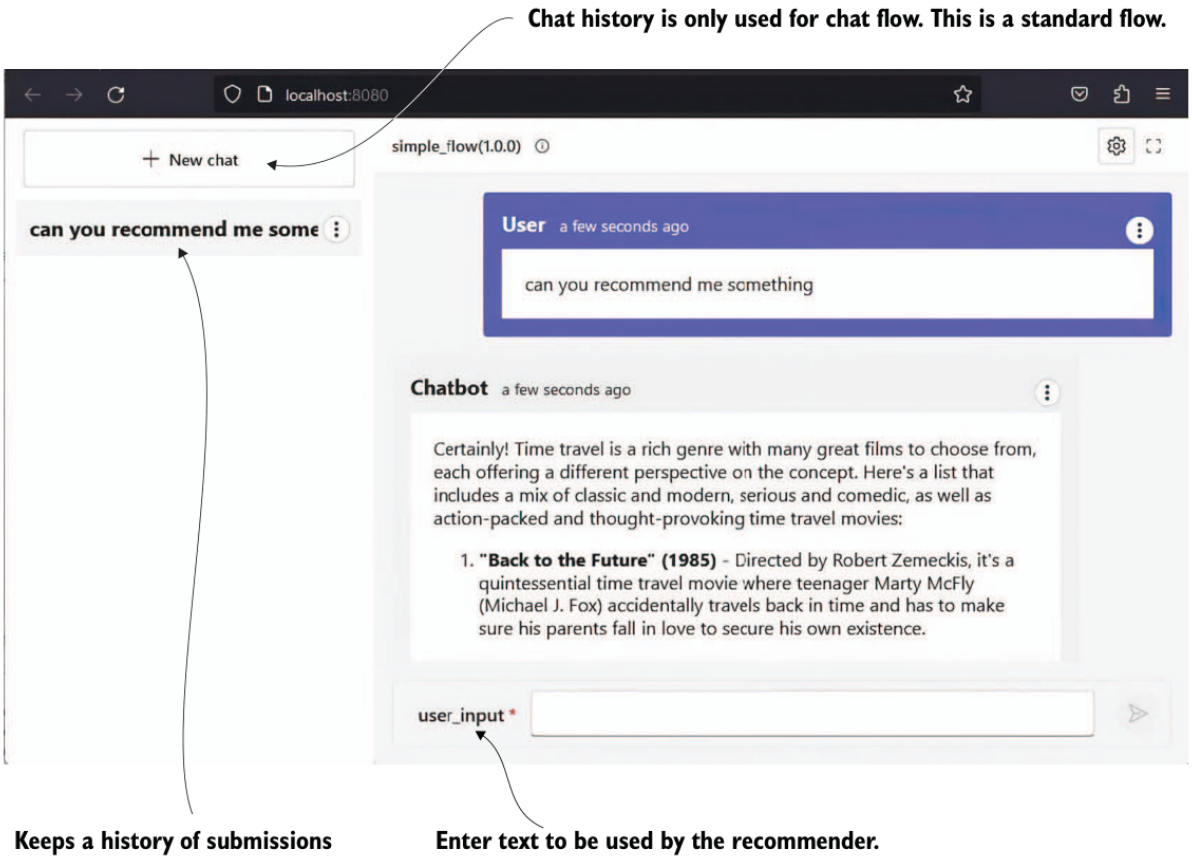
Vì prompt flow cũng được thiết kế để triển khai như một dịch vụ, nó hỗ trợ vài cách triển khai nhanh dưới dạng ứng dụng hoặc API. Prompt flow có thể được triển khai thành ứng dụng web và API cục bộ chạy từ terminal, hoặc dưới dạng Docker container.

Quay lại file `flow.dag.yaml` trong visual editor của VS Code. Ở đầu cửa sổ, cạnh nút Play có vài tuỳ chọn mà ta muốn xem kỹ hơn. Bấm nút Build như trong Hình 10.13, rồi chọn triển khai thành local app (ứng dụng cục bộ). Một file YAML mới sẽ được tạo để cấu hình ứng dụng. Giữ các giá trị mặc định và bấm liên kết Start Local App.



Hình 10.13: Build và khởi chạy flow dưới dạng ứng dụng cục bộ

Thao tác này khởi chạy flow như một ứng dụng web cục bộ, và bạn sẽ thấy một tab trình duyệt mở ra, như trong Hình 10.14. Nhập văn bản vào trường `user_input`, trường bắt buộc được đánh dấu bằng dấu sao đỏ. Nhấn Enter và đợi vài giây để nhận phản hồi.



Hình 10.14: Chạy flow dưới dạng ứng dụng web cục bộ

Bạn sẽ thấy một phản hồi giống như trong Hình 10.12 trước đó, nơi flow hay agent trả lời bằng một danh sách phim du hành thời gian. Thật tuyệt: ta vừa phát triển agent profile đầu tiên, tương đương với một proxy agent. Tuy nhiên, ta cần xác định các gợi ý đó thành công hay có giá trị đến đâu. Ở mục tiếp theo, ta tìm hiểu cách đánh giá prompt và profile.

10.4 Đánh giá profile: rubric và grounding

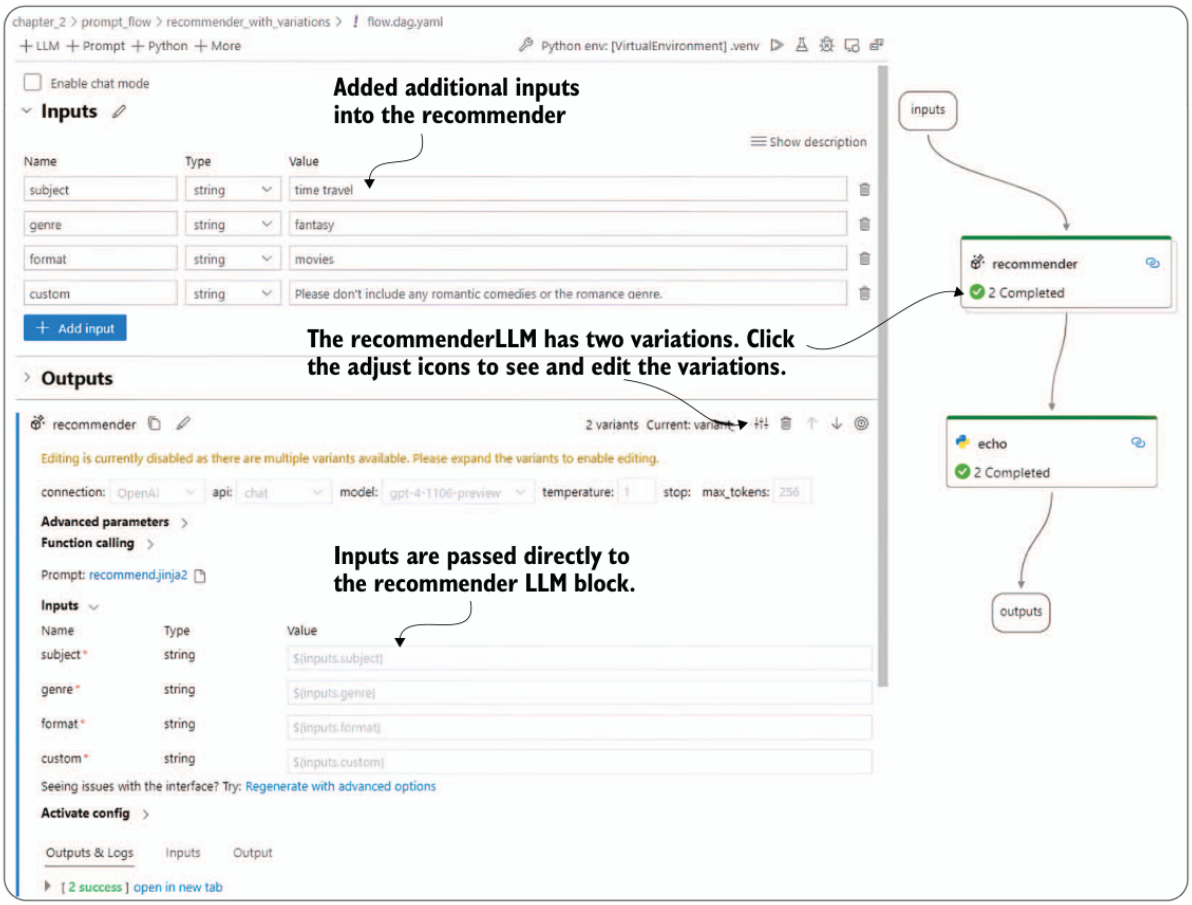
Một yếu tố then chốt của bất kỳ prompt hay agent profile nào là mức độ nó hoàn thành nhiệm vụ được giao. Như trong ví dụ gợi ý của ta, yêu cầu một agent profile đưa ra danh sách gợi ý là tương đối dễ, nhưng muốn biết các gợi ý đó có hữu ích không thì ta phải đánh giá phản hồi.

May mắn là prompt flow được thiết kế để đánh giá prompt/profile ở quy mô lớn. Hạ tầng vững chắc của nó cho phép việc đánh giá các tương tác LLM được song song hoá và quản lý như các worker, nhờ đó hàng trăm lượt đánh giá profile và biến thể có thể diễn ra nhanh chóng.

Phần tiếp theo xem cách cấu hình prompt flow để chạy các biến thể (variation) prompt/profile đối chiếu với nhau. Ta cần hiểu điều này trước khi đánh giá hiệu năng của profile.

Prompt flow cung cấp cơ chế cho phép có nhiều biến thể trong một LLM prompt/profile. Công cụ này rất tốt để so sánh khác biệt nhỏ hoặc lớn giữa các biến thể profile. Khi dùng trong đánh giá hàng loạt (bulk evaluation), nó có thể vô cùng hữu ích để nhanh chóng đánh giá hiệu năng của một profile.

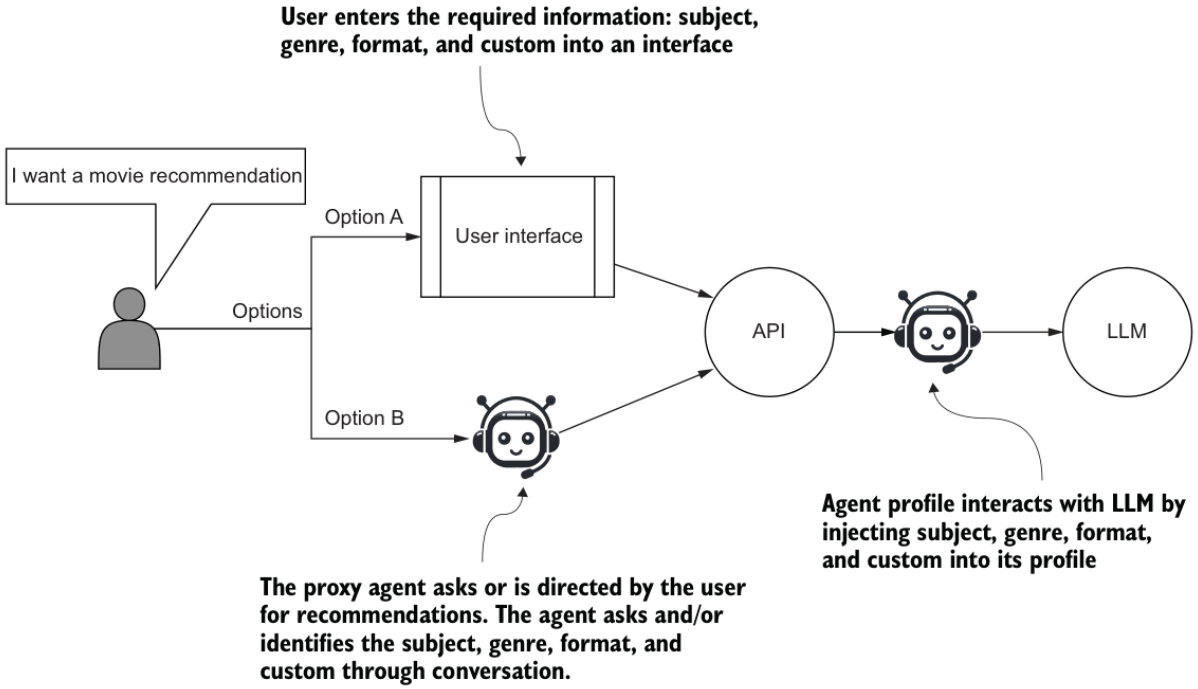
Mở file `recommender_with_variations/flow.dag.yaml` trong VS Code và visual editor của flow, như trong Hình 10.15. Lần này, ta làm cho profile tổng quát hơn và cho phép tuỳ biến ở mức đầu vào. Nhờ vậy, ta có thể mở rộng việc gợi ý sang bất cứ thứ gì chứ không chỉ phim du hành thời gian.



Hình 10.15: Recommender với các biến thể trong flow và các đầu vào mở rộng

Các đầu vào mới Subject (chủ đề), Genre (thể loại), Format (định dạng) và Custom (tuỳ chỉnh) cho phép ta định nghĩa một profile dễ điều chỉnh cho bất kỳ kiểu gợi ý nào. Điều này cũng có nghĩa là ta phải chuẩn bị sẵn (prime) các đầu vào dựa trên trường hợp gợi ý. Có nhiều cách để prime các đầu vào này; Hình 10.16 trình bày hai ví dụ. Hình cho thấy hai phương án A và B để prime đầu vào. Phương án A là giao diện người dùng (UI) cổ điển; chẳng hạn có các đối tượng để người dùng chọn chủ đề hoặc thể loại. Phương án B đặt một proxy/chat agent để tương tác với người dùng, nhằm hiểu rõ hơn chủ đề, thể loại,

v.v. mà họ mong muốn.



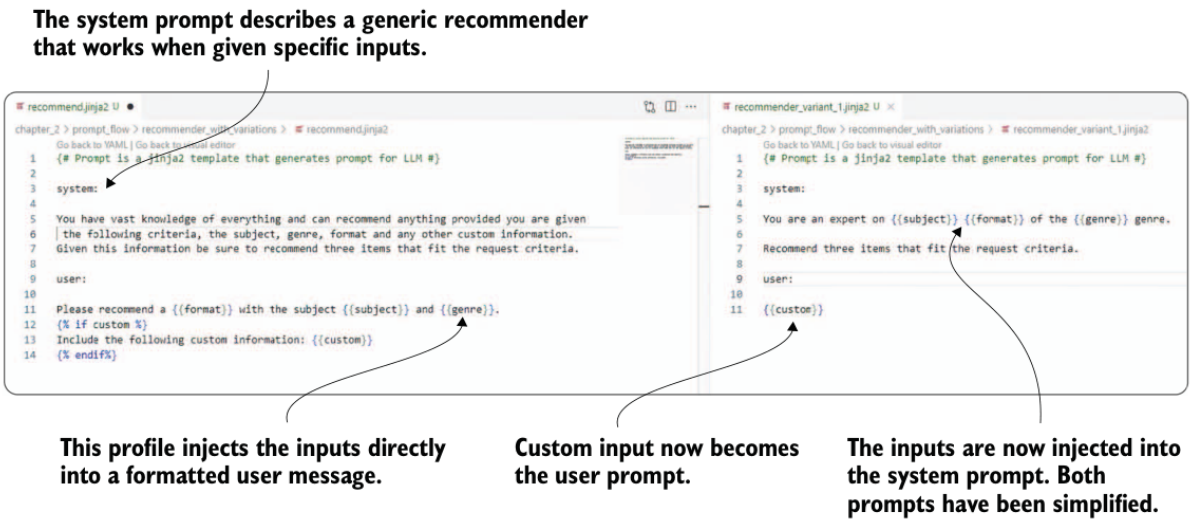
Hình 10.16: Các phương án tương tác với người dùng để giao tiếp với agent profile và prime đầu vào cho agent profile

Dù LLM mạnh mẽ đến đâu, có thể bạn vẫn muốn hoặc cần dùng phương án A. Lợi ích của phương án A là bạn có thể ràng buộc và kiểm tra hợp lệ (validate) đầu vào như với bất kỳ UI hiện đại nào. Ngược lại, nhược điểm của phương án A là hành vi bị ràng buộc có thể giới hạn và hạn chế các trường hợp sử dụng trong tương lai.

Phương án B là cách linh hoạt và tự nhiên hơn, không cần UI truyền thống. Nó mạnh mẽ và dễ mở rộng hơn nhiều so với phương án A, nhưng cũng đưa vào nhiều ẩn số hơn khi đánh giá. Tuy vậy, nếu proxy agent mà phương án B dùng được viết tốt, nó có thể giúp rất nhiều trong việc thu thập thông tin tốt hơn từ người dùng.

Phương án bạn chọn sẽ quyết định cách bạn cần đánh giá profile. Nếu bạn chấp nhận một UI có ràng buộc, nhiều khả năng các đầu vào cũng bị ràng buộc trong một tập giá trị rời rạc. Hiện tại, ta giả định dùng phương án B để prime đầu vào, nghĩa là giá trị đầu vào được xác định theo tên của chúng.

Để quay lại VS Code và chế độ xem trực quan của flow recommender with variants, bấm biểu tượng đã chỉ trong Hình 10.15 để mở các biến thể và cho phép chỉnh sửa. Sau đó bấm các liên kết `recommend.jinja2` và `recommender_variant_1.jinja2` để mở hai file cạnh nhau, như trong Hình 10.17.



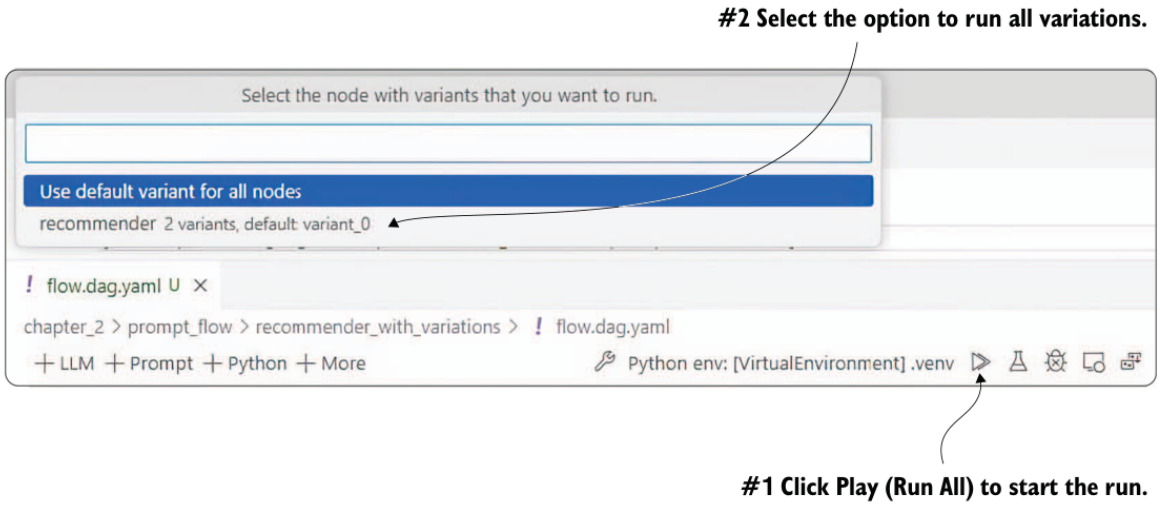
Hình 10.17: So sánh song song các template profile biến thể của recommender

Hình 10.17 cho thấy sự khác biệt giữa các profile biến thể. Một profile đưa các đầu vào vào user prompt, profile còn lại đưa chúng vào system prompt. Tuy nhiên, điều cốt yếu cần hiểu là biến thể có thể bao gồm nhiều thứ hơn thiết kế profile, như liệt kê trong Bảng 10.1.

Bảng 10.1: Các lựa chọn biến thể LLM trong prompt flow

Lựa chọn	Ví dụ về phương án đánh giá	Ghi chú
Jinja2 prompt template	So sánh các biến thể system prompt, biến thể user prompt, hoặc biến thể prompt kết hợp.	Có vô số tổ hợp và kỹ thuật có thể áp dụng ở đây. Prompt engineering luôn không ngừng phát triển.
LLM	So sánh GPT-3.5 với GPT-4. So sánh GPT-4 với GPT-4 Turbo. So sánh mô hình mã nguồn mở với mô hình thương mại.	Đây là cách hữu ích để đánh giá và grounding hiệu năng mô hình đối với một prompt. Nó cũng giúp bạn tinh chỉnh profile để chạy được với các mô hình mã nguồn mở và/hoặc rẻ hơn.
Temperature	So sánh temperature bằng 0 (không ngẫu nhiên) với 1 (ngẫu nhiên tối đa).	Thay đổi temperature có thể làm thay đổi đáng kể phản hồi của một số prompt, theo hướng cải thiện hoặc làm giảm hiệu năng.
Max tokens	So sánh giới hạn token nhỏ với kích thước token lớn hơn.	Cách này cho phép bạn giảm và tối đa hoá việc sử dụng token.
Advanced parameters	So sánh khác biệt giữa các tuỳ chọn như `top_p`, `presence_penalty`, `frequency_penalty` và `logit_bias`.	Ta sẽ bàn cách dùng các tham số nâng cao này ở các chương sau.
Function calls	So sánh các lời gọi function thay thế.	Function call sẽ được đề cập ở phần sau của chương này.

Trong ví dụ đơn giản này, ta chỉ dùng biến thể prompt bằng cách thay đổi việc đưa đầu vào vào system prompt hay user prompt. Xem lại Hình 10.17 để thấy chúng trông như thế nào. Sau đó ta có thể nhanh chóng chạy cả hai biến thể bằng cách bấm nút Play (Run All) ở trên cùng và chọn cả hai, như trong Hình 10.18.



Hình 10.18: Chạy đồng thời cả hai biến thể prompt

Trong cửa sổ terminal, bạn sẽ thấy kết quả của cả hai lượt chạy. Các kết quả nhiều khả năng trông giống nhau, vì vậy ở mục tiếp theo ta phải chuyển sang cách đánh giá sự khác biệt giữa các biến thể.

10.5 Hiểu về rubric và grounding

Thông thường, ta không thể đánh giá hiệu năng của prompt/profile bằng một thước đo độ chính xác hay tỷ lệ phần trăm đúng. Đo hiệu năng của một profile phụ thuộc vào trường hợp sử dụng và kết quả mong muốn. Nếu chỉ đơn giản là xác định phản hồi đúng hay sai thì càng tốt. Tuy nhiên, trong hầu hết trường hợp, việc đánh giá sẽ không đơn giản như vậy.

Trong giáo dục, khái niệm **rubric** định nghĩa một tập tiêu chí và tiêu chuẩn có cấu trúc mà học sinh phải đạt được để nhận một mức điểm nhất định. Rubric cũng có thể dùng để định nghĩa một chuẩn hướng dẫn cho hiệu năng của profile hay prompt. Ta có thể làm theo các bước sau để định nghĩa một rubric dùng để đánh giá hiệu năng của profile hoặc prompt:

1. *Xác định mục đích và mục tiêu.* Xác định các mục tiêu bạn muốn profile hay agent đạt được. Ví dụ, bạn muốn đánh giá chất lượng gợi ý cho một nhóm khán giả nhất định, hay chất lượng tổng thể với một chủ đề, định dạng hoặc đầu vào khác?

2. *Định nghĩa tiêu chí.* Xây dựng một tập tiêu chí hoặc chiều đánh giá mà bạn sẽ dùng để đánh giá profile. Các tiêu chí này cần khớp với mục tiêu của bạn và đưa ra hướng dẫn đánh giá rõ ràng. Mỗi tiêu chí cần cụ thể và đo được. Ví dụ, bạn có thể muốn đo một gợi ý theo mức độ phù hợp với thể loại, rồi đến chủ đề và định dạng.

3. *Tạo thang đo.* Thiết lập một thang đánh giá mô tả các mức hiệu năng cho từng tiêu chí. Thang thường dùng gồm thang số (ví dụ 1–5) hoặc thang mô tả (ví dụ Excellent, Good, Fair, Poor, tức xuất sắc, tốt, khá, kém).

4. *Viết mô tả.* Với mỗi mức trên thang, đưa ra mô tả rõ ràng, súc tích cho biết thế nào là hiệu năng tốt và thế nào là hiệu năng yếu hơn ở từng tiêu chí.

5. *Áp dụng rubric.* Khi đánh giá một prompt hay profile, dùng rubric để đánh giá hiệu năng của prompt dựa trên các tiêu chí đã thiết lập. Chấm điểm hoặc xếp hạng cho từng tiêu chí, có xét tới mô tả của từng mức.

6. *Tính tổng điểm.* Tuỳ rubric, bạn có thể tính tổng điểm bằng cách cộng điểm từng tiêu chí, hoặc dùng trung bình có trọng số nếu một số tiêu chí quan trọng hơn các tiêu chí khác.

7. *Đảm bảo tính nhất quán khi đánh giá.* Nếu có nhiều người cùng đánh giá profile, điều cốt yếu là đảm bảo việc chấm điểm nhất quán.

8. *Xem xét, sửa đổi và lặp lại.* Định kỳ xem xét và sửa đổi rubric để đảm bảo nó khớp với mục đích và mục tiêu đánh giá. Điều chỉnh khi cần để nâng cao hiệu quả của rubric.

Grounding là một khái niệm có thể áp dụng vào việc đánh giá profile và prompt. Nó xác định mức độ một phản hồi khớp với các tiêu chí và tiêu chuẩn cụ thể của một rubric cho trước. Bạn cũng có thể coi grounding là kỳ vọng nền (baseline) đối với đầu ra của một prompt hay profile.

Danh sách sau tóm tắt vài điểm quan trọng khác khi dùng grounding trong đánh giá profile:

- Grounding là việc làm cho phản hồi khớp với các tiêu chí, mục tiêu và ngữ cảnh do rubric và prompt định nghĩa.
- Grounding bao gồm việc đánh giá xem phản hồi có trực tiếp đáp ứng các tiêu chí của rubric, bám sát chủ đề và tuân thủ mọi chỉ dẫn được đưa ra hay không.
- Người đánh giá và các bài đánh giá đo độ chính xác, mức liên quan và sự tuân thủ tiêu chuẩn khi đánh giá grounding.
- Grounding đảm bảo đầu ra của phản hồi bám chắc vào ngữ cảnh đã chỉ định, giúp quá trình đánh giá khách quan và có ý nghĩa hơn.

Một phản hồi được grounding tốt sẽ khớp với mọi tiêu chí của rubric trong ngữ cảnh và mục tiêu đã cho. Phản hồi grounding kém sẽ không đạt hoặc bỏ lỡ toàn bộ tiêu chí, ngữ cảnh và mục tiêu.

Vì khái niệm rubric và grounding có thể vẫn còn trừu tượng, hãy xem cách áp dụng chúng vào ví dụ recommender hiện tại. Danh sách sau đi theo quy trình định nghĩa rubric, áp dụng cho ví dụ recommender của ta:

1. *Xác định mục đích và mục tiêu.* Mục đích của profile/prompt là gợi ý ba mục hàng đầu, cho trước subject (chủ đề), format (định dạng), genre (thể loại) và đầu vào custom (tuỳ chỉnh).

2. *Định nghĩa tiêu chí.* Để đơn giản, ta đánh giá mức độ một gợi ý cụ thể khớp với các tiêu chí đầu vào đã cho: subject, format và genre. Ví dụ, nếu một profile gợi ý một cuốn sách khi được yêu cầu định dạng phim, ta kỳ vọng điểm thấp ở tiêu chí format.

3. *Tạo thang đo.* Vẫn giữ mọi thứ đơn giản, ta dùng thang 1–5 (1 là kém, 5 là xuất sắc).

4. *Viết mô tả.* Xem các mô tả chung cho thang đánh giá trong Bảng 10.2.

5. *Áp dụng rubric.* Khi đã có rubric ở giai đoạn này, một bài tập rất tốt là tự tay đánh giá các gợi ý theo rubric.

6. *Tính tổng điểm.* Với rubric của ta, ta lấy trung bình điểm của mọi tiêu chí làm tổng điểm.

7. *Đảm bảo tính nhất quán khi đánh giá.* Kỹ thuật đánh giá mà ta sẽ dùng cho kết quả rất nhất quán.

8. *Xem xét, sửa đổi và lặp lại.* Ta sẽ xem xét, so sánh và lặp lại trên các profile, các rubric và chính các bài đánh giá.

Bảng 10.2: Các mức đánh giá trong rubric

Mức	Mô tả
1	Poor alignment (khớp kém): ngược hẳn với điều được kỳ vọng theo tiêu chí.
2	Bad alignment (khớp tệ): không phù hợp với tiêu chí đã cho.
3	Mediocre alignment (khớp trung bình): có thể phù hợp hoặc không phù hợp lắm với tiêu chí đã cho.
4	Good alignment (khớp tốt): có thể không khớp 100% với tiêu chí nhưng nhìn chung là phù hợp.
5	Excellent alignment (khớp xuất sắc): đây là một gợi ý tốt cho các tiêu chí đã cho.

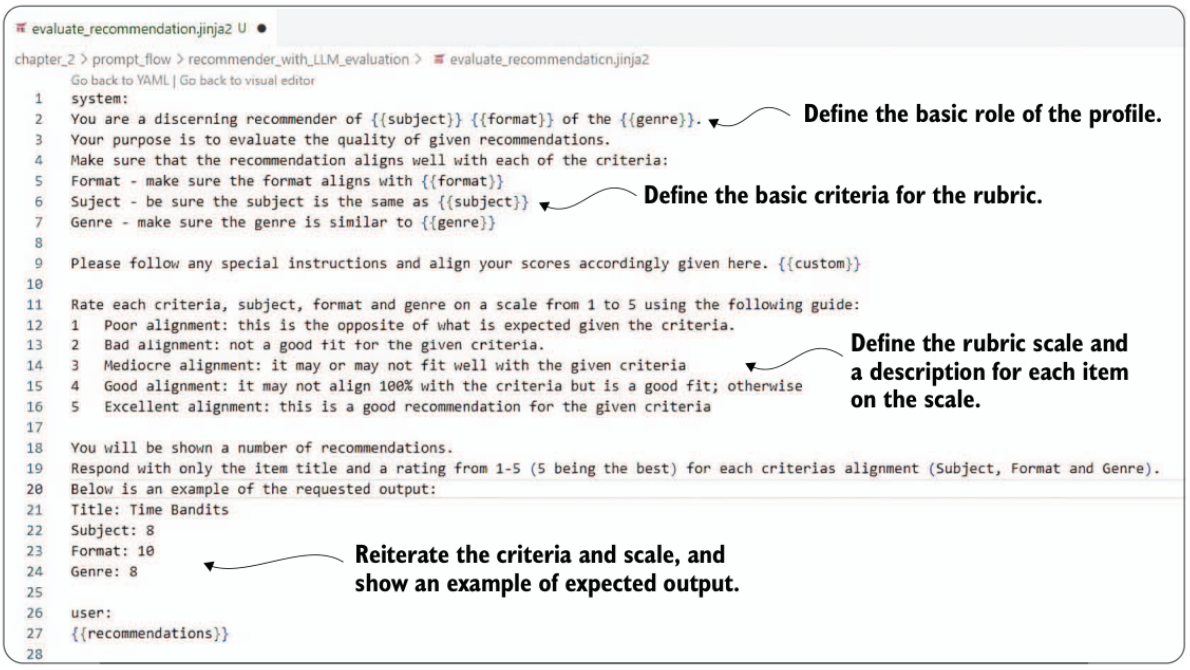
Giờ có thể áp dụng rubric cơ bản này để đánh giá phản hồi của profile. Bạn có thể làm thủ công, hoặc như sẽ thấy ở mục tiếp theo, dùng một LLM profile thứ hai.

10.6 Đánh giá grounding bằng một LLM profile

Mục này dùng thêm một LLM prompt/profile khác để đánh giá và grounding. LLM prompt thứ hai này thêm một block nữa sau khi các gợi ý được sinh ra. Nó xử lý các gợi ý đã sinh và đánh giá từng gợi ý theo rubric ở trên.

Trước khi có GPT-4 và các LLM tinh vi khác, ta sẽ không bao giờ nghĩ tới việc dùng một LLM prompt khác để đánh giá hay grounding một profile. Khi dùng LLM để grounding một profile, thường bạn nên dùng một mô hình khác. Tuy nhiên, nếu bạn đang so sánh các profile với nhau, việc dùng cùng một LLM cho đánh giá và grounding là phù hợp.

Mở file `recommender_with_LLM_evaluation\flow.dag.yaml` trong visual editor của prompt flow, cuộn xuống block `evaluate_recommendation`, rồi bấm liên kết `evaluate_recommendation.jinja2` để mở file, như trong Hình 10.19. Từng phần của rubric được đánh dấu trong hình.



Hình 10.19: Prompt đánh giá, với từng phần của rubric được chú thích

Ta có một rubric không chỉ được định nghĩa rõ ràng mà còn ở dạng một prompt có thể dùng để đánh giá các gợi ý. Điều này cho phép ta đánh giá hiệu quả của các gợi ý ứng với một profile một cách tự động. Dĩ nhiên, bạn cũng có thể dùng rubric để chấm điểm và đánh giá các gợi ý thủ công, nhằm có một baseline tốt hơn.

Lưu ý

Dùng LLM để đánh giá prompt và profile cho một baseline vững chắc để so sánh hiệu năng của profile. Nó còn làm được việc này mà không có thiên kiến của con người, theo cách có kiểm soát và lặp lại được. Đây là cơ chế rất tốt để thiết lập baseline grounding cho bất kỳ profile hay prompt nào.

Quay lại visual editor của flow `recommender_with_LLM_evaluation`, ta chạy flow bằng cách bấm nút Play và quan sát đầu ra. Bạn có thể chạy một gợi ý đơn lẻ hoặc chạy cả hai biến thể khi được hỏi. Đầu ra của một lượt đánh giá đơn với các đầu vào mặc định được trình bày trong listing sau.

Listing 9.1 LLM rubric evaluation output

```
{
    "recommendations": "Title: The Butterfly Effect
Subject: 5
Format: 5
Genre: 4

Title: Primer
Subject: 5
Format: 5
Genre: 4

Title: Time Bandits
Subject: 5
Format: 5
Genre: 5"
}
```

Giờ ta đã có một rubric để grounding recommender, và việc đánh giá được chạy tự động bằng một LLM prompt thứ hai. Ở mục tiếp theo, ta xem cách thực hiện nhiều lượt đánh giá cùng lúc rồi lấy tổng điểm cho tất cả.

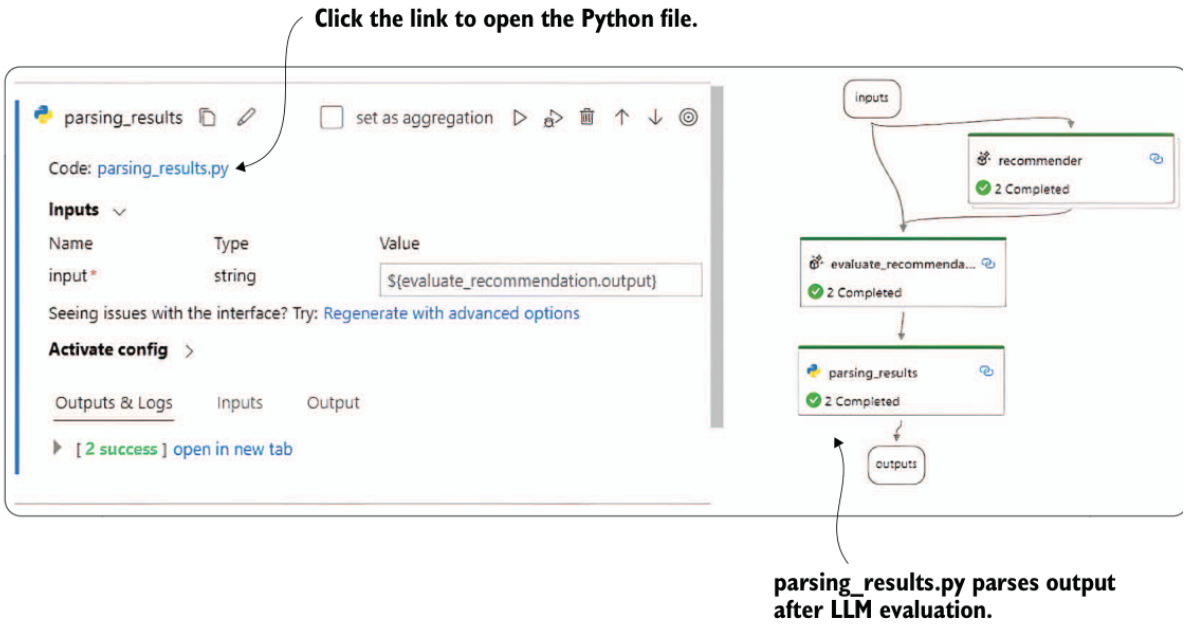
10.7 So sánh các profile: tìm ra profile hoàn hảo

Với hiểu biết về rubric và grounding, giờ ta có thể chuyển sang đánh giá và lặp để tìm profile hoàn hảo. Tuy nhiên, trước đó ta cần làm sạch đầu ra của block đánh giá LLM. Việc này đòi hỏi parse các gợi ý thành dạng gần với Python hơn, và ta sẽ làm ở mục tiếp theo.

10.7.1 Parse đầu ra đánh giá của LLM

Vì đầu ra thô của block đánh giá là văn bản, ta muốn parse nó thành dạng dễ dùng hơn. Dĩ nhiên, viết hàm parse là việc đơn giản, nhưng có những cách tốt hơn để ép phản hồi sang dạng mong muốn một cách tự động. Ta đã đề cập các phương pháp tốt hơn để trả về phản hồi ở Chương 6 về hành động của agent.

Mở file `chapter_09\prompt_flow\recommender_with_parsing\flow.dag.yaml` trong VS Code và xem flow trong visual editor. Tìm block `parsing_results`, rồi bấm liên kết để mở file Python trong trình soạn thảo, như trong Hình 10.20. Code của file `parsing_results.py` nằm trong listing 9.2.



Hình 10.20: Mở file parsing_results.py trong VS Code

Listing 9.2 parsing_results.py

```
from promptflow import tool

@tool     # Decorator đặc biệt để đánh dấu tool block
def parse(input: str) -> str:
    # Splitting the recommendations into individual movie blocks
    rblocks = input.strip().split("\n\n")     # Tách đầu vào theo hai dòng trống
    ↪ liên tiếp

    # Function to parse individual recommendation block into dictionary
    def parse_block(block):
        lines = block.split('\n')
        rdict = {}
        for line in lines:
            kvs = line.split(': ')
            key, value = kvs[0], kvs[1]
            rdict[key.lower()] = value    # Tạo một mục trong dictionary và gán
            ↪ giá trị
        return rdict

    parsed = [parse_block(block) for block in rblocks]   # Lặp qua từng block và
    ↪ parse thành dictionary key/value

    return parsed
```

Ta đang chuyển đầu ra recommendations ở listing 9.1, vốn chỉ là một chuỗi, thành một dictionary. Vì vậy code này sẽ chuyển chuỗi đó thành khối JSON trình bày dưới đây.

Trước khi parse:

```
"Title: The Butterfly Effect
Subject: 5
Format: 5
Genre: 4
```

```
Title: Primer
Subject: 5
Format: 5
Genre: 4

Title: Time Bandits
Subject: 5
Format: 5
Genre: 5"
```

Sau khi parse:

```
      {
           "title": " The Butterfly Effect",
           "subject": "5",
           "format": "5",
           "genre": "4"
       },
       {
           "title": " Primer",
           "subject": "5",
           "format": "5",
           "genre": "4"
       },
       {
           "title": " Time Bandits",
           "subject": "5",
           "format": "5",
           "genre": "5"
       }
```

Đầu ra của block `parsing_results` giờ được chuyển sang đầu ra và được bọc trong một danh sách các recommendation. Ta có thể thấy toàn bộ quá trình này bằng cách chạy flow. Mở `flow.dag.yaml` của flow trong visual editor rồi bấm nút Play (Run All). Nhớ chọn dùng cả hai biến thể recommender. Bạn sẽ thấy cả hai biến thể chạy và xuất ra terminal.

Đến đây, ta có một flow recommendation và đánh giá LLM hoàn chỉnh, xuất ra điểm cho từng tiêu chí trên mỗi đầu ra. Tuy nhiên, để đánh giá toàn diện một profile, ta muốn sinh nhiều gợi ý với các tiêu chí đa dạng. Ở mục tiếp theo, ta xem cách xử lý theo lô (batch processing) các flow.

10.7.2 Chạy batch processing trong prompt flow

Trong profile recommendation tổng quát, ta muốn đánh giá các tiêu chí đầu vào khác nhau ảnh hưởng thế nào đến các gợi ý được sinh ra. May mắn là prompt flow có thể xử lý theo lô mọi biến thể ta muốn kiểm thử. Giới hạn chỉ là thời gian và tiền bạc ta sẵn sàng bỏ ra.

Để chạy batch processing, trước tiên ta phải tạo một tài liệu JSON Lines (JSONL) hoặc danh sách JSON chứa các tiêu chí đầu vào. Nếu còn nhớ, các tiêu chí đầu vào của ta ở dạng JSON như sau:

```
{
    "subject": "time travel",
    "format": "books",
    "genre": "fantasy",
    "custom": "don't include any R rated content"
}
```

Ta muốn tạo một danh sách các đối tượng JSON như trên, tốt nhất là ngẫu nhiên. Dĩ nhiên, cách đơn giản để làm việc này là yêu cầu ChatGPT tạo tài liệu JSONL bằng prompt sau:

Prompt gửi ChatGPT

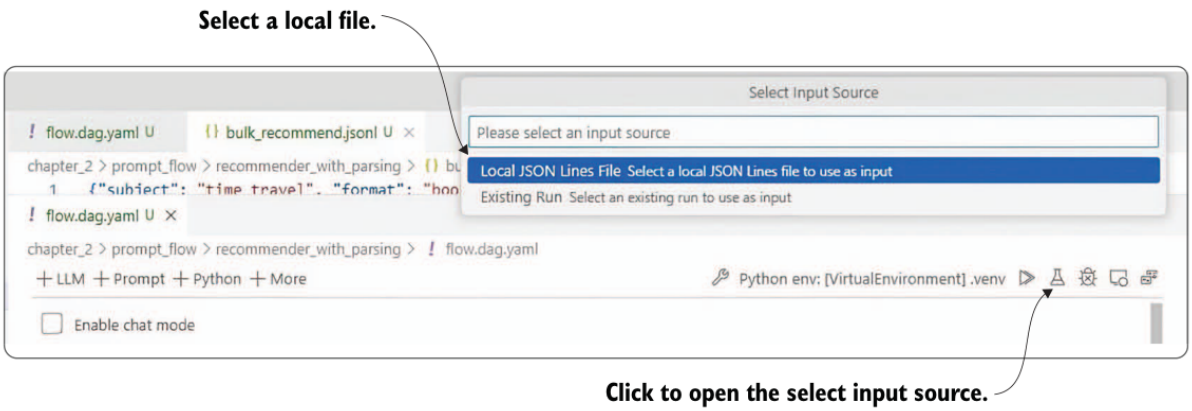
```
I am developing a recommendation agent. The agent will recommend anything
↪  given the following criteria:
1. subject - examples: time travel, cooking, vacation
2. format - examples: books, movies, games
3. genre: documentary, action, romance
4. custom: don't include any R rated content
Can you please generate a random list of these criteria and output it in the
↪  format of a JSON Lines file, JSONL. Please include 10 items in the list.
```

Hãy thử bằng cách vào ChatGPT và nhập prompt trên. Một file đã được tạo sẵn nằm trong thư mục flow, tên là `bulk_recommend.jsonl`. Nội dung file được trình bày dưới đây để tham khảo:

```
{
  "subject": "time travel",
  "format": "books",
  "genre": "fantasy",
  "custom": "don't include any R rated content"
}
{
  "subject": "space exploration",
  "format": "podcasts",
  "genre": "sci-fi",
  "custom": "include family-friendly content only"
}
{
  "subject": "mystery",
  "format": "podcasts",
  "genre": "fantasy",
  "custom": "don't include any R rated content"
}
{
  "subject": "space exploration",
  "format": "podcasts",
  "genre": "action",
  "custom": "include family-friendly content only"
}
{
  "subject": "vacation",
  "format": "books",
  "genre": "thriller",
  "custom": "don't include any R rated content"
}
{
  "subject": "mystery",
  "format": "books",
  "genre": "sci-fi",
  "custom": "don't include any R rated content"
}
{
  "subject": "mystery",
  "format": "books",
  "genre": "romance",
  "custom": "don't include any R rated content"
}
{
  "subject": "vacation",
  "format": "movies",
```

```
  "genre": "fantasy",
  "custom": "don't include any R rated content"
}
{
  "subject": "cooking",
  "format": "TV shows",
  "genre": "thriller",
  "custom": "include family-friendly content only"
}
{
  "subject": "mystery",
  "format": "movies",
  "genre": "romance",
  "custom": "include family-friendly content only"
}
```

Với file bulk này, ta có thể chạy cả hai biến thể trên các tiêu chí đầu vào khác nhau trong file JSONL. Mở file `flow.dag.yaml` trong visual editor, bấm Batch (biểu tượng ống nghiệm) để bắt đầu quá trình nạp dữ liệu hàng loạt, rồi chọn file như trong Hình 10.21. Trên một số hệ điều hành, mục này có thể hiện là `LocalDataFile`.



Hình 10.21: Nạp file JSONL hàng loạt để chạy flow trên nhiều biến thể đầu vào

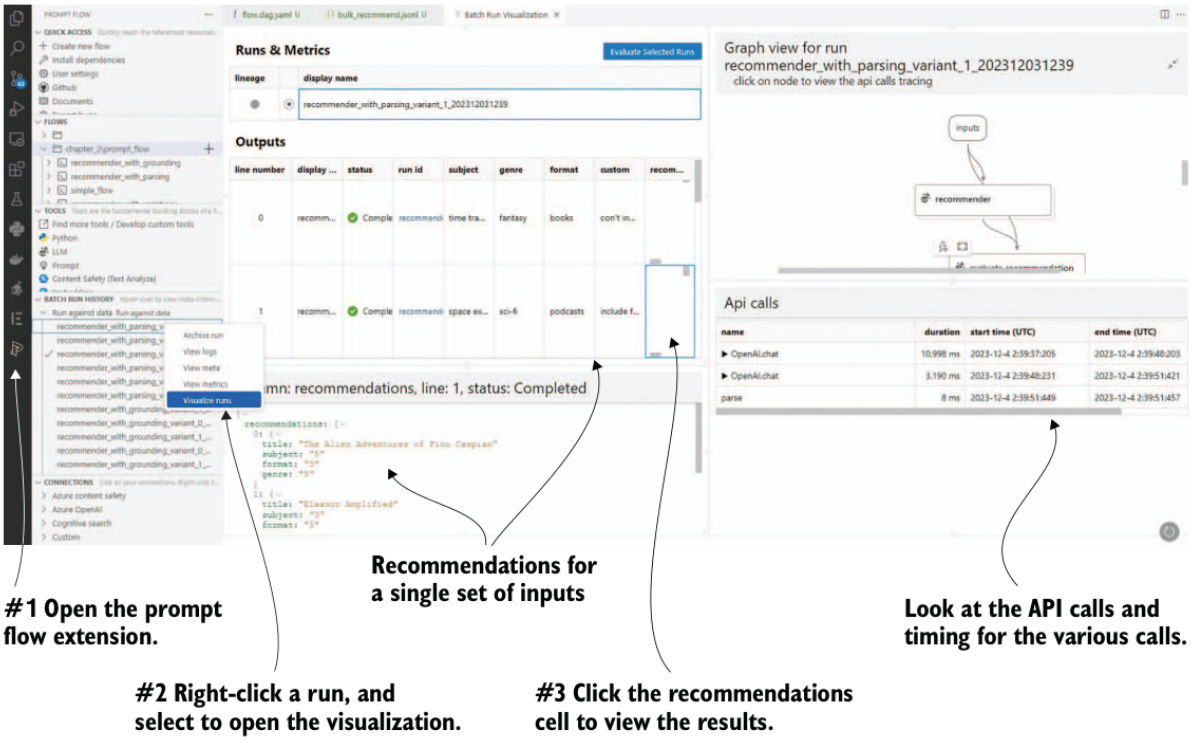
Sau khi chọn file bulk, một tài liệu YAML mới sẽ mở ra với liên kết Run được thêm ở cuối file, như trong Hình 10.22. Bấm liên kết để chạy batch với các đầu vào.

Hình 10.22: Chạy batch với các đầu vào

Lúc này sẽ có vài việc xảy ra. Visual editor của flow xuất hiện, bên cạnh là một file log mở ra cho thấy tiến độ lượt chạy. Trong cửa sổ terminal, bạn sẽ thấy các tiến trình worker được tạo ra và chạy.

Hãy kiên nhẫn. Lượt batch, dù chỉ 10 mục, có thể mất vài phút hoặc vài giây, tuỳ nhiều yếu tố như phần cứng, các lời gọi trước đó, v.v. Đợi lượt chạy hoàn tất, bạn sẽ thấy bản tóm tắt kết quả trong terminal.

Bạn cũng có thể xem kết quả lượt chạy bằng cách mở extension prompt flow và chọn lượt chạy gần nhất, như trong Hình 10.23. Sau đó, bạn có thể đi sâu vào từng lượt chạy bằng cách bấm vào các ô của bảng. Hộp thoại này hiển thị rất nhiều thông tin, giúp xử lý sự cố cho flow và profile.



Hình 10.23: Hình hoá lượt chạy ban đầu và xem xét một lượt chạy batch

Rất nhiều thông tin được ghi lại trong một lượt batch, và bạn có thể khám phá phần lớn qua công cụ trực quan hoá (visualizer). Có thể xem thêm thông tin bằng cách bấm liên kết thư mục output từ cửa sổ

terminal. Thao tác này mở một phiên VS Code khác tại thư mục output để bạn xem log của lượt chạy và các chi tiết khác.

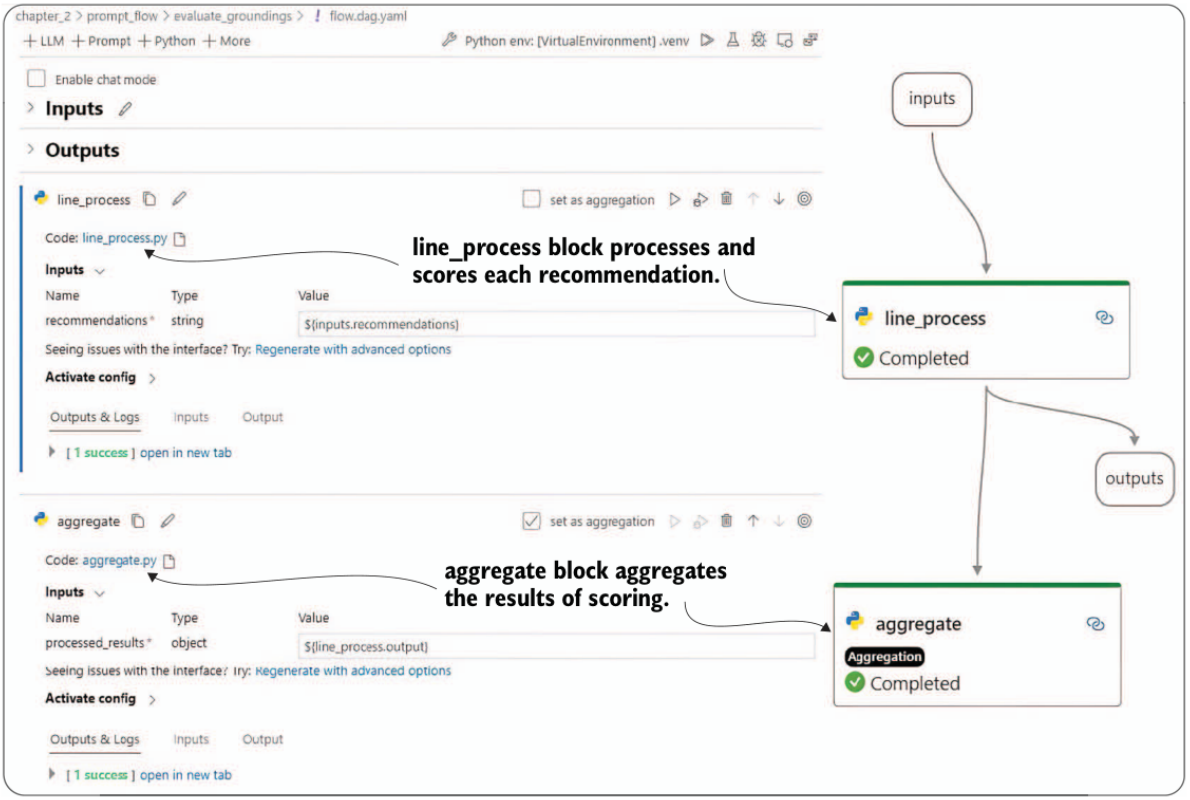
Giờ khi đã hoàn tất lượt batch cho từng biến thể, ta có thể áp dụng grounding và đánh giá kết quả của cả hai prompt. Mục tiếp theo dùng một flow mới để thực hiện đánh giá profile/prompt.

10.7.3 Tạo evaluation flow cho grounding

Mở file `chapter_3\prompt_flow\evaluate_groundings\flow.dag.yaml` trong visual editor, như trong Hình 10.24. Trong evaluation flow (flow đánh giá) này không có block LLM nào, chỉ có các block code Python chạy việc chấm điểm rồi tổng hợp (aggregate) các điểm. Giờ ta xem code của hai block `scoring` và `aggregate`, bắt đầu với code chấm điểm ở Listing 9.3. Code chấm điểm này lấy trung bình điểm của các tiêu chí thành một điểm trung bình. Đầu ra của hàm là một danh sách các recommendation đã được xử lý.

Listing 9.3 line_process.py

```
@tool
def line_process(recommendations: str):    # Một tập ba recommendation được đưa
↪  vào hàm
    inputs = recommendations
    output = []
    for data_dict in inputs:               # Lặp qua từng recommendation và từng
    ↪  tiêu chí
        total_score = 0
        score_count = 0

        for key, value in data_dict.items():     # Lặp qua từng recommendation và
        ↪  từng tiêu chí
                if key != "title":    # Title không phải là tiêu chí nên bỏ qua
                    try:
                        total_score += float(value)    # Cộng dồn điểm của mọi
                        ↪  tiêu chí
                        score_count += 1
                        data_dict[key] = float(value)    # và gán giá trị float
                        ↪  cho key
                    except:
                        pass

        avg_score = total_score / score_count if score_count > 0 else 0

        data_dict["avg_score"] = round(avg_score, 2)   # Thêm điểm trung bình làm
        ↪  điểm grounding của recommendation
        output.append(data_dict)

    return output
```



Hình 10.24: Xem flow evaluate_groundings dùng để grounding các lượt chạy recommendation

Từ các recommendation đã được grounding, ta chuyển sang tổng hợp điểm bằng block `aggregate`. Code của block `aggregate` nằm trong listing sau.

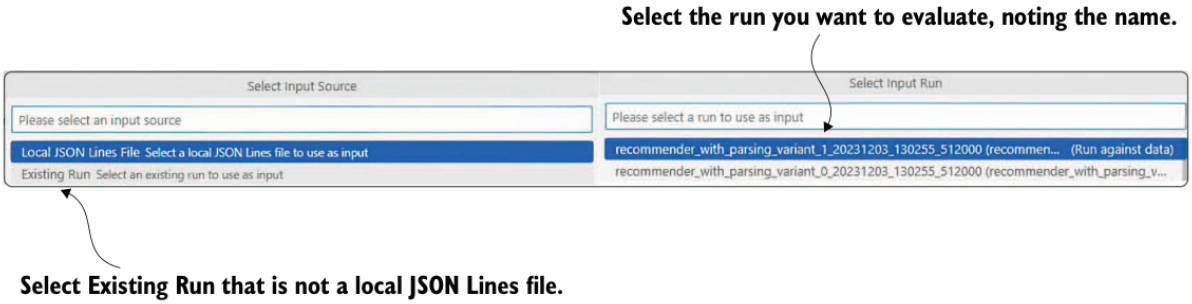
Listing 9.4 aggregate.py

```
@tool
def aggregate(processed_results: List[str]):
    items = [item for sublist in processed_results      # Đầu vào là danh sách
    ↪  các danh sách; làm phẳng thành danh sách các item
            for item in sublist]

    aggregated = {}

    for item in items:
        for key, value in item.items():
            if key == 'title':
                continue

            if isinstance(value, (float, int)):     # Kiểm tra giá trị có phải số
            ↪  không và cộng dồn điểm cho từng key tiêu chí
                if key in aggregated:
                    aggregated[key] += value
                else:
                    aggregated[key] = value

    for key, value in aggregated.items():     # Lặp qua các điểm tiêu chí đã được
    ↪  cộng dồn
        value = value / len(items)
        log_metric(key=key, value=value)    # Ghi log tiêu chí như một metric
        aggregated[key] = value
```

```
    return aggregated
```

Kết quả của việc tổng hợp sẽ là một điểm tóm tắt cho từng tiêu chí và điểm trung bình. Vì evaluation/grounding flow là một flow riêng biệt, nó có thể chạy trên bất kỳ lượt recommendation nào ta thực hiện. Điều này cho phép ta dùng kết quả batch run của bất kỳ biến thể nào để so sánh kết quả.

Ta chạy grounding flow bằng cách mở `flow.dag.yaml` trong visual editor và bấm Batch (biểu tượng ống nghiệm). Sau đó, khi được hỏi, ta chọn một lượt chạy có sẵn (existing run), rồi chọn lượt chạy cần đánh giá, như trong Hình 10.25. Thao tác này mở một file YAML với liên kết Run ở cuối, như ta đã thấy trước đó. Bấm liên kết Run để chạy đánh giá.

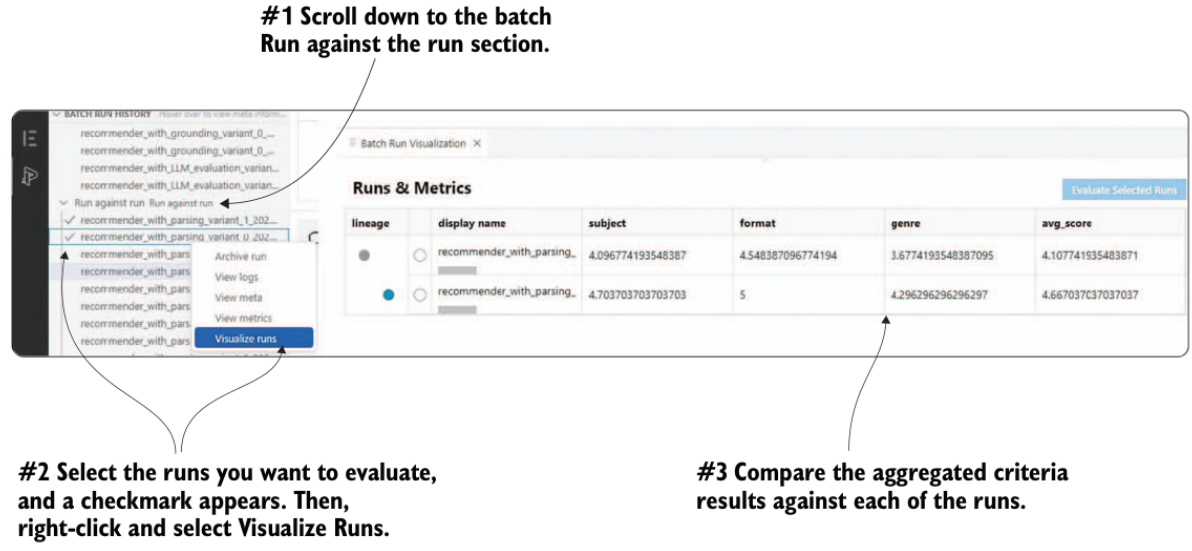


Hình 10.25: Nạp một lượt chạy trước đó để grounding và đánh giá

Sau khi lượt chạy hoàn tất, bạn sẽ thấy bản tóm tắt kết quả trong cửa sổ terminal. Bạn có thể bấm liên kết output để mở thư mục trong VS Code và phân tích kết quả, nhưng có cách tốt hơn để so sánh chúng.

Mở extension prompt flow, chuyển tới cửa sổ Batch Run History, rồi cuộn xuống phần Run against Run, như trong Hình 10.26. Chọn các lượt chạy muốn so sánh, thường là các lượt gần đầu danh sách, sao cho dấu tích xuất hiện. Sau đó bấm chuột phải vào lượt chạy và chọn tuỳ chọn Visualize Runs. Cửa sổ Batch Run Visualization mở ra, và bạn sẽ thấy các metric của từng lượt chạy ở phía trên.

Giờ ta có thể thấy sự khác biệt đáng kể giữa biến thể profile/prompt 0 (user prompt) và biến thể 1 (system prompt). Xem lại Hình 10.15 nếu cần nhớ lại các prompt/profile trông như thế nào. Đến đây, có thể thấy rõ rằng đưa các tham số đầu vào vào system prompt cho các gợi ý tốt hơn.



Hình 10.26: Hình hoá các metric của nhiều lượt chạy và so sánh chúng

Giờ bạn có thể quay lại thử các profile khác hoặc các tuỳ chọn biến thể khác để xem chúng ảnh hưởng thế nào đến các gợi ý. Khả năng gần như vô tận, và hy vọng bạn thấy prompt flow là một công cụ tuyệt vời để xây dựng agent profile và prompt.

10.7.4 Bài tập

Hãy dùng các bài tập sau để củng cố kiến thức về nội dung đã học:

Bài tập 1: Tạo một prompt variant mới cho recommender flow (Trung cấp)

Mục tiêu. Cải thiện kết quả gợi ý bằng cách tạo và kiểm thử một prompt variant mới trong prompt flow.

Nhiệm vụ.

- Tạo một prompt variant mới cho recommender flow trong prompt flow.
- Chạy flow ở chế độ batch.
- Đánh giá kết quả để xác định tốt hơn hay tệ hơn so với prompt gốc.

Bài tập 2: Thêm một trường tuỳ chỉnh vào rubric và đánh giá (Trung cấp)

Mục tiêu. Nâng cao tiêu chí đánh giá bằng cách đưa một trường tuỳ chỉnh (custom field) vào rubric và cập nhật evaluation flow.

Nhiệm vụ.

- Thêm trường tuỳ chỉnh làm tiêu chí mới của rubric.
- Cập nhật evaluation flow để chấm điểm tiêu chí mới.
- Đánh giá kết quả và phân tích ảnh hưởng của tiêu chí mới lên việc đánh giá.

Bài tập 3: Phát triển một use case mới và rubric đánh giá (Nâng cao)

Mục tiêu. Mở rộng ứng dụng của prompt engineering bằng cách phát triển một use case mới và tạo rubric đánh giá.

Nhiệm vụ.

- Phát triển một use case mới ngoài bài toán gợi ý (recommendation).
- Xây dựng prompt cho use case mới.
- Tạo rubric để đánh giá prompt mới.
- Cập nhật hoặc thay đổi evaluation flow để tổng hợp và so sánh kết quả của use case mới với các use case hiện có.

Bài tập 4: Đánh giá các LLM khác bằng LM Studio (Trung cấp)

Mục tiêu. Đánh giá hiệu năng của các LLM mã nguồn mở khác nhau bằng cách host một server cục bộ với LM Studio.

Nhiệm vụ.

- Dùng LM Studio để host một server cục bộ phục vụ việc đánh giá các LLM.
- Đánh giá các LLM mã nguồn mở khác.
- Xem Chương 3 nếu cần trợ giúp thiết lập server và thực hiện đánh giá.

Bài tập 5: Xây dựng và đánh giá prompt bằng prompt flow (Trung cấp)

Mục tiêu. Áp dụng các chiến lược prompt engineering để xây dựng và đánh giá prompt hoặc profile mới bằng prompt flow.

Nhiệm vụ.

- Xây dựng các prompt hoặc profile mới để đánh giá bằng prompt flow.
- Áp dụng chiến lược prompt engineering Write Clear Instructions từ Chương 3.
- Đánh giá các prompt và profile bằng prompt flow.
- Xem Chương 3 để biết các chiến thuật và chi tiết hiện thực nếu cần ôn lại.

Tóm tắt

- Một agent profile gồm nhiều prompt thành phần khác, có thể điều khiển các chức năng như action/tool, knowledge, memory, evaluation, reasoning, feedback và planning.
- Prompt flow có thể dùng để đánh giá các prompt thành phần của agent.
- Prompt engineering có hệ thống (systemic prompt engineering) là một quá trình lặp, đánh giá một prompt và agent profile.
- Chiến lược Test Changes Systematically mô tả việc lặp và đánh giá prompt, còn system prompt engineering hiện thực chiến lược này.
- Agent profile và prompt engineering có nhiều điểm tương đồng. Ta định nghĩa agent profile là sự kết hợp của các thành phần prompt engineering dẫn dắt và giúp agent hoàn thành nhiệm vụ.
- Prompt flow là công cụ mã nguồn mở của Microsoft, cung cấp nhiều tính năng để phát triển và đánh giá profile và prompt.
- Một LLM connection trong prompt flow hỗ trợ các tham số bổ sung, gồm temperature, stop token, max tokens và các tham số nâng cao khác.
- Các block LLM hỗ trợ prompt và profile variant (biến thể), cho phép đánh giá các thay đổi đối với prompt/profile hoặc các tham số connection khác.
- Rubric áp dụng cho một LLM prompt là tập tiêu chí và tiêu chuẩn mà một prompt/profile phải đáp ứng để được grounding. Grounding là việc chấm điểm và đánh giá theo một rubric.
- Prompt flow hỗ trợ chạy nhiều biến thể dưới dạng lượt chạy đơn (single run) hoặc batch run.
- Trong prompt flow, một evaluation flow được chạy sau một generative flow để chấm điểm và tổng hợp kết quả. Tuỳ chọn Visualize Runs có thể so sánh các tiêu chí đã tổng hợp từ việc chấm điểm rubric trên nhiều lượt chạy.

Chương 11

Suy luận và đánh giá của agent

Chương này bao quát các nội dung:

- Dùng nhiều kỹ thuật prompt engineering khác nhau để mở rộng chức năng của mô hình ngôn ngữ lớn (LLM).
- Tác động lên LLM bằng các kỹ thuật prompt engineering khơi gợi khả năng suy luận (reasoning).
- Dùng một prompt đánh giá (evaluation prompt) để thu hẹp và xác định lời giải cho một bài toán chưa biết đáp án.

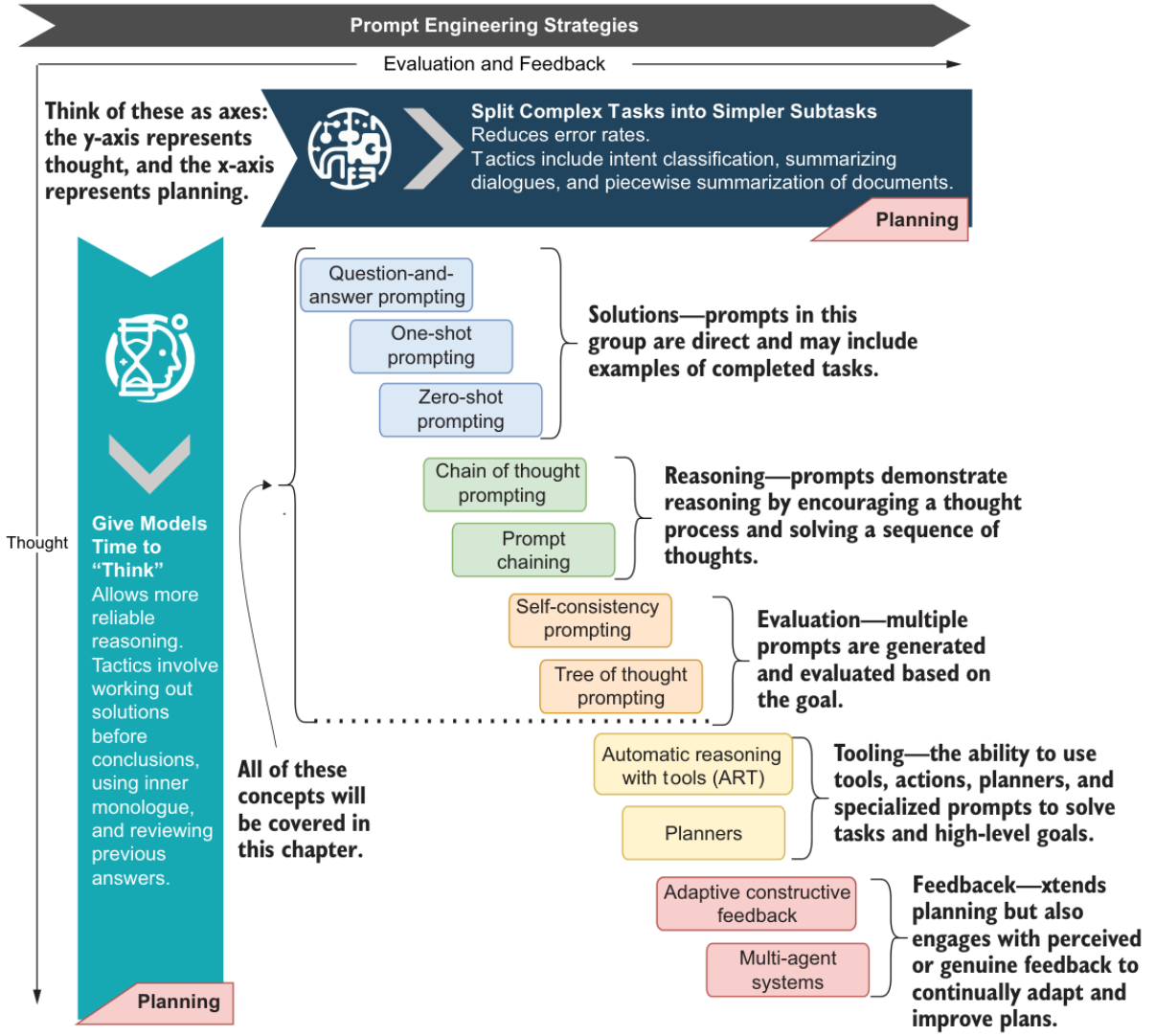
Sau khi đã xem xét các mẫu (pattern) về bộ nhớ và truy xuất tạo nên thành phần bộ nhớ ngữ nghĩa (semantic memory) của agent, giờ ta có thể xem thành phần cuối cùng và quan trọng nhất của agent: **planning** (lập kế hoạch). Planning bao gồm nhiều khía cạnh, từ suy luận, thấu hiểu, đánh giá cho tới phản hồi (feedback).

Để khám phá cách prompt LLM để nó suy luận, thấu hiểu và lập kế hoạch, ta sẽ minh hoạ cách khơi gợi suy luận bằng prompt engineering, rồi mở rộng sang planning. Giải pháp planning do Semantic Kernel (SK) cung cấp bao gồm nhiều dạng planning. Cuối chương, ta đưa phản hồi thích ứng (adaptive feedback) vào một planner mới.

Hình 11.1 trình bày các chiến lược prompt engineering cấp cao sẽ được đề cập trong chương này và mối liên hệ của chúng với các kỹ thuật cụ thể. Mỗi phương pháp trong hình đều sẽ được khám phá trong chương, từ những điều cơ bản của solution/direct prompting (prompt giải trực tiếp) ở góc trên bên trái, tới self-consistency (tự nhất quán) và tree of thought (ToT, cây suy nghĩ) ở góc dưới bên phải.

11.1 Hiểu về direct solution prompting

Direct solution prompting (prompt giải trực tiếp) thường là dạng prompt engineering đầu tiên người dùng áp dụng khi đặt câu hỏi cho LLM hoặc giải một bài toán cụ thể. Với bất kỳ ai đã dùng LLM, các kỹ thuật này có vẻ hiển nhiên, nhưng vẫn đáng ôn lại để đặt nền móng cho tư duy và planning. Trong mục tiếp theo, ta bắt đầu từ điểm khởi đầu: đặt câu hỏi và chờ câu trả lời.



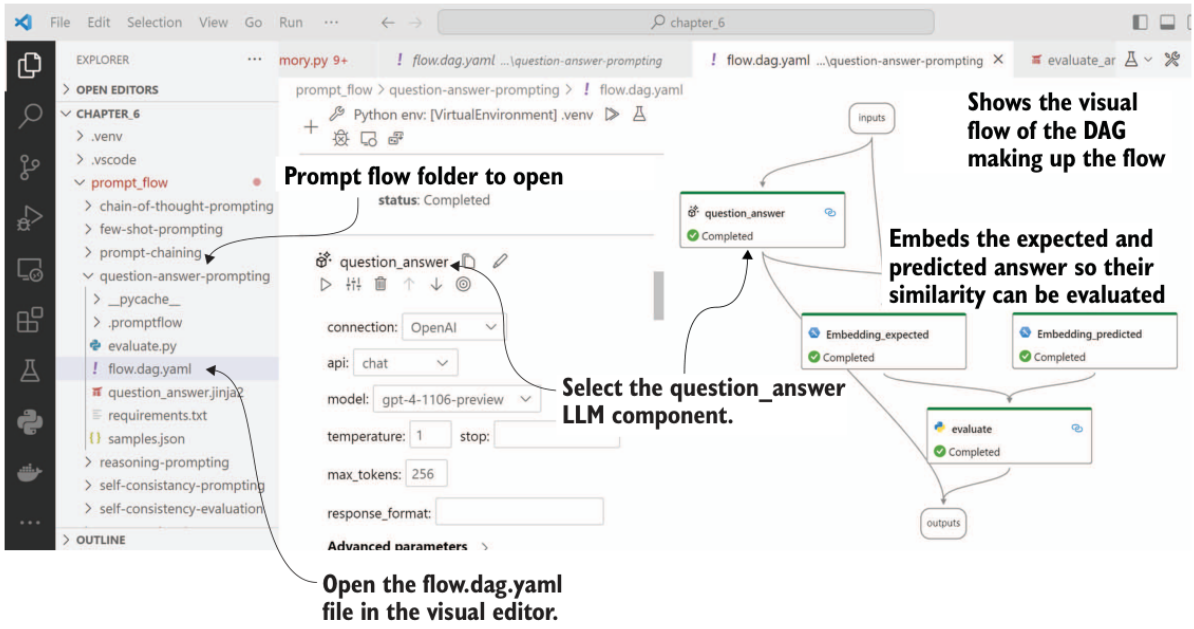
Hình 11.1: Cách hai chiến lược prompt engineering cho planning tương ứng với các kỹ thuật khác nhau

11.1.1 Prompt hỏi–đáp

Trong các bài thực hành của chương này, ta dùng prompt flow để xây dựng và đánh giá các kỹ thuật khác nhau. (Công cụ này đã được trình bày kỹ ở chương 9, hãy xem lại chương đó nếu cần.) Prompt flow là công cụ rất tốt để hiểu cách các kỹ thuật này hoạt động và khám phá luồng của quá trình planning và suy luận.

Mở Visual Studio Code (VS Code) tại thư mục mã nguồn `chapter10`. Tạo một môi trường ảo (virtual environment) mới cho thư mục và cài file `requirements.txt`. Nếu cần trợ giúp thiết lập môi trường Python cho một chương, hãy xem phụ lục B.

Ta sẽ xem flow đầu tiên trong thư mục `prompt_flow/question-answering-prompting`. Mở file `flow.dag.yaml` trong visual editor (trình soạn thảo trực quan), như trong Hình 11.2. Bên phải là luồng các thành phần. Trên cùng là LLM prompt `question_answer`, tiếp theo là hai thành phần `Embedding` và cuối cùng là một LLM prompt làm nhiệm vụ đánh giá, tên là `evaluate`.



Hình 11.2: File `flow.dag.yaml` mở trong visual editor, làm nổi bật các thành phần khác nhau của flow

Phần phân tích trong Listing 10.1 trình bày chi tiết hơn cấu trúc và các thành phần của flow bằng một dạng mã giả (pseudocode) rút gọn theo kiểu YAML. Bạn cũng có thể thấy đầu vào, đầu ra của các thành phần và một kết quả mẫu khi chạy flow.

Listing 10.1 question-answer-prompting flow

```
   Inputs:
        context  : the content to ask the question about
        question : question asked specific to the content
        expected : the expected answer

   LLM: Question-Answer (the prompt used to ask the question)
        inputs:
               context and question
        outputs:
               the prediction/answer to the question

   Embeddings: uses an LLM embedding model to create the embedding
representation of the text

     Embedding_predicted: embeds the output of the Question-Answer LLM
     Embedding_expected: embeds the output of the expected answer

   Python: Evaluation (Python code to measure embedding similarity)
     Inputs:
            Embedding_predicted output
            Embedding_expected output
     Outputs:
            the similarity score between predicted and expected

   Outputs:
        context: -> input.context
        question: -> input.question
     expected: -> input.expected
     predicted: -> output.question_answer
```

```
    evaluation_score: output.evaluation

### Example Output
{
    "context": "Back to the Future (1985)…",
    "evaluation_score": 0.9567478002354606,
    "expected": "Marty traveled back in time 30 years.",
    "predicted": "Marty traveled back in time 30 years from 1985 to 1955
in the movie \"Back to the Future.\"",
    "question": "How far did Marty travel back in time in the movie
Back to the Future (1985)"
}
```

Trước khi chạy flow này, hãy chắc chắn khối LLM đã được cấu hình đúng. Việc này có thể đòi hỏi thiết lập một connection (kết nối) tới LLM bạn chọn. Một lần nữa, xem lại chương 9 nếu cần ôn cách làm. Bạn cần cấu hình khối LLM và các khối `Embedding` với connection của mình nếu không dùng OpenAI.

Sau khi cấu hình connection tới LLM, chạy flow bằng cách bấm nút Play trong visual editor hoặc dùng liên kết Test (Shift-F5) trong cửa sổ YAML editor. Nếu mọi thứ được kết nối và cấu hình đúng, bạn sẽ thấy kết quả giống như trong Listing 10.1.

Mở file `question_answer.jinja2` trong VS Code, như trong Listing 10.2. Listing này cho thấy prompt cơ bản theo kiểu hỏi–đáp. Ở kiểu prompt này, system message mô tả các quy tắc cơ bản và cung cấp ngữ cảnh (context) để trả lời câu hỏi. Ở chương 4, ta đã khám phá mẫu retrieval augmented generation (RAG), và prompt này theo một mẫu tương tự.

Listing 10.2 question_answer.jinja2

```
system:
Answer the users question based on the context below. Keep the answer
short and concise. Respond "Unsure about answer" if not sure about the
answer.

Context: {{context}}

user:
Question: {{question}}
```

Chú thích cho Listing 10.2:

- `Context: {{context}}`: thay bằng nội dung mà LLM cần dựa vào để trả lời câu hỏi.
- `Question: {{question}}`: thay bằng câu hỏi.

Bài thực hành này cho thấy phương pháp đơn giản: dùng LLM để đặt câu hỏi về một đoạn nội dung. Sau đó, câu trả lời được đánh giá bằng một điểm so khớp độ tương đồng (similarity matching score). Từ kết quả trong Listing 10.1, ta thấy LLM trả lời khá tốt câu hỏi về ngữ cảnh. Trong mục tiếp theo, ta khám phá một kỹ thuật tương tự cũng dùng prompt trực tiếp.

11.1.2 Triển khai few-shot prompting

Few-shot prompting (prompt với vài ví dụ) giống prompt hỏi–đáp, nhưng thành phần của prompt thiên về cung cấp vài ví dụ hơn là cung cấp dữ kiện hay ngữ cảnh. Cách này cho phép LLM uốn theo những mẫu hoặc nội dung nó chưa từng thấy. Dù nghe giống hỏi–đáp, cách triển khai khá khác, và kết quả có thể rất mạnh.

Zero-shot, one-shot và few-shot learning

Một trong những "chén thánh" của học máy và AI là khả năng huấn luyện một mô hình với càng ít mẫu càng tốt. Ví dụ, với các mô hình thị giác truyền thống, hàng triệu ảnh được đưa vào mô hình để giúp nó phân biệt mèo với chó.
Mô hình **one-shot** là mô hình chỉ cần một ảnh duy nhất để huấn luyện. Chẳng hạn, cho xem một bức ảnh con mèo, rồi mô hình có thể nhận ra bất kỳ ảnh mèo nào. Mô hình **few-shot** chỉ cần vài mẫu để huấn luyện. Và tất nhiên, **zero-shot** chỉ khả năng nhận ra một thứ mà không cần ví dụ nào trước đó. LLM là những learner (bộ học) hiệu quả và có thể thực hiện cả ba kiểu học này.

Mở `prompt_flow/few-shot-prompting/flow.dag.yaml` trong VS Code và visual editor. Phần lớn flow trông giống flow ở Hình 11.2, còn các điểm khác biệt được làm nổi bật trong Listing 10.3, ở dạng mã giả YAML. Khác biệt chính giữa flow này và flow trước nằm ở đầu vào và LLM prompt.

Listing 10.3 few-shot-prompting flow

```
   Inputs:
       statement  : introduces the context and then asks for output
       expected : the expected answer to the statement

   LLM: few_shot (the prompt used to ask the question)
       inputs:statement
       outputs: the prediction/answer to the statement

   Embeddings: uses an LLM embedding model to create the embedding
representation of the text

        Embedding_predicted: embeds the output of the few_shot LLM
        Embedding_expected: embeds the output of the expected answer

   Python: Evaluation (Python code to measure embedding similarity)
        Inputs:
               Embedding_predicted output
               Embedding_expected output
        Outputs: the similarity score between predicted and expected

Outputs:
        statement: -> input.statement
        expected: -> input.expected
        predicted: -> output.few_shot
        evaluation_score: output.evaluation

### Example Output
{
    "evaluation_score": 0.906647282920417,    // Điểm đánh giá: độ tương đồng
    ↪  giữa expected và predicted
    "expected": "We ate sunner and watched the setting sun.",
    "predicted": "After a long hike, we sat by the lake
and enjoyed a peaceful sunner as the sky turned
brilliant shades of orange and pink.",    // Dùng từ sunner trong một câu
    "statement": "A sunner is a meal we eat in Cananda
at sunset, please use the word in a sentence"    // Đây là phát biểu sai, nhưng
↪  mục đích là khiến LLM dùng từ này như thể nó có thật
}
```

Chạy flow bằng Shift-F5 hoặc bấm nút Play/Test trong visual editor. Bạn sẽ thấy kết quả giống Listing 10.3, trong đó LLM đã dùng đúng từ *sunner* (một từ bịa ra) trong một câu, dựa trên phát biểu ban đầu.

Bài thực hành này cho thấy khả năng dùng prompt để thay đổi hành vi của LLM theo hướng trái với những gì nó đã học. Ta đang thay đổi điều mà LLM hiểu là đúng. Hơn nữa, ta dùng góc nhìn đã bị thay đổi đó để khiến nó sử dụng một từ bịa ra.

Mở prompt `few_shot.jinja2` trong VS Code, như trong Listing 10.4. Listing này minh hoạ việc dựng một persona (vai) đơn giản, một người soạn từ điển lập dị, rồi đưa ra các ví dụ về những từ mà người này đã định nghĩa và dùng trước đó. Phần nền của prompt cho phép LLM mở rộng các ví dụ và tạo ra kết quả tương tự với những từ khác.

Listing 10.4 few_shot.jinja2

```
system:
You are an eccentric word dictionary maker. You will be asked to
construct a sentence using the word.
The following are examples that demonstrate how to craft a sentence using
the word.
A "whatpu" is a small, furry animal native to Tanzania.
An example of a sentence that uses the word whatpu is:
We were traveling in Africa and we saw these very cute whatpus.
To do a "farduddle" means to jump up and down really fast. An example of a
sentence that uses the word farduddle is:
I was so excited that I started to farduddle.

Please only return the sentence requested by the user.

user:
{{statement}}
```

Chú thích cho Listing 10.4:

- Đoạn về `"whatpu"`: minh hoạ một ví dụ định nghĩa một từ bịa ra và dùng nó trong một câu.
- Đoạn về `"farduddle"`: minh hoạ một ví dụ khác.
- `Please only return the sentence requested by the user.`: một quy tắc ngăn LLM xuất thêm thông tin thừa.
- `{{statement}}`: phát biểu đầu vào định nghĩa một từ mới và yêu cầu dùng từ đó.

Bạn có thể nói rằng ở đây ta đang ép LLM bịa đặt (hallucinate), nhưng kỹ thuật này là nền tảng để thay đổi hành vi. Nó cho phép xây dựng prompt dẫn dắt LLM làm mọi thứ trái với những gì nó đã học. Nền tảng prompt này cũng đặt cơ sở cho các kỹ thuật thay đổi hành vi theo những dạng khác. Từ khả năng thay đổi nhận thức và bối cảnh của LLM, trong mục tiếp theo ta chuyển sang ví dụ cuối cùng về direct solution.

11.1.3 Rút ra tính khái quát với zero-shot prompting

Zero-shot prompting (hay zero-shot learning) là khả năng tạo một prompt sao cho LLM có thể khái quát hoá (generalize). Khả năng khái quát hoá này nằm sẵn bên trong LLM và được thể hiện qua zero-shot prompting: không đưa ví dụ nào, mà thay vào đó đưa một tập hướng dẫn hoặc quy tắc để dẫn dắt LLM.

Kỹ thuật này đơn giản và hiệu quả trong việc dẫn LLM tạo câu trả lời dựa trên kiến thức nội tại, không cần ngữ cảnh nào khác. Đây là một kỹ thuật tinh tế nhưng mạnh, áp dụng kiến thức của LLM cho các ứng dụng khác. Kết hợp với các chiến lược prompt khác, kỹ thuật này đang tỏ ra hiệu quả trong việc thay thế các mô hình phân loại ngôn ngữ khác, chẳng hạn mô hình nhận diện cảm xúc hay sắc thái (sentiment) trong văn bản.

Mở `prompt_flow/zero-shot-prompting/flow.dag.yaml` trong prompt flow visual editor của VS Code. Flow này một lần nữa gần như giống hệt flow đã thấy ở Hình 11.1 nhưng hơi khác về cách triển khai, như trong Listing 10.5.

Listing 10.5 zero-shot-prompting flow

```
   Inputs:
        statement  : the statement to be classified
        expected : the expected classification of the statement

    LLM: zero_shot (the prompt used to classify)
        inputs: statement
        outputs: the predicted class given the statement

    Embeddings: uses an LLM embedding model to create the embedding
representation of the text

    Embedding_predicted: embeds the output of the zero_shot LLM
    Embedding_expected: embeds the output of the expected answer

    Python: Evaluation (Python code to measure embedding similarity)
        Inputs:
               Embedding_predicted output
            Embedding_expected output
          Outputs: the similarity score between predicted and expected

   Outputs:
        statement: -> input.statement
        expected: -> input.expected
        predicted: -> output.few_shot
        evaluation_score: output.evaluation

   ### Example Output
{
       "evaluation_score": 1,    // Điểm đánh giá hoàn hảo: 1.0
       "expected": "neutral",
       "predicted": "neutral",
       "statement": "I think the vacation is okay. "    // Phát biểu mà ta yêu
       ↪  cầu LLM phân loại
   }
```

Chạy flow bằng Shift-F5 trong prompt flow visual editor của VS Code. Bạn sẽ thấy kết quả tương tự Listing 10.5.

Giờ hãy mở prompt `zero_shot.jinja2` như trong Listing 10.6. Prompt này đơn giản và không dùng ví dụ nào để rút ra sắc thái của văn bản. Điều đặc biệt thú vị là prompt thậm chí không nhắc tới từ *sentiment*, vậy mà LLM dường như vẫn hiểu ý định.

Listing 10.6 zero_shot.jinja2

```
system:
Classify the text into neutral, negative or positive.
Return on the result and nothing else.

user:
{{statement}}
```

Chú thích cho Listing 10.6:

- Hai dòng của system message: đưa ra hướng dẫn cốt yếu để thực hiện việc phân loại.
- `{{statement}}`: đoạn văn bản cần phân loại.

Zero-shot prompt engineering là việc tận dụng khả năng khái quát hoá rộng của LLM dựa trên dữ liệu huấn luyện. Bài thực hành này cho thấy kiến thức bên trong LLM có thể được đưa vào làm những tác vụ khác. Khả năng tự đặt ngữ cảnh (self-contextualize) và áp dụng kiến thức của LLM có thể vươn ra ngoài phạm vi huấn luyện. Trong mục tiếp theo, ta mở rộng khái niệm này hơn nữa bằng cách xem LLM có thể suy luận như thế nào.

11.2 Suy luận trong prompt engineering

Các LLM như ChatGPT được phát triển để hoạt động như mô hình chat completion: nội dung văn bản được đưa vào mô hình, và phản hồi của mô hình nhằm hoàn thành yêu cầu đó. LLM chưa bao giờ được huấn luyện để suy luận, lập kế hoạch, tư duy hay có suy nghĩ.

Tuy nhiên, giống như các ví dụ ở mục trước, LLM có thể được prompt để rút ra tính khái quát của nó và được mở rộng vượt ra ngoài thiết kế ban đầu. Dù LLM không được thiết kế để suy luận, dữ liệu huấn luyện đưa vào mô hình mang theo hiểu biết về suy luận, lập kế hoạch và tư duy. Vì vậy, theo lẽ đó, LLM hiểu suy luận là gì và có thể vận dụng khái niệm suy luận.

Suy luận và lập kế hoạch

Reasoning (suy luận) là khả năng của một trí tuệ, nhân tạo hay không, hiểu được quá trình tư duy hoặc quá trình suy nghĩ thấu đáo một vấn đề. Một trí tuệ có thể hiểu rằng hành động dẫn tới kết quả, và có thể dùng khả năng này để suy luận xem hành động nào trong một tập hành động có thể áp dụng để giải một tác vụ cho trước.
Planning (lập kế hoạch) là khả năng của trí tuệ suy luận ra thứ tự các hành động hoặc tác vụ và áp dụng đúng tham số để đạt mục tiêu hoặc kết quả. Mức độ lập kế hoạch của một trí tuệ phụ thuộc vào phạm vi của bài toán. Một trí tuệ có thể kết hợp nhiều cấp độ planning, từ chiến lược (strategic), chiến thuật (tactical) tới vận hành (operational) và dự phòng (contingent).

Ta sẽ xem một tập kỹ thuật prompt engineering khác, cho phép hoặc mô phỏng hành vi suy luận, để minh hoạ khả năng này. Thông thường, khi đánh giá việc áp dụng suy luận, ta cho LLM giải những bài toán khó mà nó không được thiết kế để giải. Một nguồn tốt cho loại bài này là các bài toán logic, toán học và bài toán đố bằng lời (word problem).

Với chủ đề du hành thời gian (time travel), còn lớp bài toán độc đáo nào hợp hơn việc hiểu về du hành thời gian? Hình 11.3 mô tả một ví dụ về bài toán du hành thời gian đặc biệt khó. Mục tiêu của ta là có được khả năng prompt LLM theo cách cho phép nó giải đúng bài toán.

Bài toán du hành thời gian là những bài tập tư duy có thể khó giải hơn vẻ ngoài. Ví dụ trong Hình 11.3 rất khó với LLM, nhưng chỗ nó giải sai có thể khiến bạn ngạc nhiên. Mục tiếp theo sẽ dùng suy luận trong prompt để giải những bài toán đặc biệt này.

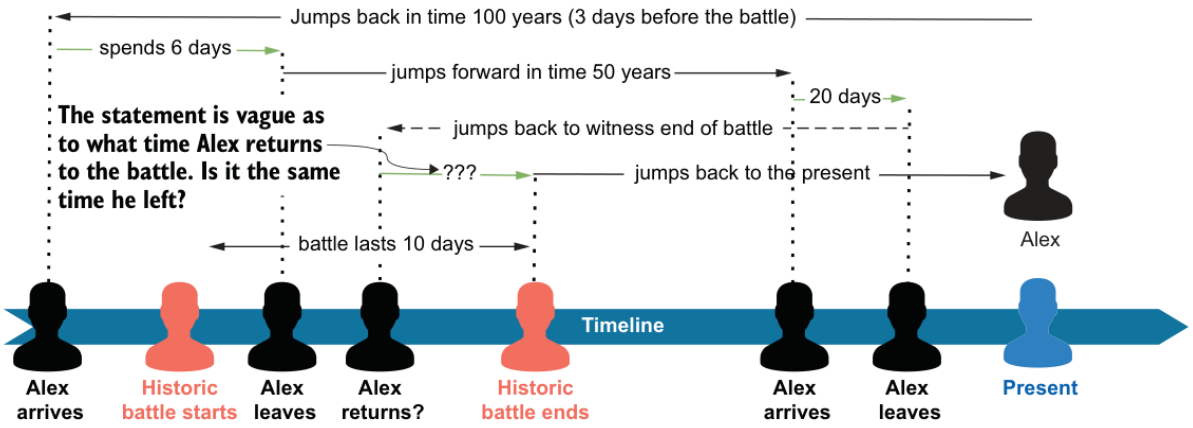
11.2.1 Chain of thought prompting

Chain of thought (CoT, chuỗi suy nghĩ) prompting là một kỹ thuật prompt engineering dùng các ví dụ one-shot hoặc few-shot mô tả quá trình suy luận và các bước để đạt một mục tiêu mong muốn. Qua việc được minh hoạ cách suy luận, LLM có thể khái quát hoá nguyên tắc này và suy luận qua những bài toán, mục tiêu tương tự. Dù LLM không được huấn luyện với mục tiêu suy luận, ta có thể khơi gợi mô hình suy luận bằng prompt engineering.

Bài toán được nêu trong Hình 11.3 như sau:

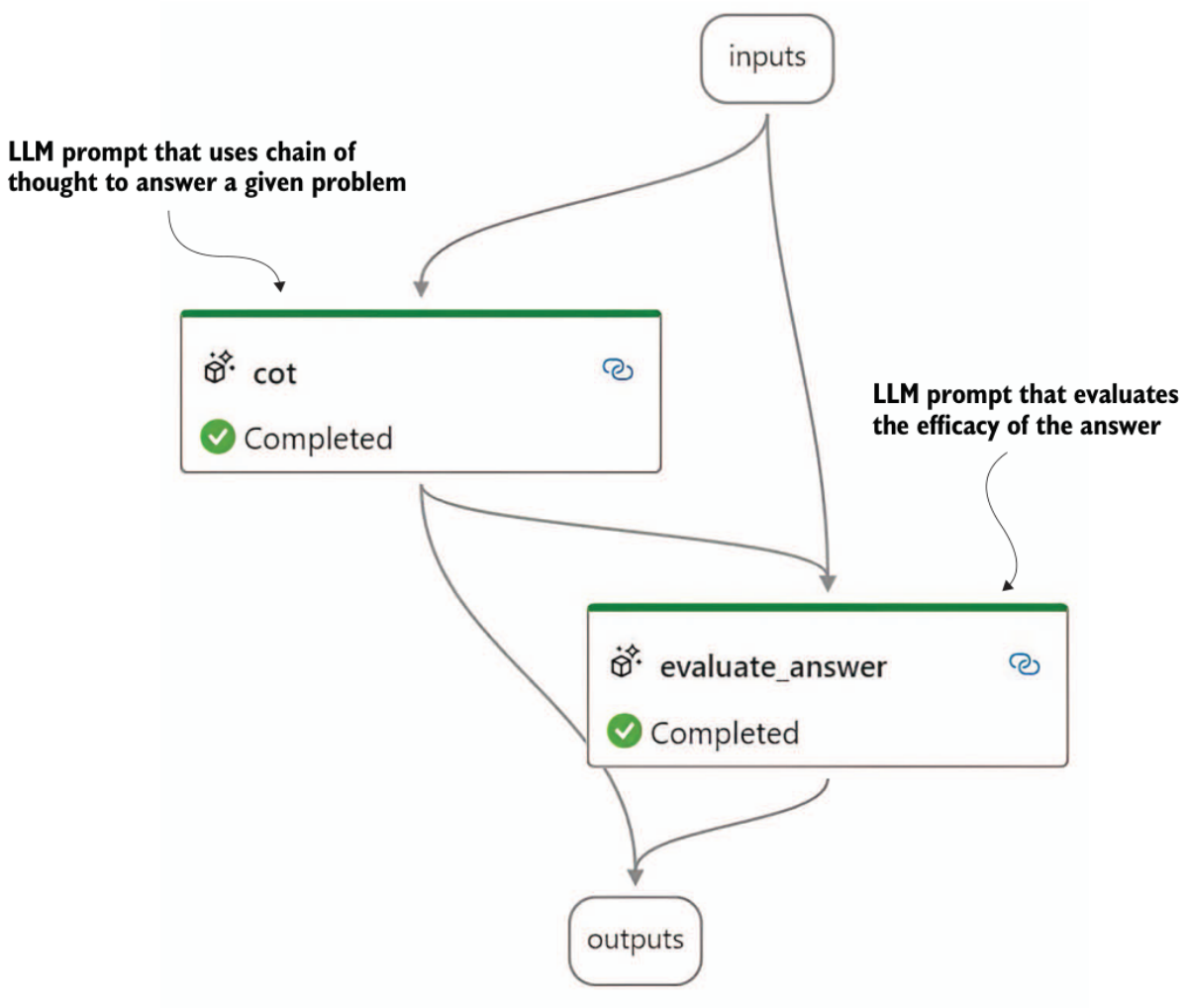
```
  In a sci-fi film, Alex is a time traveler who decides to go back in
time to witness a famous historical battle that took place 100 years ago,
which lasted for 10 days. He arrives three days before the battle starts.
However, after spending six days in the past, he jumps forward in time by
50 years and stays there for 20 days. Then, he travels back to witness the
```

```
end of the battle. How many days does Alex spend in the past before he
                        sees the end of the battle?
```



Hình 11.3: Độ phức tạp của các bài toán du hành thời gian mà ta định giải bằng LLM có suy luận và lập kế hoạch

Mở `prompt_flow/chain-of-thought-prompting/flow.dag.yaml` trong prompt flow visual editor của VS Code. Các thành phần của flow này đơn giản, như trong Hình 11.4. Chỉ với hai khối LLM, flow trước tiên dùng một CoT prompt để giải một câu hỏi phức tạp, sau đó LLM prompt thứ hai đánh giá câu trả lời.



Hình 11.4: Luồng của CoT

Listing 10.7 trình bày mã giả YAML mô tả chi tiết hơn các khối và đầu vào/đầu ra của flow. Bài toán mặc định trong ví dụ này không giống bài toán ở Hình 11.3.

Listing 10.7 chain-of-thought-prompting flow

```
Inputs:
     statement  : the statement problem to be solved
     expected : the expected solution to the problem

LLM: cot (the prompt used to solve the problem)
     inputs: statement
     outputs: the predicted answer given the problem statement

LLM: evaluate_answer (the prompt used to evaluate the solution)
     inputs:
            statement: -> input.statement
            expected: -> input.expected
            predicted: -> output.cot

     outputs: a score of how well the problem was answered

Outputs:
```

```
        statement: -> input.statement
        expected: -> input.expected
        predicted: -> output.cot
        evaluation_score: output.evaluate_answer

   ### Example Output
{
    "evaluation_score": "0.5",    // Điểm đánh giá cho lời giải đã cho
    "expected": "After the final jump, Max finds himself
in the year 1980 and he is 75 years old.",    // Đáp án mong đợi của bài toán
    "predicted": " Max starts in the year 2300 and
travels backward in 40-year increments, spending 5 years
in each period. The journeys will be as follows:
\n\n- From 2300 to 2260: Max is 25 + 5 = 30 years old.
\n- From 2260 to 2220: Max is 30 + 5 = 35 years old.
\n- From 2220 to 2180: Max is 35 + 5 = 40 years old.
\n- From 2180 to 2140: Max is 40 + 5 = 45 years old.
\n- From 2140 to 2100: Max is 45 + 5 = 50 years old.
\n- From 2100 to 2060: Max is 50 + 5 = 55 years old.
\n- From 2060 to 2020: Max is 55 + 5 = 60 years old.
\n- From 2020 to 1980: Max is 60 + 5 = 65 years old.
\n- From 1980 to 1940: Max is 65 + 5 = 70 years old.
\n- From 1940 to 1900: Max is 70 + 5"    // Câu trả lời dự đoán cho thấy các bước
↪  suy luận và kết quả
}
```

Hãy xem kỹ các đầu vào và đọc bài toán; thử tự giải bài toán. Sau đó chạy flow bằng Shift-F5. Bạn sẽ thấy kết quả tương tự Listing 10.7.

Mở file prompt `cot.jinja2` như trong Listing 10.8. Prompt này đưa ra vài ví dụ bài toán du hành thời gian, kèm lời giải đã được suy nghĩ và lập luận kỹ. Việc cho LLM thấy các bước để giải bài toán chính là cơ chế tạo ra suy luận.

Listing 10.8 cot.jinja2

```
system:
"In a time travel movie, Sarah travels back in time to
prevent a historic event from happening. She arrives
2 days before the event. After spending a day preparing,
she attempts to change the event but realizes she has
actually arrived 2 years early, not 2 days. She then
decides to wait and live in the past until the event's
original date. How many days does Sarah spend in the past
before the day of the event?"

Chain of Thought:

    Initial Assumption: Sarah thinks she has arrived 2 days before the event.
    Time Spent on Preparation: 1 day spent preparing.
    Realization of Error: Sarah realizes she's actually 2 years early.
    Conversion of Years to Days:
2 years = 2 × 365 = 730 days (assuming non-leap years).
    Adjust for the Day Spent Preparing: 730 - 1 = 729 days.
    Conclusion: Sarah spends 729 days in the past before the day of the event.

"In a sci-fi film, Alex is a time traveler who decides
to go back in time to witness a famous historical battle
that took place 100 years ago, which lasted for 10 days.
He arrives three days before the battle starts. However,
```

```
after spending six days in the past, he jumps forward in
time by 50 years and stays there for 20 days. Then, he
travels back to witness the end of the battle. How many
days does Alex spend in the past before he sees the end of
 the battle?"

Chain of Thought:

    Initial Travel: Alex arrives three days before the battle starts.
    Time Spent Before Time Jump: Alex spends six days in the past.
The battle has started and has been going on for 3 days (since he
arrived 3 days early and has now spent 6 days, 3 + 3 = 6).
    First Time Jump: Alex jumps 50 years forward and stays for 20 days.
 This adds 20 days to the 6 days he's already spent in the past
(6 + 20 = 26).
    Return to the Battle: When Alex returns, he arrives back on the same
day he left (as per time travel logic). The battle has been going on for
3 days now.
    Waiting for the Battle to End: The battle lasts 10 days. Since he's
already witnessed 3 days of it, he needs to wait for 7 more days.
    Conclusion: Alex spends a total of 3 (initial wait) + 3 (before the
first jump) + 20 (50 years ago) + 7 (after returning) = 33 days in the
past before he sees the end of the battle.
Think step by step but only show the final answer to the statement.

user:
{{statement}}
```

Chú thích cho Listing 10.8:

- Hai đoạn nằm trong dấu ngoặc kép (bài toán của Sarah và bài toán của Alex): các bài toán ví dụ mẫu (few-shot).
- Hai phần `Chain of Thought:`: lời giải cho bài toán đứng ngay trước, được trình bày dưới dạng một chuỗi các bước suy luận.
- `{{statement}}`: phát biểu bài toán mà LLM được yêu cầu giải.

Bạn có thể nhận thấy lời giải cho bài toán ở Hình 11.3 cũng được đưa vào làm một ví dụ trong Listing 10.8. Cũng nên quay lại xem Listing 10.7 để thấy câu trả lời của LLM cho bài toán đó. Từ đây, bạn có thể thấy các bước suy luận mà LLM đã áp dụng để đi tới đáp án cuối cùng.

Giờ ta xem prompt đánh giá mức độ lời giải giải được bài toán. Mở `evaluate_answer.jinja2`, trình bày trong Listing 10.9, để xem prompt được dùng. Prompt này đơn giản, dùng zero-shot prompting và để LLM tự khái quát cách chấm điểm giữa kết quả mong đợi (expected) và kết quả dự đoán (predicted). Ta cũng có thể đưa ví dụ kèm điểm, qua đó biến nó thành một bài phân loại few-shot.

Listing 10.9 evaluate_answer.jinja2

```
system:

Please confirm that expected and predicted results are
the same for the given problem.
Return a score from 0 to 1 where 1 is a perfect match and 0 is no match.
Please just return the score and not the explanation.

user:
Problem: {{problem}}
```

```
Expected result: {{expected}}

Predicted result: {{predicted}}
```

Chú thích cho Listing 10.9:

- `Please confirm that expected and predicted results are the same ...`: các quy tắc để đánh giá lời giải.
- `Please just return the score and not the explanation.`: chỉ dẫn chỉ trả về điểm và không gì khác.
- `Problem: {{problem}}`: phát biểu bài toán ban đầu.
- `Expected result: {{expected}}`: đáp án mong đợi, hay đáp án chuẩn (grounded answer).
- `Predicted result: {{predicted}}`: đầu ra từ prompt CoT ở trên.

Nhìn vào đầu ra của LLM trong Listing 10.7 ở trên, bạn có thể thấy vì sao bước đánh giá có thể gây nhầm lẫn. Một cách khắc phục có thể là gợi ý LLM đưa câu trả lời cuối cùng trong một câu duy nhất. Trong mục tiếp theo, ta chuyển sang một ví dụ khác về suy luận bằng prompt.

11.2.2 Zero-shot CoT prompting

Như ví dụ du hành thời gian cho thấy, CoT prompting có thể tốn kém ở khâu tạo prompt cho một lớp bài toán cụ thể. Có những kỹ thuật tương tự CoT, tuy kém hiệu quả hơn nhưng không dùng ví dụ và có thể khái quát hơn. Mục này xem xét một cụm từ đơn giản dùng để khơi gợi suy luận trong LLM.

Mở `prompt_flow/zero-shot-cot-prompting/flow.dag.yaml` trong prompt flow visual editor của VS Code. Flow này rất giống flow CoT trước, như trong Hình 11.4. Listing tiếp theo trình bày mã giả YAML mô tả flow.

Listing 10.10 zero-shot-CoT-prompting flow

```
Inputs:
     statement  : the statement problem to be solved
     expected : the expected solution to the problem

LLM: cot (the prompt used to solve the problem)
     inputs: statement
     outputs: the predicted answer given the problem statement

LLM: evaluate_answer (the prompt used to evaluate the solution)
     inputs:
            statement: -> input.statement
            expected: -> input.expected
            predicted: -> output.cot

      outputs: a score of how well the problem was answered


 Outputs:
     statement: -> input.statement
     expected: -> input.expected
     predicted: -> output.cot
     evaluation_score: output.evaluate_answer
```

```
    ### Example Output
    {
        "evaluation_score": "1",    // Điểm đánh giá cuối cùng
        "expected": "After the final jump,
           Max finds himself in the year 1980 and    he is 75 years old.",    //
            ↪  Đáp án mong đợi
        "predicted": "Max starts in…
           Therefore, after the final jump,
           Max is 75 years old and in the year 1980.",    // Đáp án dự đoán (các
            ↪  bước đã được lược bớt, chỉ còn đáp án cuối)
        "statement": "In a complex time travel …"    // Phát biểu bài toán ban
         ↪  đầu
    }
```

Chạy/kiểm thử flow trong VS Code bằng Shift-F5 khi đang ở visual editor. Flow sẽ chạy và bạn sẽ thấy kết quả tương tự Listing 10.10. Ví dụ này cho kết quả tốt hơn ví dụ trước trên cùng bài toán.

Mở prompt `cot.jinja2` trong VS Code, như trong Listing 10.11. Đây là prompt đơn giản hơn nhiều so với ví dụ trước vì nó chỉ dùng zero-shot. Tuy nhiên, một cụm từ then chốt biến prompt đơn giản này thành một "động cơ suy luận" mạnh. Dòng `Let's think step by step` trong prompt kích thích LLM xét đến ngữ cảnh nội tại thể hiện sự suy luận. Điều này, đến lượt nó, dẫn LLM suy luận bài toán theo từng bước.

Listing 10.11 cot.jinja2

```
system:
You are an expert in solving time travel problems.
You are given a time travel problem and you have to solve it.
Let's think step by step.
Please finalize your answer in a single statement.

user:
{{statement}}
```

Chú thích cho Listing 10.11:

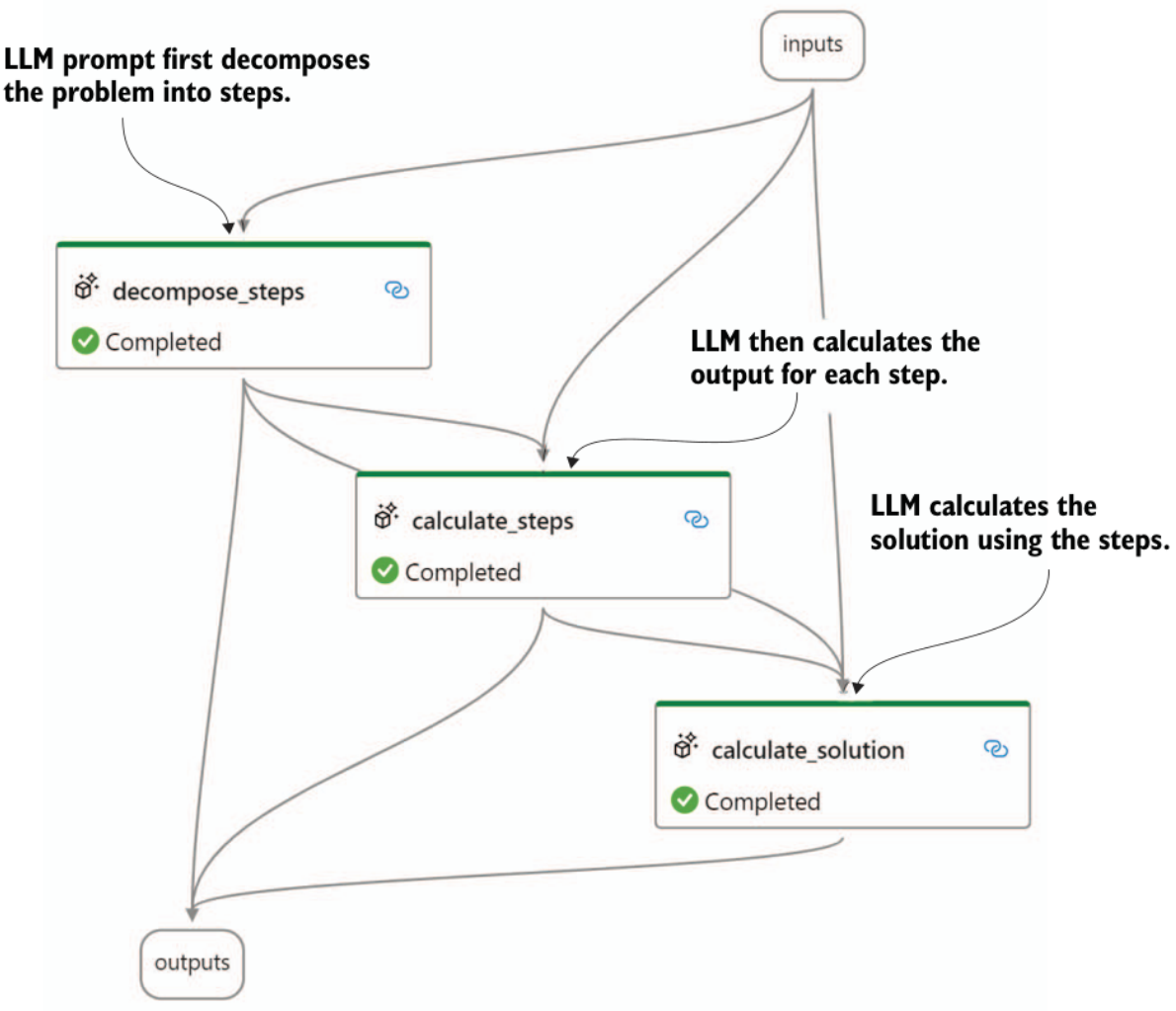
- `Let's think step by step.`: "dòng thần chú" (magic line) hình thành khả năng suy luận từ LLM.
- `Please finalize your answer in a single statement.`: yêu cầu LLM đưa ra câu kết luận cuối cùng cho đáp án.
- `{{statement}}`: phát biểu bài toán mà LLM được yêu cầu giải.

Những cụm từ tương tự, yêu cầu LLM nghĩ về các bước hoặc trả lời theo từng bước, cũng khơi gợi được suy luận. Ở mục tiếp theo, ta minh hoạ một kỹ thuật tương tự nhưng công phu hơn.

11.2.3 Từng bước với prompt chaining

Ta có thể mở rộng hành vi yêu cầu LLM nghĩ từng bước thành một chuỗi các prompt buộc LLM giải bài toán theo từng bước. Trong mục này, ta xem một kỹ thuật gọi là **prompt chaining** (xâu chuỗi prompt), buộc LLM xử lý bài toán theo từng bước.

Mở file `prompt_flow/prompt-chaining/flow.dag.yaml` trong visual editor, như trong Hình 11.5. Prompt chaining chia phương pháp suy luận dùng để giải một bài toán thành các chuỗi prompt. Kỹ thuật này buộc LLM trả lời bài toán theo từng bước.



Hình 11.5: Luồng prompt chaining

Listing 10.12 trình bày mã giả YAML mô tả flow chi tiết hơn một chút. Flow này nối đầu ra của khối LLM thứ nhất vào khối thứ hai, rồi từ khối thứ hai vào khối thứ ba. Việc buộc LLM xử lý bài toán theo cách này làm lộ ra mẫu suy luận, nhưng cũng có thể khiến đầu ra quá dài dòng.

Listing 10.12 prompt-chaining flow

```
Inputs:
     statement  : the statement problem to be solved

LLM: decompose_steps (the prompt used to decompose the problem)
     inputs:
            statement: -> input.statement    # Bắt đầu chuỗi prompt

     outputs: the breakdown of steps to solve the problem

LLM: calculate_steps (the prompt used to calculate the steps)
     inputs:
            statement: -> input.statement
            decompose_steps: -> output.decompose_steps    # Đầu ra của bước
            ↪  trước được đưa vào bước này

            outputs: the calculation for each step
```

```
   LLM: calculate_solution (attempts to solve the problem)
        inputs:
               statement: -> input.statement
               decompose_steps: -> output.decompose_steps    # Đầu ra của hai
                ↪  bước trước được đưa vào bước này
               calculate_steps: -> output.calculate_steps

         outputs: the final solution statement

   Outputs:
        statement: -> input.statement
        decompose_steps: -> output.decompose_steps
        calculate_steps: -> output.calculate_steps
        calculate_solution: -> output.calculate_solution

   ### Example Output
{
    "calculate_steps": "1. The days spent by Alex",
    "decompose_steps": "To figure out the …",
    "solution": "Alex spends 13 days in the
           past before the end of the battle.",    // Câu lời giải cuối cùng,
            ↪  tuy sai nhưng đã gần đúng hơn
    "statement": "In a sci-fi film, Alex …"
}
```

Chạy flow bằng Shift-F5 từ visual editor và bạn sẽ thấy kết quả như Listing 10.12. Đáp án vẫn chưa đúng với bài toán Alex, nhưng ta có thể thấy toàn bộ công việc mà LLM thực hiện để suy luận bài toán.

Mở cả ba prompt: `decompose_steps.jinja2`, `calculate_steps.jinja2` và `calculate_solution.jinja2` (lần lượt là Listing 10.13, 10.14 và 10.15). Có thể so sánh cả ba prompt trong các listing này để thấy các đầu ra được nối chuỗi với nhau như thế nào.

Listing 10.13 decompose_steps.jinja2

```
system:
You are a problem solving AI assistant.
Your job is to break the users problem down into smaller steps and list
the steps in the order you would solve them.
Think step by step, not in generalities.
Do not attempt to solve the problem, just list the steps.

user:
{{statement}}
```

Chú thích cho Listing 10.13:

- `Do not attempt to solve the problem, just list the steps.`: buộc LLM chỉ liệt kê các bước và không làm gì khác.
- `{{statement}}`: phát biểu bài toán ban đầu.

Listing 10.14 calculate_steps.jinja2

```
system:
You are a problem solving AI assistant.
You will be given a list of steps that solve a problem.
Your job is to calculate the output for each of the steps in order.
```

```
Do not attempt to solve the whole problem,
just list output for each of the steps.
Think step by step.

user:
{{statement}}

{{steps}}
```

Chú thích cho Listing 10.14:

- `Do not attempt to solve the whole problem, just list output for each of the steps.`: yêu cầu LLM không giải cả bài toán, chỉ liệt kê đầu ra của từng bước.
- `Think step by step.`: dùng "câu thần chú" để khơi gợi suy luận.
- `{{steps}}`: chèn các bước do khối `decompose_steps` tạo ra.

Listing 10.15 calculate_solution.jinja2

```
system:
You are a problem solving AI assistant.
You will be given a list of steps and the calculated output for each step.
Use the calculated output from each step to determine the final
solution to the problem.
Provide only the final solution to the problem in a
single concise sentence. Do not include any steps
in your answer.

user:
{{statement}}

{{steps}}

{{calculated}}
```

Chú thích cho Listing 10.15:

- `Provide only the final solution ... Do not include any steps in your answer.`: yêu cầu LLM chỉ xuất đáp án cuối cùng và không xuất bất kỳ bước nào.
- `{{steps}}`: các bước đã được phân rã.
- `{{calculated}}`: các bước đã được tính toán.

Trong ví dụ này, ta không thực hiện đánh giá và chấm điểm. Không có đánh giá, ta thấy chuỗi prompt này vẫn gặp khó khi giải bài toán du hành thời gian khó hơn đã nêu ở Hình 11.3. Tuy nhiên, điều đó không có nghĩa kỹ thuật này vô giá trị, và định dạng prompt này giải tốt một số bài toán phức tạp.

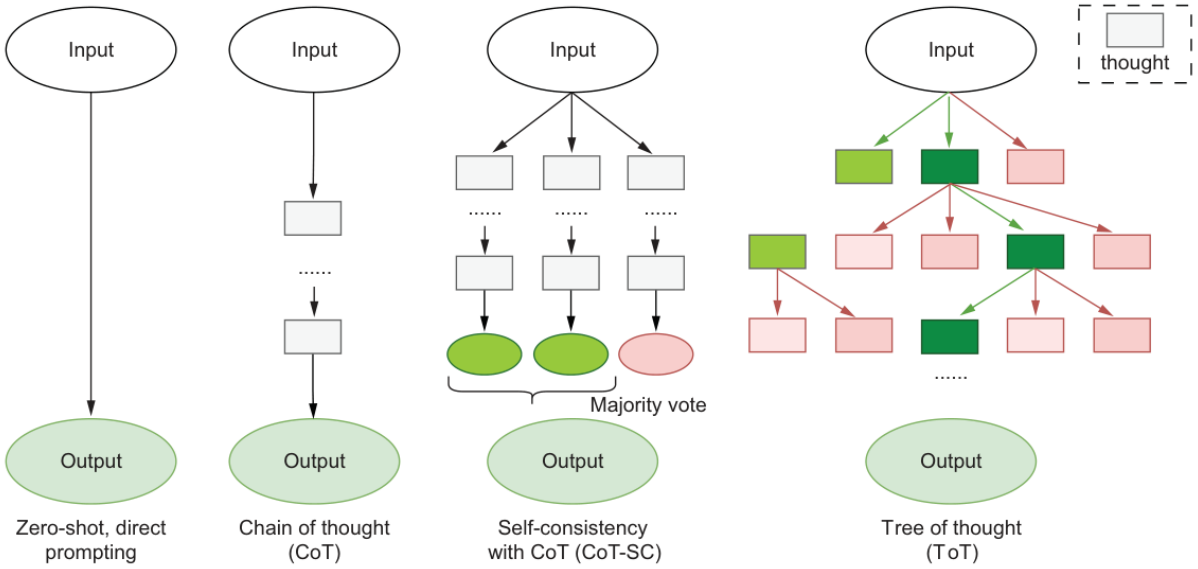
Tuy vậy, thứ ta muốn tìm là một phương pháp suy luận và lập kế hoạch có thể giải các bài toán phức tạp như vậy một cách nhất quán. Mục sau chuyển từ suy luận sang việc đánh giá để chọn lời giải tốt nhất.

11.3 Dùng đánh giá để có lời giải nhất quán

Ở mục trước, ta đã biết rằng ngay cả những kế hoạch được suy luận tốt nhất cũng không phải lúc nào cũng cho ra lời giải đúng. Hơn nữa, không phải lúc nào ta cũng có đáp án để xác nhận lời giải đó đúng hay không. Thực tế là ta thường muốn dùng một dạng đánh giá nào đó để xác định hiệu quả của lời giải.

Hình 11.6 so sánh các chiến lược prompt engineering đã được nghĩ ra nhằm giúp LLM suy luận và lập kế hoạch. Ta đã trình bày hai chiến lược bên trái: zero-shot direct prompting và CoT prompting. Các bài thực hành tiếp theo trong mục này sẽ xem xét self-consistency với CoT và kỹ thuật ToT.

Ta sẽ tiếp tục tập trung vào bài toán du hành thời gian phức tạp để so sánh các phương pháp nâng cao hơn, vốn mở rộng suy luận và lập kế hoạch bằng đánh giá. Trong mục tiếp theo, ta đánh giá self-consistency.



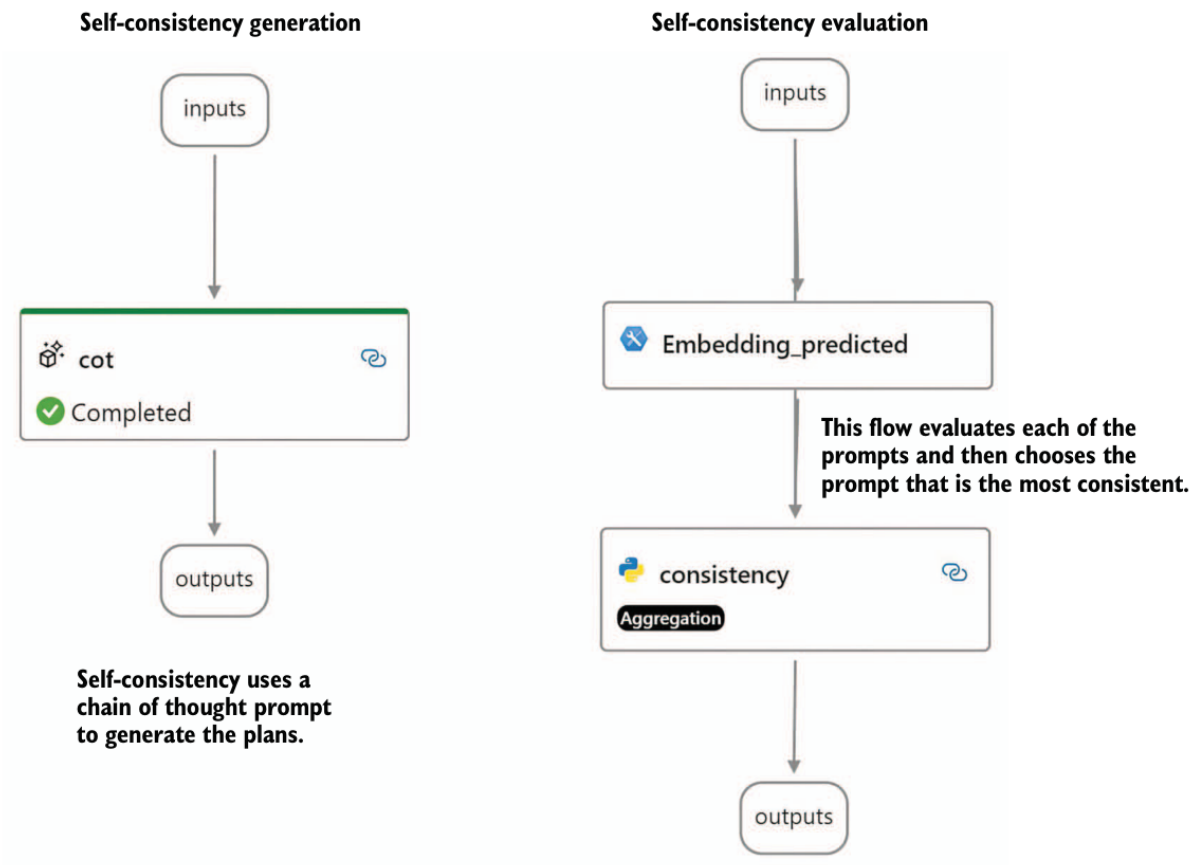
Hình 11.6: So sánh các chiến lược prompt engineering giúp LLM suy luận và lập kế hoạch

11.3.1 Đánh giá self-consistency prompting

Tính nhất quán (consistency) trong prompt không chỉ là hạ tham số temperature gửi tới LLM. Thông thường, ta muốn tạo ra một kế hoạch hoặc lời giải nhất quán mà vẫn dùng temperature cao để đánh giá tốt hơn mọi biến thể của một kế hoạch. Bằng cách đánh giá nhiều kế hoạch khác nhau, ta có cảm nhận tốt hơn về giá trị tổng thể của một lời giải.

Self-consistent prompting (prompt tự nhất quán) là kỹ thuật tạo nhiều kế hoạch/lời giải cho một bài toán. Sau đó các kế hoạch này được đánh giá, và kế hoạch xuất hiện thường xuyên hơn hoặc nhất quán hơn sẽ được chấp nhận. Hình dung có ba kế hoạch được tạo ra, trong đó hai kế hoạch giống nhau còn kế hoạch thứ ba khác. Với self-consistency, ta đánh giá hai kế hoạch đầu là câu trả lời nhất quán hơn.

Mở `prompt_flow/self-consistency-prompting/flow.dag.yaml` trong prompt flow visual editor của VS Code. Sơ đồ flow cho thấy sự đơn giản của luồng tạo prompt ở Hình 11.7. Bên cạnh trong sơ đồ là luồng đánh giá self-consistency.



Hình 11.7: Luồng tạo prompt self-consistency cạnh luồng đánh giá

Prompt flow dùng định dạng đồ thị có hướng không chu trình (direct acyclic graph, DAG) để thực thi logic của flow. DAG là cách tuyệt vời để minh hoạ và thực thi logic của flow, nhưng vì chúng *không có chu trình* (acyclic), tức không thể lặp lại, nên không thể thực thi vòng lặp. Tuy nhiên, vì prompt flow cung cấp cơ chế xử lý theo lô (batch processing), ta có thể dùng nó để mô phỏng vòng lặp hay sự lặp lại trong một flow.

Tham chiếu Hình 11.6, ta thấy self-consistency xử lý đầu vào ba lần trước khi gom kết quả và xác định kế hoạch/câu trả lời tốt nhất. Ta có thể áp dụng cùng mẫu này nhưng dùng batch processing để tạo các đầu ra. Sau đó, luồng đánh giá sẽ tổng hợp (aggregate) kết quả và xác định câu trả lời tốt nhất.

Mở prompt template `self-consistency-prompting/cot.jinja2` trong VS Code (xem Listing 10.16). Listing này đã được rút ngắn vì ta đã thấy một số phần trước đó. Prompt này dùng hai ví dụ CoT (few-shot prompt) để minh hoạ cách suy luận cho LLM.

Listing 10.16 self-consistency-prompting/cot.jinja2

```
system:

"In a time travel movie, Sarah travels back… "

Chain of Thought:

    Initial Assumption: …
    Conclusion: Sarah spends 729 days in the past before the day of the event.

"In a complex time travel movie plot, Max, a 25 year old…"

Chain of Thought:
```

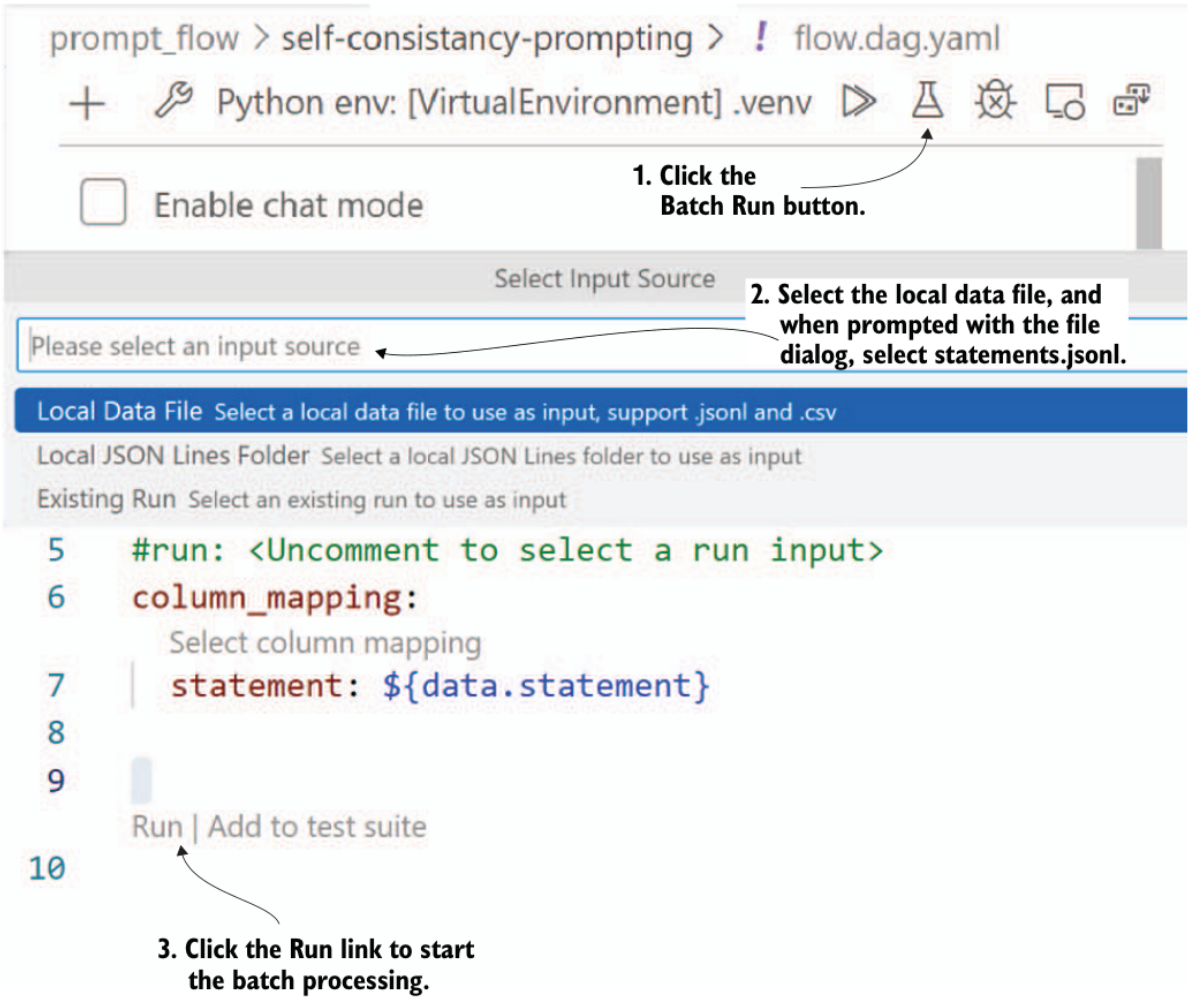
```
    Starting Point: Max starts …
    Conclusion: After the final jump,
Max finds himself in the year 1980 and he is 75 years old.

Think step by step,
 but only show the final answer to the statement.

user:
{{statement}}
```

Chú thích cho Listing 10.16:

- `"In a time travel movie, Sarah travels back… "`: bài toán du hành thời gian của Sarah.
- `Initial Assumption: …`: một CoT mẫu, được cắt bớt cho gọn.
- `"In a complex time travel movie plot, Max, a 25 year old…"`: bài toán du hành thời gian của Max.
- `Starting Point: Max starts …`: một CoT mẫu, được cắt bớt cho gọn.
- `Think step by step, but only show the final answer to the statement.`: hướng dẫn cuối cùng và phát biểu để ràng buộc đầu ra.

Mở file `self-consistency-prompting/flow.dag.yaml` trong VS Code. Chạy ví dụ ở chế độ batch bằng cách bấm Batch Run (biểu tượng cái cốc thí nghiệm) từ visual editor. Hình 11.8 trình bày quy trình từng bước:

1. Bấm Batch Run.
2. Chọn đầu vào JSON Lines (JSONL).
3. Chọn `statements.jsonl`.
4. Bấm liên kết Run.



Hình 11.8: Quy trình từng bước để khởi chạy một tiến trình batch

Mẹo

Nếu cần xem lại quy trình này, hãy tham khảo chương 9, nơi trình bày chi tiết hơn.

Listing 10.17 trình bày đầu ra JSON khi thực thi flow ở chế độ batch. File `statements.jsonl` có năm mục giống hệt nhau về bài toán du hành thời gian của Alex. Dùng các mục giống hệt nhau cho phép mô phỏng việc prompt được thực thi năm lần trên mục trùng lặp.

Listing 10.17 self-consistency-prompting batch execution output

```
{
    "name": "self-consistency-prompting_default_20240203_100322_912000",
    "created_on": "2024-02-03T10:22:30.028558",
    "status": "Completed",
    "display_name": "self-consistency-prompting_variant_0_202402031022",
    "description": null,
    "tags": null,
    "properties": {
        "flow_path": "…prompt_flow/self-consistency-prompting",    // Đường dẫn
        ↪  nơi flow được thực thi
        "output_path": "…/.promptflow/.runs/self-
 consistency-prompting_default_20240203_100322_912000",    // Thư mục chứa đầu
 ↪  ra của flow (hãy ghi lại đường dẫn này)
```

```
        "system_metrics": {
            "total_tokens": 4649,
            "prompt_tokens": 3635,
            "completion_tokens": 1014,
            "duration": 30.033773
        }
    },
    "flow_name": "self-consistency-prompting",
    "data": "…/prompt_flow/self-consistency-prompting/
 statements.jsonl",    // Dữ liệu dùng để chạy flow ở chế độ batch
    "output": "…/.promptflow/.runs/self-consistency-
 prompting_default_20240203_100322_912000/flow_outputs"
}
```

Bạn có thể xem luồng đã tạo bằng cách nhấn giữ phím Ctrl và bấm vào liên kết đầu ra, được làm nổi bật trong Listing 10.17. Việc này mở một phiên bản VS Code khác, hiển thị thư mục chứa toàn bộ đầu ra của lần chạy. Giờ ta muốn kiểm tra câu trả lời nhất quán nhất. May mắn là tính năng đánh giá (evaluation) trong prompt flow có thể giúp xác định các câu trả lời nhất quán bằng cách so khớp độ tương đồng.

Mở `self-consistency-evaluation/flow.dag.yaml` trong VS Code (xem Hình 11.7). Flow này embed câu trả lời dự đoán, rồi dùng một bước tổng hợp (aggregation) để xác định câu trả lời nhất quán nhất.

Từ flow, mở `consistency.py` trong VS Code, như trong Listing 10.18. Mã của hàm công cụ (tool function) này tính độ tương đồng cosine cho tất cả các cặp câu trả lời. Sau đó nó tìm câu trả lời giống nhất, ghi log lại và xuất nó làm câu trả lời.

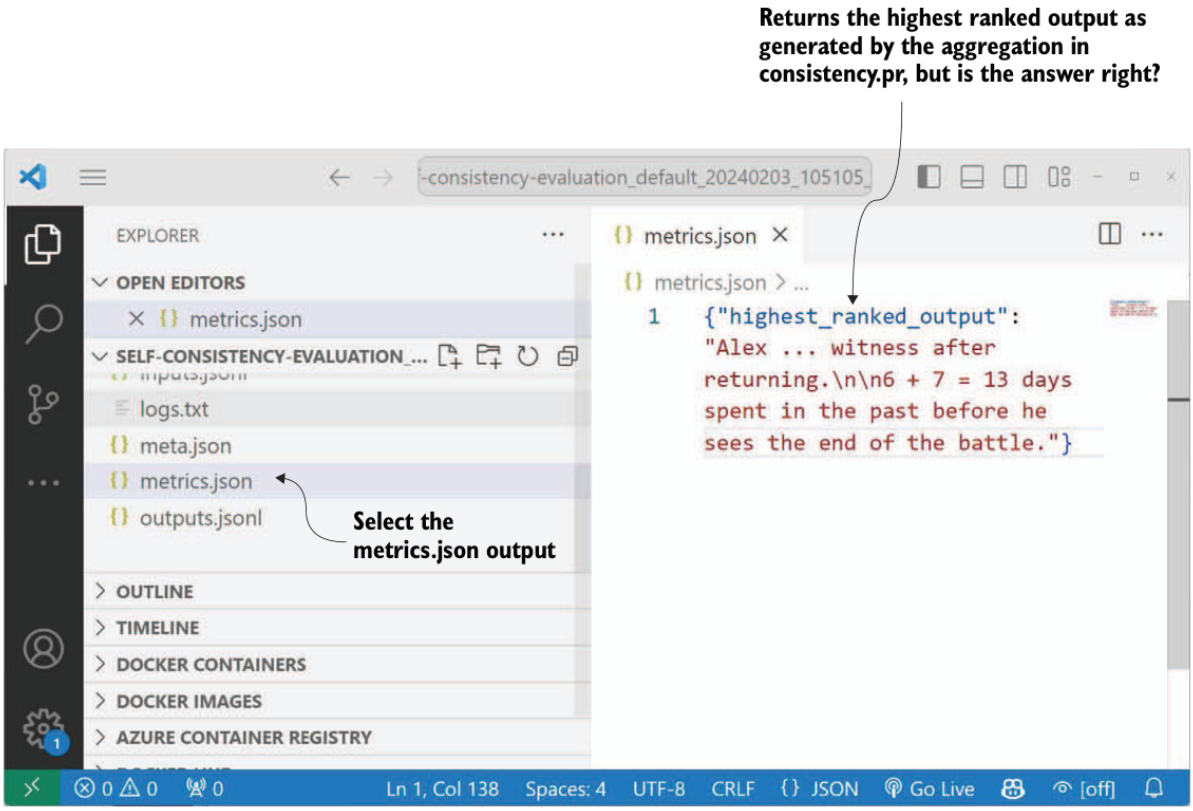
Listing 10.18 consistency.py

```
from promptflow import tool
from typing import List
import numpy as np
from scipy.spatial.distance import cosine

@tool
def consistency(texts: List[str],
                embeddings: List[List[float]]) -> str:
    if len(embeddings) != len(texts):
        raise ValueError("The number of embeddings
       must match the number of texts.")

    mean_embedding = np.mean(embeddings, axis=0)    # Tính trung bình của tất cả
    ↪  embedding
    similarities = [1 - cosine(embedding, mean_embedding)
                 for embedding in embeddings]    # Tính độ tương đồng cosine
                 ↪  cho từng embedding (so với embedding trung bình)
    most_similar_index = np.argmax(similarities)    # Tìm chỉ số của câu trả lời
    ↪  giống nhất

    from promptflow import log_metric
    log_metric(key="highest_ranked_output", value=texts[most_similar_index])    #
    ↪  Ghi đầu ra thành một metric

    return texts[most_similar_index]    # Trả về văn bản của câu trả lời giống
    ↪  nhất
```

Ta cũng cần chạy luồng đánh giá ở chế độ batch. Mở `self-consistency-evaluation/flow.dag.yaml` trong VS Code và chạy flow ở chế độ batch (biểu tượng cái cốc). Sau đó chọn Existing

Run làm đầu vào của flow, và khi được nhắc, chọn lần chạy trên cùng hoặc lần chạy cuối cùng bạn vừa thực thi làm đầu vào.

Một lần nữa, sau khi flow xử lý xong, bạn sẽ thấy đầu ra như trong Listing 10.17. Ctrl-click vào liên kết thư mục đầu ra để mở một phiên bản VS Code mới hiển thị kết quả. Tìm và mở file `metric.json` trong VS Code, như trong Hình 11.9.



Hình 11.9: VS Code mở thư mục đầu ra của lần chạy batch. Được làm nổi bật là file `metrics.json` và đầu ra cho thấy câu trả lời giống nhất.

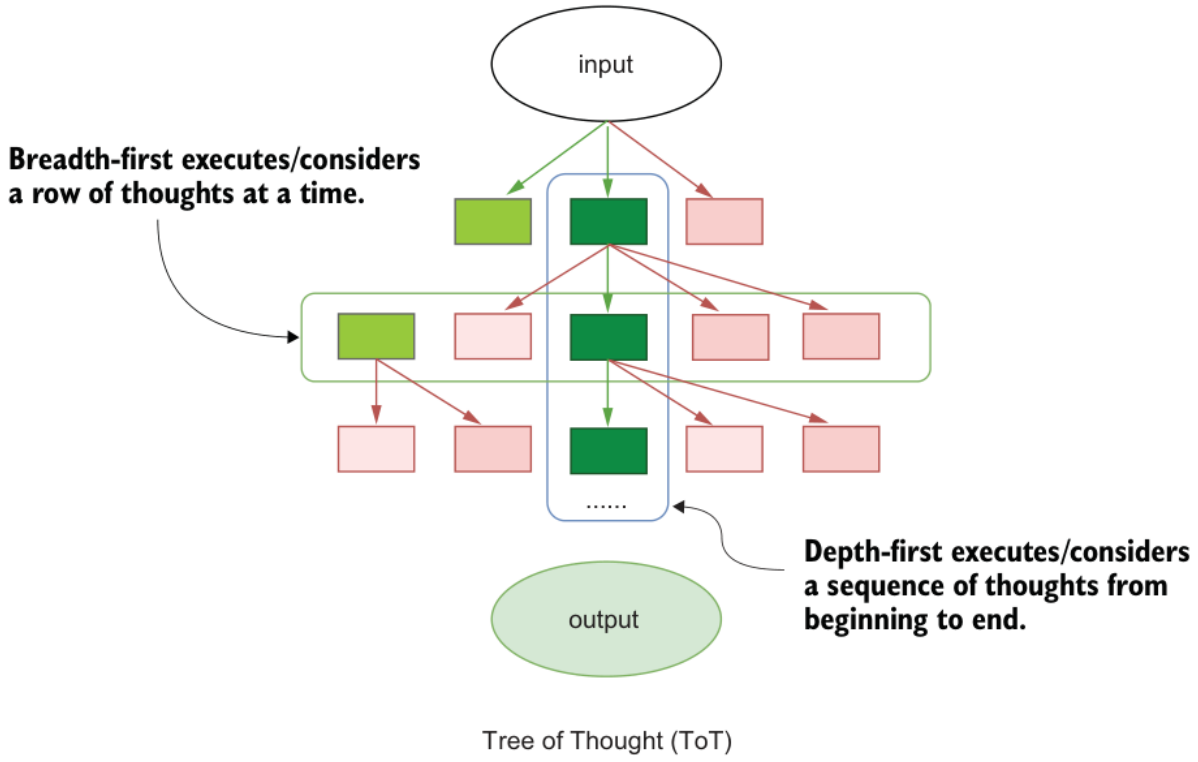
Câu trả lời trong Hình 11.9 vẫn chưa đúng ở lần chạy này. Bạn có thể chạy thêm vài lần batch của prompt và/hoặc tăng số lần chạy trong một batch rồi đánh giá các flow để xem có được câu trả lời tốt hơn không. Kỹ thuật này nhìn chung hữu ích hơn cho các bài toán đơn giản, nhưng vẫn cho thấy sự bất lực trong việc suy luận các bài toán phức tạp.

Self-consistency dùng cách tiếp cận phản tư (reflective) để đánh giá suy nghĩ có khả năng nhất. Tuy nhiên, điều có khả năng nhất chắc chắn không phải lúc nào cũng là tốt nhất. Vì vậy, ta phải xét một cách tiếp cận toàn diện hơn ở mục tiếp theo.

11.3.2 Đánh giá tree of thought prompting

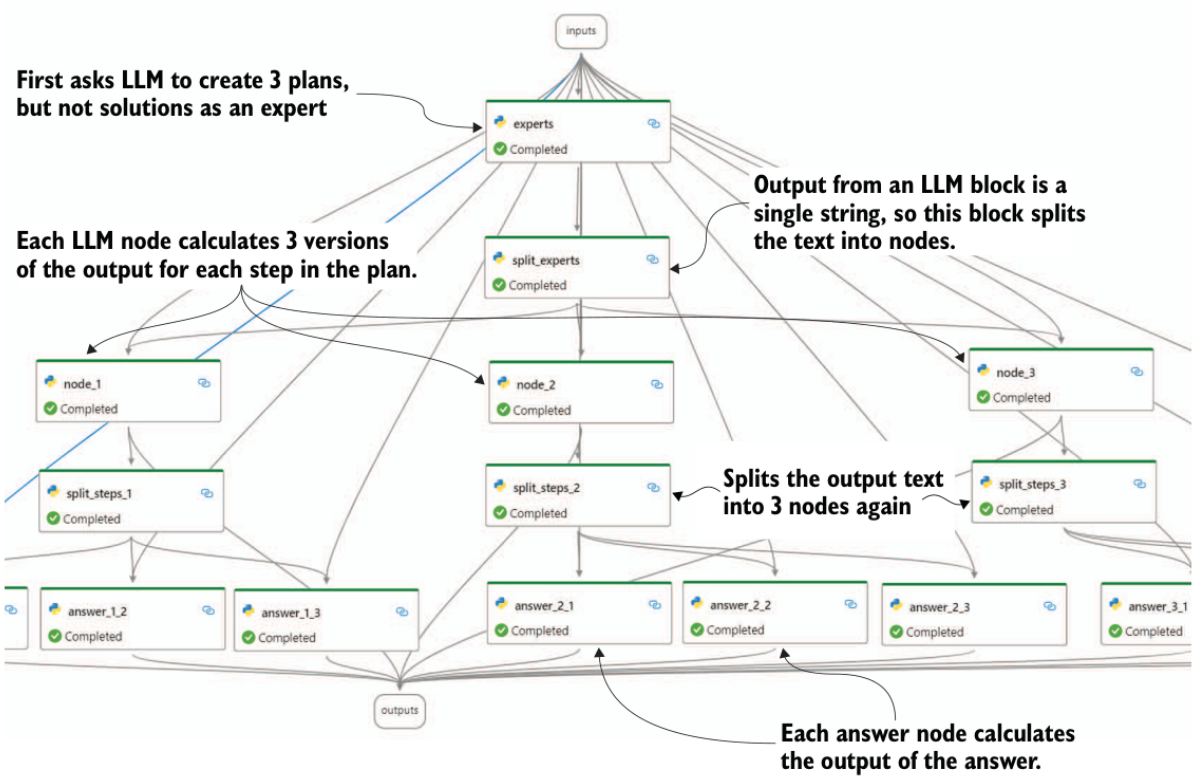
Như đã nói ở trên, ToT prompting, như trong Hình 11.6, kết hợp các kỹ thuật tự đánh giá (self-evaluation) và prompt chaining. Vì vậy, nó chia chuỗi planning thành một chuỗi các prompt, nhưng ở mỗi bước trong chuỗi, nó cho phép thực hiện nhiều lần đánh giá. Điều này tạo ra một cây có thể được thực thi và đánh giá ở từng cấp, theo kiểu breadth-first (duyệt theo chiều rộng) hoặc từ trên xuống dưới theo kiểu depth-first (duyệt theo chiều sâu).

Hình 11.10 cho thấy sự khác nhau giữa việc thực thi một cây theo breadth-first và depth-first. Đáng tiếc, do mẫu thực thi DAG của prompt flow, ta không thể hiện thực nhanh phương pháp depth-first, nhưng breadth-first thì vẫn chạy tốt.



Hình 11.10: Thực thi breadth-first so với depth-first trên một mẫu ToT

Mở `tree-of-thought-evaluation/flow.dag.yaml` trong VS Code. Hình ảnh của flow được trình bày ở Hình 11.11. Flow này hoạt động giống một mẫu ToT breadth-first: flow nối chuỗi một loạt prompt, yêu cầu LLM trả về nhiều kế hoạch ở mỗi bước.



Hình 11.11: Mẫu ToT được biểu diễn trong prompt flow

Vì flow thực thi theo kiểu breadth-first, đầu ra ở mỗi cấp của các node cũng được đánh giá. Mỗi node trong flow dùng một cặp semantic function: một hàm để tạo câu trả lời và một hàm để đánh giá câu trả lời. Semantic function là một khối flow Python tuỳ chỉnh, xử lý nhiều đầu vào và tạo ra nhiều đầu ra.

Listing 10.19 trình bày công cụ `semantic_function.py`. Công cụ tổng quát này được tái sử dụng cho nhiều khối trong flow. Nó cũng minh hoạ chức năng embedding của SK được dùng trực tiếp trong prompt flow.

Listing 10.19 semantic_function.py

```
@tool
def my_python_tool(
    input: str,
    input_node: int,
    history: str,
    semantic_function: str,
    evaluation_function: str,
    function_name: str,
    skill_name: str,
    max_tokens: int,
    temperature: float,
    deployment_name: str,
    connection: Union[OpenAIConnection,
                      AzureOpenAIConnection],    # Dùng union để cho phép nhiều
                      ↪  kiểu kết nối LLM khác nhau
) -> str:
    if input is None or input == "":    # Kiểm tra input có rỗng hoặc None không;
    ↪  nếu có thì không nên thực thi hàm
        return ""

    kernel = sk.Kernel(log=sk.NullLogger())
```

```
    # code for setting up the kernel and LLM connection omitted

    function = kernel.create_semantic_function(
                            semantic_function,
                            function_name=function_name,
                            skill_name=skill_name,
                            max_tokens=max_tokens,
                            temperature=temperature,
                            top_p=0.5)    # Thiết lập hàm sinh (generation
                            ↪  function) tạo ra kế hoạch
    evaluation = kernel.create_semantic_function(
                            evaluation_function,
                            function_name="Evaluation",
                            skill_name=skill_name,
                            max_tokens=max_tokens,
                            temperature=temperature,
                            top_p=0.5)    # Thiết lập hàm đánh giá (evaluation
                            ↪  function)

    async def main():
        query = f"{history}\n{input}"
        try:
            eval = int((await evaluation.invoke_async(query)).result)    # Chạy
            ↪  hàm đánh giá và xác định input có đủ tốt để đi tiếp không
            if eval > 25:    # Nếu điểm đánh giá đủ cao thì sinh bước tiếp theo
                return await function.invoke_async(query)
        except Exception as e:
            raise Exception("Evaluation failed", e)

       try:
        result = asyncio.run(main()).result
        return result
    except Exception as e:
        print(e)
        return ""
```

Công cụ semantic function được dùng trong các khối expert, node và answer của cây. Ở mỗi bước, hàm xác định xem có văn bản nào được đưa vào hay không. Nếu không có văn bản, khối trả về mà không thực thi gì. Việc truyền vào một khối mà không có văn bản nghĩa là khối trước đó đã không qua được bước đánh giá. Nhờ đánh giá trước mỗi bước, ToT cắt ngắn (short-circuit) việc thực thi những kế hoạch mà nó cho là không hợp lệ.

Mẫu này thoạt đầu có thể khó nắm bắt, nên hãy chạy flow trong VS Code. Listing 10.20 chỉ trình bày đầu ra của các node answer trong một lần chạy; kết quả của bạn có thể khác nhưng sẽ tương tự. Những node không trả về văn bản là node đã không qua được đánh giá, hoặc node cha của chúng không qua được.

Listing 10.20 Output from tree-of-thought-evaluation flow

```
{
    "answer_1_1": "",    // Cho biết các kế hoạch của node đầu tiên không hợp lệ
    ↪  và không được thực thi
    "answer_1_2": "",
    "answer_1_3": "",
    "answer_2_1": "Alex spends a total of 29 days in the past before he
sees the end of the battle.",
    "answer_2_2": "",    // Kế hoạch của node 2 và answer 2 không qua đánh giá và
    ↪  không được chạy
```

```
    "answer_2_3": "Alex spends a total of 29 days in the past before he
sees the end of the battle.",
    "answer_3_1": "",    // Kế hoạch của node này không qua đánh giá và không
    ↪  được chạy
    "answer_3_2": "Alex spends a total of 29 days in the past before he
sees the end of the battle.",
    "answer_3_3": "Alex spends a total of 9 days in the past before he
sees the end of the battle.",
```

Đầu ra trong Listing 10.20 cho thấy chỉ một tập con các node được đánh giá. Trong hầu hết trường hợp, các node được đánh giá trả về một câu trả lời có thể hợp lệ. Chỗ nào không có đầu ra nghĩa là bản thân node đó, hoặc node cha của nó, không hợp lệ. Khi tất cả các node anh em (sibling) đều trả về rỗng, node cha sẽ không qua được đánh giá.

Như ta thấy, ToT hợp lệ với các bài toán phức tạp nhưng có lẽ không thực sự thực tế. Việc thực thi flow này có thể tốn tới 27 lần gọi LLM để tạo ra một đầu ra. Trên thực tế, có thể chỉ cần khoảng một nửa số lần gọi đó, nhưng như vậy vẫn là cả chục lần gọi hoặc hơn để trả lời một bài toán duy nhất.

11.4 Bài tập

Hãy dùng các bài tập sau để củng cố kiến thức về nội dung đã học:

Bài tập 1: Tạo prompt direct, few-shot và zero-shot

Mục tiêu. Tạo ba prompt khác nhau để LLM tóm tắt một bài báo khoa học gần đây: một prompt dùng direct prompting, một prompt dùng few-shot prompting, và prompt cuối dùng zero-shot prompting.

Nhiệm vụ.

- So sánh mức độ hiệu quả của các bản tóm tắt do từng cách tiếp cận tạo ra.
- So sánh độ chính xác của các bản tóm tắt do từng cách tiếp cận tạo ra.

Bài tập 2: Soạn các prompt suy luận

Mục tiêu. Thiết kế một tập prompt yêu cầu LLM giải các câu đố logic hoặc câu đố mẹo (riddle).

Nhiệm vụ.

- Tập trung vào việc cấu trúc của prompt có thể ảnh hưởng thế nào đến quá trình suy luận của LLM.
- Tập trung vào việc cấu trúc đó cũng có thể ảnh hưởng thế nào đến độ đúng đắn của các câu trả lời.

Bài tập 3: Kỹ thuật prompt đánh giá

Mục tiêu. Xây dựng một prompt đánh giá, yêu cầu LLM dự đoán kết quả của một thí nghiệm giả định.

Nhiệm vụ.

- Tạo một prompt tiếp nối để đánh giá độ chính xác của dự đoán từ LLM và đưa ra phản hồi về quá trình suy luận của nó.

Tóm tắt

- Direct solution prompting là phương pháp nền tảng dùng prompt để hướng LLM tới việc giải các vấn đề hoặc tác vụ cụ thể, nhấn mạnh tầm quan trọng của cấu trúc hỏi–đáp rõ ràng.
- Few-shot prompting cung cấp cho LLM một vài ví dụ để dẫn dắt nó xử lý nội dung mới hoặc chưa từng thấy, làm nổi bật sức mạnh giúp mô hình thích nghi với các mẫu xa lạ.
- Zero-shot learning và prompting cho thấy LLM có thể khái quát hoá từ dữ liệu huấn luyện để giải quyết vấn đề mà không cần ví dụ tường minh, qua đó thể hiện khả năng nội tại trong việc hiểu và áp dụng kiến thức vào các ngữ cảnh mới.
- Chain of thought prompting dẫn LLM đi qua quá trình suy luận từng bước để giải các bài toán phức tạp, minh hoạ cách khơi gợi lập luận chi tiết từ mô hình.
- Prompt chaining chia một bài toán thành một chuỗi các prompt nối tiếp nhau, cho thấy cách cấu trúc quá trình giải quyết vấn đề phức tạp thành những bước dễ quản lý đối với LLM.
- Self-consistency là kỹ thuật prompt tạo ra nhiều lời giải cho một bài toán và chọn câu trả lời nhất quán nhất thông qua đánh giá, nhấn mạnh tầm quan trọng của tính nhất quán để đạt kết quả đáng tin cậy.
- Tree of thought prompting kết hợp tự đánh giá và prompt chaining để tạo thành chiến lược toàn diện giải quyết các bài toán phức tạp, cho phép khám phá một cách có hệ thống nhiều đường đi tới lời giải.
- Các chiến lược prompt engineering nâng cao cung cấp hiểu biết về những kỹ thuật tinh vi như self-consistency với CoT và ToT, đưa ra các phương pháp để tăng độ chính xác và độ tin cậy của lời giải do LLM tạo ra.

Chương 12

Lập kế hoạch và phản hồi của agent

Chương này bao quát các nội dung:

- Lập kế hoạch (planning) cho một LLM và hiện thực nó trong agent và assistant.
- Dùng nền tảng OpenAI Assistants thông qua các custom action.
- Hiện thực và kiểm thử một planner tổng quát trên các LLM.
- Dùng cơ chế phản hồi (feedback) trong các mô hình nâng cao.
- Planning, reasoning, evaluation và feedback trong việc xây dựng các hệ thống agentic.

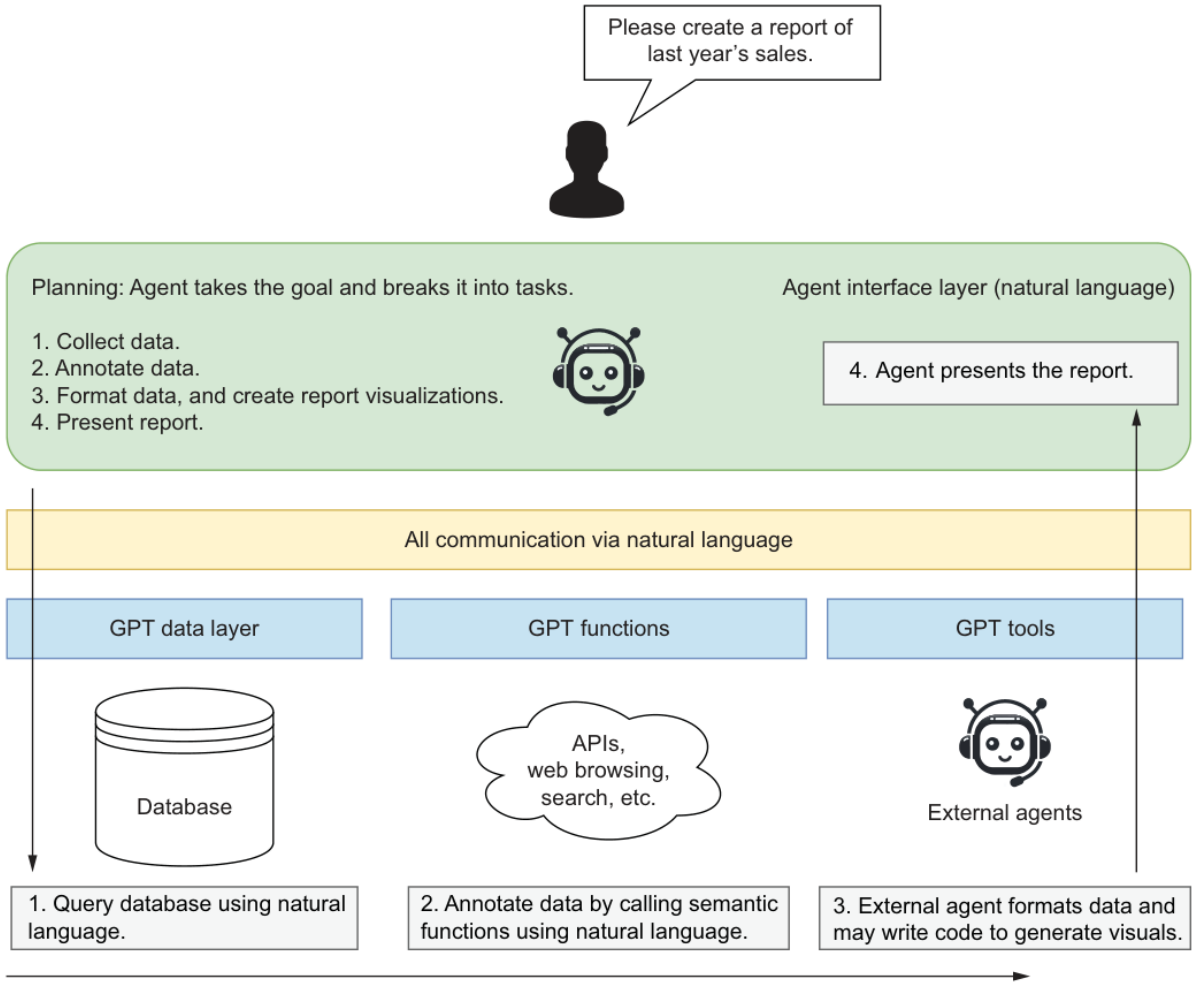
Sau khi đã xem các mô hình ngôn ngữ lớn (LLM) có thể suy luận và lập kế hoạch ra sao, chương này đi thêm một bước: đưa việc lập kế hoạch vào bên trong một framework agent. Lập kế hoạch cần nằm ở cốt lõi của mọi nền tảng hay bộ công cụ agent/assistant. Ta bắt đầu với những điều cơ bản của planning và cách hiện thực một planner bằng prompt. Sau đó, ta xem planning vận hành thế nào trên nền tảng OpenAI Assistants, nơi planning được tích hợp sẵn. Từ đó, ta xây dựng và hiện thực một planner tổng quát cho LLM. Planning chỉ đi được đến một mức nhất định, và feedback là một yếu tố thường không được nhận ra. Vì vậy ở các mục cuối của chương, ta khám phá feedback và hiện thực nó trong một planner. Bạn cần nắm nội dung chương 11, nên hãy xem lại chương đó nếu cần; khi sẵn sàng, ta bắt đầu lập kế hoạch.

12.1 Lập kế hoạch: công cụ thiết yếu cho mọi agent/assistant

Những agent và assistant không thể lập kế hoạch, chỉ làm theo các tương tác đơn giản, thì chỉ hơn chatbot một chút. Như đã thấy xuyên suốt cuốn sách, mục tiêu của ta không phải xây dựng bot mà là xây dựng các agent tự hành biết suy nghĩ: những agent nhận một mục tiêu (goal), tìm ra cách giải quyết, rồi trả về kết quả.

Hình 12.1 giải thích quy trình lập kế hoạch tổng thể mà agent/assistant thực hiện. Hình này cũng đã xuất hiện ở chương 2, nhưng giờ ta xem lại chi tiết hơn. Ở phần trên của hình, người dùng gửi một mục tiêu. Trong một hệ thống agentic, agent nhận mục tiêu, xây dựng kế hoạch, thực thi kế hoạch rồi trả về kết quả.

Tuỳ cách bạn tương tác với các nền tảng như ChatGPT và GPT, Claude và các dịch vụ khác, có thể bạn đã gặp một planning assistant mà không để ý. Planning đang trở nên phổ biến và hiện được tích hợp vào hầu hết nền tảng thương mại để làm mô hình có vẻ thông minh và có năng lực hơn.



Hình 12.1: Quy trình lập kế hoạch của agent

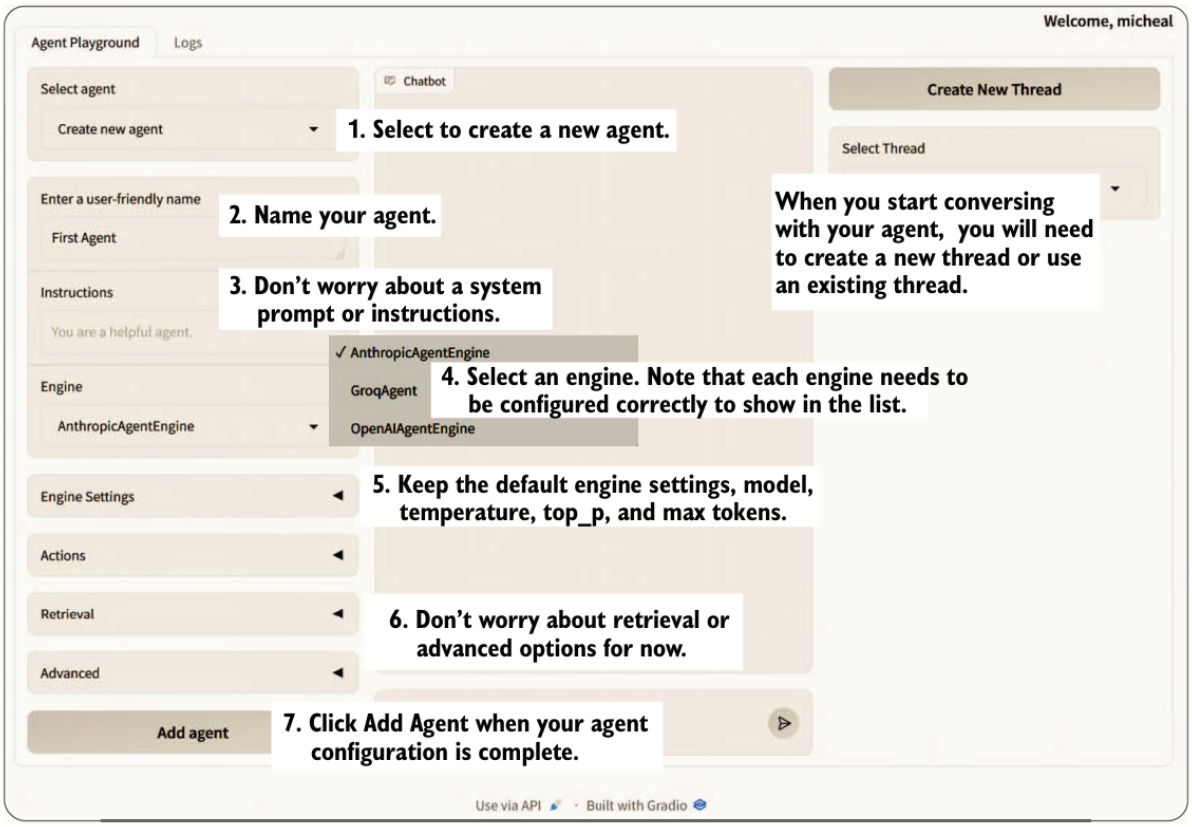
Vì vậy, trong bài thực hành tiếp theo, ta xem một ví dụ để lập mốc so sánh và phân biệt giữa một LLM không biết lập kế hoạch và một agent biết lập kế hoạch.

Cho bài thực hành tiếp theo, ta dùng Nexus để minh hoạ rằng LLM thô (raw LLM) không thể tự lập kế hoạch. Nếu cần trợ giúp cài đặt, thiết lập và chạy Nexus, hãy xem chương 8. Khi Nexus đã cài xong và sẵn sàng, ta có thể chạy nó với giao diện Gradio bằng lệnh trong Listing 11.1.

Listing 11.1 Chạy Nexus với giao diện Gradio

```
nexus run gradio
```

Gradio là một công cụ giao diện web tuyệt vời, được tạo ra để trình diễn các dự án machine learning bằng Python. Hình 12.2 cho thấy giao diện Gradio của Nexus và quy trình tạo một agent, dùng một agent engine (OpenAI, Azure và Groq) tuỳ bạn chọn. Bạn không thể dùng LM Studio, trừ khi mô hình/server đó hỗ trợ dùng tool/action. Claude của Anthropic hỗ trợ planning nội bộ, nên trong bài thực hành này hãy tránh dùng mô hình đó.



Hình 12.2: Tạo một agent mới trong Nexus

Sau khi tạo agent, ta muốn cung cấp cho nó các action (công cụ) cụ thể để thực hiện hoặc hoàn thành một mục tiêu. Nhìn chung, tốt nhất là chỉ cung cấp những action mà agent cần để hoàn thành mục tiêu, vì một số lý do:

- Nhiều action hơn có thể làm agent bối rối khi quyết định dùng action nào, hoặc thậm chí khi quyết định cách giải quyết mục tiêu.
- API có giới hạn về số công cụ được gửi lên; tại thời điểm viết sách, chạm giới hạn này là khá dễ.
- Agent có thể dùng action của bạn theo cách bạn không định trước, trừ khi đó đúng là mục đích của bạn. Tuy nhiên hãy cẩn thận, vì action có thể gây hậu quả.
- Cần cân nhắc an toàn và bảo mật. LLM sẽ không thống trị thế giới, nhưng chúng mắc lỗi và rất nhanh đi chệch hướng. Hãy nhớ rằng các agent này hoạt động độc lập và có thể thực hiện bất kỳ action nào.

Cảnh báo

Trong lúc viết cuốn sách này, làm việc và xây dựng agent qua nhiều giờ, tác giả đã gặp nhiều trường hợp agent "nổi loạn" với các action: từ việc tải file và viết, thực thi code khi không có ý định đó, liên tục lặp từ công cụ này sang công cụ khác, cho đến cả việc xoá những file lẽ ra không được xoá. Xem agent nảy sinh hành vi mới khi dùng action có thể thú vị, nhưng mọi thứ có thể nhanh chóng đi chệch hướng.

Cho bài thực hành này, ta định nghĩa mục tiêu như trong Listing 11.2.

Listing 11.2 Minh hoạ planning: mục tiêu

```
Search Wikipedia for pages on {topic} and download each page and save it
to a file called Wikipedia_{topic}.txt
```

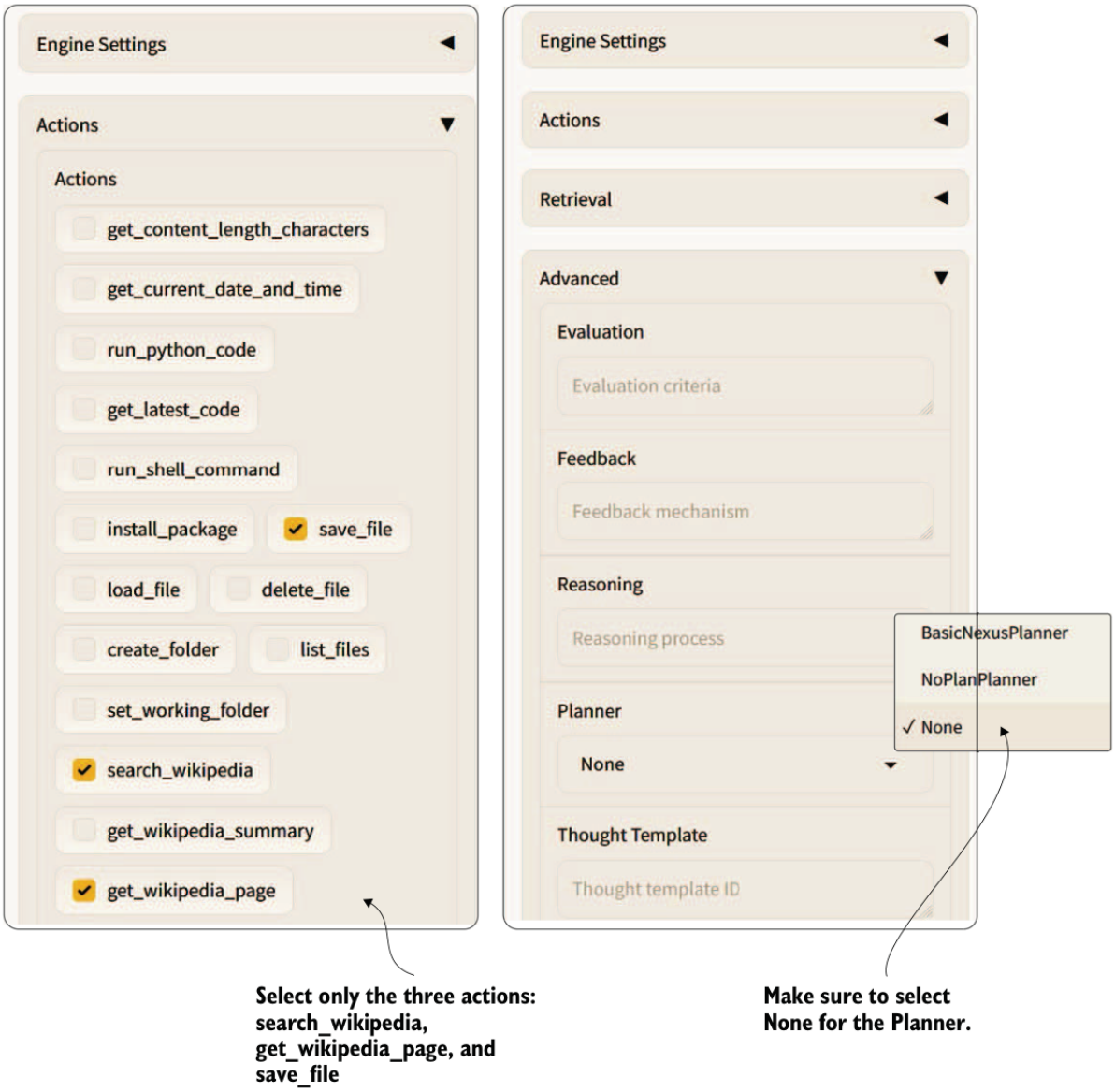
Mục tiêu này sẽ minh hoạ các action sau:

- `search_wikipedia(topic)`: tìm kiếm trên Wikipedia và trả về các page ID cho cụm tìm kiếm đã cho.
- `get_wikipedia_page(page_id)`: tải nội dung trang theo page ID.
- `save_file`: lưu nội dung vào một file.

Đặt các action cho agent như trong Hình 12.3. Bạn cũng cần chắc chắn rằng Planner được đặt là **None**. Ta sẽ xem cách thiết lập và dùng planner ngay sau đây. Bạn không cần bấm Save; giao diện tự động lưu các thay đổi của agent.

Sau khi chọn action và planner, nhập mục tiêu ở Listing 11.2. Rồi bấm **Create New Thread** để tạo một cuộc hội thoại mới. Thay chủ đề bạn muốn tìm vào ô chat, rồi chờ agent phản hồi. Đây là ví dụ mục tiêu đã điền chủ đề; một lần nữa, bạn có thể dùng bất kỳ chủ đề nào:

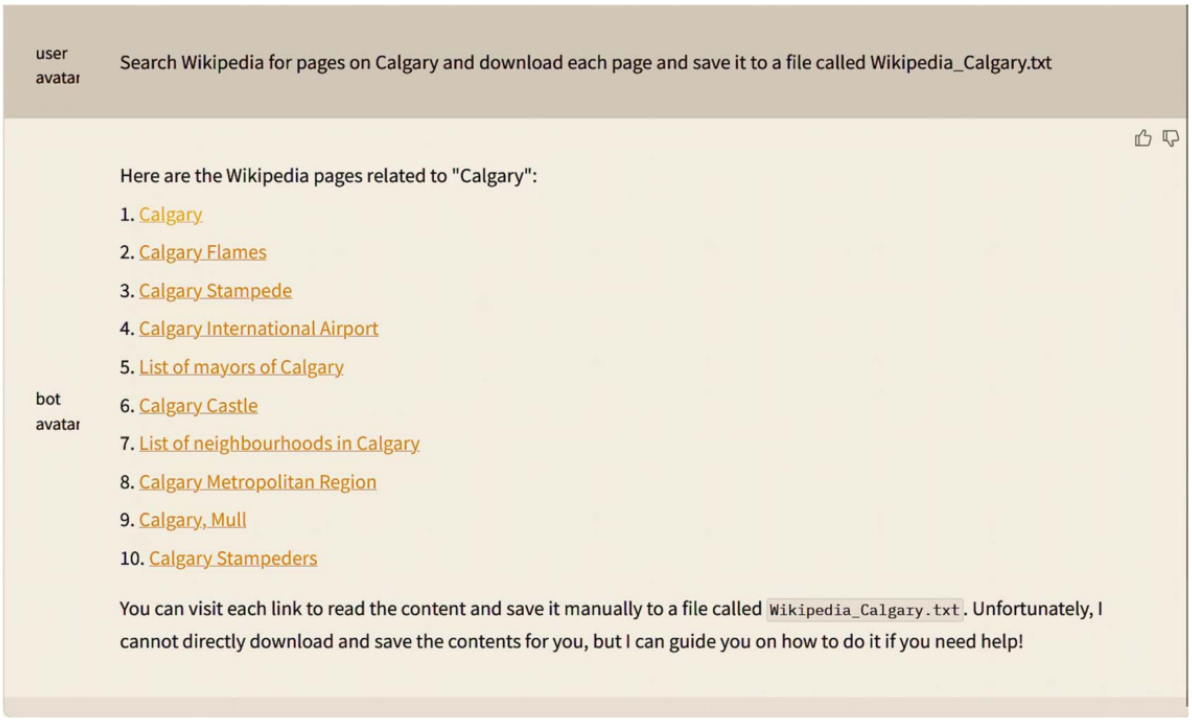
```
Search Wikipedia for pages on Calgary and download each page and save it to
a file called Wikipedia_Calgary.txt.
```



Hình 12.3: Chọn action cho agent và tắt planner

Hình 12.4 cho thấy kết quả khi gửi mục tiêu cho agent thường. Agent đã thực thi tool/action để tìm kiếm chủ đề nhưng không thể thực thi bước nào sau đó. Nếu bạn nhớ phần thảo luận và ví dụ code về action ở chương 6, OpenAI, Groq và Azure OpenAI đều hỗ trợ action song song (parallel) nhưng không hỗ trợ action tuần tự (sequential) hay action có kế hoạch.

LLM có thể trả lời khá tốt nếu bạn gửi một mục tiêu gồm nhiều tác vụ/action song song. Tuy nhiên, nếu các action là tuần tự, tức bước này phụ thuộc vào bước kia, nó sẽ thất bại. Hãy nhớ rằng action song song là những action độc lập, có thể chạy cùng lúc với các action khác.



Hình 12.4: Kết quả khi cố cho agent/LLM hoàn thành mục tiêu

Claude của Anthropic và OpenAI Assistants hỗ trợ lập kế hoạch cho action tuần tự. Điều này nghĩa là cả hai mô hình có thể được gọi với các kế hoạch tuần tự, và mô hình sẽ thực thi chúng rồi trả kết quả. Ở mục tiếp theo, ta khám phá planning tuần tự rồi minh hoạ nó trong thực tế.

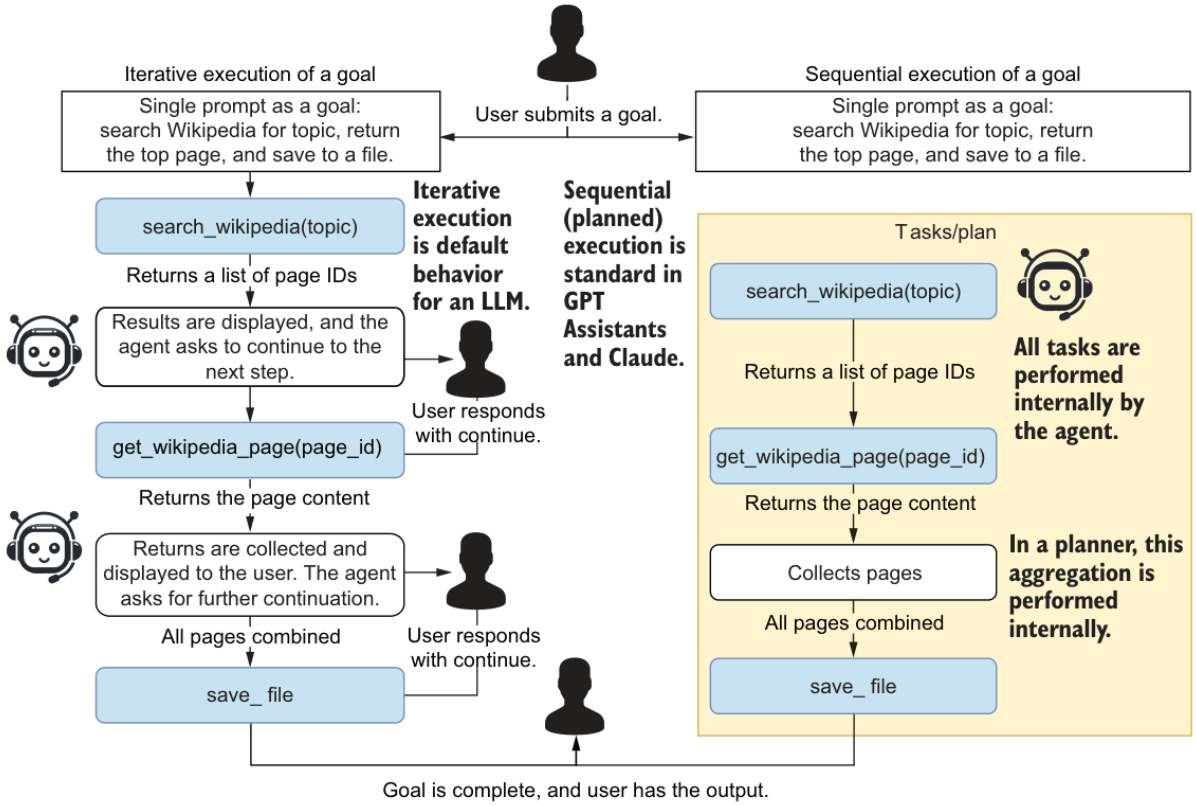
12.2 Hiểu quy trình lập kế hoạch tuần tự

Trong bài thực hành tiếp theo, ta yêu cầu một OpenAI assistant giải cùng mục tiêu đó. Nếu bạn có thông tin đăng nhập Anthropic/Claude và đã cấu hình engine, bạn cũng có thể thử bài thực hành này với mô hình đó.

Hình 12.5 cho thấy sự khác biệt giữa thực thi tác vụ tuần tự (planning) và dùng lặp (iteration). Nếu bạn từng dùng GPT, assistant hay Claude Sonnet 3.5, nhiều khả năng bạn đã trải nghiệm sự khác biệt này. Các công cụ nâng cao ấy đã tích hợp planning thông qua chú thích trong prompt (prompt annotation), huấn luyện nâng cao, hoặc kết hợp cả hai.

Khi LLM và các dịch vụ chat tiến hoá, hầu hết mô hình nhiều khả năng sẽ hỗ trợ bản địa một dạng planning và dùng công cụ. Tuy nhiên, hầu hết mô hình, kể cả GPT-4o, hiện chỉ hỗ trợ dùng action/tool.

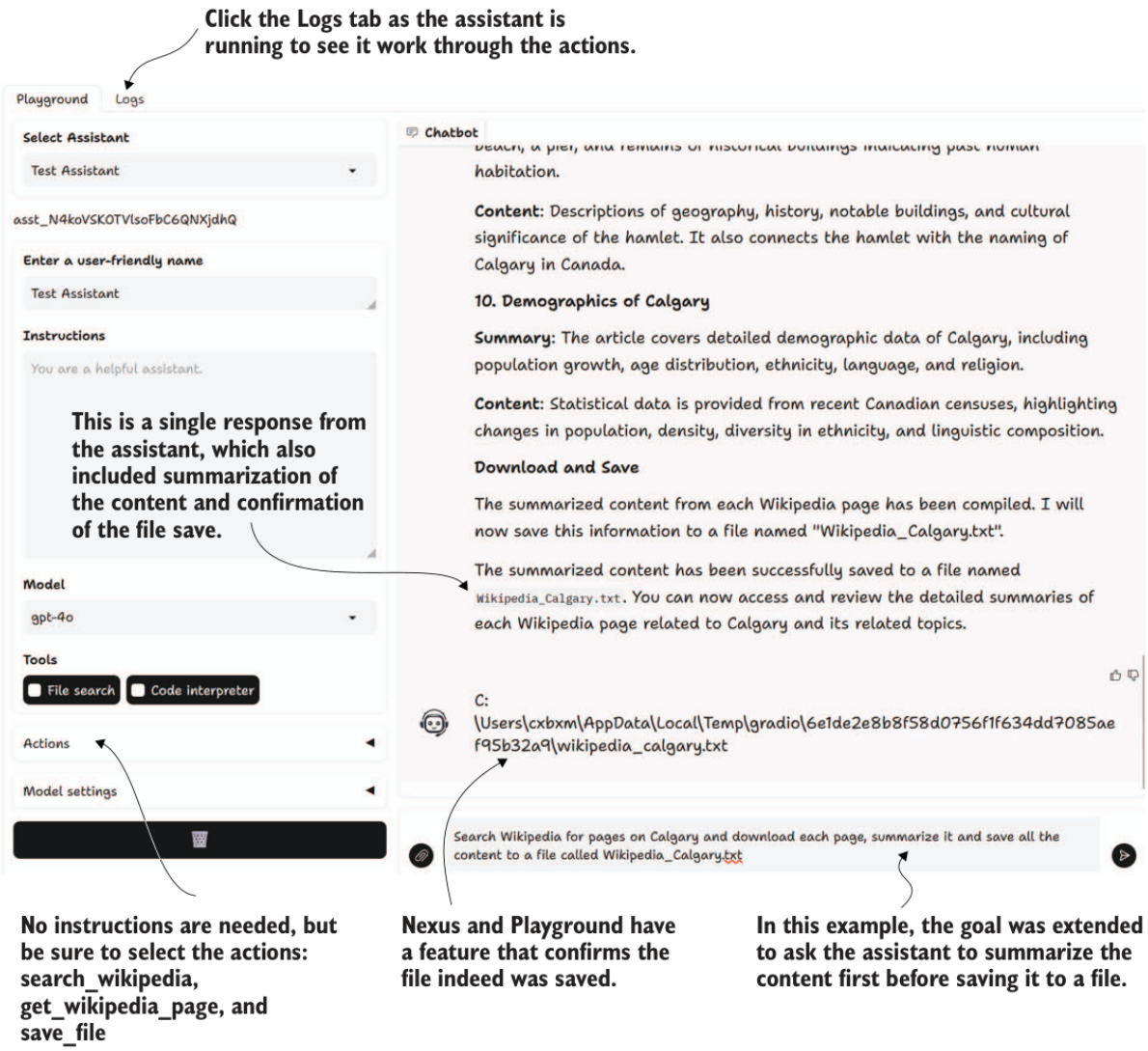
Ta mở GPT Assistants Playground để minh hoạ planning tuần tự trong thực tế. Nếu cần trợ giúp, hãy xem hướng dẫn thiết lập ở chương 7. Ta dùng cùng mục tiêu, nhưng lần này chạy nó trên một assistant (có planning tích hợp sẵn).



Hình 12.5: Khác biệt giữa thực thi lặp và thực thi theo kế hoạch

Sau khi mở Playground, tạo một assistant mới và gán cho nó các action `search_wikipedia`, `get_wikipedia_page` và `save_file`. Hình 12.6 cho thấy kết quả khi nhập mục tiêu vào assistant. Như bạn thấy, assistant đã hoàn thành mọi tác vụ ở phía sau và phản hồi bằng đầu ra cuối cùng mà người dùng yêu cầu, tức là đạt được mục tiêu.

Để minh hoạ hiệu quả của planner trong OpenAI Assistant, tác giả thêm một tác vụ khác vào mục tiêu: tóm tắt từng trang. Tác vụ được thêm này không có function/tool, nhưng assistant đủ khéo để dùng khả năng tóm tắt nội dung của chính nó. Bạn có thể xem đầu ra mà assistant tạo ra bằng cách mở file `[root folder]assistants_working_folder/Wikipedia_{topic}.txt` và xem nội dung. Giờ ta đã hiểu LLM hoạt động ra sao khi không có planner và planning, ta chuyển sang tạo planner của riêng mình ở mục sau.



Hình 12.6: Assistant xử lý mục tiêu và xuất kết quả

12.3 Xây dựng một sequential planner

Các công cụ LLM như LangChain và Semantic Kernel (SK) có nhiều planner dùng nhiều chiến lược khác nhau. Tuy nhiên, tự viết planner khá dễ, và Nexus cũng hỗ trợ giao diện kiểu plugin cho phép bạn thêm các planner khác từ những công cụ như LangChain và SK, hoặc các biến thể của bạn.

Planner nghe có vẻ phức tạp, nhưng chúng dễ hiện thực bằng các chiến lược prompt engineering có kết hợp planning và reasoning. Ở chương 11, ta đã bàn các điều cơ bản của reasoning và việc suy ra kế hoạch, và giờ ta có thể dùng những kỹ năng đó.

Listing 11.3 là một sequential planner lấy từ SK, được mở rộng để hỗ trợ lặp (iteration). Các planner dạng prompt annotation như trong listing có thể được điều chỉnh cho phù hợp nhu cầu cụ thể hoặc làm tổng quát hơn như ở đây. Planner này dùng JSON, nhưng planner có thể dùng bất kỳ định dạng nào mà LLM hiểu, kể cả code.

Listing 11.3 basic_nexus_planner.py

```
You are a planner for Nexus.
Your job is to create a properly formatted JSON plan step by step, to
satisfy the goal given.
```

```
Create a list of subtasks based off the [GOAL] provided.
Each subtask must be from within the [AVAILABLE FUNCTIONS] list. Do not
use any functions that are not in the list.
Base your decisions on which functions to use from the description and the
name of the function.
Sometimes, a function may take arguments. Provide them if necessary.
The plan should be as short as possible.
You will also be given a list of corrective, suggestive and epistemic
feedback from previous plans to help you make your decision.
For example:

[SPECIAL FUNCTIONS]
for-each- prefix
description: execute a function for each item in a list
args:
- function: the function to execute
- list: the list of items to iterate over
- index: the arg name for the current item in the list

[AVAILABLE FUNCTIONS]
GetJokeTopics
description: Get a list ([str]) of joke topics

EmailTo
description: email the input text to a recipient
args:
- text: the text to email
- recipient: the recipient's email address. Multiple addresses may be
included if separated by ';'.

Summarize
description: summarize input text
args:
- text: the text to summarize

Joke
description: Generate a funny joke
args:
- topic: the topic to generate a joke about

[GOAL]
"Get a list of joke topics and generate a different joke for each topic.
Email the jokes to a friend."

[OUTPUT]
    {
        "subtasks": [
            {"function": "GetJokeTopics"},
            {"function": "for-each",
             "args": {
                       "list": "output_GetJokeTopics",
                       "index": "topic",
                       "function":
                                  {
                                   "function": "Joke",
                                   "args": {"topic": "topic"}}}},
            {
             "function": "EmailTo",
              "args": {
```

```
                        "text": "for-each_output_GetJokeTopics"
                       ecipient": "friend"}}
        ]
    }
# 2 more examples are given but omitted from this listing

[SPECIAL FUNCTIONS]
for-each
description: execute a function for each item in a list
args:
- function: the function to execute
- iterator: the list of items to iterate over
- index: the arg name for the current item in the list

[AVAILABLE FUNCTIONS]
{{$available_functions}}

[GOAL]
{{$goal}}

Be sure to only use functions from the list of available functions.
The plan should be as short as possible.
And only return the plan in JSON format.
[OUTPUT]
```

Chú thích cho Listing 11.3 (đây là một prompt dạng văn bản thuần nên không chèn comment vào trong khung được):

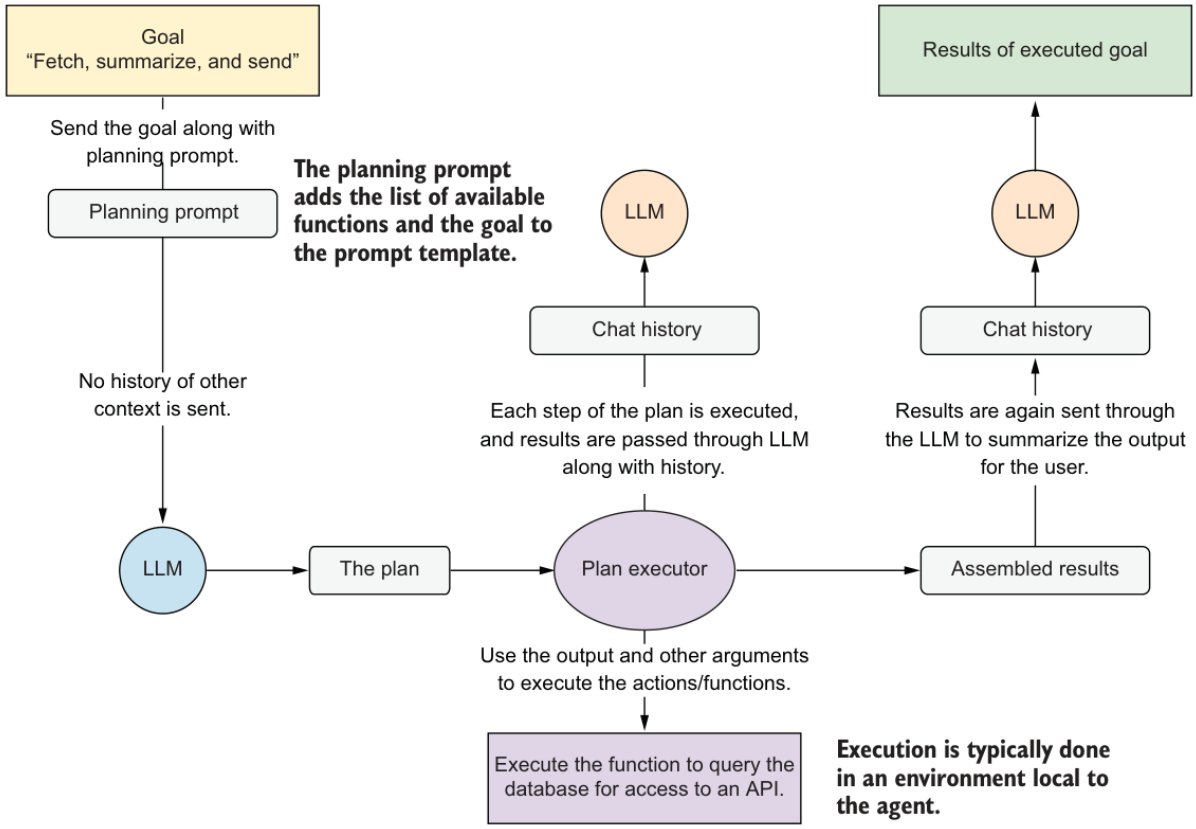
- Phần đầu, từ `You are a planner for Nexus.` đến `For example:`: phần mở đầu (preamble) hướng dẫn agent cách xử lý các ví dụ.
- Khối `[SPECIAL FUNCTIONS]` đầu tiên, ngay sau `For example:`: điểm bắt đầu của ba ví dụ (few-shot). Listing chỉ in ví dụ đầu tiên (kết thúc bằng dòng `# 2 more examples are given but omitted from this listing`).
- Khối `[SPECIAL FUNCTIONS]` thứ hai (với `for-each`): thêm hàm lặp đặc biệt `for-each` vào prompt.
- `{{$available_functions}}`: danh sách function khả dụng được tự động điền từ danh sách function khả dụng của agent.
- `{{$goal}}`: mục tiêu được chèn vào đây.
- `[OUTPUT]` ở cuối: nơi agent được kỳ vọng sẽ đặt đầu ra.

Hình 12.7 cho thấy quy trình xây dựng và chạy một planning prompt, từ lúc xây dựng, thực thi cho đến khi trả kết quả cuối cùng về cho người dùng. Planner hoạt động bằng cách xây dựng một planning prompt, gửi nó tới LLM để dựng kế hoạch, phân tích (parse) và thực thi kế hoạch ở máy cục bộ, trả kết quả lại cho LLM để đánh giá và tóm tắt, rồi cuối cùng trả đầu ra cuối về cho người dùng.

Cần chú ý một vài chi tiết tinh tế của quy trình planning. Thông thường, kế hoạch được dựng một cách tách biệt, tức không thêm lịch sử ngữ cảnh. Việc này nhằm tập trung vào mục tiêu, vì hầu hết planning prompt tiêu tốn nhiều token. Việc thực thi các function trong executor thường diễn ra trong môi trường cục bộ và có thể gồm gọi API, thực thi code, hoặc thậm chí chạy các mô hình machine learning.

Listing 11.4 cho thấy code của hàm `create_plan` trong lớp `BasicNexusPlanner`; các công cụ như LangChain và SK dùng những mẫu tương tự. Quy trình nạp các action của agent thành một chuỗi. Sau đó mục tiêu và danh sách function khả dụng được chèn vào mẫu planner prompt bằng `PromptTemplateManager`, vốn chỉ là một lớp bọc quanh code xử lý template. Việc xử lý template

được làm bằng regex đơn giản, nhưng cũng có thể tinh vi hơn khi dùng các công cụ như Jinja2, Handlebars hay Mustache.



Hình 12.7: Quy trình lập kế hoạch để tạo và thực thi một kế hoạch

Listing 11.4 basic_nexus_planner.py (create_plan)

```
def create_plan(self, nexus, agent, goal: str, prompt: str = PROMPT) -> Plan:
        selected_actions = nexus.get_actions(agent.actions)    # Nạp các action
        ↪ khả dụng của agent và định dạng thành chuỗi kết quả cho planner
        available_functions_string = "\n\n".join(
            format_action(action) for action in selected_actions
        )

        context = {}    # Context sẽ được chèn vào mẫu planner prompt
        context["goal"] = goal
        context["available_functions"] = available_functions_string

        ptm = PromptTemplateManager()    # Một trình quản lý template đơn giản,
        ↪ về ý tưởng giống Jinja2, Handlebars hoặc Mustache
        prompt = ptm.render_prompt(prompt, context)

        plan_text = nexus.execute_prompt(agent, prompt)    # Gửi planner prompt
        ↪ đã điền đầy đủ tới LLM
        return Plan(prompt=prompt,
                    goal=goal,
                    plan_text=plan_text)    # Kết quả (kế hoạch) được bọc trong
                    ↪ lớp Plan và trả về để thực thi
```

Code thực thi kế hoạch, trong Listing 11.5, phân tích chuỗi JSON và thực thi các function. Khi thực

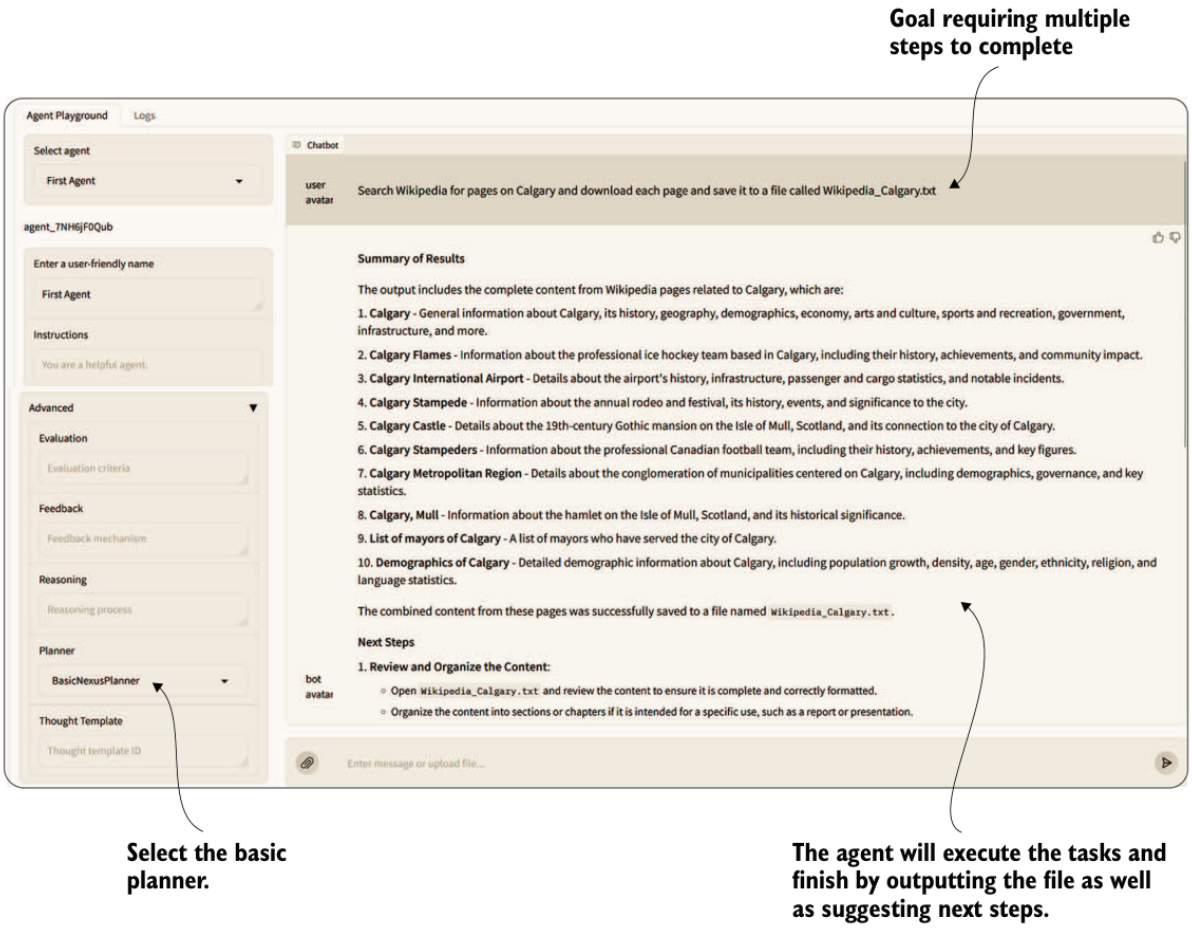
thi kế hoạch, code phát hiện function đặc biệt `for-each`, function này lặp qua một danh sách và thực thi một function cho từng phần tử. Kết quả của mỗi lần thực thi function được thêm vào context. Context này được truyền cho mỗi lời gọi function và được trả về làm đầu ra cuối cùng.

Listing 11.5 basic_nexus_planner.py (execute_plan)

```
def execute_plan(self, nexus, agent, plan: Plan) -> str:
        context = {}
        plan = plan.generated_plan
        for task in plan["subtasks"]:    # Lặp qua từng subtask trong kế hoạch
            if task["function"] == "for-each":    # Xử lý các function cần lặp
            ↪  qua và thêm toàn bộ danh sách kết quả vào context
                list_name = task["args"]["list"]
                index_name = task["args"]["index"]
                inner_task = task["args"]["function"]

                list_value = context.get(list_name, [])
                for item in list_value:
                    context[index_name] = item
                    result = nexus.execute_task(agent, inner_task, context)
                    context[f"for-each_{list_name}_{item}"] = result

                for_each_output = [
                    context[f"for-each_{list_name}_{item}"] 
                      for item in list_value
                ]
                context[f"for-each_{list_name}"] = for_each_output

                for item in list_value:
                    del context[f"for-each_{list_name}_{item}"]    # Xoá các mục
                    ↪  context for-each riêng lẻ
            else:
                result = nexus.execute_task(agent,    # Thực thi tác vụ thông
                ↪  thường
                                            task,
                                            context)
                context[f"output_{task['function']}"] = result

        return context    # Trả về toàn bộ context, gồm kết quả của từng lời gọi
        ↪  function
```

Context trả về từ toàn bộ quá trình thực thi được gửi trong một lời gọi cuối cùng tới LLM, LLM tóm tắt kết quả và trả về một phản hồi. Nếu mọi thứ diễn ra đúng kế hoạch, LLM sẽ phản hồi bằng bản tóm tắt kết quả. Nếu có lỗi hoặc thiếu gì đó, LLM có thể thử sửa vấn đề hoặc báo lỗi cho người dùng.

Giờ ta mở lại Nexus và kiểm thử một planner đang hoạt động. Nạp cùng agent bạn đã dùng lần trước, nhưng lần này chọn planner trong mục Advanced options, như trong Hình 12.8. Sau đó nhập prompt mục tiêu như trước và để agent tự làm.



Hình 12.8: Kết quả khi yêu cầu hoàn thành mục tiêu trong Nexus bằng basic planner

Sau vài phút, agent trả về file đã lưu, và trong một số trường hợp nó có thể đưa thêm thông tin, chẳng hạn các bước tiếp theo và việc cần làm với đầu ra. Lý do là agent được cung cấp một bản tổng quan cấp cao về những gì nó đã hoàn thành. Tuy nhiên hãy nhớ rằng việc thực thi kế hoạch diễn ra ở cấp cục bộ, và chỉ context, kế hoạch và mục tiêu được gửi tới LLM.

Điều này có nghĩa là việc thực thi kế hoạch có thể do bất kỳ tiến trình nào hoàn tất, không nhất thiết do agent. Thực thi kế hoạch bên ngoài LLM làm giảm số token và lượt dùng tool mà agent phải thực hiện. Nó cũng có nghĩa là một LLM không cần hỗ trợ dùng tool vẫn có thể dùng planner.

Bên trong, khi bật một planner trong Nexus, công cụ agent engine bị bỏ qua. Thay vào đó, planner hoàn tất việc thực thi action, và agent chỉ biết về các action thông qua việc nhận context đầu ra. Điều này có thể tốt với những mô hình hỗ trợ dùng tool nhưng không biết lập kế hoạch. Tuy nhiên, planner có thể hạn chế chức năng của những mô hình hỗ trợ cả dùng tool lẫn planning, chẳng hạn Claude.

Nhìn chung, bạn nên hiểu năng lực của LLM đang dùng. Nếu không chắc về các chi tiết đó, thử và sai một chút cũng có ích: yêu cầu agent hoàn thành một mục tiêu nhiều bước khi bật và khi tắt planning, rồi xem kết quả.

Planning cho phép agent hoàn thành nhiều tác vụ tuần tự để đạt các mục tiêu phức tạp hơn. Vấn đề của planning bên ngoài hay planning bằng prompt là nó bỏ qua vòng lặp phản hồi (feedback iteration loop), thứ có thể giúp sửa vấn đề nhanh chóng. Vì vậy, OpenAI và những bên khác hiện đang tích hợp trực tiếp reasoning và planning ở cấp LLM, như ta sẽ thấy ở mục tiếp theo.

12.4 Xem xét một stepwise planner: OpenAI Strawberry

Việc phát hành mô hình o1-preview, tên mã Strawberry, đã đem lại một thay đổi đáng kể về loại LLM sẵn có cho các hệ thống agentic. Strawberry không chỉ được công bố là hiệu quả hơn ở các tác vụ toán, khoa học và tính toán tổng quát, mà còn có thể thực hiện reasoning, planning, evaluation và feedback trực tiếp bên trong LLM.

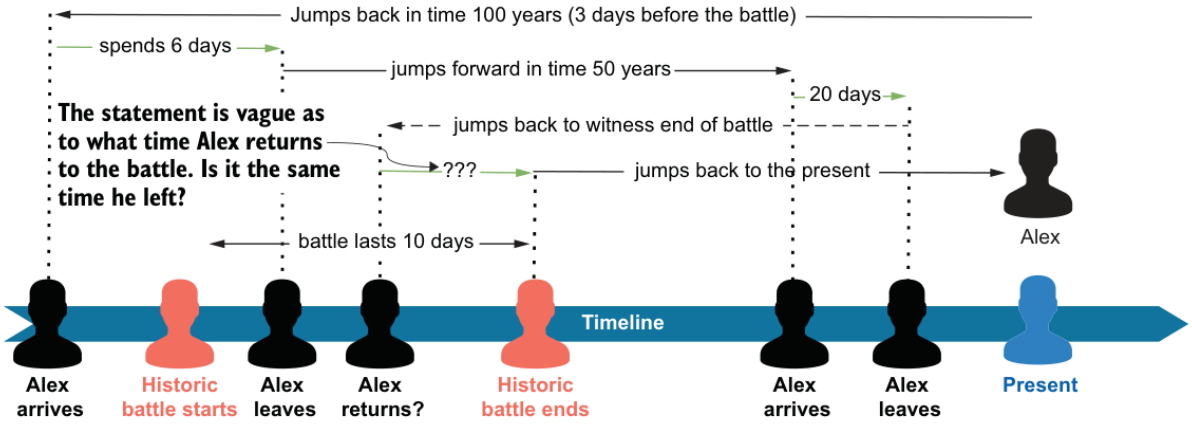
Hãy xem lại bài toán du hành thời gian ở chương 11, được trình bày lại trong Hình 12.9. Nếu nhớ lại, bài toán này khó giải bằng GPT-4 và các LLM tương tự. Tuy nhiên, khi áp dụng reasoning và feedback, ta tạo được đầu ra thỉnh thoảng đúng.

Như một thí nghiệm, nhập bài toán này vào ChatGPT bằng mô hình o1-preview, như trong Listing 11.6. Ngồi chờ vài giây để có câu trả lời. Đúng vậy, mô hình vẫn trả lời sai.

Listing 11.6 Bài toán reasoning/planning về du hành thời gian

```
In a sci-fi film, Alex is a time traveler who decides to go back in time
to witness a famous historical battle that took place 100 years ago,
which lasted for 10 days. He arrives three days before the battle starts.
However, after spending six days in the past, he jumps forward in time
by 50 years and stays there for 20 days. Then, he travels back to
witness the end of the battle. How many days does Alex spend in the past
before he sees the end of the battle?
```

Dù có phần đáng thất vọng khi mô hình trả lời sai, nhưng nó phân tách bài toán và trình bày câu trả lời tốt hơn nhiều. Listing 11.7 cho thấy đầu ra mẫu khi đưa bài toán ở Listing 11.6 cho mô hình Strawberry. Lưu ý rằng bạn có thể nhận được câu trả lời khác do tính ngẫu nhiên (stochastic) của LLM.



Hình 12.9: Bài toán du hành thời gian, xem lại

Listing 11.7 Phản hồi của o1-preview cho bài toán du hành thời gian

```
To determine how many days Alex spends in the past before witnessing the
end of the battle, let's reconstruct the timeline step by step:
Arrival in the Past:
Alex arrives 3 days before the battle begins.
He spends these 3 days waiting for the battle to start.
During the Battle:
The battle starts, and Alex experiences the first 3 days of it.
At this point, he has spent a total of 6 days in the past (3 days before
the battle and 3 days during the battle).
Time Jump Forward:
```

```
After these 6 days, Alex jumps 50 years forward in time.
He spends 20 days in this future time.
Return to the Past:
Alex then returns to the past to witness the end of the battle.
Since the battle lasts 10 days and he left after day 3 of the battle,
there are 7 days remaining in the battle.
However, because he was away for 20 days, which is longer than the
remaining 7 days of the battle, he must return to a point in time before
the battle ends to witness its conclusion.
To see the end of the battle, he returns to the past on the last day of
the battle.
Final Days Spent Before the Battle Ends:
Upon returning, he spends 1 more day in the past to witness the battle's
end.
Total Days Spent in the Past Before Seeing the End:
6 days before jumping forward in time.
1 day after returning to the past.
Total: 6 + 1 = 7 days.
Answer: 7
```

Chú thích cho Listing 11.7: tại mục `Time Jump Forward:` (đoạn mô hình xử lý bước nhảy tới tương lai), có thể thấy rõ mô hình mắc lỗi ở đâu.

Vì biết đáp án đúng là 27, ta biết LLM trả lời sai; nhưng nếu không biết, ta cũng dễ dàng cho rằng toàn bộ công việc và lập luận đều đúng. Những vấn đề như vậy có thể xảy ra khi ta bỏ feedback khỏi các tương tác với LLM và các hệ thống agentic. Feedback có thể hướng dẫn mô hình tự sửa mình.

Tuy nhiên, nếu ta không biết đáp án đúng là 27 (26, nếu bạn giả định anh ta không dành ngày đó để chứng kiến trận chiến) và cho rằng LLM hay agent đúng thì sao? Đây là vấn đề ta có thể khắc phục bằng vài prompt đơn giản để LLM tham gia vào reasoning và planning feedback. Tuy nhiên, các kỹ thuật này hiệu quả hơn với LLM hoặc các lớp bọc (wrapper) như OpenAI Assistants, nơi reasoning và planning được cung cấp bên trong mô hình.

Điều ta muốn là cung cấp feedback cho LLM, nhưng việc hiểu feedback đó là gì nhiều khả năng sẽ khó đối với ta. May mắn là ta có thể gợi ra feedback trực tiếp từ LLM, miễn là ta đưa cho nó đáp án đúng. Listing 11.8 cho thấy cách sinh feedback mang tính xây dựng từ LLM về bài toán du hành thời gian.

Listing 11.8 Sinh feedback

```
the correct answer is 27, please review what you did wrong and suggest
feedback you could give yourself when trying to solve similar future problems
```

Nhập đoạn này sau khi mô hình trả lời sai sẽ sinh ra feedback mà bạn có thể dùng để hướng dẫn mô hình thông qua prompt hoặc như một phần của chỉ dẫn hệ thống (system instruction). Listing 11.9 là ví dụ về feedback do o1-preview đưa ra. Sau đó bạn có thể trích feedback này và bổ sung vào chỉ dẫn ở lần tới khi muốn xử lý các bài toán du hành thời gian phức tạp.

Listing 11.9 Feedback được sinh ra

```
Feedback for Future Problems
Understanding Time Frames in Time Travel:
Relative Time Frames: Always consider the time periods relative to the
original point of reference. In this problem, even though Alex jumps
forward 50 years, he is still in the past relative to his original
present time.
Including All Relevant Periods:
Ensure that all time periods spent in the past (or future) are included
in the total calculation, even if they involve moving to different
points in time.
```

```
Time Continuity:
Remember that while the traveler moves through time, events in other
time periods continue to progress unless specified otherwise.
Application:
Careful Reading:
Pay close attention to the details provided, such as the durations and
the sequences of time jumps.
Visualization:
It may help to draw a timeline to visualize the sequence of events and
time periods involved.
Double-Checking Calculations:
After solving, revisit each step to ensure all time periods are
accounted for correctly.
```

Kỹ thuật feedback này sẽ hoạt động ổn định trên các mô hình như o1-preview, nhưng các mô hình khác vẫn có thể gặp khó khi trả lời đúng, kể cả khi đã có feedback này. Theo thời gian, khi các mô hình thông minh hơn, kỹ thuật này nhiều khả năng sẽ nhìn chung hiệu quả trên hầu hết mô hình. Tuy vậy, cơ chế feedback này nhiều khả năng vẫn cần thiết ngay cả khi mô hình ngày càng sáng dạ hơn, vì ngôn ngữ có nhiều sắc thái, và không phải bài toán nào ta đưa cho LLM cũng có một đáp án tuyệt đối rõ ràng. Hãy lấy chính bài toán ví dụ: đây là ví dụ rất rõ về việc đòi hỏi người giải phải đưa ra giả định và suy ra các mối liên hệ từ câu hỏi. Vẫn còn nhiều lĩnh vực khoa học, từ địa chất đến khoa học hành vi, mà cùng một bài toán có thể cho ra một khoảng các đáp án. Tiếp theo, ta xem một vài kỹ thuật áp dụng reasoning, planning, evaluation và feedback vào các hệ thống agentic.

12.5 Áp dụng planning, reasoning, evaluation và feedback vào các hệ thống assistant và agentic

Trong các chương gần đây, ta đã xem cách hiện thực các thành phần agentic gồm planning, reasoning, feedback và evaluation. Giờ ta xem cách thức, thời điểm và vị trí tích hợp các thành phần đó vào hệ thống assistant và agentic cho môi trường production thời gian thực, nghiên cứu hoặc phát triển.

Không phải mọi thành phần đều phù hợp như nhau với mọi ứng dụng, nhưng hiểu khi nào và ở đâu nên áp dụng thành phần nào là điều hữu ích. Ở mục tiếp theo, ta xem planning có thể được tích hợp vào hệ thống assistant/agentic ra sao.

12.5.1 Áp dụng planning cho assistant/agent

Planning là thành phần cho phép một assistant hay agent lên kế hoạch thực hiện một tập tác vụ, dù tuần tự, song song hay một tổ hợp nào khác. Ta thường gắn planning với việc dùng tool, và đúng là mọi hệ thống dùng tool nhiều khả năng đều cần một planner có năng lực. Tuy nhiên, không phải hệ thống nào cũng giống nhau, nên ở Bảng 12.1 ta xem xét nên hiện thực planner ở đâu, khi nào và như thế nào.

Bảng 12.1: Planning được dùng khi nào và ở đâu trong các ứng dụng khác nhau

Ứng dụng	Hiện thực (Implemented)	Môi trường	Mục đích	Thời điểm	Cấu hình
Personal assistant	At or within the LLM	Personal device	Facilitate tool use	During the response	As part of the prompt or LLM
Customer service bot	Not typical; restricted environment	Restricted environment, no tool use			
Autonomous agent	As part of the agent prompt and within the LLM	Server or service	Facilitate complex tool use and task planning	As part of constructing the agent and/or during the response	Within the agent or LLM
Collaborative workflows	As part of the LLM	Shared canvas or coding	Facilitate complex tool use	During the response	Within the LLM
Game AI	As part of the LLM	Server or application	Complex tool use and planning	Before or during the response	Within the LLM
Research	Anywhere	Server	Facilitate tool use and engage in complex task workflows	Before, during, and after response generation	Anywhere

Bảng 12.1 cho thấy nhiều kịch bản ứng dụng khác nhau mà ở đó ta có thể gặp một assistant hoặc agent được triển khai để hỗ trợ theo một cách nào đó. Để cung cấp thêm thông tin và hướng dẫn, danh sách sau nêu chi tiết hơn về cách planning có thể được dùng trong từng ứng dụng:

- *Personal assistant*: dù ứng dụng này triển khai còn chậm, các personal assistant dùng LLM hứa hẹn sẽ vượt Alexa và Siri trong tương lai. Planning sẽ rất cần thiết cho các assistant/agent mới này để điều phối nhiều tác vụ phức tạp và thực thi tool (action) theo chuỗi hoặc song song.
- *Customer service bot*: do tính chất được kiểm soát của môi trường này, các assistant tiếp xúc trực tiếp với khách hàng khó có khả năng dùng tool một cách được kiểm soát và rất cụ thể. Điều đó có nghĩa các loại assistant này nhiều khả năng sẽ không cần planning nhiều.
- *Autonomous agent*: như đã thấy ở các chương trước, agent có khả năng lập kế hoạch có thể hoàn thành một chuỗi tác vụ phức tạp cho nhiều mục tiêu khác nhau. Planning sẽ là một yếu tố thiết yếu của mọi hệ thống agentic tự hành.
- *Collaborative workflows* (workflow cộng tác): hãy nghĩ đến các agent hay assistant ngồi cạnh lập trình viên hoặc người viết. Dù các workflow này còn ở giai đoạn phát triển ban đầu, hãy hình dung một workflow trong đó agent được giao tự động viết và thực thi code kiểm thử bên cạnh lập trình viên. Planning sẽ là phần thiết yếu trong việc thực thi các workflow phức tạp của tương lai này.
- *Game AI*: dù việc áp dụng LLM vào game còn ở giai đoạn đầu, không khó để hình dung các agent hay assistant trong game có thể hỗ trợ hoặc thách thức người chơi. Cho các agent này khả năng lập kế hoạch và thực thi workflow phức tạp có thể thay đổi cách ta chơi game và chơi với ai.
- *Research*: tương tự collaborative workflow, các agent này sẽ chịu trách nhiệm rút ra ý tưởng mới từ các nguồn thông tin sẵn có. Việc tìm thông tin đó nhiều khả năng được hỗ trợ bởi việc dùng tool rộng rãi, và sẽ được lợi từ việc điều phối bằng planning.

Như bạn thấy, planning là một phần thiết yếu của nhiều ứng dụng LLM, dù thông qua việc điều phối dùng tool hay cách khác. Ở mục tiếp theo, ta xem thành phần kế tiếp là reasoning và cách áp dụng nó cho cùng các loại ứng dụng.

12.5.2 Áp dụng reasoning cho assistant/agent

Reasoning, dù thường gắn chặt với planning và việc hoàn thành tác vụ, là một thành phần cũng có thể đứng riêng. Khi LLM trưởng thành và thông minh hơn, reasoning thường được đưa sẵn vào bên trong LLM. Tuy nhiên, không phải ứng dụng nào cũng được lợi từ reasoning mở rộng, vì nó thường tạo ra một chu kỳ suy nghĩ (thinking cycle) trong phản hồi của LLM. Bảng 12.2 mô tả ở mức cao cách tích hợp thành phần reasoning vào các loại ứng dụng LLM khác nhau.

Bảng 12.2: Reasoning được dùng khi nào và ở đâu trong các ứng dụng khác nhau

Ứng dụng	**Hiện thực (Implemented)**	**Môi trường**	**Mục đích**	**Thời điểm**	**Cấu hình**
Personal assistant	Within the LLM	Personal device	Breaking down work into steps	During the response	As part of the prompt or LLM
Customer service bot	Not typical; usually just informational	Limited tool use and need for composite tool use			
Autonomous agent	As part of the agent prompt and within the LLM	Server or service	Facilitate complex tool use and task planning	As part of LLM, external reasoning not well suited	Within the agent or LLM
Collaborative workflows	As part of the LLM	Shared canvas or coding	Assists in breaking work down	During the response	Within the LLM
Game AI	As part of the LLM	Server or application	Essential for undertaking complex actions	Before or during the response	Within the LLM
Research	Anywhere	Server	Understand how to solve complex problems and engage in complex task workflows	Before, during, and after response generation	Anywhere

Bảng 12.2 cho thấy nhiều kịch bản ứng dụng khác nhau mà ở đó ta có thể gặp một assistant hoặc agent được triển khai để hỗ trợ theo một cách nào đó. Để cung cấp thêm thông tin và hướng dẫn, danh sách sau nêu chi tiết hơn về cách reasoning có thể được dùng trong từng ứng dụng:

- *Personal assistant*: tuỳ ứng dụng, lượng reasoning mà agent dùng có thể bị hạn chế. Reasoning là quá trình đòi hỏi LLM suy nghĩ qua một vấn đề, và việc này thường kéo dài thời gian phản hồi tuỳ độ phức tạp của vấn đề và độ dài của prompt. Trong nhiều tình huống, các phản hồi cần gần với thời gian thực có thể bị tắt hoặc giảm reasoning. Dù điều này có thể hạn chế độ phức tạp mà agent tương tác được, việc hạn chế hoặc không dùng reasoning có thể cải thiện thời gian phản hồi và tăng sự hài lòng của người dùng.

- *Customer service bot*: một lần nữa, do tính chất được kiểm soát của môi trường này, các assistant tiếp xúc trực tiếp với khách hàng khó có khả năng cần thực hiện reasoning phức tạp, hoặc bất kỳ dạng reasoning nào.

- *Autonomous agent*: dù reasoning là một thành phần mạnh của agent tự hành, ta vẫn chưa biết bao nhiêu reasoning là quá nhiều. Khi các mô hình như Strawberry có thể dùng cho workflow agentic, ta có thể đánh giá đến mức nào thì reasoning mở rộng không còn cần thiết. Điều này chắc chắn đúng với các workflow agent tự hành được định nghĩa rõ ràng.

- *Collaborative workflows*: một lần nữa, áp dụng reasoning tạo thêm chi phí phụ (overhead) trong tương tác với LLM. Reasoning mở rộng có thể mang lại lợi ích cho một số workflow, trong khi các workflow được định nghĩa rõ ràng khác có thể bị ảnh hưởng xấu. Điều này có thể có nghĩa các loại workflow này sẽ được lợi từ nhiều agent, gồm agent có reasoning và agent không có.

- *Game AI*: tương tự các ứng dụng khác, những ứng dụng nặng về reasoning có thể không phù hợp với hầu hết game AI. Game đặc biệt đòi hỏi thời gian phản hồi của LLM phải nhanh, và điều này chắc chắn sẽ là cách áp dụng reasoning cho các agent chiến thuật (tactical) tổng quát. Dĩ nhiên, điều đó không loại trừ việc dùng các agent reasoning khác có thể cung cấp khả năng kiểm soát mang tính chiến lược hơn.
- *Research*: reasoning nhiều khả năng sẽ thiết yếu cho mọi tác vụ nghiên cứu phức tạp vì nhiều lý do. Một ví dụ tốt là việc áp dụng mô hình Strawberry, mà ta đã thấy trong các nghiên cứu về toán học và khoa học.

Dù thường xét reasoning song song với planning, có thể có những điều kiện mà mức độ hiện thực của mỗi thành phần khác nhau. Ở mục tiếp theo, ta xét trụ cột evaluation của agent trong các ứng dụng khác nhau.

12.5.3 Áp dụng evaluation cho hệ thống agentic

Evaluation là thành phần của hệ thống agentic/assistant có thể chỉ ra hệ thống hoạt động tốt đến đâu. Dù ta đã minh hoạ việc đưa evaluation vào một số workflow agentic, evaluation thường là một thành phần bên ngoài trong các hệ thống agentic. Tuy nhiên, nó cũng là thành phần cốt lõi của hầu hết ứng dụng LLM và không nên bị bỏ qua trong hầu hết các dự án phát triển. Bảng 12.3 mô tả ở mức cao cách tích hợp thành phần evaluation vào các loại ứng dụng LLM khác nhau.

Bảng 12.3: Evaluation được dùng khi nào và ở đâu trong các ứng dụng khác nhau

Ứng dụng	**Hiện thực (Implemented)**	**Môi trường**	**Mục đích**	**Thời điểm**	**Cấu hình**
Personal assistant	External	Server	Determine how well the system is working	After the interaction	Often developed externally
Customer service bot	External monitor	Server	Evaluate the success of each interaction	After the interaction	External to the agent system
Autonomous agent	External or internal	Server or service	Determine the success of the system after or during task completion	After the interaction	External or internal
Collaborative workflows	External	Shared canvas or coding	Evaluate the success of the collaboration	After the interaction	External service
Game AI	External or internal	Server or application	Evaluate the agent or evaluate the success of a strategy or action	After the interaction	External or as part of the agent or another agent
Research	Combined manual and LLM	Server and human	Evaluate the output of the research developed	After the generated output	Depends on the complexity of the problem and research undertaken

Bảng 12.3 cho thấy nhiều kịch bản ứng dụng khác nhau mà ở đó ta có thể gặp một assistant hoặc agent được triển khai để hỗ trợ theo một cách nào đó. Để cung cấp thêm thông tin và hướng dẫn, danh sách sau nêu chi tiết hơn về cách evaluation có thể được dùng trong từng ứng dụng:

- *Personal assistant*: trong hầu hết trường hợp, một thành phần evaluation sẽ được dùng để xử lý và hướng dẫn hiệu năng của các phản hồi của agent. Với các hệ thống chủ yếu dùng retrieval

augmented generation (RAG) để khám phá tài liệu, evaluation cho biết assistant phản hồi các yêu cầu thông tin tốt đến đâu.

- *Customer service bot*: đánh giá các service bot là điều then chốt để hiểu bot phản hồi yêu cầu của khách hàng tốt đến đâu. Trong nhiều trường hợp, một thành phần tri thức RAG mạnh có thể là một phần của hệ thống và sẽ cần được đánh giá thường xuyên, liên tục. Một lần nữa, như với hầu hết thành phần evaluation, phần này nằm bên ngoài hệ thống làm việc chính và thường được chạy như một phần của việc giám sát hiệu năng chung theo nhiều chỉ số.
- *Autonomous agent*: trong hầu hết trường hợp, việc xem xét thủ công đầu ra của agent sẽ là thước đo chính cho thành công của một agent tự hành. Tuy nhiên, trong một số trường hợp, evaluation nội bộ có thể giúp hướng dẫn agent khi nó thực hiện các tác vụ phức tạp hoặc là cách cải thiện đầu ra cuối cùng. Các hệ thống đa agent như CrewAI và AutoGen là ví dụ về agent tự hành dùng feedback nội bộ để cải thiện đầu ra được sinh ra.
- *Collaborative workflows*: trong hầu hết trường hợp trực tiếp, việc đánh giá thủ công diễn ra liên tục trong các loại workflow này. Người dùng thường sẽ ngay lập tức và gần như theo thời gian thực chỉnh sửa assistant/agent bằng cách đánh giá đầu ra. Có thể thêm các agent bổ sung tương tự như với agent tự hành cho các workflow cộng tác mở rộng hơn.
- *Game AI*: evaluation thường được chia thành evaluation phát triển (đánh giá cách agent tương tác với game) và evaluation trong game (đánh giá agent hoàn thành một tác vụ tốt đến đâu). Hiện thực dạng evaluation sau tương tự agent tự hành nhưng nhằm cải thiện một số chiến lược hoặc cách thực thi. Các evaluation trong game như vậy nhiều khả năng cũng được lợi từ bộ nhớ và một cơ chế feedback.
- *Research*: evaluation ở mức này nhìn chung diễn ra như một nỗ lực thủ công sau khi hoàn thành tác vụ nghiên cứu. Một agent có thể dùng một dạng evaluation tương tự agent tự hành để cải thiện đầu ra được sinh ra, thậm chí có thể tự cân nhắc nội bộ việc evaluation đầu ra có thể được mở rộng hay nghiên cứu thêm thế nào. Vì đây hiện là một lĩnh vực mới của phát triển agentic, việc này sẽ được thực hiện tốt đến đâu vẫn còn phải chờ xem.

Evaluation là một yếu tố thiết yếu của mọi hệ thống agentic hay assistant, đặc biệt nếu hệ thống đó cung cấp cho người dùng thông tin thực và mang tính nền tảng. Phát triển các hệ thống evaluation cho agent và assistant có lẽ là chủ đề có thể, hoặc nên, có riêng một cuốn sách. Ở mục cuối của chương, ta xem việc hiện thực feedback cho các ứng dụng LLM khác nhau.

12.5.4 Áp dụng feedback cho các ứng dụng agentic/assistant

Feedback là một thành phần của hệ thống agentic thường, nếu không nói là luôn, được hiện thực như một thành phần bên ngoài, ít nhất là ở thời điểm hiện tại. Có thể độ tin cậy vào các hệ thống evaluation sẽ cải thiện đến mức feedback được đưa thường xuyên vào các hệ thống đó. Bảng 12.4 trình bày cách feedback có thể được hiện thực trong các ứng dụng LLM khác nhau.

Bảng 12.4: Feedback được dùng khi nào và ở đâu trong các ứng dụng khác nhau

Ứng dụng	Hiện thực (Implemented)	Môi trường	Mục đích	Thời điểm	Cấu hình
Personal assistant	External or by the user	Aggregated to the server or as part of the system	Provides means of system improvement	After or during the interaction	Internal and external
Customer service bot	External monitor	Aggregated to the server	Qualifies and provides a means for system improvement	After the interaction	External to the agent system
Autonomous agent	External	Aggregated at the server	Provides a means for system improvement	After the interaction	External
Collaborative workflows	While interacting	Shared canvas or coding	Provides a mechanism for immediate feedback	During the interaction	External service
Game AI	External or internal	Server or application	As part of internal evaluation feedback provided for dynamic improvement	After or during the interaction	External or as part of the agent or another agent
Research	Combined manual and LLM	Server and human	Evaluate the output of the research developed	After the generated output	Depends on the complexity of the problem and the research undertaken

Bảng 12.4 cho thấy nhiều kịch bản ứng dụng mà ở đó ta có thể gặp một assistant hoặc agent được triển khai để hỗ trợ theo một cách nào đó. Để cung cấp thêm thông tin và hướng dẫn, danh sách sau nêu chi tiết hơn về cách feedback có thể được dùng trong từng ứng dụng:

- *Personal assistant*: nếu assistant hoặc agent tương tác với người dùng qua giao diện kiểu chat, người dùng có thể đưa feedback trực tiếp và tức thì. Dù feedback này có được duy trì qua các cuộc hội thoại hay tương tác sau hay không, nó thường được hình thành trong bộ nhớ của agent (agentic memory). Các assistant như ChatGPT hiện đã tích hợp bộ nhớ và có thể được lợi từ feedback tường minh của người dùng.
- *Customer service bot*: feedback của người dùng hoặc hệ thống thường được thu thập qua một khảo sát sau khi tương tác kết thúc. Điều này thường có nghĩa feedback được chuyển cho một hệ thống bên ngoài để tổng hợp, phục vụ cải tiến về sau.
- *Autonomous agent*: giống bot, feedback trong agent tự hành thường được dồn về sau khi agent hoàn thành một tác vụ mà người dùng sau đó xem xét. Cơ chế feedback có thể khó thu thập hơn vì nhiều thứ có thể mang tính chủ quan. Các phương pháp tạo feedback đã khám phá trong chương này có thể được dùng để cải thiện prompt engineering.
- *Collaborative workflows*: tương tự personal assistant, các ứng dụng loại này có thể được lợi từ feedback trực tiếp và tức thì của người dùng. Một lần nữa, cách thông tin này được duy trì qua các phiên thường là một hiện thực của bộ nhớ agent.
- *Game AI*: feedback có thể được hiện thực song song với evaluation thông qua các agent bổ sung và nhiều agent. Dạng feedback này một lần nữa có thể chỉ dùng một lần và tồn tại trong tương tác hiện tại, hoặc có thể được lưu như bộ nhớ. Hãy hình dung một game AI có thể đánh giá hành động của chính nó, cải thiện chúng bằng feedback và ghi nhớ các cải thiện đó. Dù mẫu này không lý tưởng cho game, chắc chắn nó sẽ cải thiện trải nghiệm chơi.

- *Research*: tương tự evaluation trong bối cảnh nghiên cứu, feedback thường được thực hiện ngoại tuyến (offline) sau khi đầu ra được đánh giá. Dù đã có một số phát triển dùng các hệ thống đa agent kết hợp các agent cho evaluation và feedback, những hệ thống này không phải lúc nào cũng hoạt động tốt, ít nhất là với các mô hình tiên tiến nhất hiện nay. Thay vào đó, thường tốt hơn là tách riêng feedback và evaluation ở cuối để tránh vấn đề vòng lặp feedback thường gặp (feedback looping problem).

Feedback là một thành phần mạnh khác của hệ thống agentic và assistant, nhưng không phải lúc nào cũng cần ngay ở bản phát hành đầu tiên. Tuy nhiên, việc đưa vào các cơ chế feedback và evaluation chặt chẽ có thể mang lại nhiều lợi ích về lâu dài, liên quan đến việc giám sát liên tục và tạo sự tự tin để cải thiện các khía cạnh khác nhau của hệ thống.

Cách bạn hiện thực từng thành phần này trong hệ thống agentic của mình có thể một phần được định hướng bởi kiến trúc của nền tảng agentic bạn chọn. Giờ bạn đã hiểu các sắc thái của từng thành phần, bạn cũng có kiến thức để chọn hệ thống agent phù hợp với ứng dụng và trường hợp kinh doanh của mình. Dù ứng dụng của bạn là gì, bạn nên dùng nhiều thành phần agentic trong hầu hết mọi trường hợp.

Khi các hệ thống agentic trưởng thành và bản thân LLM thông minh hơn, một số thành phần mà ngày nay ta coi là bên ngoài có thể sẽ được tích hợp chặt chẽ hơn. Ta đã thấy reasoning và planning được tích hợp vào một mô hình như Strawberry. Chắc chắn, khi tiến gần đến cột mốc lý thuyết là trí tuệ nhân tạo tổng quát (artificial general intelligence), ta có thể thấy các mô hình có khả năng tự đánh giá (self-evaluation) và feedback dài hạn.

Dù thế nào, tác giả hy vọng bạn đã thích hành trình cùng ông đến biên giới đáng kinh ngạc của một công nghệ mới đang nổi, thứ chắc chắn sẽ thay đổi nhận thức của ta về công việc và cách ta thực hiện nó thông qua agent.

12.6 Bài tập

Hãy dùng các bài tập sau để củng cố kiến thức về nội dung đã học:

Bài tập 1: Hiện thực một planning agent đơn giản (Cơ bản)

Mục tiêu. Học cách hiện thực một planning agent cơ bản, dùng một prompt để sinh ra chuỗi action.

Nhiệm vụ.

- Tạo một agent nhận một mục tiêu, chia nó thành các bước và thực thi các bước đó tuần tự.
- Định nghĩa một mục tiêu đơn giản, chẳng hạn lấy thông tin từ Wikipedia và lưu vào một file.
- Hiện thực agent bằng một planner prompt cơ bản (xem ví dụ planner ở mục 12.3, mục 11.3).
- Chạy agent, rồi đánh giá nó lập kế hoạch và thực thi từng bước tốt đến đâu.

Bài tập 2: Kiểm thử việc tích hợp feedback trong một planning agent (Trung cấp)

Mục tiêu. Hiểu cách các cơ chế feedback có thể cải thiện hiệu năng của một hệ thống agentic.

Nhiệm vụ.

- Sửa agent ở bài tập 1 để thêm một vòng lặp feedback sau mỗi tác vụ.
- Dùng feedback để điều chỉnh hoặc sửa tác vụ kế tiếp trong chuỗi.
- Kiểm thử agent bằng cách giao một tác vụ phức tạp hơn, chẳng hạn thu thập dữ liệu từ nhiều nguồn, và quan sát feedback cải thiện hiệu năng của nó ra sao.
- Ghi lại và so sánh hành vi của agent trước và sau khi thêm feedback.

Bài tập 3: Thử nghiệm planning song song và tuần tự (Trung cấp)

Mục tiêu. Học sự khác biệt giữa action song song và action tuần tự, và chúng ảnh hưởng thế nào đến hành vi của agent.

Nhiệm vụ.

- Thiết lập hai agent bằng Nexus: một agent thực thi tác vụ song song và một agent thực hiện tác vụ tuần tự.
- Định nghĩa một mục tiêu nhiều bước, trong đó một số action phụ thuộc vào kết quả của các action trước (tuần tự), và một số có thể làm đồng thời (song song).
- So sánh hiệu năng và đầu ra của cả hai agent, ghi nhận mọi lỗi hoặc điểm kém hiệu quả khi thực thi song song trong trường hợp cần các bước tuần tự.

Bài tập 4: Xây dựng và tích hợp một planner tuỳ chỉnh vào Nexus (Nâng cao)

Mục tiêu. Học cách xây dựng một planner tuỳ chỉnh và tích hợp nó vào một nền tảng agent.

Nhiệm vụ.

- Viết một planner tuỳ chỉnh dùng các chiến lược prompt engineering từ mục 12.3 (mục 11.3), bảo đảm nó hỗ trợ thực thi tác vụ tuần tự.
- Tích hợp planner này vào Nexus và tạo một agent dùng nó.
- Kiểm thử planner với một mục tiêu phức tạp gồm nhiều bước và công cụ (ví dụ: truy xuất dữ liệu, xử lý và lưu).
- Đánh giá planner tuỳ chỉnh hoạt động thế nào so với các planner có sẵn trong Nexus hoặc các nền tảng khác.

Bài tập 5: Hiện thực xử lý lỗi và feedback trong sequential planning (Nâng cao)

Mục tiêu. Học cách hiện thực xử lý lỗi và feedback để tinh chỉnh planning tuần tự trong một hệ thống agentic.

Nhiệm vụ.

- Dùng một sequential planner, thiết lập một agent thực hiện một mục tiêu có thể gặp các lỗi thường gặp (ví dụ: lời gọi API thất bại, thiếu dữ liệu hoặc đầu vào không hợp lệ).
- Hiện thực các cơ chế xử lý lỗi trong planner để nhận biết và phản ứng với những lỗi này.
- Thêm các vòng lặp feedback để điều chỉnh kế hoạch hoặc thử lại action dựa trên lỗi gặp phải.
- Kiểm thử hệ thống bằng cách cố ý gây lỗi trong lúc thực thi, và quan sát agent phục hồi hoặc điều chỉnh kế hoạch ra sao.

Tóm tắt

- Planning là trung tâm của agent và assistant, cho phép chúng nhận một mục tiêu, chia nhỏ thành các bước và thực thi. Không có planning, agent bị thu hẹp thành các tương tác đơn giản giống chatbot.
- Agent phải phân biệt được action song song và action tuần tự. Nhiều LLM xử lý được action song song, nhưng chỉ các mô hình nâng cao mới hỗ trợ planning tuần tự, vốn rất quan trọng để hoàn thành các tác vụ phức tạp.
- Feedback đóng vai trò then chốt trong việc hướng dẫn agent sửa hướng đi và cải thiện hiệu năng theo thời gian. Chương này cho thấy các cơ chế feedback có thể được tích hợp với agent như thế nào để tinh chỉnh quá trình ra quyết định của chúng.
- Các nền tảng như OpenAI Assistants và Claude của Anthropic hỗ trợ planning nội bộ và có thể thực thi các tác vụ nhiều bước phức tạp. Agent dùng các nền tảng này có thể dùng planning cho action tuần tự để xây dựng các workflow tinh vi.
- Chọn đúng và giới hạn các action của agent là điều thiết yếu để tránh nhầm lẫn và hành vi ngoài ý muốn. Quá nhiều action có thể làm agent choáng ngợp, trong khi các công cụ không cần thiết có thể bị dùng sai.
- Nexus cho phép tạo và quản lý agent thông qua một giao diện linh hoạt, nơi người dùng có thể hiện thực planner tuỳ chỉnh, đặt mục tiêu và gán công cụ. Chương này có các ví dụ thực hành dùng Nexus để làm nổi bật sự khác biệt giữa LLM thô và agent được tăng cường bằng planner.
- Viết planner tuỳ chỉnh khá đơn giản khi dùng các chiến lược prompt engineering. Các công cụ như LangChain và Semantic Kernel cung cấp nhiều planner có thể điều chỉnh hoặc mở rộng cho nhu cầu agentic cụ thể.
- Các mô hình như OpenAI Strawberry tích hợp reasoning, planning, evaluation và feedback trực tiếp vào LLM, mang lại khả năng giải quyết vấn đề chính xác hơn.
- Evaluation giúp xác định một hệ thống agentic hoạt động tốt đến đâu và có thể được hiện thực nội bộ hoặc bên ngoài, tuỳ trường hợp sử dụng.
- Khi LLM tiến hoá, các cơ chế reasoning, planning và feedback có thể được tích hợp sâu vào mô hình, mở đường cho các hệ thống agent tự hành và thông minh hơn.

Phụ lục A

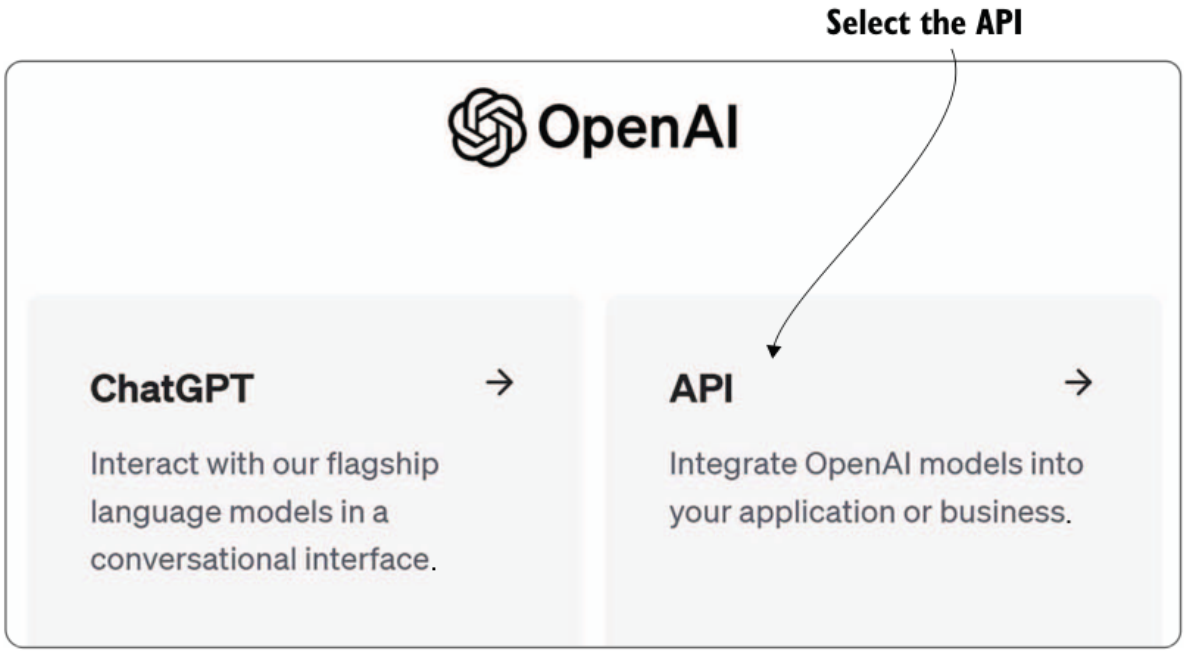
Truy cập các mô hình ngôn ngữ lớn của OpenAI

Hiện có nhiều dịch vụ mô hình ngôn ngữ lớn (LLM) thương mại, nhưng cuốn sách này khuyên dùng trực tiếp dịch vụ của OpenAI hoặc dùng qua Azure OpenAI Studio. Để truy cập một trong hai dịch vụ, bạn phải tạo tài khoản và đăng ký một phương thức thanh toán; phần này không nằm trong phụ lục. Họ mô hình GPT-4 được xem là tốt nhất trong nhóm và phù hợp hơn cho việc phát triển agent. Dùng các dịch vụ mã nguồn mở hoặc dịch vụ thay thế luôn là một lựa chọn, nhưng nhìn chung chỉ nên làm sau khi bạn đã làm việc với GPT-4 một thời gian.

A.1 Truy cập tài khoản và khoá của OpenAI

Các bước chung dưới đây giúp bạn nhanh chóng thiết lập để dùng các LLM của OpenAI cho việc phát triển agent. Dùng OpenAI và các LLM thương mại khác có tính phí, nhưng bạn có thể kỳ vọng chi dưới 100 USD để hoàn thành toàn bộ bài tập trong sách:

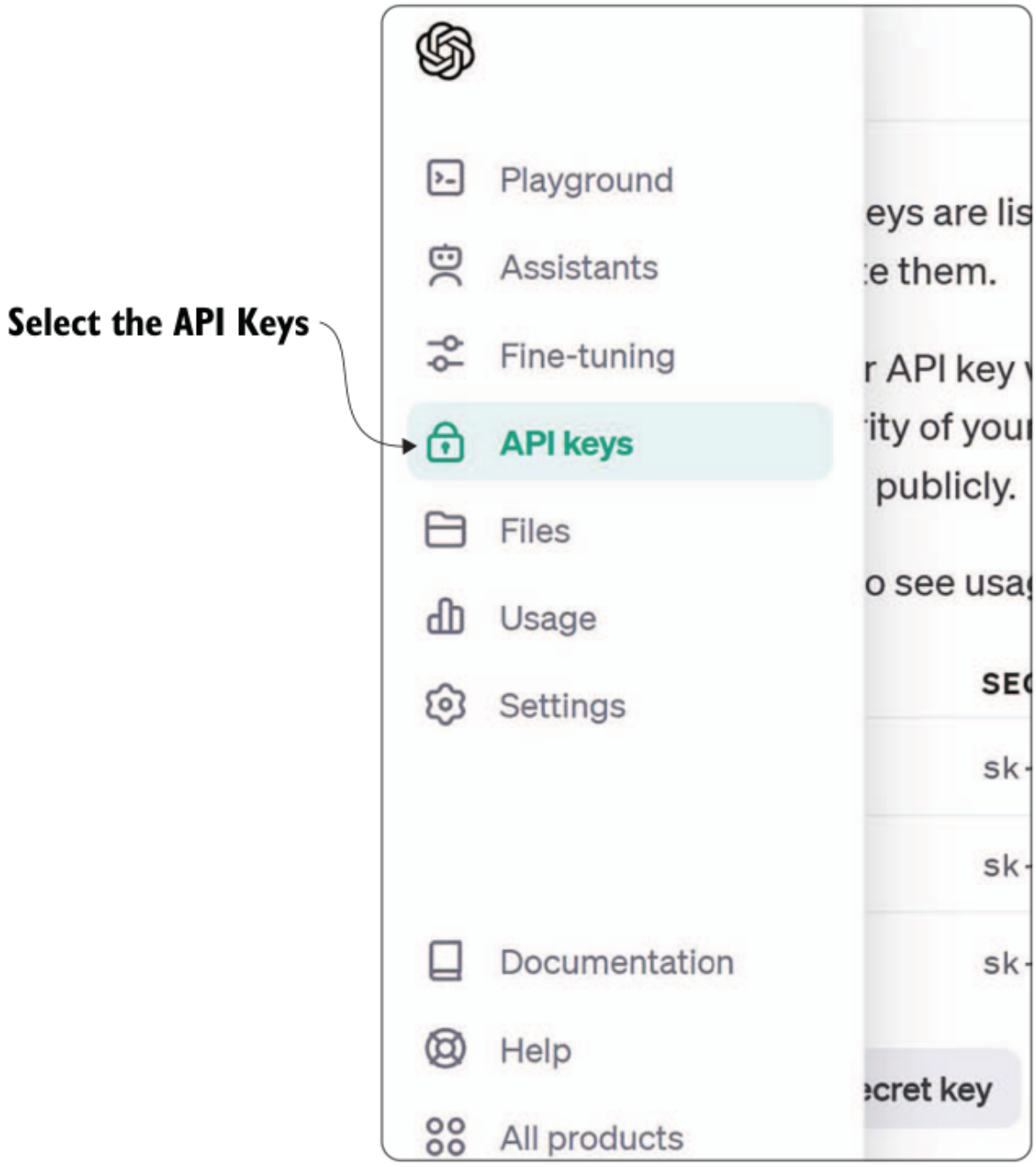
1. Truy cập `https://openai.com` và đăng nhập, hoặc đăng ký tài khoản rồi đăng nhập. Nếu đây là lần đầu tạo tài khoản, nhiều khả năng bạn sẽ được tặng một khoản tín dụng miễn phí nào đó. Nếu đã có tài khoản, bạn phải đăng ký phương thức và loại thanh toán. Nhìn chung nên mua trước một lượng tín dụng (credit) mỗi lần, vì cách này giúp quản lý chi phí tốt hơn và tránh vượt ngân sách.
2. Sau khi đăng nhập vào nền tảng, chọn ChatGPT hoặc API, như trong Hình A.1. Hãy chọn API.



Hình A.1: Chọn phần API của nền tảng OpenAI

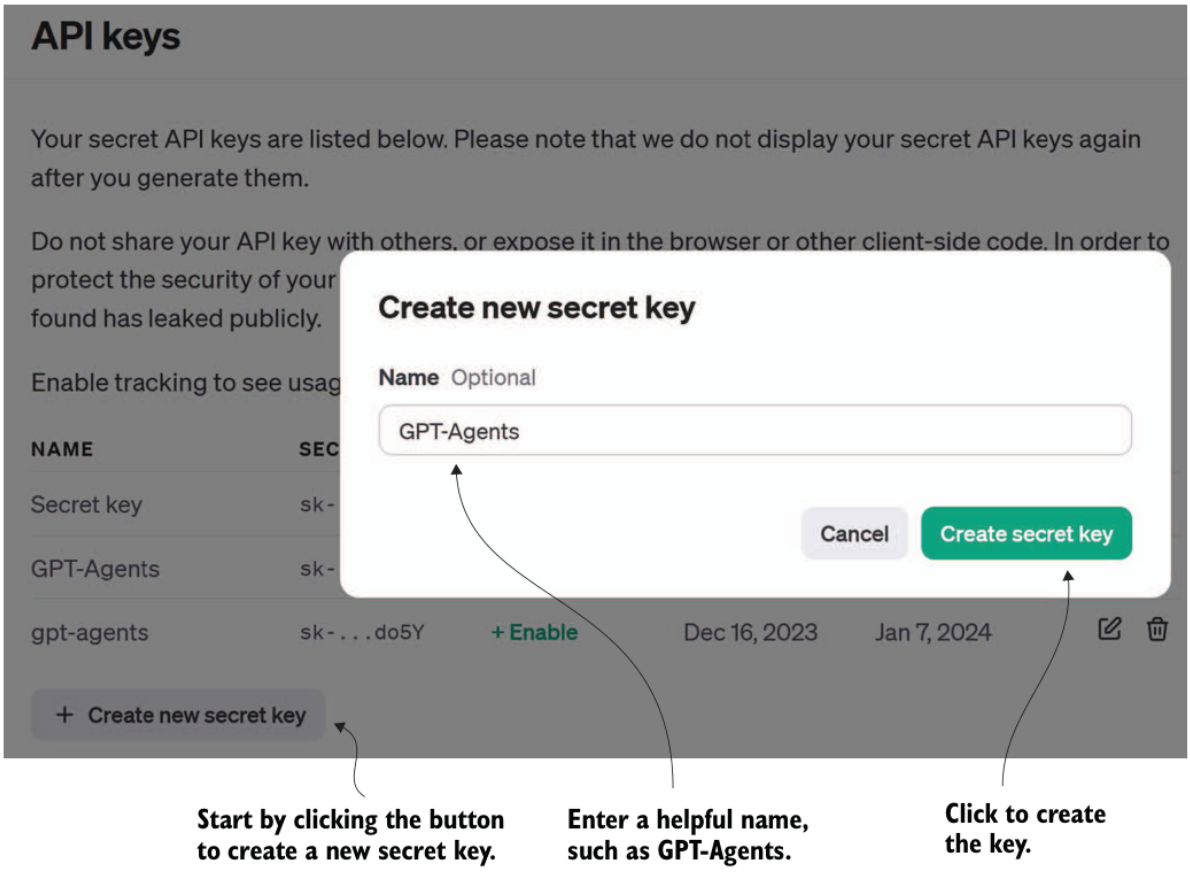
Trong hình, lựa chọn cần nhấn được ghi nhãn *Select the API* (chọn API).

3. Mở menu bên trái và chọn mục **API Keys**, như trong Hình A.2.



Hình A.2: Chọn mục API Keys

4. Nhấn nút **Create** để tạo một khoá mới, nhập tên cho khoá, rồi nhấn nút **Create Secret Key**, như trong Hình A.3.

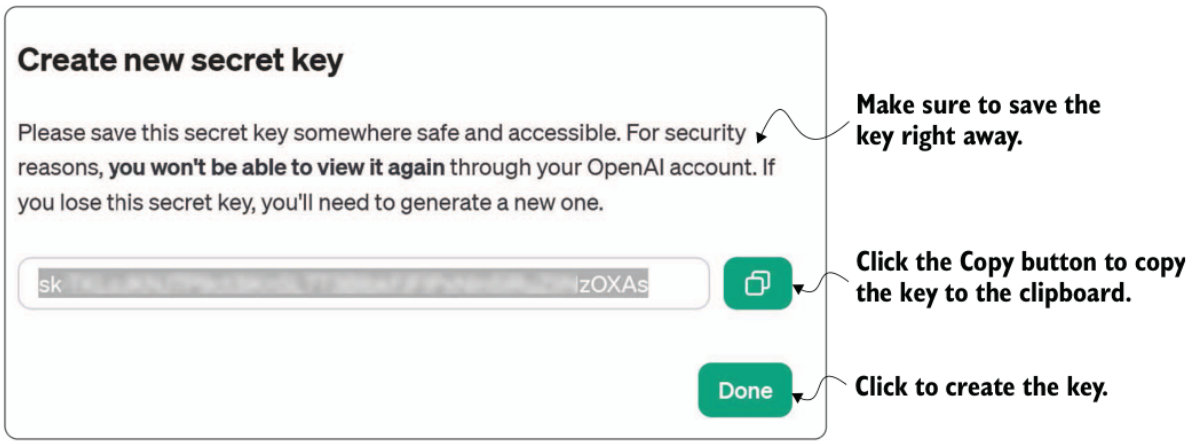


Hình A.3: Tạo khoá API bí mật (secret key)

Các chú thích trong Hình A.3:

- Bắt đầu bằng cách nhấn nút để tạo một secret key mới.
- Nhập một tên dễ nhớ, chẳng hạn `GPT-Agents`.
- Nhấn để tạo khoá.

5. Dùng nút **Copy** để sao chép khoá rồi dán vào notepad hoặc một nơi khác để cất giữ, như trong Hình A.4. Hãy giữ khoá này bí mật và bảo đảm nó chỉ nằm trên máy phát triển của bạn.



Hình A.4: Sao chép và dán khoá vào một nơi an toàn đã biết rõ

Các chú thích trong Hình A.4:

- Nhấn nút Copy để sao chép khoá vào clipboard.
- Hãy lưu khoá ngay lập tức.
- Nhấn để tạo khoá.

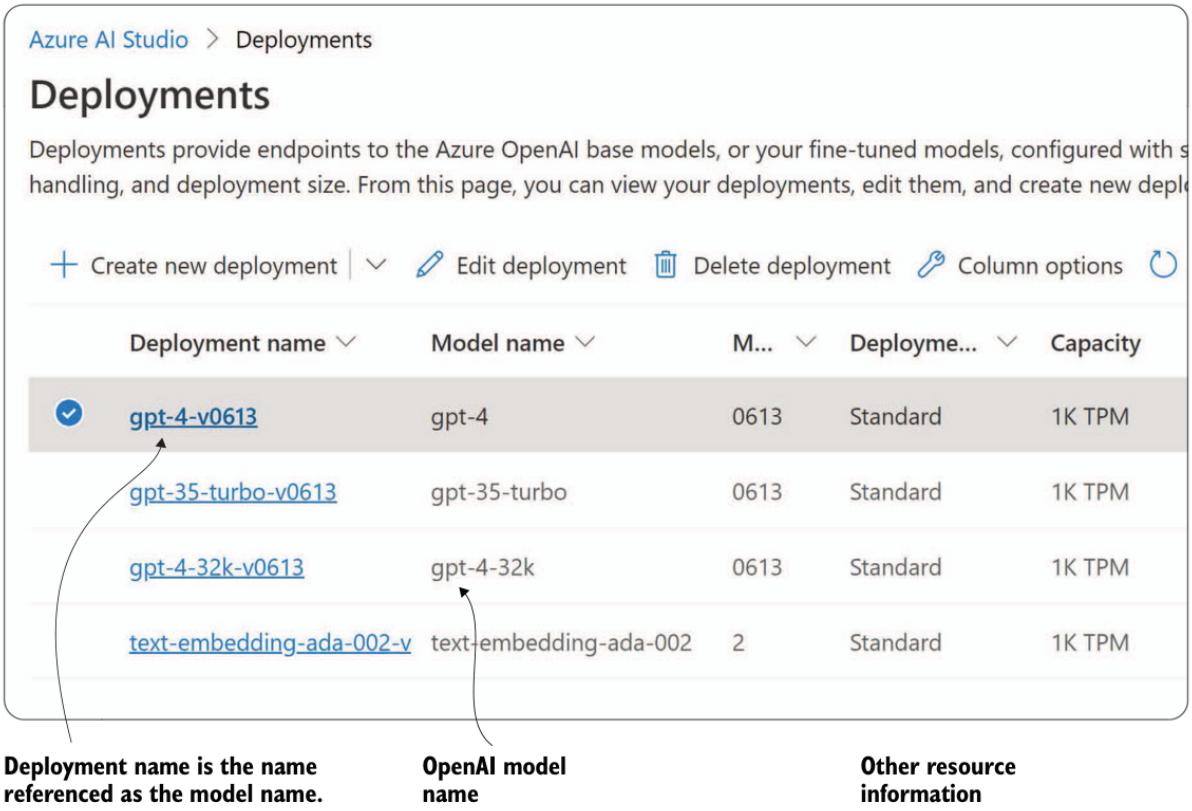
Sau khi tạo khoá, bạn có thể tiếp tục dùng nó trong file cấu hình `.env` hoặc qua các cách đăng ký khoá OpenAI khác. Với hầu hết các gói (package) dùng trong sách, cấu hình OpenAI thường chỉ cần khoá. Các dịch vụ khác, chẳng hạn Azure OpenAI, đòi hỏi cấu hình thêm một deployment (triển khai) mô hình và một base URL, như trình bày ở mục tiếp theo.

A.2 Azure OpenAI Studio, khoá và deployment

Nhờ mối quan hệ hợp tác lâu dài với OpenAI, Microsoft cung cấp cùng các mô hình với cùng mức giá trong Azure OpenAI Studio. Đôi khi Azure có thể chậm hơn một phiên bản mô hình, nhưng nhìn chung Microsoft luôn cập nhật các mô hình mới nhất của OpenAI. Các hướng dẫn dưới đây mang tính tổng quát hơn, vì có nhiều cách truy cập Azure cũng như nhiều cách tạo tài khoản và vào studio (muốn biết hướng dẫn cụ thể, hãy xem tài liệu của Microsoft):

1. Đăng nhập vào subscription (gói đăng ký) tài khoản Azure portal của bạn.
2. Tạo một tài nguyên (resource) Azure OpenAI Studio mới ở vùng (region) phù hợp với bạn. Tại thời điểm viết sách, không phải vùng nào cũng cho truy cập mọi mô hình, nên có thể bạn cần kiểm tra trước xem vùng của mình có những mô hình nào. Điều này cũng phụ thuộc vào tài khoản và cách sử dụng của bạn.

   Trong Azure OpenAI, các mô hình được cung cấp thông qua một cơ chế cấp phát tài nguyên gọi là **deployment**. Deployment bao bọc (wrap) một mô hình, chẳng hạn GPT-4, và cho phép truy cập tài nguyên đó. Hình A.5 cho thấy ví dụ về nhiều mô hình được cung cấp qua các deployment.
3. Nhấn nút **Create New Deployment** để tạo một deployment mới, rồi chọn mô hình bạn muốn triển khai.

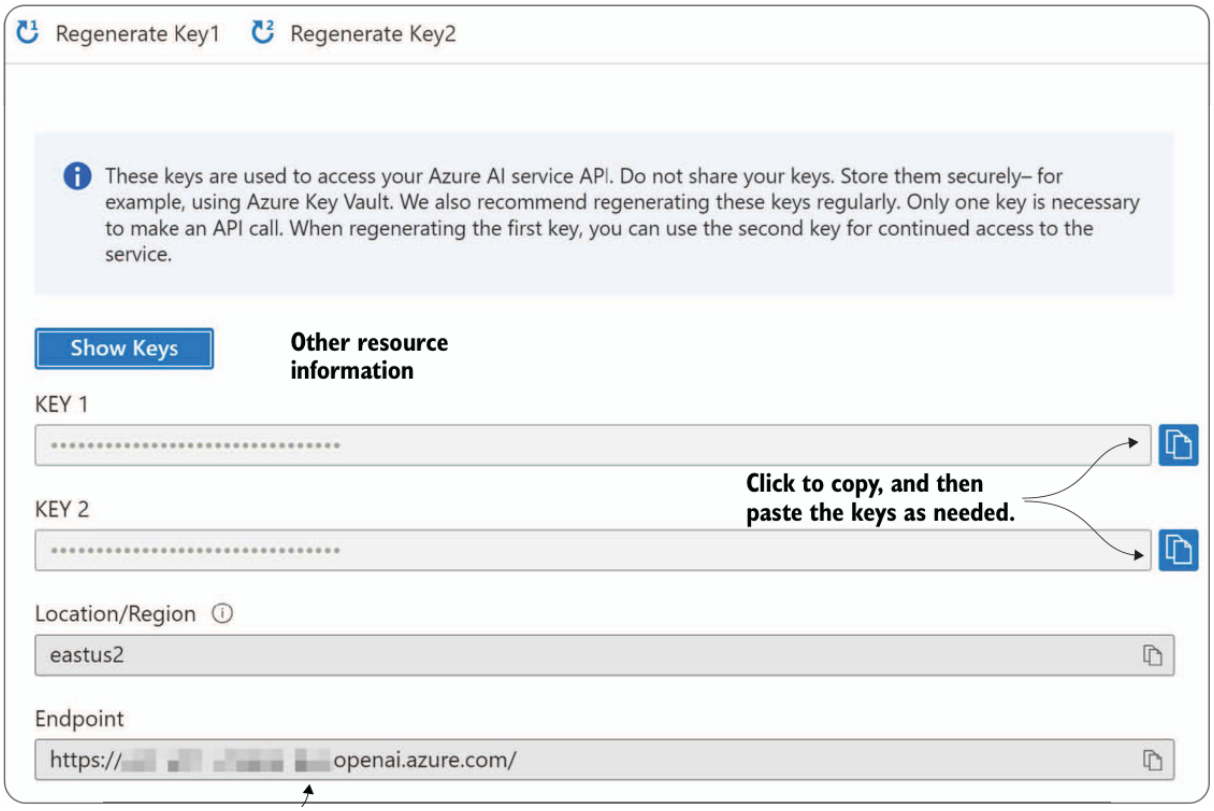


Hình A.5: Triển khai một mô hình thông qua deployment trong Azure OpenAI Studio

Các chú thích trong Hình A.5:

- Tên deployment (deployment name) là tên được dùng làm tên mô hình khi gọi.
- Tên mô hình OpenAI (OpenAI model name).
- Các thông tin khác của tài nguyên (other resource information).

4. Sau khi mô hình đã được bao bọc trong một deployment, bạn phải truy cập tài nguyên Azure OpenAI cha. Từ đó, bạn lấy được khoá (key), endpoint hoặc base URL cần để cấu hình kết nối, như trong Hình A.6.



Hình A.6: Lấy khoá và base URL (endpoint) từ tài nguyên Azure OpenAI cha

Các chú thích trong Hình A.6:

- Nhấn để sao chép, rồi dán khoá vào nơi cần dùng.
- Base URL của dịch vụ mô hình.
- Các thông tin khác của tài nguyên.

Một lần nữa, nếu gặp khó khăn, tài liệu của Microsoft có thể chỉ cho bạn hướng đi đúng. Khi kết nối tới một tài nguyên như Azure OpenAI Studio, hoặc tới một LLM khác bằng bộ công cụ của OpenAI, cần nhớ ba thông tin khác biệt then chốt sau:

- `api key` để truy cập mô hình.
- Base `url` hay `endpoint` nơi mô hình được đặt.
- Tên của `model`, hoặc tên deployment.

Nếu vì lý do nào đó bạn không truy cập được một mô hình, một lựa chọn thay thế tốt là các mô hình mã nguồn mở. Việc thiết lập và sử dụng các LLM mã nguồn mở được trình bày ở Chương 3.

Phụ lục B

Môi trường phát triển Python

Cuốn sách giả định người đọc là lập trình viên Python có kinh nghiệm, nhưng điều này có thể mang nhiều nghĩa khác nhau. Trong phụ lục này, ta xem cách cấu hình một môi trường phát triển Python chạy được với các ví dụ code trong sách. Bạn có thể dùng các môi trường phát triển tích hợp (IDE) khác, nhưng không phải công cụ nào, đặc biệt là các extension (phần mở rộng), cũng hoạt động trong mọi IDE.

B.1 Tải mã nguồn

Để tải và chạy mã nguồn, hãy cài Git rồi kéo (pull) repository về máy. Các bước ở mức tổng quát để lấy code từ repository GitHub của sách như sau:

1. Cài Git nếu cần. Git có thể cài từ nhiều nguồn, nhưng một lựa chọn tốt là bản phát hành chính, tại `https://git-scm.com/downloads`. Làm theo hướng dẫn để tải và cài công cụ cho hệ điều hành của bạn.
2. Mở terminal trong thư mục bạn muốn tải mã nguồn về, rồi nhập lệnh sau:

```
git clone https://github.com/cxbxmxcx/GPT-Agents.git
```

3. Sau khi code được tải về, bạn có thể bắt đầu bằng cách mở thư mục chương đang học trong Visual Studio Code (VS Code). Nếu cần cài VS Code hoặc cần biết cách nạp một thư mục chương làm workspace, hãy xem mục B.5 trong phụ lục này.

B.2 Cài đặt Python

Python có nhiều phiên bản và nhiều cách triển khai. Sách này dựa trên bản cài Python tiêu chuẩn, phiên bản 3.10. Anaconda là một bản triển khai Python khác rất phổ biến và cũng có thể dùng. Tuy vậy, toàn bộ tài liệu trong sách đã được chạy và kiểm thử với môi trường ảo (virtual environment) Python 3.10:

1. Truy cập `www.python.org/downloads/`.
2. Tìm và tải bản phát hành mới nhất của Python 3.10 cho hệ điều hành của bạn.
3. Cài bản phát hành này lên máy theo hướng dẫn cho hệ điều hành của bạn.
4. Để xác nhận việc cài đặt, mở terminal và chạy lệnh sau:

```
python --version
```

Phiên bản hiển thị nên là 3.10, nhưng nếu không phải thì cũng đừng lo, vì máy bạn có thể cài nhiều phiên bản Python. Ta cũng sẽ xác nhận lại việc cài đặt khi thiết lập VS Code.

B.3 Cài đặt VS Code

Cài VS Code khá đơn giản và chỉ cần vài bước:

1. Truy cập `https://code.visualstudio.com`.
2. Tải một bản ổn định (stable release) của VS Code cho hệ điều hành của bạn.
3. Sau khi tải xong, làm theo hướng dẫn cài đặt cho hệ điều hành của bạn.
4. Khởi chạy VS Code trên hệ điều hành của bạn và bảo đảm không có cảnh báo hay lỗi nào xuất hiện. Nếu gặp sự cố, hãy thử khởi động lại máy và/hoặc cài lại.

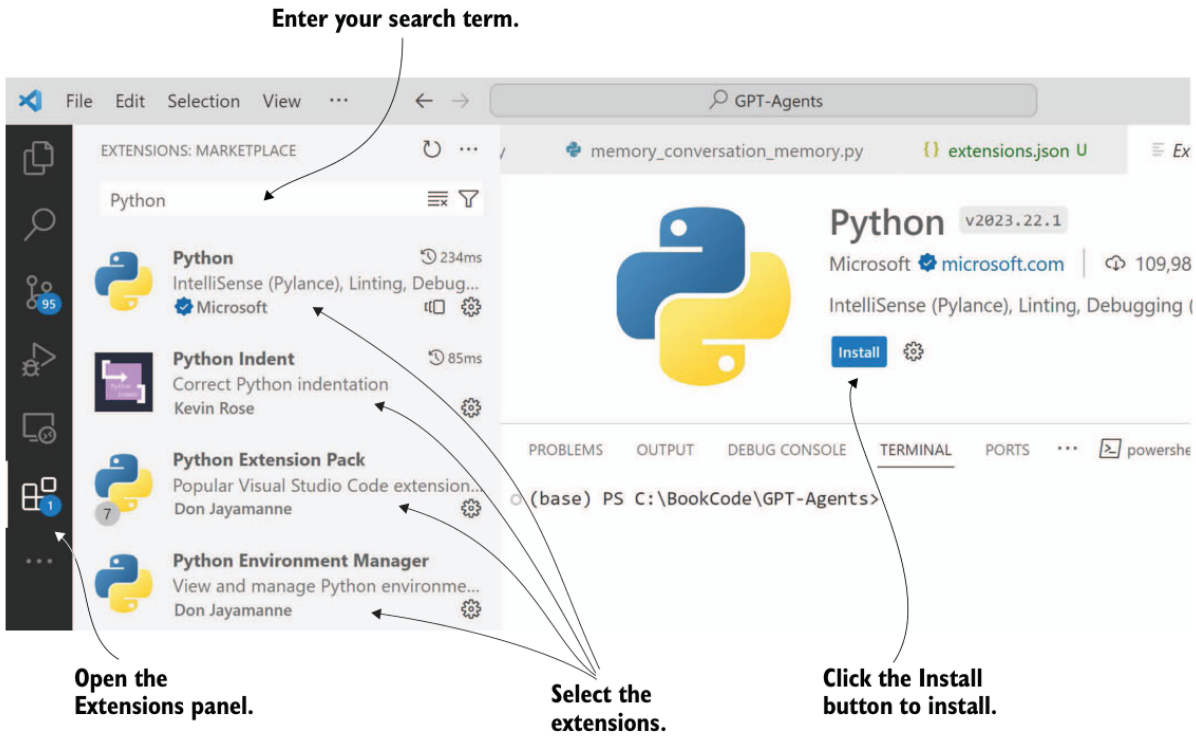
Khi VS Code đã chạy, ta có thể cài các extension cần thiết. Phần tiếp theo trình bày các extension này.

B.4 Cài các extension Python cho VS Code

Có hàng nghìn extension cho VS Code có thể tạo nên một môi trường lập trình Python xuất sắc. Các extension được khuyến nghị chỉ là điểm khởi đầu cho những gì bạn có thể tự khám phá thêm. Tuy nhiên, hãy lưu ý rằng không phải extension nào cũng có chất lượng như nhau. Khi cài extension mới, hãy xem số lượt cài đặt và điểm đánh giá. Nhìn chung nên tránh các extension dưới bốn sao. Để cài các extension, làm theo các bước sau:

1. Khởi chạy VS Code và mở bảng Extensions, như trong Hình B.1.
2. Cài danh sách extension sau:
    - **Python**: hỗ trợ môi trường và ngôn ngữ.
    - **Python Extension Pack**: gom các extension khác.
    - **Python Environment Manager**: quản lý môi trường.
    - **Python Indent**: định dạng code.
    - **Flake8**: định dạng code/linting.
    - **Prompt Flow**: kiểm thử prompt cho LLM.
    - **Semantic Kernel Tools**: làm việc với framework Semantic Kernel.
    - **Docker**: quản lý các container Docker.
    - **Dev Containers**: chạy môi trường phát triển bằng container.

Bạn chỉ cần cài extension cho mỗi môi trường VS Code mà bạn chạy. Thông thường, điều này có nghĩa là chỉ cài cho bản VS Code cài trên hệ điều hành của bạn. Tuy nhiên, nếu bạn chạy VS Code trong container, bạn phải cài extension cho từng container đang chạy. Việc làm việc với Python trong extension Dev Containers sẽ được trình bày ở phần sau của phụ lục này.



Hình B.1: Cài đặt các extension của VS Code

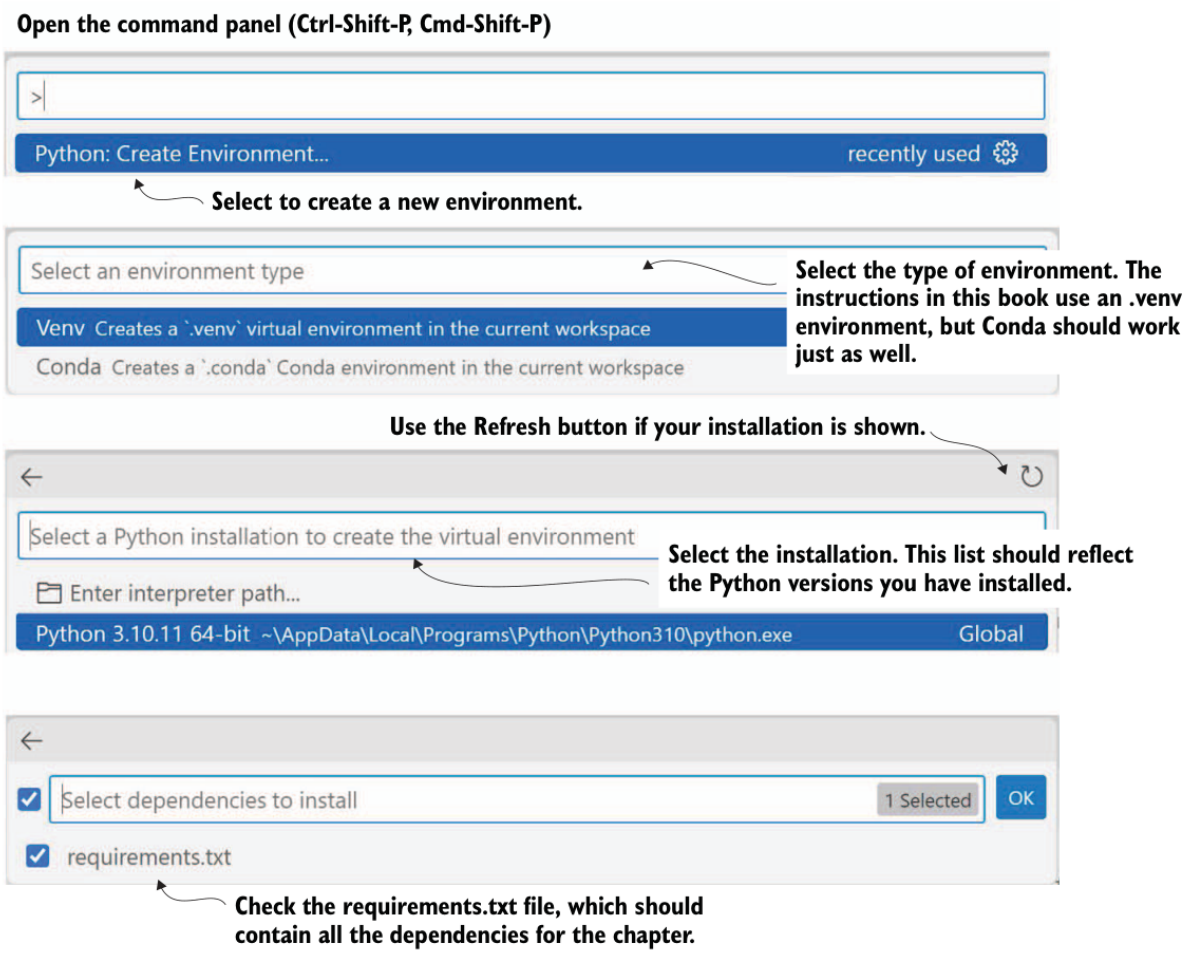
Các chú thích trong Hình B.1:

- Mở bảng Extensions.
- Chọn các extension.
- Nhập từ khoá tìm kiếm.
- Nhấn nút Install để cài.

B.5 Tạo môi trường Python mới bằng VS Code

Khi phát triển các dự án Python, bạn thường muốn tạo các môi trường ảo cô lập. Điều này giúp quản lý nhiều phụ thuộc gói (package dependency) khác nhau cho nhiều tác vụ và công cụ. Trong sách này, nên tạo một môi trường ảo mới cho mỗi chương mới. VS Code giúp bạn tạo và quản lý nhiều môi trường Python một cách nhanh chóng, hiệu quả qua các bước sau:

1. Nhấn Ctrl-Shift-P (Cmd-Shift-P) để mở bảng lệnh (command panel), rồi chọn **Python: Create Environment**, như trong Hình B.2.
2. Chọn loại môi trường: Venv hoặc Conda. Sách minh hoạ bằng Venv, nhưng Conda cũng nên hoạt động.



Hình B.2: Các bước thiết lập môi trường ảo cho một chương

Các chú thích trong Hình B.2:

- Mở bảng lệnh (Ctrl-Shift-P, Cmd-Shift-P).
- Chọn tạo một môi trường mới.
- Dùng nút Refresh nếu bản cài đặt của bạn chưa hiện trong danh sách.
- Chọn loại môi trường. Hướng dẫn trong sách dùng môi trường `.venv`, nhưng Conda cũng hoạt động tốt tương tự.
- Chọn bản cài đặt Python. Danh sách này phản ánh các phiên bản Python bạn đã cài.
- Đánh dấu file `requirements.txt`, file này nên chứa mọi phụ thuộc của chương.

3. Chọn bản cài đặt Python. Code trong sách đã được chạy với Python 3.10 trở lên. Các công cụ và framework agent trong sách đều rất mới (cutting edge), nên chúng hẳn sẽ hỗ trợ các phiên bản Python mới hơn.
4. Kiểm tra rằng file `requirements.txt` trong thư mục chương đã được chọn. Việc này sẽ cài mọi thư viện yêu cầu cho chương hiện tại.

Bạn nên thực hiện các bước này cho mỗi chương mới của sách. Cách thay thế là dùng các dev container của VS Code, sẽ được trình bày ở mục tiếp theo.

B.6 Dùng VS Code Dev Containers (Docker)

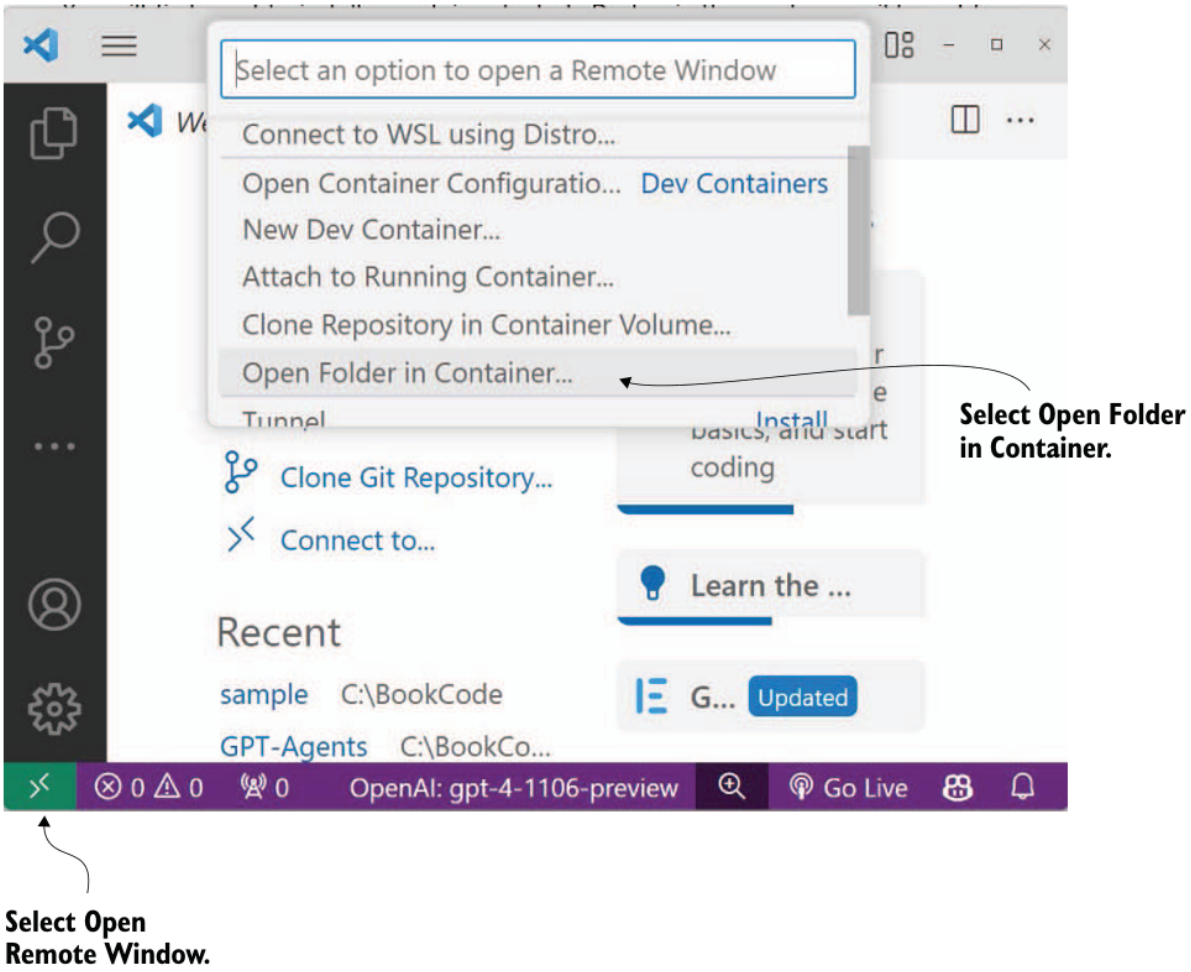
Khi làm việc với các agent nâng cao, đặc biệt là agent có thể sinh và thực thi code, nhìn chung nên chạy chúng trong các container cô lập. Cô lập bằng container giúp tránh làm gián đoạn hoặc hỏng hệ điều hành, đồng thời tạo nền tảng để triển khai agent. Làm quen với container và các nền tảng như Docker có thể là việc lớn nếu muốn nắm hết mọi thứ. May mắn là chỉ cần rất ít kiến thức để bắt đầu dùng container, và các extension của VS Code làm việc này dễ tiếp cận hơn nữa.

Trước tiên bạn cần cài một bộ công cụ container. Docker miễn phí (với điều kiện bạn dùng cho sở thích cá nhân hoặc là sinh viên) và là lựa chọn dễ tiếp cận nhất. Làm theo các hướng dẫn sau để cài Docker và bắt đầu làm việc với container:

1. Truy cập trang tải Docker Desktop tại `www.docker.com/products/docker-desktop`.
2. Tải và cài Docker cho hệ điều hành của bạn. Làm theo mọi hướng dẫn khác nếu được yêu cầu.
3. Khởi chạy ứng dụng Docker Desktop. Bước này xác nhận Docker đã được cài và hoạt động như mong đợi.
4. Mở VS Code và xác nhận các extension Docker đã liệt kê ở mục 1.4 đã được cài.

Khi Docker và VS Code đã được cấu hình, bạn có thể chuyển sang dùng Dev Containers theo các bước sau:

1. Mở một phiên bản (instance) VS Code mới.
2. Chọn mở một cửa sổ từ xa (remote window), như trong Hình B.3.
3. Chọn **Open Folder in Container** để khởi động một container từ một thư mục, hoặc chọn **New Dev Container** để khởi động mà không cần thư mục. Sau khi container được khởi chạy, môi trường VS Code của bạn sẽ được kết nối. Nhờ đó bạn có thể phát triển code trên container mà không lo các phụ thuộc không hoạt động.



Hình B.3: Mở cửa sổ từ xa tới một container trong VS Code

Các chú thích trong Hình B.3:

- Chọn *Open Folder in Container*.
- Chọn *Open Remote Window*.